

SYSTEM DESIGN CORE

Thiết Kế Hệ Thống

Từ con số 0 đến chuyên gia

Biên soạn lại từ systemdesigncore.site — nội dung gốc của Steve Bang.

Thuật ngữ chuyên ngành được Việt hoá, kèm thuật ngữ gốc ở lần xuất hiện đầu tiên mỗi bài.

Mục lục

Lời mở đầu	Thiết Kế Hệ Thống: Bắt Đầu Đúng Ngay Từ Ngày Đầu	5
Phần 0	Chuyển Đổi Mô Hình Tư Duy	10
Phần 0 · Bài 1	Từ Ưu Tiên Viết Mã đến Ưu Tiên Vấn Đề: Bước Đầu Thay Đổi Tư Duy	16
Phần 0 · Bài 2	Từ Giải Pháp Hoàn Hảo đến Tư Duy Đánh Đổi	24
Phần 0 · Bài 3	Thành Phần và Giao Tiếp: Hiểu Bản Chất Của Một Hệ Thống	38
Phần 0 · Bài 4	Tư Duy Theo Ràng Buộc: Kỹ Thuật Là Quản Lý Giới Hạn	54
Phần 0 · Bài 5	Trực Giác Về Quy Mô: Xây Dựng Cảm Giác Về Các Con Số	70
Phần 0 · Bài 6	Khung Tư Duy Hệ Thống: Nhìn Hệ Thống Như Một Tổng Thể	91
Phần 0 · Bài 7	Khung Câu Hỏi Hệ Thống – Danh Sách Kiểm Tra Chuẩn Khi Thiết Kế Hệ Thống	109
Phần 0 · Bài 8	Tổng Kết Mô Hình Tư Duy – Củng Cố Tư Duy Thiết Kế Hệ Thống	119
Phần 1	Nền Tảng: Tư Duy Theo Hệ Thống	130
Phần 1 · Bài 1	Các Thành Phần Trong Một Hệ Thống Web	134
Phần 1 · Bài 2	Các Mẫu Giao Tiếp: Đồng Bộ và Bất Đồng Bộ	151
Phần 1 · Bài 3	Luồng Dữ Liệu & Điểm Nghẽn: Tìm Và Tối Ưu Điểm Nghẽn	172
Phần 1 · Bài 4	Khái Niệm Quan Trọng Trong Thiết Kế Hệ Thống	194
Phần 1 · Bài 5	Bài Tập Tổng Hợp: Từ Lý Thuyết Đến Thực Hành	210
Phần 2	Các Khối Xây Dựng Cốt Lõi	239
Phần 2 · Bài 1	Cân Bằng Tải: Phân Phối Lưu Lượng Trong Hệ Thống Phân Tán	244
Phần 2 · Bài 2	Lưu Đệm: Tăng Hiệu Năng Quan Trọng Nhất	278
Phần 2 · Bài 3	Mở Rộng Cơ Sở Dữ Liệu: Chiến Lược Mở Rộng Tầng Dữ Liệu	301
Phần 2 · Bài 4	Hàng Đợi Thông Điệp & Xử Lý Bất Đồng Bộ: Tách Rời Các Dịch Vụ	322
Phần 2 · Bài 5	Tích Hợp & Tư Duy Đánh Đổi: Kết Hợp Các Thành Phần Thành Hệ Thống	348

Phần 3	Nền Tảng Hệ Thống Phân Tán	377
Phần 3 · Bài 1	Thực Tế Của Hệ Thống Phân Tán: Mô Hình Sự Cố Và Tư Duy CAP	382
Phần 3 · Bài 2	Các Mô Hình Nhất Quán: Nhất Quán Mạnh và Nhất Quán Cuối Cùng	403
Phần 3 · Bài 3	Đồng Thuận & Bầu Chọn Nút Chủ: Thỏa Thuận Trong Hệ Thống Phân Tán	421
Phần 3 · Bài 4	Giao Dịch Phân Tán & Toàn Vẹn Dữ Liệu	440
Phần 3 · Bài 5	ID Phân Tán Và Thứ Tự: Vấn Đề Định Danh Trong Hệ Thống Phân Tán	470
Phần 3 · Bài 6	Thời Gian & Thứ Tự Sự Kiện – Bài Toán Đồng Hồ Trong Hệ Thống Phân Tán	494
Phần 4	Khả Năng Mở Rộng & Hiệu Năng	512
Phần 4 · Bài 1	Hiệu Năng & Tư Duy Điểm Nghẽn – Tư Duy Tìm Và Giải Quyết Điểm Nghẽn	517
Phần 4 · Bài 2	Chiến Lược Lưu Đệm – Đánh Đổi Tính Nhất Quán Để Lấy Tốc Độ	532
Phần 4 · Bài 3	CDN & Điện Toán Biên – Bộ Nhớ Đệm Phân Tán Gần Người Dùng	553
Phần 4 · Bài 4	Mở Rộng Và Tối Ưu Cơ Sở Dữ Liệu – Từ Tinh Chỉnh Truy Vấn Đến Phân Mảnh Dữ Liệu	575
Phần 4 · Bài 5	Giới Hạn Tần Suất & Kiểm Soát Tải – Bảo Vệ Hệ Thống Khỏi Quá Tải	602
Phần 4 · Bài 6	Xử Lý Bất Đồng Bộ & Hàng Đợi – Trì Hoãn Công Việc Để Giảm Độ Trễ	633
Phần 4 · Bài 7	Giám Sát, Khả Năng Quan Sát & Gỡ Lỗi Hiệu Năng – Đo Lường Để Mở Rộng	659
Phần 5	Các Mẫu Kiến Trúc Thực Tế	687
Phần 5 · Bài 1	Tư Duy Tiến Hoá Kiến Trúc – Bắt Đầu Đơn Giản, Mở Rộng Dần Dần	691
Phần 5 · Bài 2	Hệ Thống Thiên Về Đọc – Thiết Kế Cho 99% Lướt Đọc	718
Phần 5 · Bài 3	Hệ Thống Bảng Tin Mạng Xã Hội: Kiến Trúc Phức Tạp Nhất	745
Phần 6	Làm Chủ Thiết Kế Hệ Thống	772

Phần 6 · Bài 1	Khung Tư Duy Thiết Kế Hệ Thống: Từ Biết Mẫu Thiết Kế Đến Biết Tư Duy	777
Phần 6 · Bài 2	Khung Tư Duy SNAKE: Phương Pháp Để Chinh Phục Buổi Phỏng Vấn Thiết Kế Hệ Thống	808
Phần 6 · Bài 3	Đánh Đổi & Ra Quyết Định Kiến Trúc	838
Phần 6 · Bài 4	Chiến Lược Phỏng Vấn Thực Chiến: Áp Dụng Khung Tư Duy Vào Tình Huống Thực Tế	863
Phần 6 · Bài 5	Các Chủ Đề Kiến Trúc Nâng Cao – Kiến Thức Phân Biệt Kỹ Sư Lâu Năm Và Kỹ Sư Cấp Staff	887
Phần 6 · Bài 6	Tư Duy Kiến Trúc Sư & Tư Duy Vận Hành Thực Tế – Từ Kỹ Sư Đến Kiến Trúc Sư Đáng Tin Cậy	921
Phụ lục A	Chuẩn Bị Cho Phỏng Vấn Thiết Kế Hệ Thống	960
Phụ lục B	Bảng thuật ngữ Anh – Việt	969

Thiết Kế Hệ Thống: Bắt Đầu Đúng Ngay Từ Ngày Đầu

Nền tảng này được xây dựng để giúp lập trình viên Việt Nam chuyển từ tư duy **lấy mã nguồn làm trung tâm** (*code-first*) sang **tư duy hệ thống** (*system thinking*).

Bạn không chỉ học cách vẽ sơ đồ hệ thống. Bạn học cách tư duy như một kỹ sư thiết kế sản phẩm thật: hiểu bài toán, nhìn ra sự đánh đổi (*trade-off*), và ra quyết định kiến trúc phù hợp với bối cảnh.

Sứ mệnh: Giúp bạn đi từ người "làm tính năng" thành người có thể thiết kế, giải thích và bảo vệ kiến trúc hệ thống trong môi trường vận hành thực tế (*production*).

1. Bạn Sẽ Học Được Gì

- **Tư duy hệ thống:** nhìn hệ thống như một tổng thể thay vì từng dịch vụ (*service*) rời rạc.
- **Tư duy kiến trúc** (*Architecture mindset*): biết cách chia thành phần, xác định ranh giới trách nhiệm và luồng dữ liệu.
- **Lập luận về đánh đổi:** hiểu vì sao không có "giải pháp tốt nhất", chỉ có giải pháp phù hợp.
- **Thiết kế hệ thống thực tế:** học theo các tình huống sát với môi trường vận hành thực tế: quy mô (*scale*), độ tin cậy (*reliability*), khả năng quan sát (*observability*), chi phí (*cost*).
- **Kỹ thuật gắn với sản phẩm:** kết nối quyết định kỹ thuật với nhu cầu người dùng và mục tiêu sản phẩm.

2. Trang Này Dành Cho Ai

Phù Hợp Nhất Với

- Kỹ sư backend muốn nâng cấp tư duy thiết kế hệ thống.
- Kỹ sư fullstack/frontend muốn hiểu kiến trúc từ đầu đến cuối (*end-to-end*).
- Lập trình viên đang chuẩn bị cho vòng phỏng vấn thiết kế hệ thống (*system design interview*).
- Lập trình viên từ mới vào nghề (*junior*) đến lâu năm (*senior*) muốn phát triển sự nghiệp dài hạn theo hướng kiến trúc.

Có Thể Chưa Phù Hợp Nếu

- Bạn chỉ muốn "khuôn mẫu trả lời phỏng vấn" mà không muốn hiểu bản chất.
- Bạn không sẵn sàng rèn luyện tư duy đặt câu hỏi và phân tích đánh đổi.

3. Triết Lý Học Tập

Bắt Đầu Đơn Giản, Tiến Hoá Dần

Thiết kế hệ thống tốt không bắt đầu từ sự phức tạp. Nó bắt đầu từ phiên bản đơn giản nhất có thể chạy được, rồi mở rộng theo nhu cầu thật.

Mô Hình Tư Duy (*Mental Model*) Trước, Công Nghệ Sau

Các framework và công cụ thay đổi liên tục. Mô hình tư duy về độ trễ (*latency*), tính nhất quán (*consistency*), thông lượng (*throughput*), khả năng chịu lỗi (*fault tolerance*) mới là thứ giữ giá trị lâu dài.

Bài Toán Trước, Công Cụ Sau

Đừng hỏi "dùng Redis hay Kafka trước". Hãy hỏi: "Bài toán là gì, ràng buộc (*constraint*) là gì, và cái giá của sự đánh đổi là gì?"

Tư Duy Vận Hành Thực Tế

Mọi quyết định đều cần nghĩ tới vận hành thật: giám sát (*monitor*) thế nào, gặp sự cố (*fail*) thì sao, hoàn tác (*rollback*) ra sao, mở rộng bao nhiêu là đủ.

4. Lộ Trình Học: Từ Con Số 0 Đến Chuyên Gia

Phần	Trọng tâm	Thời lượng gợi ý
Phần 0: Chuyển Đổi Mô Hình Tư Duy	Chuyển tư duy từ lấy mã nguồn làm trung tâm sang lấy bài toán làm trung tâm (<i>problem-first</i>), tư duy đánh đổi	2 tuần
Phần 1: Nền Tảng — Tư Duy Theo Hệ Thống	Các thành phần, mẫu giao tiếp (<i>communication patterns</i>), luồng dữ liệu (<i>data flow</i>), điểm nghẽn (<i>bottlenecks</i>), CAP/tính sẵn sàng (<i>availability</i>)	3-4 tuần
Phần 2: Các Khối Xây Dựng Cốt Lõi	Cân bằng tải (<i>Load balancing</i>), lưu đệm (<i>caching</i>), mở rộng cơ sở dữ liệu (<i>database scaling</i>), hàng đợi (<i>queue</i>), đánh đổi (<i>trade-offs</i>) khi tích hợp	4-5 tuần
Phần 3: Nền Tảng Hệ Thống Phân Tán (<i>Distributed Systems</i>)	Mô hình sự cố (<i>Failure model</i>), mô hình nhất quán (<i>consistency model</i>), đồng thuận (<i>consensus</i>), giao dịch phân tán (<i>distributed transaction</i>), ID	5-6 tuần
Phần 4: Khả Năng Mở Rộng (<i>Scalability</i>) & Hiệu Năng (<i>Performance</i>)	Hiệu năng, chiến lược lưu đệm (<i>caching strategy</i>), CDN/điểm biên (<i>edge</i>), giới hạn tần suất (<i>rate limit</i>), bất đồng bộ (<i>async</i>), khả năng quan sát	4-5 tuần
Phần 5: Các Mẫu Kiến Trúc Thực Tế	Tư duy tiến hoá kiến trúc, hệ thống thiên về đọc (<i>read-heavy</i>), kiến trúc mạng xã hội/bảng tin (<i>feed</i>)	4-5 tuần
Phần 6: Làm Chủ Thiết Kế Hệ Thống	Khung tư duy (<i>Framework</i>), phương pháp SNAKE, ra quyết định, chiến lược phỏng vấn, tư duy kiến trúc sư (<i>architect mindset</i>)	Liên tục

5. Cách Học Hiệu Quả

Nhịp Học Khuyến Nghị

- 4-5 buổi/tuần
- 60-90 phút/buổi
- Mỗi tuần dành 1 buổi để ôn lại và viết lại lập luận của bản thân

Quy Trình Học Cho Mỗi Bài

1. Đọc mục tiêu và bối cảnh của bài học.

2. Tự tóm tắt bài toán trong 3-5 dòng.
3. Liệt kê 2-3 phương án khả thi.
4. Chọn 1 phương án và ghi rõ sự đánh đổi.
5. So sánh với nội dung bài học và cập nhật mô hình tư duy.

Nguyên Tắc Quan Trọng

- Học để **ra quyết định**, không phải học để nhớ từ khoá.
- Nếu gặp khái niệm khó, hãy quay về nguyên lý gốc (luồng dữ liệu, điểm nghẽn (*bottleneck*), ràng buộc).
- Viết lại bằng ngôn ngữ của bạn sau mỗi bài để biến kiến thức thành kỹ năng.

6. Kiến Thức Cần Có Trước

Bạn không cần biết hết về hệ thống phân tán ngay từ đầu, nhưng nên có:

- Kiến thức lập trình cơ bản (backend hoặc fullstack).
- Hiểu HTTP/API ở mức làm sản phẩm thực tế.
- Nền tảng cơ sở dữ liệu (*database*) cơ bản (SQL/NoSQL ở mức sử dụng).
- Tư duy chủ động đặt câu hỏi "vì sao".

7. Kết Quả Kỳ Vọng Sau Lộ Trình

Sau khi hoàn thành, bạn có thể:

- Phân tích đề bài thiết kế hệ thống một cách có cấu trúc.
- Thiết kế kiến trúc từ phiên bản nền đơn giản (*simple baseline*) đến kiến trúc có khả năng mở rộng (*scalable architecture*).
- Trình bày sự đánh đổi rõ ràng với nhóm kỹ thuật, nhóm sản phẩm và các bên liên quan (*stakeholder*).
- Tự tin hơn khi tham gia phỏng vấn thiết kế hệ thống.
- Ra quyết định kỹ thuật gắn với giá trị sản phẩm và mục tiêu dài hạn.

8. Cam Kết Thời Gian Thực Tế

- Tổng thời gian để hoàn thành từ Phần 0 đến Phần 5: khoảng **5-7 tháng** nếu học đều đặn.
- Phần 6 là quá trình liên tục để nâng tư duy lên tầm kiến trúc sư.
- Bạn không cần học nhanh. Bạn cần học **đúng cách và bền vững**.

9. Bắt Đầu Ngay

Nếu bạn mới đến với nền tảng, hãy bắt đầu từ **Phần 0 — Chuyển Đổi Mô Hình Tư Duy**.

Đây là phần quan trọng nhất để làm mới tư duy trước khi đi sâu vào kiến trúc phần mềm (*software architecture*) và hệ thống phân tán.

PHẦN 0

Chuyển Đổi Mô Hình Tư Duy

Giai đoạn khởi đầu giúp bạn chuyển từ tư duy lấy mã nguồn làm trung tâm (*code-first*) sang tư duy hệ thống (*system thinking*) trước khi học sâu về thiết kế hệ thống.

1) Giới Thiệu Phần

Phần 0 là cánh cửa mở vào toàn bộ hành trình thiết kế hệ thống.

Giai đoạn này không tập trung vào công cụ, framework hay chi tiết cài đặt (*implementation*). Mục tiêu chính là thay đổi cách bạn nhìn nhận và giải quyết vấn đề — từ **tư duy lấy mã nguồn làm trung tâm** (*code-centric thinking*) sang **tư duy ở tầm hệ thống** (*system-level thinking*).

Phần lớn lập trình viên gặp khó khi học thiết kế hệ thống không phải vì thiếu kỹ thuật, mà vì chưa có mô hình tư duy (*mental model*) đúng về hệ thống. Phần này giúp xây dựng nền tảng tư duy đó trước khi đi sâu vào kiến trúc và hệ thống phân tán (*distributed systems*).

Nếu không có bước chuyển này, bạn dễ:

- Chọn công nghệ trước khi hiểu vấn đề
- Tối ưu giải pháp cục bộ (*local solution*) thay vì toàn hệ thống
- Bỏ qua các ràng buộc (*constraint*) thực tế của môi trường vận hành thực tế (*production*)
- Không thấy được sự đánh đổi (*trade-off*) trong các quyết định kiến trúc

Phần 0 giúp bạn tránh những sai lầm này.

2) Vì Sao Phần Này Tồn Tại

Lập Trình Và Thiết Kế Hệ Thống Là Hai Lớp Tư Duy Khác Nhau

- **Lập trình:** tập trung vào cài đặt và logic chức năng.
- **Thiết kế hệ thống:** tập trung vào quy mô (*scale*), độ tin cậy (*reliability*), ràng buộc và sự đánh đổi.

Một hệ thống tốt không chỉ chạy đúng — mà còn phải chạy ổn định, mở rộng được và phù hợp với bối cảnh thực tế.

Tư Duy Kỹ Sư Và Tư Duy Kiến Trúc Sư Khác Nhau

- **Tư duy kỹ sư** (*Engineer mindset*): giải quyết nhiệm vụ trước mắt.
- **Tư duy kiến trúc sư** (*Architect mindset*): nhìn toàn hệ thống, dự đoán điểm nghẽn (*bottleneck*) và tối ưu dài hạn.

Phần này giúp bạn chuyển sang góc nhìn của kiến trúc sư.

Đánh Đổi Là Trung Tâm Của Mọi Quyết Định Kiến Trúc

Không có giải pháp hoàn hảo.

Chỉ có giải pháp phù hợp với:

- quy mô
- ràng buộc
- chi phí (*cost*)
- độ trễ (*latency*)
- độ tin cậy
- bối cảnh kinh doanh

3) Mục Tiêu Học Tập

Sau phần này, bạn sẽ:

- Chuyển từ tư duy lấy mã nguồn làm trung tâm (*code-first*) sang lập luận lấy bài toán làm trung tâm (*problem-first*).
- Hiểu rằng thiết kế hệ thống là quản lý ràng buộc và sự đánh đổi.
- Hình thành trực giác về quy mô và điểm nghẽn.
- Nhìn hệ thống như một tổng thể thay vì từng thành phần (*component*) riêng lẻ.
- Hiểu kiến trúc tổng quan (*high-level architecture*) của hệ thống phân tán.
- Có khung tư duy (*framework*) trước khi đi vào kỹ thuật chuyên sâu.

4) Bạn Sẽ Học Gì Trong Phần Này

Phần này xây dựng nền tảng tư duy cốt lõi của một kiến trúc sư hệ thống:

- **Tư duy lấy bài toán làm trung tâm** (*Problem-first thinking*) — bắt đầu từ bài toán, không bắt đầu từ công cụ.
 - **Tư duy đánh đổi** — hiểu mặt được và mặt mất trong mỗi quyết định.
 - **Các thành phần hệ thống & cách giao tiếp** — các thành phần chính của hệ thống hiện đại.
 - **Tư duy theo ràng buộc** — thiết kế dựa trên giới hạn thực tế.
 - **Trực giác về quy mô** — hiểu cách hệ thống hoạt động ở quy mô lớn.
 - **Khung tư duy hệ thống** (*System thinking framework*) — nhìn hệ thống như một luồng (*flow*) hoàn chỉnh.
-

5) Tổng Quan Các Bài Học

Bài 1 – Từ Lấy Mã Nguồn Làm Trung Tâm Đến Lấy Bài Toán Làm Trung Tâm – Thay Đổi Tư Duy Kiến Trúc Sư

Xây nền tư duy bắt đầu từ bài toán, yêu cầu và ràng buộc trước khi chọn giải pháp kỹ thuật.

Bài 2 – Từ Giải Pháp Hoàn Hảo Đến Tư Duy Đánh Đổi – Nghệ Thuật Của Quyết Định Kiến Trúc

Hiểu rằng mọi lựa chọn kiến trúc đều có chi phí và giới hạn.

Học cách ra quyết định trong môi trường vận hành thực tế.

Bài 3 – Tư Duy Theo Ràng Buộc (*Thinking in Constraints*) – Kỹ Thuật Là Quản Lý Giới Hạn

Hiểu rằng làm kỹ thuật thực chất là quản lý tài nguyên và ràng buộc: năng lực tính toán, bộ nhớ, mạng và dữ liệu.

Bài 4 – Trực Giác Về Quy Mô – Xây Dựng Cảm Giác Về Các Con Số

Xây dựng trực giác về lưu lượng (*traffic*), tốc độ tăng trưởng dữ liệu, thông lượng (*throughput*) và điểm nghẽn khi hệ thống mở rộng.

Bài 5 – Khung Tư Duy Hệ Thống – Nhìn Hệ Thống Như Một Tổng Thể

Học cách nhìn hệ thống theo luồng: đầu vào → xử lý → lưu trữ → đầu ra, thay vì từng mô-đun rời rạc.

Bài 6 – Khung Câu Hỏi Hệ Thống (*System Question Framework*) – Danh Sách Kiểm Tra Chuẩn Khi Thiết Kế Hệ Thống

Học cách đặt câu hỏi đúng khi thiết kế hệ thống.

Bài 7 – Tổng Kết Mô Hình Tư Duy – Củng Cố Tư Duy Thiết Kế Hệ Thống

Tổng kết Phần 0 với 4 bước chuyển tư duy quan trọng nhất: không có giải pháp tốt nhất, luôn hỏi "tại sao", sự đánh đổi quan trọng hơn công nghệ, và tư duy hệ thống (*system thinking*) đặt trên mã nguồn. Đây là nền tảng tư duy cho một kiến trúc sư hệ thống.

6) Kết Quả Sau Khi Hoàn Thành Phần 0

Bạn có thể:

- Nhìn hệ thống dưới góc độ bài toán và ràng buộc.
- Giải thích quyết định kiến trúc bằng sự đánh đổi thay vì cảm tính.
- Hiểu kiến trúc tổng quan của một hệ thống.
- Dự đoán các điểm nghẽn và rủi ro cơ bản.
- Thảo luận về thiết kế hệ thống với tư duy có cấu trúc.

Bạn sẽ không còn chỉ "viết mã", mà bắt đầu **thiết kế hệ thống**.

7) Chuẩn Bị Cho Phần 1

Phần 0 giúp bạn có **cách nghĩ đúng**.

Phần 1 — **Nền Tảng: Tư Duy Theo Hệ Thống** sẽ mở rộng từ tư duy sang nền tảng kỹ thuật:

- các thành phần hệ thống
- các mẫu giao tiếp (*communication patterns*)
- luồng dữ liệu (*data flow*)
- độ trễ và thông lượng
- điểm nghẽn (*bottlenecks*)
- tính sẵn sàng (*availability*)
- tính nhất quán (*consistency*)
- trực giác về CAP

Bạn sẽ chuyển từ tư duy → kiến thức hệ thống cốt lõi.

Từ Ưu Tiên Viết Mã đến Ưu Tiên Vấn Đề: Bước Đầu Thay Đổi Tư Duy

Tôi còn nhớ rõ lần đầu tiên bị một kiến trúc sư lâu năm (*senior architect*) hỏi: "Tại sao em chọn Redis?"

Câu trả lời của tôi lúc đó? "Vì Redis nhanh ạ."

Anh ấy mỉm cười: "Nhanh hơn cái gì? Giải quyết vấn đề gì? Chi phí (*cost*) của nó là gì?"

Tôi câm nín. Đó là lúc tôi nhận ra: mình đang nghĩ như một lập trình viên, không phải một kiến trúc sư (*architect*).

Tại Sao Phải Thay Đổi Tư Duy?

Khi bạn giỏi viết mã, bạn có xu hướng nhảy thẳng vào giải pháp. Thấy bộ nhớ đệm (*cache*)? Nghĩ ngay đến Redis. Thấy hàng đợi (*queue*)? RabbitMQ. Vi dịch vụ (*Microservices*)? Kubernetes.

Nhưng đó là **tư duy ưu tiên giải pháp** (*solution-first thinking*) - tư duy nguy hiểm nhất trong thiết kế hệ thống (*system design*).

Tại sao? Vì bạn đang chọn công cụ trước khi hiểu vấn đề. Giống như mua thuốc trước khi biết mình bệnh gì.

Một kiến trúc sư lâu năm không bắt đầu bằng "Tôi sẽ dùng X". Họ bắt đầu bằng "Vấn đề tôi cần giải quyết là gì?"

Tư Duy Ưu Tiên Viết Mã (*Code-First Thinking*): Tư Duy Lập Trình Viên

Đây là cách một lập trình viên (*developer*) thường nghĩ:

Kịch bản: Trang web bị chậm

Tư duy lập trình viên:

"Trang web chậm → Cài bộ nhớ đệm → Dùng Redis → Xong!"

Nghe có vẻ hợp lý, nhưng thiếu rất nhiều thứ:

- Chậm ở đâu? Truy vấn cơ sở dữ liệu (*database queries*)? Mạng (*network*)? Lời gọi API (*API calls*)?
- Chậm khi nào? Giờ cao điểm (*peak hours*)? Mọi lúc?
- Chậm bao nhiêu? 100ms hay 5s?
- Bộ nhớ đệm thực sự giải quyết được không?
- Chi phí của bộ nhớ đệm là gì? Bộ nhớ (*memory*)? Độ phức tạp (*complexity*)? Dữ liệu cũ (*stale data*)?

Đó là 5 câu hỏi mà tư duy ưu tiên viết mã bỏ qua. Và đó chính là lý do nhiều giải pháp kỹ thuật thất bại.

Ví Dụ Thực Tế: Thảm Họa Vi Dịch Vụ

Tôi từng chứng kiến một công ty khởi nghiệp (*startup*) với 0 người dùng quyết định xây dựng hệ thống bằng vi dịch vụ từ ngày đầu.

Lý do? "Netflix dùng vi dịch vụ mà."

Kết quả sau 6 tháng:

- 15 dịch vụ (*services*) cho một tính năng đơn giản
- Triển khai (*deploy*) mất 2 giờ
- Gỡ lỗi (*debug*) là một cơn ác mộng (lỗi nằm ở dịch vụ nào?)
- Cả nhóm kiệt sức vì độ phức tạp

Họ đã giải quyết được vấn đề gì? Không có. Vì họ chưa hề có vấn đề về quy mô (*scale*). Họ cần một hệ thống nguyên khối (*monolith*) đơn giản, ra mắt nhanh, cải tiến nhanh.

Bài học: Netflix dùng vi dịch vụ để giải quyết vấn đề của Netflix (hàng triệu người dùng, hàng trăm nhóm phát triển). Công ty khởi nghiệp kia không có vấn đề đó.

Tư Duy Ưu Tiên Vấn Đề (*Problem-First Thinking*): Tư Duy Kiến Trúc Sư

Đây là cách một kiến trúc sư tiếp cận:

Kịch bản: Trang web bị chậm

Tư duy kiến trúc sư:

1. Xác định vấn đề:

- Chậm ở đâu? → Đo đạc: 80% thời gian nằm ở truy vấn cơ sở dữ liệu
- Truy vấn nào? → 3 truy vấn chậm nhất chiếm 90% tải

2. Hiểu nguyên nhân:

- Tại sao chậm? → Thiếu chỉ mục (index)
- Tại sao lại thiếu? → Thiết kế lược đồ (schema) không tối ưu cho cách truy vấn

3. Đánh giá các phương án:

- Phương án A: Thêm chỉ mục → Đơn giản, hiệu quả
- Phương án B: Lưu đệm kết quả truy vấn → Phức tạp, có thể không cần
- Phương án C: Viết lại truy vấn → Tốn thời gian

4. Chọn giải pháp:

- Bắt đầu với chỉ mục (đơn giản, tác động lớn)
- Giám sát
- Thêm bộ nhớ đệm nếu vẫn chưa đủ

Thấy sự khác biệt chưa?

Kiến trúc sư không nhảy thẳng vào Redis. Họ **đo đạc, phân tích, đánh giá, rồi mới quyết định**.

Khung tư duy (*Framework*): 5 Câu Hỏi Vàng

Trước khi chọn bất kỳ giải pháp kỹ thuật nào, hãy tự hỏi:

1. Vấn đề thực sự là gì?

Không phải "trang web chậm". Mà là "thời gian tải trang chủ là 3 giây, người dùng kỳ vọng dưới 1 giây".

Cụ thể, đo đạc được, có mục tiêu (*target*) rõ ràng.

2. Tại sao vấn đề này xảy ra?

Đào sâu. Hỏi "Tại sao?" ít nhất 3 lần.

Trang web chậm

- Tại sao? Truy vấn cơ sở dữ liệu chậm
- Tại sao? Phải quét toàn bộ bảng (full table scan)
- Tại sao? Không có chỉ mục trên cột email
- Nguyên nhân gốc: Thiếu chỉ mục

3. Giải pháp này giải quyết vấn đề gì?

"Redis làm trang web nhanh" là câu trả lời tồi.

"Redis lưu đệm kết quả truy vấn để giảm tải (*load*) cơ sở dữ liệu từ 1000 req/s xuống 100 req/s" là câu trả lời tốt.

4. Chi phí của giải pháp là gì?

Mọi giải pháp đều có chi phí:

- Redis: Thêm hạ tầng (*infrastructure*), độ phức tạp của việc vô hiệu hoá bộ nhớ đệm (*cache invalidation*), chi phí bộ nhớ
- Ví dụ: Độ trễ (*latency*) mạng, độ phức tạp của hệ thống phân tán (*distributed system*), gỡ lỗi khó hơn
- Phân mảnh dữ liệu (*Sharding*): Truy vấn xuyên nhiều mảnh rất khó, phân mảnh lại (*resharding*) là cơn ác mộng

Không có bữa trưa nào miễn phí. Sự đánh đổi (*Trade-off*) luôn luôn tồn tại.

5. Có cách đơn giản hơn không?

Đây là câu hỏi quan trọng nhất.

Trước khi thêm Redis, bạn đã thử thêm chỉ mục chưa? Trước khi chuyển sang vi dịch vụ, bạn đã thử mở rộng hệ thống nguyên khối chưa?

"Giải pháp đơn giản trước. Giải pháp phức tạp khi thật sự cần."

Ví Dụ So Sánh: Thiết Kế Tính Năng "Đếm Lượt Xem"

Hãy xem hai cách tiếp cận (*approach*) khác nhau cho cùng một tính năng.

Cách tiếp cận 1: Ưu Tiên Viêt Mã (Tệ)

Tính năng: Đếm số lượt xem video

Tư duy lập trình viên:

"Cần đếm lượt xem → Dùng bộ đếm Redis → Tăng mỗi lượt xem → Xong"

Cài đặt:

- Mỗi lượt xem → `redis.incr('video:123:views')`
- Hiển thị: `redis.get('video:123:views')`

Trông ổn đấy. Nhưng sau 1 tháng chạy trên môi trường thực tế (*production*):

Vấn đề phát sinh:

- Máy chủ (*server*) Redis khởi động lại → Mất hết dữ liệu (lượt xem về 0)
- Không có dữ liệu lịch sử (*historical data*) (lượt xem theo ngày/tháng)
- Không theo dõi được xu hướng (*trending*) (video có lượt xem tăng nhanh)
- Lưu lượng (*traffic*) từ bot spam tăng bộ đếm

Lập trình viên bây giờ phải tái cấu trúc (*refactor*) toàn bộ, di chuyển dữ liệu (*migrate data*), xử lý thời gian ngừng hoạt động (*downtime*).

Cách tiếp cận 2: Ưu Tiên Vấn Đề (Tốt)

Tính năng: Đếm số lượt xem video

Tư duy kiến trúc sư:

1. Yêu cầu:

- Độ chính xác: Có cần chính xác 100%? (Có lẽ là không)
- Quy mô: Bao nhiêu video? Bao nhiêu lượt xem/giây?
- Phân tích: Có cần theo dõi theo thời gian không?
- Gian lận: Có cần chặn bot spam không?

2. Thiết kế:

- Ghi vào cơ sở dữ liệu (lưu bền vững, là nguồn dữ liệu gốc)
- Ghi theo lô bất đồng bộ (gom 10 giây) để giảm tải CSDL
- Lưu đệm trong Redis (đọc nhanh, có thể sai lệch 1-2%)
- Tác vụ nền đồng bộ Redis ↔ CSDL mỗi phút

3. Các đánh đổi chấp nhận:

- Số lượt xem có thể trễ vài giây (chấp nhận được)
- Có thể sai lệch nhỏ (chấp nhận được với chỉ số mạng xã hội)
- Phức tạp hơn giải pháp 1 (cần thiết để đảm bảo độ tin cậy)

Cách tiếp cận này xử lý được:

- Redis gặp sự cố → Chuyển sang đọc từ cơ sở dữ liệu
- Dữ liệu lịch sử → Có sẵn trong cơ sở dữ liệu
- Xu hướng → Truy vấn cơ sở dữ liệu theo khoảng thời gian
- Bot spam → Giới hạn tần suất (*rate limit*) + lớp kiểm tra hợp lệ

Bài Tập Thực Hành: Thay Đổi Tư Duy (*Mindset*)

Bài tập 1: Phân Tích Quyết Định Của Bạn

Nhớ lại một quyết định kỹ thuật gần đây bạn đã làm. Trả lời:

1. **Bạn chọn giải pháp gì?** (VD: Dùng MongoDB thay vì PostgreSQL)
2. **Lý do bạn chọn?** Viết ra lý do thật sự, không phải lý do "đẹp".
3. **Bạn đã đo đạc vấn đề chưa?** Có số liệu đo (*metrics*) cụ thể không?
4. **Bạn có xem xét các phương án thay thế** (*alternatives*) **không?** Có so sánh với PostgreSQL không?
5. **Chi phí và đánh đổi là gì?** Bạn đã nghĩ đến chưa?

Nếu câu trả lời cho câu 3, 4, 5 là "không" → Đó là tư duy ưu tiên viết mã.

Bài tập 2: Phân Tích Ngược (*Reverse Engineering*)

Chọn một app bạn hay dùng (Facebook, Instagram, Shopee...).

Nhiệm vụ: Dùng đoán họ dùng công nghệ gì. Thay vào đó:

1. **Xác định các vấn đề họ phải giải quyết:**
 - Quy mô: Bao nhiêu người dùng?
 - Tính nhất quán (*Consistency*): Số lượt thích có cần chính xác tuyệt đối không?
 - Độ trễ: Bảng tin (*feed*) cần tải nhanh đến mức nào?
2. **Lập luận:**
 - Với quy mô này, họ không thể chỉ dùng 1 cơ sở dữ liệu
 - Với yêu cầu về độ trễ, họ phải có lớp bộ nhớ đệm (*cache layer*)
 - Với yêu cầu về tính nhất quán, họ có thể chấp nhận nhất quán cuối cùng (*eventual consistency*)

3. Dự đoán giải pháp:

- Có lẽ dùng CDN cho tài nguyên tĩnh (*static assets*)
- Có lẽ dùng bộ nhớ đệm cho bảng tin
- Có lẽ dùng nhất quán cuối cùng cho số lượt thích

Thấy sự khác biệt chưa? Bạn đang lập luận từ vấn đề → giải pháp, không phải ngược lại.

Danh Sách Tự Kiểm Tra: Bạn Đã Thay Đổi Tư Duy Chưa?

Đánh dấu những câu bạn đồng ý:

- ☐ Tôi hỏi "Vấn đề là gì?" trước khi nghĩ giải pháp
- ☐ Tôi đo đạc vấn đề trước khi tối ưu (*optimize*)
- ☐ Tôi có thể giải thích "Tại sao" cho mọi quyết định kỹ thuật
- ☐ Tôi xem xét ít nhất 2-3 phương án thay thế trước khi chọn
- ☐ Tôi hiểu các đánh đổi của giải pháp tôi chọn
- ☐ Tôi bắt đầu đơn giản, chỉ thêm độ phức tạp khi cần thiết
- ☐ Tôi có thể bảo vệ quyết định của mình bằng dữ liệu và lập luận

Nếu bạn đánh dấu được 5/7 → Bạn đang đi đúng hướng.

Những Điểm Cốt Lõi

Tư duy ưu tiên vấn đề không phải là biết thật nhiều công nghệ.

Nó về việc:

- Hiểu vấn đề sâu sắc
- Đo đạc và phân tích
- Cân nhắc các đánh đổi
- Chọn giải pháp phù hợp (không phải "tốt nhất", mà là "vừa vặn")

Tư duy ưu tiên viết mã là phản xạ: Thấy vấn đề → Nghĩ ngay đến giải pháp quen thuộc.

Tư duy ưu tiên vấn đề là có chủ đích: Hiểu vấn đề → Phân tích → Đánh giá các phương án → Quyết định.

Sự khác biệt này quyết định bạn là một lập trình viên giỏi hay một kiến trúc sư giỏi.

Câu hỏi quan trọng nhất mà một architect luôn tự hỏi:

"Tôi đang giải quyết vấn đề gì? Và đây có thực sự là cách tốt nhất không?"

Bắt đầu hỏi câu này từ hôm nay. Mọi thứ sẽ thay đổi.

Từ Giải Pháp Hoàn Hảo đến Tư Duy Đánh Đổi

Có một câu hỏi mà mọi kỹ sư mới vào nghề (*junior engineer*) đều từng hỏi tôi: "Cách tốt nhất để thiết kế hệ thống này là gì?"

Câu trả lời của tôi luôn làm họ thất vọng: "Không có 'cách tốt nhất'. Chỉ có cách phù hợp nhất với các ràng buộc (*constraints*) của bạn."

Đó là bài học khó nhất mà mọi kiến trúc sư (*architect*) phải học: **không có giải pháp hoàn hảo.**

Tại Sao "Giải Pháp Hoàn Hảo" Không Tồn Tại?

Khi tôi mới bắt đầu, tôi tin rằng với đủ thời gian và kiến thức, tôi có thể tìm ra "kiến trúc hoàn hảo" - một kiến trúc không có nhược điểm.

Sau 10 năm thiết kế hệ thống, tôi nhận ra một sự thật tàn khốc:

Mọi quyết định kiến trúc đều là một sự đánh đổi.

Không phải vì bạn không đủ giỏi. Mà vì trong thế giới thực, bạn luôn phải chọn giữa những thứ mâu thuẫn nhau:

- **Hiệu năng** (*Performance*) **hay Tính nhất quán** (*Consistency*): Nhanh hay chính xác?
- **Khả năng mở rộng** (*Scalability*) **hay Sự đơn giản**: Dễ mở rộng hay dễ bảo trì (*maintain*)?
- **Tính linh hoạt hay Hiệu năng**: Linh hoạt hay tối ưu?
- **Chi phí** (*Cost*) **hay Độ tin cậy** (*Reliability*): Rẻ hay ổn định?

Bạn không thể có tất cả. Và đó không phải là vấn đề. Đó là thực tế.

Huyền Thoại Về "Thực Hành Tốt Nhất" (Best Practices)"

Tôi thường nghe các lập trình viên (*developers*) nói: "Vi dịch vụ (*Microservices*) là thực hành tốt nhất" hoặc "NoSQL tốt hơn SQL".

Không. Sai hoàn toàn.

Vi dịch vụ là thực hành tốt nhất cho Netflix với 200 triệu người dùng và 500 nhóm kỹ sư. Nó là **thực hành tệ nhất** cho một công ty khởi nghiệp (*startup*) 5 người với 100 người dùng.

NoSQL tốt cho Facebook với hàng tỷ bài đăng không cần quan hệ chặt chẽ. Nó là **lựa chọn tồi tệ** cho hệ thống ngân hàng cần giao dịch (*transactions*) ACID.

"Thực hành tốt nhất" không tồn tại trong chân không. Bối cảnh (*Context*) **quyết định tất cả.**

Ví Dụ Thực Tế: Hệ Thống Nguyên Khối (*Monolith*) hay Vi Dịch Vụ

Hãy phân tích sự đánh đổi (*trade-off*) của hai cách tiếp cận (*approaches*) này.

Hệ thống nguyên khối:

Ưu điểm:

- Triển khai đơn giản (1 ứng dụng)
- Gỡ lỗi dễ dàng (1 kho mã nguồn)
- Không có độ trễ mạng giữa các thành phần
- Giao dịch đơn giản (cùng một cơ sở dữ liệu)
- Phát triển nhanh ở giai đoạn đầu

Nhược điểm:

- Khó mở rộng độc lập từng phần
- Triển khai kiểu "tất cả hoặc không có gì"
- Các nhóm xung đột trên cùng kho mã nguồn
- Bị khoá chặt vào một công nghệ
- Phải khởi động lại toàn bộ ứng dụng khi cập nhật

Vi dịch vụ:

Ưu điểm:

- Mở rộng độc lập
- Triển khai độc lập
- Đa dạng công nghệ
- Các nhóm tự chủ
- Cô lập lỗi

Nhược điểm:

- Độ trễ mạng
- Giao dịch phân tán là cơn ác mộng
- Triển khai phức tạp
- Giám sát phức tạp
- Thách thức về tính nhất quán dữ liệu

Câu hỏi không phải là "cái nào tốt hơn?". Câu hỏi là:

"Với bối cảnh hiện tại của tôi, tôi cần những ưu điểm nào và có thể chấp nhận những nhược điểm nào?"

Khung tư duy (Framework): Ma Trận Đánh Đổi

Đây là khung tư duy tôi dùng để đánh giá mọi quyết định kiến trúc.

Bước 1: Xác Định Các Ràng Buộc Của Bạn

Trước khi so sánh giải pháp, hãy hiểu rõ các ràng buộc của bạn:

Ràng buộc kỹ thuật:

- Quy mô (Scale) hiện tại: 1 nghìn người dùng hay 100 triệu người dùng?
- Tốc độ tăng trưởng: gấp 2 lần/năm hay gấp 10 lần/tháng?
- Quy mô nhóm: 2 lập trình viên hay 50 lập trình viên?
- Yêu cầu về độ trễ (latency): dưới 100ms hay dưới 1 giây là ổn?

Ràng buộc kinh doanh:

- Ngân sách: Không giới hạn hay eo hẹp?
- Thời gian ra thị trường: Ra mắt trong 1 tháng hay 6 tháng?
- Quy định pháp lý: GDPR, tuân thủ PCI?

Ràng buộc về nhóm:

- Chuyên môn: Nhóm biết những gì?

- Thời gian học: Có thời gian để học không?
- Năng lực vận hành: Ai sẽ bảo trì?

Bước 2: Liệt Kê Các Đánh Đổi Một Cách Rõ Ràng

Với mỗi phương án, viết ra rõ ràng:

Phương án A: PostgreSQL

Được:

- Đảm bảo ACID
- Khả năng truy vấn phong phú (JOIN, tổng hợp)
- Hệ sinh thái trưởng thành
- Nhóm đã biết

Mất:

- Giới hạn khi mở rộng theo chiều dọc
- Thay đổi lược đồ (schema) tốn kém
- Không tối ưu cho dữ liệu phi cấu trúc

Phương án B: MongoDB

Được:

- Mở rộng theo chiều ngang dễ hơn
- Lược đồ linh hoạt
- Phù hợp để cải tiến nhanh

Mất:

- Không hỗ trợ JOIN (phải xử lý ở tầng ứng dụng)
- Độ phức tạp của nhất quán cuối cùng
- Nhóm phải học mới

Bước 3: Đối Chiếu Với Các Ưu Tiên

Xếp hạng các ưu tiên của dự án:

Ưu tiên 1: Ra mắt nhanh (MVP trong 2 tháng)

Ưu tiên 2: Năng suất của nhóm (ít phải học thêm)

Ưu tiên 3: Toàn vẹn dữ liệu (dữ liệu tài chính)

Với các ưu tiên này → PostgreSQL thắng

Tại sao? Nhóm đã biết, ACID là then chốt, có thể mở rộng sau.

Nếu các ưu tiên thay đổi:

Ưu tiên 1: Mở rộng lên 100 triệu người dùng (đã đạt PMF)
Ưu tiên 2: Xử lý dữ liệu phi cấu trúc (nội dung do người dùng tạo)
Ưu tiên 3: Hiệu năng đọc

Với các ưu tiên này → MongoDB có thể thắng
Tại sao? Khả năng mở rộng + tính linh hoạt quan trọng hơn hạn chế về JOIN.

Thấy chưa? Cùng 2 phương án, khác ưu tiên → khác quyết định.

Tình Huống Thực Tế: Có Dùng Bộ Nhớ Đệm (Cache) Hay Không?

Hãy xem một quyết định tưởng chừng đơn giản: "Có nên dùng bộ nhớ đệm không?"

Hầu hết lập trình viên nghĩ: "Bộ nhớ đệm làm nhanh hơn → Lúc nào cũng nên dùng!"

Sai rồi. Hãy phân tích các đánh đổi.

Kịch bản: Trang Sản Phẩm Thương Mại Điện Tử

Không có bộ nhớ đệm:

Yêu cầu của người dùng → Truy vấn CSDL → Trả về

Ưu điểm:

- Dữ liệu luôn mới nhất (giá, tồn kho)
- Logic đơn giản (không cần vô hiệu hoá bộ nhớ đệm)
- Không tốn thêm hạ tầng

Nhược điểm:

- Tải lên CSDL cao
- Phản hồi chậm (mỗi yêu cầu đều truy vấn CSDL)
- Khó mở rộng khả năng đọc

Có bộ nhớ đệm (Redis):

Yêu cầu của người dùng → Kiểm tra Redis → Nếu không có: Truy vấn CSDL → Trả về`

Ưu điểm:

- Phản hồi nhanh ($< 10\text{ms}$)
- Giảm tải cho CSDL
- Dễ mở rộng khả năng đọc

Nhược điểm:

- Nguy cơ dữ liệu cũ (người dùng thấy sản phẩm đã hết hàng vẫn còn hàng)
- Độ phức tạp của việc vô hiệu hoá bộ nhớ đệm
- Chi phí hạ tầng (máy chủ Redis)
- Giới hạn bộ nhớ

Quyết Định Đánh Đổi

Nếu bạn là Amazon:

→ **Dùng bộ nhớ đệm** vì:

- Hàng triệu yêu cầu/giây → CSDL không chịu nổi
- Dữ liệu hơi cũ một chút vẫn chấp nhận được (số tồn kho có thể sai 1-2 sản phẩm)
- Có ngân sách cho hạ tầng
- Nhóm có chuyên môn

Nếu bạn là công ty khởi nghiệp nhỏ:

→ **Bỏ qua bộ nhớ đệm** vì:

- 100 yêu cầu/giây → CSDL xử lý được
- Dữ liệu mới nhất quan trọng hơn (tồn kho ít)
- Tiết kiệm chi phí hạ tầng
- Giữ mọi thứ đơn giản

Cùng một vấn đề, khác bối cảnh, khác quyết định.

Những Đánh Đổi Phổ Biến Mọi Kiến Trúc Sư Phải Biết

1. Tính Nhất Quán hay Tính Sẵn Sàng (*Availability*) (Định lý CAP)

Bạn không thể có cả hai trong một hệ thống phân tán (*distributed system*) khi xảy ra phân vùng mạng (*network partition*).

Chọn tính nhất quán (CP):

Ngân hàng, thanh toán, quản lý tài khoản
→ Đúng quan trọng hơn nhanh
→ Chấp nhận ngừng hoạt động nếu cần để đảm bảo tính đúng đắn

Chọn tính sẵn sàng (AP):

Bảng tin mạng xã hội, đếm lượt xem, gợi ý nội dung
→ Nhanh quan trọng hơn chính xác tuyệt đối
→ Nhất quán cuối cùng là chấp nhận được

Tư duy đánh đổi:

- Tính năng này CẦN nhất quán mạnh (*strong consistency*) không? (Thường là KHÔNG)
- Người dùng có bị ảnh hưởng nếu dữ liệu sai trong vài giây không?
- Chi phí để đảm bảo tính nhất quán là gì?

2. Chuẩn Hoá (*Normalization*) hay Phi Chuẩn Hoá (*Denormalization*)

Cơ sở dữ liệu chuẩn hoá:

```
-- 3 tables, proper 3NF
users: id, name
posts: id, user_id, content
likes: id, post_id, user_id

-- Query post with like count
SELECT p.*, COUNT(l.id) as likes
FROM posts p
LEFT JOIN likes l ON p.id = l.post_id
WHERE p.id = 123
GROUP BY p.id;
```

Toàn vẹn dữ liệu, không trùng lặp
Truy vấn chậm (JOIN tốn kém)

Phi chuẩn hoá:

```
-- 1 table
posts: id, user_id, content, like_count

-- Query post with like count
SELECT * FROM posts WHERE id = 123;
```

Đọc nhanh (không cần JOIN)
Nguy cơ thiếu nhất quán, tốn công khi cập nhật

Khi nào nên phi chuẩn hoá?

- Đọc >> Ghi (99% đọc, 1% ghi)
- Hiệu năng là yếu tố sống còn
- Có thể chấp nhận thiếu nhất quán đôi chút

Khi nào nên giữ chuẩn hoá?

- Ghi nhiều
- Toàn vẹn dữ liệu là yếu tố sống còn
- Quan hệ dữ liệu phức tạp

3. Xử Lý Bất Đồng Bộ (Async) hay Đồng Bộ (Sync)

Đồng bộ:

```
def upload_video(video):
    save_to_s3(video)          # 2s
    generate_thumbnail(video)  # 3s
    transcode_720p(video)     # 30s
    send_notification()        # 1s
    return "Success"          # User waits 36s
```

Đơn giản, phản hồi ngay lập tức
Trải nghiệm người dùng kém, nguy cơ hết thời gian chờ (timeout)

Bất đồng bộ:

```
def upload_video(video):
    save_to_s3(video)          # 2s
    queue.add_job('process_video', video.id)
    return "Success"          # User waits 2s

# Background worker processes rest
```

Trải nghiệm người dùng tốt hơn, chịu được đột biến tải (*spikes*)
Phức tạp, xử lý trễ, khó xử lý lỗi hơn

Quyết định đánh đổi:

- Người dùng có cần kết quả ngay không?
- Tiến trình có thể thất bại và thử lại (*retry*) được không?
- Nhóm có kinh nghiệm với các mẫu bất đồng bộ không?

Nghệ Thuật Của "Đủ Tốt"

Một trong những kỹ năng quan trọng nhất của kiến trúc sư: **Biết khi nào dừng tối ưu** (*optimize*).

Tôi từng thấy kỹ sư bỏ ra 2 tuần để tối ưu một truy vấn từ 50ms xuống 20ms.

Tác động? Người dùng không thấy khác biệt (ngưỡng cảm nhận của con người là khoảng 100ms).

Chi phí? 2 tuần làm việc = các tính năng khác bị trì hoãn.

"Hoàn hảo là kẻ thù của tốt."

Quy Tắc 80/20 Trong Kiến Trúc

20% công sức thường giải quyết 80% vấn đề.

Ví dụ: Tối Ưu Thời Gian Phản Hồi API

Hiện tại: 800ms
Mục tiêu: < 200ms

Phương án A: Thêm chỉ mục cho cơ sở dữ liệu

- Công sức: 2 giờ
- Kết quả: 800ms → 300ms (cải thiện 62%)

Phương án B: Thêm và triển khai bộ nhớ đệm Redis

- Công sức: 1 tuần
- Kết quả: 300ms → 150ms (cải thiện thêm 50%)

Phương án C: Chuyển sang Cassandra + viết lại truy vấn

- Công sức: 2 tháng
- Kết quả: 150ms → 100ms (cải thiện thêm 33%)

Quyết định khôn ngoan:

- Bắt đầu với Phương án A (hiệu quả đầu tư cao)
- Giám sát (*Monitor*)
- Phương án B nếu 300ms vẫn chưa đủ
- Có lẽ sẽ không bao giờ làm Phương án C (lợi ích giảm dần)

Bài Tập: Phân Tích Đánh Đổi

Bài tập 1: Thiết Kế Xác Thực (*Authentication*) Người Dùng

Yêu cầu: Hệ thống đăng nhập người dùng

Phương án A: JWT (**Phi trạng thái** (*Stateless*))

Liệt kê các đánh đổi:

- Được: ?
- Mất: ?
- Phù hợp nhất cho: ?

Phương án B: Phiên (*Session*) (**Có trạng thái** (*Stateful*))

Liệt kê các đánh đổi:

- Được: ?
- Mất: ?

- Phù hợp nhất cho: ?

Viết ra lập luận của bạn. Không có đúng/sai, chỉ có phù hợp/không phù hợp.

Bài tập 2: Tình Huống Thực Tế

Bối cảnh:

- Công ty khởi nghiệp, 5 kỹ sư
- Hiện tại: 10 nghìn người dùng
- Dự kiến: 100 nghìn người dùng trong 6 tháng
- Tính năng: Trò chuyện thời gian thực (*real-time*)

Các phương án:

1. Tự xây máy chủ WebSocket
2. Dùng Firebase Realtime Database
3. Dùng Twilio/Stream Chat (dịch vụ bên thứ ba)

Nhiệm vụ của bạn:

1. Liệt kê các đánh đổi của mỗi phương án
2. Xếp hạng các ưu tiên (thời gian, chi phí, khả năng mở rộng, quyền kiểm soát)
3. Đưa ra quyết định kèm lập luận

Không có "đáp án đúng". Hãy rèn luyện tư duy đánh đổi.

Các Mô Hình Tư Duy (*Mental Models*) để Đưa Ra Quyết Định

Mô hình 1: Khả Năng Đảo Ngược

Câu hỏi: Quyết định này có dễ thay đổi sau không?

Dễ đảo ngược (thay đổi dễ dàng):

- Lựa chọn bộ nhớ đệm (Redis → Memcached)
- Thư viện ORM
- Khung phát triển giao diện người dùng

→ Đừng nghĩ quá nhiều. Chọn nhanh, cải tiến sau.

Khó đảo ngược (thay đổi tốn kém):

- Lựa chọn cơ sở dữ liệu (SQL → NoSQL)
- Hệ thống nguyên khối → Vi dịch vụ
- Nhà cung cấp điện toán đám mây

→ Suy nghĩ kỹ. Cái giá của quyết định sai rất cao.

Mô hình 2: Tác Động và Công Sức

Tác động cao, Công sức thấp → LÀM NGAY
Tác động cao, Công sức cao → LẬP KẾ HOẠCH & ƯU TIÊN
Tác động thấp, Công sức thấp → LÀM NẾU CÓ THỜI GIAN
Tác động thấp, Công sức cao → KHÔNG LÀM

Ví dụ:

- Thêm chỉ mục cơ sở dữ liệu: Tác động cao, Công sức thấp → LÀM NGAY
- Viết lại thành vi dịch vụ: Tác động cao, Công sức cao → LẬP KẾ HOẠCH cẩn thận
- Tái cấu trúc (*Refactor*) tên biến: Tác động thấp, Công sức thấp → LÀM nếu có thời gian
- Tự xây hệ thống xác thực: Tác động thấp, Công sức cao → KHÔNG LÀM (dùng Auth0)

Mô hình 3: Giảm Thiểu Hối Tiếc

Câu hỏi: 6 tháng sau, quyết định nào tôi sẽ hối hận nhất?

Kịch bản: Chọn bộ công nghệ (*tech stack*) cho MVP của công ty khởi nghiệp

Quyết định A: Dùng framework mới đang thịnh hành (Deno, Bun...)

- Nguy cơ: Hệ sinh thái chưa trưởng thành, nhóm phải học, nguy cơ thất bại
- Khả năng hối tiếc: CAO (nếu thất bại, mất thời gian quý báu)

Quyết định B: Dùng bộ công nghệ "nhàm chán" đã được kiểm chứng (Node.js + Express + PostgreSQL)

- Nguy cơ: Không "ngầu", có thể bỏ lỡ tính năng mới
- Khả năng hối tiếc: THẤP (dù có chậm, nó vẫn chạy ổn định)

→ Với MVP của công ty khởi nghiệp: **Giảm thiểu hối tiếc** → **Chọn B**

Danh Sách Tự Kiểm Tra: Bạn Đã Tư Duy Theo Đánh Đổi Chưa?

Trước khi chốt bất kỳ quyết định kiến trúc nào:

- ☐ Tôi đã liệt kê ít nhất 2-3 phương án thay thế (*alternatives*)
- ☐ Tôi đã viết ra các đánh đổi (được và mất) cho mỗi phương án
- ☐ Tôi hiểu rõ bối cảnh và các ràng buộc hiện tại
- ☐ Tôi đã xếp hạng các ưu tiên rõ ràng
- ☐ Tôi có thể giải thích "tại sao phương án này PHÙ HỢP" (không phải "tốt nhất")
- ☐ Tôi đã cân nhắc khả năng đảo ngược của quyết định
- ☐ Tôi biết mình đang hy sinh cái gì để đổi lấy cái gì

Nếu đánh dấu được 6/7 → Bạn đang tư duy như một kiến trúc sư.

Những Điểm Cốt Lõi

Không có giải pháp hoàn hảo, chỉ có sự đánh đổi tối ưu.

Mọi quyết định kiến trúc là một sự đánh đổi. Công việc của kiến trúc sư không phải là tìm "thực hành tốt nhất", mà là:

1. **Hiểu rõ bối cảnh:** Quy mô, nhóm, ràng buộc
2. **Liệt kê đánh đổi một cách rõ ràng:** Được và Mất
3. **Đối chiếu với ưu tiên:** Tác động và Công sức
4. **Ra quyết định có cơ sở:** "Phù hợp" chứ không phải "hoàn hảo"
5. **Chấp nhận hệ quả:** Đón nhận cái giá của lựa chọn

Câu nói yêu thích của tôi:

"Không có giải pháp. Chỉ có sự đánh đổi." — Thomas Sowell

Một kiến trúc sư lâu năm giỏi không phải là người biết mọi mẫu thiết kế (*pattern*). Mà là người biết **mẫu nào hợp với bối cảnh nào, và tại sao**.

Đủ tốt, đã ra mắt và được cải tiến liên tục còn hơn hoàn hảo nhưng không bao giờ ra mắt.

Hãy bắt đầu nghĩ theo sự đánh đổi từ hôm nay. Mỗi khi bạn nói "giải pháp tốt nhất", hãy dừng lại và hỏi:

"Tốt nhất cho ai? Trong bối cảnh nào? Tôi đang đánh đổi cái gì?"

Thành Phần (Components) và Giao Tiếp (Communication): Hiểu Bản Chất Của Một Hệ Thống

Lần đầu tiên tôi phải gỡ lỗi (*debug*) một sự cố trên môi trường thực tế (*production*), tôi hoàn toàn choáng váng.

Phản hồi API chậm. Nhưng chậm ở đâu? Cơ sở dữ liệu (*Database*)? Ứng dụng? Mạng (*Network*)? Bộ nhớ đệm (*Cache*)?

Một kiến trúc sư lâu năm (*senior architect*) ngồi bên cạnh, vẽ một sơ đồ đơn giản:

Máy khách → Bộ cân bằng tải → Máy chủ ứng dụng → Bộ nhớ đệm → Cơ sở dữ liệu

Rồi anh hỏi: "Yêu cầu (*Request*) đi qua bao nhiêu bước? Mỗi bước mất bao lâu?"

Đó là lần đầu tiên tôi hiểu: **Hệ thống không phải là một khối mã. Nó là tập hợp các thành phần tương tác với nhau.**

Tại Sao Phải Hiểu Các Thành Phần?

Khi bạn viết mã, bạn nhìn vào hàm, lớp, mô-đun. Đó là **góc nhìn ở cấp độ mã nguồn**.

Khi bạn thiết kế hệ thống, bạn phải nhìn vào **các thành phần và cách chúng tương tác**. Đó là **góc nhìn ở cấp độ kiến trúc**.

Sự khác biệt?

Cấp độ mã nguồn:

```
def get_user(user_id):  
    return db.query("SELECT * FROM users WHERE id = ?", user_id)
```

Cấp độ kiến trúc:

Máy khách → API Gateway → Dịch vụ Người dùng → Cơ sở dữ liệu
(50ms) (10ms) (200ms)

Tổng độ trễ: 260ms

Điểm nghẽn: Cơ sở dữ liệu (200ms = 77% tổng thời gian)

Góc nhìn cấp độ kiến trúc giúp bạn:

- Nhanh chóng xác định điểm nghẽn (*bottlenecks*)
- Hiểu các điểm có thể gặp sự cố
- Thiết kế để mở rộng (*scale*)
- Trao đổi với nhóm

Bạn không thể tối ưu (*optimize*) cái bạn không hiểu.

Giải Phẫu Một Hệ Thống Web

Mọi hệ thống web, dù đơn giản hay phức tạp, đều có cùng những khối xây dựng (*building blocks*) cơ bản.

Thành phần 1: Máy Khách (*Client*) (Trình duyệt/Ứng dụng di động)

Vai trò: Giao diện người dùng, gửi yêu cầu, hiển thị phản hồi (*responses*)

Đặc điểm:

- Không đáng tin cậy (người dùng có thể sửa mã)
- Không kiểm soát được môi trường (mạng, thiết bị)
- Có trạng thái (*Stateful*) (lưu dữ liệu cục bộ)

Hàm ý cho kiến trúc sư (*architect*):

KHÔNG BAO GIỜ tin máy khách

- Chỉ kiểm tra hợp lệ ở máy khách là chưa đủ
- Không lưu dữ liệu nhạy cảm
- Không dựa vào logic phía máy khách cho các quy tắc nghiệp vụ

LUÔN kiểm tra hợp lệ phía máy chủ

- Kiểm tra ở máy khách chỉ để cải thiện trải nghiệm người dùng
- Kiểm tra ở máy chủ là yêu cầu bảo mật

Ví dụ thực tế:

Tôi từng thấy một ứng dụng kiểm tra giá sản phẩm ở máy khách. Kẻ tấn công sửa JavaScript, đổi giá từ \$100 thành \$1, rồi gửi biểu mẫu.

Máy chủ (Server) tin tưởng máy khách → Cơ sở dữ liệu lưu đơn hàng với giá \$1 → Công ty mất tiền.

Bài học: Máy khách là tầng hiển thị (*presentation layer*), không phải tầng nghiệp vụ (*business logic layer*).

Thành phần 2: Bộ Cân Bằng Tải (*Load Balancer*)

Vai trò: Phân phối lưu lượng (*traffic*) đều giữa nhiều máy chủ

Khi nào cần?

- Khi 1 máy chủ không đủ sức xử lý lưu lượng
- Khi cần tính sẵn sàng cao (*high availability*) (1 máy chủ chết, vẫn còn máy chủ khác)

Khi nào KHÔNG cần?

- Công ty khởi nghiệp với dưới 1000 người dùng
- Công cụ nội bộ với ít lưu lượng
- Bản thử nghiệm/MVP

Các đánh đổi (*trade-offs*):

Được:

- Phân phối tải đồng đều
- Không có điểm lỗi duy nhất
- Triển khai cuốn chiếu (cập nhật từng máy chủ)

Mất:

- Thêm 1 tầng phức tạp
- Bản thân bộ cân bằng tải có thể là điểm lỗi duy nhất (nếu không có dự phòng)
- Độ trễ mạng (+10-50ms)

Các thuật toán phổ biến:

Round Robin (xoay vòng):

Yêu cầu 1 → Máy chủ A
Yêu cầu 2 → Máy chủ B
Yêu cầu 3 → Máy chủ C
Yêu cầu 4 → Máy chủ A (lặp lại)

Đơn giản, phân phối đều. Nhưng không quan tâm máy chủ nào đang quá tải.

Least Connections (ít kết nối nhất):

Máy chủ A: 50 kết nối đang hoạt động
Máy chủ B: 20 kết nối đang hoạt động
Máy chủ C: 35 kết nối đang hoạt động

→ Yêu cầu mới được chuyển đến Máy chủ B

Thông minh hơn, nhưng cần theo dõi trạng thái.

Weighted Round Robin (xoay vòng có trọng số):

Máy chủ A (16GB RAM): trọng số' = 2
Máy chủ B (8GB RAM): trọng số' = 1

→ A nhận 2 yêu cầu cho mỗi 1 yêu cầu đến B

Hữu ích khi các máy chủ có cấu hình khác nhau.

Góc nhìn cá nhân: Trong thực tế, tôi thấy Round Robin là đủ cho 90% trường hợp. Least Connections chỉ cần khi các yêu cầu có thời gian xử lý rất khác nhau.

Thành phần 3: Máy Chủ Ứng Dụng

Vai trò: Xử lý nghiệp vụ, xử lý yêu cầu, điều phối luồng dữ liệu (*data flow*)

Phi trạng thái (Stateless) hay Có trạng thái - Quyết định quan trọng:

Máy chủ phi trạng thái:

```
# Không lưu user session trong memory
def handle_request(request):
    user_id = jwt.decode(request.token) # Get from token
    user = db.get_user(user_id)         # Get from DB
    return process(user)

# Benefit: Any server can handle any request
# Can scale horizontally easily
```

Máy chủ có trạng thái:

```
# Lưu user session trong memory
sessions = {} # In-memory storage

def handle_login(user_id):
    sessions[user_id] = UserSession(user_id)

def handle_request(user_id):
    session = sessions[user_id] # Retrieve from memory
    return process(session)

# Problem: User phải hit same server
# Scaling is hard (need sticky sessions)
```

Quyết định đánh đổi:

Chọn phi trạng thái khi:

- Cân mở rộng theo chiều ngang
- Nhiều máy chủ đứng sau bộ cân bằng tải
- Môi trường đám mây (máy chủ có thể chết bất cứ lúc nào)

Chọn có trạng thái khi:

- Cân độ trễ cực thấp (không phải gọi CSDL)
- Kết nối WebSocket (cần kết nối duy trì liên tục)
- Quy mô nhỏ (1-2 máy chủ)

Tôi khuyên: Mặc định chọn phi trạng thái. Chỉ chọn có trạng thái khi có lý do thật sự thuyết phục.

Thành phần 4: Lớp Bộ Nhớ Đệm (*Cache Layer*)

Vai trò: Lưu dữ liệu hay được truy cập trong bộ nhớ để giảm tải (*load*) cho cơ sở dữ liệu

Khi nào dùng bộ nhớ đệm?

Dùng bộ nhớ đệm khi:

- Đọc >> Ghi (tỷ lệ 10:1 trở lên)
- Dữ liệu không thay đổi thường xuyên
- Truy vấn tốn kém (JOIN phức tạp, tổng hợp)
- Các endpoint có lưu lượng cao

Không dùng bộ nhớ đệm khi:

- Tải công việc chủ yếu là ghi
- Dữ liệu thay đổi liên tục (giá cổ phiếu thời gian thực)
- Dữ liệu cá nhân hoá theo từng người dùng (mỗi người mỗi khác)

Ví dụ thực tế: Trang sản phẩm thương mại điện tử

```
# Product info: Perfect for cache
product = cache.get(f"product:{id}")
if not product:
    product = db.query("SELECT * FROM products WHERE id = ?", id)
    cache.set(f"product:{id}", product, ttl=3600) # 1 hour

# Inventory count: DON'T cache (changes frequently)
stock = db.query("SELECT stock FROM inventory WHERE product_id = ?", id)

# User's cart: DON'T cache (personalized)
cart = db.query("SELECT * FROM carts WHERE user_id = ?", user_id)
```

Vô Hiệu Hoá Bộ Nhớ Đệm (Cache Invalidation) - Bài Toán Khó:

Phil Karlton từng nói: "Chỉ có hai thứ khó trong Khoa học Máy tính: vô hiệu hoá bộ nhớ đệm và đặt tên."

Ông ấy đúng. Vô hiệu hoá bộ nhớ đệm là một cơn ác mộng.

Vấn đề:

1. Người dùng cập nhật hồ sơ → Cơ sở dữ liệu được cập nhật
2. Bộ nhớ đệm vẫn giữ dữ liệu cũ
3. Người dùng tải lại trang → Thấy dữ liệu cũ (trúng bộ nhớ đệm)
4. Người dùng bối rối, gửi báo cáo lỗi

Giải pháp:

Phương án 1: TTL (Thời gian sống (Time To Live))

```
cache.set("user:123", user, ttl=300) # Expire after 5 minutes
```

Simple

Data stale for up to 5 minutes

Phương án 2: Chủ động vô hiệu hoá

```
def update_user(user_id, data):
    db.update("users", user_id, data)
    cache.delete(f"user:{user_id}") # Delete immediately
```

Always fresh

Must remember to invalidate everywhere

Phương án 3: Mẫu Cache-Aside (Khuyến dùng)

```
def get_user(user_id):
    user = cache.get(f"user:{user_id}")
    if user:
        return user

    user = db.query("SELECT * FROM users WHERE id = ?", user_id)
    cache.set(f"user:{user_id}", user, ttl=3600)
    return user

def update_user(user_id, data):
    db.update("users", user_id, data)
    cache.delete(f"user:{user_id}") # Invalidate
    # Next read will miss cache → Fetch fresh from DB → Cache again
```

Nguyên tắc cá nhân: Bắt đầu đơn giản với TTL. Chỉ thêm cơ chế chủ động vô hiệu hoá khi độ tươi mới của dữ liệu thực sự quan trọng.

Thành phần 5: Cơ Sở Dữ Liệu

Vai trò: Lưu trữ bền vững, nguồn dữ liệu gốc (*source of truth*)

Cơ sở dữ liệu là điểm nghẽn phổ biến nhất.

Tại sao? Vì đọc/ghi đĩa (I/O) chậm hơn bộ nhớ 100.000 lần.

Đọc bộ nhớ: 100 nano giây
Đọc đĩa: 10 mili giây

10ms / 100ns = chậm hơn 100.000 lần

Hàm ý:

1. Giảm thiểu số lần gọi cơ sở dữ liệu

- Dùng bộ nhớ đệm
- Gộp truy vấn theo lô
- Tải sớm (eager loading) để tránh vấn đề N+1

2. Tối ưu truy vấn

- Thêm chỉ mục (indexes)
- Tránh SELECT *
- Dùng EXPLAIN ANALYZE

3. Mở rộng cơ sở dữ liệu sau cùng

- Tối ưu ứng dụng trước
- Thêm bộ nhớ đệm thứ hai
- Chỉ mở rộng CSDL khi thật cần thiết (tốn kém)

Ví dụ thực tế:

Một nhóm phản nàn: "Cơ sở dữ liệu quá chậm, cần nâng cấp máy chủ!"

Tôi kiểm tra nhật ký truy vấn:

```
-- Running 1000 times per second
SELECT * FROM users WHERE email = 'john@example.com';
-- Execution time: 500ms
```

Vấn đề? Không có chỉ mục trên cột email → Phải quét toàn bộ bảng.

Giải pháp:

```
CREATE INDEX idx_users_email ON users(email);
-- Execution time: 5ms (100x faster!)
```

Chi phí: \$0 và 30 giây thiết lập.

Thay vì nâng cấp máy chủ (\$500/tháng), sửa truy vấn (miễn phí). **Luôn tối ưu trước khi mở rộng.**

Các Mẫu Giao Tiếp (*Communication Patterns*): Đồng Bộ hay Bất Đồng Bộ

Đây là một trong những quyết định quan trọng nhất trong thiết kế hệ thống (*system design*).

Đồng Bộ (*Synchronous*) (Yêu cầu - Phản hồi)

Cách hoạt động:

```
Máy khách → [Yêu cầu HTTP] → Máy chủ
Máy khách ← [Chờ...] ←
Máy khách ← [Phản hồi HTTP] ← Máy chủ
```

Đặc điểm:

- Máy khách bị chặn và chờ phản hồi
- Nhận phản hồi ngay lập tức
- Liên kết chặt (*Tight coupling*) (máy khách phụ thuộc máy chủ)

Khi nào dùng:

- Người dùng cần kết quả ngay lập tức
- Quy trình đơn giản
- Các thao tác CRUD

Ví dụ:

```
# User login
def login(email, password):
    user = authenticate(email, password) # Must know now
    if user:
        token = generate_token(user)
        return {"token": token} # Return immediately
    return {"error": "Invalid credentials"}
```

Bất Đồng Bộ (Asynchronous) (Hàng Đợi Thông điệp (Message Queue))

Cách hoạt động:

Máy khách → [Yêu cầu HTTP] → Máy chủ → [Đưa vào hàng đợi] → Trả về ngay
↓
Worker nên → Xử lý công việc

Đặc điểm:

- Máy khách không phải chờ
- Được xử lý sau
- Liên kết lỏng (*Loose coupling*) (máy khách không quan tâm ai xử lý)

Khi nào dùng:

- Xử lý mất nhiều thời gian (> 5 giây)
- Người dùng không cần kết quả ngay
- Cần chịu được các đợt tăng đột biến lưu lượng (*traffic spikes*)

Ví dụ:

```
# User uploads video
def upload_video(video_file):
    video_id = save_to_s3(video_file)          # Quick (2s)
    queue.add({
        "job": "process_video",
        "video_id": video_id,
        "tasks": ["thumbnail", "transcode", "notify"]
    })
    return {"status": "processing", "id": video_id} # Return immediately

# Background worker (runs separately)
def process_video(video_id):
    generate_thumbnail(video_id)    # 5s
    transcode_720p(video_id)       # 30s
    transcode_1080p(video_id)     # 60s
    send_notification(video_id)    # 1s
```

Trải nghiệm người dùng:

- Tải lên → Nhận phản hồi sau 2 giây (tốt)
- Video sẵn sàng → Nhận thông báo sau 1-2 phút (chấp nhận được)

Nếu làm đồng bộ → Người dùng phải chờ 98 giây → Hết thời gian chờ (Timeout) → Trải nghiệm tệ.

Ma Trận Đánh Đổi

Đồng bộ:
Logic đơn giản
Phản hồi ngay lập tức
Dễ gỡ lỗi
Trải nghiệm kém với thao tác chậm
Nguy cơ hết thời gian chờ
Không chịu được lưu lượng tăng đột biến

Bất đồng bộ:
Trải nghiệm tốt hơn (phản hồi nhanh)
Chịu được tăng đột biến (hàng đợi làm vùng đệm)
Có sẵn cơ chế 'thư' lại
Phức tạp (cần hạ tầng hàng đợi)
Nhật quán cuối cùng
Khó gỡ lỗi hơn (cần truy vết phân tán)

Khung ra quyết định:

```
if user_needs_result_immediately:
    use_synchronous()
elif process_time > 5_seconds:
    use_asynchronous()
elif traffic_spiky: # Black Friday, viral post
    use_asynchronous()
else:
    use_synchronous() # Simpler
```

Luồng Dữ Liệu & Phát Hiện Điểm Nghẽn

Nguyên tắc vàng: Hệ thống chỉ nhanh bằng thành phần chậm nhất của nó.

Đo Lường Luồng Dữ Liệu

Ví dụ luồng: Người dùng tải trang chủ

Bước 1: Tra cứu DNS → 20ms
Bước 2: Bắt tay TCP → 50ms
Bước 3: Thương lượng TLS → 100ms
Bước 4: Gửi yêu cầu HTTP → 10ms
Bước 5: Máy chủ xử lý → 30ms
Bước 6: Truy vấn CSDL → 500ms ← ĐIỂM NGHẸN
Bước 7: Máy chủ dựng HTML → 20ms
Bước 8: Phản hồi về máy khách → 10ms
Bước 9: Trình duyệt phân tích/hiển thị → 200ms

Tổng: 940ms

Điểm nghẽn: Cơ sở dữ liệu (500ms = 53% tổng)

Kế hoạch hành động:

1. Tối ưu truy vấn cơ sở dữ liệu (thêm chỉ mục)
2. Thêm lớp bộ nhớ đệm
3. Đo lại

Sau khi tối ưu:

Bước 6: Trúng bộ nhớ đệm → 5ms

Tổng: 445ms (nhanh hơn 52%!)

Bài học: Đừng tối ưu bừa. Đo lường → Tìm điểm nghẽn → Tối ưu điểm nghẽn.

Các Điểm Nghẽn Phổ Biến & Giải Pháp

Điểm nghẽn 1: Cơ sở dữ liệu

Triệu chứng: Độ trễ truy vấn cao, CPU/IO chạm ngưỡng tối đa

Giải pháp:

- Thêm chỉ mục
- Tối ưu truy vấn
- Thêm bản sao chỉ đọc (read replica)
- Triển khai bộ nhớ đệm

Điểm nghẽn 2: Mạng

Triệu chứng: Độ trễ cao, hết thời gian chờ

Giải pháp:

- CDN cho tài nguyên tĩnh
- Giảm kích thước dữ liệu truyền (nén)
- Ít yêu cầu HTTP hơn (gộp tệp)

Điểm nghề 3: Logic ứng dụng

Triệu chứng: CPU sử dụng cao, phản hồi chậm

Giải pháp:

- Phân tích hiệu năng mã (profiling)
- Tối ưu thuật toán ($O(n^2) \rightarrow O(n \log n)$)
- Thêm bộ nhớ đệm
- Mở rộng theo chiều ngang

Bài Tập Thực Hành: Phân Tích Một Hệ Thống Thật

Hãy phân tích luồng đăng bài của Instagram.

Hành động của người dùng: Đăng một bức ảnh

Luồng hệ thống:

1. Ứng dụng di động → Tải ảnh lên → CDN
2. CDN → Trả về URL
3. Ứng dụng → API: Tạo bài đăng (URL, chú thích)
4. API → Lưu vào cơ sở dữ liệu (bảng posts)
5. API → Đưa vào hàng đợi: "fanout_post"
6. API → Trả về thành công cho người dùng
7. Worker nên → Lấy danh sách người theo dõi
8. Worker → Ghi bài đăng vào bảng tin của từng người theo dõi (bộ nhớ đệm)
9. Người theo dõi → Mở ứng dụng → Thấy bài đăng mới

Phân tích:

Các thành phần:

- Ứng dụng di động (máy khách)
- CDN (lưu trữ ảnh)
- Máy chủ API (phi trạng thái)
- Cơ sở dữ liệu (lưu trữ bền vững)
- Hàng đợi thông điệp (xử lý bất đồng bộ)

- Các tiến trình xử lý nền (*background workers*)
- Redis (bộ nhớ đệm cho bảng tin (*feed*))

Giao tiếp:

- Bước 1-6: Đồng bộ (người dùng cần xác nhận ngay)
- Bước 7-9: Bất đồng bộ (người theo dõi không cần cập nhật tức thì)

Điểm nghẽn:

- Nếu là người nổi tiếng với 10 triệu người theo dõi → Phải phân phát (*fanout*) tới 10 triệu bảng tin
- Giải pháp: Không phân phát cho người nổi tiếng, tải theo yêu cầu khi cần

Các đánh đổi:

- Người dùng bình thường: Đọc nhanh (bảng tin tạo sẵn), ghi chậm
- Người nổi tiếng: Ghi nhanh (không phân phát), đọc chậm hơn (truy vấn khi cần)

Mô Hình Tư Duy (*Mental Model*): Mọi Thứ Đều Là Một Hệ Thống

Áp dụng tư duy thành phần vào mọi thứ.

Quán cà phê:

- Thành phần: Thu ngân, Pha chế, Thiết bị, Kho nguyên liệu
- Giao tiếp: Phiếu gọi món (bất đồng bộ!)
- Điểm nghẽn: 1 người pha chế vs 10 khách hàng → Hàng chờ dài ra

Giải pháp (theo kiểu thiết kế hệ thống):

- Thêm người pha chế (mở rộng theo chiều ngang (*horizontal scaling*))
- Máy pha espresso tốt hơn (mở rộng theo chiều dọc (*vertical scaling*))
- Ki-ốt tự phục vụ (giảm tải cho thu ngân)

Nhà kho:

- Thành phần: Nhận hàng, Lưu kho, Lấy hàng, Giao hàng

- Giao tiếp: Hệ thống đơn hàng
- Điểm nghẽn: Lấy hàng thủ công → Chậm

Giải pháp:

- Robot lấy hàng (tự động hoá)
- Hệ thống kho tốt hơn (tối ưu lộ trình)

Khi bạn nhìn thế giới qua lăng kính hệ thống, bạn bắt đầu thấy những mẫu giống nhau ở khắp nơi.

Những Điểm Cốt Lõi

Mọi hệ thống đều có 3 yếu tố:

1. **Thành phần** - Những khối xây dựng thực hiện công việc
2. **Giao tiếp** - Cách các thành phần tương tác
3. **Ràng buộc** - Giới hạn về tài nguyên, độ trễ, thông lượng (*throughput*)

Để hiểu một hệ thống:

1. Xác định các thành phần
2. Vẽ luồng dữ liệu
3. Đo từng bước
4. Tìm điểm nghẽn
5. Tối ưu điểm nghẽn (không phải mọi thứ)

Hướng dẫn chọn đồng bộ hay bất đồng bộ:

- Đồng bộ: Người dùng cần kết quả ngay, quy trình đơn giản
- Bất đồng bộ: Xử lý lâu, lưu lượng tăng đột biến, cần tách rời các thành phần

Hướng dẫn dùng bộ nhớ đệm:

- Nên dùng: Đọc nhiều, truy vấn tốn kém, dữ liệu tĩnh
- Không nên dùng: Ghi nhiều, dữ liệu cá nhân hoá, thời gian thực

Tư duy thành phần không chỉ dành cho phần mềm. Nó là mô hình tư duy để phân tích bất kỳ hệ thống phức tạp nào.

Từ bây giờ, mỗi khi bạn nhìn một ứng dụng, hãy tự hỏi:

"Có những thành phần nào? Chúng giao tiếp thế nào? Điểm nghẽn ở đâu?"

Hãy luyện tập mô hình tư duy này. Nó sẽ thay đổi cách bạn nhìn nhận các hệ thống.

Tư Duy Theo Ràng Buộc (*Thinking in Constraints*): Kỹ Thuật Là Quản Lý Giới Hạn

Tôi còn nhớ lần đầu tiên thiết kế một hệ thống "cho 1 triệu người dùng".

Tôi vẽ một sơ đồ kiến trúc thật đẹp: vi dịch vụ (*microservices*), Kafka, Kubernetes, bộ nhớ đệm phân tán (*distributed cache*), tự động mở rộng (*auto-scaling*)...

Một kiến trúc sư lâu năm (*senior architect*) nhìn vào, hỏi một câu đơn giản: "Em có ngân sách bao nhiêu?"

Tôi: "Ngân sách ạ? Em chưa tính..."

Anh ấy: "Kiến trúc này tốn \$15.000/tháng. Ngân sách của em là \$2.000/tháng. Em nghĩ sao?"

Đó là lúc tôi học được bài học quan trọng nhất: Thiết kế không phải là vẽ sơ đồ đẹp. Thiết kế là làm việc với các ràng buộc (*constraints*).

Tại Sao Ràng Buộc Là Điểm Bắt Đầu?

Đối Chiếu Với Thực Tế

Lý thuyết: "Thiết kế hệ thống chịu được 1 triệu người dùng!"

Thực tế:

- Ngân sách: \$2.000/tháng (không phải vô hạn)
- Nhóm: 2 lập trình viên (không phải 50)
- Thời hạn: 2 tháng (không phải 2 năm)
- Hạ tầng sẵn có: PostgreSQL đã được cài đặt
- Chuyên môn: Nhóm chỉ biết Python và SQL

→ Kiến trúc hoàn hảo không tồn tại

→ Chỉ có kiến trúc phù hợp với các ràng buộc

Điểm mấu chốt: Ràng buộc không phải là kẻ thù. Ràng buộc là **các tham số thiết kế**.

Khung Tư Duy (Framework): Hệ Thống = Ràng Buộc + Đánh Đổi (Trade-offs)



Quy trình thiết kế hệ thống: Ràng buộc định hình quyết định, đánh đổi quyết định giải pháp

Hiểu ràng buộc → **Ra quyết định đúng** → **Tránh thiết kế thừa** (over-engineering) hoặc **thiết kế thiếu** (under-engineering)

Ràng buộc 1: Giới Hạn Tính Toán

Thực tế: CPU không vô hạn.

Những Con Số Bạn Phải Biết

Năng lực của một lõi CPU:

- Máy chủ web thông thường: ~1.000 yêu cầu/giây
- Truy vấn cơ sở dữ liệu: ~10.000 truy vấn đơn giản/giây
- Tuân tự hoá JSON: ~50.000 đối tượng/giây
- Tính toán hàm băm: ~1.000.000 lần băm/giây

Các con số này có thể dao động, nhưng bậc độ lớn mới là điều quan trọng!

Kịch bản ví dụ:

Yêu cầu: Xử lý 10.000 yêu cầu/giây

Phương án 1: Một máy chủ cực mạnh (32 lõi)

- Năng lực lý thuyết: 32.000 req/s
- Chi phí: \$800/tháng
- Rủi ro: Điểm lõi duy nhất

Phương án 2: 10 máy chủ tầm trung (mỗi máy 4 lõi)

- Năng lực: $10 \times 4.000 = 40.000$ req/s
- Chi phí: $10 \times \$200 = \2.000 /tháng
- Lợi ích: Có dự phòng, dễ mở rộng hơn

Phương án 3: Bắt đầu với 2 máy chủ, mở rộng sau

- Năng lực hiện tại: $2 \times 4.000 = 8.000$ req/s
- Chi phí: \$400/tháng
- Đối chiếu thực tế: Bạn có thực sự có 10 nghìn req/s không?

Chọn phương án nào?

→ Tùy vào các ràng buộc khác (ngân sách, tốc độ tăng trưởng, mức chịu rủi ro)

Ví Dụ Về Ràng Buộc Tính Toán

Trường hợp 1: Xử lý ảnh

Nhiệm vụ: Thay đổi kích thước 1 triệu ảnh

Một máy chủ: 100 ảnh/giây

Thời gian: $1 \text{ triệu} / 100 = 10.000$ giây = 2,8 giờ

Ràng buộc: Phải xong trong 30 phút

Tính toán: $1 \text{ triệu} / 1.800 \text{ giây} =$ cần 556 ảnh/giây

Giải pháp: 6 máy chủ ($6 \times 100 = 600$ ảnh/giây)

Phương án khác: Tối ưu thuật toán

- Xử lý song song: 400 ảnh/giây trên cùng phân cứng
- Chỉ cần 2 máy chủ thay vì 6
- Đánh đổi: Thời gian phát triển và chi phí hạ tầng

Trường hợp 2: Phân tích dữ liệu thời gian thực

Yêu cầu: Xử lý 100 nghìn sự kiện/giây

Đơn luồng: Tối đa 10 nghìn sự kiện/giây

Ràng buộc: Không chịu nổi tải

Các phương án:

1. Mở rộng theo chiều dọc (máy lớn hơn)
 - Máy chủ 16 lõi → 160 nghìn sự kiện/giây
 - Chi phí: \$1.200/tháng
2. Mở rộng theo chiều ngang (thêm máy)
 - 10 máy chủ 2 lõi → 200 nghìn sự kiện/giây
 - Chi phí: $10 \times \$100 = \1.000 /tháng
3. Tối ưu mã + dùng hàng đợi làm vùng đệm
 - Tối ưu lên 50 nghìn sự kiện/giây mỗi lõi
 - Máy chủ 2 lõi xử lý được 100 nghìn
 - Chi phí: \$200/tháng
 - Đánh đổi: Thời gian phát triển

Quyết định dựa trên ràng buộc:

- Ngân sách eo hẹp → Phương án 3
- Thời gian gấp → Phương án 1 hoặc 2
- Dài hạn → Phương án 3 rồi mở rộng theo Phương án 2

Ràng buộc 2: Giới Hạn Bộ Nhớ

Thực tế: RAM không miễn phí và có giới hạn vật lý.

Tính Toán Bộ Nhớ

Yêu cầu thông thường:

Mỗi phiên người dùng: ~1KB

1 triệu người dùng: 1GB RAM

Mỗi đối tượng lưu đệm: ~10KB

100 nghìn đối tượng lưu đệm: 1GB RAM

Mỗi kết nối cơ sở dữ liệu: ~10MB

100 kết nối: 1GB RAM

Một máy chủ 16GB:

- Hệ điều hành chiếm: ~2GB
 - Ứng dụng: ~2GB
 - Kết nối cơ sở dữ liệu (100): ~1GB
 - Bộ nhớ đệm: còn ~10GB
 - Phiên người dùng (10 triệu): Cần 10GB
- Vừa khít, hoặc phải nén/loại bỏ bớt

Ví dụ: Chiến lược lưu đệm (Caching strategy)

Thương mại điện tử? 1 triệu sản phẩm

Lưu đệm toàn bộ:

- Dữ liệu sản phẩm: mỗi cái 1KB
- Tổng: 1GB RAM
- Chi phí: Vừa vặn trong một instance Redis (\$50/tháng)
- Lợi ích: Mọi sản phẩm đều được lưu đệm

Lưu đệm một phần (sản phẩm "nóng"):

- 10% sản phẩm = 90% lưu lượng
- Bộ nhớ đệm: 100 nghìn sản phẩm = 100MB
- Chi phí: Nằm sẵn trong RAM của máy chủ ứng dụng
- Lợi ích: Miễn phí, đủ tốt

Chọn phương án nào?

- Kiểm tra ràng buộc: Ngân sách? Bộ nhớ còn trống?
- Nếu ngân sách ỏn, lưu đệm toàn bộ = logic đơn giản hơn
- Nếu ngân sách eo hẹp, lưu đệm một phần = đủ dùng

Các Quyết Định Về Ràng Buộc Bộ Nhớ

Trường hợp: Bộ nhớ đệm dòng thời gian của người dùng

Ứng dụng mạng xã hội: 10 triệu người dùng

Phương án 1: Lưu đệm toàn bộ dòng thời gian

- 10 triệu người dùng $\times$ 100 bài $\times$ 1KB = 1TB RAM
- Chi phí: \$10.000/tháng (cụm Redis)
- Lợi ích: Dòng thời gian tải tức thì

Phương án 2: Chỉ lưu đệm người dùng đang hoạt động

- 1 triệu người dùng hoạt động $\times$ 100 bài $\times$ 1KB = 100GB RAM
- Chi phí: \$1.000/tháng
- Lợi ích: Tỷ lệ trùng bộ nhớ đệm 90% (đủ tốt)

Phương án 3: Lưu đệm theo nhu cầu

- Bộ nhớ đệm LRU, giới hạn 10GB
- Chi phí: \$100/tháng
- Lợi ích: Bao phủ dữ liệu "nóng" nhất
- Đánh đổi: Có một số lần trượt bộ nhớ đệm

Kiểm tra ràng buộc:

- 10 triệu người dùng, nhưng chỉ 1 triệu hoạt động hằng ngày
- Ngân sách: tổng \$2.000/tháng
- Phương án 2 hoặc 3 là hợp lý
- Phương án 1 là thiết kế thừa

Ràng buộc 3: Giới Hạn Mạng

Thực tế: Mạng có giới hạn về độ trễ (latency) và băng thông (bandwidth).

Các Định Luật Vật Lý

Tốc độ ánh sáng: 300.000 km/s

Nhưng trong cáp quang: 200.000 km/s

New York ↔ San Francisco: 4.000 km

Độ trễ tối thiểu theo lý thuyết: $4.000 / 200.000 = 20\text{ms}$

Thực tế: 40-70ms (chi phí định tuyến)

→ Không thể thắng được vật lý

→ Đây là ràng buộc nên tăng

Ngân sách độ trễ:

Mức chịu đựng của người dùng: 100-200ms cho một lần tải trang web

Phân bổ ngân sách:

- Tra cứu DNS: 20ms
- Bắt tay TCP: 20ms
- Bắt tay TLS: 40ms
- Yêu cầu/Phản hồi: 20ms
- Máy chủ xử lý: ???

Đã dùng: 100ms

Còn lại cho máy chủ: 0-100ms

Ràng buộc: Máy chủ phải phản hồi trong tối đa 100ms

Ví Dụ Về Ràng Buộc Mạng

Trường hợp 1: Người dùng toàn cầu

Ứng dụng có người dùng khắp thế giới

Không có CDN:

- Người dùng Mỹ → Máy chủ Singapore: độ trễ ~200ms
- Tài nguyên tĩnh (ảnh, JS, CSS): 5MB
- Thời gian tải: $200\text{ms} + (5\text{MB} / 10\text{Mbps}) = 200\text{ms} + 4\text{s} = 4,2\text{s}$
- Trải nghiệm người dùng: TỆ HẠI

Có CDN:

- Người dùng Mỹ → Điểm biên CDN ở Mỹ: độ trễ ~20ms
- Tài nguyên tĩnh được lưu đệm tại điểm biên
- Thời gian tải: $20\text{ms} + (5\text{MB} / 50\text{Mbps}) = 20\text{ms} + 0,8\text{s} = 0,82\text{s}$
- Trải nghiệm người dùng: TỐT

Đánh đổi chi phí:

- Không có CDN: \$0
- Có CDN: \$50/tháng
- Quyết định: CDN đáng tiền (trải nghiệm người dùng là then chốt)

Trường hợp 2: Vị trí cơ sở dữ liệu

Máy chủ ứng dụng ở Mỹ, cơ sở dữ liệu ở châu Âu

Mỗi truy vấn:

- Mỹ → châu Âu: độ trễ ~100ms
- Thực thi truy vấn: 10ms
- Châu Âu → Mỹ: độ trễ ~100ms
- Tổng: 210ms mỗi truy vấn

Trang cần 10 truy vấn:

- Tổng: $2.100\text{ms} = 2,1\text{ giây}$
- Không thể chấp nhận!

Giải pháp:

1. Chuyển cơ sở dữ liệu về Mỹ

- Độ trễ: 10ms mỗi truy vấn
- Tổng: 100ms cho 10 truy vấn
- Đánh đổi: Vương vấn đề chủ quyền dữ liệu?

2. Nhân bản cơ sở dữ liệu (bản sao chỉ đọc đặt ở Mỹ)

- Đọc: 10ms
- Ghi: Vẫn mất 100ms tới máy chủ chính
- Đánh đổi: Nhất quán cuối cùng

3. Lớp bộ nhớ đệm

- Trúng bộ nhớ đệm: 1ms
- Trượt bộ nhớ đệm: 210ms
- Đánh đổi: Có thể gặp dữ liệu cũ

Quyết định dựa trên ràng buộc:

- Yêu cầu độ trễ < 200ms → Cần giải pháp
- Dữ liệu phải ở lại châu Âu → Phương án 2 hoặc 3
- Cần nhất quán mạnh → Phương án 2

Ràng buộc 4: Giới Hạn Cơ Sở Dữ Liệu

Thực tế: Cơ sở dữ liệu có giới hạn về thông lượng (throughput) và dung lượng lưu trữ.

Năng Lực Của Cơ Sở Dữ Liệu

Giới hạn thông thường của PostgreSQL:

Năng lực ghi:

- Một máy chủ: ~10.000 lượt ghi/giây
- Với SSD: ~50.000 lượt ghi/giây
- Ghi theo lô: ~100.000 lượt ghi/giây

Năng lực đọc:

- Truy vấn đơn giản: ~100.000 lượt đọc/giây
- Truy vấn phức tạp: ~1.000 lượt đọc/giây
- Có chỉ mục: cải thiện gấp 10 lần

Lưu trữ:

- Giới hạn thực tế: ~10TB mỗi máy chủ
- Vượt mức đó: Cân nhắc phân mảnh dữ liệu (sharding)

Kết nối:

- Số kết nối tối đa: ~500
- Thực tế: ~200 kết nối hoạt động

Ví dụ: Ứng dụng ghi nhiều

Hệ thống ghi log: 50.000 lượt ghi/giây

Một PostgreSQL duy nhất:

- Tối đa: 10.000 lượt ghi/giây
- Ràng buộc: Không chịu nổi tải

Các phương án:

1. Ghi theo lô

- Gom 100 lượt ghi, chèn cùng lúc
- Hiệu quả: 100.000 lượt ghi/giây
- Đánh đổi: Trễ một chút (chấp nhận được với log)

2. Dùng CSDL tối ưu cho ghi (Cassandra)

- Năng lực: 1.000.000 lượt ghi/giây
- Đánh đổi: Mô hình truy vấn khác, nhóm phải học thêm

3. Hàng đợi + worker chạy nền

- Nhận lượt ghi vào hàng đợi (tức thì)
- Worker ghi vào CSDL (tốc độ có kiểm soát)
- Đánh đổi: Dữ liệu được lưu bên vững sau một khoảng trễ

Kiểm tra ràng buộc:

- Nhóm thành thạo PostgreSQL
- Log có thể trễ ~1-2 giây
- Phương án 1 (ghi theo lô) là phù hợp nhất
- Đơn giản, tận dụng kiến thức sẵn có, đáp ứng yêu cầu

Ràng buộc 5: Giới Hạn Con Người (Quy Mô Nhóm)

Ràng buộc hay bị bỏ qua nhất: Năng lực của chính nhóm bạn.

Thực Tế Về Quy Mô Nhóm

Quy mô nhóm: 2 lập trình viên

Có thể duy trì:

- Hệ thống nguyên khối: ☒ Dễ dàng
- 3-5 vi dịch vụ: ☐ Khó khăn
- Hơn 10 vi dịch vụ: ☒ Không thể
- Kubernetes: ☒ Quá phức tạp
- Triển khai đơn giản: ☒ Thiết yếu

Tại sao?

- Môi dịch vụ cần: triển khai, giám sát, gỡ lỗi
- 10 dịch vụ = gấp 10 lần gánh nặng vận hành
- 2 lập trình viên không thể quản lý tốt 10 dịch vụ

Khung Tư Duy Về Ràng Buộc Nhóm

Nhóm 1-3 người:

- ✓ Hệ thống nguyên khối
- ✓ Bộ công nghệ đơn giản (những gì nhóm đã biết)
- ✓ Dịch vụ được quản lý sẵn (giảm việc vận hành)
- x Vi dịch vụ
- x Hạ tầng phức tạp
- x Công nghệ mới

Nhóm 5-10 người:

- ✓ Hệ thống nguyên khối chia mô-đun
- ✓ Vi dịch vụ đơn giản (2-3 dịch vụ)
- ✓ Kết hợp công nghệ mới + quen thuộc
- ☐ Vi dịch vụ toàn phần (rủi ro)

Nhóm từ 20 người trở lên:

- ✓ Vi dịch vụ
- ✓ Nhiều bộ công nghệ
- ✓ Có nhóm vận hành riêng
- ✓ Hạ tầng phức tạp

Ví dụ thực tế:

Công ty khởi nghiệp: 3 lập trình viên

Phương án A: Kiến trúc vi dịch vụ

- 8 dịch vụ
- Triển khai trên Kubernetes
- Service mesh
- Truy vết phân tán

Thời gian phát triển: 6 tháng

Bảo trì: 2 lập trình viên toàn thời gian chỉ để vận hành

Phát triển tính năng: CHẬM

Phương án B: Hệ thống nguyên khối có cấu trúc tốt

- Một kho mã nguồn, các mô-đun rõ ràng
- Triển khai trên Heroku
- Giám sát đơn giản

Thời gian phát triển: 6 tuần

Bảo trì: 0,5 lập trình viên cho vận hành

Phát triển tính năng: NHANH

Thực tế ràng buộc:

- 3 lập trình viên không thể duy trì 8 dịch vụ
 - Tốc độ ra thị trường là sống còn với công ty khởi nghiệp
- Phương án B là lựa chọn khả thi duy nhất

Danh Sách Kiểm Tra Ràng Buộc

Trước khi thiết kế bất kỳ hệ thống nào, hãy hỏi:

Ràng buộc kỹ thuật:

- ☐ Cần năng lực CPU bao nhiêu? (yêu cầu/giây)
- ☐ Bộ nhớ còn bao nhiêu? (kích thước bộ nhớ đệm, lưu phiên)
- ☐ Độ trễ mạng có chấp nhận được không? (vị trí người dùng, vị trí máy chủ)
- ☐ Thông lượng cơ sở dữ liệu có đủ không? (lượt ghi/đọc mỗi giây)
- ☐ Dung lượng lưu trữ? (kích thước dữ liệu hiện tại và 2 năm sau)

Ràng buộc tài nguyên:

- ☐ Ngân sách? (\$X/tháng cho hạ tầng)
- ☐ Thời hạn? (X tuần đề ra mắt)
- ☐ Cái giá của sự cố? (có chịu được ngừng hoạt động không?)

Ràng buộc con người:

- ☐ Quy mô nhóm? (X lập trình viên)
- ☐ Chuyên môn của nhóm? (biết Y, cần học Z?)
- ☐ Năng lực vận hành? (ai duy trì hệ thống?)
- ☐ Gánh nặng trực sự cố? (có giám sát 24/7 được không?)

Ràng buộc tăng trưởng:

- ☐ Quy mô hiện tại? (X người dùng, Y yêu cầu/ngày)
- ☐ Tăng trưởng dự kiến? (gấp 2 lần/năm hay 10 lần/năm?)
- ☐ Tải đỉnh so với trung bình? (mẫu hình lưu lượng?)

Quy Trình Thiết Kế Dựa Trên Ràng Buộc

Khung tư duy từng bước:

1. Xác định TẤT CẢ ràng buộc
 - Giới hạn kỹ thuật
 - Giới hạn tài nguyên (ngân sách, thời gian)
 - Giới hạn con người (nhóm, chuyên môn)
2. Xếp hạng ưu tiên các ràng buộc
 - Đầu là giới hạn cứng? (không được vi phạm)
 - Đầu là giới hạn mềm? (có thể thương lượng)
3. Thiết kế trong phạm vi ràng buộc
 - Bắt đầu đơn giản
 - Chỉ thêm độ phức tạp khi ràng buộc đòi hỏi
4. Kiểm chứng với các ràng buộc
 - Giải pháp có vừa ngân sách không?
 - Nhóm có duy trì được không?
 - Có đáp ứng yêu cầu hiệu năng không?
5. Ghi lại các quyết định về ràng buộc
 - Tại sao chọn X thay vì Y?
 - Ràng buộc nào đã ảnh hưởng đến quyết định?

Ví dụ áp dụng:

Yêu cầu: Xây dựng bảng điều khiển phân tích dữ liệu

Các ràng buộc đã xác định:

- Ngân sách: \$500/tháng (giới hạn cứng)
- Nhóm: 1 lập trình viên (giới hạn cứng)
- Dữ liệu: 100GB (hiện tại), tăng 10GB/tháng
- Truy vấn: Tổng hợp phức tạp
- Người dùng: 50 người dùng nội bộ
- Thời hạn: 2 tuần để có MVP

Các quyết định dựa trên ràng buộc:

Chọn cơ sở dữ liệu:

- PostgreSQL ✓
 - Xử lý 100GB dễ dàng
 - Nhóm biết SQL
 - Truy vấn phức tạp: hỗ trợ sẵn
 - Chi phí: \$50/tháng cho bản được quản lý sẵn
- Cassandra ✗
 - Quá mức cần thiết cho 100GB
 - Nhóm không biết
 - Tổng hợp phức tạp: khó
 - Đắt hơn

Triển khai:

- Heroku ✓
 - Đơn giản, được quản lý sẵn
 - \$200/tháng (trong ngân sách)
 - 1 lập trình viên quản lý được
- Kubernetes ✗
 - Thiết lập phức tạp
 - Cần người vận hành chuyên trách
 - Vượt ngân sách nếu dùng bản quản lý sẵn

Bộ nhớ đệm:

- Bỏ qua cho MVP ✓
 - 50 người dùng = lưu lượng thấp
 - Có thể thêm sau nếu chậm
 - Tiết kiệm thời gian phát triển

Kết quả:

- Tổng chi phí: \$250/tháng (dưới ngân sách)
- Phát triển: 2 tuần (kịp thời hạn)
- Bảo trì: Tối thiểu (nhóm quản lý được)
- Hiệu năng: Đủ tốt cho quy mô hiện tại

Sai Lầm Phổ Biến: Phốt Lờ Ràng Buộc

Sai lầm 1: Thiết kế cho quy mô tưởng tượng

✗ Tệ:

"Biết đâu một ngày nào đó ta sẽ có 10 triệu người dùng, vậy dùng Kubernetes ngay bây giờ đi."

Thực tế:

- Hiện tại: 100 người dùng
- Thời hạn: 6 tháng để chứng minh sản phẩm phù hợp thị trường
- Ngân sách: \$2.000/tháng

✓ Tốt:

"Bắt đầu với Heroku. Nếu đạt 100 nghìn người dùng (thành công!), ta sẽ có doanh thu để chi trả việc chuyển sang Kubernetes. Đừng tối ưu cho những vấn đề ta chưa có."

Sai lầm 2: Phốt lờ ràng buộc về nhóm

✗ Tệ:

"Dùng Rust để đạt hiệu năng tối đa đi!"

Thực tế:

- Nhóm biết Python
- Yêu cầu hiệu năng: 1.000 req/s
- Python xử lý 1.000 req/s dễ dàng

✓ Tốt:

"Dùng Python. Nhóm làm việc hiệu quả ngay lập tức. Rust sẽ tốn 3 tháng để học. Ràng buộc hiệu năng không bị vi phạm."

Sai lầm 3: Mặc định ngân sách là vô hạn

✗ Tệ:

"Ta sẽ dùng Kubernetes quản lý sẵn, CSDL quản lý sẵn, bộ nhớ đệm quản lý sẵn, mọi thứ đều quản lý sẵn!"

Chi phí: \$8.000/tháng

Thực tế:

- Ngân sách: \$2.000/tháng
- Vi phạm ràng buộc cứng

✓ Tốt:

"Dùng Heroku (\$200), PostgreSQL quản lý sẵn (\$50), và bộ nhớ đệm ở tầng ứng dụng (miễn phí).

Tổng: \$250/tháng. Vừa ngân sách."

Những Điểm Cốt Lõi

Kỹ thuật = Quản lý ràng buộc

Không phải: "Kiến trúc nào là tốt nhất?"

Mà là: "Kiến trúc nào phù hợp với các ràng buộc của tôi?"

5 nhóm ràng buộc:

1. Tính toán: Giới hạn năng lực CPU
2. Bộ nhớ: Giới hạn RAM sẵn có
3. Mạng: Giới hạn độ trễ và băng thông
4. Cơ sở dữ liệu: Giới hạn thông lượng và lưu trữ
5. Con người: Giới hạn quy mô và chuyên môn của nhóm

Khung tư duy:

Thiết kế hệ thống = Ràng buộc + Đánh đổi

Ràng buộc → Xác định điều gì khả thi

Đánh đổi → Chọn giữa các khả năng

Vừa vặn → Giải pháp tôn trọng các ràng buộc

Quy trình thiết kế:

1. Xác định ràng buộc (tất cả!)
2. Xếp ưu tiên (giới hạn cứng hay mềm)
3. Thiết kế trong phạm vi ràng buộc
4. Kiểm chứng giải pháp có phù hợp
5. Ghi lại các quyết định

Hãy nhớ:

Kiến trúc hoàn hảo nhưng phớt lờ ràng buộc = Vô dụng
Kiến trúc đủ tốt và tôn trọng ràng buộc = Có thể ra mắt

Ràng buộc không phải kẻ thù
Ràng buộc là tham số thiết kế

Làm việc CÙNG ràng buộc, đừng CHÔNG LẠI chúng

Sự thay đổi tư duy cần có:

Từ: "Làm sao xây hệ thống hoàn hảo?"

Sang: "Làm sao xây đúng hệ thống với các ràng buộc của tôi?"

Từ: "Thực hành tốt nhất bảo X"

Sang: "Ràng buộc của tôi đòi hỏi Y"

Từ: "Chuẩn bị sẵn cho mọi thứ trong tương lai"

Sang: "Giải quyết ràng buộc hôm nay, thích ứng ngày mai"

Bạn đã học tư duy ưu tiên vấn đề (Bài 1) và tư duy đánh đổi (*trade-off thinking*) (Bài 2).

Giờ với tư duy ràng buộc (bài này), bạn đã có một mô hình tư duy (*mental model*) hoàn chỉnh:

Vấn đề → Ràng buộc → Đánh đổi → Giải pháp

Đây là cách các kiến trúc sư lâu năm thực sự suy nghĩ.

Trực Giác Về Quy Mô (Scale Intuition): Xây Dựng Cảm Giác Về Các Con Số

Tôi còn nhớ lần đầu tiên được hỏi trong một buổi phỏng vấn (interview):

"Hệ thống của em cần xử lý bao nhiêu yêu cầu (requests) mỗi giây?"

Tôi: "Ừm... nhiều ạ?"

Người phỏng vấn (Interviewer): "Nhiều là bao nhiêu? 10? 1.000? 1.000.000?"

Tôi: "... Em không biết ạ."

Tôi trượt câu hỏi đó.

Không phải vì không biết viết mã. Mà vì **không có trực giác về quy mô**.

Một kiến trúc sư lâu năm (senior architect) ngồi bên cạnh sau đó nói: "Em biết không, khác biệt giữa 1 nghìn người dùng và 1 triệu người dùng không phải là 1000 lần. Nó là khác biệt giữa một chiếc xe đạp và một chiếc Boeing 747."

Đó là lúc tôi bắt đầu học về quy mô (scale).

Tại Sao Trực Giác Về Quy Mô Lại Quan Trọng?

Vấn Đề: Các Con Số Quá Trừu Tượng

Khi bạn nghe "1 triệu người dùng", bạn nghĩ gì?

Hầu hết lập trình viên (developers): "Ồ, nhiều quá!"

Nhưng "nhiều" không giúp bạn thiết kế hệ thống (design system).

Bạn cần biết:

- 1 triệu người dùng = bao nhiêu yêu cầu/giây?
- Cần bao nhiêu máy chủ (servers)?
- Cần bao nhiêu dung lượng cơ sở dữ liệu (database capacity)?
- Cần bao nhiêu lưu trữ (storage)?

- Tổng bao nhiêu chi phí (cost)?

Giải Pháp: Rèn Luyện Trực Giác

Trực giác về quy mô = Khả năng ước lượng (estimate) nhanh các yêu cầu của hệ thống

Kỹ năng: "Ước chừng 1 triệu người dùng cần 10 máy chủ ứng dụng, 2TB lưu trữ, chi phí ~\$3K/tháng"

Chứ không phải: "1 triệu người dùng... cần nhiều máy chủ... ừm..."

Có trực giác → Thiết kế một cách tự tin

Không có trực giác → Thiết kế bằng phỏng đoán

Bài này sẽ dạy bạn xây dựng trực giác đó.

Phổ Quy Mô: 1 Nghìn vs 1 Triệu vs 1 Tỷ Người Dùng

Hình Dung Sự Khác Biệt



Mỗi bậc quy mô là một thế giới hoàn toàn khác nhau

1.000 Người Dùng (MVP Của Công Ty Khởi Nghiệp)

Người dùng hoạt động hằng ngày (DAU): ~500

Người dùng đồng thời: ~50

Số yêu cầu mỗi giây: ~5-10

Hạ tầng:

- 1 máy chủ ứng dụng (\$50/tháng)
- 1 máy chủ cơ sở dữ liệu (\$50/tháng)
- Không cần bộ cân bằng tải
- Không cần bộ nhớ đệm

Tổng chi phí: ~\$100/tháng

Điểm nghề: Có lẽ là không có

Kiến trúc: Hệ thống nguyên khối đơn giản

Nhóm: 1-2 lập trình viên là đủ

Ví dụ thực tế:

Ứng dụng cho một quán cà phê địa phương:

- 500 khách hàng
- Mỗi người xem thực đơn 2 lần/ngày
- 1.000 yêu cầu/ngày
- $1.000 / 86.400 \text{ giây} \approx 0,01 \text{ yêu cầu/giây}$

Chạy được trên Raspberry Pi! 😊

10.000 Người Dùng (Công Ty Khởi Nghiệp Đang Tăng Trưởng)

Người dùng hoạt động hằng ngày: ~5.000

Người dùng đồng thời: ~500

Số yêu cầu mỗi giây: ~50-100

Hạ tầng:

- 2-3 máy chủ ứng dụng (\$150/tháng)
- 1 máy chủ cơ sở dữ liệu (\$100/tháng)
- Bộ cân bằng tải (\$50/tháng)
- Bộ nhớ đệm Redis (tuỳ chọn) (\$50/tháng)

Tổng chi phí: ~\$350/tháng

Điểm nghẽn: Truy vấn cơ sở dữ liệu (thêm chỉ mục)

Kiến trúc: Hệ thống nguyên khối + bộ nhớ đệm

Nhóm: 2-5 lập trình viên

Khác biệt chính so với mốc 1 nghìn:

- Cần bộ cân bằng tải (*load balancer*) (tính sẵn sàng cao (*high availability*))
- Chỉ mục (*Indexes*) cơ sở dữ liệu (*database*) trở nên quan trọng
- Giám sát (*Monitoring*) bắt đầu trở nên quan trọng

100.000 Người Dùng (Sản Phẩm Đã Có Chỗ Đứng)

Người dùng hoạt động hằng ngày: ~50.000

Người dùng đồng thời: ~5.000

Số yêu cầu mỗi giây: ~500-1.000

Hạ tầng:

- 10 máy chủ ứng dụng (\$500/tháng)
- Cơ sở dữ liệu với bản sao chỉ đọc (\$500/tháng)
- Bộ cân bằng tải (\$100/tháng)
- Cụm Redis (\$300/tháng)
- CDN (\$100/tháng)

Tổng chi phí: ~\$1.500/tháng

Điểm nghẽn: Ghi vào cơ sở dữ liệu, vô hiệu hoá bộ nhớ đệm

Kiến trúc: Hệ thống nguyên khối dạng mô-đun hoặc vi dịch vụ đơn giản

Nhóm: 10-20 lập trình viên

Khác biệt chính so với mốc 10 nghìn:

- Cần có bản sao chỉ đọc (*read replicas*) cho cơ sở dữ liệu
- Lưu đệm (*Caching*) trở nên sống còn
- CDN cho tài nguyên tĩnh (*static assets*)
- Tự động mở rộng (*Auto-scaling*) bắt đầu có ý nghĩa

1.000.000 Người Dùng (Quy Mô Lớn)

Người dùng hoạt động hằng ngày: ~500.000

Người dùng đồng thời: ~50.000

Số yêu cầu mỗi giây: ~5.000-10.000

Hạ tầng:

- 50+ máy chủ ứng dụng (\$2.500/tháng)
- Cụm cơ sở dữ liệu đã phân mảnh (\$2.000/tháng)
- Nhiều bộ cân bằng tải (\$300/tháng)
- Bộ nhớ đệm phân tán (\$1.000/tháng)
- CDN (\$500/tháng)
- Hàng đợi thông điệp (\$200/tháng)

Tổng chi phí: ~\$6.500/tháng trở lên

Điểm nghẽn: Mọi thứ! Cần cách tiếp cận phân tán

Kiến trúc: Nhiều khả năng là vi dịch vụ

Nhóm: 50+ lập trình viên, có nhóm vận hành chuyên trách

Khác biệt chính so với mốc 100 nghìn:

- Cần phân mảnh dữ liệu (*sharding*) cho cơ sở dữ liệu

- Những thách thức của hệ thống phân tán (*distributed systems*)
- Có nhóm DevOps/SRE chuyên trách
- Giám sát và cảnh báo (*alerting*) phức tạp

Quy Luật (*The Pattern*)

Mỗi khi quy mô tăng 10 lần:

- Chi phí tăng ~3-5 lần (không tuyến tính!)
- Độ phức tạp tăng đáng kể
- Quy mô nhóm tăng
- Kiến trúc thay đổi về căn bản

1K → 10K: Thêm máy chủ, tối ưu cơ bản

10K → 100K: Thêm bộ nhớ đệm, bản sao chỉ đọc

100K → 1M: Phân mảnh dữ liệu, vi dịch vụ, hệ thống phân tán

1M → 10M: Phân bố theo địa lý, điện toán biên

Ước Tính Nhanh (*Back-of-Envelope Calculations*)

Kỹ năng cốt lõi: Ước lượng nhanh các yêu cầu của hệ thống

Khung Tính Toán (*Calculation Framework*)

1. Bắt đầu với số người dùng
2. Ước lượng thói quen sử dụng
3. Tính số yêu cầu
4. Ước lượng dung lượng lưu trữ
5. Tính băng thông

Bài tập 1: Trang Web Thương Mại Điện Tử

Dữ kiện:

- 1 triệu người dùng đã đăng ký
- 20% hoạt động hằng ngày (200 nghìn DAU)
- Mỗi người dùng xem 10 trang sản phẩm/ngày
- Mỗi người dùng mua hàng 1 lần/tháng

Hãy tính:

Số yêu cầu mỗi giây:

Lượt xem trang:

$200K \text{ người dùng} \times 10 \text{ trang/ngày} = 2M \text{ lượt xem trang/ngày}$

$2M / 86.400 \text{ giây/ngày} \approx 23 \text{ yêu cầu/giây}$

Lưu lượng cao điểm (giả sử gấp 3 lần trung bình):

$23 \times 3 = 69 \text{ yêu cầu/giây}$

Lời gọi API (giả sử 5 lời gọi API cho mỗi lượt xem trang):

$23 \times 5 = 115 \text{ yêu cầu/giây}$

Cao điểm: 345 yêu cầu/giây

→ Cần xử lý ~350 req/s vào giờ cao điểm

Lưu trữ:

Danh mục sản phẩm:

$100K \text{ sản phẩm} \times 10KB \text{ mỗi sản phẩm} = 1GB$

Ảnh sản phẩm:

$100K \text{ sản phẩm} \times 5 \text{ ảnh} \times 500KB = 250GB$

Dữ liệu người dùng:

$1M \text{ người dùng} \times 1KB = 1GB$

Đơn hàng (1 năm):

$1M \text{ người dùng} \times 12 \text{ lần mua/năm} \times 5KB = 60GB$

Tổng: ~312GB

→ Một cơ sở dữ liệu xử lý dễ dàng

→ Ảnh cần CDN/S3

Băng thông:

Lượt xem trang: 2M/ngày

Kích thước trang trung bình: 2MB (HTML + ảnh + JS)

Tổng băng thông: $2M \times 2MB = 4TB/ngày$

Với bộ nhớ đệm CDN (tỷ lệ trùng bộ nhớ đệm 90%):

$4TB \times 0,1 = 400GB/ngày$ từ máy chủ gốc

= 16GB/giờ từ các máy chủ

= Kiểm soát được nhờ CDN

Ước lượng hạ tầng:

Máy chủ ứng dụng:

- Cao điểm 350 req/s
- Mỗi máy chủ: ~100 req/s
- Cân: 4 máy chủ (có dư phòng)

Cơ sở dữ liệu:

- 312GB dữ liệu (một máy chủ là ổn)
- Thiên về đọc (thêm 2 bản sao chỉ đọc)

CDN: Thiết yếu (90% lưu lượng)

Tổng chi phí: ~\$800/tháng

Bài tập 2: Ứng Dụng Mạng Xã Hội

Dữ kiện:

- 10 triệu người dùng
- 1 triệu người dùng hoạt động hằng ngày
- Mỗi người dùng đăng bài 2 lần/ngày
- Mỗi người dùng xem 100 bài đăng/ngày

Hãy tính:

Lưu lượng ghi (Write traffic):

Số bài đăng được tạo:

$1M \text{ người dùng} \times 2 \text{ bài/ngày} = 2M \text{ bài/ngày}$

$2M / 86.400 \approx 23 \text{ lượt ghi/giây}$

Cao điểm: 69 lượt ghi/giây

→ Một cơ sở dữ liệu xử lý được

Lưu lượng đọc (Read traffic):

Số bài đăng được xem:

$1M \text{ người dùng} \times 100 \text{ bài/ngày} = 100M \text{ lượt xem/ngày}$

$100M / 86.400 \approx 1.157 \text{ lượt đọc/giây}$

Cao điểm: 3.471 lượt đọc/giây

→ Cân bộ nhớ đệm!

Tốc độ tăng lưu trữ:

Số bài đăng mỗi ngày: 2M

Kích thước bài đăng: 500 byte (văn bản + siêu dữ liệu)

Lưu trữ mỗi ngày: $2M \times 500 \text{ byte} = 1\text{GB/ngày}$

Ảnh (50% bài đăng có 1 ảnh):

$1M \text{ ảnh/ngày} \times 2\text{MB} = 2\text{TB/ngày}$

Tăng trưởng hằng năm:

Văn bản: $1\text{GB} \times 365 = 365\text{GB}$

Ảnh: $2\text{TB} \times 365 = 730\text{TB}$

→ Cần lưu trữ phân tán (S3)

→ Cơ sở dữ liệu: Phân mảnh sau 1-2 năm

Tỷ lệ Đọc:Ghi:

Đọc: 1.157/s

Ghi: 23/s

Tỷ lệ: 50:1

→ Cần tối ưu mạnh cho việc đọc

→ Chiến lược lưu đệm quyết liệt

→ CDN cho nội dung đa phương tiện

Trực Giác Về Số Yêu Cầu Mỗi Giây (RPS)

Các Mức RPS Khác Nhau Có Ý Nghĩa Gì

1-10 RPS:

- Công cụ nội bộ nhỏ
- Một máy chủ là đủ
- Không cần tối ưu gì đặc biệt

10-100 RPS:

- Ứng dụng web nhỏ
- 1-2 máy chủ
- Lưu đệm cơ bản là có ích

100-1.000 RPS:

- Ứng dụng web cỡ vừa
- 5-10 máy chủ
- Lưu đệm là thiết yếu
- Tối ưu cơ sở dữ liệu trở nên quan trọng

1.000-10.000 RPS:

- Ứng dụng web lớn
- 20-50 máy chủ
- Bộ nhớ đệm phân tán
- Bản sao chỉ đọc cho cơ sở dữ liệu
- Cân bằng tải là sống còn

10.000-100.000 RPS:

- Quy mô rất lớn
- 100+ máy chủ
- Phân mảnh cơ sở dữ liệu
- Phân bố theo địa lý
- Bắt buộc có CDN

100.000+ RPS:

- Quy mô khổng lồ (cỡ Google, Facebook)
- Hàng nghìn máy chủ
- Hạ tầng tự xây dựng
- Điện toán biên
- Tối ưu nâng cao ở mọi nơi

Quy Tắc Kinh Nghiệm Về Năng Lực Máy Chủ

Máy chủ web thông thường:

- Truy vấn đơn giản: ~1.000 req/s
- Độ phức tạp trung bình: ~500 req/s
- Logic phức tạp: ~100 req/s

Cơ sở dữ liệu:

- Đọc đơn giản: ~10.000 req/s
- Ghi đơn giản: ~5.000 req/s
- Truy vấn phức tạp: ~100 req/s
- Giao dịch: ~1.000 req/s

Bộ nhớ đệm (Redis):

- Đọc: ~100.000 req/s mỗi nút
- Ghi: ~50.000 req/s mỗi nút

Dùng các con số này để ước lượng:

Cần 5.000 req/s với độ phức tạp trung bình

$5.000 / 500 =$ tối thiểu 10 máy chủ

Thêm 50% dư phòng: 15 máy chủ

Cơ sở dữ liệu không theo kịp?

Dùng bộ nhớ đệm để giảm 90% tải lên CSDL

$5.000 \times 0,1 = 500$ req/s tới CSDL → Một CSDL là ổn

Trực Giác Về Tốc Độ Tăng Lưu Trữ

Tính Toán Lưu Trữ

Kích thước dữ liệu phổ biến:

Bản ghi người dùng: ~1KB

- tên người dùng, email, mật khẩu đã băm, siêu dữ liệu

Tweet/Bài đăng: ~500 byte

- nội dung văn bản, user_id, dấu thời gian, siêu dữ liệu

Ảnh (đã nén): ~2MB

- JPEG, tối ưu cho web

Video (1 phút, 720p): ~50MB

- Nén H.264

Một dòng log: ~200 byte

- dấu thời gian, mức độ, thông điệp

Ví Dụ Tính Toán Tăng Trưởng

Kịch bản: Ứng dụng chia sẻ ảnh

Người dùng: 1 triệu
Hoạt động hằng ngày: 200K
Số ảnh tải lên mỗi người dùng hoạt động: 2

Lượt tải lên mỗi ngày:
 $200K \times 2 = 400K$ ảnh/ngày

Lưu trữ mỗi ngày:
 $400K \times 2MB = 800GB/ngày$

Hàng tháng: $800GB \times 30 = 24TB/tháng$
Hàng năm: $24TB \times 12 = 288TB/năm$

Chi phí (AWS S3):
\$0,023/GB/tháng
 $288TB = 288.000GB$
Chi phí: $288.000 \times \$0,023 = \$6.624/tháng$

→ Đáng kê! Cần nén, CDN, phân tầng lưu trữ

Các chiến lược tối ưu (optimization):

Tính toán ban đầu: 288TB/năm

Sau khi tối ưu:

1. Nén mạnh tay: -30% = 200TB
2. Xoá ảnh cũ/không dùng: -20% = 160TB
3. Dùng phân tầng lưu trữ (lưu trữ lạnh): -40% chi phí
4. Tổng: 160TB với giá \$0,014/GB = \$2.240/tháng

Tiết kiệm: \$4.384/tháng (giảm 66%!)

Mẫu Hình Thiên Về Đọc (Read Heavy) và Thiên Về Ghi (Write Heavy)

Nhận Diện Mẫu Hình (Pattern)



Mỗi mẫu hình khác nhau cần một chiến lược tối ưu (optimization strategy) khác nhau

Hệ Thống Thiên Về Đọc (Read-Heavy) (90%+ Lướt Đọc)

Ví dụ: Mạng xã hội, tin tức, blog, tài liệu

Đặc điểm:

- Nhiều người dùng xem cùng một nội dung
- Nội dung không thay đổi thường xuyên
- Tỷ lệ trùng bộ nhớ đệm rất cao

Chiến lược tối ưu:

- ✓ Lưu đệm quyết liệt (Redis, Memcached)
- ✓ CDN cho nội dung tĩnh
- ✓ Bản sao chỉ đọc cho cơ sở dữ liệu
- ✓ Nhất quán cuối cùng là chấp nhận được
- ✗ Đừng tối ưu việc ghi (không phải điểm nghẽn)

Trọng tâm hạ tầng:

- Tầng bộ nhớ đệm (quan trọng nhất)
- CDN (cho nội dung đa phương tiện)
- Nhiều bản sao chỉ đọc

Ví dụ: Reddit

- 1 bài đăng → 10.000 lượt xem
- Lưu đệm bài đăng trong 5 phút
- Tỷ lệ trùng bộ nhớ đệm 99,99%
- Tải lên cơ sở dữ liệu rất nhỏ

Hệ Thống Thiên Về Ghi (Write-Heavy) (50%+ Lướt Ghi)

Ví dụ: Phân tích dữ liệu, ghi log, cảm biến IoT, giao dịch tài chính

Đặc điểm:

- Dữ liệu được nạp vào liên tục
- Việc đọc ít thường xuyên hơn
- Dữ liệu thường là chuỗi thời gian

Chiến lược tối ưu:

- ✓ Cơ sở dữ liệu tối ưu cho ghi (Cassandra, CSDL chuỗi thời gian)
- ✓ Ghi bất đồng bộ (dùng hàng đợi làm vùng đệm)
- ✓ Xur'ly theo lô
- ✓ Phân mảnh/phân vùng dữ liệu
- x Lưu đệm kém hiệu quả hơn

Trọng tâm hạ tầng:

- Thông lượng ghi (cơ sở dữ liệu)
- Hàng đợi thông điệp (vùng đệm)
- Xur'ly theo lô

Ví dụ: Hệ thống giám sát

- 10K máy chủ × 100 chỉ số/phút
- 1M lướt ghi/phút = 16K/giây
- Đọc: Một lần/ngày cho bảng điều khiển
- Tối ưu cho thông lượng ghi

Hệ Thống Cân Bằng (50-80% Lướt Đọc)

Ví dụ: Thương mại điện tử, CRM, công cụ năng suất

Đặc điểm:

- Kết hợp cả đọc và ghi
- Cận tối ưu cả hai
- Các đánh đổi phức tạp hơn

Chiến lược tối ưu:

- ✓ Lưu đệm có chọn lọc (chỉ dữ liệu "nóng")
- ✓ Tối ưu cơ sở dữ liệu (chỉ mục)
- ✓ Bản sao chỉ đọc cho báo cáo
- ✓ Tối ưu ghi cho các giao dịch

Trọng tâm hạ tầng:

- Cách tiếp cận cân bằng
- Lưu đệm các luồng xur'ly "nóng"
- Tối ưu các truy vấn quan trọng

Tư Duy Về Điểm Nghẽn (Bottleneck)

Nguyên tắc cốt lõi: Hệ thống chỉ nhanh bằng thành phần chậm nhất của nó

Chuỗi Điểm Nghẽn

Luồng xử lý yêu cầu:

Máy khách → (Mạng 50ms)

→ Bộ cân bằng tải (5ms)

→ Máy chủ ứng dụng (20ms)

→ Truy vấn CSDL (500ms) ← ĐIỂM NGHẼN

→ Máy chủ ứng dụng (10ms)

→ Máy khách (50ms)

Tổng: 635ms

Điểm nghẽn: Cơ sở dữ liệu (79% thời gian)

Thứ tự ưu tiên tối ưu:

1. Cơ sở dữ liệu (tác động lớn nhất)
2. Mạng (nếu không sửa được CSDL)
3. Logic ứng dụng (tác động rất nhỏ)

Đừng phí thời gian tối ưu máy chủ ứng dụng từ 20ms → 10ms
khi cơ sở dữ liệu mất tới 500ms!

Xác Định Điểm Nghẽn (Bottlenecks)

Hãy đặt những câu hỏi sau:

1. Cái gì tốn nhiều thời gian nhất?
 - Đo từng thành phần
 - Tìm ra thành phần chậm nhất
2. Cái gì chịu tải cao nhất?
 - Mức dùng CPU
 - Mức dùng bộ nhớ
 - Mạng bị bão hoà
 - Disk I/O
3. Cái gì hỏng đầu tiên khi tải tăng?
 - Kết nối cơ sở dữ liệu đã chạm mức tối đa?
 - Bộ nhớ máy chủ đầy?
 - Băng thông mạng bị bão hoà?

Điểm nghẽn là nơi bạn chạm giới hạn đầu tiên.

Các Điểm Nghẽn Thường Gặp Theo Quy Mô

1K người dùng:

- Điểm nghẽn: Thường là không có
- Nếu có: Truy vấn kém hiệu quả

10K người dùng:

- Điểm nghẽn: Truy vấn cơ sở dữ liệu
- Cách khắc phục: Thêm chỉ mục, bộ nhớ đệm

100K người dùng:

- Điểm nghẽn: Ghi vào cơ sở dữ liệu
- Cách khắc phục: Bản sao chỉ đọc, tối ưu

1M người dùng:

- Điểm nghẽn: Năng lực cơ sở dữ liệu
- Cách khắc phục: Phân mảnh dữ liệu, bộ nhớ đệm phân tán

10M+ người dùng:

- Điểm nghẽn: Mọi thứ
- Cách khắc phục: Kiến trúc phân tán, CDN, điện toán biên

Bài Tập Thực Hành

Bài tập 1: Ước Lượng Một Dịch Vụ Kiểu YouTube

Dữ kiện:

- 1 tỷ người dùng
- 100 triệu người dùng hoạt động hằng ngày
- Mỗi người dùng xem 10 video/ngày (trung bình 5 phút mỗi video)
- 1% người dùng tải lên 1 video/ngày

Hãy tính:

1. Số lượt xem video mỗi giây
2. Băng thông tải lên cần thiết
3. Dung lượng lưu trữ tăng thêm mỗi ngày
4. Chi phí hạ tầng ước chừng

Hãy tự thử trước!

Đáp án:

1. Số lượt xem video mỗi giây:
 $100M \text{ người dùng} \times 10 \text{ video/ngày} = 1B \text{ lượt xem/ngày}$
 $1B / 86.400 \text{ giây} \approx 11.574 \text{ lượt xem/giây}$
2. Băng thông tải lên:
 $1M \text{ lượt tải lên/ngày (1\% của 100M)}$
Video trung bình: $5 \text{ phút} \times 10MB/\text{phút} = 50MB$
 $50MB \times 1M = 50TB/\text{ngày}$ băng thông tải lên
 $50TB / 86.400s \approx 580 \text{ MB/giây tải lên}$
3. Dung lượng lưu trữ tăng thêm:
 $1M \text{ video/ngày} \times 50MB = 50TB/\text{ngày}$ dữ liệu thô
Nhiều độ phân giải (360p, 720p, 1080p): $\times 3$
 $= 150TB/\text{ngày}$
 $= 4,5PB/\text{tháng}$
4. Chi phí hạ tầng (ước chừng):
Lưu trữ: $4,5PB \times \$0,02/GB = \$90.000/\text{tháng}$
CDN: $1B \text{ lượt xem} \times 50MB = 50PB \text{ truyền tải}$
Chi phí CDN: $\sim \$500.000/\text{tháng}$
Tính toán: $\sim \$200.000/\text{tháng}$
Tổng: tối thiểu $\sim \$800.000/\text{tháng}$

Bài tập 2: Ước Lượng Một Ứng Dụng Nhắn Tin

Dữ kiện:

- 500 triệu người dùng
- 200 triệu người dùng hoạt động hằng ngày
- Mỗi người dùng gửi 50 tin nhắn/ngày
- Mỗi người dùng nhận 50 tin nhắn/ngày

Hãy tính:

1. Số tin nhắn mỗi giây
2. Dung lượng lưu trữ mỗi ngày
3. Số lượt ghi cơ sở dữ liệu mỗi giây
4. Thách thức khi gửi tin theo thời gian thực

Đến lượt bạn!

Đáp án:

1. Số tin nhắn mỗi giây:
 $200M \text{ người dùng} \times 50 \text{ tin nhắn} = 10B \text{ tin nhắn/ngày}$
 $10B / 86.400 \approx 115.740 \text{ tin nhắn/giây}$
2. Dung lượng lưu trữ mỗi ngày:
 $10B \text{ tin nhắn} \times 100 \text{ byte} = 1TB/\text{ngày}$
(tin nhắn văn bản có kích thước nhỏ)
3. Số lượt ghi cơ sở dữ liệu:
 $115.740 \text{ lượt ghi/giây}$
→ Cần cơ sở dữ liệu phân tán hoặc vùng đệm cho việc ghi
4. Gửi tin theo thời gian thực:
 $115.740 \text{ tin nhắn/giây}$ cần được chuyển đi
Mỗi tin cần được đẩy qua WebSocket
→ Cần hệ thống pub/sub hiệu quả
→ Hàng đợi thông điệp là thiết yếu

Xây Dựng Trực Giác: Hướng Dẫn Luyện Tập

Cách rèn luyện trực giác về quy mô:

Bước 1: Ghi Nhớ Các Con Số Then Chốt

Bắt buộc phải biết:

- 1 triệu giây ≈ 12 ngày
- 1 tỷ giây ≈ 32 năm
- 1KB = 1.000 byte
- 1MB = 1.000KB = 1 triệu byte
- 1GB = 1.000MB = 1 tỷ byte
- 1TB = 1.000GB

Máy chủ thông thường:

- 1.000 yêu cầu/giây
- 16GB RAM
- 1TB lưu trữ

Chi phí thông thường:

- Máy chủ: \$50-500/tháng
- Cơ sở dữ liệu: \$50-1.000/tháng
- CDN: \$10-1.000/tháng
- Lưu trữ: \$0,02/GB/tháng

Bước 2: Luyện Ước Lượng (Estimation) Mỗi Ngày

Mỗi ngày, hãy ước lượng:

Thứ Hai: "Instagram có 500M DAU. Mỗi ngày có bao nhiêu ảnh được tải lên?"

Thứ Ba: "Twitter xử lý bao nhiêu tweet/giây?"

Thứ Tư: "Băng thông mỗi giây của Netflix là bao nhiêu?"

Thứ Năm: "Google xử lý bao nhiêu lượt tìm kiếm mỗi giây?"

Thứ Sáu: "Dung lượng lưu trữ của Facebook tăng bao nhiêu mỗi ngày?"

Có công mài sắt, có ngày nên kim!

Bước 3: Kiểm Chứng Các Ước Lượng (Estimates) Của Bạn

Sau khi ước lượng, hãy tìm hiểu con số thực tế:

- Instagram: 95M ảnh/ngày
- Twitter: 6.000 tweet/giây
- Netflix: băng thông cao điểm 200Gbps
- Google: 40.000 lượt tìm kiếm/giây

So sánh với ước lượng của bạn

Hiệu chỉnh lại trực giác của bạn

Những Điểm Cốt Lỗi

Trực giác về quy mô = Ước lượng nhanh các yêu cầu của hệ thống

Kỹ năng then chốt cho:

- Phỏng vấn thiết kế hệ thống
- Lập kế hoạch kiến trúc
- Hoạch định năng lực
- Ước lượng chi phí

Phổ quy mô:

1K người dùng: Xe đạp (đơn giản)

10K người dùng: Xe hơi (thêm cấu trúc)

100K người dùng: Xe buýt (cần tối ưu)

1M người dùng: Máy bay (hệ thống phân tán)

10M+ người dùng: Hạm đội máy bay (hạ tầng phức tạp)

Mỗi lần tăng 10 lần = kiến trúc khác biệt về căn bản

Ước tính nhanh:

Khung tư duy:

1. Người dùng → Hoạt động → Yêu cầu
2. Yêu cầu → Năng lực máy chủ
3. Dữ liệu → Lưu trữ → Chi phí
4. Xác định điểm nghẽn

Luyện tập cho đến khi thành phản xạ!

Mẫu hình đọc (Read) và ghi (Write):

Thiên về đọc (90%+):

- Lưu đệm là vua
- CDN là thiết yếu
- Bản sao chỉ đọc

Thiên về ghi (50%+):

- Tối ưu việc ghi
- Đùng hàng đợi làm vùng đệm
- Cơ sở dữ liệu chuyên dụng

Biết mẫu hình của mình → Biết chiến lược của mình

Tư duy về điểm nghẽn:

Tốc độ hệ thống = Tốc độ của thành phần chậm nhất

Luôn tự hỏi:

- Điểm nghẽn là gì?
- Cái gì chậm giới hạn đầu tiên?
- Nên tối ưu cái gì trước?

Tối ưu điểm nghẽn, đừng tối ưu mọi thứ!

Xây dựng trực giác:

1. Ghi nhớ các con số then chốt
2. Luyện ước lượng mỗi ngày
3. Kiểm chứng với dữ liệu thực tế
4. Hiệu chỉnh và cải thiện

Sau 100 lần ước lượng, bạn sẽ có trực giác vững vàng

Hãy nhớ:

Trực giác không đến từ lý thuyết
Trực giác đến từ thực hành

Hãy tính toán mỗi ngày
Sau 1 tháng, bạn sẽ "cảm" được quy mô

Khung Tư Duy Hệ Thống (System Thinking Framework): Nhìn Hệ Thống Như Một Tổng Thể

Tôi còn nhớ lần đầu tiên nhìn vào mã nguồn (codebase) của một hệ thống lớn.

Hàng trăm tệp. Hàng chục vi dịch vụ (microservices). Cơ sở dữ liệu (Database), bộ nhớ đệm (cache), hàng đợi (queues), API...

Tôi hỏi một kỹ sư lâu năm (senior): "Làm sao anh hiểu được hết những thứ này?"

Anh ấy cười: "Em không cần hiểu từng dòng mã. Em cần hiểu hệ thống như một **tổng thể**."

Tôi: "Nghĩa là sao ạ?"

Anh ấy vẽ một sơ đồ đơn giản:

```
graph LR
    User --> API
    API --> Cache
    Cache --> Database
    API --> Queue
    Queue --> Worker
```

"Đây là toàn bộ hệ thống. Dữ liệu chảy (Data flow) từ người dùng, qua API, kiểm tra bộ nhớ đệm, truy vấn cơ sở dữ liệu, các tác vụ bất đồng bộ (async jobs) được đưa vào hàng đợi. Hiểu luồng này quan trọng hơn hiểu từng hàm."

Đó là lần đầu tiên tôi học về tư duy hệ thống (System Thinking).

Tại Sao Phải Tư Duy Theo Hệ Thống?

Vấn Đề: Tư Duy Theo Mã

Hầu hết lập trình viên (developers) đều nghĩ như thế này:

```
# Code-level thinking
def get_user(user_id):
    user = cache.get(user_id)
    if not user:
        user = db.query("SELECT * FROM users WHERE id = ?", user_id)
        cache.set(user_id, user)
    return user
```

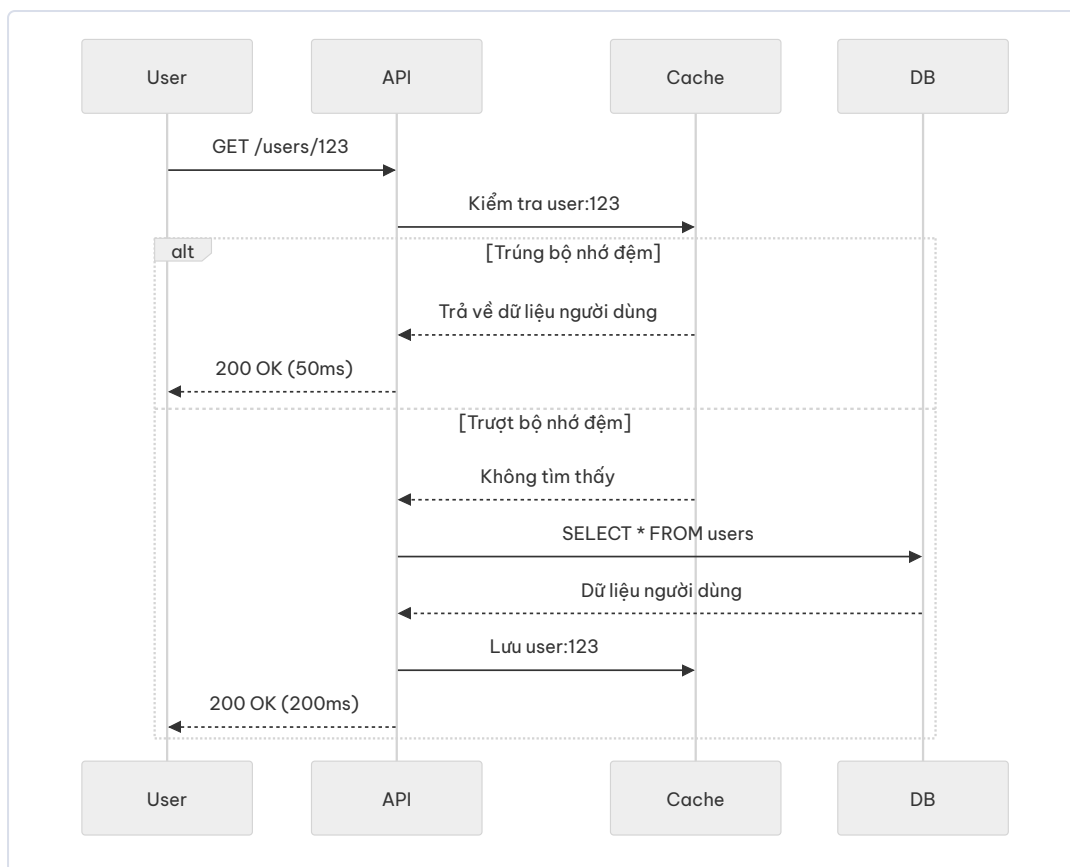
Bạn thấy: hàm, logic lưu đệm, truy vấn cơ sở dữ liệu.

Nhưng bạn không thấy:

- Yêu cầu (*Request*) đi từ đâu đến?
- Phản hồi (*Response*) đi về đâu?
- Bộ nhớ đệm gặp sự cố (*fail*) thì sao?
- Cơ sở dữ liệu chậm thì sao?
- Mở rộng (*Scale*) như thế nào?

Giải Pháp: Góc Nhìn Ở Cấp Độ Hệ Thống

Tư duy hệ thống nhìn như thế này:



Góc nhìn hệ thống: Thấy được luồng xử lý (flow), các tương tác (interactions) và các kịch bản sự cố (failure scenarios)

Thấy sự khác biệt chưa?

Tư duy hệ thống giúp bạn:

- Hiểu luồng dữ liệu
- Phát hiện điểm nghẽn (bottlenecks)
- Thiết kế với giả định sự cố sẽ xảy ra
- Mở rộng một cách thông minh
- Gỡ lỗi (Debug) các vấn đề trong môi trường thực tế (production)

Mẫu (Pattern) Hệ Thống Phổ Quát

Mọi hệ thống đều tuân theo một mẫu cơ bản:

Đầu vào → Xử lý → Lưu trữ → Đầu ra



Mẫu phổ quát: Mọi hệ thống đều nhận đầu vào, xử lý, lưu trữ và trả về đầu ra

Ví Dụ Về Mẫu Đây

Ứng dụng web:

Đầu vào: Người dùng nhấn "Đặt hàng"
Xử lý: Kiểm tra hợp lệ, tính tổng tiền, kiểm tra tồn kho
Lưu trữ: Lưu đơn hàng vào cơ sở dữ liệu
Đầu ra: Hiển thị trang xác nhận

Hệ thống nhắn tin:

Đầu vào: Người dùng gửi tin nhắn
Xử lý: Mã hoá, định tuyến đến người nhận
Lưu trữ: Lưu vào hàng đợi thông điệp
Đầu ra: Chuyển đến thiết bị của người nhận

Đường ống phân tích dữ liệu (Analytics Pipeline):

Đầu vào: Sự kiện từ các ứng dụng
Xử lý: Tổng hợp, biến đổi dữ liệu
Lưu trữ: Ghi vào kho dữ liệu (data warehouse)
Đầu ra: Cập nhật bảng điều khiển

Nền tảng video:

Đầu vào: Người dùng tải video lên
Xử lý: Mã hoá sang nhiều định dạng
Lưu trữ: Lưu vào kho lưu trữ đối tượng (S3)
Đầu ra: Video sẵn sàng để phát trực tuyến

Điểm mấu chốt: Hiểu mẫu này → Hiểu mọi hệ thống.

Các Thành Phần Cốt Lõi Của Hệ Thống

Mọi hệ thống web đều có 6 thành phần (*components*) cơ bản:

Thành Phần 1: Máy Khách (*Client*) (Tầng Đầu Vào)

Vai trò: Khởi tạo yêu cầu, nhận phản hồi

Các loại:

- Trình duyệt web (JavaScript)
- Ứng dụng di động (iOS/Android)
- Ứng dụng máy tính
- Các dịch vụ khác (máy khách API)

Đặc điểm:

- Không đáng tin cậy (người dùng có thể 'sử' đổi)
- Không kiểm soát được (mạng, thiết bị)
- Có trạng thái (có trạng thái cục bộ)

Nguyên tắc cốt lõi: KHÔNG BAO GIỜ tin tưởng đầu vào từ máy khách

Ví dụ về luồng xử lý:

Hành động của người dùng:

1. Người dùng nhấn "Đăng nhập"
2. Trình duyệt gửi POST /login kèm thông tin đăng nhập
3. Chờ phản hồi
4. Hiện thị kết quả

Những gì có thể 'sai':

- Người dùng sửa đổi yêu cầu (đổi `user_id`)
- Hết thời gian chờ mạng
- Phản hồi chậm
- Thiết bị bị treo

Hệ thống phải xử lý được tất cả những điều này!

Thành Phần 2: API (Tầng Xử Lý)

Vai trò: Xử lý yêu cầu, thực thi logic nghiệp vụ (*business logic*), điều phối luồng dữ liệu

Trách nhiệm:

- Kiểm tra hợp lệ đầu vào (máy khách không đáng tin cậy!)
- Thực thi các quy tắc nghiệp vụ
- Điều phối giữa các dịch vụ
- Định dạng phản hồi
- Xử lý lỗi

Đặc điểm:

- Phi trạng thái (nên là phi trạng thái!)
- Mở rộng được theo chiều ngang
- Chứa logic nghiệp vụ

Ví dụ công nghệ:

- REST API (Express, FastAPI, Spring)
- GraphQL API
- Dịch vụ gRPC

Ví dụ:

```
# API Server handles request
@app.post("/orders")
def create_order(order_data):
    # 1. Validate (client không tin cậy)
    if not validate_order(order_data):
        return 400, {"error": "Invalid order"}

    # 2. Business logic
    total = calculate_total(order_data)

    # 3. Check inventory
    if not check_inventory(order_data.items):
        return 400, {"error": "Out of stock"}

    # 4. Save to database
    order = db.create_order(order_data, total)

    # 5. Async processing (queue)
    queue.publish("order.created", order.id)

    # 6. Return response
    return 201, {"order_id": order.id}
```

Thành Phần 3: Cơ Sở Dữ Liệu (Tầng Lưu Trữ)

Vai trò: Lưu bền dữ liệu một cách đáng tin cậy

Các loại:

- Quan hệ (PostgreSQL, MySQL)
 - Dữ liệu có cấu trúc, giao dịch
- NoSQL (MongoDB, Cassandra)
 - Lược đồ linh hoạt, mở rộng theo chiều ngang
- Chuỗi thời gian (InfluxDB, TimescaleDB)
 - Số liệu đo, log, sự kiện

Đặc điểm:

- Thuộc tính ACID (SQL) hoặc nhất quán cuối cùng (NoSQL)
- Chậm hơn bộ nhớ (I/O đĩa)
- Thường là điểm nghẽn
- Dữ liệu phải tồn tại được qua các sự cố/sập máy

Nguyên tắc cốt lõi: Nguồn dữ liệu gốc (source of truth)

Khi nào dùng loại nào:

SQL (PostgreSQL):

- ✓ Cần giao dịch
 - ✓ Quan hệ phức tạp
 - ✓ Tính toàn vẹn dữ liệu là then chốt
- Ví dụ: Thương mại điện tử, ngân hàng

NoSQL (MongoDB):

- ✓ Lược đồ linh hoạt
 - ✓ Mở rộng theo chiều ngang
 - ✓ Truy vấn đơn giản
- Ví dụ: Hồ sơ người dùng, nội dung

Chuỗi thời gian (InfluxDB):

- ✓ Số liệu đo/giám sát
 - ✓ Dữ liệu cảm biến IoT
 - ✓ Truy vấn theo thời gian
- Ví dụ: Khả năng quan sát, phân tích dữ liệu

Thành Phần 4: Bộ Nhớ Đệm (Tăng Hiệu Năng)

Vai trò: Lưu dữ liệu được truy cập thường xuyên trong bộ nhớ

Tại sao cần bộ nhớ đệm?

- Bộ nhớ: 100 nano giây
- SSD: 150 micro giây (chậm hơn 1.500 lần)
- HDD: 10 mili giây (chậm hơn 100.000 lần)

→ Bộ nhớ đệm = Tăng tốc thao tác đọc đáng kể

Các cách triển khai phổ biến:

- Redis (phổ biến nhất)
- Memcached (key-value đơn giản)
- Ở cấp ứng dụng (trong tiến trình)

Đặc điểm:

- Dễ mất (dữ liệu có thể bị mất)
- Dung lượng hạn chế (RAM đắt đỏ)
- Có thể có dữ liệu cũ
- Vô hiệu hoá bộ nhớ đệm là việc khó

Nguyên tắc cốt lõi: Bộ nhớ đệm là đề' tối ưu, không phải nguồn dữ liệu gốc

Các mẫu lưu đệm (Cache patterns):

```
# Pattern 1: Cache-Aside (Lazy Loading)
def get_user(user_id):
    # Check cache first
    user = cache.get(f"user:{user_id}")
    if user:
        return user # Cache hit

    # Cache miss → Query database
    user = db.query("SELECT * FROM users WHERE id = ?", user_id)

    # Store in cache for next time
    cache.set(f"user:{user_id}", user, ttl=3600)
    return user

# Pattern 2: Write-Through
def update_user(user_id, data):
    # Update database
    db.update("users", user_id, data)

    # Update cache immediately
    cache.set(f"user:{user_id}", data)
```

Thành Phần 5: Hàng Đợi (Queue) (Tầng Bất Đồng Bộ)

Vai trò: Tách rời (Decouple) các dịch vụ, cho phép xử lý bất đồng bộ (async processing)

Tại sao cần hàng đợi?

- Tác vụ chạy lâu (không chặn người dùng)
- Lưu lượng tăng đột biến (làm vùng đệm cho yêu cầu)
- Tách rời dịch vụ (bên gửi không cần bên nhận phải trực tuyến)
- Logic thư' lại (xư' lý sự cố)

Các cách triển khai phổ' biến:

- RabbitMQ (message broker)
- Kafka (truyền luồng sự kiện)
- AWS SQS (hàng đợi được quản lý)
- Redis (hàng đợi đơn giản)

Đặc điểm:

- Bất đồng bộ (gửi rồi quên)
- Chuyển phát ít nhất một lần (thường là vậy)
- Có thứ tự hoặc không có thứ tự
- Lưu bền hoặc chỉ trong bộ nhớ

Nguyên tắc cốt lõi: Dừng cho công việc có thể' trì hoãn

Ví dụ:

```
# API: Quick response to user
@app.post("/orders")
def create_order(order_data):
    order = db.create_order(order_data)

    # Don't send email synchronously (slow!)
    # Put in queue instead
    queue.publish("order.created", {
        "order_id": order.id,
        "user_email": order_data.email
    })

    return 201, {"order_id": order.id}
    # User gets response immediately!

# Worker: Process async
@worker.task("order.created")
def on_order_created(message):
    # This runs in background
    send_confirmation_email(message.user_email)
    update_inventory(message.order_id)
    notify_warehouse(message.order_id)
```

Thành Phần 6: Lưu Trữ (Storage) (Tăng Tập)

Vai trò: Lưu các tệp lớn (hình ảnh, video, tài liệu)

Tại sao tách riêng khỏi cơ sở dữ liệu?

- Cơ sở dữ liệu không phù hợp với tệp lớn
- Cần khả năng mở rộng (terabyte/petabyte)
- Cần tích hợp với CDN
- Tối ưu chi phí

Các cách triển khai phổ biến:

- S3 (lưu trữ đối tượng của AWS)
- Google Cloud Storage
- Azure Blob Storage
- MinIO (tự vận hành, tương thích S3)

Đặc điểm:

- Rẻ (so với cơ sở dữ liệu)
- Mở rộng được (dung lượng không giới hạn)
- Bền vững (độ bền dữ liệu 99,99999999%)
- Chậm với tệp nhỏ (chi phí phụ về độ trễ)

Nguyên tắc cốt lõi: Lưu siêu dữ liệu trong CSDL, tệp trong kho lưu trữ đối tượng

Ví dụ:

```
# Upload flow
@app.post("/upload")
def upload_photo(file):
    # 1. Generate unique filename
    filename = f"{uuid.uuid4()}.jpg"

    # 2. Upload to S3
    s3_url = s3.upload(filename, file)

    # 3. Save metadata to database
    photo = db.create({
        "filename": filename,
        "s3_url": s3_url,
        "size": file.size,
        "uploaded_at": datetime.now()
    })

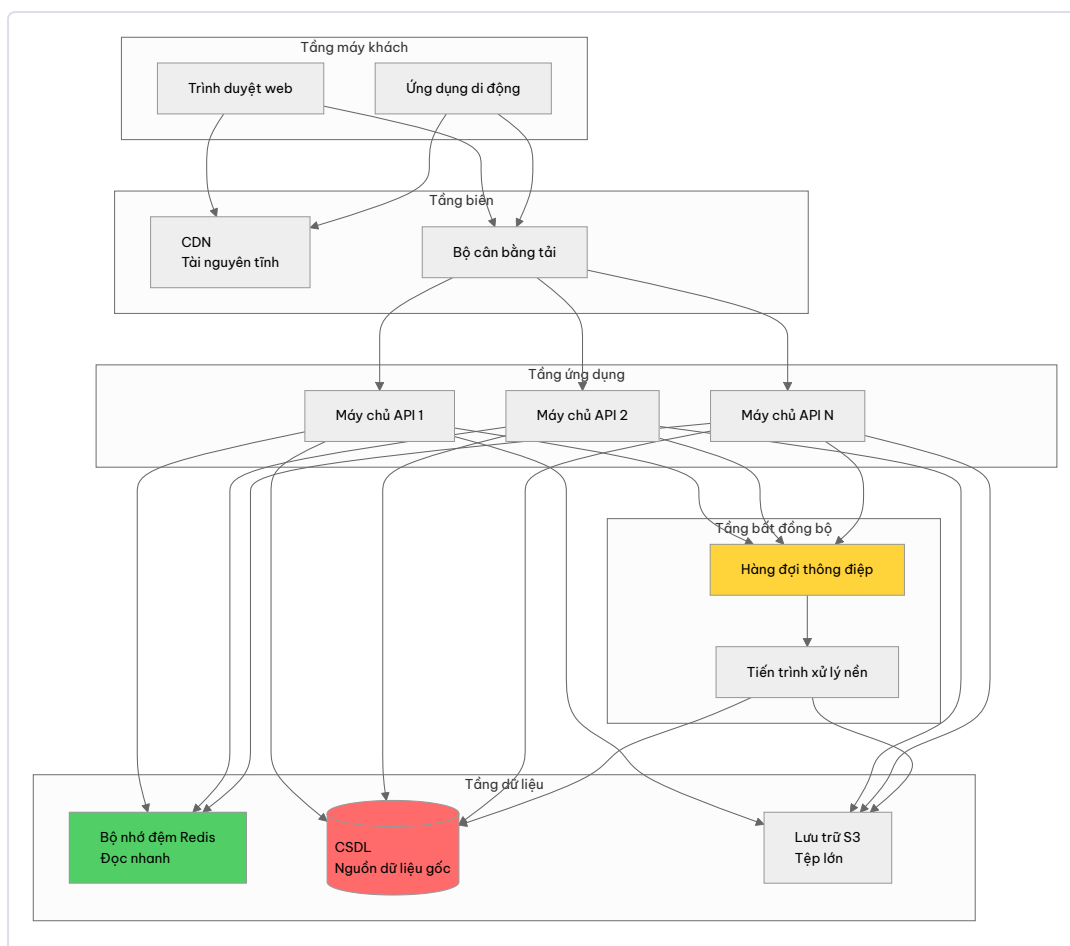
    return {"photo_id": photo.id, "url": s3_url}

# Retrieve flow
@app.get("/photos/{id}")
def get_photo(id):
    # Get metadata from database
    photo = db.get_photo(id)

    # Client downloads from S3 directly
    return {"url": photo.s3_url}
```


Ghép Tất Cả Lại: Kiến Trúc Hệ Thống (System Architecture)

Kiến trúc (Architecture) **diễn hình** của một ứng dụng web:



Hệ thống hoàn chỉnh: Tất cả các thành phần phối hợp với nhau

Ví Dụ Về Luồng Yêu Cầu: Người Dùng Cập Nhật Hồ Sơ

Lần theo một yêu cầu đi qua toàn bộ hệ thống:

Bước 1: Người dùng nhấn "Lưu hồ sơ"

Thành phần: Máy khách (Trình duyệt)

Hành động: POST /users/123 kèm dữ liệu mới

Bước 2: Yêu cầu đến bộ cân bằng tải

Thành phần: Bộ cân bằng tải

Hành động: Định tuyến đến Máy chủ API 2 (ít kết nối nhất)

Thời gian: +10ms

Bước 3: API kiểm tra hợp lệ và xử lý

Thành phần: Máy chủ API

Hành động:

- Kiểm tra hợp lệ đầu vào (định dạng email, v.v.)
- Kiểm tra quyền (người dùng có được phép không?)
- Làm sạch dữ liệu

Thời gian: +20ms

Bước 4: Cập nhật cơ sở dữ liệu

Thành phần: Cơ sở dữ liệu

Hành động:

- Thực thi truy vấn UPDATE
- Cam kết giao dịch
- Xác nhận thành công

Thời gian: +100ms

Bước 5: Vô hiệu hoá bộ nhớ đệm

Thành phần: Bộ nhớ đệm

Hành động:

- Xoá khoá bộ nhớ đệm "user:123"
- Buộc đọc dữ liệu mới ở lần sau

Thời gian: +5ms

Bước 6: Đưa các tác vụ bất đồng bộ vào hàng đợi

Thành phần: Hàng đợi

Hành động:

- Xuất bản sự kiện "user.updated"
- Không chặn (trả về ngay lập tức)

Thời gian: +2ms

Bước 7: Trả phản hồi cho người dùng

Thành phần: API → Máy khách

Phản hồi: 200 OK

Tổng thời gian: 137ms (trải nghiệm người dùng tốt!)

Bước 8: Xử lý nền

Thành phần: Tiến trình xử lý (bất đồng bộ)

Hành động:

- Gửi email "hồ sơ đã được cập nhật"
- Cập nhật chỉ mục tìm kiếm
- Đồng bộ sang hệ thống phân tích

Thời gian: Không ảnh hưởng đến trải nghiệm người dùng

Những nhận xét chính:

- ✓ Người dùng nhận phản hồi trong 137ms (nhANH!)
- ✓ Công việc nặng được xử lý bất đồng bộ (email, lập chỉ mục)
- ✓ Bộ nhớ đệm đã được vô hiệu hoá (tính nhất quán)
- ✓ Mỗi thành phần có vai trò rõ ràng
- ✓ Sự có'có thể' được cô lập

Đây chính là tư duy hệ thống trong thực tế!

Mô Hình Tư Duy (*Mental Model*): Đầu Vào → Xử Lý → Lưu Trữ → Đầu Ra

Áp dụng cho mọi thứ:

Ví Dụ 1: Thanh Toán Thương Mại Điện Tử

Đầu vào:

- Người dùng nhấn "Đặt hàng"
- Các món trong giỏ hàng, địa chỉ giao hàng, thông tin thanh toán

Xử lý:

- Kiểm tra tồn kho
- Tính tổng tiền (hàng hoá + thuế + phí vận chuyển)
- Xử lý thanh toán
- Sinh mã đơn hàng

Lưu trữ:

- Lưu đơn hàng vào cơ sở dữ liệu
- Lưu biên lai thanh toán vào S3
- Lưu đệm chi tiết đơn hàng

Đầu ra:

- Trang xác nhận cho người dùng
- Email xác nhận (bất đồng bộ qua hàng đợi)
- Thông báo đến kho hàng (bất đồng bộ)

Ví Dụ 2: Bài Đăng Mạng Xã Hội

Đầu vào:

- Người dùng viết bài + tải ảnh lên

Xử lý:

- Làm sạch văn bản (loại bỏ từ ngữ xấu)
- Thay đổi kích thước/nén ảnh
- Sinh mã bài đăng

Lưu trữ:

- Lưu bài đăng vào cơ sở dữ liệu
- Tải ảnh lên S3
- Lưu đếm các bài đăng "nóng" (đang thịnh hành)

Đầu ra:

- Hiển thị bài đăng cho người dùng
- Phân phát (fan-out) đến người theo dõi (bắt đồng bộ qua hàng đợi)
- Cập nhật bộ nhớ đếm bảng tin (bắt đồng bộ)

Ví Dụ 3: Tải Video Lên

Đầu vào:

- Người dùng tải tệp video lên

Xử lý:

- Kiểm tra hợp lệ tệp (định dạng, kích thước)
- Tạo ảnh thu nhỏ
- Đưa tác vụ mã hoá vào hàng đợi

Lưu trữ:

- Lưu bản gốc vào S3
- Lưu siêu dữ liệu vào cơ sở dữ liệu
- Lưu tác vụ mã hoá trong hàng đợi

Đầu ra:

- Hiển thị "Đang xử lý..." cho người dùng
- Gửi email khi sẵn sàng (bắt đồng bộ)
- Nhiều độ phân giải trong S3 (bắt đồng bộ)

Mẫu này nhất quán trên mọi hệ thống!

Các Mẫu Hệ Thống Phổ Biến

Mẫu 1: Hệ Thống Thiên Về Đọc (*Read-Heavy*)

Đặc điểm:

- Trên 90% là đọc, < 10% là ghi
- Ví dụ: Trang tin tức, tài liệu, bảng tin mạng xã hội

Tối ưu:

- Lưu đệm triệt đề' (Redis)
- CDN cho nội dung tĩnh
- Bản sao chỉ đọc của cơ sở dữ liệu
- Kiến trúc ưu tiên bộ nhớ đệm

Kiến trúc:

Máy khách → CDN (trúng 90%) → Bộ cân bằng tải → API → Bộ nhớ đệm (trúng 95%) → CSDL (5% còn lại)

Kết quả: 99,5% yêu cầu không bao giờ chạm đến cơ sở dữ liệu!

Mẫu 2: Hệ Thống Thiên Về Ghi (*Write-Heavy*)

Đặc điểm:

- Trên 50% là ghi
- Ví dụ: Phân tích dữ liệu, ghi log, cảm biến IoT

Tối ưu:

- Đệm thao tác ghi (hàng đợi)
- Xử lý theo lô
- Cơ sở dữ liệu tối ưu cho ghi (Cassandra)
- Bất đồng bộ mọi thứ

Kiến trúc:

Máy khách → API → Hàng đợi → Tiến trình xử lý → CSDL (ghi theo lô)

Kết quả: Xử lý 100K lượt ghi/giây nhờ vùng đệm

Mẫu 3: Hệ Thống Thời Gian Thực (Real-Time)

Đặc điểm:

- Độ trễ thấp là then chốt ($< 100\text{ms}$)
- Ví dụ: Chat, trò chơi, giao dịch chứng khoán

Tối ưu:

- Kết nối WebSocket
- Mọi thứ nằm trong bộ nhớ
- Phân bổ theo địa lý
- Xử lý tối thiểu

Kiến trúc:

Máy khách $\leftrightarrow$ Máy chủ WebSocket $\rightarrow$ Redis (pub/sub) $\rightarrow$ CSDL (nhất quán cuối cùng)

Kết quả: Chuyển tin nhắn dưới 50ms

Gỡ Lỗi Với Tư Duy Hệ Thống

Khi xảy ra sự cố trong môi trường thực tế:

Trước Khi Có Tư Duy Hệ Thống:

"API chậm quá. Để mình xem mã nguồn..."

Đọc 1000 dòng mã

Không biết vấn đề nằm ở đâu

Sau Khi Có Tư Duy Hệ Thống:

"API chậm quá. Để mình lần theo yêu cầu:

1. Máy khách → API: 50ms (bình thường)
2. API → Bộ nhớ đệm: 2ms (bình thường)
3. API → CSDL: 5000ms (CÓ VẤN ĐỀ!)
4. CSDL → API: 2ms
5. API → Máy khách: 10ms

Điều nghi ngờ: Truy vấn cơ sở dữ liệu mất 5 giây

Tiếp theo: Kiểm tra cơ sở dữ liệu

- Log truy vấn chậm
- Thiếu chỉ mục?
- Tranh chấp khoá?

Phát hiện: Thiếu chỉ mục trên user_id

Sửa: CREATE INDEX idx_user_id

Kết quả: Truy vấn giờ chỉ còn 50ms"

→ Giải quyết trong 10 phút!

Tư duy hệ thống = Cách tiếp cận gỡ lỗi có cấu trúc

Những Điểm Cốt Lõi

Hệ thống = Thành phần + Tương tác

6 thành phần cốt lõi:

1. Máy khách (tầng đầu vào)
2. API (tầng xử lý)
3. Cơ sở dữ liệu (tầng lưu trữ)
4. Bộ nhớ đệm (tầng hiệu năng)
5. Hàng đợi (tầng bất đồng bộ)
6. Lưu trữ (tầng tệp)

Mỗi thành phần có vai trò riêng

Cùng nhau tạo nên hệ thống hoàn chỉnh

Mẫu phổ quát:

Đầu vào → Xử lý → Lưu trữ → Đầu ra

Mọi hệ thống đều tuân theo mẫu này

Hiểu mẫu → Hiểu hệ thống

Lợi ích của tư duy hệ thống:

- ✓ Nhìn thấy bức tranh toàn cảnh
- ✓ Xác định điểm nghẽn nhanh hơn
- ✓ Thiết kế kiến trúc tốt hơn
- ✓ Gỡ lỗi các vấn đề trong môi trường thực tế
- ✓ Giao tiếp với nhóm
- ✓ Đưa ra các đánh đôi có cơ sở

Chuyển đổi tư duy:

Từ: "Hàm này hoạt động thế nào?"
Sang: "Yêu cầu đi qua hệ thống như thế nào?"

Từ: "Đoạn mã này làm gì?"
Sang: "Vai trò của thành phần này là gì?"

Từ: "Sửa lỗi này"
Sang: "Thành phần nào đang gây ra vấn đề?"

Hãy nhớ:

Mã nguồn là chi tiết triển khai
Hệ thống là bức tranh toàn cảnh

Làm chủ mã nguồn → Lập trình viên giỏi
Làm chủ hệ thống → Kiến trúc sư giỏi

Bạn cần cả hai, nhưng góc nhìn hệ thống phải đi trước

Khung Câu Hỏi Hệ Thống (*System Question Framework*) – Danh Sách Kiểm Tra Chuẩn Khi Thiết Kế Hệ Thống

Khi bắt đầu thiết kế một hệ thống, nhiều kỹ sư (*engineer*) mới thường nhảy thẳng vào giải pháp mà quên mất một bước cực kỳ quan trọng: **đặt câu hỏi đúng**.

Tôi từng mắc lỗi này. Khi thiết kế một hệ thống chat, tôi nghĩ ngay đến WebSocket, Redis Pub/Sub, mở rộng theo chiều ngang (*horizontal scaling*)... rồi giữa chừng mới phát hiện: khách hàng chỉ cần 100 người dùng trực tuyến đồng thời. Tôi đã thiết kế thừa (*over-engineer*) một cách thảm hại.

Vấn đề không phải là thiếu kiến thức kỹ thuật. Vấn đề là tôi đã không hỏi đủ câu hỏi trước khi thiết kế.

Tại Sao Cần Một Khung (*Framework*) Câu Hỏi?

Khi thiết kế hệ thống, có hàng trăm thứ cần cân nhắc. Không ai có thể nhớ hết. Bạn cần một **danh sách kiểm tra** để đảm bảo không bỏ sót điểm quan trọng nào.

Hãy nghĩ về nó như danh sách kiểm tra của phi công trước khi cất cánh. Dù đã bay 10.000 giờ, phi công vẫn phải kiểm tra từng bước một. Tại sao? Vì danh sách kiểm tra đảm bảo sự nhất quán và an toàn.

Thiết kế hệ thống (*System design*) cũng vậy. Khung câu hỏi là tấm lưới an toàn của bạn.

7 Câu Hỏi Cốt Lõi

Đây là 7 câu hỏi tôi luôn đặt ra khi bắt đầu thiết kế bất kỳ hệ thống nào. Thứ tự có ý nghĩa — nó đi từ bối cảnh tổng thể đến chi tiết kỹ thuật.

1. Quy Mô (Scale): Hệ Thống Sẽ Lớn Đến Đâu?

Đây là câu hỏi đầu tiên vì một lý do đơn giản: **kiến trúc** (architecture) **cho 1.000 người dùng hoàn toàn khác kiến trúc cho 10 triệu người dùng**.

Các câu hỏi cụ thể:

- Hiện tại có bao nhiêu người dùng?
- Mức tăng trưởng dự kiến trong 1-2 năm tới?
- Số người dùng đồng thời vào cao điểm (peak)?
- Số lượng giao dịch (transactions)/yêu cầu (requests) mỗi ngày?

Tại sao quan trọng: Quy mô quyết định các lựa chọn công nghệ. Với 1.000 người dùng, một hệ thống nguyên khối (monolith) + MySQL là quá đủ. Với 10 triệu người dùng, bạn cần vi dịch vụ (microservices), các lớp bộ nhớ đệm (caching layers), phân mảnh dữ liệu (database sharding).

Ví dụ thực tế: Một công ty khởi nghiệp (startup) với 5.000 người dùng không cần Kubernetes và bộ nhớ đệm phân tán (distributed cache). Nhưng nếu họ kỳ vọng tăng trưởng gấp 50 lần trong năm tới, bạn cần thiết kế với dư địa (headroom).

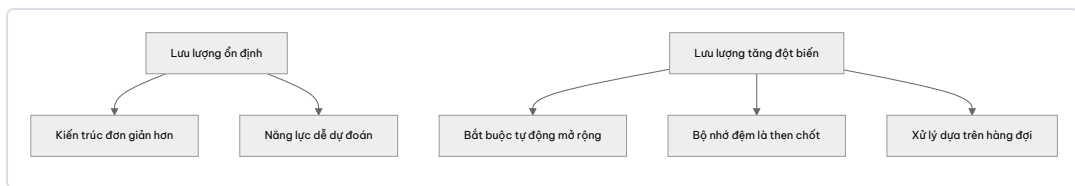
2. Mô Hình Lưu Lượng (Traffic Pattern): Người Dùng Hoạt Động Như Thế Nào?

Quy mô chỉ cho bạn con số. Mô hình lưu lượng cho bạn biết **hành vi**.

Các câu hỏi cụ thể:

- Lưu lượng (Traffic) đều đặn hay có tăng đột biến (spike)?
- Những giờ cao điểm (peak hours) nào trong ngày?
- Có quy luật theo mùa không? (Black Friday, Tết)
- Thiên về đọc (Read-heavy) hay thiên về ghi (write-heavy)?
- Tỷ lệ giữa lượt đọc và lượt ghi?

Tại sao quan trọng: Mô hình lưu lượng ảnh hưởng đến chiến lược lưu đệm (caching strategy), chính sách tự động mở rộng (auto-scaling policies) và thiết kế cơ sở dữ liệu (database).



Ví dụ: Một hệ thống thương mại điện tử có lưu lượng tăng đột biến vào 12 giờ trưa và 8 giờ tối. Bạn không thể cấp phát tài nguyên cho tải (load) cao điểm suốt 24/7 — quá tốn kém. Bạn cần tự động mở rộng (auto-scaling) + lưu đệm (caching) mạnh tay.

Ngược lại, một công cụ nội bộ với lưu lượng ổn định có thể dùng năng lực cố định, đơn giản hơn.

3. Kích Thước Dữ Liệu: Bao Nhiêu Dữ Liệu Và Tăng Nhanh Đến Đâu?

Đây là câu hỏi về lưu trữ (storage) và vòng đời dữ liệu (data lifecycle).

Các câu hỏi cụ thể:

- Tổng kích thước dữ liệu hiện tại?
- Tốc độ tăng trưởng dữ liệu? (GB/ngày, TB/tháng)
- Kích thước trung bình mỗi bản ghi?
- Chính sách lưu giữ dữ liệu (Data retention policy)? (giữ mãi hay xóa sau X tháng?)
- Tỷ lệ dữ liệu nóng (hot data) so với dữ liệu lạnh (cold data)?

Tại sao quan trọng: Kích thước dữ liệu quyết định lựa chọn cơ sở dữ liệu, chiến lược lưu trữ (storage strategy) và chính sách lưu trữ dài hạn (archival policies).

Ví dụ thực tế: Một nền tảng phân tích sinh ra 100GB log mỗi ngày. Sau 1 năm là 36TB. Bạn không thể truy vấn (query) trực tiếp trên 36TB. Bạn cần:

- Lưu trữ nóng (Hot storage) cho 30 ngày gần nhất (SSD tốc độ cao)
- Lưu trữ lạnh (Cold storage) cho dữ liệu lịch sử (S3/Glacier)
- Tổng hợp dữ liệu (Data aggregation) và lấy mẫu (sampling)

4. Tính Sẵn Sàng (Availability): Thời Gian Ngừng Hoạt Động (Downtime) Có Chấp Nhận Được Không?

Tính sẵn sàng là chuyện **thời gian ngừng hoạt động chấp nhận được**.

Các câu hỏi cụ thể:

- Hệ thống có được phép ngừng hoạt động không?
- Thời gian ngừng hoạt động chấp nhận được: phút, giờ, hay giây?
- Cần thời gian hoạt động (*uptime*) 99,9% hay 99,99%?
- Tác động của việc ngừng hoạt động? (mất doanh thu, ảnh hưởng người dùng)
- Có chấp nhận khung giờ bảo trì (*maintenance window*) không?

Tại sao quan trọng: Yêu cầu về tính sẵn sàng chi phối độ phức tạp (*complexity*) của kiến trúc và chi phí (*cost*).

Uptime 99.9%	= ~8.7 giờ ngừng/năm	→ một máy chủ là ổn
Uptime 99.99%	= ~52 phút ngừng/năm	→ cần dự phòng
Uptime 99.999%	= ~5 phút ngừng/năm	→ đa vùng + chuyên đôi dự phòng phức tạp

Ví dụ: Công cụ nội bộ có thể ngừng 2-3 giờ để bảo trì lúc đêm khuya? Thiết kế đơn giản. Hệ thống thanh toán không được ngừng dù chỉ 1 giây? Cần đa vùng chủ động – chủ động (*multi-region active-active*).

Đánh đổi (*Trade-off*): Mỗi số "9" thêm vào tính sẵn sàng làm độ phức tạp và chi phí tăng theo cấp số nhân.

5. Tính Nhất Quán (*Consistency*): Dữ Liệu Phải Đúng Ngay Lập Tức?

Đây là câu hỏi về **tính đúng đắn và đồng bộ hoá** (*synchronization*) dữ liệu.

Các câu hỏi cụ thể:

- Có cần nhất quán mạnh (*strong consistency*) không?
- Nhất quán cuối cùng (*Eventual consistency*) có chấp nhận được không?
- Dữ liệu cũ (*Stale data*) trong vài giây có ổn không?
- Chiến lược giải quyết xung đột (*conflict resolution*)?
- Cần đảm bảo giao dịch (*transaction*) đến mức nào?

Tại sao quan trọng: Yêu cầu về tính nhất quán ảnh hưởng đến lựa chọn cơ sở dữ liệu và các mẫu kiến trúc (*architecture patterns*).

Ví dụ:

- **Cần nhất quán mạnh:** Giao dịch ngân hàng, hệ thống quản lý tồn kho

- **Nhất quán cuối cùng là đủ:** Bảng tin (*feeds*) mạng xã hội, bảng điều khiển (*dashboards*) phân tích, số lượt xem

Một đánh đổi kinh điển: Bạn không thể có cả nhất quán mạnh VÀ tính sẵn sàng cao (*high availability*) VÀ khả năng chịu phân vùng (*partition tolerance*) (định lý CAP). Phải chọn.

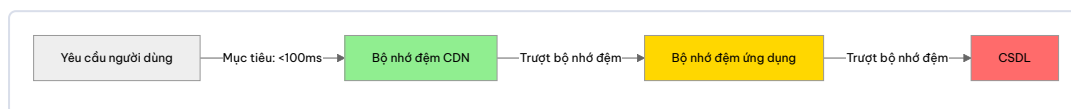
6. Độ Trễ (*Latency*): Nhanh Đến Mức Nào?

Độ trễ là chuyện **trải nghiệm người dùng và thời gian phản hồi** (*response time*).

Các câu hỏi cụ thể:

- Thời gian phản hồi chấp nhận được? (ms, giây)
- Yêu cầu độ trễ P50, P95, P99?
- Kỳ vọng về trải nghiệm người dùng?
- Cần thời gian thực hay xử lý theo lô (*batch processing*) là được?
- Người dùng phân bố địa lý thế nào?

Tại sao quan trọng: Yêu cầu về độ trễ chi phối chiến lược lưu đệm, việc đánh chỉ mục (*indexing*) cơ sở dữ liệu và việc dùng CDN.



Ví dụ:

- Công cụ tìm kiếm: <100ms là then chốt cho trải nghiệm người dùng
- Tạo báo cáo theo lô: vài phút là ổn
- Chat thời gian thực: kỳ vọng <500ms
- Bảng điều khiển (*Dashboard*) phân tích: 2-3 giây là chấp nhận được

Độ trễ P99 quan trọng hơn trung bình. Nếu 99% yêu cầu nhanh nhưng 1% chậm, trải nghiệm người dùng vẫn tệ.

7. Chi Phí: Ngân Sách Và Ràng Buộc (*Constraints*) Là Gì?

Cuối cùng nhưng cực kỳ quan trọng: **các ràng buộc về chi phí.**

Các câu hỏi cụ thể:

- Ngân sách hạ tầng (*infrastructure*) hằng tháng?
- Chi phí trên mỗi người dùng ở mức nào là chấp nhận được?
- Đánh đổi (*Trade-offs*) giữa tự xây hay mua sẵn?
- Quy mô nhóm (*team*) và chuyên môn?
- Chi phí vận hành (bảo trì, giám sát (*monitoring*))?

Tại sao quan trọng: Giải pháp kỹ thuật tốt nhất cũng chẳng có ý nghĩa nếu vượt ngân sách hoặc nhóm không bảo trì nổi.

Các kịch bản thực tế:

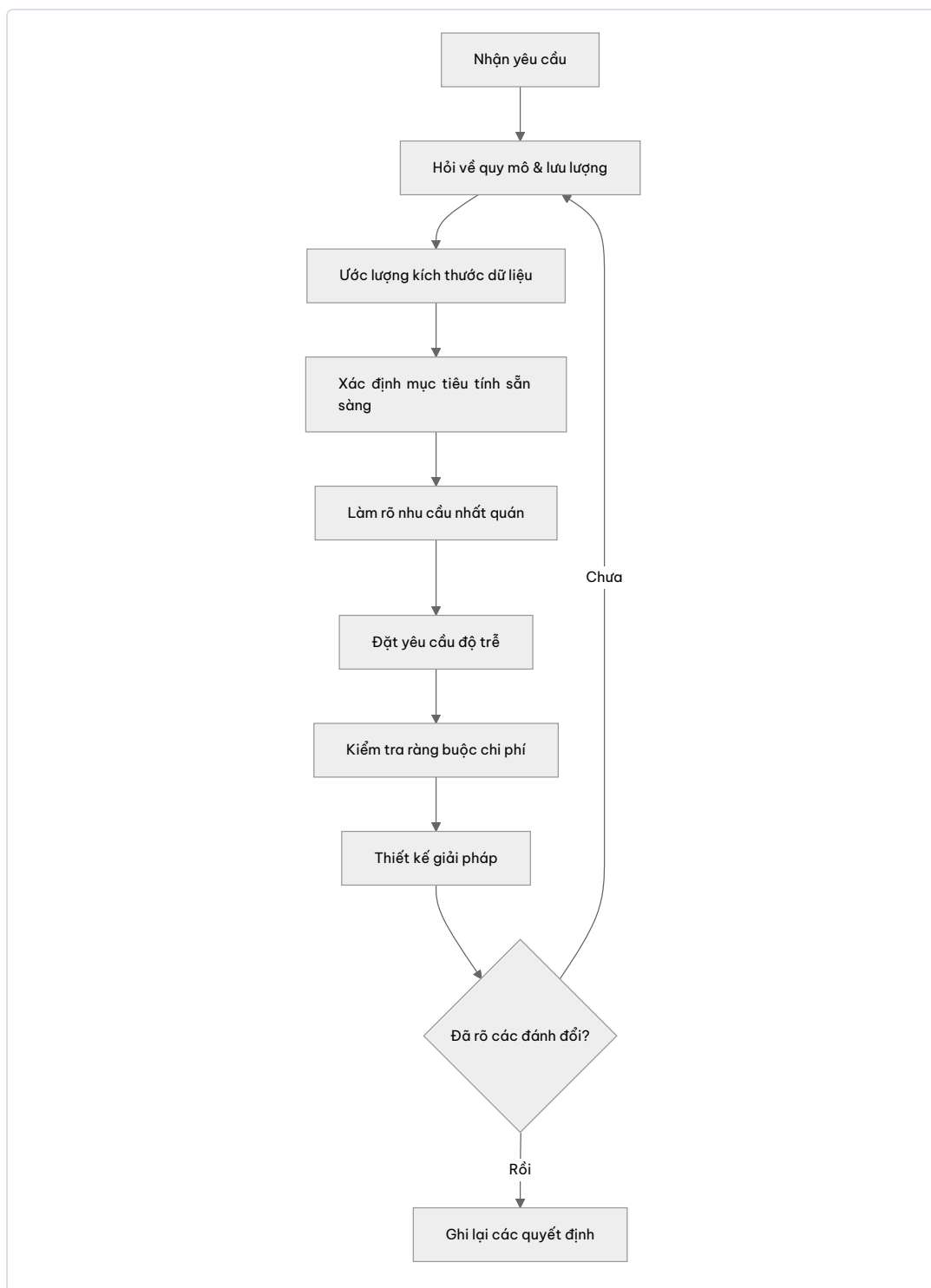
- Công ty khởi nghiệp với ngân sách hạn chế: tối ưu cho chi phí, dùng dịch vụ được quản lý (*managed services*)
- Doanh nghiệp lớn có yêu cầu tuân thủ: có thể chi nhiều hơn cho bảo mật và độ tin cậy (*reliability*)
- Công ty tăng trưởng nhanh: tối ưu cho tốc độ ra thị trường, chi phí là thứ yếu

Chi phí không chỉ là hạ tầng. Nó gồm:

- Thời gian kỹ thuật (xây dựng và bảo trì)
- Dịch vụ của bên thứ ba
- Hạ tầng (máy chủ (*servers*), cơ sở dữ liệu (*databases*), lưu trữ)
- Chi phí phụ (*Overhead*) vận hành (giám sát, trực sự cố (*on-call*))

Khung Trong Thực Tế

Khi thiết kế một hệ thống, tôi làm theo quy trình này:



Quy trình cụ thể:

1. **Lắng nghe trước** — nghe yêu cầu (*requirement*) ban đầu
2. **Hỏi có hệ thống** — đi qua 7 câu hỏi theo thứ tự
3. **Làm rõ chỗ mơ hồ** — đừng giả định, hãy hỏi cho rõ
4. **Ghi lại câu trả lời** — viết ra, đừng dựa vào trí nhớ
5. **Nhận diện mâu thuẫn** — phát hiện các điểm mâu thuẫn từ sớm
6. **Thảo luận về đánh đổi** — minh bạch về các giới hạn

Những Sai Lầm Thường Gặp Khi Đặt Câu Hỏi

Qua kinh nghiệm, đây là những lỗi phổ biến:

1. Hỏi chưa đủ sâu

✗ "Quy mô bao nhiêu?" — quá mơ hồ

✓ "Bao nhiêu người dùng đồng thời vào giờ cao điểm (*peak hour*)? Tốc độ tăng trưởng dự kiến?"

2. Chấp nhận câu trả lời mơ hồ

✗ Người dùng: "Cần tính sẵn sàng cao (*highly available*)"

✓ Hỏi tiếp: "Tính sẵn sàng cao nghĩa là chấp nhận thời gian ngừng hoạt động bao nhiêu? 1 giờ/năm hay 1 phút/năm?"

3. Không phản biện những yêu cầu phi thực tế

Nếu người dùng muốn: nhất quán mạnh + tính sẵn sàng 99,999% + độ trễ <10ms + ngân sách \$100/tháng → không thực tế. Bạn phải chỉ ra và thảo luận về các đánh đổi.

4. Quên hỏi về các ràng buộc

Đừng chỉ hỏi về mục tiêu. Hãy hỏi về các giới hạn: ngân sách, quy mô nhóm, yêu cầu tuân thủ, hệ thống cũ (*legacy systems*).

Ví Dụ Thực Hành: Thiết Kế Dịch Vụ Rút Gọn URL (*URL Shortener*)

Áp dụng khung:

Quy mô:

- 100 triệu URL được tạo mỗi tháng
- 1 tỷ lượt nhấp mỗi tháng (tỷ lệ đọc-ghi 10:1)

Mô hình lưu lượng:

- Lưu lượng ổn định, tăng nhẹ trong giờ hành chính
- Thiên về đọc (lượt nhấp >> lượt tạo)

Kích thước dữ liệu:

- ~10 byte cho mỗi URL rút gọn
- 100 triệu × 10 byte = 1GB/tháng
- Mức tăng không đáng kể

Tính sẵn sàng:

- 99,9% là ổn (ngừng hoạt động ngắn có thể chấp nhận)
- Không phải hệ thống sống còn

Tính nhất quán:

- Nhất quán cuối cùng là đủ cho phần phân tích
- Nhất quán mạnh cho việc tạo URL (không trùng lặp)

Độ trễ:

- Chuyển hướng: <100ms là then chốt
- Tạo mới: <500ms là ổn

Chi phí:

- Ngân sách công ty khởi nghiệp: tối ưu cho chi phí
- Ưu tiên dịch vụ được quản lý

Các quyết định dựa trên câu trả lời:

- Một cơ sở dữ liệu duy nhất là đủ (dữ liệu 1GB rất nhỏ)
 - Lưu đệm mạnh tay cho lượt đọc (tỷ lệ 10:1)
 - Kiến trúc đơn giản (tính sẵn sàng 99,9% là đủ)
 - Dùng cơ sở dữ liệu được quản lý (*managed database*) (tối ưu chi phí, nhóm nhỏ)
-

Những Điểm Cốt Lõi

Khung Câu Hỏi Hệ Thống không phải là công thức thần kỳ. Nó là **công cụ tư duy** giúp bạn định hướng cuộc trao đổi và không bỏ sót những yếu tố quan trọng.

Hãy nhớ:

Luôn bắt đầu bằng câu hỏi — đừng nhảy ngay vào giải pháp

Hỏi có hệ thống — làm theo danh sách kiểm tra để không bỏ sót gì

Làm rõ những câu trả lời mơ hồ — đòi hỏi những con số cụ thể

Ghi lại mọi thứ — viết ra để tham khảo về sau

Nhận diện đánh đổi từ sớm — mâu thuẫn thường lộ ra từ chính các câu trả lời

Thực tế — giải pháp hoàn hảo không tồn tại, chỉ có giải pháp đủ tốt trong các ràng buộc hiện có

Khung này không đảm bảo một thiết kế hoàn hảo. Nhưng nó đảm bảo bạn **đã đặt đúng câu hỏi** trước khi thiết kế — và đó là điểm khởi đầu quan trọng nhất.

Lần tới khi thiết kế hệ thống, hãy lấy danh sách kiểm tra này ra. Đi qua từng câu hỏi một. Bạn sẽ bất ngờ về số giả định (*assumption*) mình đã tránh được và số đánh đổi mình nhận diện được từ sớm.

Đó chính là sức mạnh của những câu hỏi hay.

Tổng Kết Mô Hình Tư Duy (*Mental Model*) — Củng Cố Tư Duy Thiết Kế Hệ Thống

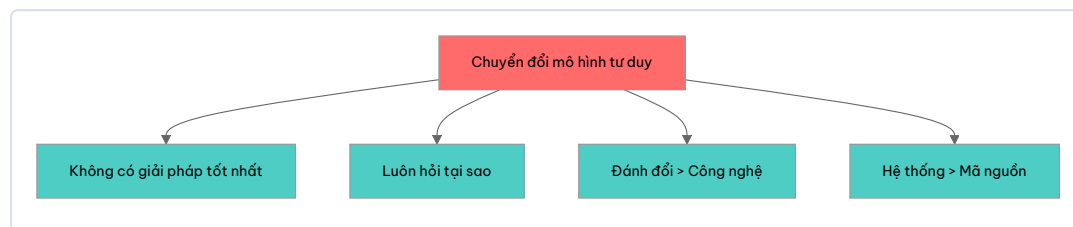
Chúng ta đã đi qua 7 bài học trong Phần 0. Đây không phải là phần dạy công nghệ hay mẫu thiết kế (*design patterns*). Đây là phần **thay đổi cách bạn tư duy**.

Trước khi chuyển sang Phần 1, tôi muốn dừng lại và tổng kết những bước chuyển tư duy quan trọng nhất. Những bước chuyển này sẽ theo bạn suốt hành trình từ đây trở đi.

Hãy nghĩ về nó như nền móng của một toà nhà. Bạn có thể học hết các công cụ, mẫu thiết kế (*patterns*), công nghệ — nhưng nếu nền móng sai, mọi thứ khác sẽ chông chênh.

Bốn Tư Duy (*Mindsets*) Cốt Lõi

Phần 0 xoay quanh 4 bước chuyển tư duy cốt lõi. Đây không phải danh sách kiểm tra (*checklist*) để học thuộc. Đây là những **lăng kính** để nhìn nhận vấn đề.



Hãy đi sâu vào từng tư duy.

1. Không Có "Giải Pháp Tốt Nhất"

Đây là bước chuyển khó chấp nhận nhất, đặc biệt khi bạn vừa chuyển mình từ vai trò lập trình viên.

Tư duy cũ:

"Có một giải pháp tốt nhất. Tôi cần tìm ra nó."

Tư duy mới:

"Mọi giải pháp đều có sự đánh đổi (*trade-offs*). Tôi cần tìm giải pháp phù hợp nhất với bối cảnh này."

Tại Sao Tư Duy Này Quan Trọng?

Khi bạn tin rằng có "giải pháp tốt nhất", bạn sẽ:

- Lãng phí thời gian tìm kiếm sự hoàn hảo
- Sao chép những giải pháp không phù hợp với bối cảnh
- Cứng nhắc khi yêu cầu thay đổi
- Bỏ lỡ cơ hội tối ưu cho những nhu cầu cụ thể

Khi bạn chấp nhận "không có giải pháp tốt nhất", bạn sẽ:

- Tập trung vào việc hiểu các ràng buộc (*constraints*)
- Thiết kế cho bối cảnh hiện tại
- Linh hoạt khi cần thích ứng
- Đưa ra các sự đánh đổi một cách có ý thức

Ví Dụ Thực Tế

Câu hỏi: "Cơ sở dữ liệu (*Database*) nào tốt nhất cho ứng dụng của tôi?"

Cách tiếp cận sai: "PostgreSQL là tốt nhất vì X, Y, Z"

Cách tiếp cận đúng:

"Tuỳ vào nhu cầu của bạn:

- Dữ liệu quan hệ cần ACID? → PostgreSQL
- Dữ liệu dạng tài liệu với lược đồ (*schema*) linh hoạt? → MongoDB
- Ghi với thông lượng (*throughput*) cao? → Cassandra
- Lưu trữ khoá-giá trị đơn giản? → Redis

Mỗi lựa chọn đều có sự đánh đổi. Lựa chọn tốt nhất phụ thuộc vào các ràng buộc CỦA BẠN."

2. Luôn Hỏi "Tại Sao"

Bước chuyển tư duy này nói về việc **đặt câu hỏi cho mọi thứ** — không chấp nhận câu trả lời theo bề nổi.

Tư duy cũ:

"Kỹ sư lâu năm (*Senior engineer*) nói dùng vi dịch vụ (*microservices*) → tôi dùng vi dịch vụ"

Tư duy mới:

"Tại sao lại là vi dịch vụ? Nó giải quyết vấn đề gì? Đánh đổi những gì? Có phương án thay thế nào không?"

Vì Sao Câu Hỏi "Tại Sao" Quan Trọng

Trong thiết kế hệ thống, mọi quyết định đều có hiệu ứng lan toả. Nếu bạn không hiểu **lập luận** đằng sau các quyết định, bạn sẽ:

- Lặp lại sai lầm của người khác
- Không thích ứng được khi bối cảnh thay đổi
- Đưa ra các quyết định thiếu nhất quán
- Không bảo vệ được lựa chọn của mình

Kỹ Thuật "Năm Lần Tại Sao"

Khi ai đó đề xuất một giải pháp, hãy đào sâu:

```
"Chúng ta nên dùng Redis."  
→ Tại sao lại là Redis?  
  
"Đề'luu đệm dữ liệu."  
→ Tại sao cần lưu đệm? Vấn đề'hiệu năng là gì?  
  
"Thời gian phản hồi API khoảng 2 giây."  
→ Tại sao chậm? Truy vấn CSDL chậm à?  
  
"Đúng, truy vấn quét toàn bộ bảng."  
→ Tại sao không thêm chỉ mục trước?  
  
"À... chưa thu'thêm chỉ mục."
```

Thường thì sau 2-3 lần hỏi "tại sao", bạn sẽ phát hiện ra một giải pháp đơn giản hơn.

Áp Dụng Cho Mọi Thứ

- Tại sao dùng vi dịch vụ thay vì hệ thống nguyên khối (*monolith*)?
- Tại sao dùng NoSQL thay vì SQL?
- Tại sao dùng Kubernetes thay vì các máy ảo (VM) đơn giản?
- Tại sao dùng kiến trúc hướng sự kiện (*event-driven*) thay vì yêu cầu-phản hồi (*request-response*)?
- Tại sao phải lưu đệm dữ liệu này?
- Tại sao phải tách dịch vụ này?

Không có câu hỏi nào là "ngớ ngẩn". Mọi câu hỏi "tại sao" đều chính đáng.

3. Sự Đánh Đổi Quan Trọng Hơn Công Nghệ

Đây là bước chuyển từ tư duy **lấy công nghệ làm trung tâm** (*technology-first*) sang **tư duy lấy bài toán làm trung tâm** (*problem-first*).

Tư duy cũ:

"Công nghệ X đang hot và mới. Tôi muốn dùng X."

Tư duy mới:

"Vấn đề là gì? Những sự đánh đổi nào quan trọng? Công nghệ nào phù hợp nhất với những sự đánh đổi này?"

Vì Sao Điều Này Quan Trọng

Công nghệ chỉ là công cụ. Chọn công nghệ mà KHÔNG hiểu sự đánh đổi cũng giống như:

- Mua Ferrari để đi chợ hàng ngày
- Dùng búa tạ để đóng đinh
- Thiết kế thừa (*Over-engineer*) cho những vấn đề đơn giản

Kiến trúc sư giỏi nghĩ về sự đánh đổi TRƯỚC, công nghệ SAU.

Các Sự Đánh Đổi Phổ Biến

Mọi quyết định thiết kế đều liên quan đến sự đánh đổi:

Tính nhất quán (*Consistency*) và **Tính sẵn sàng** (*Availability*):

- Nhất quán mạnh (*Strong consistency*) → có thể phải hy sinh tính sẵn sàng
- Tính sẵn sàng cao (*High availability*) → có thể phải chấp nhận nhất quán cuối cùng (*eventual consistency*)

Hiệu năng (*Performance*) **và Chi phí** (*Cost*):

- Hiệu năng tốt hơn → hạ tầng (*infrastructure*) đắt hơn
- Chi phí thấp hơn → chấp nhận thời gian phản hồi chậm hơn

Đơn giản và Linh hoạt:

- Kiến trúc đơn giản → khó mở rộng thêm tính năng hơn
- Kiến trúc linh hoạt → độ phức tạp cao hơn ngay từ đầu

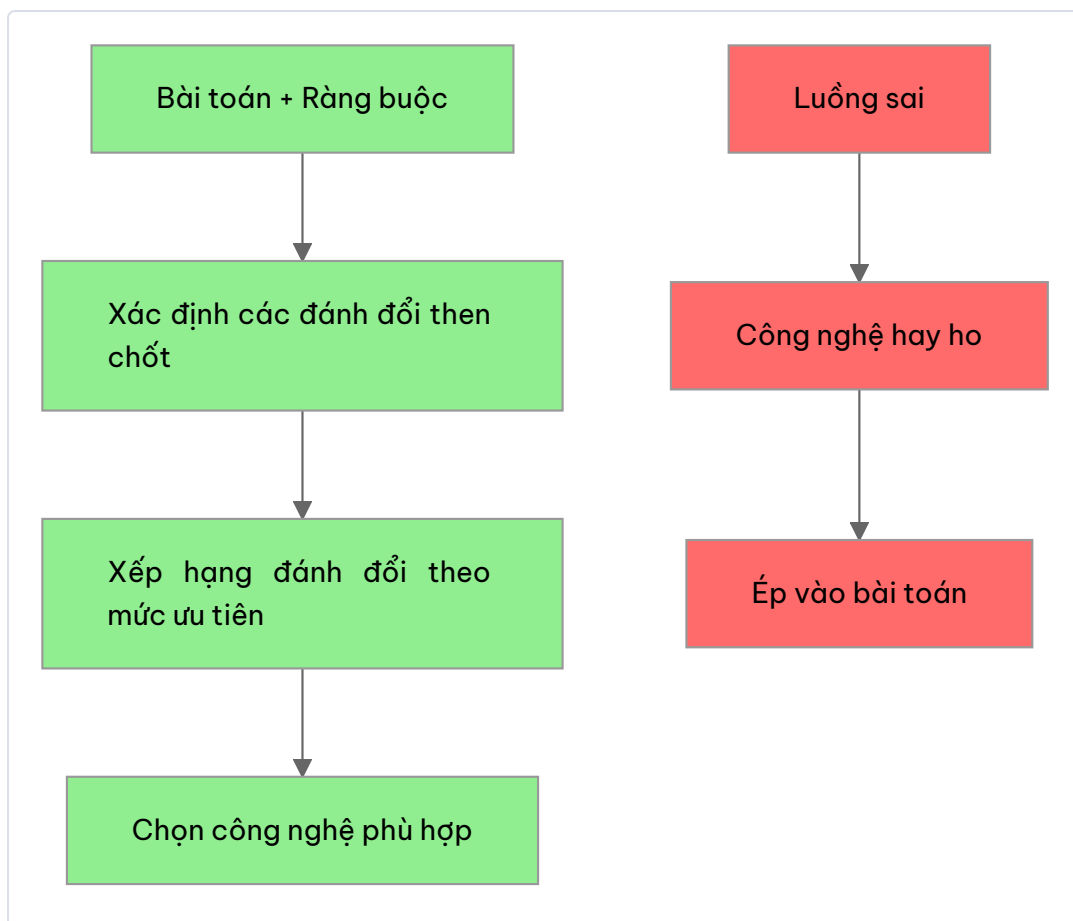
Tự xây dựng hay Mua:

- Tự xây dựng → toàn quyền kiểm soát, chi phí cao hơn
- Dùng dịch vụ được quản lý (*managed services*) → ít quyền kiểm soát hơn, giao hàng nhanh hơn

Tư Duy Theo Khung (*Framework*)

Thay vì nghĩ "dùng công nghệ nào?", hãy nghĩ:

1. Sự đánh đổi (*Trade-off*) nào là then chốt cho bài toán này?
2. Sự đánh đổi nào có thể chấp nhận được?
3. Công nghệ nào hỗ trợ những sự đánh đổi đó?



4. Tư Duy Hệ Thống (System Thinking) Đặt Trên Tư Duy Mã Nguồn

Đây là bước chuyển lớn nhất — từ **cấp độ vi mô** (mã nguồn) lên **cấp độ vĩ mô** (hệ thống).

Tư duy cũ:

"Tôi viết mã sạch (*clean code*), tối ưu thuật toán, tuân theo các thực hành tốt nhất (*best practices*). Thế là đủ."

Tư duy mới:

"Mã nguồn chỉ là một phần. Tôi cần nghĩ về: tương tác giữa các thành phần (*components*), luồng dữ liệu (*data flow*), các kiểu sự cố (*failure modes*), mở rộng (*scaling*), giám sát (*monitoring*), vận hành."

Vì Sao Bước Chuyển Này Khó

Lập trình viên dành nhiều năm để tối ưu mã nguồn:

- Kiến trúc sạch (*Clean architecture*)
- Mẫu thiết kế
- Tối ưu thuật toán
- Chất lượng mã nguồn

Đột nhiên, thiết kế hệ thống lại nói: "Chất lượng mã nguồn kém quan trọng hơn chất lượng hệ thống."

Điều này không có nghĩa mã nguồn không quan trọng. Nhưng **mã tốt trong một kiến trúc tồi vẫn thất bại.**

Tư Duy Hệ Thống Là Gì?

Tư duy hệ thống nghĩa là nhìn vào **bức tranh tổng thể**, không chỉ từng phần riêng lẻ:

Tư duy mã nguồn:

- Hàm này có độ phức tạp $O(n^2)$ → tối ưu xuống $O(n \log n)$
- Lớp này vi phạm SOLID → tái cấu trúc (*refactor*)

Tư duy hệ thống:

- Dịch vụ này trở thành điểm nghẽn (*bottleneck*) khi ở quy mô lớn → cần tách ra hay mở rộng?
- Cơ sở dữ liệu này là điểm lỗi duy nhất (*single point of failure*) → cần nhân bản (*replication*)?
- Độ trễ mạng (*Network latency*) giữa các dịch vụ → có nên gộp các dịch vụ lại?
- Sự cố (*Failure*) ở thành phần A ảnh hưởng đến thành phần B như thế nào?

Ví Dụ: Xử Lý Thanh Toán

Tư duy ở cấp độ mã nguồn:

```
PaymentService.process(payment):  
- Validate payment data  
- Call payment gateway  
- Update database  
- Return response
```

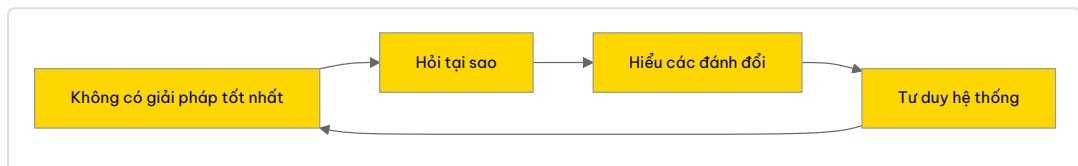
Tư duy ở cấp độ hệ thống:

1. Nếu công thanh toán hết thời gian chờ thì sao?
→ Thu' lại? Hàng đợi? Tính lũy thừa?
2. Nếu CSDL sập đúng lúc cập nhật thì sao?
→ Đã trừ tiền khách hàng nhưng chưa lưu bản ghi?
3. Xử lý tải cao như thế nào?
→ Xử lý dựa trên hàng đợi? Giới hạn tần suất?
4. Giám sát sự cố như thế nào?
→ Số liệu đo? Cảnh báo? Bảng điều khiển?
5. Gỡ lỗi trong môi trường thực tế như thế nào?
→ Ghi log? Truy vết? Mã tương quan (correlation ID)?

Tư duy mã nguồn tập trung vào logic. Tư duy hệ thống tập trung vào **độ tin cậy** (reliability), **khả năng mở rộng** (scalability), **khả năng quan sát** (observability).

Kết Nối 4 Tư Duy

4 tư duy này không tách biệt. Chúng liên kết chặt chẽ với nhau:



Dòng chảy tự nhiên:

1. Chấp nhận **không có giải pháp tốt nhất** → bạn trở nên cởi mở
2. **Hỏi tại sao** → bạn hiểu được lập luận
3. Tập trung vào **sự đánh đổi** → bạn đưa ra quyết định có cơ sở
4. Áp dụng **tư duy hệ thống** → bạn nhìn thấy bức tranh tổng thể

Mỗi tư duy củng cố cho những tư duy còn lại.

Từ Tư Duy Đến Thực Hành

Được rồi, bạn đã hiểu 4 tư duy. Giờ làm sao để áp dụng?

Thực Hành Hằng Ngày

1. Xem Lại Các Quyết Định Trong Quá Khứ

Chọn một quyết định kiến trúc trong dự án của bạn. Hãy hỏi:

- Tại sao lại có quyết định đó?
- Đã đánh đổi những gì?
- Nhìn lại bây giờ, quyết định đó còn đúng không?
- Học được gì từ nó?

2. Tự Thử Thách Bản Thân

Mỗi khi bạn muốn dùng một công nghệ:

- Viết ra 3 lý do tại sao
- Viết ra 2 phương án thay thế
- So sánh các sự đánh đổi
- Biện minh cho lựa chọn của mình

3. Tư Duy Theo Hệ Thống

Khi gỡ lỗi (*debug*) một sự cố trong môi trường vận hành thực tế (*production*):

- Đừng chỉ sửa lỗi
- Hãy hỏi: tại sao lỗi này xảy ra?
- Thiết kế hệ thống nào sẽ ngăn được nó?
- Có sự cố tương tự nào khác không?

4. Học Hỏi Từ Người Khác

Đọc các blog về kiến trúc, các nghiên cứu tình huống (*case studies*):

- Không sao chép giải pháp
- Tập trung vào lập luận
- Hiểu các ràng buộc của họ

- Ghi lại những sự đánh đổi mà họ đã chọn

Những Dấu Hiệu Cảnh Báo Cần Để Ý

Dấu hiệu cho thấy bạn chưa thực sự thấm nhuần các tư duy này:

"Thực hành tốt nhất (*Best practice*) là X, nên tôi dùng X"

→ Thiếu: bối cảnh và sự đánh đổi

"Kỹ sư lâu năm nói như vậy"

→ Thiếu: hỏi tại sao

"Công nghệ Y đang hot, tôi muốn học"

→ Thiếu: tư duy lấy bài toán làm trung tâm

"Đoạn mã này sạch và nhanh"

→ Thiếu: các mối quan tâm ở cấp độ hệ thống

Khi bạn bắt gặp bản thân đang nghĩ như vậy — hãy dừng lại và áp dụng khung tư duy.

Những Điểm Cốt Lõi

Nếu chỉ nhớ 4 điều từ Phần 0:

1. Không Có Giải Pháp Tốt Nhất

→ Bối cảnh quyết định thế nào là "tốt nhất". Tập trung vào sự phù hợp, không phải sự hoàn hảo.

2. Luôn Hỏi Tại Sao

→ Hiểu lập luận > ghi nhớ giải pháp.

3. Sự Đánh Đổi > Công Nghệ

→ Chọn dựa trên ràng buộc, không phải theo xu hướng.

4. Hệ Thống > Mã Nguồn

→ Mã tốt trong một kiến trúc tồi vẫn thất bại.

Lời Kết

Thay đổi tư duy không xảy ra sau một đêm. Tôi mất 2 năm để thực sự thấm nhuần những nguyên tắc này.

Bạn sẽ bắt gặp bản thân suy nghĩ theo lối cũ. Điều đó là bình thường. Mỗi lần như vậy, hãy chủ động áp dụng tư duy mới. Theo thời gian, nó sẽ trở thành phản xạ tự nhiên.

Hãy nhớ: **Thiết kế hệ thống không phải là biết hết mọi công nghệ. Đó là tư duy rõ ràng về sự đánh đổi trong những bối cảnh phức tạp.**

PHẦN 1

Nền Tảng: Tư Duy Theo Hệ Thống

Nền tảng tư duy hệ thống để chuyển từ hiểu khái niệm sang phân tích hệ thống thực tế.

1) Giới Thiệu Phần

Phần 1 là nền tảng bắt buộc trước khi đi sâu vào hệ thống phân tán (*distributed systems*) và kiến trúc quy mô lớn.

Mục tiêu của phần này là giúp bạn nhìn hệ thống như một dòng chảy hoàn chỉnh: từ yêu cầu (*request*) đầu vào, qua các thành phần xử lý, đến dữ liệu và phản hồi cuối cùng.

Khi học xong, bạn không chỉ "biết thuật ngữ" mà còn có thể phân tích hệ thống theo tư duy của một kỹ sư thiết kế.

2) Mục Tiêu Học Tập

- Hiểu vai trò của từng thành phần trong một hệ thống web.
- Phân biệt khi nào nên dùng đồng bộ (*sync*), khi nào nên dùng bất đồng bộ (*async*).
- Theo dõi luồng dữ liệu (*data flow*) để tìm chính xác điểm nghẽn (*bottleneck*).
- Đọc và đánh giá hệ thống bằng các chỉ số nền tảng: độ trễ (*latency*), thông lượng (*throughput*), tính sẵn sàng (*availability*).
- Nhận diện ràng buộc (*constraint*) và sự đánh đổi (*trade-off*) khi ra quyết định kiến trúc.

3) Bạn Sẽ Học Gì Trong Phần Này

- **Các thành phần hệ thống:** máy khách (*client*), API, dịch vụ (*service*), cơ sở dữ liệu (*database*), bộ nhớ đệm (*cache*), hàng đợi (*queue*) và mối quan hệ giữa chúng.
- **Các mẫu giao tiếp** (*Communication patterns*): cách các dịch vụ giao tiếp đồng bộ và bất đồng bộ trong bối cảnh thực tế.
- **Luồng dữ liệu:** yêu cầu đi qua hệ thống như thế nào và điểm nghẽn thường xuất hiện ở đâu.
- **Điểm nghẽn** (*Bottlenecks*): cách phát hiện, ưu tiên và xử lý điểm nghẽn theo mức độ ảnh hưởng.

- **Nền tảng về hiệu năng** (*performance*): đọc đúng các chỉ số quan trọng thay vì tối ưu theo cảm tính.
- **Ràng buộc** (*Constraints*) & **đánh đổi** (*trade-offs*): vì sao mỗi quyết định luôn có mặt được và mặt mất.

4) Tổng Quan Các Bài Học

Bài 1 – Các Thành Phần Trong Một Hệ Thống Web

Hiểu kiến trúc cơ bản của hệ thống web hiện đại và trách nhiệm của từng thành phần.

Bài 2 – Các Mẫu Giao Tiếp – Đồng Bộ Và Bất Đồng Bộ Trong Hệ Thống Phân Tán

Nắm cách chọn mô hình giao tiếp phù hợp theo yêu cầu về độ trễ, độ tin cậy (*reliability*) và độ phức tạp (*complexity*).

Bài 3 – Luồng Dữ Liệu & Điểm Nghẽn – Tìm Và Tối Ưu Điểm Nghẽn

Phân tích đường đi của dữ liệu để xác định đúng điểm nghẽn và đưa ra hướng tối ưu thực dụng.

Bài 4 – Độ Trễ, Thông Lượng, Tính Sẵn Sàng Và Định Lý CAP

Xây nền tảng tư duy về hiệu năng, tính sẵn sàng và các giới hạn cốt lõi trong hệ phân tán (*distributed system*).

5) Kết Quả Sau Khi Hoàn Thành Phần 1

Bạn có thể:

- Phân tích một hệ thống ở mức kiến trúc cơ bản.
- Đặt câu hỏi đúng về quy mô (*scale*), hiệu năng và độ ổn định.
- Giải thích rõ quyết định chọn đồng bộ/bất đồng bộ theo bối cảnh.
- Nhìn thấy điểm nghẽn trước khi tối ưu.

- Thảo luận về sự đánh đổi bằng tư duy hệ thống thay vì tư duy từng tính năng đơn lẻ.

6) Chuẩn Bị Cho Phần 2

Sau Phần 1, bạn đã có nền tư duy đủ vững để bước vào **Phần 2 — Các Khối Xây Dựng Cốt Lõi**.

Ở phần tiếp theo, bạn sẽ đi sâu vào từng khối cốt lõi như cân bằng tải (*load balancing*), lưu đệm (*caching*), mở rộng cơ sở dữ liệu (*database scaling*) và hàng đợi thông điệp (*message queue*), với trọng tâm là cách dùng đúng theo bài toán thực tế.

Các Thành Phần Trong Một Hệ Thống Web

Chào mừng bạn đến với Phần 1 — Nền Tảng: Tư Duy Theo Hệ Thống.

Sau Phần 0, bạn đã thay đổi tư duy. Giờ là lúc xây dựng nền tảng thực sự: **Hiểu các khối xây dựng** (*building blocks*) **của mọi hệ thống**.

Tôi còn nhớ lần đầu gỡ lỗi (*debug*) một sự cố trong môi trường vận hành thực tế (*production*). API chậm. Nhưng chậm ở đâu?

Một kiến trúc sư lâu năm (*senior architect*) vẽ sơ đồ:

```
Máy khách → Bộ cân bằng tải → Máy chủ ứng dụng → Bộ nhớ đệm → CSDL
```

Rồi hỏi: "Yêu cầu (*Request*) đi qua bao nhiêu bước? Mỗi bước mất bao lâu?"

Đó là lần đầu tiên tôi hiểu ra: **Hệ thống là tập hợp các thành phần** (*components*) **tương tác với nhau**. Hiểu từng thành phần = hiểu cả hệ thống.

Tại Sao Phải Học Về Các Thành Phần?

Góc nhìn ở cấp độ mã nguồn:

```
def get_user(user_id):  
    return db.query("SELECT * FROM users WHERE id = ?", user_id)
```

Bạn thấy các hàm. Nhưng kiến trúc sư thấy:

Góc nhìn ở cấp độ hệ thống:

Máy khách (50ms)
↓
Bộ cân bằng tải (10ms)
↓
Máy chủ ứng dụng (20ms)
↓
Truy vấn CSDL (500ms) ← ĐIỂM NGHẼN
↓
Tổng: 580ms

Tư duy ở cấp độ hệ thống giúp bạn:

- **Xác định điểm nghẽn** (*bottlenecks*) nhanh chóng
- **Thiết kế để mở rộng** (*scale*) ngay từ đầu
- **Gỡ lỗi trong môi trường thực tế** hiệu quả
- **Giao tiếp** rõ ràng với nhóm

Bạn không thể tối ưu thứ mà bạn không hiểu.

Thành Phần 1: Máy Khách (*Client*) (Trình Duyệt/Ứng Dụng Di Động)

Vai Trò

Máy khách là lớp trình bày (*presentation layer*) — nơi người dùng tương tác với hệ thống.

Đặc điểm:

- Không đáng tin cậy (người dùng có thể sửa mã)
- Không kiểm soát được môi trường (thiết bị, mạng)
- Có trạng thái (*Stateful*) (có thể lưu dữ liệu cục bộ)

Các Quy Tắc Then Chốt

Quy tắc 1: KHÔNG BAO GIỜ tin tưởng máy khách

```
// BAD: Validate price ở client only
function checkout(items) {
  let total = items.reduce((sum, item) => sum + item.price, 0)
  submitOrder(total) // Hacker có thể modify price
}

// GOOD: Always validate server-side
function checkout(items) {
  submitOrder(items) // Server recalculate price
}
```

Tại sao? Tôi từng thấy hacker sửa JavaScript, đổi giá từ \$100 → \$1 rồi gửi đi. Máy chủ (Server) tin máy khách → Công ty mất tiền.

Quy tắc 2: Kiểm tra dữ liệu ở máy khách chỉ để cải thiện trải nghiệm người dùng (UX)

```
// Client validation: Immediate feedback (UX)
if (!email.includes('@')) {
  showError("Invalid email")
}

// Server PHẢI validate lại
if (!isValidEmail(email)) {
  return 400 // Never trust client
}
```

Các Sự Đánh Đổi (Trade-offs)

Ưu điểm:

- Tương tác phong phú
- Có thể hoạt động ngoại tuyến
- Giảm tải cho máy chủ (kết xuất phía máy khách)

Nhược điểm:

- Rủi ro bảo mật
- Môi trường không đồng nhất
- Khó cập nhật (người dùng không tải lại trang)

Khi Nào Nên Tối Ưu Máy Khách?

Tối ưu khi:

- Kích thước gói mã (bundle) lớn (> 1MB)

- Thời gian hiển thị đầu tiên (*first paint*) chậm ($> 3s$)
- Người dùng có thiết bị/mạng chậm

Các kỹ thuật:

- Chia nhỏ mã (*Code splitting*)
- Tải lười (*Lazy loading*)
- Service worker (*Service workers*) (lưu đệm (*caching*))
- CDN cho tài nguyên tĩnh (*static assets*)

Thành Phần 2: Bộ Cân Bằng Tải (*Load Balancer*)

Vai Trò

Phân phối lưu lượng (*traffic*) đồng đều giữa nhiều máy chủ (*servers*).

Khi Nào Cần?

Kịch bản 1: Một máy chủ duy nhất
1 máy chủ xử lý 1.000 yêu cầu/giây
Lưu lượng tăng lên 5.000 yêu cầu/giây
→ Máy chủ quá tải → Chậm/sập

Kịch bản 2: Nhiều máy chủ + Bộ cân bằng tải
5 máy chủ, mỗi máy xử lý 1.000 yêu cầu/giây
Bộ cân bằng tải phân phối đồng đều
→ Hệ thống xử lý trơn tru 5.000 yêu cầu/giây

Cần bộ cân bằng tải khi:

- 1 máy chủ không đủ năng lực xử lý (*capacity*)
- Cần tính sẵn sàng cao (*high availability*) (1 máy chủ chết, các máy khác tiếp quản)
- Triển khai cuộn chiếu (*Rolling deployments*) (cập nhật từng máy chủ một)

Khi Nào KHÔNG Cần?

Không cần khi:

- < 1.000 người dùng
- Công cụ nội bộ
- MVP/Bản mẫu thử nghiệm
- Ngân sách hạn chế

Hãy bắt đầu đơn giản. Thêm LB khi đã chứng minh là cần thiết.

Các Thuật Toán Cốt Lõi

Round Robin (xoay vòng):

Yêu cầu 1 → Máy chủ A
Yêu cầu 2 → Máy chủ B
Yêu cầu 3 → Máy chủ C
Yêu cầu 4 → Máy chủ A (lặp lại)

Đơn giản, phân phối đều
Không xét đến tải của máy chủ

Least Connections (ít kết nối nhất):

Máy chủ A: 50 kết nối
Máy chủ B: 20 kết nối ← Chọn máy này
Máy chủ C: 35 kết nối

Cân bằng tải tốt hơn
Phức tạp hơn, phải theo dõi trạng thái

Weighted Round Robin (xoay vòng có trọng số):

Máy chủ A (16GB RAM): trọng số 2
Máy chủ B (8GB RAM): trọng số 1

A nhận 2 yêu cầu cho mỗi 1 yêu cầu gửi tới B

Tận dụng nhiều hơn các máy chủ mạnh
Phải cấu hình trọng số

Các Sự Đánh Đổi

Được:

- Tăng năng lực xử lý
- Tính sẵn sàng cao
- Triển khai không gián đoạn (zero-downtime)

Mất:

- Thêm độ phức tạp
- Có thể trở thành điểm lỗi duy nhất (chính LB)
- Tăng độ trễ mạng (+10-50ms)

Khuyến Nghị Cá Nhân

Bắt đầu với Round Robin. Đơn giản và hiệu quả cho 80% trường hợp.

Chỉ dùng Least Connections khi các yêu cầu (*requests*) có thời gian xử lý rất khác nhau (WebSocket, tải tệp lên).

Thành Phần 3: Máy Chủ Ứng Dụng (*Application Server*)

Vai Trò

Lớp logic nghiệp vụ (*business logic*). Xử lý yêu cầu, điều phối luồng dữ liệu (*data flow*).

Quyết Định Then Chốt: Phi Trạng Thái (*Stateless*) Hay Có Trạng Thái

Máy chủ phi trạng thái:

```
# Không lưu user session trong memory
def handle_request(request):
    user_id = jwt.decode(request.token) # Get from token
    user = db.get_user(user_id)        # Get from DB
    return process(user)

# Benefit: Any server can handle any request
```

Máy chủ có trạng thái:

```
# Lưu session trong memory
sessions = {} # In-memory

def handle_login(user_id):
    sessions[user_id] = UserSession(user_id)

def handle_request(user_id):
    session = sessions[user_id] # Must hit same server!
    return process(session)
```

Phân Tích Sự Đánh Đổi (Trade-off)

Phi trạng thái:

- Dễ mở rộng theo chiều ngang
- Máy chủ nào cũng xử lý được mọi yêu cầu
- Thân thiện với đám mây (máy chủ có thể thuê)
- Tốn thêm một lần gọi CSDL để lấy dữ liệu người dùng

Có trạng thái:

- Độ trễ thấp (không cần đi một vòng tới CSDL)
- Có thể lưu đệm dữ liệu riêng của từng người dùng
- Khó mở rộng (cần sticky session)
- Mất phiên khi máy chủ khởi động lại

Quan Điểm Mạnh Mẽ Của Tôi

Mặc định chọn phi trạng thái. Chỉ chọn có trạng thái khi có lý do thật sự chính đáng (kết nối WebSocket, máy chủ game).

Hệ thống hiện đại với vi dịch vụ (*microservices*)? Phi trạng thái là bắt buộc.

Cấu Hình Máy Chủ

Mô hình xử lý đồng thời (*Concurrency model*) **rất quan trọng:**

Dựa trên luồng (Java, kiểu truyền thống):

- 1 luồng cho mỗi yêu cầu
- Bị giới hạn bởi kích thước thread pool
- Tốt: Mô hình quen thuộc
- Dở: Tốn bộ nhớ

Vòng lặp sự kiện (Node.js, Python asyncio):

- 1 luồng xử lý nhiều yêu cầu
- I/O không chặn (non-blocking)
- Tốt: Khả năng xử lý đồng thời cao
- Dở: Tác vụ nặng CPU chặn tất cả

Dựa trên worker (Gunicorn, uWSGI):

- Tạo sẵn các worker (pre-fork)
- Mỗi worker tự xử lý yêu cầu
- Tốt: Có lập sự cô'
- Dở: Tốn nhiều bộ nhớ hơn

Chọn theo khối lượng công việc (*workload*):

- Nặng về I/O (gọi API, truy vấn CSDL) → Vòng lặp sự kiện (*Event-loop*)
- Nặng về CPU (xử lý ảnh) → Dựa trên tiến trình xử lý (*worker*)
- Hỗn hợp → Dựa trên tiến trình xử lý, bên trong dùng bất đồng bộ (*async*)

Thành Phần 4: Lớp Bộ Nhớ Đệm (*Cache*)

Vai Trò

Lưu dữ liệu được truy cập thường xuyên trong bộ nhớ (in-memory) để giảm tải cho CSDL.

Câu Hỏi Nền Tảng

Có nên dùng bộ nhớ đệm không?

Trả lời bằng lưu đồ:

Có thiên về đọc không? (đọc:ghi > 10:1)
↓ CÓ
Dữ liệu có thay đổi thường xuyên không?
↓ KHÔNG
Truy vấn có tốn kém không? (> 100ms)
↓ CÓ
→ DỪNG BỘ NHỚ ĐỆM

Có câu trả lời KHÔNG nào → Cân nhắc kỹ

Các Chiến Lược Lưu Đệm

Chiến lược 1: Cache-Aside (tải lười — Lazy Loading)

```
def get_product(product_id):  
    # 1. Check cache  
    product = cache.get(f"product:{product_id}")  
    if product:  
        return product # Cache hit  
  
    # 2. Cache miss → Query DB  
    product = db.query("SELECT * FROM products WHERE id = ?", product_id)  
  
    # 3. Store in cache  
    cache.set(f"product:{product_id}", product, ttl=3600)  
    return product  
  
Only cache what's needed  
First request slow (cache miss)
```

Chiến lược 2: Write-Through (ghi xuyên)

```
def update_product(product_id, data):  
    # 1. Update DB  
    db.update("products", product_id, data)  
  
    # 2. Update cache immediately  
    cache.set(f"product:{product_id}", data)  
  
Cache always fresh  
Every write slower (2 operations)
```

Chiến lược 3: Write-Behind (ghi sau)

```
def update_product(product_id, data):
    # 1. Update cache immediately
    cache.set(f"product:{product_id}", data)

    # 2. Async write to DB
    queue.add({"action": "update_product", "id": product_id, "data": data})

Writes very fast
Risk data loss if cache crashes
```

Vô Hiệu Hoá Bộ Nhớ Đệm (Cache Invalidation) – Bài Toán Khó

Phil Karlton: "Trong khoa học máy tính chỉ có hai việc khó: vô hiệu hoá bộ nhớ đệm và đặt tên."

Vấn đề:

1. Người dùng cập nhật hồ sơ → CSDL được cập nhật
2. Bộ nhớ đệm vẫn giữ dữ liệu cũ
3. Người dùng tải lại trang → Thấy dữ liệu cũ (trúng bộ nhớ đệm)
4. Người dùng bối rối

Giải pháp:

Phương án 1: TTL (Thời gian sống — Time To Live)

```
cache.set("user:123", user, ttl=300) # Expire after 5min

Simple
Data stale for up to 5 minutes
```

Phương án 2: Vô hiệu hoá chủ động (Active Invalidation)

```
def update_user(user_id, data):
    db.update("users", user_id, data)
    cache.delete(f"user:{user_id}") # Delete immediately

Always fresh
Must remember to invalidate everywhere
```

Phương án 3: Hướng sự kiện (Event-Driven)

```
# Service A updates user
db.update("users", user_id, data)
event_bus.publish("user.updated", user_id)

# Service B listens
@event_bus.subscribe("user.updated")
def on_user_updated(user_id):
    cache.delete(f"user:{user_id}")

Decoupled, scalable
Complex infrastructure
```

Quy Tắc Cá Nhân

Bắt đầu với TTL. Đơn giản và hiệu quả.

Thêm vô hiệu hoá chủ động khi độ mới của dữ liệu là yếu tố then chốt.

Chỉ dùng cách hướng sự kiện khi có nhiều dịch vụ (*services*) cần phản ứng.

Khi Nào KHÔNG Nên Dùng Bộ Nhớ Đệm

Không lưu đệm:

- Dữ liệu ghi nhiều (số lượng tồn kho thay đổi liên tục)
- Dữ liệu thời gian thực (giá cổ phiếu)
- Dữ liệu cá nhân hoá (khác nhau theo từng người, tỷ lệ trùng thấp)
- Tập dữ liệu nhỏ (< 1000 bản ghi, CSDL đã đủ nhanh)

Thành Phần 5: Cơ Sở Dữ Liệu (*Database*)

Vai Trò

Lưu trữ bền vững (*Persistent storage*). Nguồn dữ liệu gốc (*Source of truth*).

Vì Sao Cơ Sở Dữ Liệu Thường Là Điểm Nghẽn (Bottleneck)

Đọc từ bộ nhớ: 100 nano giây
Đọc từ SSD: 150 micro giây (chậm hơn 1.500 lần)
Đọc từ HDD: 10 mili giây (chậm hơn 100.000 lần)

Cơ sở dữ liệu phải:

- Đọc từ đĩa
- Phân tích cú pháp truy vấn
- Thực thi kế hoạch truy vấn
- Trả về kết quả

→ Về bản chất là chậm so với bộ nhớ

Thứ Bậc Tối Ưu

Cấp 1: Tối ưu truy vấn (Query Optimization) (Làm việc này trước)

```
-- BAD: No index, full table scan
SELECT * FROM users WHERE email = 'john@example.com';
-- 10M rows → 5 seconds

-- GOOD: Add index
CREATE INDEX idx_users_email ON users(email);
-- Same query → 10ms (500x faster!)
```

Cấp 2: Gộp kết nối (Connection Pooling)

```
# BAD: New connection per request
def get_user(user_id):
    conn = create_connection() # Expensive!
    user = conn.query(...)
    conn.close()
    return user

# GOOD: Reuse connections
pool = ConnectionPool(min=5, max=20)

def get_user(user_id):
    with pool.get_connection() as conn:
        return conn.query(...)
```

Cấp 3: Bản sao chỉ đọc (Read Replicas)

Máy chủ chính (Ghi)

↓ nhân bản

Bản sao 1, Bản sao 2, Bản sao 3 (Đọc)

Ghi → Máy chủ chính

Đọc → Cân bằng tải giữa các bản sao

Dễ dàng mở rộng khả năng đọc

Nhất quán cuối cùng (độ trễ nhân bản)

Cấp 4: Phân mảnh dữ liệu (*Sharding*) (Phương án cuối cùng)

Chỉ khi:

- Một CSDL không chịu nổi tải
- Đã mở rộng theo chiều dọc tới mức tối đa
- Dữ liệu quá lớn so với 1 máy

Cái giá:

- Rất phức tạp
- Truy vấn xuyên mảnh rất khó
- Phân mảnh lại là cơn ác mộng

Các Sự Đánh Đổi: SQL Và NoSQL

SQL (PostgreSQL, MySQL):

Giao dịch ACID

Truy vấn phong phú (JOIN, tổng hợp)

Toàn vẹn dữ liệu

Khó mở rộng theo chiều ngang

Thay đổi lược đồ tốn kém

NoSQL (MongoDB, DynamoDB):

Dễ mở rộng theo chiều ngang

Lược đồ linh hoạt

Thông lượng ghi cao

Không có JOIN (phải xử lý ở tầng ứng dụng)

Nhất quán cuối cùng

Khả năng truy vấn hạn chế hơn

Khung ra quyết định (*Decision framework*):

Chọn SQL khi:

- Cần giao dịch (ngân hàng, thương mại điện tử)
- Quan hệ dữ liệu phức tạp
- Truy vấn tùy biến (ad-hoc)
- Toàn vẹn dữ liệu là then chốt

Chọn NoSQL khi:

- Cần quy mô cực lớn
- Khối lượng công việc thiên về ghi
- Lược đồ linh hoạt
- Truy vấn đơn giản (tra cứu khoá-giá trị)

Quan Điểm Của Tôi

Hãy bắt đầu với SQL (PostgreSQL).

Nó xử lý được 99% tình huống sử dụng (*use cases*). Cột JSONB mang lại sự linh hoạt. Hãy mở rộng theo chiều dọc (*scale vertically*) trước, thêm bản sao chỉ đọc sau.

Chỉ chuyển sang NoSQL khi đã chứng minh được SQL không chịu nổi tải.

Ghép Tất Cả Lại Với Nhau: Luồng Yêu Cầu (*Request Flow*)

Hãy lần theo một yêu cầu từ đầu đến cuối:

Ví Dụ: Người Dùng Tải Trang Chủ

1. Trình duyệt → Tra cứu DNS
Thời gian: 20ms
 2. Trình duyệt → HTTPS tới Bộ cân bằng tải
Thời gian: 50ms (TCP + TLS)
 3. Bộ cân bằng tải → Chọn máy chủ ứng dụng (Round Robin)
Thời gian: 5ms
 4. Máy chủ ứng dụng → Kiểm tra bộ nhớ đệm cho dữ liệu trang chủ
Thời gian: 2ms
Kết quả: TRÚNG bộ nhớ đệm
 5. Máy chủ ứng dụng → Trả HTML về LB
Thời gian: 5ms
 6. LB → Trả về trình duyệt
Thời gian: 10ms
 7. Trình duyệt → Phân tích và hiển thị
Thời gian: 200ms
- Tổng: ~292ms
Điểm nghẽn: Trình duyệt hiển thị trang (200ms)

Nếu Trượt Bộ Nhớ Đệm (Cache MISS):

4. Máy chủ ứng dụng → Kiểm tra bộ nhớ đệm
Kết quả: TRƯỢT
 5. Máy chủ ứng dụng → Truy vấn CSDL
Thời gian: 300ms ← ĐIỂM NGHẼN MỚI
 6. Máy chủ ứng dụng → Lưu vào bộ nhớ đệm
Thời gian: 2ms
 7. Máy chủ ứng dụng → Trả về HTML
Thời gian: 5ms
- Tổng: ~594ms (chậm gấp 2 lần)

Chiến Lược Tối Ưu

Điểm nghẽn hiện tại: CSDL (300ms)

Các phương án:

- A. Thêm chỉ mục → 300ms → 50ms (nhanh gấp 6 lần, ít công sức)
- B. Tối ưu truy vấn → 50ms → 20ms (gấp 2,5 lần, công sức trung bình)
- C. Thêm lưu đệm → Giảm số lần truy cập CSDL xuống gần bằng 0

Hành động: Làm A trước (lợi ích/chi phí cao nhất), rồi theo dõi.

Mô Hình Tư Duy (*Mental Model*): Mọi Thứ Đều Là Một Hệ Thống

Hãy áp dụng tư duy theo thành phần (*component*) ở mọi nơi:

Nhà hàng:

- Máy khách: Thực khách
- Bộ cân bằng tải: Nhân viên đón khách
- Máy chủ: Bồi bàn
- Bộ nhớ đệm: Hiểu biết về thực đơn
- Cơ sở dữ liệu: Nhà bếp

Điểm nghẽn? Nhà bếp (mất 20 phút chuẩn bị món)

Giải pháp? Thêm đầu bếp (mở rộng theo chiều ngang (*horizontal scaling*)) hoặc trang bị thiết bị tốt hơn (mở rộng theo chiều dọc (*vertical scaling*)).

Quán cà phê:

- Điểm nghẽn: 1 nhân viên pha chế, 10 khách hàng
- Giải pháp: Thêm nhân viên pha chế hoặc ki-ốt tự phục vụ

Khi bạn nhìn ra được những mẫu (*patterns*) này, bạn sẽ hiểu hệ thống một cách sâu sắc.

Những Điểm Cốt Lõi

Mọi hệ thống web đều có 5 thành phần cơ bản:

1. **Máy khách** — Trình bày, không đáng tin cậy
2. **Bộ cân bằng tải** — Phân phối lưu lượng
3. **Máy chủ ứng dụng** — Logic nghiệp vụ, ưu tiên phi trạng thái
4. **Bộ nhớ đệm** — Đọc nhanh, nhưng vô hiệu hoá (*invalidation*) rất khó
5. **Cơ sở dữ liệu** — Nguồn dữ liệu gốc, thường là điểm nghẽn

Nguyên tắc thiết kế:

- **Bắt đầu đơn giản** — Đừng thiết kế thừa (*over-engineer*)
- **Đo lường trước** — Tìm ra điểm nghẽn thật sự
- **Tối ưu điểm nghẽn** — Không phải tối ưu mọi thứ
- **Mở rộng dần dần** — Theo chiều dọc → Theo chiều ngang

Ý thức về sự đánh đổi:

- Máy chủ phi trạng thái và có trạng thái
- Độ phức tạp của bộ nhớ đệm và hiệu năng (*performance*)
- Tính nhất quán (*Consistency*) của SQL và khả năng mở rộng của NoSQL
- Hệ thống nguyên khối (*Monolith*) đơn giản và hệ thống phân tán (*distributed*) phức tạp

Các bước tiếp theo:

Hãy thực hành với một hệ thống thật. Lần theo các yêu cầu. Xác định các thành phần. Đo độ trễ (*latency*). Tìm điểm nghẽn.

Tư duy theo thành phần là nền móng. Hãy làm chủ nó trước khi học các mẫu phức tạp hơn.

Các Mẫu Giao Tiếp (*Communication Patterns*): Đồng Bộ (*Sync*) và Bất Đồng Bộ (*Async*)

Sau khi hiểu các thành phần (*components*) trong hệ thống, câu hỏi tiếp theo là: **Chúng giao tiếp với nhau như thế nào?**

Tôi còn nhớ lần đầu thiết kế tính năng "tải video lên". Cách tiếp cận của tôi:

```
def upload_video(video_file):
    save_to_s3(video_file)           # 2 giây
    generate_thumbnail(video_file)  # 5 giây
    transcode_720p(video_file)     # 30 giây
    transcode_1080p(video_file)    # 60 giây
    send_notification()             # 1 giây
    return "Success"                # User đợi 98 giây!
```

Một kiến trúc sư lâu năm (*senior architect*) nhìn code rồi hỏi: "Người dùng có cần đợi tất cả những thứ này không?"

Tôi nhận ra: **Không**. Người dùng (*User*) chỉ cần biết video đã được tải lên. Phần xử lý có thể làm sau.

Đó là lúc tôi học về **giao tiếp đồng bộ** (*synchronous communication*) và **giao tiếp bất đồng bộ** (*asynchronous communication*) - một trong những quyết định quan trọng nhất trong thiết kế hệ thống (*system design*).

Tại Sao Mẫu Giao Tiếp (*Communication Pattern*) Lại Quan Trọng?

Mẫu (*Pattern*) bạn chọn ảnh hưởng đến:

Hiệu năng (*Performance*):

- Đồng bộ: Người dùng phải đợi cho đến khi xong
- Bất đồng bộ: Người dùng nhận phản hồi (*response*) ngay, xử lý sau

Khả năng mở rộng (Scalability):

- Đồng bộ: Khó xử lý các đợt lưu lượng tăng đột biến (*traffic spikes*)
- Bất đồng bộ: Hàng đợi (*Queue*) đệm các yêu cầu (*requests*) một cách tự nhiên

Độ tin cậy (Reliability):

- Đồng bộ: Sự cố (*Failure*) = lỗi ngay lập tức
- Bất đồng bộ: Có sẵn cơ chế thử lại (*retry*)

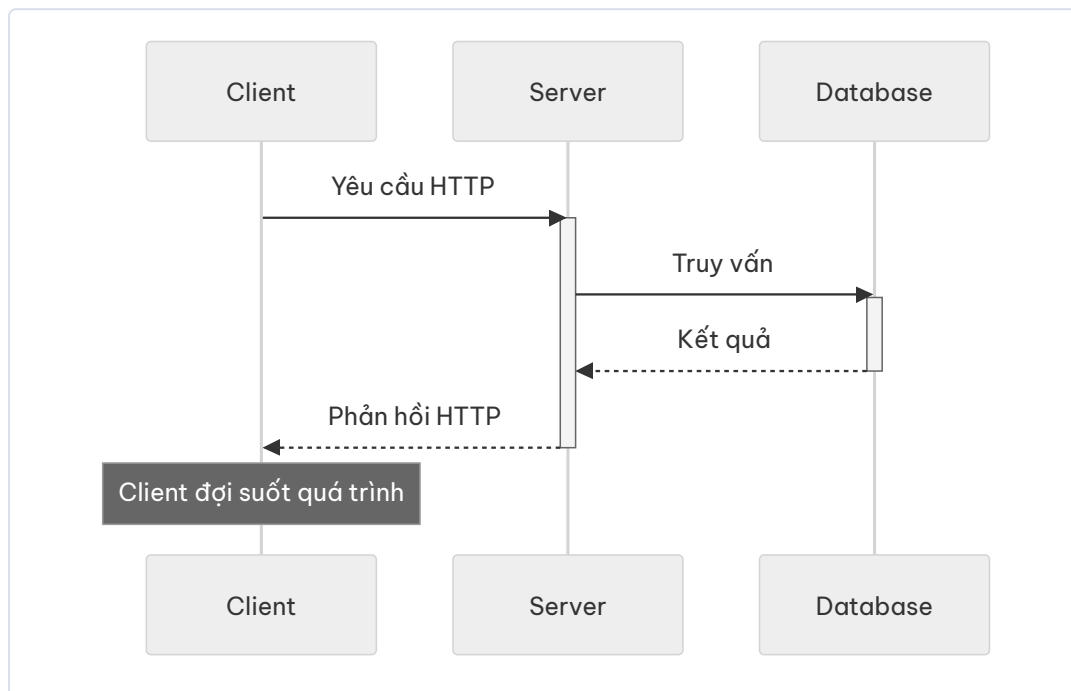
Độ phức tạp (Complexity):

- Đồng bộ: Đơn giản, dễ hiểu
- Bất đồng bộ: Phức tạp, nhất quán cuối cùng (*eventual consistency*)

Không có mẫu "tốt hơn". Chỉ có mẫu "phù hợp".

Giao Tiếp Đồng Bộ (Yêu Cầu-Phản Hồi (Request-Response))

Cách Hoạt Động



Máy khách (Client) gửi yêu cầu (request) → đợi → nhận phản hồi. Đây là thao tác chặn (blocking).

Ví Dụ Code

```
# Client code
def get_user_profile(user_id):
    response = requests.get(f"/api/users/{user_id}") # Blocks here
    return response.json()

# Server code
@app.route('/api/users/<user_id>')
def get_user(user_id):
    user = database.query(f"SELECT * FROM users WHERE id = {user_id}")
    return jsonify(user)
```

Luồng xử lý:

1. Máy khách gửi yêu cầu
2. Máy khách **chờ đợi** (bị chặn (blocks))
3. Máy chủ (Server) xử lý
4. Máy chủ trả về phản hồi
5. Máy khách nhận kết quả

Tổng thời gian = Tổng của tất cả các bước.

Khi Nào Dùng Đồng Bộ (Synchronous)

Dùng đồng bộ khi:

Người dùng cần kết quả ngay lập tức

Ví dụ:

- Đăng nhập (cần biết thành công/thất bại ngay)
- Tìm kiếm (người dùng đang đợi kết quả)
- Gửi biểu mẫu (cần phản hồi kiểm tra hợp lệ)
- Gọi API (máy khách cần dữ liệu để tiếp tục)

Thao tác nhanh (< 5 giây)

- Tra cứu cơ sở dữ liệu
- Tính toán đơn giản
- Đọc tệp (tệp nhỏ)

Quy trình đơn giản

Yêu cầu → Xử lý → Phản hồi
Không có logic rẽ nhánh phức tạp

Khi Nào KHÔNG Nên Dùng Đồng Bộ

Xử lý lâu (> 5 giây)

- Mã hoá video
 - Xử lý tệp lớn
 - Báo cáo phức tạp
 - Các thao tác theo lô
- Người dùng bị hết thời gian chờ, trải nghiệm tệ

Các phụ thuộc bên ngoài (*external dependencies*) không đáng tin cậy

- API của bên thứ ba (có thể chậm/sập)
 - Công thức toán
 - Gửi email
- API của bạn chậm vì phụ thuộc bị chậm

Các đợt lưu lượng tăng đột biến không dự đoán được

Đợt giảm giá Black Friday, bài đăng lan truyền
→ Mọi yêu cầu xếp hàng chờ
→ Hệ thống quá tải

Các Đánh Đổi (Trade-offs)

Ưu điểm:

- Logic đơn giản (dễ hiểu)
- Phản hồi tức thì
- Dễ gỡ lỗi (luồng tuyến tính)
- Hỗ trợ giao dịch (ACID)

Nhược điểm:

- Trải nghiệm tệ với các thao tác chậm
- Liên kết chặt (máy khách phụ thuộc vào tính sẵn sàng của máy chủ)
- Khó mở rộng (chặn đồng bộ)
- Rủi ro hết thời gian chờ

Ví Dụ Thực Tế: Luồng Đăng Nhập

```
@app.route('/login', methods=['POST'])
def login():
    email = request.json['email']
    password = request.json['password']

    # Step 1: Validate credentials (50ms)
    user = db.query("SELECT * FROM users WHERE email = ?", email)
    if not user or not verify_password(password, user.password_hash):
        return {"error": "Invalid credentials"}, 401

    # Step 2: Generate token (10ms)
    token = generate_jwt(user.id)

    # Step 3: Log login event (20ms)
    db.insert("INSERT INTO login_logs (user_id, timestamp) VALUES (?, ?)",
             user.id, datetime.now())

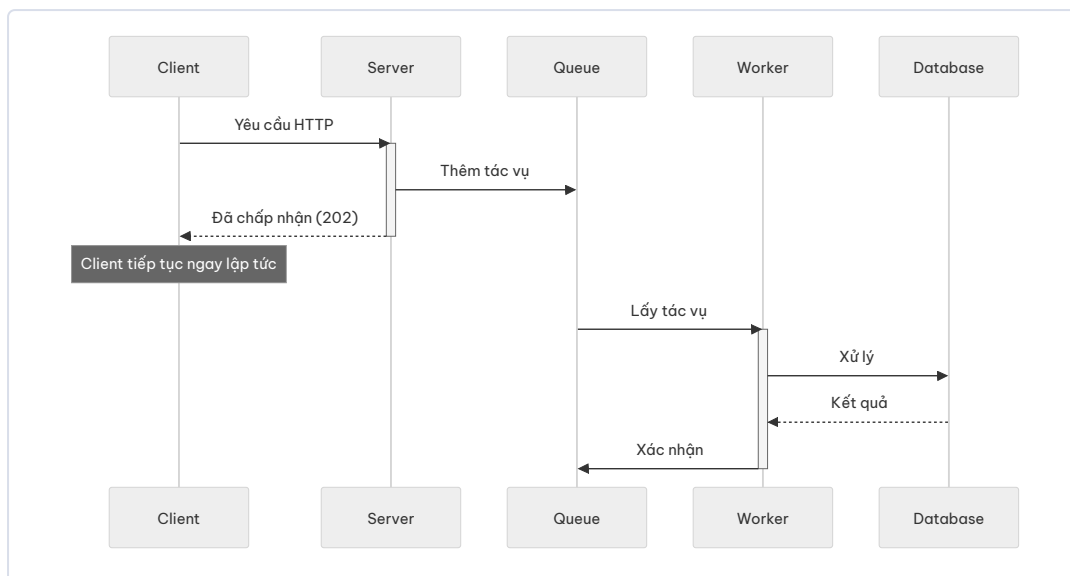
    return {"token": token, "user": user}, 200
# Total: ~80ms - Acceptable cho sync
```

Tại sao đồng bộ là phù hợp:

- Người dùng cần kết quả ngay (không thể tiếp tục nếu chưa có token)
- Thao tác nhanh (< 100ms)
- Quy trình đơn giản

Giao Tiếp Bất Đồng Bộ (Hàng Đợi Thông Điệp (Message Queue))

Cách Hoạt Động



Máy khách gửi yêu cầu → máy chủ đưa vào hàng đợi → trả về ngay. Tiến trình xử lý (Worker) sẽ xử lý sau.

Ví Dụ Code

```
# Server code
from celery import Celery
app = Celery('tasks', broker='redis://localhost:6379')

@app.route('/upload-video', methods=['POST'])
def upload_video():
    video_file = request.files['video']

    # Quick operation: Save original
    video_id = save_to_s3(video_file) # 2 seconds

    # Add processing to queue
    process_video_task.delay(video_id)

    return {"status": "processing", "video_id": video_id}, 202
    # User gets response sau 2s (not 98s!)

# Worker code (runs separately)
@app.task
def process_video_task(video_id):
    video = get_from_s3(video_id)
    generate_thumbnail(video) # 5s
    transcode_720p(video) # 30s
    transcode_1080p(video) # 60s
    send_notification(video_id) # 1s
    # Total: 96s, but user doesn't wait
```

Luồng xử lý:

1. Máy khách gửi yêu cầu
2. Máy chủ lưu lại và đưa tác vụ vào hàng đợi
3. Máy chủ trả về ngay lập tức (202 Accepted)
4. Máy khách tiếp tục (không đợi)
5. Tiến trình xử lý nền (*Background worker*) nhận tác vụ
6. Tiến trình xử lý xử lý (có thể mất nhiều thời gian)
7. (Tuỳ chọn) Thông báo cho máy khách khi xong

Khi Nào Dùng Bất Đồng Bộ (*Asynchronous*)

Dùng bất đồng bộ khi:

Xử lý lâu (> 5 giây)

Ví dụ:

- Xử lý video/hình ảnh
- Tạo báo cáo
- Nhập dữ liệu theo lô
- Đồng bộ hoá dữ liệu

Người dùng không cần kết quả ngay

- Gửi email
- Tạo hoá đơn
- Cập nhật số liệu phân tích
- Đồng bộ chạy nền

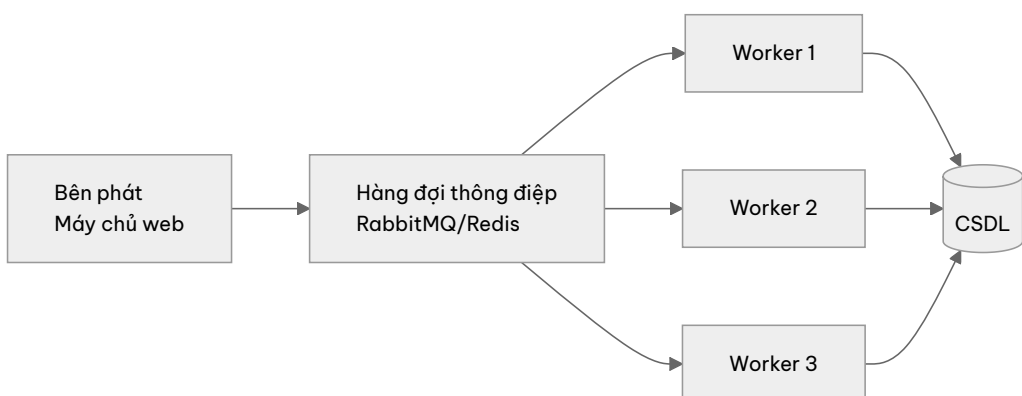
Cần xử lý các đợt lưu lượng tăng đột biến

Hàng đợi làm vùng đệm cho các yêu cầu:
1000 yêu cầu/giây → Hàng đợi giữ lại
Worker xử lý với tốc độ ổn định (100/giây)
Máy chủ không bị quá tải

Các phụ thuộc bên ngoài không đáng tin cậy

- Gọi API của bên thứ ba
 - Gửi webhook
 - Xử lý thanh toán
- Cơ chế 'thư' lại có sẵn trong hàng đợi

Các Thành Phần Của Hàng Đợi Thông Điệp



Bên phát (*Producer*): Dịch vụ (*Service*) tạo ra các tác vụ (máy chủ web (*web server*))

Hàng đợi: Lưu các tác vụ đang chờ được xử lý

Bên tiêu thụ (*Consumer*)/**Tiến trình xử lý:** Các dịch vụ (*services*) xử lý tác vụ

Các Đảm Bảo Của Hàng Đợi

Tối đa một lần (*At-Most-Once*):

Thông điệp có thể bị mất
Không thu lại
Tình huống sử dụng: Số liệu đo, log (mất một ít cũng không sao)

Ít nhất một lần (*At-Least-Once*) (Phổ biến):

Thông điệp không bị mất
Có thể bị xử lý nhiều lần
Tình huống sử dụng: Phân lớn logic nghiệp vụ (kèm tính lũy đẳng)

Đúng một lần (*Exactly-Once*) (Hiếm):

Thông điệp được xử lý đúng 1 lần
Rất khó triển khai
Tình huống sử dụng: Giao dịch tài chính

Tính Lũy Đẳng (*Idempotency*) - Khái Niệm Then Chốt

Vấn đề: Tiến trình xử lý bị sập sau khi xử lý nhưng trước khi xác nhận (*acknowledge*).

Tác vụ: "Trừ \$100 của người dùng"
Worker 1: Xử lý → Trừ \$100 → Sập (chưa ACK)
Hàng đợi: Gửi lại tác vụ
Worker 2: Xử lý → Lại trừ \$100 nữa!
Người dùng bị trừ \$200 (sai!)

Giải pháp: Các thao tác lũy đẳng (*idempotent*)

```

@app.task
def charge_user(order_id, amount):
    # Check if already processed
    existing = db.query("SELECT * FROM charges WHERE order_id = ?", order_id)
    if existing:
        return "Already charged" # Idempotent!

    # Process
    charge_credit_card(amount)
    db.insert("INSERT INTO charges (order_id, amount) VALUES (?, ?)",
              order_id, amount)

    return "Charged successfully"

```

Khi thiết kế các thao tác bất đồng bộ, hãy luôn làm cho chúng lũy đẳng.

Các Đánh Đổi

Ưu điểm:

- Trải nghiệm tốt hơn (phản hồi nhanh)
- Chịu được lưu lượng tăng đột biến (hàng đợi làm vùng đệm)
- Cơ chế 'thư' lại (độ tin cậy)
- Tách rời các dịch vụ

Nhược điểm:

- Kiến trúc phức tạp
- Nhất quán cuối cùng
- Khó gỡ lỗi hơn (phân tán)
- Cần giám sát (độ dài hàng đợi, sức khỏe worker)

Ví Dụ Thực Tế: Đơn Hàng Thương Mại Điện Tử

```
@app.route('/orders', methods=['POST'])
def create_order():
    items = request.json['items']
    user_id = request.json['user_id']

    # Sync: Critical operations
    order_id = db.insert("INSERT INTO orders (...) VALUES (...)")
    reserve_inventory(items) # Must happen now

    # Async: Non-critical operations
    send_confirmation_email.delay(order_id)
    update_analytics.delay(order_id)
    notify_warehouse.delay(order_id)

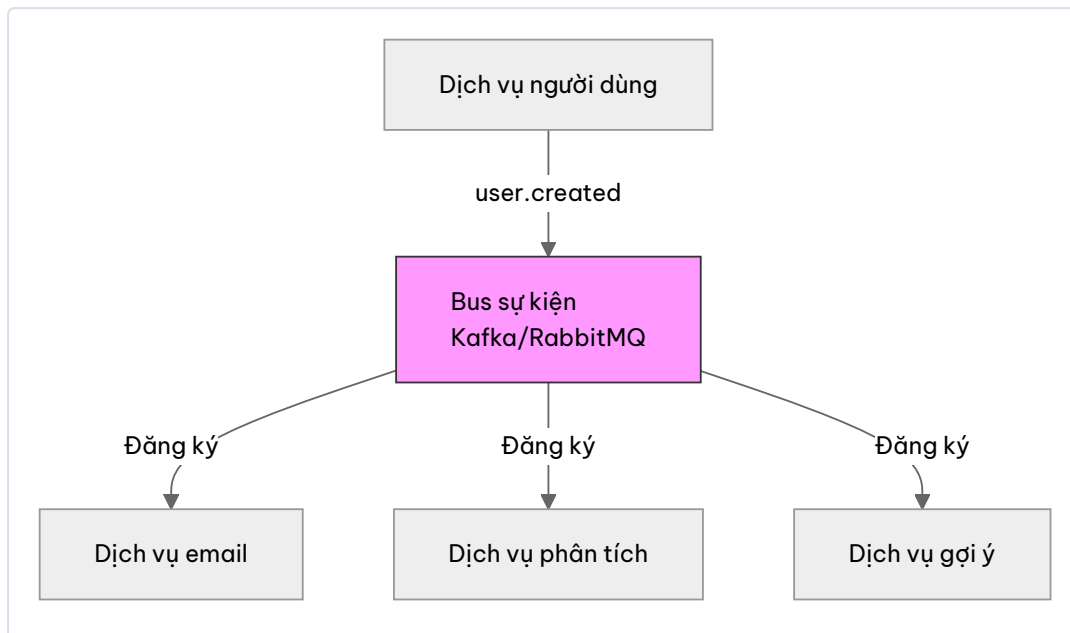
    return {"order_id": order_id, "status": "confirmed"}, 201
    # User gets response immediately
    # Email/analytics happen in background
```

Tại sao lại thiết kế như vậy?

- Tạo đơn hàng: Đồng bộ (người dùng cần order_id)
- Tồn kho: Đồng bộ (phải ngăn bán vượt số lượng tồn)
- Email: Bất đồng bộ (người dùng không đợi email)
- Phân tích: Bất đồng bộ (không nằm trên đường găng (*critical path*))

Kiến Trúc Hướng Sự Kiện (Event-Driven Architecture) (Bất Đồng Bộ Năng Cao)

Cách Hoạt Động



Thay vì User Service gọi trực tiếp tới Email/Analytics, nó phát hành (*publish*) một sự kiện (*event*). Các dịch vụ khác đăng ký (*subscribe*) để nhận.

Ví Dụ Code

```
# User Service (Publisher)
def create_user(email, name):
    user_id = db.insert("INSERT INTO users (...) VALUES (...)")

    # Publish event
    event_bus.publish('user.created', {
        'user_id': user_id,
        'email': email,
        'name': name,
        'timestamp': datetime.now()
    })

    return user_id

# Email Service (Subscriber)
@event_bus.subscribe('user.created')
def on_user_created(event):
    send_welcome_email(event['email'], event['name'])

# Analytics Service (Subscriber)
@event_bus.subscribe('user.created')
def on_user_created(event):
    track_signup(event['user_id'], event['timestamp'])

# Recommendation Service (Subscriber)
@event_bus.subscribe('user.created')
def on_user_created(event):
    initialize_recommendations(event['user_id'])
```

Lợi ích:

- User Service không cần biết ai tiêu thụ (*consume*) các sự kiện (*events*)
- Dễ dàng thêm bên đăng ký (*subscribers*) mới (không phải sửa User Service)
- Mỗi dịch vụ đều độc lập

Khi Nào Dùng Hướng Sự Kiện (*Event-Driven*)

Kiến trúc vi dịch vụ (*microservices*)

Nhiều dịch vụ cần phản ứng với cùng một sự kiện

Cần liên kết lỏng (*loose coupling*)

Hệ thống nguyên khối (*monolith*) **đơn giản** (thiết kế thừa (*over-engineering*))

Nhóm nhỏ (chi phí phụ (*overhead*) bảo trì)

So Sánh Các Mẫu

Kịch Bản (Scenario): Người Dùng Tải Lên Ảnh Đại Diện

Cách tiếp cận 1: Đồng bộ

```
@app.route('/upload-photo', methods=['POST'])
def upload_photo():
    photo = request.files['photo']

    # All sync
    uploaded_url = s3.upload(photo)          # 2s
    thumbnail_url = resize_image(photo)      # 3s
    update_database(user_id, uploaded_url)   # 0.1s
    invalidate_cache(user_id)                # 0.1s

    return {"url": uploaded_url}, 200
# User waits 5.2 seconds
```

Cách tiếp cận 2: Bất đồng bộ

```
@app.route('/upload-photo', methods=['POST'])
def upload_photo():
    photo = request.files['photo']

    # Sync: Upload original only
    uploaded_url = s3.upload(photo) # 2s

    # Async: Processing
    process_photo.delay(user_id, uploaded_url)

    return {"url": uploaded_url, "status": "processing"}, 202
# User waits 2 seconds

@app.task
def process_photo(user_id, photo_url):
    thumbnail = generate_thumbnail(photo_url) # 3s
    update_database(user_id, photo_url)
    invalidate_cache(user_id)
```

Cách tiếp cận 3: Hướng sự kiện


```

@app.route('/upload-photo', methods=['POST'])
def upload_photo():
    photo = request.files['photo']
    uploaded_url = s3.upload(photo)

    # Publish event
    events.publish('photo.uploaded', {
        'user_id': user_id,
        'photo_url': uploaded_url
    })

    return {"url": uploaded_url}, 202

# Different services subscribe
@events.subscribe('photo.uploaded')
def generate_thumbnails(event):
    create_thumbnail(event['photo_url'])

@events.subscribe('photo.uploaded')
def update_ml_model(event):
    facial_recognition.train(event['photo_url'])

@events.subscribe('photo.uploaded')
def moderate_content(event):
    check_inappropriate_content(event['photo_url'])

```

Khung Ra Quyết Định (Decision Framework)

Chọn Đồng bộ khi:

- ✓ Người dùng cần kết quả ngay
- ✓ Thao tác nhanh (< 5 giây)
- ✓ Quy trình đơn giản
- ✓ Yêu cầu nhất quán mạnh

Chọn Bất đồng bộ (Hàng đợi) khi:

- ✓ Xử lý lâu (> 5 giây)
- ✓ Người dùng không cần kết quả ngay
- ✓ Cần cơ chế thử lại
- ✓ Phải chịu được lưu lượng tăng đột biến

Chọn Hướng sự kiện khi:

- ✓ Nhiều dịch vụ phản ứng với cùng một sự kiện
- ✓ Kiến trúc vi dịch vụ
- ✓ Cần liên kết lỏng
- ✓ Quy trình phức tạp

Các Mẫu (Patterns) & Phản Mẫu (Anti-Patterns) Thường Gặp

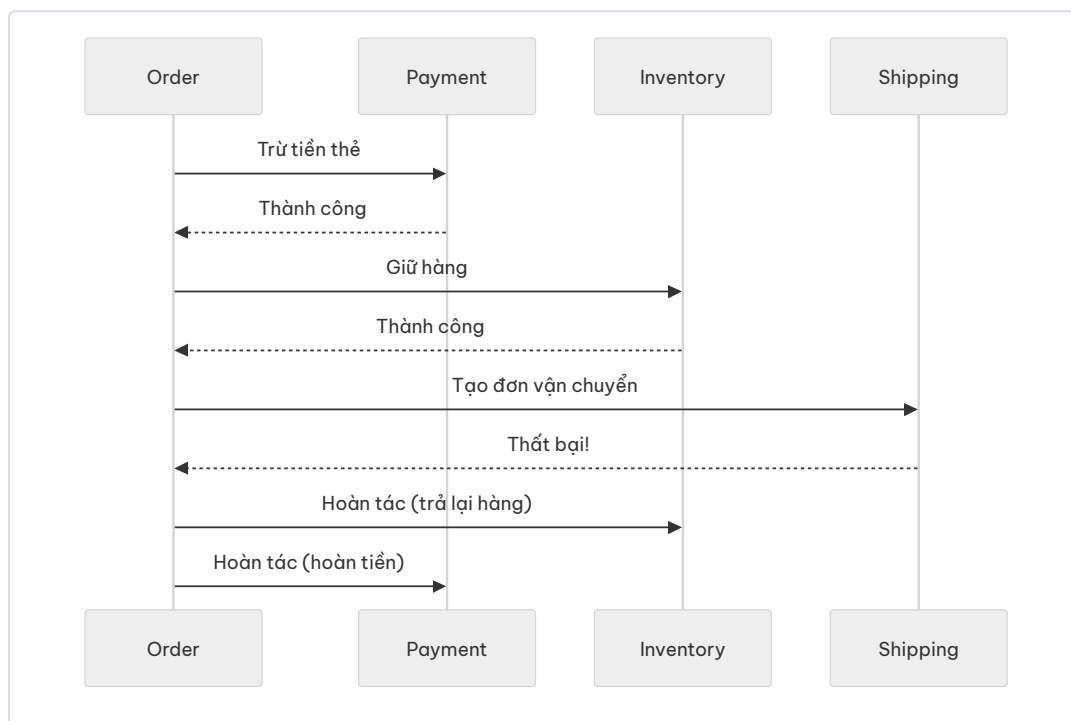
Mẫu 1: Đồng Bộ Cho Đọc, Bất Đồng Bộ Cho Ghi

```
# Read: Sync (user needs data now)
@app.route('/products/<id>')
def get_product(id):
    return db.query("SELECT * FROM products WHERE id = ?", id)

# Write: Async (can process later)
@app.route('/products/<id>/review', methods=['POST'])
def add_review(id):
    review_data = request.json
    add_review_task.delay(id, review_data)
    return {"status": "submitted"}, 202
```

Mẫu 2: Saga Pattern (Giao Dịch Phân Tán (Distributed Transactions))

Khi cần điều phối nhiều dịch vụ:



Triển khai:

```

def create_order_saga(order_data):
    order_id = create_order(order_data)

    try:
        # Step 1: Payment
        payment_id = charge_payment.delay(order_id).get()

        # Step 2: Inventory
        reserve_inventory.delay(order_id, order_data['items']).get()

        # Step 3: Shipping
        create_shipment.delay(order_id).get()

        return {"order_id": order_id, "status": "success"}

    except Exception as e:
        # Compensate (rollback)
        if payment_id:
            refund_payment.delay(payment_id)
            release_inventory.delay(order_id)
            cancel_order.delay(order_id)

        return {"error": str(e), "status": "failed"}

```

Phản Mẫu 1: Gọi Đồng Bộ Nối Chuỗi

```
# BAD: Synchronous chain
def process_order(order_id):
    order = get_order(order_id)           # 100ms
    user = get_user(order.user_id)        # 100ms
    payment = process_payment(order.total) # 500ms
    inventory = reserve_items(order.items) # 200ms
    shipping = create_shipment(order_id)   # 300ms

    return order
# Total: 1200ms (serial)

# GOOD: Parallel async
def process_order(order_id):
    order = get_order(order_id)

    # Parallel processing
    tasks = [
        get_user_task.delay(order.user_id),
        process_payment_task.delay(order.total),
        reserve_items_task.delay(order.items)
    ]

    results = [task.get() for task in tasks]

    # Finally, shipping (depends on above)
    create_shipment_task.delay(order_id)

    return order
# Total: ~500ms (parallel)
```

Phản Mẫu 2: Không Đặt Thời Gian Chờ (Timeout) Cho Lỗi Gọi Đồng Bộ

```
# BAD: No timeout
response = requests.get('https://external-api.com/data')

# GOOD: Always timeout
try:
    response = requests.get('https://external-api.com/data', timeout=5)
except requests.Timeout:
    # Fallback or queue for retry
    retry_later.delay('https://external-api.com/data')
    return {"status": "processing"}
```

Phản Mẫu 3: Không Xử Lý Sự Cố Của Hàng Đợi

```
# BAD: No error handling
@app.task
def send_email(user_id, subject, body):
    smtp.send(user_id, subject, body) # What if SMTP down?

# GOOD: Retry with backoff
@app.task(bind=True, max_retries=3)
def send_email(self, user_id, subject, body):
    try:
        smtp.send(user_id, subject, body)
    except SMTPException as exc:
        # Exponential backoff: 1min, 2min, 4min
        raise self.retry(exc=exc, countdown=2 ** self.request.retries * 60)
```

Giám Sát (Monitoring) & Khả Năng Quan Sát (Observability)

Các Chỉ Số (Metrics) Chính

Cho các API đồng bộ:

- Thời gian phản hồi (p50, p95, p99)
- Tỷ lệ lỗi
- Tần suất yêu cầu
- Tỷ lệ hết thời gian chờ

Cho các hàng đợi (queues) bất đồng bộ:

- Độ dài hàng đợi (tồn đọng)
- Thời gian xử lý mỗi tác vụ
- Tỷ lệ thất bại
- Tỷ lệ thư' lại
- Sức khỏe của worker

Cảnh Báo (Alerting)

```
# Alert khi queue backlog too high
if queue_length > 10000:
    alert("Queue backlog high - need more workers")

# Alert khi workers failing
if worker_failure_rate > 0.05: # 5%
    alert("High worker failure rate - check errors")

# Alert khi processing slow
if avg_processing_time > 60: # 60 seconds
    alert("Slow job processing - investigate")
```

Những Điểm Cốt Lỗi

Mẫu giao tiếp định hình hành vi của hệ thống:

Đồng bộ (Yêu cầu-Phản hồi):

- Đơn giản, phản hồi tức thì
- Kém với các thao tác chậm, liên kết chặt (*tight coupling*)
- **Dùng cho:** Đọc dữ liệu, thao tác nhanh, các API phục vụ trực tiếp người dùng

Bất đồng bộ (Hàng đợi thông điệp):

- Trải nghiệm tốt hơn, chịu được tăng đột biến (*spikes*), có cơ chế thử lại
- Phức tạp, nhất quán cuối cùng
- **Dùng cho:** Xử lý chậm, tác vụ nền (*background jobs*), tách rời (*decoupling*)

Hướng sự kiện:

- Liên kết lỏng, dễ mở rộng (*scalable*), linh hoạt
- Phức tạp nhất, khó gỡ lỗi (*debug*)
- **Dùng cho:** Vi dịch vụ, nhiều phản ứng với cùng các sự kiện

Những quy tắc vàng:

1. **Mặc định dùng đồng bộ** cho đơn giản
2. **Chuyển sang bất đồng bộ** khi trải nghiệm người dùng đòi hỏi
3. **Làm cho các thao tác bất đồng bộ trở nên lũy đẳng**

4. **Luôn đặt thời gian chờ** (*timeouts*) cho các lời gọi đồng bộ

5. **Giám sát sức khoẻ hàng đợi** thật nghiêm ngặt

Danh sách kiểm tra khi ra quyết định:

Trước khi chọn mẫu, hãy tự hỏi:

- Người dùng có cần kết quả ngay không?
- Thao tác mất bao lâu?
- Các phụ thuộc bên ngoài có đáng tin cậy không?
- Nhóm (*Team*) có kinh nghiệm với các mẫu bất đồng bộ (*async patterns*) không?

Bài tập thực hành:

Thiết kế mẫu giao tiếp cho các kịch bản (*scenarios*) sau:

1. Người dùng đổi mật khẩu
2. Tạo báo cáo hằng tháng
3. Xử lý thanh toán
4. Gửi thông báo đẩy (*push notification*) tới 1 triệu người dùng (*users*)
5. Tin nhắn chat thời gian thực (*real-time*)

Với mỗi kịch bản, hãy bảo vệ lựa chọn của bạn bằng các sự đánh đổi.

Luồng Dữ Liệu (*Data Flow*) & Điểm Nghẽn (*Bottlenecks*): Tìm Và Tối Ưu Điểm Nghẽn

Tôi còn nhớ lần đầu tiên bị gọi vào phòng họp khẩn cấp (*war room*) lúc 3 giờ sáng.

"Hệ thống trên môi trường thực tế (*production*) chậm kinh khủng! Người dùng (*Users*) phàn nàn thời gian phản hồi (*response time*) lên tới 10 giây!"

Cả nhóm (*team*) đang hoảng loạn. Ai cũng có giả thuyết riêng:

- Lập trình viên A: "Cơ sở dữ liệu (*Database*) chậm, cần nâng cấp máy chủ (*server*)!"
- Lập trình viên B: "Mạng bị trễ, cần thêm băng thông (*bandwidth*)!"
- Lập trình viên C: "Mã kém hiệu quả, cần tái cấu trúc (*refactor*)!"

Một kiến trúc sư lâu năm (*senior architect*) bước vào, lặng lẽ mở công cụ giám sát (*monitoring*). 5 phút sau, anh chỉ vào một biểu đồ:

"Nhóm kết nối (*Connection pool*) của Redis chỉ có 10 kết nối. Đang có 1000 yêu cầu (*requests*) phải chờ. Sửa cái này trước."

Tăng nhóm kết nối lên 100. Thời gian phản hồi giảm về 200ms. Vấn đề được giải quyết.

Bài học: Đừng tối ưu (*optimize*) bừa. **Đo lường, tìm điểm nghẽn (*bottleneck*), xử lý điểm nghẽn.**

Tại Sao Luồng Dữ Liệu & Điểm Nghẽn Quan Trọng?

Khi hệ thống chậm, 90% kỹ sư (*engineers*) làm sai một điều: **Tối ưu không đúng chỗ.**

Họ:

- Tối ưu đoạn mã vốn đã nhanh
- Thêm bộ nhớ đệm (*cache*) không cần thiết
- Nâng cấp phần cứng vốn không phải là điểm nghẽn

Kết quả: Tốn thời gian, tốn tiền, mà vấn đề vẫn còn nguyên.

Một kiến trúc sư (*architect*) **giỏi khác ở chỗ:** Họ biết **tìm điểm nghẽn** trước khi **tối ưu**.

Phân tích luồng dữ liệu chính là kỹ năng này. Nó giúp bạn:

- **Hình dung** một yêu cầu đi qua những thành phần nào
- **Đo lường** mỗi bước mất bao lâu
- **Xác định** điểm nghẽn thực sự
- **Tối ưu** đúng chỗ, tác động lớn

Luật Vàng: Hệ Thống Chậm Nhanh Bằng Thành Phần (*Component*) Chậm Nhất

Nguyên Lý Mất Xích Yếu Nhất (*The Weakest Link Principle*).

Hãy hình dung một dây chuyền sản xuất:

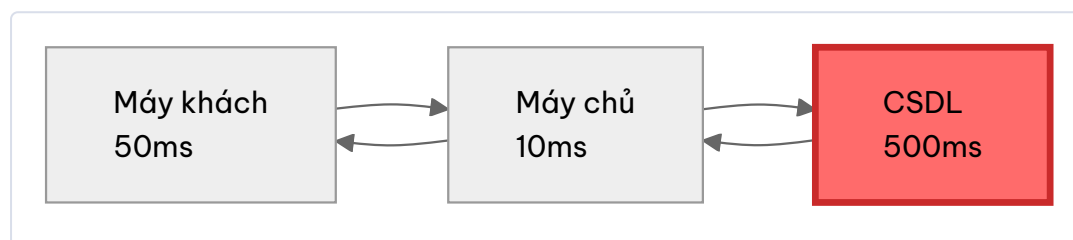
Bước 1: Cắt vải (1 phút/sản phẩm)
Bước 2: May (5 phút/sản phẩm) ← ĐIỂM NGHẼN
Bước 3: Đóng gói (30 giây/sản phẩm)

Thông lượng: 1 sản phẩm mỗi 5 phút

Tối ưu bước 1 từ 1 phút → 10 giây? **Chẳng có tác dụng gì.** Điểm nghẽn vẫn là bước 2.

Chỉ khi tối ưu bước 2 (điểm nghẽn), thông lượng (*throughput*) mới tăng.

Hệ thống phần mềm cũng vậy:



Tổng độ trễ (*latency*): $50 + 10 + 500 + 10 + 50 = 620\text{ms}$

Cơ sở dữ liệu chiếm $500\text{ms} / 620\text{ms} = 81\%$ tổng thời gian.

Tối ưu máy chủ từ 10ms → 1ms? Tiết kiệm được 18ms (cải thiện 3%).

Tối ưu cơ sở dữ liệu từ 500ms → 50ms? Tiết kiệm được 450ms (cải thiện 73%)!

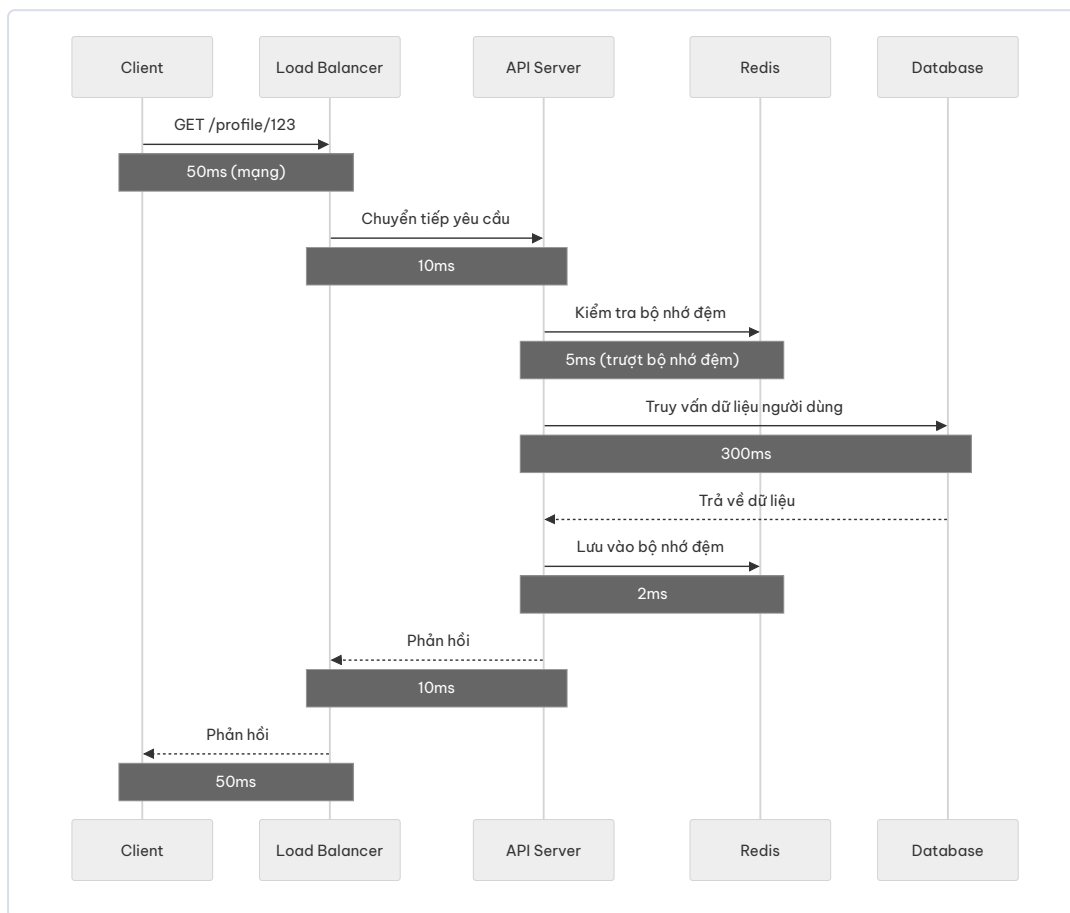
Bài học: Luôn tối ưu điểm nghẽn trước. Đó là nơi cho lợi tức đầu tư (ROI) lớn nhất.

Cách Tư Duy: Phân Tích Điểm Nghẽn Theo 3 Bước

Bước 1: Đo Thời Gian Mỗi Bước

Không đoán. Hãy đo.

Ví dụ: Người dùng tải trang hồ sơ



Phân tích chi tiết:

Máy khách → Bộ cân bằng tải:	50ms
Bộ cân bằng tải → API:	10ms
API → Kiểm tra bộ nhớ đệm:	5ms (trượt)
API → Truy vấn CSDL:	300ms ← ĐIỂM NGHẼN
API → Lưu vào bộ nhớ đệm:	2ms
API → Bộ cân bằng tải:	10ms
Bộ cân bằng tải → Máy khách:	50ms

Tổng: 427ms

CSDL: 300ms (70% tổng thời gian)

Bước 2: Tìm Điểm Nghẽn

Điểm nghẽn = Thành phần chiếm nhiều thời gian nhất.

Trong ví dụ trên: **Cơ sở dữ liệu (300ms / 427ms = 70%)**

Cách đo lường trên môi trường thực tế:

Công cụ 1: APM (Application Performance Monitoring - giám sát hiệu năng ứng dụng) (Application Performance Monitoring))

```
# New Relic, DataDog, hoặc tương tự
from apm import trace

@trace("load_profile")
def load_profile(user_id):
    with trace("cache_check"):
        user = cache.get(f"user:{user_id}")

    if not user:
        with trace("db_query"): # Measure this
            user = db.query("SELECT * FROM users WHERE id = ?", user_id)

        with trace("cache_store"):
            cache.set(f"user:{user_id}", user)

    return user

# APM dashboard sẽ show:
# - load_profile: 307ms total
# - cache_check: 5ms
# - db_query: 300ms ← BOTTLENECK FOUND
# - cache_store: 2ms
```

Công cụ 2: Log đo thời gian tự viết

```

import time

def load_profile(user_id):
    start = time.time()

    # Step 1: Cache check
    cache_start = time.time()
    user = cache.get(f"user:{user_id}")
    cache_time = time.time() - cache_start

    # Step 2: DB query if cache miss
    if not user:
        db_start = time.time()
        user = db.query("SELECT * FROM users WHERE id = ?", user_id)
        db_time = time.time() - db_start

        cache.set(f"user:{user_id}", user)
    else:
        db_time = 0

    total_time = time.time() - start

    # Log breakdown
    logger.info(f"Profile load: {total_time:.3f}s | "
                f"Cache: {cache_time:.3f}s | "
                f"DB: {db_time:.3f}s")

    return user

# Logs show:
# Profile load: 0.307s | Cache: 0.005s | DB: 0.300s

```

Công cụ 3: Phân tích hiệu năng (*profiling*) truy vấn cơ sở dữ liệu

```

-- PostgreSQL
EXPLAIN ANALYZE
SELECT * FROM users WHERE email = 'john@example.com';

-- Output shows:
Seq Scan on users (cost=0.00..18334.00 rows=1 width=123)
  (actual time=0.011..285.234 rows=1 loops=1)
    Filter: (email = 'john@example.com')
    Rows Removed by Filter: 999999
Planning Time: 0.082 ms
Execution Time: 285.256 ms ← 285ms! Missing index!

```

Bước 3: Tối Ưu Chính Điểm Nghẽn Đó

Chỉ tối ưu thành phần là điểm nghẽn.

Trong ví dụ, cơ sở dữ liệu là điểm nghẽn (300ms).

Các phương án tối ưu:

Phương án A: Thêm chỉ mục (index) (Ít công sức, Tác động lớn)

```
-- Before: Full table scan, 300ms
SELECT * FROM users WHERE email = 'john@example.com';

-- Add index
CREATE INDEX idx_users_email ON users(email);

-- After: Index scan, 5ms
-- 60x faster!
```

Phương án B: Lưu đệm mạnh tay hơn

```
# Before: Cache miss → DB query (300ms)

# After: Pre-warm cache
def warm_cache():
    popular_users = db.query("SELECT * FROM users ORDER BY login_count DESC
LIMIT 1000")
    for user in popular_users:
        cache.set(f"user:{user.id}", user, ttl=3600)

# Result: Cache hit rate 95% → Only 5% queries hit DB
```

Phương án C: Tối ưu truy vấn (query)

```
-- Before: Select all columns (waste)
SELECT * FROM users WHERE email = 'john@example.com';

-- After: Select only needed columns
SELECT id, name, email, avatar FROM users WHERE email = 'john@example.com';

-- Smaller data → Faster transfer → 300ms → 200ms
```

Đo lại sau khi tối ưu:

Ban đầu:

Tổng: 427ms | CSDL: 300ms (70%)

Sau khi thêm chỉ mục:

Tổng: 132ms | CSDL: 5ms (4%)

Cải thiện: 427ms → 132ms (nhanh hơn 69%!)

Điểm mấu chốt: Tối ưu điểm nghẽn = Tác động lớn nhất với ít công sức nhất.

Các Điểm Nghẽn Thường Gặp & Cách Phát Hiện

Điểm Nghẽn 1: Truy Vấn Cơ Sở Dữ Liệu (*Database Queries*)

Triệu chứng:

- Độ trễ truy vấn cao (> 100ms)
- CPU/IO của cơ sở dữ liệu luôn ở mức tối đa
- Thời gian phản hồi chậm khi chịu tải (*load*)

Phát hiện:

```
-- PostgreSQL: Find slow queries
SELECT
    query,
    calls,
    total_time,
    mean_time,
    max_time
FROM pg_stat_statements
ORDER BY mean_time DESC
LIMIT 10;

-- Output shows queries taking > 100ms average
```

Giải pháp:

1. Thêm chỉ mục (cách sửa phổ biến nhất)
 - Trước: 500ms quét toàn bộ bảng
 - Sau: 5ms quét theo chỉ mục
2. Tối ưu cấu trúc truy vấn
 - Tránh SELECT *
 - Dùng JOIN phù hợp
 - Thêm LIMIT khi có thể
3. Thêm bộ nhớ đệm cho truy vấn
 - Lưu đệm các truy vấn thường dùng
 - Giảm tải CSDL 80-90%
4. Tối ưu cơ sở dữ liệu
 - Phân tích kế hoạch thực thi truy vấn
 - Cập nhật số liệu thống kê
 - Vacuum/tối ưu các bảng
5. Thêm bản sao chỉ đọc (phương án cuối cùng)
 - Mở rộng lượt đọc theo chiều ngang
 - Định tuyến truy vấn đọc sang các bản sao

Điểm Nghẽn 2: Độ Trễ Mạng (Network Latency)

Triệu chứng:

- Độ trễ cao giữa các dịch vụ (services)
- Hết thời gian chờ (Timeouts) khi chịu tải
- Vấn đề do khoảng cách địa lý

Phát hiện:

```
import time
import requests

def measure_network_latency():
    start = time.time()
    response = requests.get("https://api.example.com/health")
    latency = time.time() - start

    print(f"Network latency: {latency * 1000:.0f}ms")

# Run from different regions:
# US East → US West: 50ms
# US East → Asia: 200ms
# Asia → Europe: 300ms
```

Giải pháp:

1. CDN cho tài nguyên tĩnh
 - Ảnh, CSS, JS
 - Phục vụ từ các điểm biên
 - 300ms → 20ms
2. Triển khai theo khu vực
 - Đặt máy chủ gần người dùng hơn
 - Người dùng Mỹ → máy chủ ở Mỹ
 - Người dùng châu Á → máy chủ ở châu Á
3. Giảm kích thước dữ liệu gửi kèm
 - Nén phản hồi (gzip)
 - Bỏ dữ liệu không cần thiết
 - Phân trang
4. Dùng nhóm kết nối (connection pooling)
 - Tái sử dụng kết nối
 - Tránh chi phí bắt tay (handshake)
5. HTTP/2 hoặc HTTP/3
 - Ghép kênh (multiplexing)
 - Nén tiêu đề (header)

Điểm Nghẽn 3: Logic Ứng Dụng

Triệu chứng:

- Mức sử dụng CPU cao trên các máy chủ ứng dụng
- Thời gian xử lý chậm
- Mã thực thi mất nhiều thời gian

Phát hiện:


```

import cProfile
import pstats

# Profile slow function
def profile_function():
    profiler = cProfile.Profile()
    profiler.enable()

    # Run function
    result = slow_function()

    profiler.disable()

    # Print stats
    stats = pstats.Stats(profiler)
    stats.sort_stats('cumulative')
    stats.print_stats(10) # Top 10 slowest functions

# Output shows:
# Function A: 500ms (bottleneck!)
# Function B: 50ms
# Function C: 10ms

```

Giải pháp:

1. Tối ưu thuật toán
 - $O(n^2) \rightarrow O(n \log n)$
 - Dùng cấu trúc dữ liệu tốt hơn
 - Tránh vòng lặp lồng nhau
2. Xử lý bất đồng bộ
 - Chuyển việc nặng sang chạy nền
 - Dùng hàng đợi thông điệp
 - Không chặn yêu cầu của người dùng
3. Lưu đệm các phép tính tốn kém
 - Ghi nhớ kết quả (memoization)
 - Lưu đệm kết quả đã tính
4. Mở rộng theo chiều ngang
 - Thêm máy chủ ứng dụng
 - Cân bằng tải
5. Tối ưu ngôn ngữ/môi trường chạy
 - Dùng ngôn ngữ biên dịch cho các đường xử lý "nóng"
 - Biên dịch JIT
 - Tối ưu dựa trên kết quả phân tích hiệu năng

Điểm Nghẽn 4: Lời Gọi API Bên Ngoài

Triệu chứng:

- Phải chờ các dịch vụ bên thứ ba
- API bên ngoài bị hết thời gian chờ
- Độ trễ dây chuyền

Phát hiện:

```
import httpx
import time

async def measure_external_apis():
    apis = [
        "https://api.stripe.com/health",
        "https://api.twilio.com/health",
        "https://maps.googleapis.com/health"
    ]

    async with httpx.AsyncClient() as client:
        for api in apis:
            start = time.time()
            try:
                response = await client.get(api, timeout=5.0)
                latency = time.time() - start
                print(f"{api}: {latency * 1000:.0f}ms")
            except httpx.TimeoutException:
                print(f"{api}: TIMEOUT (> 5000ms)")

# Output:
# Stripe API: 150ms
# Twilio API: 3500ms ← BOTTLENECK
# Google Maps: 80ms
```

Giải pháp:

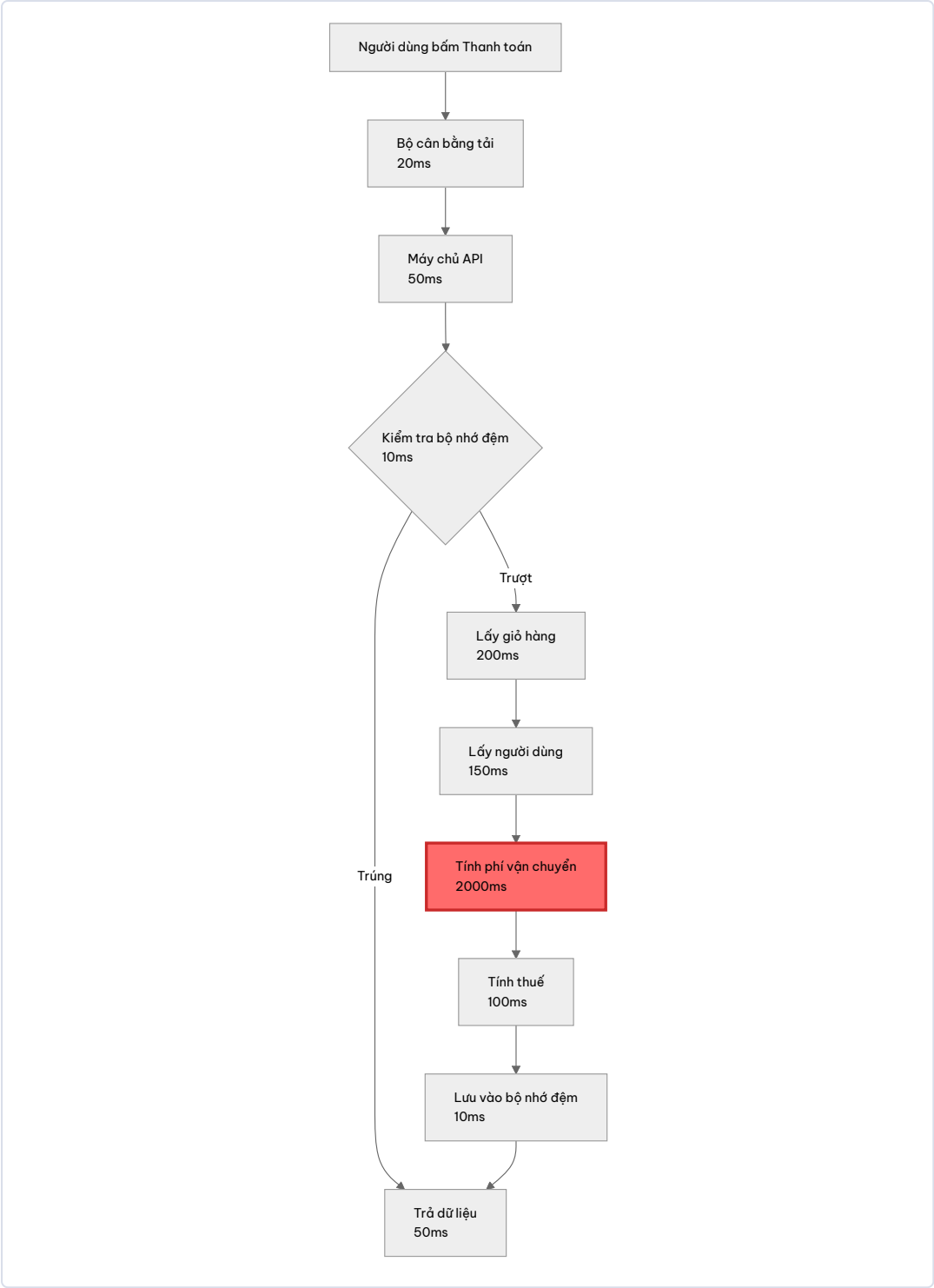
1. Gọi bất đồng bộ (song song)
Trước: Tuần tự ($3.5s + 0.15s + 0.08s = 3.73s$)
stripe_data = call_stripe()
twilio_data = call_twilio()
maps_data = call_google_maps()

Sau: Song song ($\max(3.5s, 0.15s, 0.08s) = 3.5s$)
results = await asyncio.gather(
 call_stripe(),
 call_twilio(),
 call_google_maps()
)
2. Lưu đệm phản hồi của API
 - Lưu đệm với TTL phù hợp
 - Giảm 90% lời gọi ra bên ngoài
3. Mẫu bộ ngắt mạch (circuit breaker)
 - Thất bại nhanh khi API sập
 - Không chờ đến khi hết thời gian chờ
4. Thời gian chờ (timeout)
 - Đặt thời gian chờ ngắn, quyết liệt
 - Không để API chậm chạp hệ thống
5. Cơ chế dự phòng (fallback)
 - Suy giảm có kiểm soát
 - Trả về dữ liệu đã lưu đệm/mặc định

Ví Dụ Thực Tế: Thanh Toán Trong Thương Mại Điện Tử

Kịch bản: Trang thanh toán tải chậm (3 giây)

Bước 1: Đo Luồng Dữ Liệu



Phân tích chi tiết:

Bộ cân bằng tải:	20ms
Xử lý API:	50ms
Kiểm tra bộ nhớ đệm:	10ms (trượt)
Lấy giỏ hàng từ CSDL:	200ms
Lấy người dùng từ CSDL:	150ms
Tính phí vận chuyển:	2000ms ← ĐIỂM NGHẼN (67% tổng)
Tính thuế:	100ms
Lưu vào bộ nhớ đệm:	10ms
Trả về máy khách:	50ms

Tổng: ~2,590ms

API vận chuyển: 2,000ms (77% tổng)

Bước 2: Xác Định Điểm Nghẽn

Lời gọi API vận chuyển (2000ms) rõ ràng là điểm nghẽn.

Tại sao lại chậm đến vậy?

- API bên ngoài (nhà cung cấp dịch vụ vận chuyển bên thứ ba)
- Tính giá cước theo thời gian thực từ 5 hãng vận chuyển
- Độ trễ mạng + thời gian tính toán

Bước 3: Tối Ưu Điểm Nghẽn

Giải pháp 1: Lưu đệm giá cước vận chuyển

```

# Before: Call API every time
def calculate_shipping(address, weight):
    return shipping_api.get_rates(address, weight) # 2000ms

# After: Cache by address + weight
def calculate_shipping(address, weight):
    cache_key = f"shipping:{hash(address)}:{weight}"

    rates = cache.get(cache_key)
    if rates:
        return rates # 5ms (cache hit)

    rates = shipping_api.get_rates(address, weight) # 2000ms (cache miss)
    cache.set(cache_key, rates, ttl=3600) # Cache 1 hour

    return rates

# Result: Cache hit rate 85%
# Average time: 0.85 * 5ms + 0.15 * 2000ms = 304ms
# Improvement: 2000ms → 304ms (85% faster!)

```

Giải pháp 2: Tính toán bất đồng bộ (*async*)

```

# Before: User waits for shipping calculation
def checkout():
    cart = get_cart()
    user = get_user()
    shipping = calculate_shipping() # User waits 2s
    tax = calculate_tax()
    return render_page(cart, user, shipping, tax)

# After: Calculate shipping in background
def checkout():
    cart = get_cart()
    user = get_user()

    # Return page immediately với estimated shipping
    estimated_shipping = get_estimated_shipping(user.zipcode)

    # Calculate real shipping in background
    task_queue.add({
        "type": "calculate_shipping",
        "user_id": user.id,
        "cart_id": cart.id
    })

    return render_page(cart, user, estimated_shipping, tax)

# User sees page immediately (500ms)
# Real shipping rates update via WebSocket (2s later)

```

Giải pháp 3: Gọi API song song

```

# Before: Sequential (2000 + 100 = 2100ms)
shipping = calculate_shipping()
tax = calculate_tax()

# After: Parallel (max(2000, 100) = 2000ms)
shipping, tax = await asyncio.gather(
    calculate_shipping(),
    calculate_tax()
)

# Saved: 100ms

```

Kết quả sau khi tối ưu:

Ban đầu: 2,590ms
Sau khi lưu đệm (85% trúng): 590ms
Sau khi chuyển sang bất đồng bộ: 500ms (cảm nhận)

Cải thiện: 2590ms → 500ms (nhanh hơn 81%!)

Mức hài lòng của người dùng: 

Phản Mẫu (Anti-Patterns): Những Sai Lầm Khi Tối Ưu

Sai Lầm 1: Tối Ưu Quá Sớm (Premature Optimization)

```
# BAD: Optimize before measuring
def get_users():
    # Add complex caching without knowing if DB is slow
    # Add pagination without knowing if data is large
    # Use redis cluster without measuring redis load
    pass

# GOOD: Measure first
def get_users():
    # Measure: DB query takes 500ms → bottleneck found
    # Then optimize: Add cache → 5ms
    pass
```

Bài học: "Tối ưu quá sớm là cội nguồn của mọi tội lỗi" - Donald Knuth

Đo lường → Tìm điểm nghẽn → Rồi mới tối ưu.

Sai Lầm 2: Tối Ưu Sai Thành Phần

```
# Example: API slow (1000ms total)
# Breakdown:
# - Server processing: 50ms
# - Database query: 950ms ← Real bottleneck

# BAD: Optimize server code
def handle_request():
    # Refactor code: 50ms → 10ms
    # Total: 1000ms → 960ms (4% improvement)
    # Waste of time!
    pass

# GOOD: Optimize database
# Add index: 950ms → 50ms
# Total: 1000ms → 100ms (90% improvement)
```

Bài học: Tối ưu điểm nghẽn, không phải các thành phần ngẫu nhiên.

Sai Lầm 3: Tối Ưu Mà Không Đo Lại

```
# Add optimization
add_cache_layer()

# BAD: Assume it worked, move on

# GOOD: Measure improvement
before = measure_latency() # 500ms
add_cache_layer()
after = measure_latency() # 50ms

improvement = (before - after) / before * 100
print(f"Improvement: {improvement:.1f}%") # 90%

# If improvement < 10%, optimization không có effect
# Investigate why hoặc try different approach
```

Bài học: Luôn kiểm chứng xem việc tối ưu có hiệu quả không.

Sai Lầm 4: Tối Ưu Quá Mức

```
# API currently: 100ms (already fast)
# User perception threshold: ~100ms

# BAD: Spend 2 weeks optimizing 100ms → 50ms
# Users don't notice difference
# Developer time wasted

# GOOD: Optimize only if:
# - Current performance < acceptable
# - Optimization has high ROI
# - User experience improves noticeably
```

Bài học: Đủ tốt > Hoàn hảo. Biết khi nào nên dừng.

Bài Tập Thực Hành: Phân Tích Hệ Thống Của Bạn

Bài tập 1: Truy Vết Một Yêu Cầu

Chọn một điểm cuối (*endpoint*) trong hệ thống bạn đang làm.

Các bước:

1. Gắn mã đo thời gian vào code:

```
import time

def track_time(func):
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        duration = time.time() - start
        print(f"{func.__name__}: {duration * 1000:.0f}ms")
        return result
    return wrapper

@track_time
def get_product(product_id):
    user = get_current_user() # ?ms
    product = get_product_from_db(product_id) # ?ms
    reviews = get_reviews(product_id) # ?ms
    recommendations = get_recommendations() # ?ms
    return render(product, reviews, recommendations)
```

2. Chạy và ghi lại thời gian:

```
get_current_user: 50ms
get_product_from_db: 200ms
get_reviews: 150ms
get_recommendations: 800ms ← ĐIỂM NGHẼN
render: 30ms

Tổng: 1,230ms
```

3. Xác định điểm nghẽn:

get_recommendations chiếm $800/1230 = 65\%$ tổng thời gian.

4. Tối ưu điểm nghẽn:

Thêm bộ nhớ đệm, xử lý bất đồng bộ, hoặc cải thiện thuật toán.

5. Đo lại:

```
Trước: 1,230ms
Sau: 430ms (nhanh hơn 65%)
```

Bài tập 2: Tìm Các Điểm Cuối (Endpoints) Chậm Nhất Của Bạn

Truy vấn công cụ APM hoặc log truy cập (access logs):

```
-- Find slowest API endpoints (if logging to DB)
SELECT
    endpoint,
    COUNT(*) as request_count,
    AVG(response_time_ms) as avg_time,
    MAX(response_time_ms) as max_time
FROM api_logs
WHERE created_at > NOW() - INTERVAL '1 day'
GROUP BY endpoint
ORDER BY avg_time DESC
LIMIT 10;

-- Output:
-- /api/search: 2,500ms avg (BOTTLENECK)
-- /api/checkout: 1,200ms avg
-- /api/products: 300ms avg
```

Tập trung tối ưu 3 điểm cuối chậm nhất.

Những Điểm Cốt Lõi

Luật vàng của việc tối ưu điểm nghẽn:

1. **Đo lường trước** - Không đoán, đo thực tế
2. **Tìm điểm nghẽn** - Thành phần chậm nhất
3. **Tối ưu điểm nghẽn** - Đừng tối ưu ngẫu nhiên
4. **Đo lại** - Kiểm chứng mức cải thiện
5. **Lặp lại** - Tìm điểm nghẽn tiếp theo

Các điểm nghẽn thường gặp:

- **Cơ sở dữ liệu** - Truy vấn chậm, thiếu chỉ mục (*indexes*)
- **Mạng** - Độ trễ, API bên ngoài
- **Ứng dụng** - Thuật toán kém hiệu quả
- **Dịch vụ bên ngoài** - API của bên thứ ba

Các chiến lược tối ưu:

- **Chỉ mục** - Truy vấn nhanh hơn 100 lần
- **Lưu đệm** (*Caching*) - Đọc nhanh hơn 10-100 lần
- **Xử lý bất đồng bộ** (*Async processing*) - Không chặn người dùng
- **Tối ưu truy vấn** - Viết SQL tốt hơn
- **Mở rộng theo chiều ngang** (*Horizontal scaling*) - Thêm năng lực xử lý

Hãy nhớ: Hệ thống chỉ nhanh bằng thành phần chậm nhất. Tối ưu điểm nghẽn = Tác động lớn nhất.

Mô hình tư duy (*Mental model*):

Hãy tư duy như một thám tử:

1. Có vấn đề (hệ thống chậm)
2. Thu thập bằng chứng (các phép đo)
3. Tìm thủ phạm (điểm nghẽn)
4. Giải quyết (tối ưu)
5. Kiểm chứng (đo lại)

Bước tiếp theo: Áp dụng vào hệ thống đang chạy thực tế của bạn. Đo lường, tìm điểm nghẽn, tối ưu. Lặp lại.

Khái Niệm Quan Trọng Trong Thiết Kế Hệ Thống (System Design)

Sau khi học về các thành phần (*components*), các mẫu giao tiếp (*communication patterns*) và luồng dữ liệu (*data flow*), giờ là lúc củng cố kiến thức bằng những khái niệm quan trọng nhất trong thiết kế hệ thống.

Tôi còn nhớ lần đầu tiên PM hỏi: "Hệ thống của chúng ta có đạt 99,9% thời gian hoạt động (*uptime*) không?"

Tôi trả lời: "Có chứ, máy chủ (*server*) hiếm khi sập mà."

Anh ấy cười: "Em có biết 99,9% nghĩa là gì không? Và chi phí (*cost*) để đạt 99,99% là bao nhiêu?"

Tôi im lặng. Đó là lúc tôi nhận ra: **Không hiểu rõ các khái niệm = Không thể đưa ra quyết định đúng.**

Bài này sẽ đi sâu vào 4 khái niệm mà mọi kiến trúc sư (*architect*) phải nắm vững.

Tại Sao Các Khái Niệm Này Quan Trọng?

Trong các buổi phỏng vấn (*interviews*) thiết kế hệ thống, bạn sẽ nghe:

"Hãy thiết kế một hệ thống có độ trễ (*latency*) thấp và thông lượng (*throughput*) cao."

Nếu bạn không hiểu rõ độ trễ và thông lượng, bạn sẽ thiết kế sai. Hoặc tệ hơn, thiết kế đúng nhưng không giải thích được.

Trong thực tế:

- **Độ trễ** quyết định trải nghiệm người dùng (*user experience*)
- **Thông lượng** quyết định năng lực hệ thống (*system capacity*)
- **Tính sẵn sàng** (*Availability*) quyết định SLA với khách hàng
- **Định lý CAP** (*CAP Theorem*) quyết định việc lựa chọn cơ sở dữ liệu (*database*)

Hiểu các khái niệm = Đưa ra quyết định có cơ sở.

Độ Trễ và Thông Lượng: Hai Chỉ Số (Metrics) Khác Nhau

Định Nghĩa

Độ trễ:

- Thời gian để xử lý **1 yêu cầu** (*request*)
- Đơn vị: mili giây (ms)
- Góc nhìn người dùng: "Tôi phải đợi bao lâu?"

Thông lượng:

- Số yêu cầu (*requests*) xử lý được **trong 1 giây**
- Đơn vị: yêu cầu/giây (req/s) hoặc giao dịch (*transactions*)/giây (TPS)
- Góc nhìn hệ thống: "Hệ thống xử lý được bao nhiêu yêu cầu cùng lúc?"

Tại Sao Phải Phân Biệt?

Nhiều người nghĩ: "Độ trễ thấp = Thông lượng cao."

Sai rồi. Chúng là hai thứ độc lập.

Phép So Sánh Đời Thực

Xe buýt:

Độ trễ: CAO (đợi xe buýt, xe dừng nhiều điểm)

- Từ nhà đến công ty: 60 phút

Thông lượng: CAO (chở nhiều người cùng lúc)

- 1 xe buýt chở 50 người
- 1 giờ có 10 chuyến = 500 người/giờ

Taxi:

Độ trễ: THẤP (đi ngay, đường thẳng)

- Từ nhà đến công ty: 20 phút

Thông lượng: THẤP (chỉ chở 4 người)

- 1 taxi chở 4 người
- 1 giờ có 3 chuyến = 12 người/giờ

Ví Dụ Trong Thiết Kế Hệ Thống

Kịch bản 1: Hệ thống xử lý theo lô (Batch Processing)

```
# Process 1000 images
for image in images:
    process(image) # Takes 10 seconds each
```

Latency per image: 10 seconds (HIGH)
Total time: $1000 * 10s = 10,000s \approx 2.8$ hours
Throughput: $1000 / 10,000s = 0.1$ images/second (LOW)

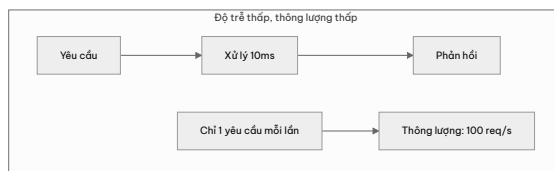
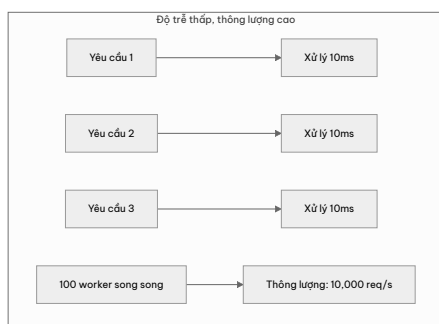
Kịch bản 2: Xử lý song song (Parallel Processing)

```
# Process 1000 images in parallel (100 workers)
workers = create_workers(100)
for worker in workers:
    worker.process(batch_of_10_images)
```

Latency per image: still 10 seconds
Total time: $1000 / 100 * 10s = 100s$
Throughput: $1000 / 100s = 10$ images/second (HIGH!)

Bài học: Tăng thông lượng bằng song song hoá (*parallelization*), nhưng độ trễ của từng phần tử không thay đổi.

Sơ Đồ: Độ Trễ và Thông Lượng



Độ trễ = thời gian xử lý 1 yêu cầu. Thông lượng = số lượng yêu cầu xử lý được đồng thời.

Các Đánh Đổi (Trade-offs)

Tối ưu cho độ trễ:

Kỹ thuật:

- Lưu đệm (giảm số lần gọi CSDL)
- Chỉ mục cơ sở dữ liệu
- CDN (giảm độ trễ mạng)
- Tối ưu mã

Chi phí:

- Độ phức tạp
- Chi phí hạ tầng

Tối ưu cho thông lượng:

Kỹ thuật:

- Mở rộng theo chiều ngang (thêm máy chủ)
- Xử lý bất đồng bộ (hàng đợi)
- Gộp kết nối (connection pooling)
- Cân bằng tải

Chi phí:

- Nhiều máy chủ hơn = chi phí cao hơn
- Độ phức tạp của hệ thống phân tán

Khi Nào Ưu Tiên Cái Nào?

Ưu tiên độ trễ khi:

- Các tính năng hướng tới người dùng (phản hồi API, tải trang)
- Ứng dụng thời gian thực (*real-time*) (game, gọi video)
- Trải nghiệm người dùng là yếu tố then chốt

Ưu tiên thông lượng khi:

- Xử lý theo lô (đường ống dữ liệu (*data pipelines*))
- Tác vụ nền (*Background jobs*)
- Hệ thống khối lượng lớn (hàng triệu yêu cầu)

Lý tưởng: Độ trễ thấp VÀ thông lượng cao. Nhưng thường phải đánh đổi (*trade-off*).

Góc Nhìn Cá Nhân

Tôi từng tối ưu một API từ 500ms xuống 50ms (cải thiện độ trễ gấp 10 lần).

Người dùng hài lòng? Có.

Nhưng thông lượng vẫn chỉ 1.000 req/s. Khi lưu lượng (traffic) tăng đột biến (spike) lên 5.000 req/s → Hệ thống quá tải.

Bài học rút ra: Tối ưu cả hai. Nhưng phải biết đâu là điểm nghẽn (bottleneck) thực sự.

Tính Sẵn Sàng: Bạn Cần Bao Nhiêu "Số 9"?

Định Nghĩa

Tính sẵn sàng (tính khả dụng): Phần trăm thời gian hệ thống hoạt động bình thường.

Tính sẵn sàng = Thời gian hoạt động / (Thời gian hoạt động + Thời gian ngừng hoạt động)

Các Mức Tính Sẵn Sàng

Mức	Tính sẵn sàng	Thời gian ngừng hoạt động (Downtime)/năm	Thời gian ngừng hoạt động/tháng	Tình huống sử dụng (Use Case)
90%	90%	36,5 ngày	3 ngày	Phát triển/Kiểm thử
99%	"Hai số 9"	3,65 ngày	7,2 giờ	Công cụ nội bộ
99,9%	"Ba số 9"	8,76 giờ	43 phút	Thương mại điện tử
99,99%	"Bốn số 9"	52 phút	4,3 phút	Ngân hàng, thanh toán
99,999%	"Năm số 9"	5,26 phút	26 giây	Hệ thống sống còn

Tại Sao Mỗi "9" Lại Quan Trọng?

99% và 99,9%:

- 99%: 7.2 giờ ngừng hoạt động/tháng
 - Có thể sập trọn một ngày làm việc
 - Chấp nhận được cho công cụ quản trị nội bộ
- 99.9%: 43 phút ngừng hoạt động/tháng
 - Sập < 1 giờ/tháng
 - Mức tối thiểu cho ứng dụng hướng tới khách hàng

99,9% và 99,99%:

- 99.9%: 43 phút ngừng hoạt động/tháng
 - Khung bảo trì hàng tuần vẫn ổn
- 99.99%: 4.3 phút ngừng hoạt động/tháng
 - Không thể lên lịch bảo trì
 - Cần triển khai không gián đoạn (zero-downtime)

Chi Phí Của Mỗi "Số 9"

Sự thật then chốt: Mỗi "số 9" thêm vào, chi phí tăng gấp đôi (hoặc hơn).

- 99%:
 - 1 máy chủ
 - Không có dự phòng
 - Chi phí: \$100/tháng
- 99.9%:
 - Nhiều máy chủ
 - Bộ cân bằng tải
 - Giám sát
 - Chi phí: \$500/tháng
- 99.99%:
 - Triển khai đa vùng
 - Tự động chuyển đổi dự phòng
 - Đội trực 24/7
 - Chi phí: \$5,000/tháng
- 99.999%:
 - Hạ tầng toàn cầu
 - Kỹ thuật hỗn loạn (chaos engineering)
 - Đội SRE chuyên trách
 - Chi phí: \$50,000+/tháng

Câu Hỏi Tư Duy Quan Trọng

"Bạn THỰC SỰ cần bao nhiêu số 9?"

Tư duy sai:

"Chúng ta cần 99.999% vì muốn tốt nhất"

Tư duy đúng:

"Chúng ta cần 99.9% vì:

- Đây là trang thương mại điện tử
- Ngừng hoạt động 43 phút/tháng là chấp nhận được
- Khách hàng không bị ảnh hưởng lớn
- Chi phí cho 99.99% tăng 10 lần nhưng lợi ích chỉ thêm 40 phút/tháng"

Ví Dụ Thực Tế

Stripe (Thanh toán):

- Mục tiêu: 99,99%+ (4 số 9)
- Tại sao? Giao dịch tài chính, khách hàng phụ thuộc vào nó
- Chi phí: Xứng đáng vì thời gian ngừng hoạt động = doanh thu bị mất của các nhà bán hàng

Blog cá nhân:

- Mục tiêu: 99% là ổn
- Tại sao? Không quan trọng, người đọc có thể quay lại sau
- Chi phí: Tiết kiệm tiền hạ tầng (*infrastructure*)

Netflix:

- Mục tiêu: 99,9%+
- Tại sao? Dịch vụ giải trí hướng tới khách hàng
- Chiến lược: Suy giảm có kiểm soát (*Graceful degradation*) (một số tính năng lỗi, ứng dụng vẫn chạy)

Làm Sao Để Đạt Tính Sẵn Sàng Cao (*High Availability*)

Các khối xây dựng:

1. Dự phòng (*Redundancy*)

Một máy chủ: 1 máy lỗi = sập 100%
Nhiều máy chủ: 1 máy lỗi = các máy khác gánh thay

2. Cân bằng tải (Load Balancing)

Phân phối lưu lượng → Không có điểm lỗi duy nhất

3. Kiểm tra sức khỏe (Health Checks)

Phát hiện sự cố nhanh → Chuyển hướng lưu lượng đi nơi khác

4. Tự động mở rộng (Auto-scaling)

Xử lý lưu lượng tăng đột biến → Không bị quá tải

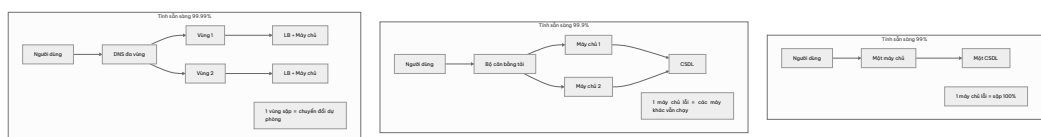
5. Triển khai đa vùng (Multi-region Deployment)

1 vùng sập → Chuyển đổi dự phòng sang vùng khác

6. Giám sát (Monitoring) & Cảnh báo (Alerting)

Phát hiện vấn đề trước khi người dùng nhận ra

Sơ Đồ: Kiến Trúc Tính Sẵn Sàng



Càng nhiều "số 9", càng cần nhiều dự phòng và độ phức tạp (complexity) càng cao.

Quan Điểm Mạnh Mẽ Của Tôi

Đừng thiết kế thừa (over-engineer).

Hãy bắt đầu với 99,9%. Đủ cho hầu hết doanh nghiệp.

Chỉ nhắm tới 99,99%+ khi:

- Nhu cầu đã được chứng minh (khách hàng phàn nàn về thời gian ngừng hoạt động)
- Tác động tài chính rõ ràng
- Ngân sách cho phép

Hãy nhớ: **Tính sẵn sàng hoàn hảo là điều không thể**. Ngay cả AWS, Google cũng có sự cố ngừng dịch vụ (*outages*).

Định Lý CAP: Chọn 2 Trong 3

Định Nghĩa

Định lý CAP nói rằng trong một hệ thống phân tán (*distributed system*), bạn chỉ có thể đảm bảo 2/3:

- **C (Tính nhất quán (Consistency))**: Mọi thao tác đọc đều nhận được lần ghi gần nhất (hoặc lỗi)
- **A (Tính sẵn sàng)**: Mọi yêu cầu đều nhận được phản hồi (*response*) (không lỗi)
- **P (Khả năng chịu phân vùng (Partition Tolerance))**: Hệ thống vẫn hoạt động khi có phân vùng mạng (*network partition*)

Tại Sao Chỉ Chọn Được 2?

Điểm mấu chốt: Phân vùng mạng (*Network partitions*) LUÔN xảy ra trong hệ thống phân tán (*distributed systems*).

Server A <--X--> Server B
(Phân vùng mạng: không giao tiếp được với nhau)

Khi phân vùng xảy ra, bạn PHẢI chọn:

- Tính nhất quán: Từ chối yêu cầu cho đến khi phân vùng được khắc phục
- Tính sẵn sàng: Chấp nhận yêu cầu nhưng dữ liệu có thể không nhất quán

Vì P (Khả năng chịu phân vùng) là **BẮT BUỘC** → Thực tế bạn chỉ chọn giữa C hoặc A.

Hệ Thống CP: Tính Nhất Quán + Khả Năng Chịu Phân Vùng

Hành vi:

Phân vùng mạng xảy ra

- Hệ thống từ chối thao tác ghi để duy trì tính nhất quán
- Người dùng thấy lỗi
- Khi phân vùng được khắc phục, dữ liệu nhất quán

Ví dụ:

- Hệ thống ngân hàng
- Xử lý thanh toán
- Quản lý tồn kho (ngăn bán vượt tồn kho (*overselling*))
- HBase, MongoDB (trong một số cấu hình)

Đánh đổi:

Dữ liệu luôn đúng

Tính sẵn sàng thấp hơn (lỗi trong lúc phân vùng)

Tình huống sử dụng:

Số dư tài khoản ngân hàng

Người dùng A: Rút \$100

Người dùng B: Kiểm tra số dư

CP đảm bảo:

- Số dư luôn đúng
- Không có tiền "ma"
- Có thể tạm thời báo lỗi

Hệ Thống AP: Tính Sẵn Sàng + Khả Năng Chịu Phân Vùng

Hành vi:

Phân vùng mạng xảy ra

- Hệ thống chấp nhận đọc/ghi ở cả hai phía
- Người dùng không bao giờ thấy lỗi
- Khi phân vùng được khắc phục, dữ liệu có thể xung đột

Ví dụ:

- Bảng tin (*Feeds*) mạng xã hội (Facebook, Twitter)
- DNS
- Cassandra, DynamoDB

- Giỏ hàng

Đánh đổi:

Luôn sẵn sàng
Nhất quán cuối cùng (tạm thời không nhất quán)

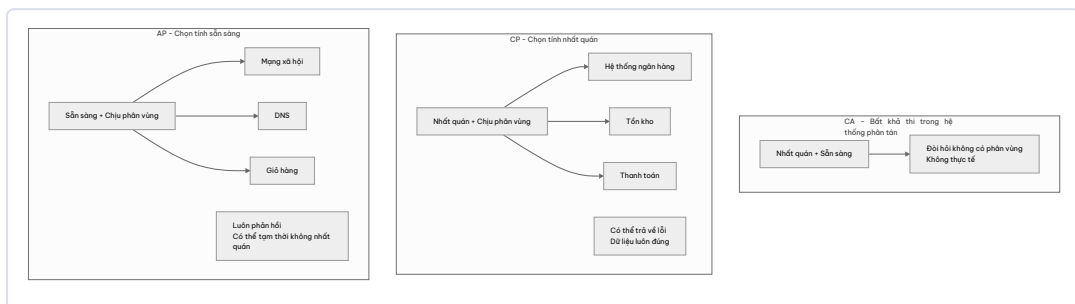
Tình huống sử dụng:

Số lượt thích trên Facebook
Người dùng A thấy: 100 lượt thích
Người dùng B thấy: 99 lượt thích (khác trung tâm dữ liệu)

AP cho phép:

- Cả hai người dùng đều thấy dữ liệu
- Số đếm cuối cùng sẽ đồng bộ
- Chấp nhận việc tạm thời không nhất quán

Sơ Đồ: Các Đánh Đổi Trong CAP



Trong hệ thống phân tán, phân vùng mạng luôn xảy ra → chọn C hoặc A.

Chọn Như Thế Nào?

Khung tư duy (Framework):


```
def choose_cap_model(feature):
    if financial_transaction(feature):
        return "CP" # Correctness critical

    if user_can_tolerate_stale_data(feature):
        return "AP" # Availability critical

    if strong_consistency_required(feature):
        return "CP"

    return "AP" # Default: availability usually better UX
```

Những câu hỏi cần đặt ra:

1. Nếu dữ liệu sai, người dùng bị ảnh hưởng như thế nào?

- Nghiêm trọng (tiền, tồn kho) → CP
- Nhỏ (số lượt thích, lượt xem) → AP

2. Nếu hệ thống sập, người dùng bị ảnh hưởng như thế nào?

- Không thể hoạt động → AP
- Có thể chờ được → CP

3. Người dùng có nhận ra sự không nhất quán không?

- Có và họ quan tâm → CP
- Không, hoặc họ không quan tâm → AP

Ví Dụ Thực Tế

Trang sản phẩm thương mại điện tử:

Thông tin sản phẩm: AP

- Mô tả sản phẩm, hình ảnh
- Nhặt quán cuối cùng là ổn
- Trải nghiệm người dùng: luôn tải được

Số lượng tồn kho: CP

- Phải chính xác để tránh bán vượt tồn kho
- Thà hiện "hết hàng" còn hơn bán thứ bạn không có

Giỏ hàng: AP

- Người dùng thêm sản phẩm, thấy ngay lập tức
- Giỏ hàng có thể nhặt quán cuối cùng giữa các thiết bị

Ứng dụng ngân hàng:

Số dư tài khoản: CP

- Lúc nào cũng phải chính xác
- Báo lỗi trong lúc bảo trì là chấp nhận được
- Không bao giờ hiển thị số dư sai

Lịch sử giao dịch: CP

- Phải đúng và đúng thứ tự
- Là dấu vết kiểm toán tài chính

Vị trí ATM: AP

- Có thể nhất quán cuối cùng
- Dữ liệu cũ không phải vấn đề nghiêm trọng

Khung Tư Duy Cá Nhân Của Tôi

90% tính năng có thể là AP. Người dùng thích một hệ thống chạy được (dù dữ liệu hơi cũ) hơn là gặp lỗi.

Chỉ chọn CP khi:

- Có liên quan đến tiền
- Đặt chỗ tồn kho/tài nguyên
- Yêu cầu pháp lý/kiểm toán

Sai lầm phổ biến: Áp dụng CP quá đà. Lập trình viên (*Developers*) nghĩ "phải nhất quán" cho mọi tính năng.

Thực tế: Bảng tin (*Feed*) Facebook không cần CP. Lướt thích Instagram không cần CP. Số email đã đọc không cần CP.

Hãy tự hỏi: "Điều tệ nhất có thể xảy ra nếu dữ liệu sai trong 5 giây là gì?"

Nếu câu trả lời là "không có gì nghiêm trọng" → AP là ổn.

Kết Hợp Các Khái Niệm

Các khái niệm này không độc lập. Chúng tác động qua lại lẫn nhau.

Ví Dụ: Thiết Kế API Chịu Lưu Lượng Cao

Yêu cầu (*Requirements*):

- Xử lý 100.000 req/s (thông lượng)
- Thời gian phản hồi < 100ms (độ trễ)
- 99,9% thời gian hoạt động (tính sẵn sàng)
- Giao dịch tài chính (cần tính nhất quán)

Các quyết định thiết kế:

Yêu cầu thông lượng (100K req/s):

- Cân mở rộng theo chiều ngang
- Nhiều máy chủ ứng dụng
- Cân bằng tải

Yêu cầu độ trễ (< 100ms):

- Cân tăng bộ nhớ đệm
- Chỉ mục cơ sở dữ liệu
- Truy vấn được tối ưu

Yêu cầu tính sẵn sàng (99.9%):

- Triển khai nhiều máy chủ
- Kiểm tra sức khỏe
- Tự động chuyển đổi dự phòng

Yêu cầu tính nhất quán (CP):

- Cơ sở dữ liệu nhất quán mạnh
- Giao dịch ACID
- Có thể hy sinh một phần tính sẵn sàng khi xảy ra phân vùng

Các đánh đổi:

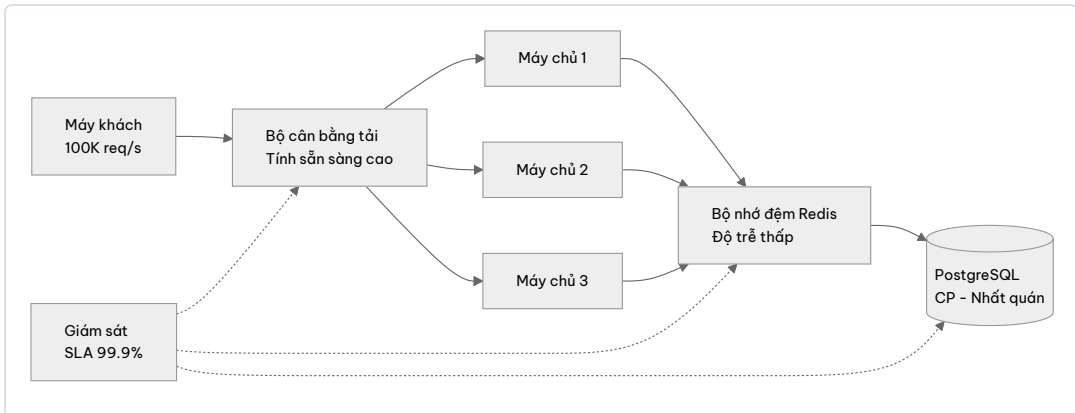
Thông lượng cao + Độ trễ thấp:

- Cân lưu đệm (độ phức tạp)
- Thêm hạ tầng (chi phí)

Tính sẵn sàng cao + Nhất quán mạnh:

- Xung đột theo định lý CAP
- Chọn tính nhất quán (CP)
- Chấp nhận thỉnh thoảng không sẵn sàng

Sơ Đồ: Thiết Kế Hệ Thống Tích Hợp



Mỗi thành phần (*component*) giải quyết một yếu tố khác nhau: thông lượng, độ trễ, tính sẵn sàng, tính nhất quán.

Những Điểm Cốt Lõi

Độ trễ và thông lượng:

- **Độ trễ:** Thời gian xử lý 1 yêu cầu
- **Thông lượng:** Số yêu cầu xử lý được mỗi giây
- Hai chỉ số độc lập, tối ưu riêng
- Thường phải đánh đổi giữa hai thứ

Tính sẵn sàng:

- **99%:** 7,2 giờ ngừng hoạt động/tháng - công cụ nội bộ
- **99,9%:** 43 phút ngừng hoạt động/tháng - thương mại điện tử
- **99,99%:** 4,3 phút ngừng hoạt động/tháng - ngân hàng
- Mỗi "số 9" làm chi phí tăng gấp đôi
- Đừng thiết kế thừa: chọn mức phù hợp với nhu cầu kinh doanh

Định lý CAP:

- **CP (Nhất quán + Chịu phân vùng (*Partition*)):** Ngân hàng, thanh toán
- **AP (Sẵn sàng + Chịu phân vùng):** Mạng xã hội, lưu đệm (*caching*)
- Phân vùng mạng luôn xảy ra → chọn C hoặc A

- Mặc định chọn AP cho hầu hết tính năng

Khung Ra Quyết Định (*Decision Framework*):

1. Đo lường yêu cầu cụ thể (bằng con số, không phải cảm giác)
2. Hiểu các đánh đổi của mỗi lựa chọn
3. Lựa chọn dựa trên các ưu tiên kinh doanh
4. Giám sát (*Monitor*) và điều chỉnh

Quan trọng nhất: Không có giải pháp hoàn hảo. Hiểu các khái niệm → Đưa ra những đánh đổi có cơ sở.

Bài Tập Tổng Hợp: Từ Lý Thuyết Đến Thực Hành

Chúc mừng bạn đã hoàn thành tất cả các bài học trong Phần 1!

Bạn đã học về các thành phần (*components*), các mẫu giao tiếp (*communication patterns*), luồng dữ liệu (*data flow*), và các khái niệm quan trọng như độ trễ (*latency*), thông lượng (*throughput*), định lý CAP (*CAP theorem*).

Nhưng có một sự thật mà tôi phải nói thẳng: **Đọc hiểu ≠ Làm được.**

Tôi từng có một người được tôi kèm cặp đọc xong tất cả tài liệu, nói "Em hiểu hết rồi!" Nhưng khi tôi cho một bài tập đơn giản: "Thiết kế dịch vụ rút gọn URL (*URL shortener*)", anh ta ngồi im 20 phút không biết bắt đầu từ đâu.

Tại sao? Vì thiếu thực hành. Thiếu việc áp dụng các khái niệm vào những kịch bản thực tế.

Bài học này sẽ khắc phục điều đó. Ba bài tập được thiết kế để:

1. **Buộc bạn suy nghĩ như một kiến trúc sư** (*architect*) (không còn sao chép – dán lời giải)
2. **Áp dụng tất cả các khái niệm đã học** (thành phần, luồng dữ liệu, đánh đổi (*trade-offs*))
3. **Xây dựng sự tự tin** (sau khi làm xong, bạn sẽ thấy "Mình làm được đây!")

Đừng bỏ qua. Đây là phần quan trọng nhất của Phần 1.

Cách Làm Bài Tập Hiệu Quả

Quy tắc:

1. **Không Google ngay** - Thử tự suy nghĩ 15 phút trước
2. **Vẽ ra giấy/bảng trắng** - Việc vẽ buộc bạn phải nghĩ rõ ràng
3. **Giải thích cho người khác** - Hoặc cho một chú vịt cao su (*rubber duck*)
4. **So sánh với lời giải** - Sau khi làm xong

5. **Lặp lại** - Làm lại với một cách tiếp cận khác

Tư duy (*Mindset*):

Không có "đúng/sai" tuyệt đối. Chỉ có "phù hợp/không phù hợp" với bối cảnh.

Nếu bạn có lập luận tốt cho các quyết định của mình, đó là một thiết kế tốt.

Bài 1: Phân Tích Hệ Thống Thực Tế

Mục Tiêu

Phát triển tư duy ở cấp độ hệ thống bằng cách phân tích ngược (*reverse engineer*) một ứng dụng bạn dùng hàng ngày.

Đề Bài

Chọn **1 ứng dụng** bạn hay dùng:

- Facebook
- Instagram
- Shopee
- Grab
- TikTok

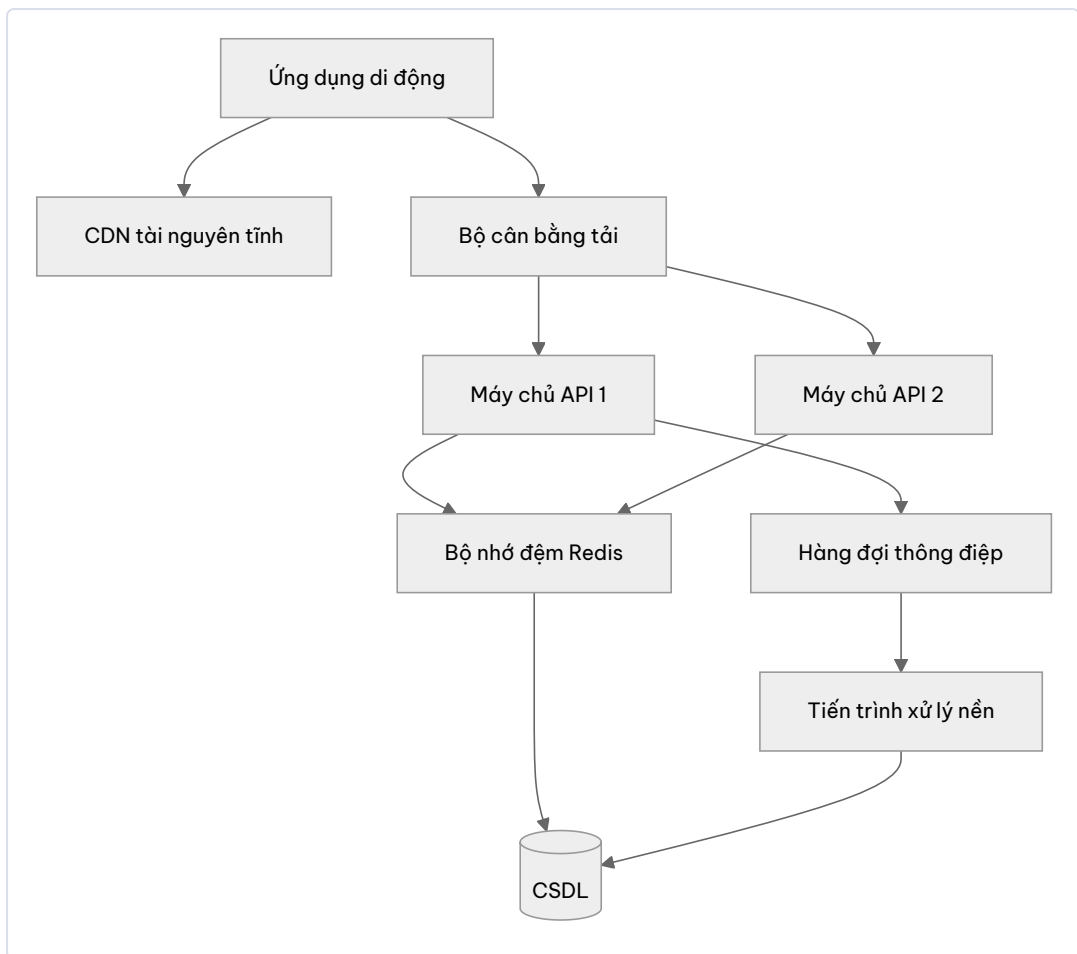
Nhiệm vụ:

Phần A: Vẽ Sơ Đồ Các Thành Phần

Xác định và vẽ:

1. Máy khách (ứng dụng web/di động)
2. (Các) bộ cân bằng tải
3. Các máy chủ ứng dụng
4. (Các) tầng bộ nhớ đệm
5. (Các) cơ sở dữ liệu
6. CDN (nếu có)
7. Hàng đợi thông điệp (nếu có)

Cấu trúc ví dụ:



Mẹo:

Hãy suy luận dựa trên:

- **Quy mô** (Scale) - Bao nhiêu người dùng? → Cần bộ nhớ đệm (cache), bộ cân bằng tải (LB), nhiều máy chủ
- **Tính năng** - Trò chuyện thời gian thực? → Máy chủ WebSocket, hàng đợi thông điệp (message queue)
- **Hiệu năng** (Performance) - Tải nhanh? → CDN, lưu đệm (caching) mạnh tay

Phần B: Lăn Theo Luồng Dữ Liệu

Chọn 1 hành động của người dùng và lăn theo từ đầu đến cuối.

Ví dụ:

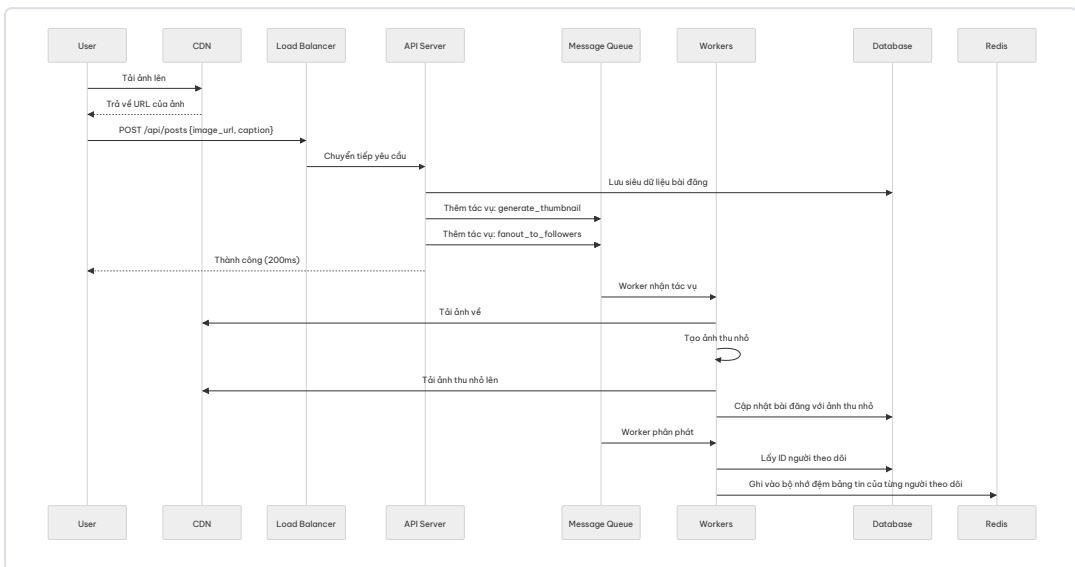
- **Instagram:** Người dùng đăng một bức ảnh

- **Shopee:** Người dùng đặt một đơn hàng
- **Grab:** Người dùng đặt một chuyến xe

Định dạng:

1. Hành động của người dùng: [Mô tả]
2. Luồng yêu cầu:
Máy khách → Thành phần A → Thành phần B → Thành phần C
3. Biến đổi dữ liệu:
 - Tại thành phần A: [Điều gì xảy ra]
 - Tại thành phần B: [Điều gì xảy ra]
 - Tại thành phần C: [Điều gì xảy ra]
4. Luồng phản hồi:
Thành phần C → Thành phần B → Thành phần A → Máy khách
5. Các tiến trình bất đồng bộ (nếu có):
 - Tác vụ nền X
 - Thông báo Y

Ví dụ: Đăng ảnh trên Instagram



Phân tích:

Phần đồng bộ (200ms):

- Tải ảnh lên CDN
- Lưu siêu dữ liệu bài đăng
- Thêm tác vụ vào hàng đợi
- Trả về thành công

Phần bất đồng bộ (1-2 phút):

- Tạo ảnh thu nhỏ
- Phân phát tới bảng tin của người theo dõi
- Gửi thông báo

Tại sao bất đồng bộ?

- Người dùng không cần chờ ảnh thu nhỏ
- Người theo dõi không cần cập nhật tức thì
- Có thể xử lý hàng triệu người theo dõi

Phần C: Dự Đoán Điểm Nghẽn (Bottlenecks)

Dựa trên sơ đồ của bạn, hãy xác định:

1. Điểm nghẽn (Bottleneck) hiện tại (ở mức tải bình thường):

Thành phần: [?]

Lý do: [Tại sao nó chậm?]

Bằng chứng: [Làm sao bạn biết?]

2. Điểm nghẽn trong tương lai (ở quy mô gấp 10 lần):

Thành phần: [?]

Lý do: [Cái gì sẽ hỏng trước?]

Tác động: [Điều gì xảy ra khi nó hỏng?]

3. Giải pháp:

Ngắn hạn: [Sửa nhanh]

Dài hạn: [Giải pháp bài bản]

Đánh đổi: [Chúng ta hy sinh điều gì?]

Giải Mẫu Tham Khảo

Ví dụ: Shopee - Người Dùng Đặt Hàng

Các thành phần:

Máy khách: Ứng dụng di động
CDN: Ảnh sản phẩm, tài nguyên tĩnh
Bộ cân bằng tải: Phân phối lưu lượng
Máy chủ API: Xử lý đơn hàng (10+ máy chủ)
Bộ nhớ đệm: Redis (thông tin sản phẩm, tồn kho)
Cơ sở dữ liệu:

- CSDL sản phẩm (bản sao chỉ đọc)
- CSDL đơn hàng (master-slave)
- CSDL người dùng (master-slave)

Hàng đợi thông điệp: RabbitMQ
Worker: Xử lý thanh toán, thông báo

Luồng Dữ Liệu:

1. Người dùng nhấn "Đặt hàng"
2. Yêu cầu:
Ứng dụng → LB → Máy chủ API
3. Máy chủ API:
 - Kiểm tra hợp lệ các mặt hàng trong giỏ (từ bộ nhớ đệm)
 - Kiểm tra tồn kho (Redis)
 - Tạo đơn hàng (CSDL đơn hàng)
 - Giữ chỗ hàng tồn kho (giảm số lượng)
 - Thêm vào hàng đợi thanh toán
 - Trả về order_id
4. Phản hồi:
API → LB → Ứng dụng (hiển thị "Đang xử lý...")
5. Bất đồng bộ:
 - Worker thanh toán trừ tiền thẻ
 - Nếu thành công: Cập nhật trạng thái đơn hàng
 - Nếu thất bại: Hoàn lại tồn kho, thông báo cho người dùng
 - Worker thông báo gửi email/SMS

Điểm nghẽn:

- Hiện tại:
- Kiểm tra tồn kho (Redis)
 - Nhiều người dùng đồng thời kiểm tra cùng sản phẩm
 - Giải pháp: Khoá lạc quan (optimistic locking)
- Ở quy mô gấp 10 lần:
- Ghi vào CSDL đơn hàng (điểm nghẽn ở máy chủ chính)
 - Giải pháp: Phân mảnh theo user_id hoặc order_id
 - Đánh đổi: Truy vấn xuyên mảnh khó hơn

Tự Kiểm Tra

Bạn đạt bài này khi:

- ☐ Sơ đồ có ít nhất 5 thành phần rõ ràng
- ☐ Luồng dữ liệu hợp lý và đầy đủ
- ☐ Các dự đoán về điểm nghẽn có lập luận
- ☐ Các giải pháp có phân tích đánh đổi (*trade-off*)

Bài 2: Thiết Kế Dịch Vụ Rút Gọn URL (bit.ly)

Mục Tiêu

Thiết kế một hệ thống hoàn chỉnh từ yêu cầu (*requirements*) đến các chi tiết triển khai.

Yêu Cầu

Chức năng:

- Người dùng gửi URL dài → Hệ thống trả về URL ngắn
- Người dùng nhấn vào URL ngắn → Chuyển hướng tới URL gốc
- URL ngắn tùy chỉnh (không bắt buộc)

Phi chức năng:

- **Ghi:** 1 triệu URL được rút gọn/ngày
- **Đọc:** 100 triệu lượt chuyển hướng/ngày
- **Độ trễ:** < 10ms cho mỗi lượt chuyển hướng
- **Tính sẵn sàng** (*Availability*): 99,9%

Phần A: Ước Lượng Năng Lực (*Capacity Estimation*)

Tính toán để hiểu rõ quy mô.

Ghi (Rút gọn):

1M URL/ngày
= $1M / (24 * 3600)$
 ≈ 12 URL/giây

Cao điểm (gấp 3 lần trung bình): 36 URL/giây

Kết luận: Lượng ghi nhẹ, dễ xử lý

Đọc (Chuyển hướng):

100M lượt chuyển hướng/ngày
= $100M / (24 * 3600)$
 $\approx 1,157$ yêu cầu/giây

Cao điểm (gấp 3): $\sim 3,500$ yêu cầu/giây

Kết luận: Thiên về đọc (tỉ lệ 100:1) → Rất hợp với bộ nhớ đệm

Lưu trữ (Storage):

1 bản ghi ảnh xạ URL ≈ 500 byte
 $1M \text{ URL/ngày} * 365 \text{ ngày} * 5 \text{ năm} = 1.825B \text{ URL}$
 $1.825B * 500 \text{ byte} \approx 912 \text{ GB} \approx 1 \text{ TB}$

Kết luận: Lưu trữ không phải vấn đề

Phần B: Thiết Kế API

Định nghĩa các giao diện rõ ràng.

```

POST /api/shorten
Request:
{
  "long_url": "https://example.com/very/long/url",
  "custom_alias": "mylink" (optional)
}

Response:
{
  "short_url": "https://short.ly/abc123",
  "long_url": "https://example.com/very/long/url",
  "created_at": "2024-01-15T10:30:00Z"
}

---

GET /{short_code}
Response: 302 Redirect to long_url

```

Phần C: Bài Toán Cốt Lõi – Sinh Mã Rút Gọn

Thách thức: Chuyển URL dài → mã ngắn duy nhất

Phương án 1: Dựa trên hàm băm (hash)

```

import hashlib

def generate_short_code(long_url):
    hash_value = hashlib.md5(long_url.encode()).hexdigest()
    short_code = hash_value[:7] # Take first 7 chars
    return short_code

# Example:
# "https://example.com/long/url" → "a3f8c2b"

```

Phân tích:

Tái định (cùng URL → cùng mã)
 Có thể xảy ra xung đột
 Cần kiểm tra trùng lặp trong CSDL
 Dễ đoán trước (vấn đề bảo mật)

Phương án 2: Tự tăng (Auto-increment) + Mã hoá Base62

```

BASE62 = "0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ"

def encode_base62(num):
    if num == 0:
        return BASE62[0]

    result = []
    while num:
        result.append(BASE62[num % 62])
        num //= 62
    return ''.join(reversed(result))

# Example:
# ID 1 → "1"
# ID 62 → "10"
# ID 1000000 → "4c92"

```

Tính độ dài:

$62^6 = 56$ tỷ tỷ tỷ tỷ tỷ tỷ
 $62^7 = 3,5$ nghìn tỷ tỷ tỷ tỷ tỷ tỷ
 Với 6 ký tự: 56 tỷ URL (đủ dùng!)

Phân tích:

Không xung đột (ID duy nhất)
 Mã ngắn (6-7 ký tự)
 Thứ tự dễ đoán (có thể là tính năng hoặc lỗi)
 Tuân tự (có thể làm lộ khối lượng)

Khuyến nghị: Dùng Phương án 2 (Base62) vì sự đơn giản và độ tin cậy (*reliability*).

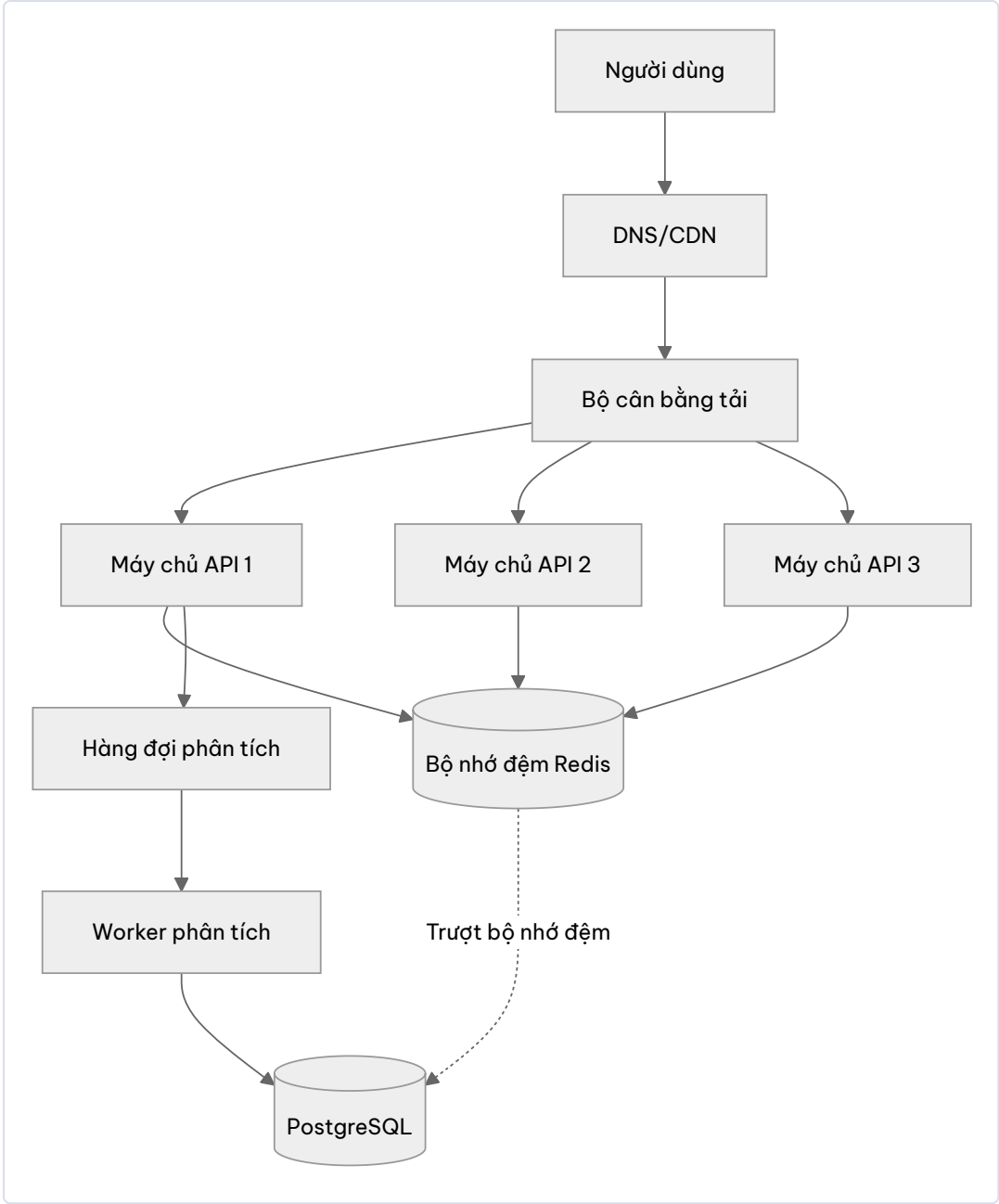
Phần D: Lược Đồ (Schema) Cơ Sở Dữ Liệu

```
CREATE TABLE urls (  
  id BIGSERIAL PRIMARY KEY,  
  short_code VARCHAR(10) UNIQUE NOT NULL,  
  long_url TEXT NOT NULL,  
  created_at TIMESTAMP DEFAULT NOW(),  
  expires_at TIMESTAMP,  
  user_id BIGINT,  
  click_count INT DEFAULT 0,  
  
  INDEX idx_short_code (short_code),  
  INDEX idx_user_id (user_id)  
);  
  
CREATE TABLE clicks (  
  id BIGSERIAL PRIMARY KEY,  
  short_code VARCHAR(10) NOT NULL,  
  clicked_at TIMESTAMP DEFAULT NOW(),  
  ip_address VARCHAR(45),  
  user_agent TEXT,  
  referer TEXT,  
  country VARCHAR(2),  
  
  INDEX idx_short_code (short_code),  
  INDEX idx_clicked_at (clicked_at)  
);
```

Tại sao thiết kế như vậy:

- **urls:** Dữ liệu cốt lõi, cần tra cứu nhanh theo short_code
- **clicks:** Dữ liệu phân tích, có thể tách riêng (không làm chậm việc chuyển hướng)
- **Chỉ mục (Indexes):** Cực kỳ quan trọng cho hiệu năng
- **click_count:** Phi chuẩn hoá (Denormalized) để hiển thị nhanh

Phần E: Thiết Kế Kiến Trúc (Architecture)



Luồng: Rút gọn URL

1. Người dùng → POST /api/shorten
2. Máy chủ API:
 - Sinh ID (tự tăng)
 - Mã hoá sang Base62
 - Lưu vào CSDL
 - Trả về URL ngắn
3. Thời gian: ~50ms

Luồng: Chuyển hướng

1. Người dùng → GET /abc123
2. Máy chủ API:
 - Kiểm tra bộ nhớ đệm Redis
 - Nếu trúng: Trả về long_url (5ms)
 - Nếu trượt: Truy vấn CSDL → Lưu kết quả vào bộ nhớ đệm → Trả về (50ms)
3. Ghi log sự kiện click (bất đồng bộ, không chờ)
4. Chuyển hướng 302

Chiến lược lưu đệm:

```

def redirect(short_code):
    # 1. Try cache
    long_url = cache.get(f"url:{short_code}")
    if long_url:
        # Log async (fire and forget)
        analytics_queue.add({
            "short_code": short_code,
            "timestamp": now(),
            "ip": request.ip
        })
        return redirect_response(long_url)

    # 2. Cache miss → DB
    url_obj = db.query(
        "SELECT long_url FROM urls WHERE short_code = ?",
        short_code
    )

    if not url_obj:
        return 404

    # 3. Cache for next time
    cache.set(
        f"url:{short_code}",
        url_obj.long_url,
        ttl=86400 # 24 hours
    )

    # 4. Log async
    analytics_queue.add({...})

    return redirect_response(url_obj.long_url)

```

Phần F: Mở Rộng Lên 100 Triệu Lượt Click/Ngày

Vấn đề:

100M lượt click/ngày = 1,157 req/s (trung bình)
Cao điểm: 3,500 req/s

Thách thức:

1. Tải đọc lên cơ sở dữ liệu
2. Dung lượng bộ nhớ đệm
3. Lượng ghi cho phân tích

Giải pháp:

1. Đọc Cơ Sở Dữ Liệu:

Vấn đề: $3,500 \text{ req/s} * 50\text{ms} = 175$ truy vấn đồng thời

Giải pháp A: Thêm bản sao chỉ đọc (3 slave)

- Phân phối lượt đọc qua các bản sao
- Mỗi bản sao xử lý $\sim 1,200 \text{ req/s}$ → Dễ dàng

Giải pháp B: Lưu đệm mạnh tay (tỉ lệ trúng 99%)

- Chỉ 1% đi tới CSDL = 35 req/s
- Rất dễ kiểm soát

2. Chiến Lược Bộ Nhớ Đệm:

Lưu đệm các URL "nóng":

- Quy tắc 80/20: 20% URL nhận 80% lưu lượng
- Lưu đệm 1 triệu URL hàng đầu
- Bộ nhớ: $1\text{M} * 500 \text{ byte} = 500 \text{ MB}$
- Rẻ, nhanh

Loại bỏ theo LRU:

- Tự động xóa mục ít được dùng gần đây nhất
- Luôn giữ lại dữ liệu "nóng"

3. Phân Tích Dữ Liệu:

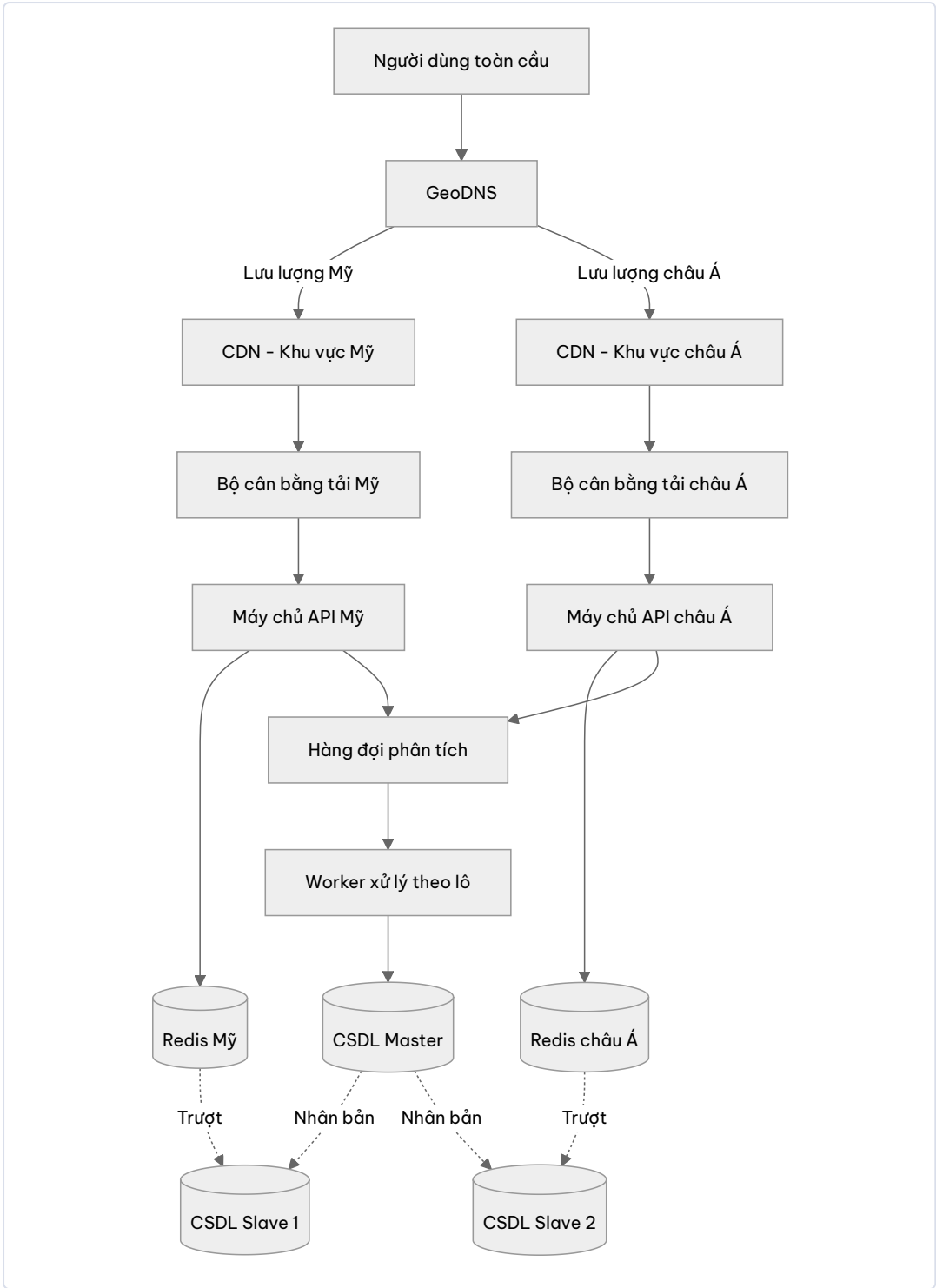
Vấn đề: 3,500 lượt ghi/giây vào bảng clicks

Giải pháp: Hàng đợi bất đồng bộ + ghi theo lô

- Gom các lượt click vào hàng đợi
- Worker chèn theo lô mỗi 10 giây
- Giảm lượng ghi vào CSDL đi 10 lần

Đánh đổi: Dữ liệu phân tích trễ khoảng ~ 10 giây (chấp nhận được)

Kiến Trúc Cuối Cùng:



Tại sao cách này hiệu quả:

- **GeoDNS:** Định tuyến người dùng tới khu vực gần nhất → Độ trễ thấp
- **CDN:** Lưu đệm các lượt chuyển hướng tại điểm biên (*edge*) → Cực nhanh
- **Nhiều khu vực:** Tính sẵn sàng cao (*High availability*)
- **Bộ nhớ đệm Redis:** Tỷ lệ trùng 99% → Tải lên CSDL tối thiểu
- **Bản sao chỉ đọc** (*Read replicas*): Dễ dàng mở rộng lượng đọc
- **Phân tích bất đồng bộ** (*async*): Không làm chậm việc chuyển hướng

Tự Kiểm Tra

Bạn đạt bài này khi:

- ☐ Phân ước lượng năng lực có các phép tính rõ ràng
- ☐ Thiết kế API gọn gàng và theo chuẩn RESTful
- ☐ Lược đồ cơ sở dữ liệu có chỉ mục đặt đúng chỗ
- ☐ Kiến trúc đáp ứng được các yêu cầu về quy mô
- ☐ Có giải thích các đánh đổi của từng quyết định

Bài 3: Phân Tích Đánh Đổi - Người Dùng Tải Ảnh Đại Diện

Mục Tiêu

Thực hành so sánh các cách tiếp cận và đưa ra quyết định có cơ sở.

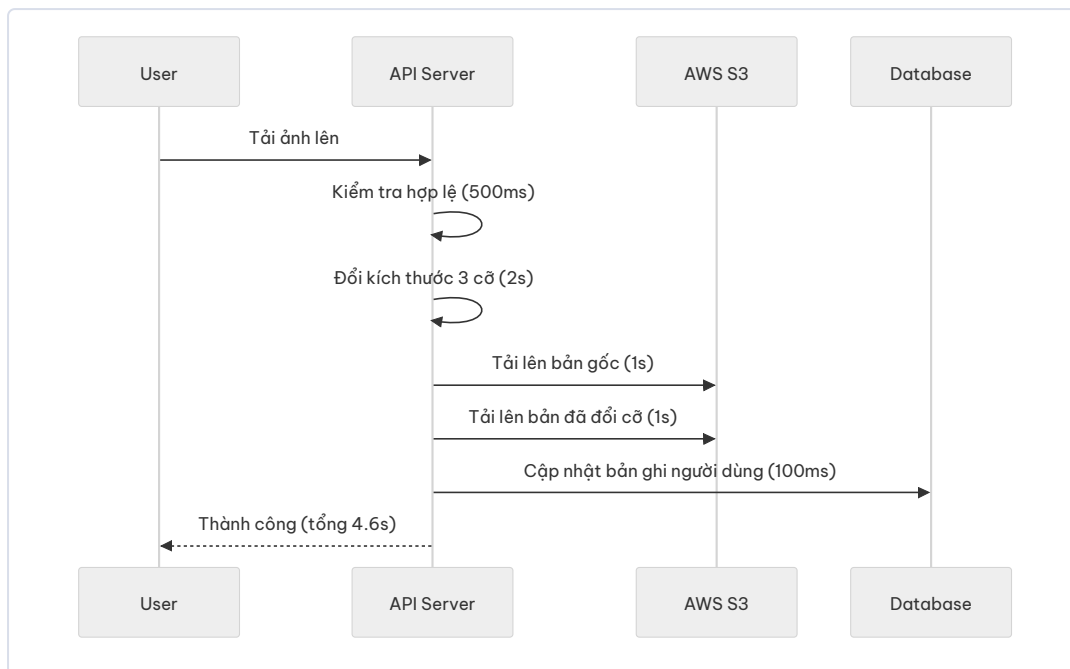
Bối Cảnh

Tính năng: Người dùng tải lên ảnh đại diện (ảnh hồ sơ)

Các bước xử lý cần thiết:

- Kiểm tra hợp lệ ảnh (định dạng, kích thước)
- Thay đổi kích thước thành nhiều cỡ (thu nhỏ, vừa, lớn)
- Tải lên S3
- Cập nhật bản ghi người dùng

Cách Tiếp Cận A: Đồng Bộ (Synchronous)



Triển khai:

```

@app.route('/upload-avatar', methods=['POST'])
def upload_avatar_sync():
    # 1. Validate
    if not is_valid_image(request.file):
        return error(400, "Invalid image")

    # 2. Resize
    thumbnail = resize(request.file, 100, 100)
    medium = resize(request.file, 300, 300)
    large = resize(request.file, 600, 600)

    # 3. Upload to S3
    original_url = s3.upload(request.file)
    thumbnail_url = s3.upload(thumbnail)
    medium_url = s3.upload(medium)
    large_url = s3.upload(large)

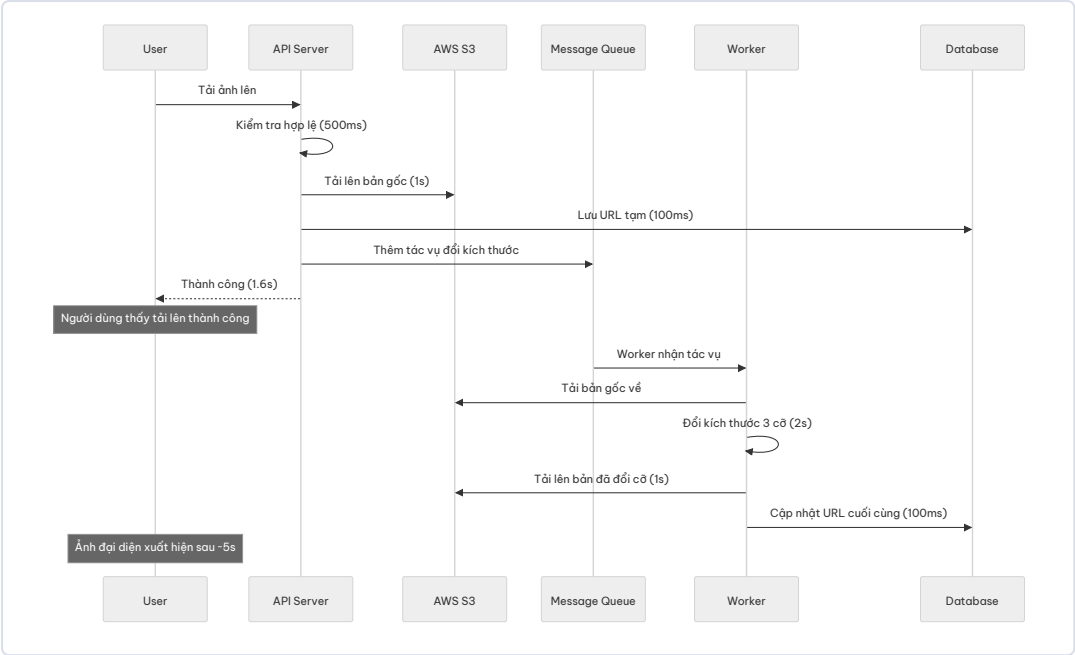
    # 4. Update DB
    db.update_user(user_id, {
        'avatar_original': original_url,
        'avatar_thumbnail': thumbnail_url,
        'avatar_medium': medium_url,
        'avatar_large': large_url
    })

    # 5. Return
    return success({
        'avatar_url': thumbnail_url
    })

# User waits 4.6 seconds

```


Cách Tiếp Cận B: Bất Đồng Bộ (Asynchronous)



Triển khai:

```

@app.route('/upload-avatar', methods=['POST'])
def upload_avatar_async():
    # 1. Quick validation
    if not is_valid_image(request.file):
        return error(400, "Invalid image")

    # 2. Upload original only
    original_url = s3.upload(request.file)

    # 3. Save temp state
    db.update_user(user_id, {
        'avatar_original': original_url,
        'avatar_status': 'processing'
    })

    # 4. Queue background job
    queue.add_job('resize_avatar', {
        'user_id': user_id,
        'original_url': original_url
    })

    # 5. Return immediately
    return success({
        'avatar_url': original_url,
        'status': 'processing'
    })

# User waits 1.6 seconds

# Background worker (separate process)
def resize_avatar_worker(job):
    # Download
    image = s3.download(job['original_url'])

    # Resize
    thumbnail = resize(image, 100, 100)
    medium = resize(image, 300, 300)
    large = resize(image, 600, 600)

    # Upload
    thumbnail_url = s3.upload(thumbnail)
    medium_url = s3.upload(medium)
    large_url = s3.upload(large)

    # Update DB
    db.update_user(job['user_id'], {
        'avatar_thumbnail': thumbnail_url,
        'avatar_medium': medium_url,
        'avatar_large': large_url,
        'avatar_status': 'completed'
    })

```

Phần A: Phân Tích So Sánh

Độ trễ:

Đồng bộ:

- Người dùng chờ: 4,6 giây
- Tốc độ cảm nhận: Chậm
- Rủi ro hết thời gian chờ: Cao (nếu > 30s)

Bất đồng bộ:

- Người dùng chờ: 1,6 giây (nhanh hơn 65%)
- Tốc độ cảm nhận: Nhanh
- Rủi ro hết thời gian chờ: Thấp

Trải Nghiệm Người Dùng:

Đồng bộ:

Có kết quả ngay lập tức
UX đơn giản (tải lên → xong)
Thời gian chờ lâu
Cần thanh tiến trình
Người dùng không làm được gì khác

Bất đồng bộ:

Phản hồi nhanh
Có thể tiếp tục dùng ứng dụng
Kết quả đến chậm hơn
Cần hiển thị trạng thái "đang xử lý"
Cần xử lý việc tải lại trang (lưu giữ trạng thái)

Độ Phức Tạp (Complexity):

Đồng bộ:

Mã đơn giản
Dễ gỡ lỗi
Không cần thêm hạ tầng
Khó mở rộng (chặn luồng của máy chủ)

Bất đồng bộ:

Có khả năng mở rộng (chịu được lưu lượng tăng đột biến)
Giải phóng luồng của máy chủ
Mã phức tạp hơn
Cần hạ tầng hàng đợi thông điệp
Xử lý lỗi khó hơn (logic thư lại)
Cần giám sát

Xử Lý Lỗi:

Đồng bộ:

- Lỗi → Trả về cho người dùng ngay lập tức
- Người dùng có thể thư' lại
- Hoàn tác đơn giản

Bất đồng bộ:

- Lỗi → Người dùng đã nhận phản hồi thành công rồi
- Cân hệ thống thông báo
- Logic thư' lại phức tạp
- Xu' lý trạng thái dở dang

Phần B: Khung Ra Quyết Định (*Decision Framework*)

Chọn Đồng bộ khi:

Việc xử' lý nhanh (< 2 giây)

Người dùng cần xác nhận ngay

Tình huống sử' dụng đơn giản

Lưu lượng thấp

Nhóm nhỏ (không duy trì nổi hạ tầng phức tạp)

Ví dụ:

- Gửi biểu mẫu
- Các thao tác CRUD đơn giản
- Đăng nhập người dùng

Chọn Bất đồng bộ khi:

Việc xử' lý chậm (> 3 giây)

Người dùng có thể chờ kết quả

Lưu lượng cao (cần chịu được các đợt tăng đột biến)

Nhiều thao tác tốn kém

Chấp nhận được nhất quán cuối cùng

Ví dụ:

- Tải tệp lên kèm xử' lý
- Tạo báo cáo
- Gửi email
- Chuyển mã video
- Nhập/xuất dữ liệu

Phần C: Cách Tiếp Cận Lai (*Hybrid*) (Tận Dụng Ưu Điểm Của Cả Hai)

Chiến lược: Đồng bộ (*sync*) nhanh + Bất đồng bộ chuyên sâu

```

@app.route('/upload-avatar', methods=['POST'])
def upload_avatar_hybrid():
    # SYNC: Fast operations
    if not is_valid_image(request.file):
        return error(400)

    original_url = s3.upload(request.file)

    # Generate quick thumbnail (500ms)
    quick_thumb = resize_fast(request.file, 100, 100)
    thumb_url = s3.upload(quick_thumb)

    # Update with quick thumbnail
    db.update_user(user_id, {
        'avatar_thumbnail': thumb_url,
        'avatar_status': 'processing'
    })

    # ASYNC: Quality resizes
    queue.add_job('resize_hq_avatar', {
        'user_id': user_id,
        'original_url': original_url
    })

    return success({
        'avatar_url': thumb_url,
        'status': 'processing'
    })

# User sees low-quality avatar immediately
# High-quality versions replace after few seconds

```

Tại sao đây thường là lựa chọn tốt nhất:

Phản hồi nhanh cho người dùng (2s)
 Có thứ để hiển thị ngay lập tức
 Chất lượng tốt hơn về sau
 Chịu được lưu lượng tăng đột biến
 Triển khai phức tạp hơn

Phần D: Ví Dụ Thực Tế

Facebook/Instagram làm thế nào?

Cách tiếp cận của Instagram:

1. Người dùng tải ảnh lên
2. Tải ngay lên CDN (bản gốc)
3. Hiện thị bản gốc ngay lập tức (có thể dung lượng lớn)
4. Chạy nền: Xử lý bộ lọc, tạo các kích cỡ
5. Đổi sang phiên bản đã xử lý khi sẵn sàng
6. Người dùng thấy tiến trình: "Đang xử lý..." → Xong

Tại sao?

- Hơn 1 tỷ người dùng → Bắt buộc phải bất đồng bộ
- Mức độ tương tác của người dùng > Chất lượng hoàn hảo
- Không thể bắt người dùng chờ 10 giây

Điểm mấu chốt: Trải nghiệm người dùng > Sự hoàn hảo về kỹ thuật.

Tự Kiểm Tra

Bạn đạt bài này khi:

- ☐ So sánh được độ trễ của 2 cách tiếp cận
- ☐ Giải thích rõ ràng sự khác biệt về UX
- ☐ Hiểu các đánh đổi về độ phức tạp
- ☐ Có khung ra quyết định rõ ràng
- ☐ Có thể đề xuất cách tiếp cận lại

Danh Sách Kiểm Tra Tổng Hợp Phần 1

Sau khi hoàn thành 3 bài tập, đánh giá bản thân:

Hiểu Biết Cốt Lõi

- ☐ **Tôi có thể vẽ sơ đồ kiến trúc** của bất kỳ ứng dụng nào tôi dùng
 - Kiểm tra: Chọn ngẫu nhiên 1 ứng dụng, vẽ trong 10 phút
- ☐ **Tôi hiểu điểm nghẽn và cách tìm ra nó**
 - Kiểm tra: Cho trước một sơ đồ, xác định điểm nghẽn và giải thích tại sao
- ☐ **Tôi có thể lần theo luồng dữ liệu** từ máy khách đến cơ sở dữ liệu
 - Kiểm tra: Giải thích luồng của 1 hành động bất kỳ của người dùng
- ☐ **Tôi hiểu các đánh đổi giữa đồng bộ và bất đồng bộ**

- Kiểm tra: Cho trước một tính năng, chọn cách tiếp cận và bảo vệ quyết định của mình

Kỹ Năng Thực Hành

- ☐ **Tôi có thể ước lượng năng lực** (*estimate capacity*)
 - Kiểm tra: Tính QPS, dung lượng lưu trữ cho các yêu cầu
- ☐ **Tôi có thể thiết kế API**
 - Kiểm tra: Định nghĩa các điểm cuối (*endpoints*) cho 1 tính năng
- ☐ **Tôi có thể thiết kế lược đồ cơ sở dữ liệu**
 - Kiểm tra: Tạo các bảng với chỉ mục phù hợp
- ☐ **Tôi biết khi nào cần bộ nhớ đệm**
 - Kiểm tra: Xác định dữ liệu có thể lưu đệm và chiến lược TTL

Nắm Vững Khái Niệm

- ☐ **Tôi hiểu Độ trễ và Thông lượng**
 - Kiểm tra: Giải thích sự khác biệt bằng một ví dụ thực tế
- ☐ **Tôi hiểu cách tính Tính sẵn sàng**
 - Kiểm tra: Tính thời gian ngừng hoạt động (*downtime*) cho 99,9% so với 99,99%
- ☐ **Tôi hiểu định lý CAP ở mức cơ bản**
 - Kiểm tra: Cho trước một hệ thống, xác định nó là CA, CP hay AP
- ☐ **Tôi có thể phân tích các đánh đổi**
 - Kiểm tra: So sánh 2 cách tiếp cận với ưu điểm/nhược điểm

Chấm Điểm

Đạt 10-12 mục: Xuất sắc - Sẵn sàng cho Phần 2 
Đạt 7-9 mục: Tốt - Ôn lại phần còn yếu, rồi đi tiếp
Đạt 4-6 mục: Cần thực hành thêm - Làm lại các bài tập
Đạt 0-3 mục: Ôn lại nội dung Phần 1

Tự đánh giá trung thực là cực kỳ quan trọng. Đừng vội vàng.

Những Sai Lầm Thường Gặp & Cách Khắc Phục

Sai Lầm 1: Thiết Kế Thừa (*Over-Engineering*)

Triệu chứng:

Bài URL shortener:

- Cụm Kubernetes
- Vi dịch vụ (5 dịch vụ)
- Event sourcing
- CQRS
- Service mesh

Cho 1 triệu URL/ngày → Quá mức cần thiết!

Cách khắc phục: Bắt đầu đơn giản. Chứng minh nhu cầu trước khi thêm độ phức tạp.

Sai Lầm 2: Mô Tả Thiếu Cụ Thể

Triệu chứng:

"Dùng cơ sở dữ liệu"

- SQL hay NoSQL?
- Lược đồ như thế nào?
- Chỉ mục ở đâu?

Cách khắc phục: Hãy cụ thể. Giải thích lý do cho các lựa chọn.

Sai Lầm 3: Bỏ Qua Các Con Số

Triệu chứng:

"Cần bộ nhớ đệm vì hiệu năng"

- Không tính QPS
- Không ước lượng tỉ lệ trùng
- Không tính bộ nhớ cần thiết

Cách khắc phục: Luôn tính toán. Các con số định hướng quyết định.

Sai Lầm 4: Không Phân Tích Đánh Đổi

Triệu chứng:

"Dùng cách tiếp cận A"

- Không nhắc đến nhược điểm
- Không so sánh với các phương án khác

Cách khắc phục: Mọi quyết định đều có đánh đổi. Hãy nêu rõ chúng.

Bước Tiếp Theo

Nếu bạn đạt danh sách kiểm tra:

🎉 Chúc mừng! Bạn đã hoàn thành Phần 1.

Phần 2: Các Khối Xây Dựng Cốt Lõi đang đợi bạn:

- Đi sâu vào các chiến lược bộ nhớ đệm
- Các mẫu (*patterns*) mở rộng (*scaling*) cơ sở dữ liệu
- Hàng đợi thông điệp (*Message queues*)
- Và nhiều hơn nữa...

Nếu chưa đạt:

Đừng vội vàng. Thiết kế hệ thống (*System design*) không phải một cuộc đua.

Khuyến nghị:

1. Ôn lại các bài học của Phần 1
2. Làm lại các bài tập với cách tiếp cận khác
3. Giải thích các khái niệm cho người khác
4. Thực hành với nhiều ứng dụng thực tế hơn
5. Xem lại danh sách kiểm tra sau 1 tuần

Những Điểm Cốt Lõi

Thực hành > Lý thuyết

Bạn có thể đọc 100 cuốn sách về thiết kế hệ thống, nhưng không thực hành thì vẫn không thiết kế được.

Hãy Tư Duy, Đừng Học Thuộc

Không có khuôn mẫu cố định. Mỗi hệ thống mỗi khác. Hãy học cách tư duy, chứ không phải ghi nhớ lời giải.

Đánh Đổi Ở Khắp Nơi

Mọi quyết định đều có đánh đổi. Một kiến trúc sư giỏi là người:

- Hiểu các đánh đổi
- Lựa chọn dựa trên bối cảnh
- Có thể bảo vệ các quyết định của mình

Bắt Đầu Đơn Giản, Cải Tiến Dần

Những thiết kế tốt nhất thường phát triển từ những thiết kế đơn giản. Đừng thiết kế thừa (*over-engineer*) ngay từ đầu.

Các Con Số Quan Trọng

Ước lượng năng lực, tính toán QPS, định cỡ lưu trữ - Không phải để trang trí. Chúng định hướng những quyết định then chốt.

Hãy dành thời gian với 3 bài tập này. Làm kỹ. Hiểu sâu.

Nền tảng vững = Mọi thứ phía sau đều dễ hơn.

Chúc may mắn! 🚀

PHẦN 2

Các Khối Xây Dựng Cốt Lõi

Nền tảng kỹ thuật cốt lõi để xây dựng hệ thống vận hành thực tế có khả năng mở rộng, tối ưu hiệu năng và tích hợp dịch vụ.

1) Giới Thiệu Phần

Phần 2 là nơi bạn học các khối hạ tầng cốt lõi đứng sau một hệ thống hiện đại.

Trong môi trường vận hành thực tế (*production*), khả năng mở rộng (*scale*) không đến từ mã ứng dụng đơn thuần. Nó đến từ cách bạn phối hợp cân bằng tải (*load balancing*), bộ nhớ đệm (*cache*), cơ sở dữ liệu (*database*) và xử lý bất đồng bộ (*async processing*) thành một kiến trúc nhất quán.

2) Vì Sao Phần Này Tồn Tại

Một ứng dụng chạy được chưa phải là một hệ thống vận hành thực tế.

Hệ thống ở quy mô thật cần:

- Lớp hạ tầng để xử lý lưu lượng (*traffic*) tăng đột biến.
- Lớp tối ưu hiệu năng (*performance*) để giảm độ trễ (*latency*) và giảm tải cho backend.
- Cơ chế tách rời dịch vụ để giảm liên kết (*coupling*) và tăng độ bền vững.
- Cách kết hợp các thành phần (*component*) đúng đắn để giữ độ tin cậy (*reliability*) khi mở rộng.

Nắm chắc các khối xây dựng (*building blocks*) ở phần này là điều kiện cần trước khi đi vào hệ thống phân tán (*distributed systems*) ở phần sau.

3) Mục Tiêu Học Tập

- Hiểu các thành phần hạ tầng cốt lõi trong hệ thống hiện đại.
- Hiểu các lớp tối ưu hiệu năng trong môi trường vận hành thực tế.
- Hiểu cơ chế tách rời (*decoupling*) dịch vụ bằng xử lý bất đồng bộ.
- Hiểu cách xử lý dữ liệu và lưu lượng khi hệ thống tăng trưởng.
- Hiểu cách tích hợp các thành phần và đánh giá sự đánh đổi (*trade-off*) tổng thể.

4) Bạn Sẽ Học Gì Trong Phần Này

- **Phân phối lưu lượng:** phân phối tải để tăng độ ổn định và khả năng mở rộng (*scalability*).
- **Lớp lưu đệm** (*Caching*): dùng bộ nhớ đệm để giảm độ trễ và bảo vệ backend.
- **Mở rộng dữ liệu:** chọn chiến lược mở rộng dữ liệu theo mẫu truy cập (*access pattern*).
- **Xử lý bất đồng bộ:** tách luồng đồng bộ và bất đồng bộ phù hợp với khối lượng công việc (*workload*).
- **Tách rời dịch vụ** (*Service decoupling*): giảm phụ thuộc trực tiếp giữa các dịch vụ.
- **Tích hợp các thành phần:** ghép các khối hạ tầng thành một hệ thống có thể vận hành.
- **Đánh đổi** (*Trade-offs*) **ở cấp hệ thống:** cân bằng hiệu năng, chi phí, độ phức tạp và tính ổn định.

5) Tổng Quan Các Bài Học

Bài 1 – Cân Bằng Tải – Phân Phối Lưu Lượng Trong Hệ Thống Phân Tán

- **Mục tiêu:** hiểu cách phân phối yêu cầu (*request*) giữa nhiều phiên bản chạy (*instance*)/dịch vụ.
- **Bạn nhận được:** tư duy chọn chiến lược cân bằng tải theo mẫu lưu lượng (*traffic pattern*) và độ tin cậy.

Bài 2 – Lưu Đệm (Lớp Hiệu Năng)

- **Mục tiêu:** hiểu bộ nhớ đệm như một lớp hiệu năng trong kiến trúc vận hành thực tế.
- **Bạn nhận được:** cách giảm độ trễ, giảm tải cho cơ sở dữ liệu và kiểm soát việc vô hiệu hoá (*invalidation*) hợp lý.

Bài 3 – Mở Rộng Cơ Sở Dữ Liệu (*Database Scaling*)

- **Mục tiêu:** nắm các hướng mở rộng dữ liệu theo mức tăng trưởng thực tế.

- **Bạn nhận được:** nền tảng để chọn giữa mở rộng theo chiều dọc (*vertical scaling*)/theo chiều ngang (*horizontal scaling*) và tối ưu việc truy cập dữ liệu.

Bài 4 – Hàng Đợi Thông Điệp (*Message Queues*) & Xử Lý Bất Đồng Bộ – Tách Rời Dịch Vụ (*Decoupling Services*)

- **Mục tiêu:** hiểu cách dùng hàng đợi (*queue*) để tách luồng xử lý và giảm liên kết.
- **Bạn nhận được:** tư duy thiết kế đường ống (*pipeline*) xử lý bất đồng bộ có khả năng chịu tải tốt hơn.

Bài 5 – Tích Hợp & Tư Duy Đánh Đổi – Kết Hợp Các Thành Phần (*Components*) Thành Hệ Thống

- **Mục tiêu:** học cách phối hợp nhiều khối xây dựng (*building block*) thành một kiến trúc thống nhất.
- **Bạn nhận được:** khả năng đánh giá sự đánh đổi ở cấp hệ thống thay vì từng thành phần riêng lẻ.

6) Kết Quả Sau Khi Hoàn Thành Phần 2

Bạn có thể:

- Giải thích rõ vì sao chọn từng thành phần hạ tầng trong một thiết kế.
- Lập luận hướng mở rộng phù hợp với lưu lượng và dữ liệu.
- Đánh giá sự đánh đổi về hiệu năng giữa bộ nhớ đệm, hàng đợi, cơ sở dữ liệu và ranh giới dịch vụ (*service boundaries*).
- Hiểu cách các thành phần tương tác với nhau trong hệ thống vận hành thực tế.

7) Chuẩn Bị Cho Phần 3

Phần 2 cho bạn nền tảng hạ tầng và tư duy tích hợp các thành phần.

Sang **Phần 3 — Nền Tảng Hệ Thống Phân Tán**, bạn sẽ đi sâu vào:

- các mô hình nhất quán (*consistency model*),
- khả năng chịu lỗi (*fault tolerance*),

- phối hợp phân tán (*distributed coordination*),
- và các thách thức về độ tin cậy trong hệ phân tán (*distributed system*).

Cân Bằng Tải (Load Balancing): Phân Phối Lưu Lượng (Traffic) Trong Hệ Thống Phân Tán (Distributed Systems)

Tóm Tắt Nhanh

- **Cân bằng tải** là cơ chế phân phối lưu lượng giữa nhiều phiên bản máy chủ (instances) để tăng **khả năng mở rộng** (scalability) và **độ tin cậy** (reliability).
- Trong môi trường thực tế (production), bộ cân bằng tải (load balancer) giúp giảm điểm lỗi duy nhất (single point of failure), hỗ trợ triển khai không gián đoạn (zero-downtime deploy) và giữ độ trễ (latency) ổn định khi lưu lượng tăng.
- Chọn thuật toán theo khối lượng công việc (workload): Round Robin, Least Connections, Weighted, hoặc IP Hash.
- Với web/API hiện đại, tầng 7 (Layer 7) thường là lựa chọn mặc định; tầng 4 (Layer 4) phù hợp khi cần thông lượng (throughput) cực cao hoặc dùng các giao thức không phải HTTP.

Cân Bằng Tải Là Gì?

Cân bằng tải trong thiết kế hệ thống (system design) là kỹ thuật phân phối yêu cầu (request) đến nhiều máy chủ phía sau (backend servers) để tránh quá tải một nút (node) và tăng khả năng mở rộng của hệ thống.

Vì Sao Cân Bằng Tải Quan Trọng?

- Tăng khả năng chịu tải khi lưu lượng tăng đột biến.
 - Tăng độ sẵn sàng của hệ thống khi một máy chủ (server) bị lỗi.
 - Giữ hiệu năng ổn định cho người dùng (user) trong môi trường hệ thống phân tán.
 - Là nền tảng cho kiến trúc có tính sẵn sàng cao (high-availability architecture) trong môi trường thực tế.
-

Khi Nào Nên Dừng?

- Khi một máy chủ không còn đủ năng lực (*capacity*) cho thông lượng hiện tại.
- Khi cần giảm thời gian ngừng hoạt động (*downtime*) lúc triển khai (*deploy*).
- Khi hệ thống có nhiều dịch vụ (*service*) hoặc nhiều vùng (đa vùng (*multi-region*)).
- Khi muốn cải thiện độ tin cậy và hiệu năng (*performance*) ở quy mô môi trường thực tế.

Nó Hoạt Động Như Thế Nào?

1. Máy khách (*Client*) gửi yêu cầu đến bộ cân bằng tải.
2. Bộ cân bằng tải chọn máy chủ phía sau (*backend server*) theo thuật toán định tuyến (*routing algorithm*).
3. Yêu cầu được chuyển tiếp (*forward*) đến máy chủ đã chọn.
4. Phản hồi (*Response*) được trả về máy khách thông qua bộ cân bằng tải.

Những Sự Đánh Đổi (*Trade-offs*) Cần Nhớ

- Tầng 4 nhanh hơn nhưng có ít ngữ cảnh để định tuyến.
- Tầng 7 linh hoạt hơn nhưng tốn thêm chi phí xử lý.
- Phiên dính (*Sticky session*) giúp các luồng có trạng thái (*stateful*) hoạt động nhưng làm giảm mức độ phân phối đều.
- Thuật toán đơn giản dễ vận hành hơn nhưng có thể không tối ưu cho khối lượng công việc phức tạp.

Thuật Ngữ Chính (Định Nghĩa Nhanh Cho AI/SEO)

- **Khả năng mở rộng:** khả năng hệ thống tăng công suất khi lưu lượng tăng.
- **Độ tin cậy:** khả năng hệ thống tiếp tục hoạt động ổn định khi có lỗi.
- **Độ trễ:** thời gian từ lúc gửi yêu cầu đến khi nhận được phản hồi.
- **Thông lượng:** số lượng yêu cầu hệ thống xử lý được trong một đơn vị thời gian.

Bạn đã học về các thành phần (*components*) và các mẫu giao tiếp (*communication patterns*). Giờ là lúc đi sâu vào một trong những thành phần quan trọng nhất: **Bộ Cân Bằng Tải**.

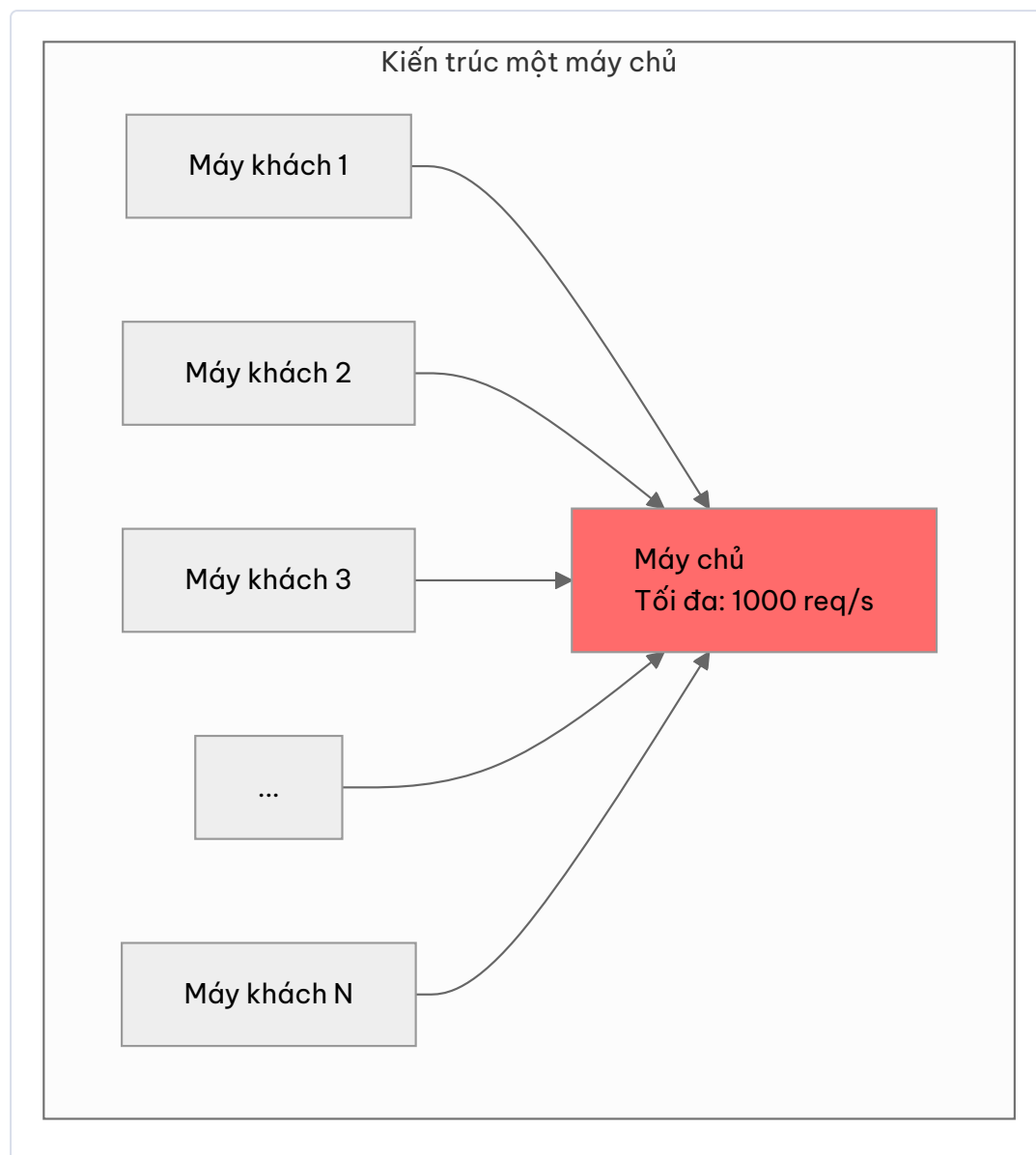
Tôi còn nhớ ngày Black Friday đầu tiên ở công ty khởi nghiệp (*startup*) tôi từng làm. Lưu lượng tăng gấp 50 lần. Một máy chủ duy nhất? Chết ngay lập tức.

Chúng tôi hoảng loạn thêm 5 máy chủ (*servers*). Nhưng rồi nhận ra: **Làm sao lưu lượng biết phải đi đến máy chủ nào?**

Đó là lúc tôi thực sự hiểu giá trị của bộ cân bằng tải. Không chỉ đơn thuần là "phân phối lưu lượng". Mà là **làm cho hệ thống phân tán** (*distributed system*) **hoạt động như một thể thống nhất**.

Tại Sao Cân Bằng Tải Tồn Tại?

Vấn Đề Cơ Bản: Giới Hạn Của Một Máy Chủ Duy Nhất



Kịch bản (Scenario) **thực tế:**

Năng lực máy chủ: 1,000 yêu cầu/giây
Lưu lượng bình thường: 800 req/s (80% năng lực)
Lưu lượng Black Friday: 5,000 req/s

Kết quả:

- Thời gian phản hồi: 200ms → 15 giây
- Hết thời gian chờ (timeout) khắp nơi
- Máy chủ sập
- Mất doanh thu

Giải Pháp 1: Mở Rộng Theo Chiều Dọc (*Vertical Scaling*) (Scale Up)

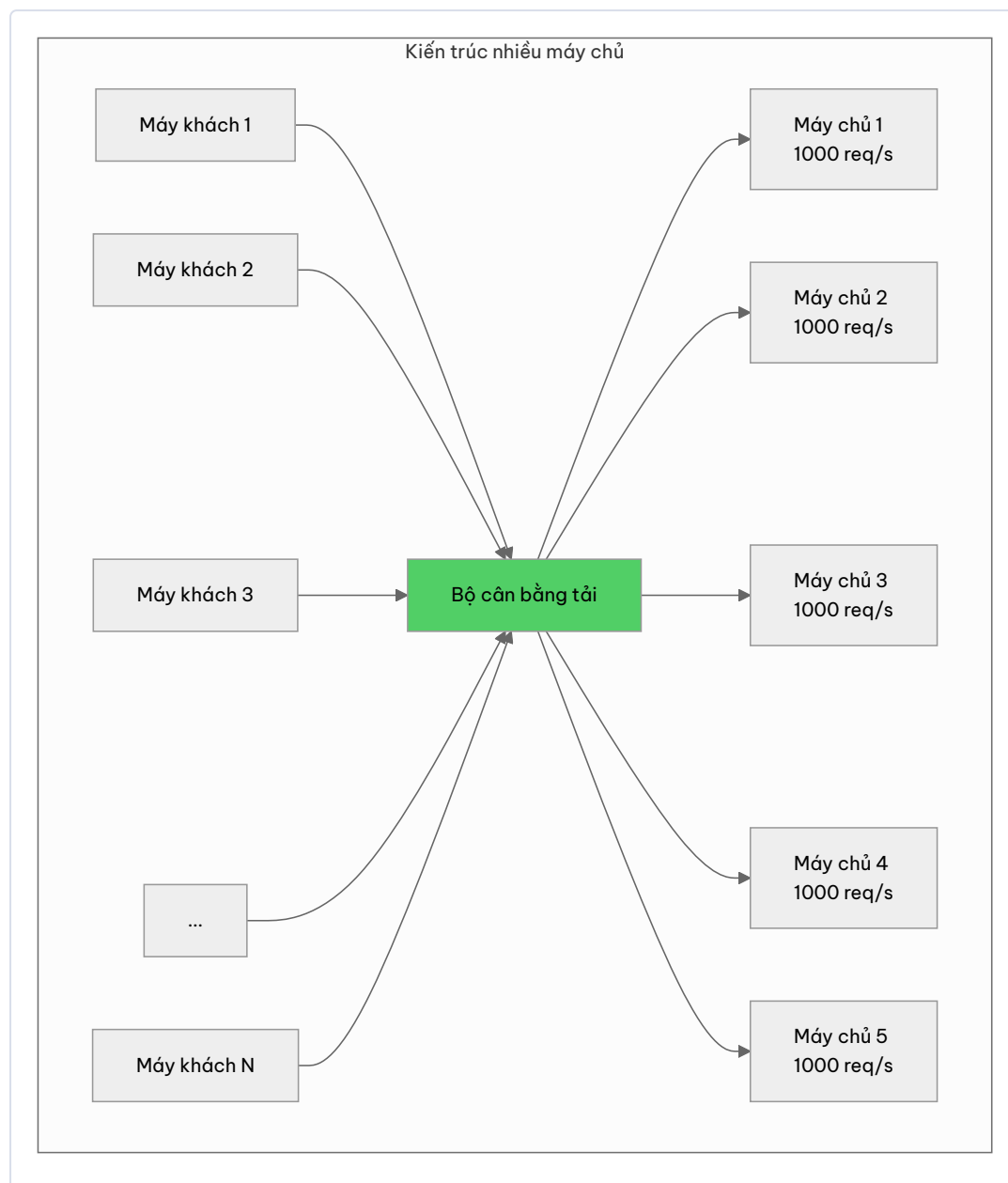
Nâng cấp máy chủ:
4 lõi CPU → 16 lõi
8GB RAM → 64GB RAM
Chi phí: \$200/tháng → \$1,200/tháng

Kết quả: Xử lý được 4,000 req/s
Vấn đề: Vẫn thiếu 1,000 req/s
Và... không thể nâng cấp mãi mãi

Hạn chế:

- Có giới hạn vật lý (không thể mua CPU vô hạn)
- Đắt đỏ (chi phí (*cost*) không tăng tuyến tính)
- Điểm lỗi duy nhất (máy chủ chết = hệ thống chết)
- Thời gian ngừng hoạt động khi nâng cấp

Giải Pháp 2: Mở Rộng Theo Chiều Ngang (Horizontal Scaling) (Scale Out)



Kết quả:

5 máy chủ × 1,000 req/s = tổng năng lực 5,000 req/s
Chi phí: 5 × \$200 = \$1,000/tháng
Có thể thêm máy chủ vô hạn

Nhưng có một vấn đề:

Với 5 máy chủ, **máy khách sẽ gửi yêu cầu đến máy chủ nào?**

Ai quyết định? Làm sao phân phối cho đều?

→ Đó chính là **nhiệm vụ của Bộ Cân Bằng Tải**.

Bộ Cân Bằng Tải Là Gì?

Định nghĩa:

Bộ cân bằng tải là thành phần (*component*) đứng giữa máy khách (*clients*) và máy chủ, nhận tất cả yêu cầu (*requests*) đến và phân phối chúng một cách đều đặn (hoặc thông minh) giữa nhiều máy chủ phía sau.

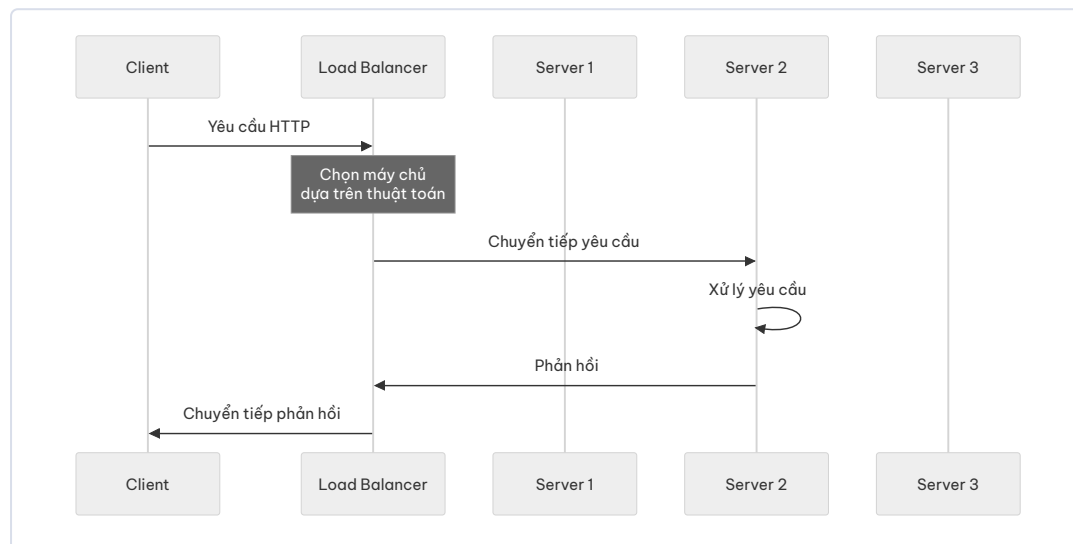
Một phép so sánh đơn giản:

Hãy hình dung quầy làm thủ tục ở sân bay:

Khách hàng đến → Nhân viên điều phối → Quầy check-in
(máy khách) (bộ cân bằng tải) (máy chủ)

Nhân viên: "Quầy 3 đang trống, mời anh qua đây"

Luồng kỹ thuật:



Bộ cân bằng tải nhận yêu cầu, chọn máy chủ theo thuật toán, chuyển tiếp yêu cầu, nhận phản hồi rồi trả về cho máy khách.

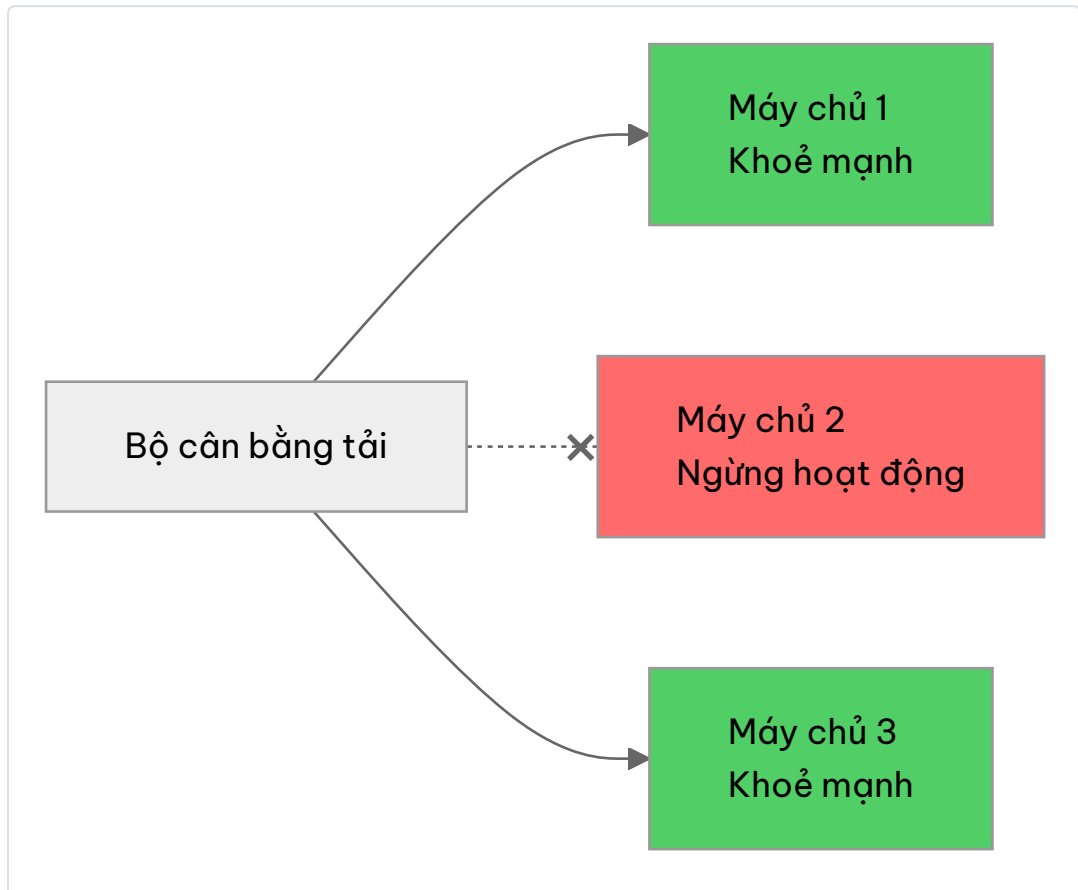
Lợi Ích Của Cân Bằng Tải

Lợi Ích 1: Tăng Năng Lực

Trước: 1 máy chủ = 1,000 req/s
Sau: 10 máy chủ = 10,000 req/s

Mở rộng tuyến tính (thêm máy chủ = thêm năng lực)

Lợi Ích 2: Tính Sẵn Sàng Cao (High Availability)



Khi Máy chủ 2 chết, bộ cân bằng tải tự động loại nó khỏi nhóm máy chủ (pool) và định tuyến lưu lượng đến các máy chủ còn lại.

Không có bộ cân bằng tải (LB):

Máy chủ chết → Người dùng gặp lỗi → Ngừng hoạt động

Có bộ cân bằng tải:

Máy chủ chết → LB phát hiện → Định tuyến đến các máy chủ khỏe mạnh
→ Người dùng không bị ảnh hưởng
→ Không có thời gian ngừng hoạt động

Lợi Ích 3: Triển Khai Không Gián Đoạn (*Zero-Downtime Deployment*)

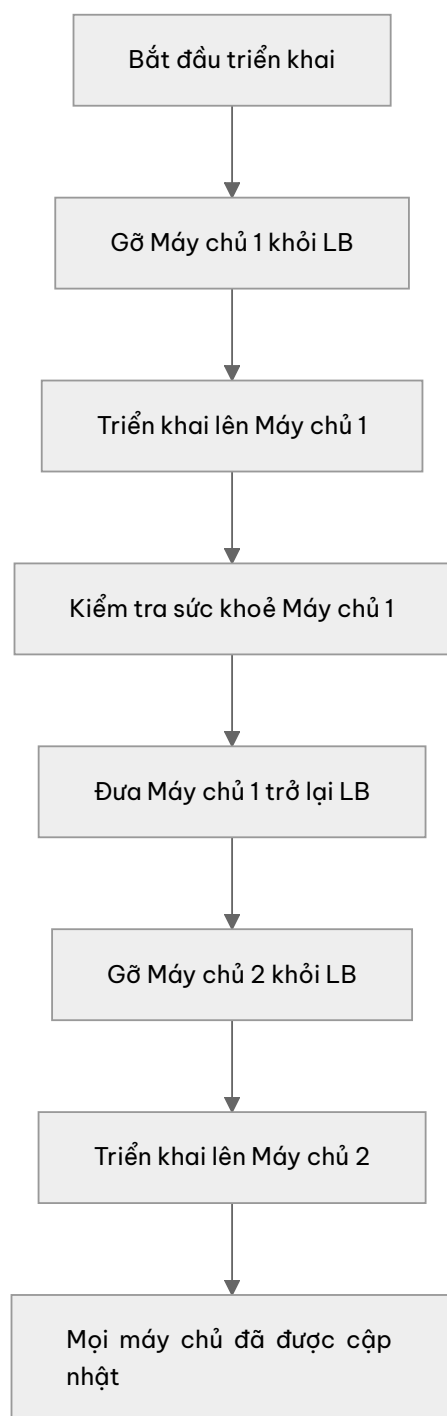
Triển khai truyền thống:

1. Dừng máy chủ
 2. Triển khai mã mới
 3. Khởi động máy chủ
- Ngừng hoạt động: 2-5 phút

Với bộ cân bằng tải:

1. Gỡ Máy chủ 1 khỏi nhóm
 2. Triển khai lên Máy chủ 1
 3. Đưa Máy chủ 1 trở lại
 4. Lặp lại với Máy chủ 2, 3, 4, 5...
- Ngừng hoạt động: 0 giây

Chiến lược triển khai cuộn chiếu (*rolling deployment*):



Lợi Ích 4: Phân Bố Theo Địa Lý

Người dùng ở Mỹ → Bộ cân bằng tải Mỹ → Máy chủ Mỹ (độ trễ~thấp)
Người dùng ở châu Âu → Bộ cân bằng tải EU → Máy chủ EU (độ trễ~thấp)
Người dùng ở châu Á → Bộ cân bằng tải châu Á → Máy chủ châu Á (độ trễ~thấp)

Tác động:

Không phân bố theo địa lý:
Người dùng ở Brazil → Máy chủ ở Singapore → độ trễ~400ms

Có phân bố theo địa lý:
Người dùng ở Brazil → Máy chủ ở São Paulo → độ trễ~20ms

Cải thiện 20 lần!

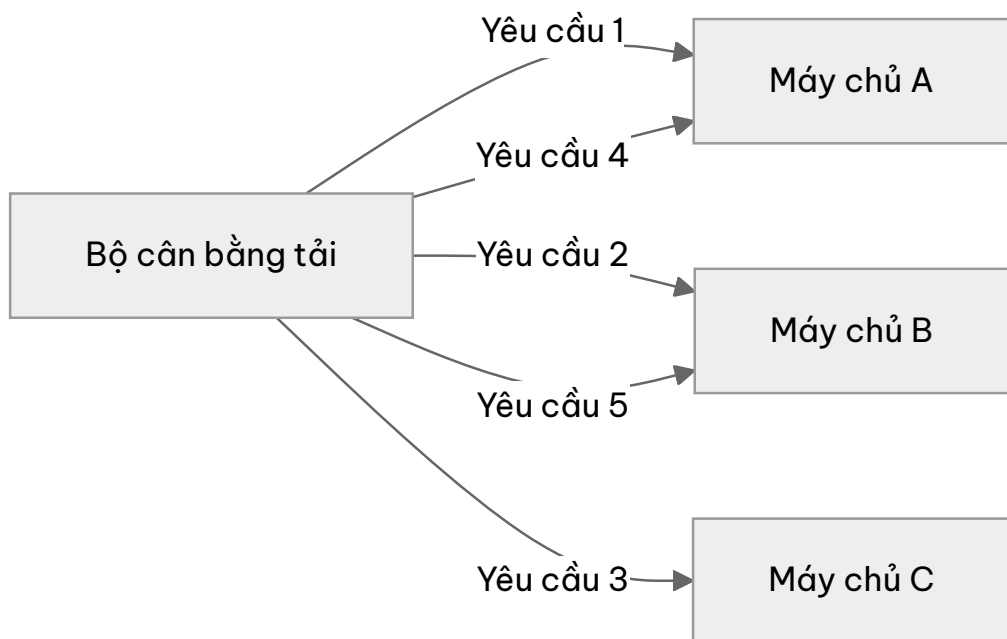
Các Thuật Toán Cân Bằng Tải

Đây là phần quan trọng nhất: **Làm thế nào bộ cân bằng tải quyết định yêu cầu nào đi đến máy chủ nào?**

Thuật Toán 1: Round Robin (Xoay Vòng)

Cách hoạt động:

Phân phối các yêu cầu tuần tự, mỗi máy chủ lần lượt nhận một yêu cầu theo vòng tròn.



Ý tưởng mã nguồn:

```
servers = ['server_a', 'server_b', 'server_c']
current_index = 0

def get_next_server():
    global current_index
    server = servers[current_index]
    current_index = (current_index + 1) % len(servers)
    return server

# Request flow
get_next_server() # → server_a
get_next_server() # → server_b
get_next_server() # → server_c
get_next_server() # → server_a (cycle repeats)
```

Ưu điểm:

Cực kỳ đơn giản

Phân phối đều về số lượng yêu cầu

Không cần theo dõi trạng thái (*state*)

Chi phí phụ (*Overhead*) thấp

Nhược điểm:

Không quan tâm đến tải (*load*) hiện tại của máy chủ

Không biết yêu cầu nào nặng, yêu cầu nào nhẹ

Giả định mọi yêu cầu và máy chủ đều giống nhau

Kịch bản có vấn đề:

Yêu cầu 1 → Máy chủ A: Tải lên tệp 5GB (mất 10 phút)
Yêu cầu 2 → Máy chủ B: Lấy hồ sơ người dùng (mất 10ms)
Yêu cầu 3 → Máy chủ C: Lấy hồ sơ người dùng (mất 10ms)
Yêu cầu 4 → Máy chủ A: Lấy hồ sơ người dùng (phải chờ sau lượt tải lên...)

Máy chủ A: Quá tải (đang xử lý yêu cầu nặng)
Máy chủ B, C: Rảnh rỗi (đã xong các yêu cầu nhanh)

Round Robin không biết máy chủ A đang bận!

Khi nào dùng:

Các yêu cầu có thời gian thực thi (*execution time*) tương tự nhau

Các máy chủ có cấu hình giống nhau

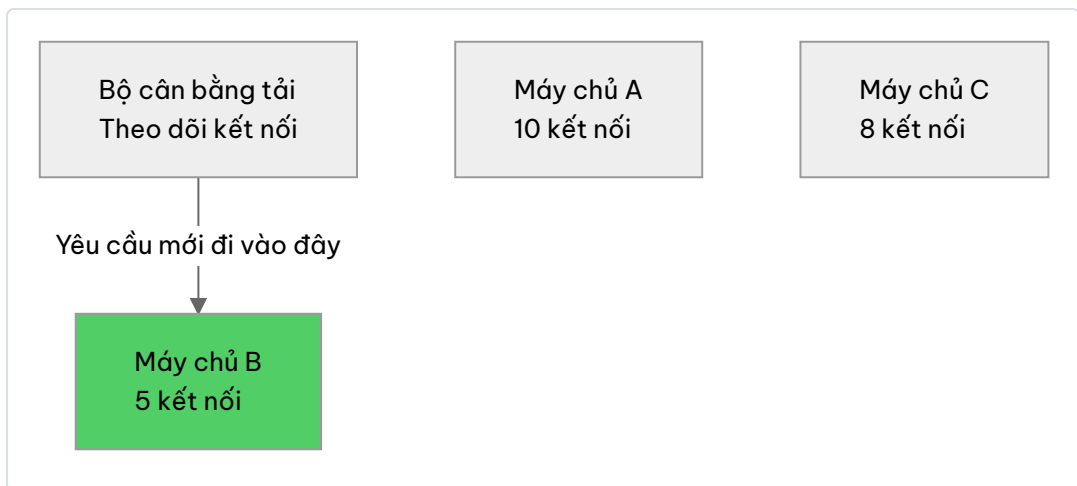
Các tình huống sử dụng (*use cases*) đơn giản

Lựa chọn mặc định khi mới bắt đầu

Thuật Toán 2: Least Connections (Ít Kết Nối Nhất)

Cách hoạt động:

Định tuyến yêu cầu đến máy chủ có **ít kết nối đang hoạt động** (*active connections*) nhất.



Ý tưởng mã nguồn:

```
servers = {
    'server_a': {'connections': 10, 'url': '192.168.1.10'},
    'server_b': {'connections': 5, 'url': '192.168.1.11'},
    'server_c': {'connections': 8, 'url': '192.168.1.12'}
}

def get_next_server():
    # Tìm server có ít connections nhất
    min_server = min(servers.items(),
                     key=lambda x: x[1]['connections'])
    return min_server[0]

# New request arrives
server = get_next_server() # → server_b (5 connections)
servers[server]['connections'] += 1 # Now 6

# Next request
server = get_next_server() # → server_b again (6 connections)
```

Ưu điểm:

Cân bằng tải tốt hơn Round Robin

Thích ứng với các yêu cầu có thời gian thực thi khác nhau

Hiệu quả cho các kết nối kéo dài (*long-lived connections*)

Nhược điểm:

Phức tạp hơn (phải theo dõi số kết nối)

Chi phí phụ để cập nhật trạng thái

Cần đồng bộ hoá (*synchronization*) giữa nhiều phiên bản bộ cân bằng tải

Khi nào dùng:

Các yêu cầu có thời gian thực thi rất khác nhau

Kết nối WebSocket

Thăm dò dài (*Long-polling*)

Tải lên/tải xuống tệp

Truyền phát (*Streaming*)

Trải nghiệm cá nhân:

Tôi từng có một API với 2 loại điểm cuối (*endpoints*):

```
GET /users/{id} → thời gian thực thi 10ms
POST /reports/generate → thời gian thực thi 60 giây
```

Dùng Round Robin → Các máy chủ nhận yêu cầu tạo báo cáo bị kẹt, trong khi các máy chủ khác thì rảnh rỗi.

Chuyển sang Least Connections → Tải được cân bằng đều, thời gian phản hồi cải thiện gấp 5 lần.

Thuật Toán 3: Weighted Round Robin (Xoay Vòng Có Trọng Số)

Phát biểu vấn đề:

Các máy chủ có cấu hình khác nhau. Round Robin đối xử với mọi máy chủ như nhau → lãng phí năng lực.

```
Máy chủ A: 16GB RAM, 8 lõi CPU (mạnh)
Máy chủ B: 8GB RAM, 4 lõi CPU (bình thường)
Máy chủ C: 8GB RAM, 4 lõi CPU (bình thường)
```

```
Round Robin: Mỗi máy chủ nhận số yêu cầu như nhau
→ Máy chủ A bị dùng chưa hết công suất
→ Máy chủ B, C có thể bị quá tải
```

Giải pháp: Gán trọng số (*weights*)

```

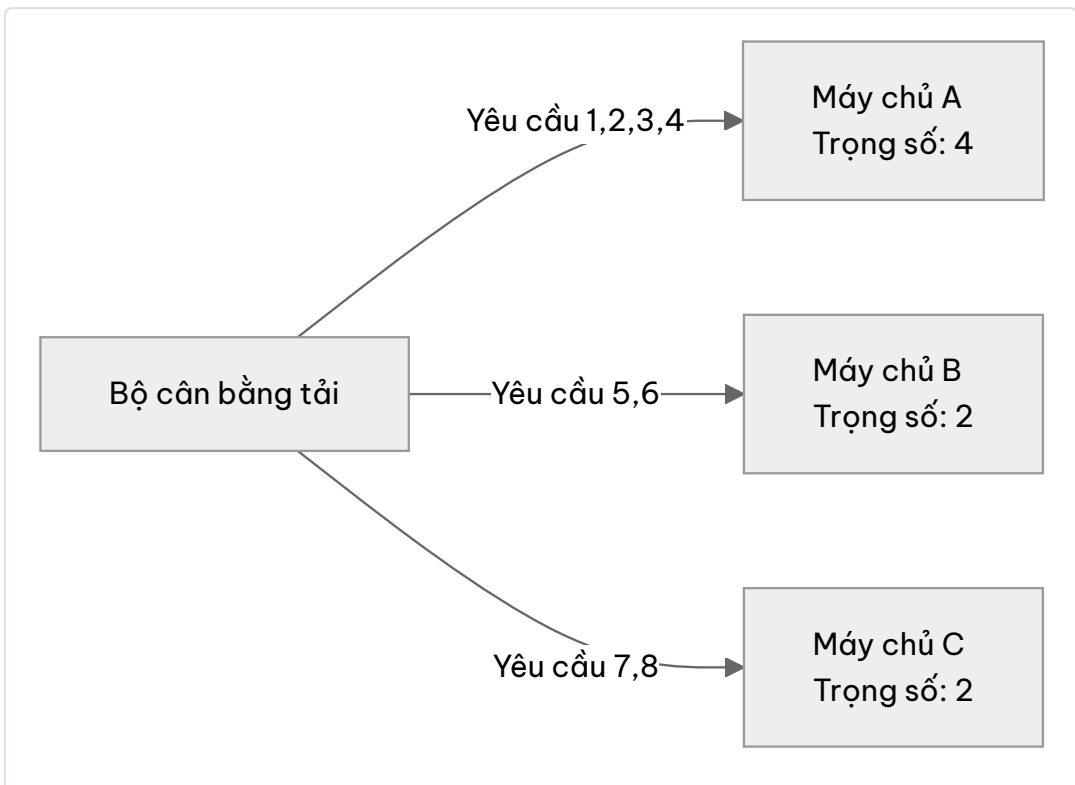
servers = [
    {'name': 'server_a', 'weight': 4}, # Powerful
    {'name': 'server_b', 'weight': 2}, # Normal
    {'name': 'server_c', 'weight': 2}  # Normal
]

# Distribution:
# Server A gets 4 requests
# Server B gets 2 requests
# Server C gets 2 requests
# Total: 8 requests per cycle

# In ratio: 4:2:2 or 2:1:1

```

Luồng yêu cầu:



Ý tưởng triển khai:

```
def weighted_round_robin():
    # Expand servers theo weight
    expanded = []
    for server in servers:
        expanded.extend([server['name']] * server['weight'])

    # expanded = ['server_a', 'server_a', 'server_a', 'server_a',
    #             'server_b', 'server_b',
    #             'server_c', 'server_c']

    # Apply Round Robin trên expanded list
    current_index = 0
    while True:
        server = expanded[current_index]
        current_index = (current_index + 1) % len(expanded)
        yield server
```

Ưu điểm:

Tận dụng các máy chủ mạnh hiệu quả hơn
 Phân bổ tài nguyên linh hoạt
 Tiết kiệm chi phí (kết hợp nhiều loại máy chủ)

Nhược điểm:

Phải cấu hình trọng số thủ công
 Cần điều chỉnh trọng số khi thêm/bớt máy chủ
 Không động (không thích ứng với tải theo thời gian thực)

Khi nào dùng:

Các máy chủ có cấu hình khác nhau
 Kết hợp máy ảo giá rẻ ngắn quãng (*spot instances*) và máy ảo đặt trước (*reserved instances*) (trên đám mây)
 Chuyển đổi (*Migration*) dần dần (máy chủ cũ trọng số (*weight*) thấp, máy chủ mới trọng số cao)

Thuật Toán 4: IP Hash (Băm Địa Chỉ IP – Băm Nhất Quán (*Consistent Hashing*))

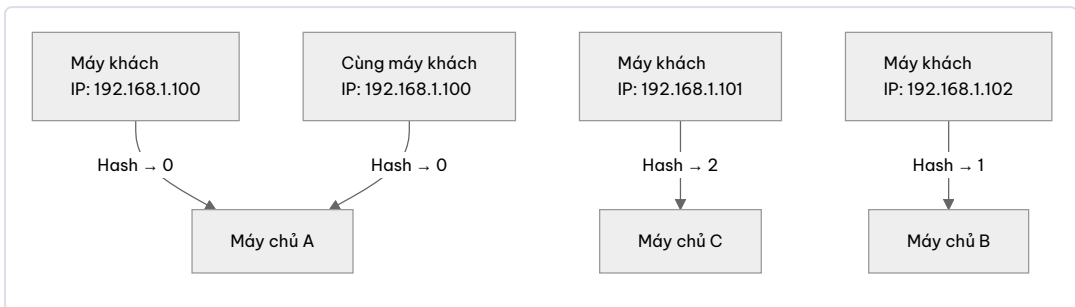
Cách hoạt động:

Băm địa chỉ IP của máy khách rồi ánh xạ tới một máy chủ. Cùng máy khách → Cùng máy chủ.


```
def get_server(client_ip):
    hash_value = hash(client_ip)
    server_index = hash_value % num_servers
    return servers[server_index]

# Example
get_server('192.168.1.100') # → server_a
get_server('192.168.1.101') # → server_c
get_server('192.168.1.100') # → server_a (same!)
```

Sơ đồ:



Cùng một IP máy khách luôn được bấm tới cùng một máy chủ.

Ưu điểm:

Duy trì phiên (session) (phiên dính (sticky sessions))

Lưu đệm (Caching) phía máy chủ hiệu quả (cùng người dùng → cùng máy chủ → trúng bộ nhớ đệm (cache hit))

Các ứng dụng có trạng thái hoạt động được

Nhược điểm:

Phân phối không đều nếu một số máy khách gửi nhiều yêu cầu

Khi thêm/bớt máy chủ → giá trị băm thay đổi → mất phiên (sessions)

Không cân bằng được tải nếu một máy khách gửi ồ ạt yêu cầu

Khi nào dùng:

Cần gắn phiên (session affinity)

Kết nối WebSocket

Lưu đệm phía máy chủ cho từng người dùng

Ứng dụng cũ không hỗ trợ phiên phân tán (distributed sessions)

Phương án tốt hơn:

Thay vì IP Hash, hãy dùng **kho lưu phiên dùng chung** (*shared session store*) (Redis):

```
# IP Hash approach
def handle_request(client_ip):
    server = hash(client_ip) % num_servers
    # Session stored on server → tied to server

# Shared session approach
def handle_request(session_id):
    session = redis.get(f"session:{session_id}")
    # Any server can handle → true stateless
```

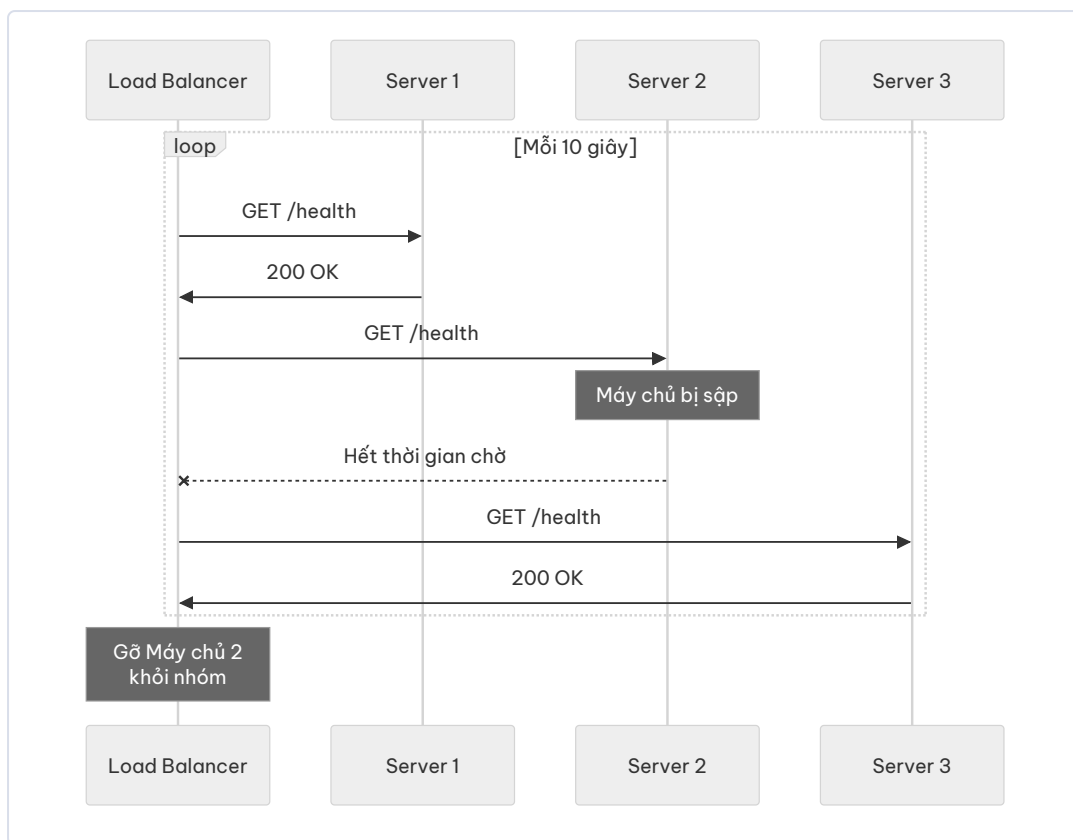
Kiểm Tra Sức Khỏe (*Health Checks*): Tránh Định Tuyển Đến Máy Chủ Đã Chết

Vấn đề:

Máy chủ 2 bị sập
Bộ cân bằng tải không biết
Vẫn tiếp tục định tuyển 33% lưu lượng đến Máy chủ 2
Người dùng gặp lỗi

Giải pháp: Kiểm Tra Sức Khỏe

Bộ cân bằng tải định kỳ gửi ping đến các máy chủ để xác minh tình trạng sức khỏe.



Triển Khai Kiểm Tra Sức Khỏe (Health Check)

Điểm cuối (Endpoint) phía máy chủ:

```

from flask import Flask, jsonify

app = Flask(__name__)

@app.route('/health')
def health_check():
    # Check critical dependencies
    checks = {
        'database': check_database_connection(),
        'cache': check_cache_connection(),
        'disk_space': check_disk_space()
    }

    if all(checks.values()):
        return jsonify({
            'status': 'healthy',
            'checks': checks
        }), 200
    else:
        return jsonify({
            'status': 'unhealthy',
            'checks': checks
        }), 503 # Service Unavailable

def check_database_connection():
    try:
        db.execute("SELECT 1")
        return True
    except:
        return False

def check_cache_connection():
    try:
        cache.ping()
        return True
    except:
        return False

```

Phía bộ cân bằng tải:

```

import requests
import time

servers = [
    {'url': 'http://server1.example.com', 'healthy': True},
    {'url': 'http://server2.example.com', 'healthy': True},
    {'url': 'http://server3.example.com', 'healthy': True}
]

def health_check_loop():
    while True:
        for server in servers:
            try:
                response = requests.get(
                    f"{server['url']}/health",
                    timeout=2 # 2 second timeout
                )

                if response.status_code == 200:
                    server['healthy'] = True
                    print(f"{server['url']}: Healthy ")
                else:
                    server['healthy'] = False
                    print(f"{server['url']}: Unhealthy ")

            except requests.exceptions.Timeout:
                server['healthy'] = False
                print(f"{server['url']}: Timeout ")

            except requests.exceptions.ConnectionError:
                server['healthy'] = False
                print(f"{server['url']}: Connection failed ")

        time.sleep(10) # Check every 10 seconds

def get_healthy_servers():
    return [s for s in servers if s['healthy']]

```

Thực Hành Tốt Nhất (Best Practices) Cho Kiểm Tra Sức Khỏe

Thực hành 1: Kiểm tra cả các thành phần phụ thuộc (dependencies), không chỉ máy chủ

```
# BAD: Chỉ return OK
@app.route('/health')
def health():
    return "OK", 200

# GOOD: Check critical dependencies
@app.route('/health')
def health():
    if not database.is_connected():
        return "Database down", 503

    if not cache.is_connected():
        return "Cache down", 503

    if disk_usage() > 90:
        return "Disk full", 503

    return "OK", 200
```

Thực hành 2: Thời gian chờ (Timeout) phù hợp

Quá ngắn (500ms):

- Âm tính giả (máy chủ khỏe mạnh nhưng phản hồi chậm)
- Gỡ bỏ các máy chủ khỏe mạnh một cách không cần thiết

Quá dài (30s):

- Dương tính giả (máy chủ đã chết nhưng mất 30 giây mới phát hiện)
- Người dùng gặp lỗi trong 30 giây

Điểm cân bằng: 2-5 giây

Thực hành 3: Logic thử lại (retry)

```
# BAD: Remove after 1 failed check
if health_check_failed:
    remove_from_pool(server)

# GOOD: Remove after N consecutive failures
server['failed_checks'] = server.get('failed_checks', 0) + 1

if server['failed_checks'] >= 3: # 3 consecutive failures
    remove_from_pool(server)
elif health_check_success:
    server['failed_checks'] = 0 # Reset counter
```

Lý do: Tránh hiện tượng chập chờn (*flapping*) (máy chủ bị gỡ → được thêm lại → bị gỡ → được thêm lại...).

Thực hành 4: Suy giảm có kiểm soát (*Graceful degradation*)

```
# Nếu TẤT CẢ servers unhealthy:
healthy_servers = get_healthy_servers()

if len(healthy_servers) == 0:
    # Option A: Route to all servers anyway (some might work)
    # Option B: Return 503 Service Unavailable
    # Option C: Route to backup servers in different region

    # Choose based on use case
```

Bài học cá nhân:

Tôi từng đặt thời gian chờ của kiểm tra sức khỏe = 500ms. Cơ sở dữ liệu (*Database*) có lúc độ trễ tăng đột biến (*spike*) lên 800ms (vẫn khỏe mạnh, chỉ chậm).

Bộ cân bằng tải phát hiện TẤT CẢ máy chủ đều không khỏe mạnh → Gỡ bỏ tất cả → Không còn máy chủ nào → **Sập toàn bộ hệ thống**.

Bài học: Kiểm tra sức khỏe phải phân biệt được "chậm" và "chết".

Cân Bằng Tải Tầng 4 và Tầng 7

Đây là khái niệm nhiều người hay nhầm lẫn. Hãy cùng phân tích cho rõ ràng.

Ôn Nhanh Mô Hình OSI

```
Tầng 7: Ứng dụng - Application (HTTP, FTP, SMTP)
Tầng 6: Trình diễn - Presentation (SSL/TLS)
Tầng 5: Phiên - Session
Tầng 4: Giao vận - Transport (TCP, UDP)
Tầng 3: Mạng - Network (IP)
Tầng 2: Liên kết dữ liệu - Data Link (Ethernet)
Tầng 1: Vật lý - Physical
```

Cân Bằng Tải Tầng 4 (Tầng Giao Vận (*Transport Layer*))

Hoạt động ở: cấp độ TCP/UDP

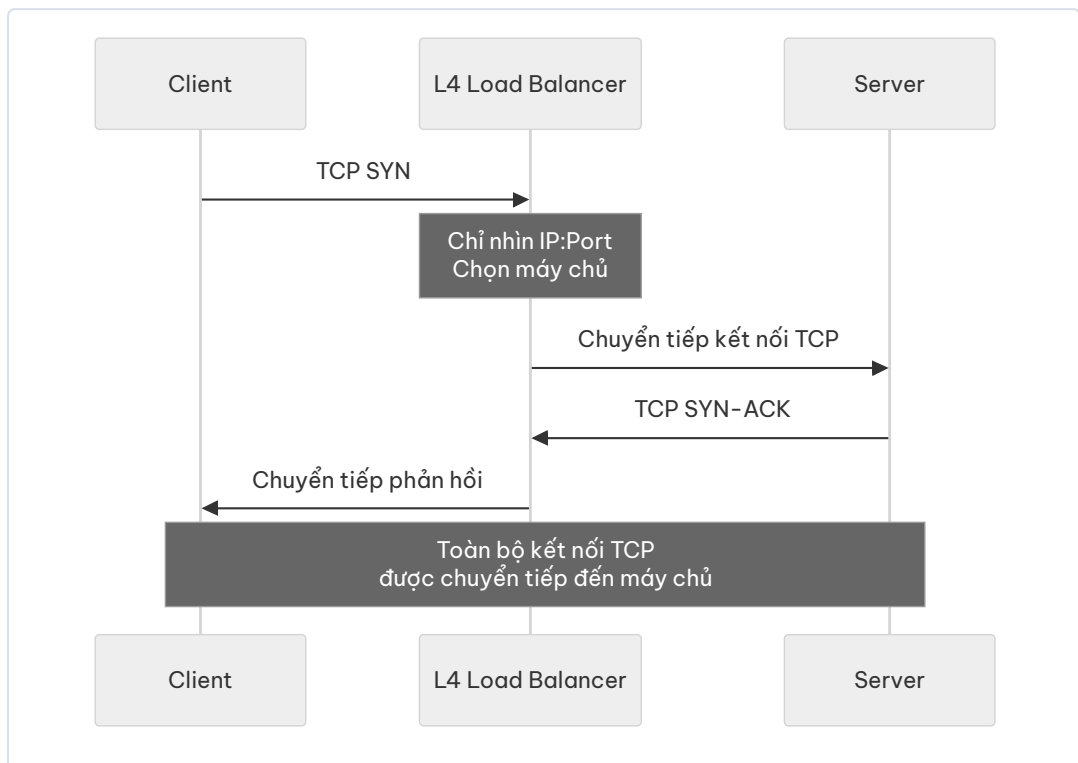
Thông tin có thể thấy:

- Địa chỉ IP nguồn
- Địa chỉ IP đích
- Cổng (Port) nguồn
- Cổng đích

Không thể thấy:

- Tiêu đề (Headers) HTTP
- Đường dẫn URL
- Cookies
- Thân yêu cầu (Request body)
- Dữ liệu ở tầng ứng dụng (application-level)

Cách hoạt động:



Ví dụ:

Mọi lưu lượng đến cổng 443 → Các máy chủ HTTPS phía sau
Mọi lưu lượng đến cổng 5432 → Các bản sao PostgreSQL
Mọi lưu lượng đến cổng 6379 → Cụm Redis

Định tuyến đơn giản dựa trên cổng

Ưu điểm:

Cực nhanh (chỉ chuyển tiếp gói tin đơn giản)

Độ trễ thấp (+5-10ms)

Không phụ thuộc giao thức (hoạt động với bất kỳ giao thức TCP/UDP nào)

Thông lượng cao (hàng triệu kết nối/giây)

Nhược điểm:

Không thể định tuyến dựa trên URL

Không thể định tuyến dựa trên tiêu đề

Không thể sửa đổi yêu cầu/phản hồi (*responses*)

Khả năng "thông minh" hạn chế

Tình huống sử dụng:

- Các giao thức không phải HTTP (cơ sở dữ liệu (*databases*), ứng dụng TCP tùy biến)
- Yêu cầu hiệu năng cực cao
- Định tuyến đơn giản (mọi lưu lượng đến cùng một nhóm máy chủ phía sau (*backends*))
- Máy chủ game, VoIP

Công cụ ví dụ:

- AWS Network Load Balancer (NLB)
- HAProxy (chế độ TCP)
- IPVS (trong nhân Linux)

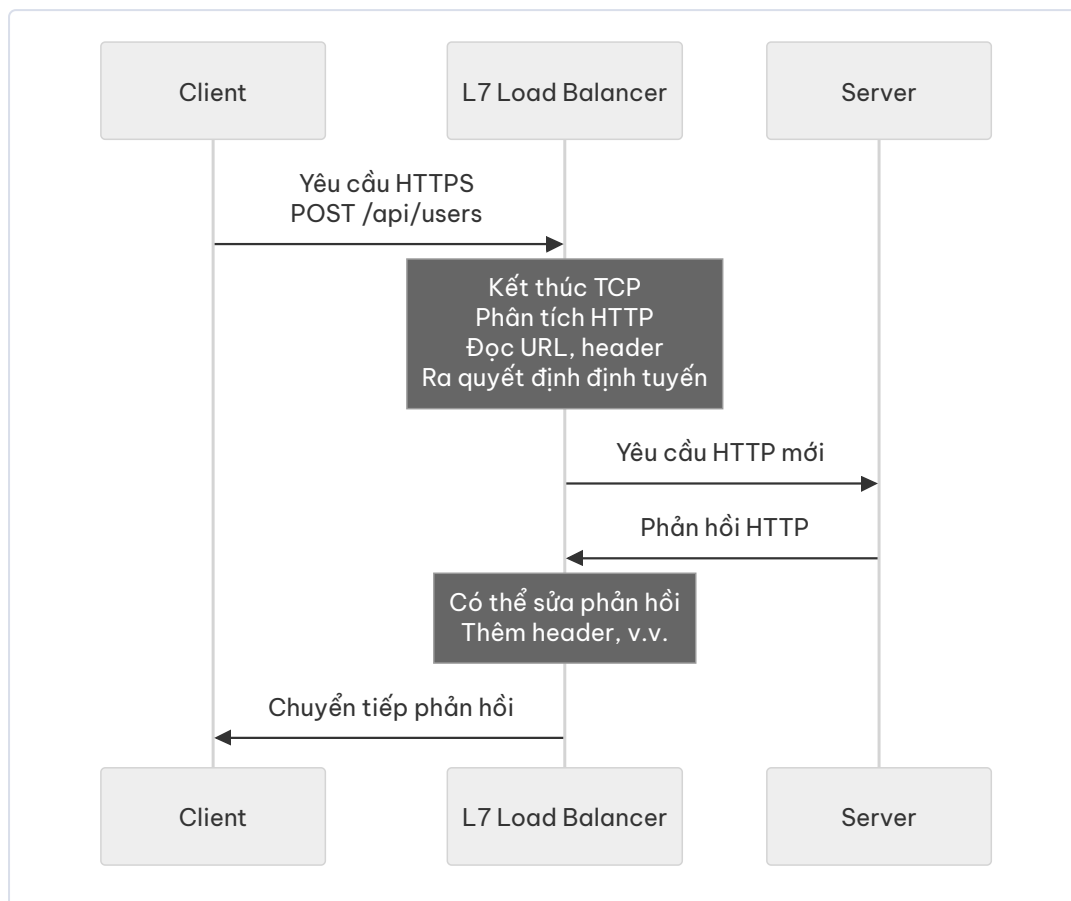
Cân Bằng Tải Tầng 7 (Tầng Ứng Dụng (*Application Layer*))

Hoạt động ở: cấp độ HTTP/HTTPS

Thông tin có thể thấy:

- Mọi thứ mà tầng 4 có
- Phương thức HTTP (GET, POST, PUT, DELETE)
- Đường dẫn URL và tham số truy vấn (*query parameters*)
- Tiêu đề HTTP (User-Agent, cookies, tiêu đề tùy biến (*custom headers*))
- Thân yêu cầu
- Thông tin SSL/TLS

Cách hoạt động:



Ví dụ quy tắc định tuyến:

```
# Nginx configuration (Layer 7)

# Route by URL path
location /api/users {
    proxy_pass http://user_service;
}

location /api/orders {
    proxy_pass http://order_service;
}

location /api/payments {
    proxy_pass http://payment_service;
}

# Route by header
if ($http_x_api_version = "v2") {
    proxy_pass http://api_v2_servers;
}

# Route by cookie
if ($cookie_beta_user = "true") {
    proxy_pass http://beta_servers;
}

# Admin traffic to special servers
location /admin {
    proxy_pass http://admin_servers;
    # Can add authentication here
}
```

Tính năng nâng cao:

```
# SSL termination
# LB handles HTTPS, forwards HTTP to backends
Client --HTTPS--> LB --HTTP--> Backend
(encrypted)          (unencrypted, fast)

# Request modification
# Add custom headers
X-Forwarded-For: client_ip
X-Request-ID: unique_id

# Response modification
# Gzip compression
# Add security headers (CORS, CSP)

# Content-based routing
if content_type == "video/mp4":
    route_to_video_servers()
elif content_type == "image/jpeg":
    route_to_image_servers()
```

Ưu điểm:

Định tuyến thông minh (*Intelligent routing*) (URL, tiêu đề, cookies)

Ra quyết định dựa trên nội dung

Có thể sửa đổi yêu cầu/phản hồi

Kết thúc SSL (*SSL termination*) (giảm tải việc mã hoá cho các máy chủ phía sau)

Hoàn hảo cho vi dịch vụ (*microservices*)

Nhược điểm:

Chậm hơn (xử lý nhiều hơn)

Độ trễ cao hơn (+10-50ms)

Chỉ hoạt động với HTTP/HTTPS

Tốn CPU hơn

Tình huống sử dụng:

- Ứng dụng web
- Kiến trúc vi dịch vụ (*Microservices architecture*)
- Định tuyến dựa trên nội dung (*Content-based routing*)
- Thử nghiệm A/B (*A/B testing*)
- Triển khai canary (*Canary deployments*)

Công cụ ví dụ:

- AWS Application Load Balancer (ALB)
- Nginx
- HAProxy (chế độ HTTP)
- Traefik
- Envoy

Bảng So Sánh

Tiêu chí	Tầng 4	Tầng 7
Tốc độ	Rất nhanh	Chậm hơn
Độ trễ	+5-10ms	+10-50ms
Định tuyến	Chỉ IP:Port	URL, header, v.v.
Giao thức hỗ trợ	Mọi TCP/UDP	Chỉ HTTP/HTTPS
Kết thúc SSL	Không	Có
Kiểm tra nội dung	Không	Có
Sửa đổi yêu cầu	Không	Có
Vi dịch vụ	Không lý tưởng	Hoàn hảo
Chi phí (xur'ly)	Thấp	Cao
Tình huống dùng	Định tuyến đơn giản	Định tuyến thông minh

Khung Ra Quyết Định (Decision Framework)

Chọn Tầng 4 khi: Cần hiệu năng tối đa Hàng triệu kết nối Giao thức không phải HTTP (cơ sở dữ liệu, game, VoIP) Định tuyến đơn giản (mọi lưu lượng đến cùng một nhóm máy chủ) Từng mili giây đều quan trọng
Chọn Tầng 7 khi: Ứng dụng web / API Vi dịch vụ (định tuyến theo đường dẫn) Cần kết thúc SSL (SSL termination) Định tuyến dựa trên nội dung Cần sửa đổi yêu cầu/phản hồi Thư' nghiệm A/B / Triển khai canary

Khuyến nghị của tôi:

Với ứng dụng web: **Mặc định dùng tầng 7** (Application Load Balancer).

Sự linh hoạt và tính năng > chênh lệch nhỏ về độ trễ.

Chỉ xuống tầng 4 khi có yêu cầu hiệu năng đã được chứng minh hoặc khi dùng giao thức không phải HTTP.

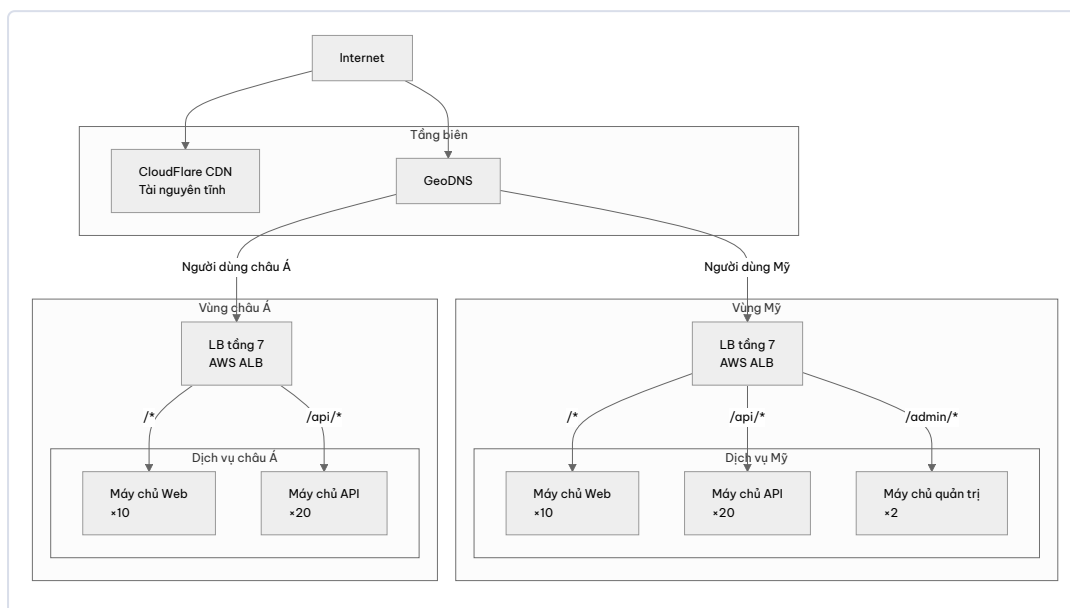
Ví Dụ Kiến Trúc Thực Tế

Hãy xem một hệ thống thương mại điện tử (*e-commerce*) thực tế.

Yêu cầu (*Requirements*):

- 100.000 yêu cầu/giây
- Nhiều dịch vụ (*services*) (Web, API, Quản trị, Thanh toán)
- Người dùng (*Users*) toàn cầu
- Thời gian hoạt động (*Uptime*) 99,99%

Kiến trúc:



Chi tiết cấu hình:

```
# Layer 7 Load Balancer config

# Web servers pool
- path: /*
  backend: web_servers
  algorithm: round_robin
  health_check:
    path: /health
    interval: 10s
    timeout: 5s
    unhealthy_threshold: 3

# API servers pool
- path: /api/*
  backend: api_servers
  algorithm: least_connections # APIs vary in execution time
  health_check:
    path: /api/health
    interval: 10s
    timeout: 5s

# Admin servers pool
- path: /admin/*
  backend: admin_servers
  algorithm: round_robin
  health_check:
    path: /admin/health
    interval: 10s
  ip_whitelist: # Security: Only allow from office IPs
    - 203.0.113.0/24
```

Tại sao thiết kế như vậy:

1. **GeoDNS**: Định tuyến người dùng đến vùng gần nhất (độ trễ thấp)
2. **CDN**: Phục vụ tài nguyên tĩnh (*static assets*) (hình ảnh, CSS, JS) từ điểm biên (*edge*) (nhANH)
3. **Bộ cân bằng tải tầng 7** (*Layer 7 LB*): Định tuyến thông minh theo đường dẫn URL
4. **Các nhóm máy chủ** (*pools*) **khác nhau**: Cô lập (phần quản trị tách biệt khỏi phần công khai)
5. **Các thuật toán khác nhau**:
 - Web: Round Robin (các yêu cầu tương tự nhau)
 - API: Least Connections (thời gian xử lý biến động)
6. **Kiểm tra sức khỏe**: Tự động gỡ bỏ các máy chủ không khoẻ mạnh

7. Danh sách IP cho phép (IP whitelist): Bảo mật cho trang quản trị

Những Điểm Cốt Lõi

Cân bằng tải giải quyết:

- Giới hạn năng lực của một máy chủ duy nhất
- Điểm lỗi duy nhất
- Yêu cầu về tính sẵn sàng cao
- Phân bố theo địa lý

Các thuật toán cốt lõi:

- **Round Robin:** Đơn giản, lựa chọn mặc định, phân phối đều
- **Least Connections:** Tốt hơn khi các yêu cầu có thời gian xử lý khác nhau, kết nối kéo dài
- **Weighted Round Robin:** Khi các máy chủ có cấu hình khác nhau
- **IP Hash:** Duy trì phiên (nhưng kho lưu phiên dùng chung tốt hơn)

Kiểm tra sức khỏe cực kỳ quan trọng:

- Ngăn định tuyến đến các máy chủ đã chết
- Kiểm tra cả các thành phần phụ thuộc, không chỉ máy chủ
- Thời gian chờ phù hợp (2-5 giây)
- Logic thử lại để tránh chập chờn

Tầng 4 và Tầng 7:

- **Tầng 4:** Nhanh, đơn giản, mọi giao thức, định tuyến hạn chế
- **Tầng 7:** Thông minh, linh hoạt, chỉ HTTP, thân thiện với vi dịch vụ
- **Mặc định dùng tầng 7 cho ứng dụng web**

Thực hành tốt nhất:

- Bắt đầu đơn giản (Round Robin + kiểm tra sức khỏe)
- Đo lường trước khi tối ưu thuật toán
- Dùng nhiều bộ cân bằng tải (load balancers) (tránh điểm lỗi duy nhất (SPOF))
- Giám sát (Monitor) các chỉ số (metrics) (tần suất yêu cầu, độ trễ, tỷ lệ lỗi)

- Kiểm thử các kịch bản (*scenarios*) chuyển đổi dự phòng (*failover*)

Tự hỏi khi thiết kế:

1. Thuật toán nào phù hợp với mẫu yêu cầu (*request pattern*)?
2. Kiểm tra sức khỏe có phát hiện sự cố (*failures*) chính xác không?
3. Bộ cân bằng tải có phải là điểm lỗi duy nhất không?
4. Cần tầng 4 hay tầng 7?

Cân bằng tải không phải là thứ tùy chọn. Nó là **nền tảng của hệ thống phân tán**.

Nắm vững nó = hệ thống của bạn có khả năng mở rộng (*scalable*) và đáng tin cậy (*reliable*).

Lưu Đệm (Caching): Tăng Hiệu Năng (Performance) Quan Trọng Nhất

Có một câu hỏi mà các kỹ sư mới vào nghề (*junior engineers*) thường hỏi tôi: "API chậm, em nên làm gì?"

10 năm trước, tôi sẽ nói: "Tối ưu các truy vấn (*queries*) cơ sở dữ liệu (*database*)."

Giờ tôi nói: "Dùng bộ nhớ đệm (*cache*) trước. Tối ưu sau."

Tại sao? Vì tôi đã học một bài học đắt giá.

Năm 2015, hệ thống tôi quản lý bị chậm. Tôi mất 2 tuần tối ưu truy vấn, viết lại chỉ mục (*indexes*), tinh chỉnh tham số cơ sở dữ liệu. Cải thiện được bao nhiêu? 20%.

Một kiến trúc sư lâu năm (*senior architect*) ghé qua, thêm tầng bộ nhớ đệm (*cache layer*) Redis trong 2 giờ. Cải thiện được bao nhiêu? **Nhanh hơn 10 lần.**

Đó là lúc tôi hiểu: **Bộ nhớ đệm không phải là một kỹ thuật tối ưu. Nó là một quyết định kiến trúc.**

Tại Sao Bộ Nhớ Đệm Tồn Tại?

Khoảng Cách Tốc Độ Cơ Bản

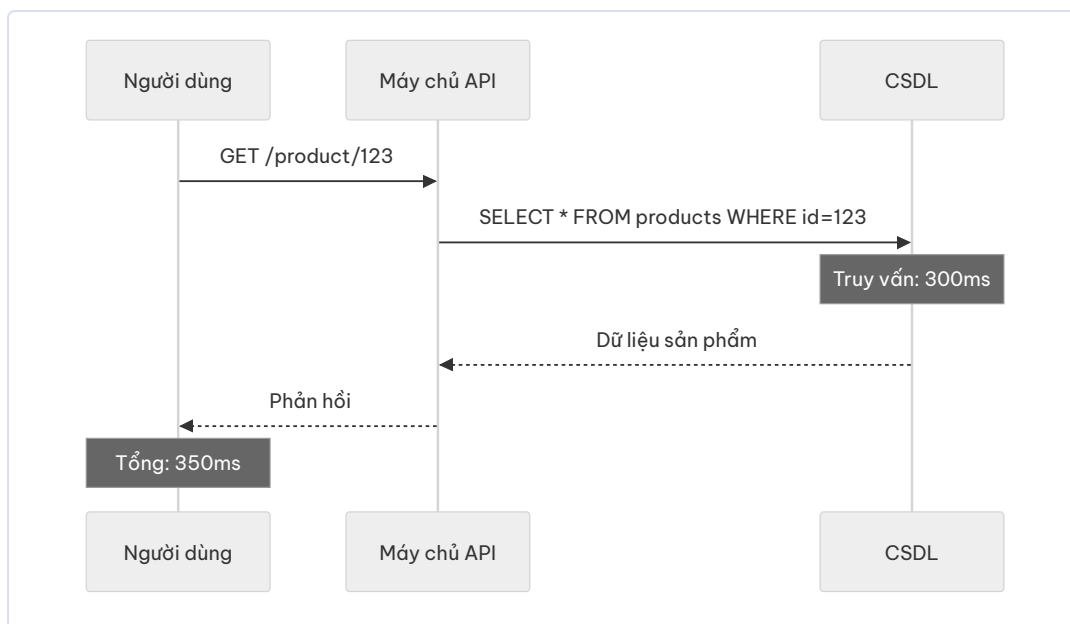
Bộ đệm CPU:	1 nano giây	
RAM:	100 nano giây	(chậm hơn 100 lần)
SSD:	150 micro giây	(chậm hơn 150.000 lần)
Mạng:	50 mili giây	(chậm hơn 50.000.000 lần)
HDD:	10 mili giây	(chậm hơn 10.000.000 lần)

Hệ quả: Mỗi lần bạn truy vấn (*query*) cơ sở dữ liệu qua mạng, bạn mất 50 triệu chu kỳ CPU.

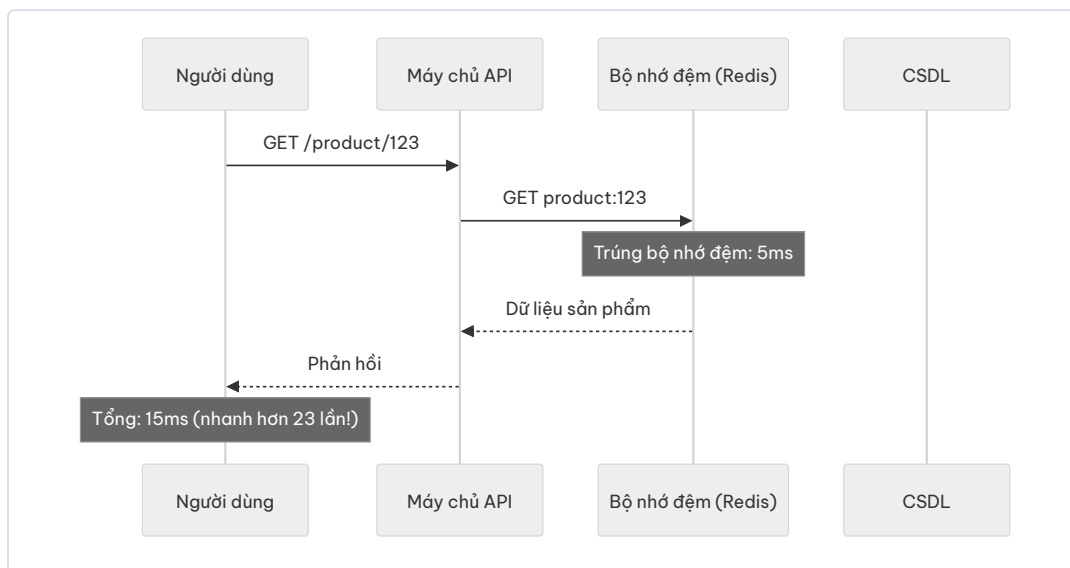
Giải pháp: Lưu kết quả ở gần người dùng hơn. Đó chính là bộ nhớ đệm.

Kịch Bản Thực Tế

Không có bộ nhớ đệm:



Có bộ nhớ đệm:



Khi Nào Bộ Nhớ Đệm Hợp Lý

Không phải mọi thứ đều nên lưu đệm. Hãy dùng khung tư duy (*framework*) này:

Ma Trận Quyết Định Lưu Đệm:

Câu hỏi 1: Thiên về đọc? (tỷ lệ đọc:ghi > 10:1)

↓ CÓ

Câu hỏi 2: Dữ liệu ít thay đổi?

↓ CÓ

Câu hỏi 3: Cùng một dữ liệu được yêu cầu nhiều lần?

↓ CÓ

Câu hỏi 4: Truy vấn tốn kém? (> 100ms)

↓ CÓ

→ DỪNG BỘ NHỚ ĐỆM

Có câu nào KHÔNG? → Cần nhắc kỹ

Ví dụ:

NÊN lưu đệm:

- Danh mục sản phẩm (đọc 1000 lần, ghi 1 lần)
- Hồ sơ người dùng (đọc nhiều, hiếm khi cập nhật)
- Trang tĩnh
- Phản hồi API cho dữ liệu công khai

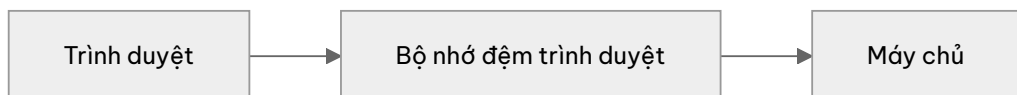
KHÔNG NÊN lưu đệm:

- Giá cổ phiếu thời gian thực (thay đổi liên tục)
- Gợi ý cá nhân hoá (mỗi người dùng một khác)
- Bộ đếm thiên về ghi (lượt thích, lượt xem tăng liên tục)
- Tập dữ liệu nhỏ (< 1000 bản ghi, CSDL đã đủ nhanh)

Các Tầng Bộ Nhớ Đệm: Từ Máy Khách (Client) Đến Cơ Sở Dữ Liệu

Bộ nhớ đệm không chỉ là Redis. Nó là một **chiến lược nhiều tầng**.

Tầng 1: Bộ Nhớ Đệm Trình Duyệt (Browser Cache)



Cách hoạt động:

```
HTTP/1.1 200 OK
Cache-Control: public, max-age=31536000, immutable
```

Các đánh đổi (Trade-offs):

Nhanh nhất (không qua mạng)
Giảm tải cho máy chủ
Khó vô hiệu hoá (người dùng không làm mới trang)
Mỗi người dùng một bộ nhớ đệm riêng

Khi nào nên dùng:

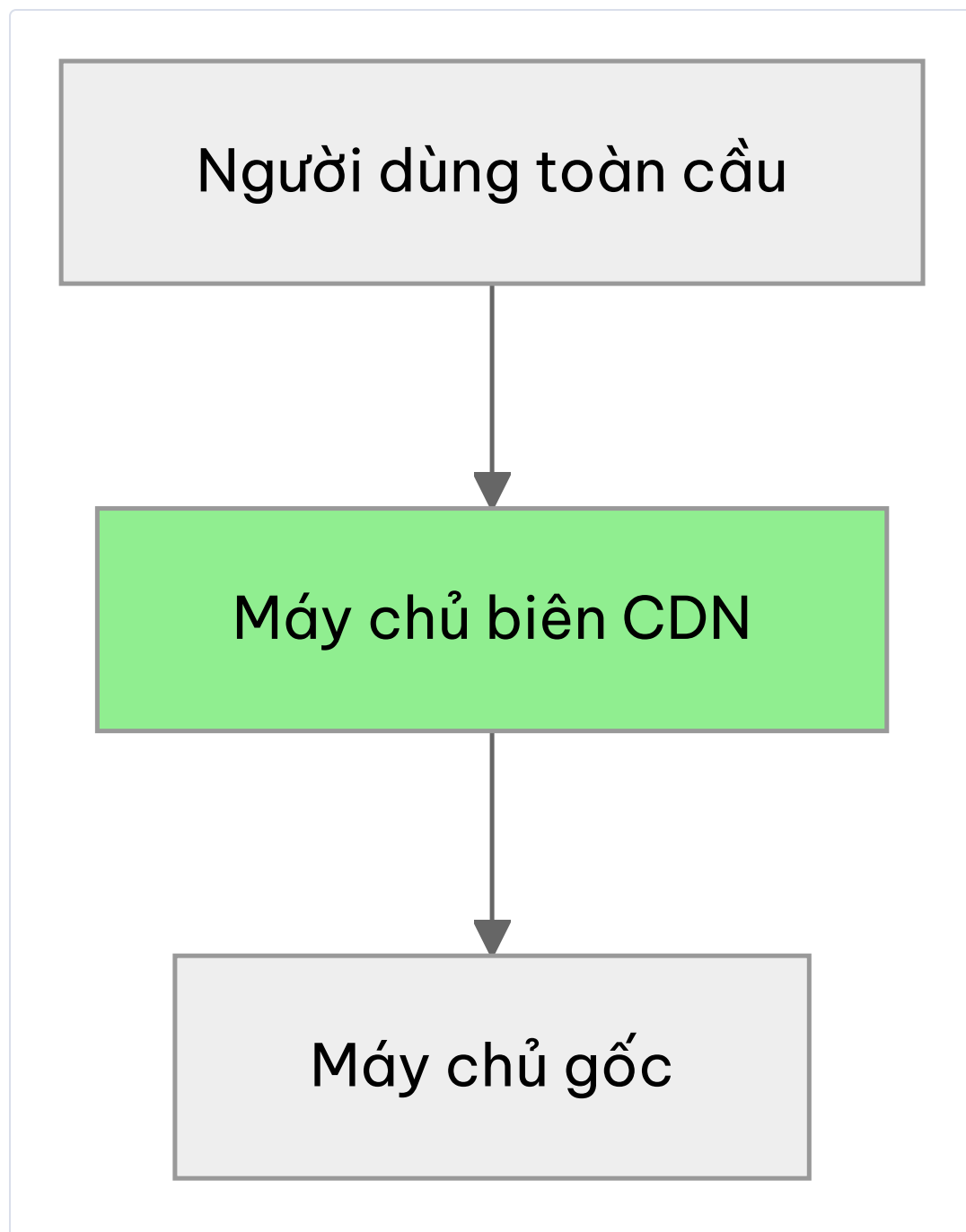
Tài nguyên tĩnh:

- Tập hình ảnh, CSS, JS
- Phong chữ, biểu tượng
- URL có phiên bản (/app.v2.js)

KHÔNG dùng cho:

- Trang HTML (cần luôn mới)
- Phản hồi API
- Nội dung cá nhân hoá

Tầng 2: Bộ Nhớ Đệm CDN



Phân bố theo địa lý:

Người dùng ở Brazil → máy chủ biên Brazil (20ms)
Người dùng ở Tokyo → máy chủ biên Tokyo (15ms)
Người dùng ở London → máy chủ biên London (10ms)

Tất cả đều phục vụ từ bộ nhớ đệm, không gọi tới máy chủ gốc

Cấu hình:

```
// Cloudflare Workers example
addEventListener('fetch', event => {
  const cache = caches.default
  const response = cache.match(event.request)

  if (response) {
    return response // Cache hit
  }

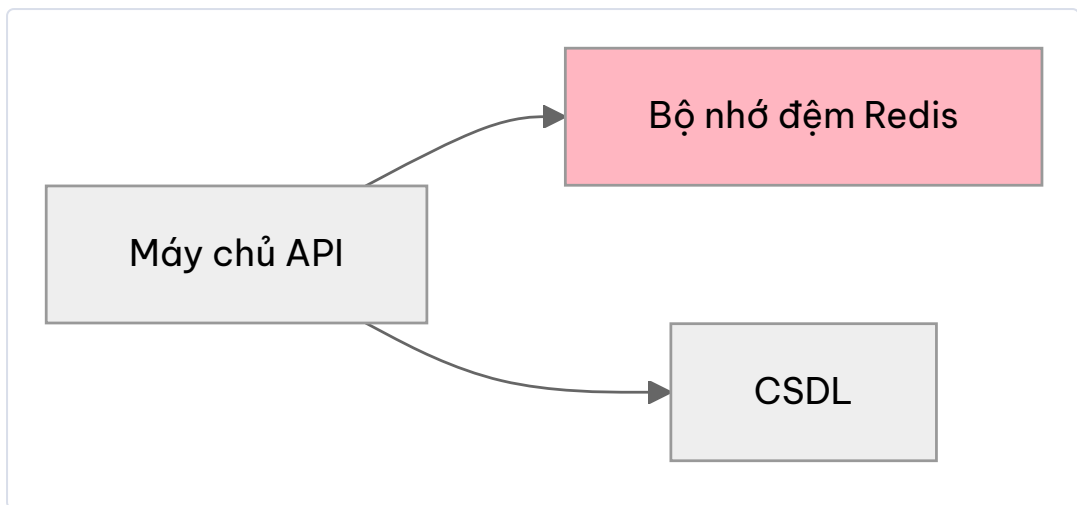
  // Cache miss → fetch from origin → cache it
  const originResponse = fetch(event.request)
  event.waitUntil(
    cache.put(event.request, originResponse.clone())
  )
  return originResponse
})
```

Các đánh đổi:

Độ trễ thấp trên toàn cầu
Giảm tải cho máy chủ gốc
Chống DDoS
Chi phí (băng thông đắt)
Độ phức tạp (nhiều tầng bộ nhớ đệm)

Tầng 3: Bộ Nhớ Đệm Ứng Dụng (*Application Cache*) (Redis/Memcached)

Đây là trọng tâm chính. Đây là tầng mà bạn toàn quyền kiểm soát.



Tại sao lại là Redis?

- Lưu trong bộ nhớ (*In-memory*) → Cực kỳ nhanh (< 1ms)
- Cấu trúc dữ liệu phong phú (strings, hashes, lists, sets)
- Hỗ trợ TTL
- Mô hình xuất bản – đăng ký (*Pub/sub*) để vô hiệu hoá bộ nhớ đệm (*cache invalidation*)
- Các tùy chọn lưu bền (*persistence*)

Mẫu (*Pattern*) cơ bản:


```

import redis

cache = redis.Redis()

def get_user(user_id):
    # 1. Try cache first
    cached = cache.get(f"user:{user_id}")
    if cached:
        return json.loads(cached) # Cache hit!

    # 2. Cache miss → Query DB
    user = db.query("SELECT * FROM users WHERE id = ?", user_id)

    # 3. Store in cache for next time
    cache.setex(
        f"user:{user_id}",
        3600, # TTL: 1 hour
        json.dumps(user)
    )

    return user

```

Tầng 4: Bộ Nhớ Đệm Truy Vấn (Query Cache) Của Cơ Sở Dữ Liệu

Hầu hết cơ sở dữ liệu (*databases*) đều có sẵn bộ nhớ đệm truy vấn.

Bộ nhớ đệm truy vấn của MySQL:

```

-- First query: 300ms (cache miss)
SELECT * FROM products WHERE category = 'electronics';

-- Same query again: 5ms (cache hit)
SELECT * FROM products WHERE category = 'electronics';

```

Các đánh đổi:

- Tự động (không cần sửa mã)
- Dùng chung giữa các kết nối
- Bị vô hiệu hoá môi khi có BẤT KỲ thao tác ghi nào vào bảng
- Không mở rộng được (chỉ một máy chủ)

Cách làm hiện đại: Tắt bộ nhớ đệm truy vấn của CSDL, thay vào đó dùng bộ nhớ đệm ứng dụng (Redis). Kiểm soát tốt hơn.

Các Chiến Lược Lưu Đệm (Cache Strategies): Khi Nào Đọc? Khi Nào Ghi?

Chiến Lược 1: Cache-Aside (Tải Lười (Lazy Loading))

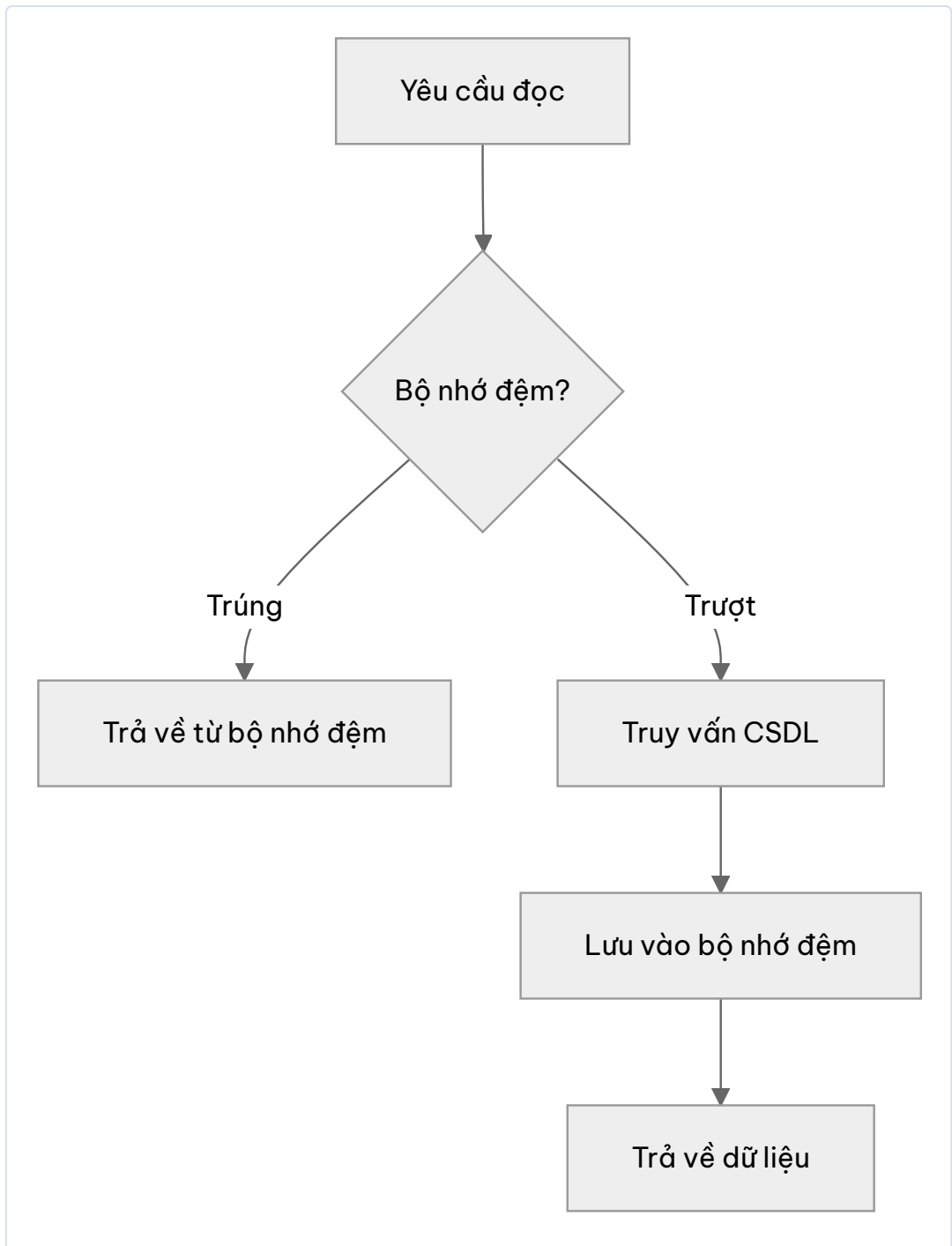
Mẫu phổ biến nhất.

```
def get_product(product_id):
    # Read path
    product = cache.get(f"product:{product_id}")
    if product:
        return product # Hit

    # Miss → Load from DB
    product = db.get(product_id)
    cache.set(f"product:{product_id}", product, ttl=3600)
    return product

def update_product(product_id, data):
    # Write path
    db.update(product_id, data)
    cache.delete(f"product:{product_id}") # Invalidate
```

Sơ đồ luồng:



Các đánh đổi:

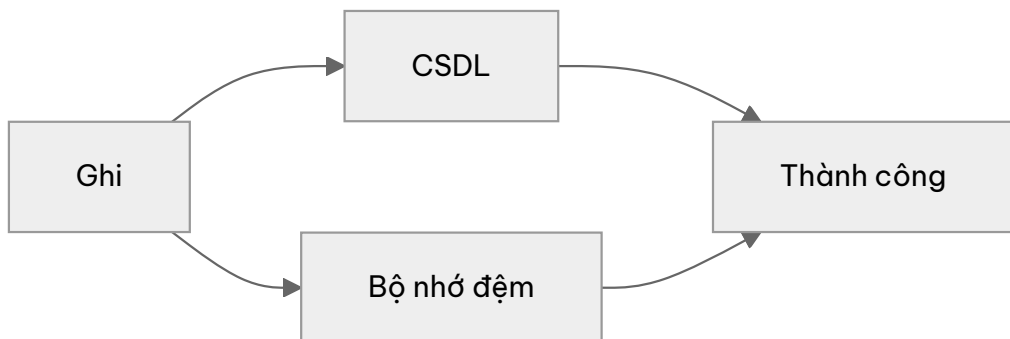
Chỉ lưu đệm những gì cần thiết
Trượt bộ nhớ đệm không gây hậu quả nghiêm trọng
Bền bỉ (bộ nhớ đệm sập → hệ thống vẫn chạy)
Yêu cầu đầu tiên luôn chậm (bộ nhớ đệm còn nguội)
Trượt bộ nhớ đệm làm tăng độ trễ

Phù hợp nhất cho: Khối lượng công việc thiên về đọc (*read-heavy*), ít cập nhật.

Chiến Lược 2: Write-Through (Ghi Xuyên)

```
def update_product(product_id, data):  
    # Write to both DB and cache  
    db.update(product_id, data)  
    cache.set(f"product:{product_id}", data, ttl=3600)  
  
def get_product(product_id):  
    # Read from cache (always up-to-date)  
    return cache.get(f"product:{product_id}")
```

Sơ đồ luồng:



Các đánh đổi:

Bộ nhớ đệm luôn mới
Độ trễ đọc dễ dự đoán
Độ trễ ghi cao hơn (2 thao tác)
Lãng phí thao tác ghi (dữ liệu có thể không được đọc)

Phù hợp nhất cho: Hệ thống thiên về đọc với yêu cầu tính nhất quán (*consistency*) nghiêm ngặt.

Chiến lược 3: Write-Behind (Write-Back, Ghi Sau)

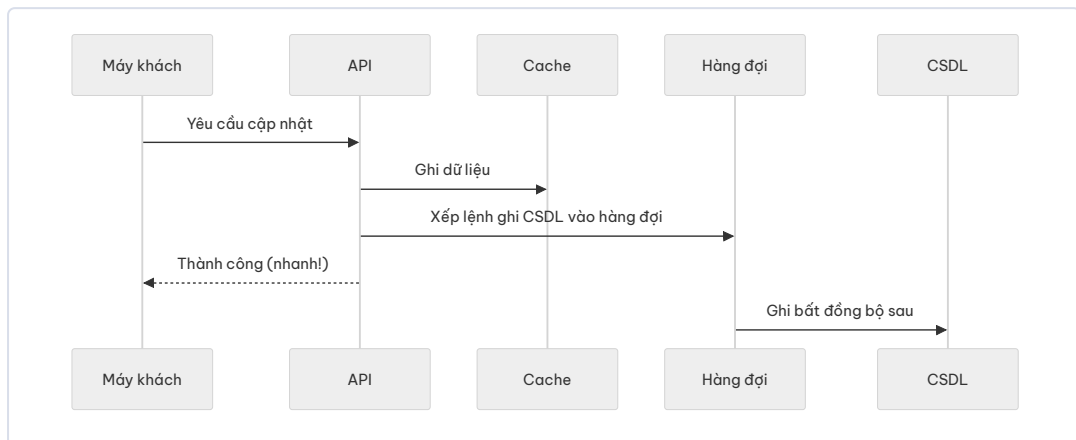
```
def update_product(product_id, data):
    # Write to cache immediately
    cache.set(f"product:{product_id}", data)

    # Queue DB write asynchronously
    queue.add({
        "type": "update_product",
        "id": product_id,
        "data": data
    })

    return "Success" # Return immediately!

# Background worker
def worker():
    while True:
        job = queue.pop()
        db.update(job["id"], job["data"])
```

Sơ đồ luồng:



Các đánh đổi:

- Ghi cực kỳ nhanh
- Hấp thụ được các đợt ghi tăng đột biến
- Rủi ro mất dữ liệu (bộ nhớ đệm sập trước khi ghi vào CSDL)
- Xử lý lỗi phức tạp
- Nhất quán cuối cùng

Phù hợp nhất cho: Khối lượng công việc thiên về ghi (*write-heavy*), phân tích dữ liệu, ghi log (*logging*).

Chiến Lược 4: Refresh-Ahead (Làm Mới Trước)

```
def get_product(product_id):
    product = cache.get(f"product:{product_id}")
    ttl_remaining = cache.ttl(f"product:{product_id}")

    # Nếu sắp expire, refresh async
    if ttl_remaining < 300: # < 5 phút
        background_refresh(product_id)

    return product

def background_refresh(product_id):
    fresh_data = db.get(product_id)
    cache.set(f"product:{product_id}", fresh_data, ttl=3600)
```

Các đánh đổi:

Bộ nhớ đệm hiếm khi trượt
Độ trễ ổn định
Độ phức tạp
Lãng phí lần làm mới (nếu dữ liệu không được truy cập)

Phù hợp nhất cho: Mẫu truy cập (*Access patterns*) để dự đoán, yêu cầu tính sẵn sàng cao (*high availability*).

Vô Hiệu Hoá Bộ Nhớ Đệm: Bài Toán Khó

Phil Karlton: "Trong Khoa học Máy tính chỉ có hai việc khó: vô hiệu hoá bộ nhớ đệm và đặt tên cho mọi thứ."

Tại Sao Nó Khó

Dòng thời gian:

- 10:00:00 - Người dùng cập nhật hồ sơ (CSDL đã cập nhật)
- 10:00:01 - Người dùng làm mới trang
 - Bộ nhớ đệm vẫn giữ dữ liệu cũ
 - Người dùng thấy hồ sơ cũ
- 10:00:02 - Người dùng bối rối, gửi báo cáo lỗi

Giải Pháp 1: Dựa Trên Thời Gian (TTL)

```
cache.setex("user:123", 300, user_data) # Expire after 5 min
```

Simple
Automatic cleanup
Stale data **for** up to TTL duration
Can't invalidate immediately

Quy tắc kinh nghiệm cho TTL:

Dữ liệu biến động rất mạnh (giá cổ'phiếu): 1-10 giây
Dữ liệu động (bảng tin): 1-5 phút
Bán tính (hồ'sơ người dùng): 10-30 phút
Tĩnh (danh mục sản phẩm): 1-24 giờ
Bất biến (hình ảnh, bài đăng cũ): Nhiều ngày/tuần

Giải Pháp 2: Vô Hiệu Hoá (*Invalidation*) Tường Minh

```
def update_user(user_id, data):  
    db.update(user_id, data)  
    cache.delete(f"user:{user_id}") # Delete immediately
```

Fresh data immediately
Full control
Must remember to invalidate everywhere
Easy to forget → stale data

Vấn đề vô hiệu hoá N+1:

```
# BAD: Easy to miss invalidations  
def update_user_email(user_id, email):  
    db.update_email(user_id, email)  
    cache.delete(f"user:{user_id}")  
    # Forgot to invalidate: user:email:{email}  
    # Forgot to invalidate: user_list  
    # Forgot to invalidate: search results
```

Mẫu: Vô hiệu hoá tập trung

```

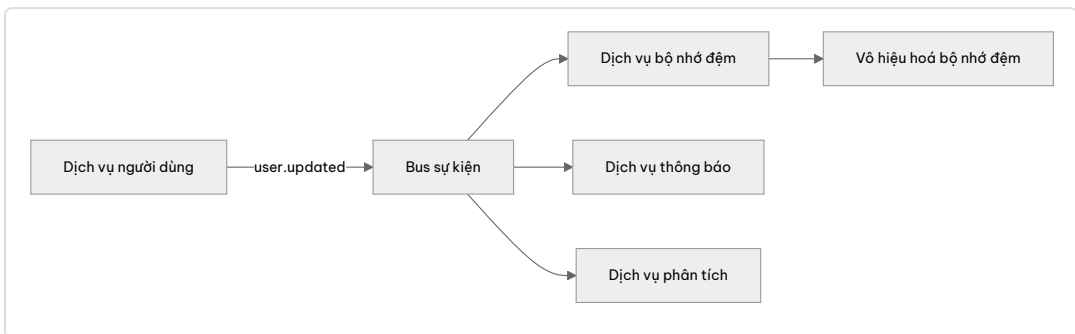
class UserCache:
    @staticmethod
    def invalidate(user_id):
        keys = [
            f"user:{user_id}",
            f"user:email:{user.email}",
            "user_list",
            "user_count"
        ]
        cache.delete_many(keys)

    def update_user(user_id, data):
        db.update(user_id, data)
        UserCache.invalidate(user_id) # One call, all caches cleared

```

Giải Pháp 3: Vô Hiệu Hoá Hướng Sự Kiện (Event-Driven)

Phù hợp nhất cho vi dịch vụ (microservices).



Triển khai:


```

# User Service
def update_user(user_id, data):
    db.update(user_id, data)
    event_bus.publish("user.updated", {
        "user_id": user_id,
        "fields": ["email", "name"]
    })

# Cache Service (separate service)
@event_bus.subscribe("user.updated")
def on_user_updated(event):
    cache.delete(f"user:{event['user_id']}")

# Notification Service
@event_bus.subscribe("user.updated")
def on_user_updated(event):
    send_notification(event['user_id'])

```

Các đánh đổi:

Các dịch vụ được tách rời
 Dễ thêm bên đăng ký mới
 Đáng tin cậy (log sự kiện được lưu bền)
 Độ phức tạp hạ tầng (Kafka, RabbitMQ)
 Nhất quán cuối cùng (trễ ~10-100ms)

Giải Pháp 4: Đánh Phiên Bản Bộ Nhớ Đệm (Cache Versioning)

Dành cho dữ liệu bất biến.

```

# Version in key
def get_user_avatar(user_id, version):
    key = f"avatar:{user_id}:v{version}"
    return cache.get(key)

# Update creates new version
def update_avatar(user_id, new_avatar):
    version = db.increment_avatar_version(user_id)
    cache.set(f"avatar:{user_id}:v{version}", new_avatar)

# Old versions auto-expire via TTL
# No explicit invalidation needed!

No invalidation needed
Old versions still work (useful for CDN)
More storage (multiple versions)

```

Vấn Đề Thundering Herd (Giẫm Đạp Bộ Nhớ Đệm (Cache Stampede))

Vấn Đề

Kịch bản:

1. Một khoá bộ nhớ đệm phổ biến hết hạn lúc 10:00:00
2. Lúc 10:00:01, 1000 yêu cầu đồng thời ập đến
3. Tất cả yêu cầu: trượt bộ nhớ đệm
4. Cả 1000 yêu cầu cùng lúc truy vấn CSDL
5. CSDL quá tải → sập

Kết quả: Bộ nhớ đệm làm hệ thống TỆ HƠN thay vì TỐT HƠN

Tác động thực tế:

Tôi từng thấy một trang Facebook có 10 triệu người theo dõi. Mỗi khi có bài đăng mới, hàng triệu người dùng (*users*) làm mới trang cùng lúc. Bộ nhớ đệm hết hạn → Cơ sở dữ liệu chết → 5 phút ngừng hoạt động (*downtime*).

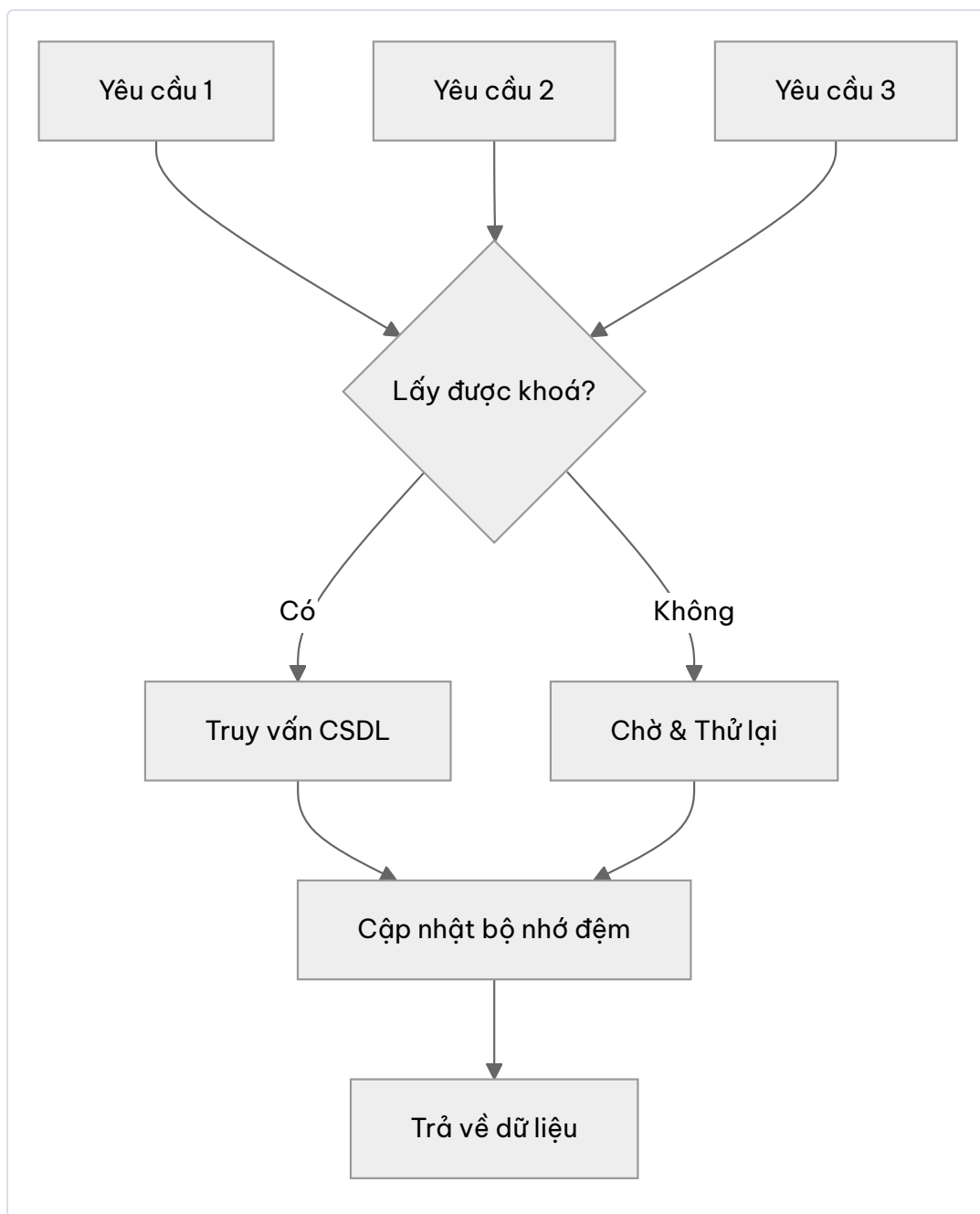
Giải Pháp 1: Dựa Trên Khoá (Lock)

```
def get_product(product_id):
    product = cache.get(f"product:{product_id}")
    if product:
        return product

    # Try to acquire lock
    lock_key = f"lock:product:{product_id}"
    locked = cache.set(lock_key, "1", nx=True, ex=10)

    if locked:
        # Only this request queries DB
        product = db.get(product_id)
        cache.set(f"product:{product_id}", product, ex=3600)
        cache.delete(lock_key)
        return product
    else:
        # Other requests wait
        time.sleep(0.1)
        return get_product(product_id) # Retry
```

Luồng:



Các đánh đổi:

Chỉ 1 truy vấn CSDL
Bảo vệ cơ sở dữ liệu
Các yêu cầu khác phải chờ (độ trễ~tăng đột biến)
Khóa có thể bị kẹt (nếu bên giữ khóa bị sập)

Giải Pháp 2: Hết Hạn Sớm Theo Xác Suất

```
import random

def get_product(product_id):
    product = cache.get(f"product:{product_id}")
    ttl = cache.ttl(f"product:{product_id}")

    # Probabilistically refresh before expiration
    if ttl < 300 and random.random() < 0.1:
        # 10% chance to refresh early
        background_refresh(product_id)

    return product
```

Spreads DB load over time
No locks needed
Still possible (but rare) stampede

Giải Pháp 3: Stale-While-Revalidate (Dùng Bản Cũ Trong Khi Làm Mới)

```
def get_product(product_id):
    product = cache.get(f"product:{product_id}")
    ttl = cache.ttl(f"product:{product_id}")

    if ttl < 0: # Expired
        # Return stale data immediately
        stale = cache.get(f"product:{product_id}:stale")

        # Refresh async
        background_refresh(product_id)

        return stale

    return product
```

No waiting
Stale data better than no data
Users might see slightly old data

Đây là cách tiếp cận tôi thích nhất cho hầu hết tình huống sử dụng (*use cases*).
Trải nghiệm người dùng (*User experience*) > độ mới hoàn hảo.

Các Đánh Đổi: Khi Nào Không Nên Lưu Đệm

Phản Mẫu (Anti-Pattern) 1: Lưu Đệm Dữ Liệu Thiên Về Ghi

```
# BAD: Cache counter that updates constantly
def increment_view_count(post_id):
    count = cache.get(f"views:{post_id}") or 0
    cache.set(f"views:{post_id}", count + 1)
```

Problem:

- 1000 increments/second
- Cache hiệu quả = 0% (mỗi read là miss)
- Complexity tăng, benefit = 0

BETTER: Write directly to DB, aggregate periodically

Phản Mẫu 2: Lưu Đệm Dữ Liệu Cá Nhân Hoá (Personalized)

```
# BAD: Cache recommendations per user
def get_recommendations(user_id):
    recs = cache.get(f"recs:{user_id}")
    if not recs:
        recs = expensive_ml_model(user_id)
        cache.set(f"recs:{user_id}", recs)
    return recs
```

Problem:

- 1M users = 1M cache entries
- Each user's recs accessed 1-2x/day
- Hit rate low
- Memory wasted

BETTER: Cache shared data (trending items), compute personal on-the-fly

Phản Mẫu 3: Lưu Đệm Quá Đầy

```
# BAD: Cache everything
cache.set("user_count", db.count_users())
cache.set("server_time", datetime.now())
cache.set("config", load_config())
```

Problem:

- Adds complexity for minimal gain
- Config thay đổi ít, DB query fast (<10ms)
- Not worth the invalidation headache

Rule: Only cache if query > 100ms hoặc called frequently (>100x/sec)

Thực Hành Tốt Nhất (Best Practices) Từ 10 Năm Kinh Nghiệm

1. Bắt Đầu Từ Số Liệu Đo (Metrics)

```
# Don't add cache blindly
"Let's cache everything!"

# Measure first
1. Log slow queries (> 100ms)
2. Identify hot paths (>100 req/s)
3. Cache those, not everything
```

2. Thiết Kế Khoá Bộ Nhớ Đệm (Cache Keys)

```
# BAD: Hard to invalidate
cache.set("user", user)

# GOOD: Namespace and version
cache.set("user:123:v2", user)
```

Benefits:

- Easy to invalidate: delete("user:123:*")
- Easy to upgrade: v2 → v3, old cache auto-expires

3. TTL Phù Hợp

```
# BAD: Same TTL cho everything
cache.setex(key, 3600, data) # Always 1 hour

# GOOD: TTL based on data characteristics
product_catalog: 24 hours (thay đổi ít)
user_session: 30 minutes (security)
trending_posts: 5 minutes (freshness)
stock_price: 5 seconds (real-time)
```

4. Suy Giảm Có Kiểm Soát (Graceful Degradation)

```
# BAD: Cache down = system down
def get_product(product_id):
    return cache.get(f"product:{product_id}") # Crash if cache down!

# GOOD: Cache down = slower, not broken
def get_product(product_id):
    try:
        cached = cache.get(f"product:{product_id}")
        if cached:
            return cached
    except CacheError:
        logger.warning("Cache unavailable, falling back to DB")

    return db.get(product_id) # Still works!
```

5. Giám Sát (Monitor) Các Chỉ Số Bộ Nhớ Đệm

```
metrics_to_track = {
    "hit_rate": cache_hits / total_requests, # Should be > 80%
    "miss_rate": cache_misses / total_requests,
    "eviction_rate": evictions / total_writes,
    "memory_usage": used / total,
    "latency_p99": p99_latency # Should be < 5ms
}

if hit_rate < 0.5:
    # Cache không effective, review strategy
```

Những Điểm Cốt Lõi

Bộ nhớ đệm là cách tối ưu hiệu năng đầu tiên bạn nên nghĩ đến:

- Nhanh hơn cơ sở dữ liệu 10-100 lần
- Dễ triển khai (Redis = 2 giờ cài đặt)
- ROI cao

Các chiến lược lưu đệm:

- **Cache-Aside:** Phổ biến nhất, thiên về đọc
- **Write-Through:** Khi tính nhất quán là tối quan trọng
- **Write-Behind:** Thiên về ghi, chấp nhận được việc mất dữ liệu
- **Refresh-Ahead:** Mẫu truy cập (*Patterns*) để dự đoán

Những bài toán khó của vô hiệu hoá bộ nhớ đệm:

- TTL: Đơn giản, dữ liệu cuối cùng cũng mới
- Tường minh: Tức thì, dễ sai sót
- Hướng sự kiện: Có khả năng mở rộng (*Scalable*), phức tạp
- Đánh phiên bản (*Versioning*): Thanh lịch, tốn thêm lưu trữ (*storage*)

Các giải pháp cho thundering herd:

- Dựa trên khoá: An toàn, độ trễ (*latency*) tăng đột biến
- Theo xác suất: Dàn trải tải (*load*)
- Stale-while-revalidate: Trải nghiệm người dùng (*UX*) tốt nhất (tôi thích nhất)

Khi nào KHÔNG nên lưu đệm:

- Dữ liệu thiên về ghi
- Nội dung cá nhân hoá (tỷ lệ trùng bộ nhớ đệm (*hit rate*) thấp)
- Dữ liệu vốn đã nhanh (< 10ms)
- Yêu cầu thời gian thực (*real-time*)

Hãy nhớ: Bộ nhớ đệm làm tăng độ phức tạp (*complexity*). Chỉ thêm khi lợi ích > chi phí (*cost*). Đo lường trước, lưu đệm sau.

Mở Rộng Cơ Sở Dữ Liệu (*Database Scaling*): Chiến Lược Mở Rộng Tầng Dữ Liệu (*Data Layer*)

Tôi còn nhớ cái ngày cơ sở dữ liệu (*database*) đầu tiên tôi thiết kế sụp đổ.

Black Friday. Lưu lượng (*Traffic*) tăng gấp 10 lần. CPU của cơ sở dữ liệu lên 100%, các truy vấn (*queries*) liên tục hết thời gian chờ (*timeout*), thời gian phản hồi (*response time*) từ 200ms vọt lên 30 giây.

3 giờ sáng, tôi gọi cho một kiến trúc sư lâu năm (*senior architect*). Anh ấy hỏi đúng một câu: "Em có mấy cơ sở dữ liệu?"

Tôi: "Một cái ạ."

Anh ấy: "Vậy là em chưa biết cách mở rộng cơ sở dữ liệu."

Đó là đêm tôi học được bài học đắt giá nhất về mở rộng cơ sở dữ liệu. Và bài học này sẽ giúp bạn tránh được sai lầm đó.

Tại Sao Cơ Sở Dữ Liệu Luôn Là Điểm Nghẽn (*Bottleneck*) Đầu Tiên?

Trước khi học cách mở rộng (*scale*), hãy hiểu tại sao cơ sở dữ liệu luôn là điểm nghẽn.

Lý do 1: I/O ổ đĩa chậm nhất trong hệ thống

Truy cập RAM:	100 nano giây	
Đọc SSD:	150 micro giây	(chậm hơn RAM 1,500 lần)
Đọc HDD:	10 mili giây	(chậm hơn RAM 100,000 lần)

Cơ sở dữ liệu phải:

- Đọc từ ô'đĩa
- Phân tích cú pháp truy vấn
- Thực thi kế'hoạch truy vấn
- Ghi kết quả

→ Bản chất đã chậm

Lý do 2: Hệ thống có trạng thái (stateful) khó mở rộng hơn hệ thống phi trạng thái (stateless)

Máy chủ ứng dụng (phi trạng thái):

- Không lưu dữ liệu
- Thêm máy chủ → Mở rộng được ngay
- Dễ!

Cơ sở dữ liệu (có trạng thái):

- Lưu dữ liệu
- Dữ liệu phải nhất quán
- Khó!

Lý do 3: Điểm lỗi duy nhất (Single point of failure)

10 máy chủ ứng dụng, 1 cơ sở dữ liệu

Cơ sở dữ liệu chết → Toàn bộ hệ thống chết

Điểm mấu chốt: Việc mở rộng cơ sở dữ liệu phải được lên kế hoạch từ đầu, chứ không phải đợi đến khi môi trường thực tế (production) đã cháy.

Mở Rộng Theo Chiều Dọc (Vertical Scaling): Đơn Giản Nhưng Có Giới Hạn

Định Nghĩa

Mở rộng theo chiều dọc = Tăng sức mạnh của máy chủ (server) hiện tại

Hiện tại: 8 CPU, 16GB RAM, 500GB SSD

Nâng cấp: 32 CPU, 128GB RAM, 2TB SSD

Máy chủ CSDL
8 CPU, 16GB RAM

Nâng cấp phần cứng

Máy chủ CSDL
32 CPU, 128GB RAM

Mở rộng theo chiều dọc: Nâng cấp phần cứng của cùng một máy chủ

Ưu Điểm

1. Cực kỳ đơn giản

Không phải sửa code
Không phải thay đổi kiến trúc
Bấm nút → Có thêm sức mạnh
10 phút là xong

2. Không có độ phức tạp (complexity) của hệ thống phân tán (distributed system)

Vẫn là một cơ sở dữ liệu duy nhất
Giao dịch ACID hoạt động bình thường
Không có vấn đề nhất quán dữ liệu
Truy vấn vẫn như cũ

3. Có kết quả ngay lập tức

Trước: CPU 90%, truy vấn 5 giây
Sau: CPU 30%, truy vấn 200ms

Nhược Điểm

1. Có giới hạn vật lý

Không thể mua CPU/RAM vô hạn
Có mức trần tối đa
Sớm muộn cũng phải chuyển sang mở rộng theo chiều ngang

2. Đắt đỏ và lợi ích giảm dần (diminishing returns)

Chi phí khi mở rộng (số liệu thực tế từ AWS):

16GB RAM → 32GB: giá x2 (\$200 → \$400/tháng)
32GB RAM → 64GB: giá x3 (\$400 → \$1,200/tháng)
64GB RAM → 128GB: giá x5 (\$1,200 → \$6,000/tháng)
128GB RAM → 256GB: giá x8 (\$6,000 → \$48,000/tháng)

Giá tăng theo cấp số nhân, hiệu năng chỉ tăng tuyến tính

3. Điểm lỗi duy nhất

Máy chủ chết → Toàn bộ hệ thống chết
Bảo trì → Ngừng hoạt động
Không có dự phòng

4. Bắt buộc phải có thời gian ngừng hoạt động (downtime)

Phải khởi động lại cơ sở dữ liệu
Ngừng hoạt động 5-30 phút
Không thể nâng cấp mà không ngừng hoạt động

Ma Trận Đánh Đổi (Trade-off)

Mở rộng theo chiều dọc:

Đơn giản:	*****
Triển khai nhanh:	*****
Tính nhất quán:	*****
Hiệu quả chi phí:	★★☆☆☆
Tính sẵn sàng:	★★☆☆☆
Giới hạn mở rộng:	★★☆☆☆

Khi Nào Nên Dừng?

Sử dụng mở rộng theo chiều dọc khi:

- ✓ Công ty khởi nghiệp/MVP (< 10 nghìn người dùng)
- ✓ Ngân sách eo hẹp
- ✓ Nhóm nhỏ (không có chuyên môn về hệ thống phân tán)
- ✓ Lưu lượng dự đoán được và có mức trần
- ✓ Dữ liệu < 1TB
- ✓ Giao dịch ACID là tối quan trọng (ngân hàng, thanh toán)

Ví dụ thực tế:

Tôi từng tư vấn cho một công ty khởi nghiệp (*startup*) ngân hàng. Nhóm (*Team*) muốn làm "vi dịch vụ (*microservices*) + cơ sở dữ liệu phân mảnh (*sharded database*) ngay từ đầu".

Tôi nói không:

- 5.000 người dùng (*users*) hiện tại
- PostgreSQL 16GB RAM chịu được 50 nghìn người dùng
- Đơn giản = ra mắt sản phẩm nhanh hơn
- Tiết kiệm ngân sách cho marketing

2 năm sau, 30 nghìn người dùng, cơ sở dữ liệu vẫn ổn. Chỉ cần nâng cấp lên 32GB RAM (ngừng hoạt động 1 giờ lúc 3 giờ sáng).

Quyết định đúng cho bối cảnh đó.

Lời Khuyên Cá Nhân

Luôn bắt đầu với mở rộng theo chiều dọc.

Chỉ chuyển sang mở rộng theo chiều ngang (*horizontal scaling*) khi:

- Chạm giới hạn CPU/bộ nhớ (thường xuyên ở mức 80% trở lên)
- Chi phí quá cao
- Cần tính sẵn sàng cao (*high availability*)

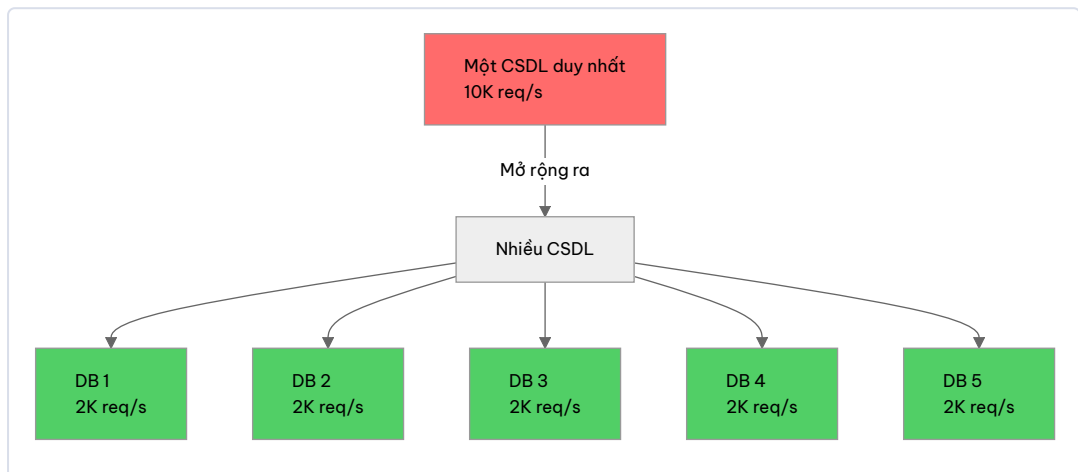
Đừng tối ưu hoá quá sớm (*prematurely optimize*).

Mở Rộng Theo Chiều Ngang: Phức Tạp Nhưng Không Giới Hạn

Định Nghĩa

Mở rộng theo chiều ngang = Thêm nhiều máy chủ (*servers*)

1 cơ sở dữ liệu xử lý 10,000 req/s
→ 5 cơ sở dữ liệu, mỗi cái xử lý 2,000 req/s



Mở rộng theo chiều ngang: Thêm nhiều máy chủ cơ sở dữ liệu (*database servers*) để phân tán tải (*load*)

Ưu Điểm

Về lý thuyết, mở rộng không giới hạn

- 10 máy chủ → năng lực gấp 10 lần
- 100 máy chủ → năng lực gấp 100 lần

Hiệu quả về chi phí khi ở quy mô lớn

- Dùng phần cứng phổ thông
- Chi phí tăng tuyến tính theo năng lực

Tính sẵn sàng cao

- 1 máy chủ chết → Các máy còn lại vẫn tiếp tục chạy
- Không có điểm lỗi duy nhất

Mở rộng không cần ngừng hoạt động

- Thêm máy chủ mà không cần khởi động lại các máy hiện có

Nhược Điểm

Độ phức tạp cực cao

- Thách thức về nhất quán dữ liệu
- Giao dịch phân tán là cơn ác mộng
- Phân vùng mạng

Phải sửa đổi ứng dụng

- Code phải biết về các mảnh (shard-aware)
- Không thể coi như một CSDL duy nhất

Chi phí vận hành phụ trội

- Giám sát nhiều máy chủ
- Sao lưu/khôi phục phức tạp
- Phân mảnh lại = cơn ác mộng

Bản Sao Chỉ Đọc (*Read Replicas*): Mở Rộng Việc Đọc Dễ Dàng

Mẫu Đọc/Ghi (*The Read/Write Pattern*)

Đối chiếu thực tế: Hầu hết các ứng dụng đều thiên về đọc (*read-heavy*).

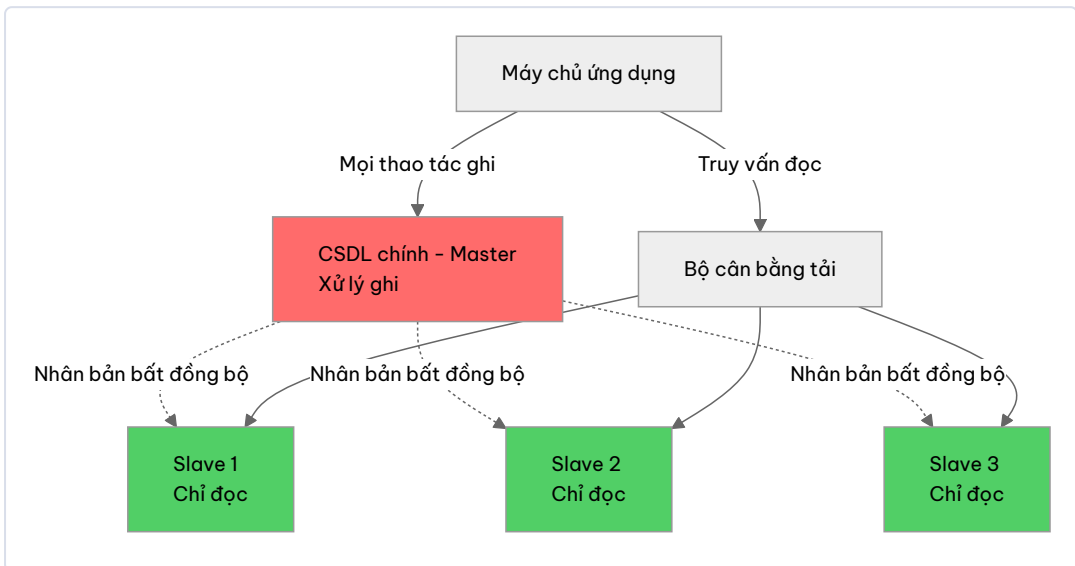
Ứng dụng web thông thường:
90% đọc (SELECT)
10% ghi (INSERT, UPDATE, DELETE)

Mạng xã hội:
95% đọc (xem bài đăng, hồ` sơ)
5% ghi (tạo bài đăng, thích)

Thương mại điện tử:
85% đọc (duyệt sản phẩm)
15% ghi (thanh toán, đánh giá)

Điểm mấu chốt: Nếu tách riêng việc đọc và ghi, ta có thể mở rộng việc đọc rất dễ dàng.

Kiến Trúc Chủ-Tớ (Master-Slave)



Nút chủ (Master) xử lý việc ghi, các nút tớ (Slaves) xử lý việc đọc. Nhân bản (Replication) diễn ra bất đồng bộ (async).

Quy trình hoạt động:

1. Ứng dụng ghi → CSDL chính (Master)
2. Master nhân bản thay đổi bất đồng bộ → các Slave
3. Ứng dụng đọc → được cân bằng tải qua các Slave
4. Slave chỉ phục vụ đọc, không nhận ghi

Ví Dụ Triển Khai

```
# Database configuration
class DatabaseRouter:
    def __init__(self):
        self.master = connect('master-db.example.com')
        self.slaves = [
            connect('slave1-db.example.com'),
            connect('slave2-db.example.com'),
            connect('slave3-db.example.com')
        ]

    def write(self, query, params):
        # Tất cả writes đi master
        return self.master.execute(query, params)

    def read(self, query, params):
        # Reads load balanced qua slaves
        slave = random.choice(self.slaves)
        return slave.execute(query, params)

# Application code
db = DatabaseRouter()

def create_user(data):
    # Write operation → Master
    db.write("INSERT INTO users (...) VALUES (...)", data)

def get_user(user_id):
    # Read operation → Slave
    return db.read("SELECT * FROM users WHERE id = ?", [user_id])

def update_user(user_id, data):
    # Write → Master
    db.write("UPDATE users SET ... WHERE id = ?", data)

    # IMPORTANT: Read ngay sau write phải từ master
    # Tránh đọc stale data từ slave (replication lag)
    return self.master.execute("SELECT * FROM users WHERE id = ?", [user_id])
```

Độ Trễ Nhân Bản (*Replication Lag*): Vấn Đề Quan Trọng Nhất

Vấn đề:

Dòng thời gian:

10:00:00.000 - Người dùng cập nhật hồ sơ trên Master
10:00:00.050 - Master nhân bản sang các Slave (trễ ~50ms)
10:00:00.010 - Người dùng tải lại trang, truy vấn đi tới Slave
→ Slave chưa có dữ liệu mới!
→ Người dùng thấy dữ liệu cũ

Người dùng bối rối: "Tôi vừa cập nhật sao không thấy thay đổi?"

Độ trễ nhân bản = Độ trễ giữa lúc nút chủ ghi và lúc nút tớ (Slave) có dữ liệu

Tốt: < 100ms
Chấp nhận được: < 1 giây
Có vấn đề: 1-5 giây
Thảm họa: > 5 giây

Giải Pháp Cho Độ Trễ Nhân Bản

Giải pháp 1: Đọc-Sau-Khi-Ghi (Read-After-Write) từ nút chủ

```
def update_profile(user_id, data):  
    # Write to master  
    db.write("UPDATE users SET ... WHERE id = ?", data)  
  
    # Read from master to guarantee consistency  
    return db.master.execute("SELECT * FROM users WHERE id = ?", [user_id])
```

User thấy changes ngay lập tức
Load trên master tăng

Giải pháp 2: Gắn phiên (Session Stickiness)

```
# Sau write, flag trong session
session['use_master_until'] = time.now() + 5_seconds

def get_user(user_id):
    if session.get('use_master_until', 0) > time.now():
        # Read from master trong 5 giây sau write
        return db.master.execute(...)
    else:
        # Fallback to slave
        return db.read(...)
```

Automatic fallback
Giảm load trên master
Phức tạp hơn

Giải pháp 3: Chấp nhận và thông báo cho người dùng (*user*)

```
def update_profile(user_id, data):
    db.write("UPDATE users ...", data)

    return {
        "status": "success",
        "message": "Cập nhật thành công. Thay đổi có thể mất vài giây để hiển thị."
    }
```

Honest với user
Simplest code
Acceptable cho most use cases
Slightly worse UX

Khi Nào Dùng Bản Sao Chỉ Đọc?

Sử dụng khi:

- ✓ Khối lượng công việc thiên về đọc (> 70% đọc)
- ✓ Chấp nhận được độ trễ nhân bản (< 1 giây là ổn)
- ✓ Master có thể xử lý toàn bộ thao tác ghi
- ✓ Cân tính sẵn sàng cao cho việc đọc

Không dùng khi:

- ✗ Khối lượng công việc thiên về ghi (> 40% ghi)
- ✗ Cân nhất quán mạnh ở mọi nơi
- ✗ Master đã quá tải vì thao tác ghi

Kinh nghiệm cá nhân:

Bản sao chỉ đọc giải quyết được 80% các vấn đề mở rộng cơ sở dữ liệu. Đơn giản, hiệu quả, ít rủi ro.

Khuyến nghị: Hãy thử bản sao chỉ đọc trước khi nghĩ đến phân mảnh dữ liệu (*sharding*).

Phân Mảnh Dữ Liệu: Quy Mô Tối Thượng, Độ Phức Tạp Tối Thượng

Vấn Đề Bản Sao Chỉ Đọc Không Giải Quyết Được

Kịch bản:

- Master xử lý 50,000 thao tác ghi/giấy
- Thêm 10 bản sao chỉ đọc → Master vẫn phải xử lý 50 nghìn thao tác ghi
- Bản sao chỉ đọc chỉ mở rộng việc đọc, KHÔNG mở rộng việc ghi

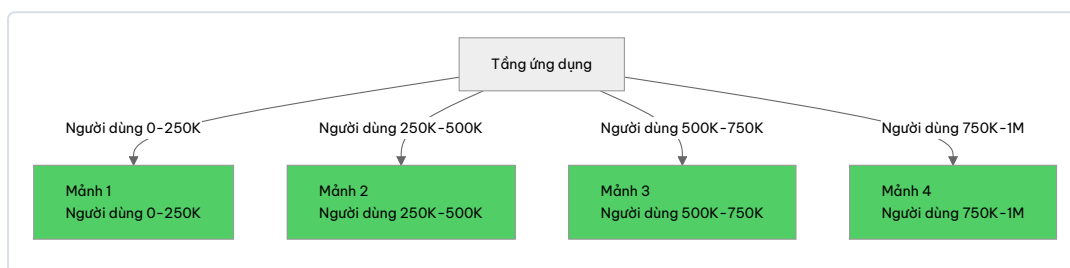
Tương tự:

- Cơ sở dữ liệu quá lớn: 10TB dữ liệu
- Một máy chủ không đủ dung lượng lưu trữ
- Truy vấn chậm vì bảng có hàng tỷ dòng

Bản sao chỉ đọc mở rộng việc đọc. Phân mảnh dữ liệu mở rộng CẢ việc ghi lẫn kích thước dữ liệu.

Khái Niệm Phân Mảnh Dữ Liệu

Phân mảnh dữ liệu = Chia dữ liệu ra nhiều cơ sở dữ liệu (*databases*), mỗi cơ sở dữ liệu lưu một tập con



Mỗi mảnh (*shard*) chứa một tập con của toàn bộ dữ liệu và hoạt động độc lập

Đặc điểm chính:

- Mỗi mảnh là một cơ sở dữ liệu độc lập
- Mỗi mảnh chứa một phần của toàn bộ dữ liệu
- Không có trạng thái dùng chung giữa các mảnh
- Ứng dụng quyết định truy vấn đi tới mảnh nào

Các Chiến Lược Phân Mảnh

Chiến lược 1: Phân mảnh theo hàm băm (*Hash-Based Sharding*)

```
def get_shard(user_id):  
    shard_id = hash(user_id) % num_shards  
    return shards[shard_id]
```

Ví dụ:

user_id = 12345

shard = hash(12345) % 4 # = 1

→ User 12345 nằm ở Shard 1

user_id = 67890

shard = hash(67890) % 4 # = 3

→ User 67890 nằm ở Shard 3

Phân phối dữ liệu đồng đều

Thuật toán đơn giản

Dự đoán được vị trí mảnh

Khó thay đổi num_shards (phân mảnh lại là cơn ác mộng)

Truy vấn theo khoảng xuyên các mảnh rất khó

Dữ liệu liên quan có thể nằm ở các mảnh khác nhau

Chiến lược 2: Phân mảnh theo khoảng (*Range-Based Sharding*)

```
def get_shard(user_id):
    if user_id < 250_000:
        return shard_0
    elif user_id < 500_000:
        return shard_1
    elif user_id < 750_000:
        return shard_2
    else:
        return shard_3
```

Ví dụ:

```
user_id = 100_000 → Shard 0
user_id = 300_000 → Shard 1
user_id = 900_000 → Shard 3
```

Đề~thêm mảnh mới (chỉ cần thêm khoảng mới)
 Truy vấn theo khoảng trong 1 mảnh rất hiệu quả
 Dữ liệu liên quan (ID tuần tự) nằm cùng mảnh

Phân phối không đều (mảnh mới có ít dữ liệu)
 Đỉnh nóng (người dùng mới dồn vào mảnh mới nhất)
 Tải không cân bằng

Chiến lược 3: Phân mảnh theo địa lý (Geography-Based Sharding)

```
def get_shard(user_country):
    shard_map = {
        'US': us_shard,
        'EU': eu_shard,
        'Asia': asia_shard
    }
    return shard_map.get(user_country, us_shard)
```

Ví dụ:

```
user_country = 'Vietnam' → asia_shard
user_country = 'Germany' → eu_shard
user_country = 'USA' → us_shard
```

Độ trê~thấp (dữ liệu ở gần người dùng)
 Tuân thủ quy định (yêu cầu lưu trữ dữ liệu trong nước)
 Dữ liệu được cô lập một cách tự nhiên

Truy vấn xuyên vùng tốn kém
 Phân phối không đều nếu người dùng phân bố không đều
 Phức tạp nếu người dùng di chuyển

Chiến lược 4: Phân mảnh theo thực thể (Entity-Based Sharding)

```
def get_shard(entity_type, entity_id):  
    if entity_type == 'user':  
        return user_shard(entity_id)  
    elif entity_type == 'order':  
        return order_shard(entity_id)  
    elif entity_type == 'product':  
        return product_shard(entity_id)
```

Ví dụ:

User data → User shards

Order data → Order shards

Product data → Product shards

Tối ưu riêng cho từng thực thể
Quyền sở hữu dữ liệu rõ ràng
Mở rộng độc lập theo từng loại thực thể

JOIN giữa các thực thể = thảm họa
Giao dịch giữa các thực thể rất khó
Nhiều mảnh hơn phải quản lý

Những Thách Thức Của Phân Mảnh Dữ Liệu

Thách thức 1: Truy vấn xuyên mảnh (Cross-Shard Queries)

```
-- Query users với followers > 1000  
SELECT * FROM users WHERE followers_count > 1000;
```

Problem:

- Data nằm ở 4 shards
- Phải query TẤT CẢ 4 shards
- Merge results từ 4 sources
- Sort/paginate across shards

Giải pháp:

- Denormalize data (lưu followers_count riêng)
- Use search engine (Elasticsearch) cho aggregations
- Accept eventual consistency

Thách thức 2: Giao dịch phân tán (Distributed Transactions)

```
# Transfer money user A → user B
def transfer(from_user_id, to_user_id, amount):
    from_shard = get_shard(from_user_id)
    to_shard = get_shard(to_user_id)

    if from_shard != to_shard:
        # Users ở different shards!
        # Không thể dùng database transactions
        # Phải implement 2-phase commit hoặc saga pattern
        pass
```

Giải pháp:

- Avoid cross-shard transactions (design data để minimize)
- Use distributed transaction protocols (2PC, Saga)
- Accept eventual consistency với compensation

Thách thức 3: Phân mảnh lại (Resharding)

Kịch bản:

- Bắt đầu với 4 mảnh
- Cân tăng lên 8 mảnh
- Phải chuyển 50% dữ liệu từ mỗi mảnh cũ

Quy trình:

1. Tạo 4 mảnh mới
2. Dừng ghi (ngừng hoạt động) hoặc ghi kép (phức tạp)
3. Sao chép dữ liệu từ mảnh cũ → mảnh mới
4. Kiểm chứng tính nhất quán dữ liệu
5. Chuyển lưu lượng sang các mảnh mới
6. Gỡ bỏ các mảnh cũ

Cơ ác mộing:

- 10TB dữ liệu → mất 1-2 tuần để di chuyển
- Rủi ro mất dữ liệu
- Ngừng hoạt động hoặc độ phức tạp cực cao

Thách thức 4: ID tự tăng (Auto-Increment IDs)


```
-- Không thể dùng auto-increment vì conflict giữa shards
CREATE TABLE users (
  id SERIAL PRIMARY KEY, -- Conflict!
  ...
);

-- Shard 1: id = 1, 2, 3, ...
-- Shard 2: id = 1, 2, 3, ... (DUPLICATE!)
```

Giải pháp:

- Use UUID (128-bit, guaranteed unique)
- Snowflake ID (64-bit, time-ordered)
- Shard-prefixed IDs (shard_id + local_id)

Khi Nào Dùng Phân Mảnh Dữ Liệu?

Chỉ dùng phân mảnh khi:

- ✓ Mở rộng theo chiều dọc đã kịch trần (512GB+ RAM)
- ✓ Bản sao chỉ đọc không đủ (quá nhiều thao tác ghi)
- ✓ Một cơ sở dữ liệu không đủ dung lượng lưu trữ (>10TB)
- ✓ Có nhóm giàu kinh nghiệm (chuyên môn về hệ thống phân tán)
- ✓ Có ngân sách cho độ phức tạp vận hành

Tránh phân mảnh khi:

- x < 1 triệu người dùng
- x < 5TB dữ liệu
- x Vẫn còn có thể mở rộng theo chiều dọc
- x Nhóm chưa có kinh nghiệm
- x Chưa thu' bản sao chỉ đọc + bộ nhớ đệm

Lời Khuyên Từ Kinh Nghiệm

Tôi đã làm việc với 3 công ty cố gắng phân mảnh cơ sở dữ liệu quá sớm. Cả 3 đều hối hận.

Công ty A: 50 nghìn người dùng, phân mảnh cơ sở dữ liệu. 6 tháng phát triển, nhiều lỗi (*bugs*), nhóm kiệt sức.

Công ty B: Đợi đến 2 triệu người dùng, mở rộng theo chiều dọc (*vertical scale*) + bản sao chỉ đọc. Chỉ phân mảnh khi thực sự cần (5 triệu người dùng).

Công ty C: 100 nghìn người dùng, kiến trúc sư (*architect*) khẳng định đòi phân mảnh. 1 năm sau, phải thu gọn về một cơ sở dữ liệu duy nhất vì độ phức tạp không đáng.

Quy tắc của tôi:

Thứ tự mở rộng cơ sở dữ liệu:

1. Mở rộng theo chiều dọc (đầu tiên)
2. Tối ưu truy vấn + đánh chỉ mục
3. Tăng bộ nhớ đệm
4. Bản sao chỉ đọc
5. Phân mảnh dữ liệu (cuối cùng, khi không còn cách nào khác)

Phân mảnh dữ liệu là **giải pháp cuối cùng**, không phải lựa chọn đầu tiên.

Những Đánh Đổi (Trade-offs) Về Tính Nhất Quán (Consistency)

Nhắc Lại Định Lý CAP (CAP Theorem)

Trong cơ sở dữ liệu phân tán (distributed database), chỉ chọn được 2 trong 3:

C (Consistency - Tính nhất quán): Mọi người đọc cùng một dữ liệu

A (Availability - Tính sẵn sàng): Hệ thống luôn phản hồi

P (Partition tolerance - Khả năng chịu phân vùng): Vẫn hoạt động khi mạng gặp sự cố

Phân vùng mạng LUÔN xảy ra → Phải chọn C hoặc A

Các Mô Hình Nhất Quán (Consistency Models)

Nhất quán mạnh (Strong Consistency) (CP)

Người dùng A ghi $X = 10$

Người dùng B đọc $X \rightarrow$ PHẢI thấy 10

Ứng dụng ngân hàng:

- Chuyển \$100
- Số dư PHẢI được phản ánh ngay
- Không chấp nhận dữ liệu cũ

Đánh đổi:

Dữ liệu chính xác

Chậm (phải chờ tất cả bản sao)

Tính sẵn sàng thấp hơn

Nhất quán cuối cùng (Eventual Consistency) (AP)

Người dùng A ghi $X = 10$
Người dùng B đọc $X \rightarrow$ Có thể thấy giá trị cũ
Sau vài giây $\rightarrow$ Mọi người đều thấy 10

Mạng xã hội:

- Thích một bài đăng
- Số lượt thích có thể trên vài giây
- Chấp nhận được

Đánh đôi:

Nhanh

Tính sẵn sàng cao

Dữ liệu có thể bị cũ

Chọn Mức Độ Nhất Quán

Cần nhất quán mạnh:

- ✓ Giao dịch tài chính
- ✓ Quản lý tồn kho
- ✓ Xác thực người dùng
- ✓ Logic nghiệp vụ trọng yếu

Chấp nhận nhất quán cuối cùng:

- ✓ Số liệu mạng xã hội (lượt thích, lượt xem)
- ✓ Hệ thống gợi ý
- ✓ Bảng điều khiển phân tích
- ✓ Bảng tin hoạt động

Khung tư duy (Framework):

Tự hỏi:

1. Người dùng có bị ảnh hưởng nếu dữ liệu sai trong vài giây không?
2. Doanh nghiệp có bị thiệt hại nếu dữ liệu không nhất quán không?
3. Có thể bù đắp/sửa lại sau không?

Nếu có bất kỳ câu "có" $\rightarrow$ Nhất quán mạnh

Tất cả đều "không" $\rightarrow$ Nhất quán cuối cùng là ổn

Những Điểm Cốt Lỗi

Thứ bậc mở rộng cơ sở dữ liệu:

Cấp 1: Mở rộng theo chiều dọc

- Đơn giản nhất, bắt đầu từ đây
- Phù hợp cho < 100 nghìn người dùng

Cấp 2: Tối ưu truy vấn + bộ nhớ đệm

- Giải quyết 70% vấn đề
- Chi phí thấp, tác động lớn

Cấp 3: Bản sao chỉ đọc

- Mở rộng việc đọc dễ dàng
- Phù hợp cho ứng dụng thiên về đọc

Cấp 4: Phân mảnh dữ liệu

- Giải pháp cuối cùng
- Chỉ khi thực sự cần
- Độ phức tạp cực cao

Các chiến lược phân mảnh:

Theo hàm băm:	Đều, khó thay đổi kích thước
Theo khoảng:	Dễ thêm mảnh, phân phối không đều
Theo địa lý:	Độ trễ thấp, tuân thủ quy định
Theo thực thể:	Sở hữu rõ ràng, khó JOIN

Những đánh đổi về tính nhất quán:

Nhất quán mạnh: Chính xác, chậm, tính sẵn sàng thấp hơn
Nhất quán cuối cùng: Nhanh, sẵn sàng, có thể bị cũ

Lựa chọn dựa trên tác động kinh doanh, không phải sở thích kỹ thuật

Những quy tắc vàng:

1. **Bắt đầu đơn giản:** Mở rộng theo chiều dọc trước
2. **Đo lường trước:** Chứng minh rằng bạn cần mở rộng
3. **Tối ưu trước khi mở rộng:** Tối ưu truy vấn (*query*) > Thêm máy chủ
4. **Bản sao chỉ đọc trước phân mảnh dữ liệu:** Giải quyết 80% vấn đề
5. **Chỉ phân mảnh khi cần thiết:** Giải pháp cuối cùng

Câu hỏi tự kiểm tra:

Trước khi mở rộng cơ sở dữ liệu, hãy tự hỏi:

1. Tôi đã tối ưu các truy vấn chưa?

2. Tôi đã dùng bộ nhớ đệm (*cache*) chưa?
3. Tôi đã thử mở rộng theo chiều dọc chưa?
4. Tôi đã thử bản sao chỉ đọc chưa?
5. Tôi có dữ liệu/kinh nghiệm chứng minh rằng cần phân mảnh dữ liệu không?

Nếu chưa trả lời "có" cho cả 4 câu đầu, đừng nghĩ đến phân mảnh dữ liệu.

Mở rộng cơ sở dữ liệu không phải là dùng công nghệ phức tạp nhất. Nó là chọn giải pháp **phù hợp** với quy mô và các ràng buộc (*constraints*) của bạn.

Đơn giản > Phức tạp. Luôn luôn.

Hàng Đợi Thông Điệp (Message Queues) & Xử Lý Bất Đồng Bộ (Async Processing): Tách Rời (Decoupling) Các Dịch Vụ (Services)

Tôi còn nhớ lần đầu tiên hệ thống môi trường thực tế (*production*) của tôi sập chỉ vì một email.

Người dùng tải lên một video. Máy chủ (*Server*) phải:

1. Lưu video (2 giây)
2. Tạo ảnh thu nhỏ (5 giây)
3. Chuyển mã (*Transcode*) 720p (30 giây)
4. Chuyển mã 1080p (60 giây)
5. Gửi email thông báo (1 giây)

Tổng cộng: 98 giây. Người dùng phải đợi 98 giây mới nhận được phản hồi (*response*).

Kết quả? Hết thời gian chờ (*Timeout*). Người dùng bức bối. Máy chủ quá tải vì mỗi yêu cầu (*request*) giữ kết nối suốt 98 giây.

Một kỹ sư lâu năm (*senior engineer*) nhìn tôi: "Em biết hàng đợi thông điệp (*message queue*) chưa?"

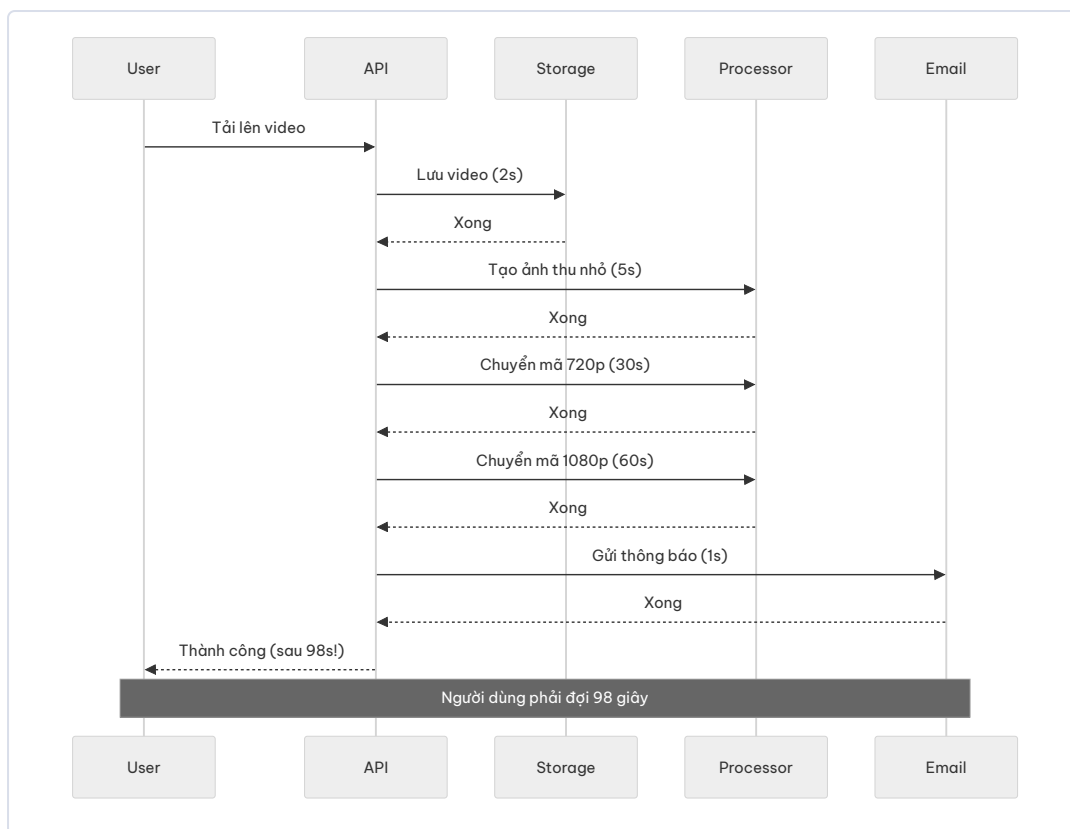
Tôi: "Dạ chưa ạ..."

Đó là cái đêm tôi học về xử lý bất đồng bộ. Và đó là một trong những mẫu thiết kế (*patterns*) quan trọng nhất trong hệ thống phân tán (*distributed systems*).

Tại Sao Cần Xử Lý Bất Đồng Bộ?

Vấn Đề Của Cách Xử Lý Đồng Bộ (*Synchronous*)

Đồng bộ = Người dùng đợi cho đến khi mọi thứ xong xuôi



Xử lý đồng bộ: Người dùng đợi cho đến khi mọi thứ hoàn thành

Các vấn đề:

Trải nghiệm người dùng tệ hại

- Người dùng đợi 98 giây
- Hết thời gian chờ sau 30-60 giây
- Người dùng nghĩ ứng dụng bị lỗi

Sử dụng tài nguyên kém hiệu quả

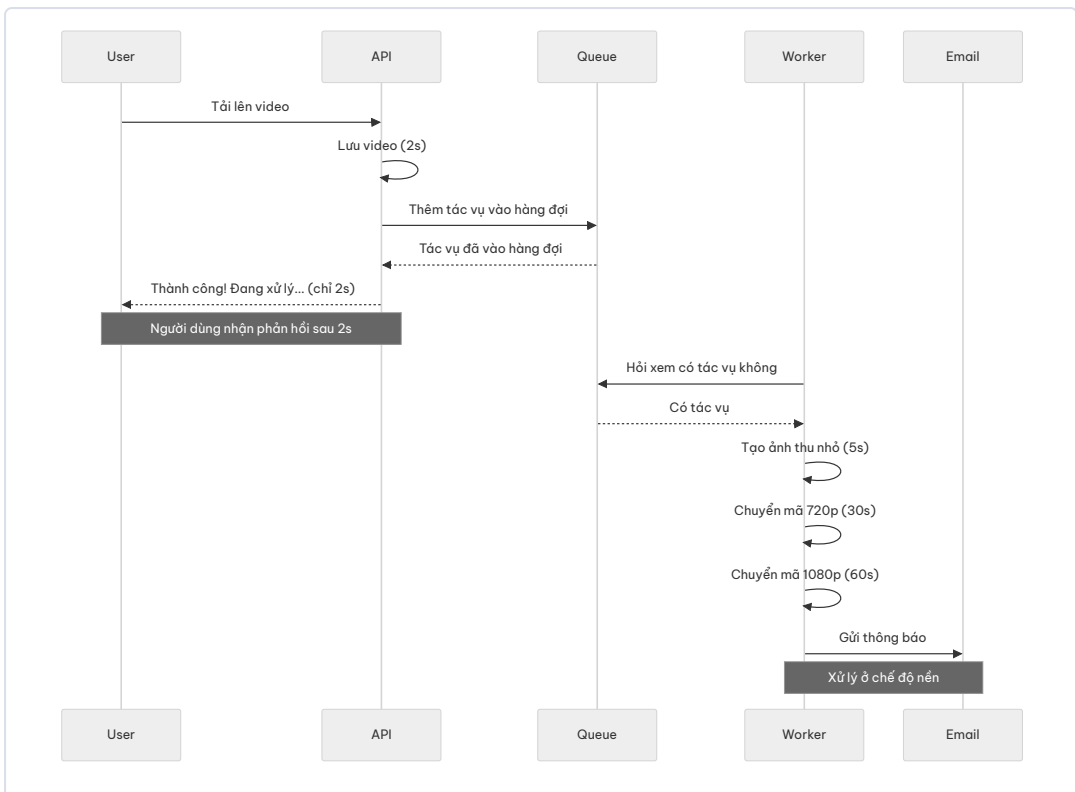
- Kết nối bị giữ suốt 98 giây
- Máy chủ chỉ xử lý được ít yêu cầu
- Cạn kiệt bộ nhớ/luồng

Sự cố dây chuyền

- Dịch vụ email sập → Toàn bộ thao tác tải lên thất bại
- Chuyển mã lỗi → Người dùng không nhận được video
- Liên kết chết

Giải Pháp Bất Đồng Bộ (Async)

Bất đồng bộ (Asynchronous) = Người dùng nhận phản hồi ngay, việc xử lý diễn ra ở chế độ nền



Xử lý bất đồng bộ: Người dùng nhận phản hồi ngay, công việc diễn ra ở chế độ nền

Lợi ích:

Trải nghiệm người dùng tuyệt vời

- Người dùng nhận phản hồi sau 2 giây
- Không bị hết thời gian chờ
- Trạng thái rõ ràng: "Đang xử lý..."

Sử dụng tài nguyên hiệu quả hơn

- Kết nối được giải phóng ngay lập tức
- Máy chủ xử lý được nhiều yêu cầu
- Các worker mở rộng độc lập

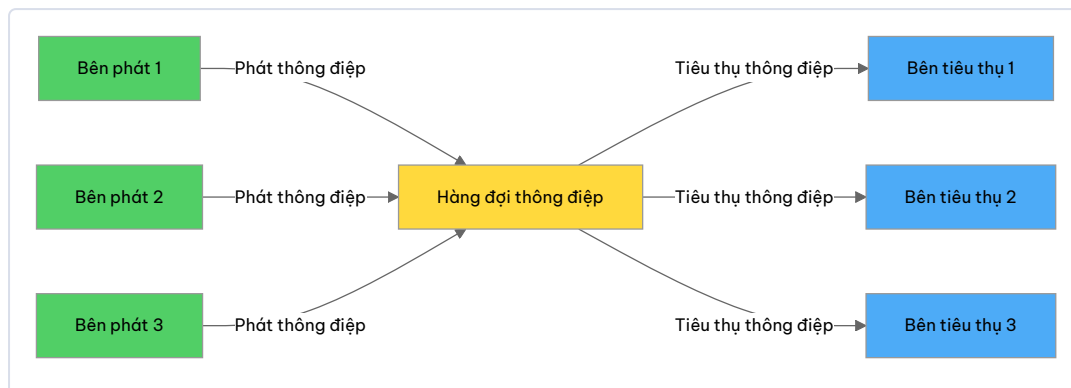
Khả năng chịu lỗi

- Email lỗi? Video vẫn được tải lên
- Có sẵn cơ chế 'thư' lại
- Liên kết lỏng

Kiến Thức Nền Tảng Về Hàng Đợi Thông Điệp

Các Khái Niệm Cốt Lõi

Hàng đợi thông điệp = Vùng đệm giữa bên phát (*producers*) và bên tiêu thụ (*consumers*)



Hàng đợi thông điệp tách rời (*decouples*) bên phát và bên tiêu thụ

Thuật ngữ:

Producer (bên phát): Dịch vụ tạo thông điệp (publish/send)
Consumer (bên tiêu thụ): Dịch vụ xử lý thông điệp (consume/receive)
Queue (hàng đợi): Vùng đệm lưu thông điệp
Message (thông điệp): Đơn vị công việc (job/task/event)
Broker (môi giới): Dịch vụ quản lý hàng đợi (RabbitMQ, Kafka, SQS)

Ví Dụ Đơn Giản

```
# Producer: API server
from queue import MessageQueue

queue = MessageQueue('video-processing')

def upload_video(video_file):
    # Save video nhanh
    video_id = save_to_storage(video_file)

    # Add job to queue
    queue.publish({
        'job': 'process_video',
        'video_id': video_id,
        'tasks': ['thumbnail', 'transcode_720p', 'transcode_1080p']
    })

    # Return ngay cho user
    return {
        'status': 'processing',
        'video_id': video_id,
        'message': 'Video is being processed'
    }

# Consumer: Background worker
def worker():
    while True:
        # Poll for messages
        message = queue.consume()

        if message:
            video_id = message['video_id']

            # Process tasks
            generate_thumbnail(video_id)
            transcode_720p(video_id)
            transcode_1080p(video_id)

            # Acknowledge message processed
            queue.ack(message)
```

Dòng thời gian:

Lời gọi API:

00:00 - Người dùng tải lên

00:02 - API trả về (người dùng vui vẻ!)

Hàng đợi có 1 tác vụ

Chế độ nhàn:

00:02 - Worker nhận tác vụ

00:07 - Xong ảnh thu nhỏ

00:37 - Xong 720p

01:37 - Xong 1080p

01:38 - Gửi thông báo

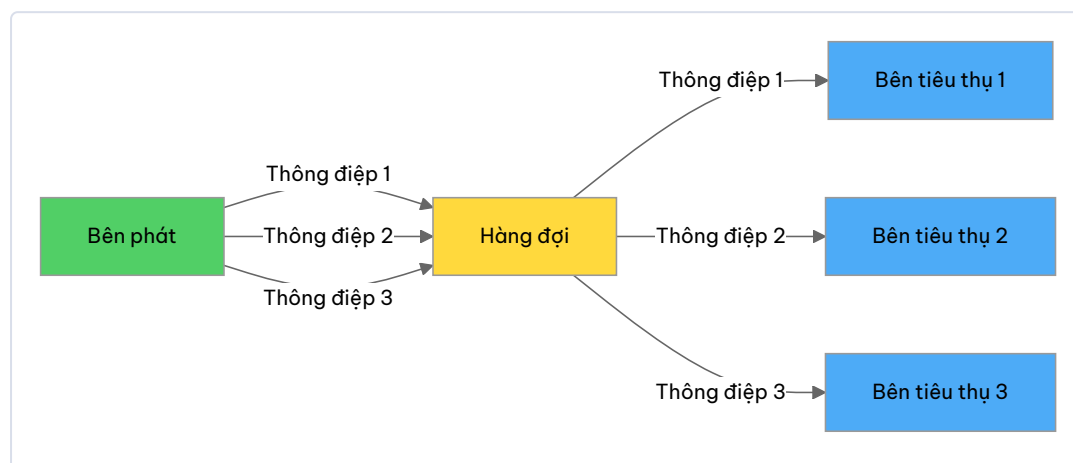
Trải nghiệm người dùng: 2 giây

Công việc thực tế: 98 giây (chạy nhàn)

Hàng Đợi (Queue) Và Pub/Sub: Hai Mẫu Thiết Kế Khác Nhau

Mẫu Hàng Đợi (Điểm-Tới-Điểm / Point-to-Point)

Một thông điệp → Một bên tiêu thụ (consumer)



Hàng đợi: Mỗi thông điệp chỉ được một bên tiêu thụ xử lý

Đặc điểm:

- Mỗi thông điệp chỉ được xử lý bởi 1 bên tiêu thụ
- Cân bằng tải giữa các bên tiêu thụ
- Dùng để phân phối tác vụ
- Các bên cạnh tranh nhau để nhận thông điệp

Tình huống sử dụng (Use cases):

- ✓ Tác vụ nền (gửi email, xử lý ảnh)
- ✓ Phân phối tác vụ (mã hoá video, tạo PDF)
- ✓ San phẳng tải (làm mượt các đợt lưu lượng tăng đột biến)
- ✓ Logic thu lại (thanh toán lỗi, lời gọi API lỗi)

Ví dụ:

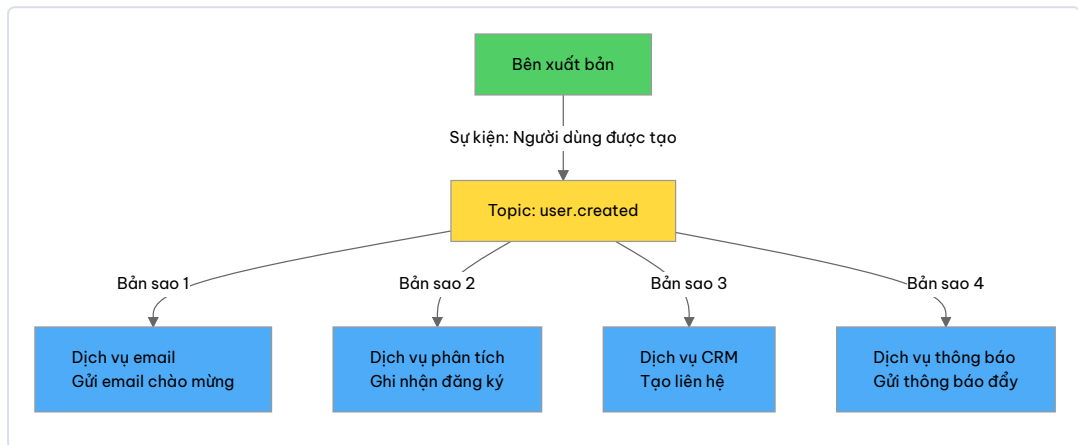
```
# Queue: email-queue
# 3 workers compete for jobs

# Producer
queue.publish({'to': 'user@example.com', 'subject': 'Welcome'})

# Consumer 1 might get it
# OR Consumer 2 might get it
# OR Consumer 3 might get it
# Only ONE processes it
```

Mẫu Pub/Sub (Xuất Bản – Đăng Ký (Publish-Subscribe))

Một thông điệp → Nhiều bên đăng ký (subscribers)



Pub/Sub: Mỗi bên đăng ký (subscriber) nhận một bản sao của thông điệp

Đặc điểm:

- Mỗi thông điệp được sao chép tới TẤT CẢ bên đăng ký
- Không có cạnh tranh
- Dùng để phát sự kiện diện rộng
- Các dịch vụ được tách rời

Tình huống sử dụng:

- ✓ Thông báo sự kiện (người dùng đăng ký, đơn hàng được đặt)
- ✓ Cập nhật thời gian thực (giá cổ phiếu, tỷ số trực tiếp)
- ✓ Phân phát (fan-out) (1 sự kiện → nhiều hành động)
- ✓ Ghi log kiểm toán (theo dõi mọi hành động)

Ví dụ:

```
# Topic: order.placed
# All subscribers receive event

# Publisher
pubsub.publish('order.placed', {
  'order_id': 12345,
  'user_id': 67890,
  'amount': 99.99
})

# Subscriber 1: Email Service
# Sends order confirmation email

# Subscriber 2: Inventory Service
# Updates stock count

# Subscriber 3: Analytics Service
# Records sale

# Subscriber 4: Shipping Service
# Creates shipping label

# ALL 4 receive and process the event
```

So Sánh Sự Đánh Đổi (Trade-off)

Hàng đợi (Điểm-tới-điểm):
Ngữ nghĩa đơn giản (1 thông điệp = 1 bên tiêu thụ)
Cân bằng tải một cách tự nhiên
Dễ suy luận
Giới hạn ở một hành động cho mỗi thông điệp
Liên kết chặt nếu cần nhiều hành động

Pub/Sub:
Tách rời hoàn toàn các dịch vụ
Dễ thêm bên đăng ký mới
Phát sự kiện diện rộng một cách tự nhiên
Phức tạp hơn (nhất quán cuối cùng)
Khó gỡ lỗi hơn (ai đã xử lý cái gì?)
Chi phí phụ do nhân bản thông điệp

Khung ra quyết định (Decision framework):

Dùng Hàng đợi khi:

- Tác vụ cần được thực hiện đúng một lần
- Cân bằng tải giữa các worker
- Quy trình đơn giản

Dùng Pub/Sub khi:

- Nhiều dịch vụ cùng quan tâm đến sự kiện
- Cần tách rời các dịch vụ
- Mẫu phân phát (fan-out)
- Có yêu cầu kiểm toán/ghi log

Bảo Đảm Phân Phối (Delivery Guarantees): Hiểu Biết Then Chốt

Phổ Các Mức Bảo Đảm

Phân Phối Tối Đa Một Lần (At-Most-Once Delivery)

Thông điệp có thể bị mất, nhưng không bao giờ bị trùng lặp

Dòng thời gian:

Bên phát → Gửi thông điệp → Hàng đợi

Hàng đợi → Giao cho bên tiêu thụ (có thể thất bại)

Hàng đợi xoá thông điệp ngay lập tức

Kết quả:

- Thành công: Thông điệp được xử lý một lần ✓
- Thất bại: Thông điệp bị mất ✗
- Không bao giờ: Thông điệp bị xử lý hai lần ✓

Nhanh nhất (không cần xác nhận)

Đơn giản

Có thể mất dữ liệu

Không thu lại

Tình huống sử dụng:

- Số liệu đo/ghi log (mất một ít dữ liệu cũng không sao)
- Lưu trữ dữ liệu thời gian thực (dữ liệu cũ không còn giá trị)
- Thông báo theo kiểu cố gắng hết mức (best-effort)

Phân Phối Ít Nhất Một Lần (At-Least-Once Delivery)

Thông điệp không bao giờ bị mất, nhưng có thể bị trùng lặp

Dòng thời gian:

Bên phát → Gửi thông điệp → Hàng đợi

Hàng đợi → Giao cho bên tiêu thụ

Bên tiêu thụ → Xử lý

Bên tiêu thụ → Xác nhận (ack) (có thể thất bại/hết thời gian chờ)

Hàng đợi → Giao lại nếu không nhận được ack

Kết quả:

- Thành công: Thông điệp được xử lý một lần ✓
- Thất bại: Thông điệp được giao lại ✓
- Có thể xảy ra: Thông điệp bị xử lý nhiều lần ⚠

Không mất thông điệp

Có sẵn logic thu lại

Có thể xử lý trùng lặp

Bắt buộc phải đảm bảo tính lũy đẳng

Tình huống sử dụng:

- Xử lý thanh toán (kèm tính lũy đẳng)
- Hoàn tất đơn hàng
- Logic nghiệp vụ quan trọng

Phân Phối Đúng Một Lần (Exactly-Once Delivery)

Thông điệp được xử lý đúng một lần, có bảo đảm

Cực kỳ khó đạt được trong hệ thống phân tán

Đòi hỏi:

- Giao dịch phân tán
- Logic khử trùng lặp
- Phối hợp giữa các dịch vụ

Kết quả:

- Luôn luôn: Thông điệp được xử lý đúng một lần ✓

Ngữ nghĩa hoàn hảo

Không mất, không trùng

Rất phức tạp để triển khai

Chi phí phụ về hiệu năng

Thường bất khả thi trong thực tế

Tình huống sử dụng:

- Giao dịch tài chính
- Nghiệp vụ quan trọng
- Khi tuyệt đối không được phép trùng lặp

Tính Lũy Đẳng (Idempotency): Giải Pháp Thực Tế

Vấn đề với phân phối ít nhất một lần (at-least-once):

```
# Non-idempotent operation
def process_payment(order_id, amount):
    charge_credit_card(amount) # Charge twice if retry!

# Message delivered twice:
# 1st: Charge $100 ✓
# 2nd: Charge $100 again! ✗
# User charged $200
```

Giải pháp: Các thao tác lũy đẳng (idempotent)


```

# Idempotent operation
def process_payment(order_id, amount):
    # Check if already processed
    if payment_exists(order_id):
        return # Skip, already done

    charge_credit_card(amount)
    record_payment(order_id) # Mark as processed

# Message delivered twice:
# 1st: Charge $100, record ✓
# 2nd: Check → Already done, skip ✓
# User charged $100 (correct!)

```

Các kỹ thuật đảm bảo tính lũy đẳng:

```

# Technique 1: Unique message ID
def process_message(message):
    message_id = message['id']

    if redis.exists(f"processed:{message_id}"):
        return # Already processed

    # Do work
    do_actual_work(message)

    # Mark as processed
    redis.set(f"processed:{message_id}", "1", ex=86400) # 24h TTL

# Technique 2: Database unique constraint
def create_order(order_data):
    try:
        db.execute("""
            INSERT INTO orders (order_id, user_id, amount)
            VALUES (?, ?, ?)
            """, order_data)
    except UniqueConstraintError:
        # Already exists, idempotent ✓
        pass

# Technique 3: Update operations (naturally idempotent)
def update_user_status(user_id, status):
    db.execute("""
        UPDATE users SET status = ?
        WHERE user_id = ?
        """, status, user_id)

    # Running twice = same result ✓

```

Quy tắc cá nhân:

Luôn giả định phân phối ít nhất một lần. Luôn đảm bảo tính lũy đẳng.

Ngay cả khi hàng đợi hứa hẹn đúng một lần (*exactly-once*), các sự cố mạng, các lần thử lại (*retries*) và lỗi (*bugs*) vẫn có thể gây ra trùng lặp.

Hàng Đợi Thư Chết (*Dead Letter Queue*) (DLQ): Xử Lý Sự Cố (*Failures*)

Vấn Đề Thông Điệp Độc (*Poison Message*)

```
# Worker consuming messages
while True:
    message = queue.consume()

    try:
        process(message)
        queue.ack(message)
    except Exception as e:
        # Message fails → Retry
        # Retry fails again
        # Retry fails again
        # ...infinite loop!
        # Queue blocked!
```

Vấn đề:

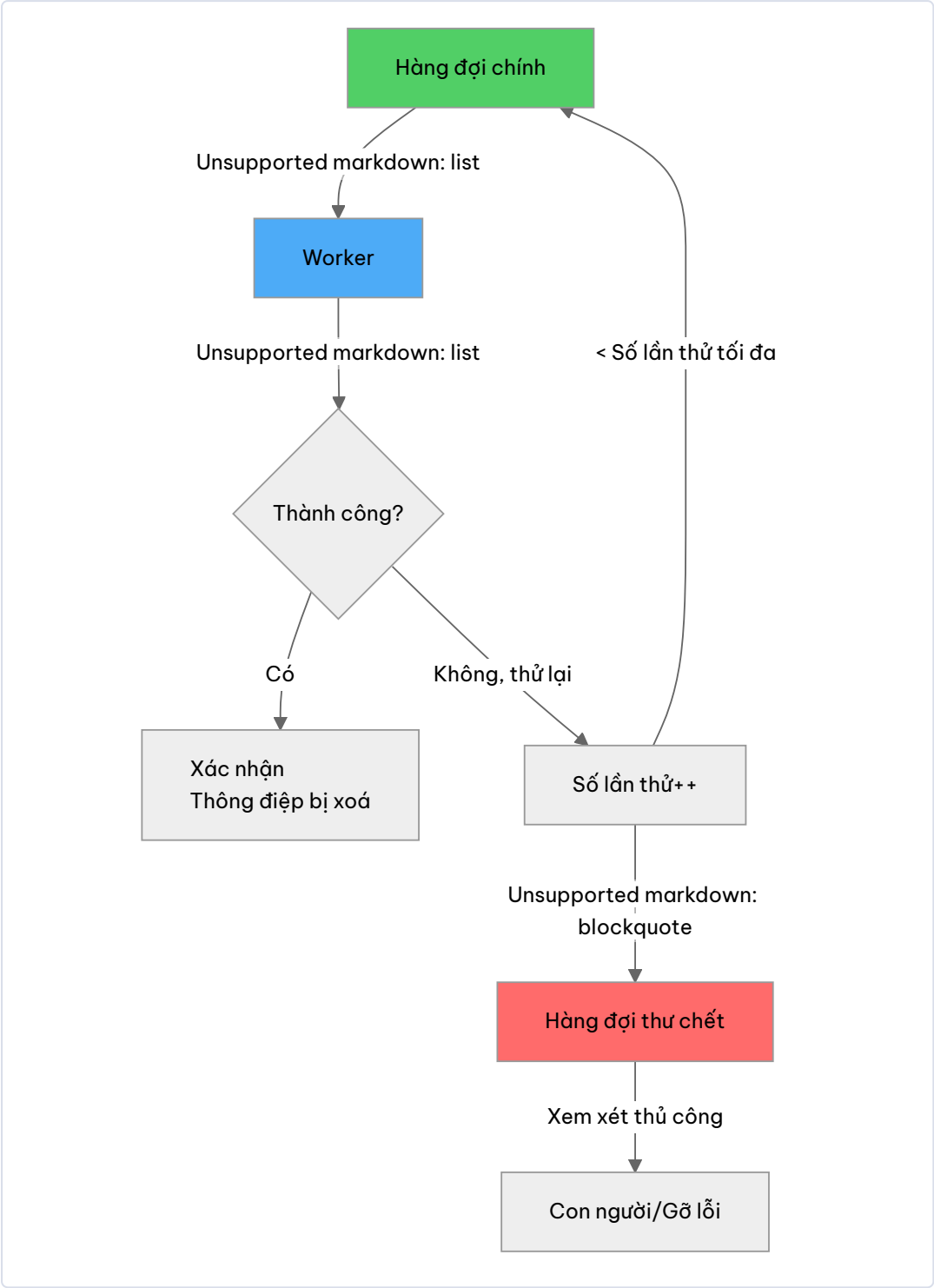
Kịch bản:

- Thông điệp có lỗi (dữ liệu sai định dạng)
- Worker xử lý → Thất bại
- Hàng đợi thư'lại → Lại thất bại
- Hàng đợi thư'lại → Lại thất bại
- ...

Kết quả:

- Thông điệp không bao giờ thành công
- Chặn hàng đợi (nếu có thứ tự)
- Lãng phí tài nguyên
- Các thông điệp khác bị kẹt phía sau

Giải Pháp DLQ



Hàng đợi thư chết: Các thông điệp thất bại nhiều lần được chuyển sang DLQ

Triển khai:

```
class MessageProcessor:
    MAX_RETRIES = 3

    def process_message(self, message):
        try:
            # Attempt processing
            result = do_work(message)

            # Success → Acknowledge
            queue.ack(message)
            return result

        except Exception as e:
            # Increment retry count
            retry_count = message.get('retry_count', 0) + 1

            if retry_count < self.MAX_RETRIES:
                # Retry
                message['retry_count'] = retry_count
                queue.nack(message) # Return to queue

                log.warning(f"Message retry {retry_count}/{self.MAX_RETRIES}")
            else:
                # Max retries exceeded → DLQ
                dlq.publish(message)
                queue.ack(message) # Remove from main queue

                log.error(f"Message moved to DLQ after {self.MAX_RETRIES}
retries")

                alert.send("DLQ message", message)
```

Giám sát (Monitoring) DLQ:

```
# Monitor DLQ regularly
def check_dlq():
    dlq_size = dlq.size()

    if dlq_size > 10:
        alert.send(f"DLQ has {dlq_size} messages!")

    # Sample messages for debugging
    messages = dlq.peek(5)
    for msg in messages:
        log.info(f"DLQ message: {msg}")
        analyze_failure(msg)

# Run every hour
schedule.every(1).hour.do(check_dlq)
```

Xử lý các thông điệp trong DLQ:

1. Điều tra nguyên nhân gốc
 - Dữ liệu sai định dạng?
 - Lỗi trong mã?
 - Dịch vụ bên ngoài bị sập?
2. Khắc phục vấn đề`
 - Triển khai bản sửa mã
 - Di chuyển dữ liệu
 - Cập nhật cấu hình
3. Phát lại thông điệp
 - Chuyển DLQ → Hàng đợi chính
 - Xử lý lại với bản sửa
 - Xác minh thành công
4. Hoặc loại bỏ
 - Nếu thực sự không hợp lệ
 - Nếu đã cũ (dữ liệu cũ)
 - Ghi lại lý do

Các Kịch Bản (Scenarios) Thực Tế

Kịch Bản 1: Xử Lý Đơn Hàng Thương Mại Điện Tử

Cơ ác mạng đồng bộ:

```
def place_order(order_data):
    # All in one request
    validate_inventory(order_data)    # 200ms
    charge_payment(order_data)        # 2s
    update_inventory(order_data)      # 500ms
    create_shipping_label(order_data) # 3s
    send_confirmation_email(order_data) # 1s
    update_analytics(order_data)      # 300ms

    return {"status": "success"}      # After 7 seconds!
```

Bất đồng bộ với hàng đợi:

```
def place_order(order_data):
    # Critical path only
    validate_inventory(order_data) # 200ms
    charge_payment(order_data)     # 2s

    # Everything else → Queue
    queue.publish({
        'event': 'order.placed',
        'order_id': order_data['id']
    })

    return {"status": "success"}    # After 2.2 seconds!

# Background workers
@queue.consumer('order.placed')
def handle_order_placed(event):
    order_id = event['order_id']

    update_inventory(order_id)
    create_shipping_label(order_id)
    send_confirmation_email(order_id)
    update_analytics(order_id)
```

Kết quả:

Trải nghiệm người dùng:

Trước: đợi 7 giây

Sau: 2.2 giây (nhanh hơn 70%!)

Năng lực hệ thống:

Trước: 10 đơn hàng/phút (bị chặn bởi quá trình xử lý dài)

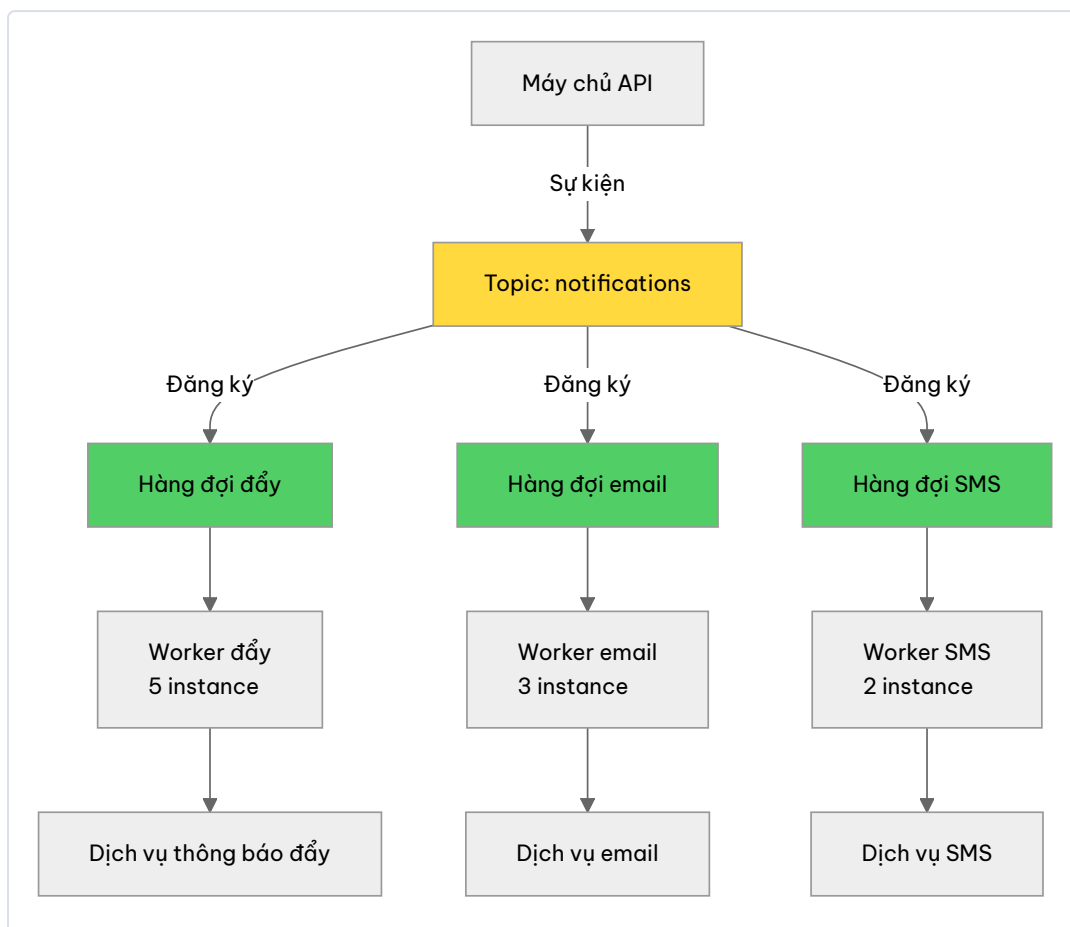
Sau: 50 đơn hàng/phút (phản hồi nhanh)

Kịch Bản 2: Hệ Thống Thông Báo

Yêu cầu (Requirements):

- Gửi thông báo đẩy đến hàng triệu người dùng
- Nhiều kênh (đẩy, email, SMS)
- Không được chặn phản hồi của API
- Cần logic thư' lại
- Theo dõi trạng thái gửi

Kiến trúc:



Mẫu Pub/Sub: Sự kiện (Event) được phân phát (fanout) đến nhiều kênh

```

# API publishes event
def send_notification(user_id, message):
    pubsub.publish('notifications', {
        'user_id': user_id,
        'message': message,
        'channels': ['push', 'email']
    })

    return {"status": "queued"} # Instant response

# Push worker
@pubsub.subscribe('notifications')
def send_push(event):
    if 'push' in event['channels']:
        try:
            push_service.send(event['user_id'], event['message'])
            metrics.increment('push.success')
        except Exception as e:
            metrics.increment('push.failure')
            raise # Retry

# Email worker (independent)
@pubsub.subscribe('notifications')
def send_email(event):
    if 'email' in event['channels']:
        try:
            email_service.send(event['user_id'], event['message'])
            metrics.increment('email.success')
        except Exception as e:
            metrics.increment('email.failure')
            raise # Retry

```

Kịch Bản 3: Đường Ống (Pipeline) Xử Lý Ảnh

Chuỗi các phép biến đổi:

Tải lên ảnh gốc → Đổi kích thước → Nén → Đóng dấu bản quyền → Lưu lên CDN

Triển khai bằng hàng đợi (queues):


```

# Step 1: Upload
def upload_image(image_file):
    image_id = save_original(image_file)

    queue.publish('image.uploaded', {
        'image_id': image_id,
        'format': 'jpg'
    })

    return {"image_id": image_id, "status": "processing"}

# Step 2: Resize
@queue.consumer('image.uploaded')
def resize_image(event):
    image_id = event['image_id']

    resized = resize(image_id, width=1200)
    save(resized, f"{image_id}_resized")

    queue.publish('image.resized', {
        'image_id': image_id
    })

# Step 3: Compress
@queue.consumer('image.resized')
def compress_image(event):
    image_id = event['image_id']

    compressed = compress(f"{image_id}_resized", quality=85)
    save(compressed, f"{image_id}_compressed")

    queue.publish('image.compressed', {
        'image_id': image_id
    })

# Step 4: Watermark
@queue.consumer('image.compressed')
def add_watermark(event):
    image_id = event['image_id']

    watermarked = add_watermark(f"{image_id}_compressed")
    save(watermarked, f"{image_id}_final")

    queue.publish('image.ready', {
        'image_id': image_id,
        'url': upload_to_cdn(f"{image_id}_final")
    })

# Step 5: Notify
@queue.consumer('image.ready')

```

```
def notify_user(event):
    send_notification(event['image_id'], event['url'])
```

Lợi ích:

Mỗi bước độc lập
Mở rộng từng bước riêng biệt
Thư' lại từng bước riêng lẻ
Dễ thêm các phép biến đổi mới
Giám sát hiệu năng của từng bước

Thực Hành Tốt Nhất (Best Practices)

1. Thiết Kế Thông Điệp

```
# BAD: Large message
queue.publish({
    'user': {
        # Entire user object (10KB)
        'id': 123,
        'name': 'John',
        'email': '...',
        # ... 50 more fields
    },
    'order': {
        # Entire order (20KB)
    }
})

# GOOD: Reference only
queue.publish({
    'user_id': 123,
    'order_id': 456
})

# Consumer fetches what it needs
@queue.consumer
def process(event):
    user = db.get_user(event['user_id'])
    order = db.get_order(event['order_id'])
```

Tại sao?

- Thông điệp nhỏ hơn = nhanh hơn

- Tránh dữ liệu cũ (*stale data*) (lấy dữ liệu mới từ cơ sở dữ liệu)
- Tốn ít băng thông (*bandwidth*) mạng hơn

2. Xử Lý Lỗi

```
@queue.consumer
def process_message(message):
    try:
        result = do_work(message)
        queue.ack(message)

    except TemporaryError as e:
        # Transient error → Retry
        log.warning(f"Temporary error: {e}, will retry")
        queue.nack(message) # Requeue

    except PermanentError as e:
        # Permanent error → DLQ
        log.error(f"Permanent error: {e}, moving to DLQ")
        dlq.publish(message)
        queue.ack(message)

    except Exception as e:
        # Unknown error → Investigate
        log.exception(f"Unknown error: {e}")
        alert.send("Unknown queue error", message, e)
        queue.nack(message)
```

3. Giám Sát

```
# Track queue metrics
metrics.gauge('queue.size', queue.size())
metrics.gauge('queue.oldest_message_age', queue.oldest_age())
metrics.gauge('dlq.size', dlq.size())

metrics.histogram('message.processing_time', processing_time)
metrics.increment('message.success')
metrics.increment('message.failure')

# Alerts
if queue.size() > 10000:
    alert.send("Queue backlog!")

if queue.oldest_age() > 3600: # 1 hour
    alert.send("Messages stuck in queue!")

if dlq.size() > 100:
    alert.send("Too many DLQ messages!")
```

4. Xử Lý Áp Lực Ngược (Backpressure)

```
# Consumer rate limiting
class RateLimitedConsumer:
    def __init__(self, max_per_second=100):
        self.rate_limiter = RateLimiter(max_per_second)

    def consume(self):
        while True:
            # Wait if rate limit exceeded
            self.rate_limiter.wait()

            message = queue.consume()
            if message:
                self.process(message)

# Circuit breaker pattern
class CircuitBreaker:
    def __init__(self):
        self.failures = 0
        self.state = 'closed' # closed, open, half-open

    def call(self, func):
        if self.state == 'open':
            raise Exception("Circuit breaker open")

        try:
            result = func()
            self.failures = 0
            return result
        except Exception:
            self.failures += 1
            if self.failures >= 5:
                self.state = 'open'
            raise
```

Những Điểm Cốt Lõi

Hàng đợi thông điệp giải quyết những vấn đề then chốt:

- ✓ Tách rời các dịch vụ
- ✓ Chịu được lưu lượng tăng đột biến (vùng đệm)
- ✓ Cho phép xử lý bất đồng bộ
- ✓ Có sẵn logic thu lại
- ✓ Cải thiện trải nghiệm người dùng (phản hồi nhanh)

Các mẫu thiết kế:

Hàng đợi (Điểm-tới-điểm):

- Phân phối tác vụ
- Cân bằng tải
- 1 thông điệp → 1 bên tiêu thụ

Pub/Sub:

- Phát sự kiện diện rộng
- Mẫu phân phát (fan-out)
- 1 thông điệp → N bên đăng ký

Bảo đảm phân phối:

Tối đa một lần: Nhanh, có thể mất dữ liệu

Ít nhất một lần: Tin cậy, có thể trùng lặp (HÃY DỪNG CÁI NÀY)

Đúng một lần: Hoàn hảo, rất phức tạp (hiếm khi đạt được)

→ Luôn đảm bảo tính lũy đẳng

Thực hành tốt nhất:

- ✓ Giữ thông điệp nhỏ gọn (tham chiếu, không phải toàn bộ dữ liệu)
- ✓ Đảm bảo tính lũy đẳng
- ✓ Dùng DLQ cho thông điệp độc
- ✓ Giám sát các chỉ số của hàng đợi
- ✓ Xử lý áp lực ngược
- ✓ Thu' lại với các lỗi tạm thời
- ✓ Cảnh báo khi có bất thường

Khi nào dùng hàng đợi thông điệp?

Dùng hàng đợi khi:

- ✓ Tác vụ chạy lâu (> 5 giây)
- ✓ Người dùng không cần kết quả ngay
- ✓ Cần tách rời các dịch vụ
- ✓ Phải chịu lưu lượng tăng đột biến
- ✓ Cần có sẵn logic thu' lại

Không dùng khi:

- x Người dùng cần phản hồi ngay lập tức
- x Quy trình đồng bộ
- x Các thao tác CRUD đơn giản

Lời khuyên cá nhân:

Hãy bắt đầu đơn giản với các dịch vụ được quản lý sẵn (AWS SQS, Google Pub/Sub). Chỉ tự vận hành hạ tầng (*infrastructure*) riêng (RabbitMQ, Kafka) khi có yêu cầu cụ thể và nhóm (*team*) có đủ chuyên môn.

Hàng đợi thông điệp là một trong những mẫu thiết kế quan trọng nhất trong hệ thống phân tán. Nắm vững nó, bạn sẽ mở khoá khả năng xây dựng những ứng dụng thực sự có khả năng mở rộng (*scalable*) và bền bỉ trước sự cố.

Tích Hợp (Integration) & Tư Duy Đánh Đổi (Trade-off Thinking): Kết Hợp Các Thành Phần (Components) Thành Hệ Thống

Đây là bài học mà 90% khoá học bỏ qua. Và đó là lý do tại sao nhiều kỹ sư (*engineers*) biết từng thành phần (*component*) nhưng không biết thiết kế hệ thống.

Tôi còn nhớ lần đầu tiên phải thiết kế một hệ thống hoàn chỉnh. Tôi biết bộ nhớ đệm (*cache*), biết cơ sở dữ liệu (*database*), biết hàng đợi thông điệp (*message queue*). Nhưng khi ngồi trước tấm bảng trắng, đầu óc tôi hoàn toàn trống rỗng.

"Tôi dùng cái nào trước? Bộ nhớ đệm hay cơ sở dữ liệu? Hàng đợi (*Queue*) đặt ở đâu? Bộ cân bằng tải (*Load balancer*) kết nối với cái gì?"

Một kiến trúc sư lâu năm (*senior architect*) ngồi bên cạnh nói: "Em biết từng viên gạch, nhưng chưa biết xây nhà."

Đó chính là bài học này. Không phải học thêm thành phần mới. Mà học cách **kết hợp chúng thành hệ thống hoàn chỉnh**.

Tại Sao Tư Duy Tích Hợp (Integration Thinking) Quan Trọng Hơn Kiến Thức Về Từng Thành Phần?

Đổi chiếu với thực tế:

Biết bộ nhớ đệm: ☒
Biết cơ sở dữ liệu: ☒
Biết hàng đợi thông điệp: ☒
Biết bộ cân bằng tải: ☒

Thiết kế được hệ thống: ☐

Tại sao?

Vì **thiết kế hệ thống không phải là về các thành phần**. Nó là về sự tương tác (*interactions*) **giữa các thành phần**.

Khoảng Trống Về Tích Hợp

Khoảng trống 1: Tương tác giữa các thành phần (*Component interactions*)

Câu hỏi: Bộ nhớ đệm nằm ở đâu trong luồng xử lý?

Tư duy sai:

"Tôi cần bộ nhớ đệm → Thêm Redis"

Tư duy đúng:

"Bộ nhớ đệm nằm giữa máy chủ ứng dụng và cơ sở dữ liệu"

→ Ứng dụng kiểm tra bộ nhớ đệm trước

→ Trượt bộ nhớ đệm → Truy vấn cơ sở dữ liệu

→ Lưu vào bộ nhớ đệm

→ Yêu cầu tiếp theo trúng bộ nhớ đệm"

Khoảng trống 2: Kết hợp các sự đánh đổi (*trade-offs*)

Mỗi thành phần có những đánh đổi riêng:

- Bộ nhớ đệm: Nhanh nhưng có rủi ro dữ liệu cũ
- Hàng đợi: Bất đồng bộ nhưng xử lý không tắc thì (xử lý sau)
- Bản sao CSDL: Mở rộng việc đọc nhưng có độ trễ nhân bản

Câu hỏi: Khi kết hợp chúng lại, các đánh đổi tương tác với nhau thế nào?

Khoảng trống 3: Ràng buộc (*Constraints*) thực tế

Sách giáo khoa: "Dùng bộ nhớ đệm để tăng hiệu năng"

Thực tế:

- Ngân sách: \$500/tháng
- Nhóm: 2 lập trình viên
- Bảo trì: Ai giám sát Redis?
- Độ phức tạp: Nhóm có kinh nghiệm không?

→ Quyết định phức tạp hơn nhiều

Điểm mấu chốt: Các thành phần là công cụ. Tích hợp là tay nghề.

Khung Tư Duy (*Thinking Framework*): Từ Bài Toán Đến Kiến Trúc (*Architecture*)

Đây là khung tư duy (*framework*) tôi dùng cho mọi bài toán thiết kế hệ thống (*system design*).

Bước 1: Xác Định Yêu Cầu (Requirements) (Bài Toán Trước Tiên)

Yêu cầu chức năng (Functional requirements):

Hệ thống phải làm Gì:

- Người dùng tải ảnh lên
- Hệ thống thay đổi kích thước ảnh
- Người dùng xem ảnh đã được thay đổi kích thước

Yêu cầu phi chức năng (Non-functional requirements):

Hệ thống phải làm TỐT ĐẾN MỨC NÀO:

- Tải lên: phản hồi < 5 giây
- Xem: độ trễ < 500ms
- Quy mô: 100 nghìn lượt tải lên/ngày
- Tính sẵn sàng: 99,9% thời gian hoạt động

Ràng buộc:

- Ngân sách: \$1.000/tháng
- Nhóm: 3 lập trình viên
- Thời hạn: Phát hành trong 2 tháng
- Chuyên môn: Nhóm biết Node.js, PostgreSQL

Bước 2: Xác Định Các Thành Phần Cốt Lõi Cần Thiết

Các thành phần bắt buộc phải có:

Lưu trữ: Cần lưu ảnh → Lưu trữ đối tượng (S3)

Cơ sở dữ liệu: Cần siêu dữ liệu → SQL hoặc NoSQL

Tính toán: Cần thay đổi kích thước ảnh → Máy chủ ứng dụng

Các thành phần tùy chọn (cần đánh giá):

Bộ nhớ đệm: Có cần không? → Tùy vào kiểu đọc

Hàng đợi: Có cần không? → Tùy vào thời gian xử lý

Bộ cân bằng tải: Có cần không? → Tùy vào lưu lượng

CDN: Có cần không? → Tùy vào việc có người dùng toàn cầu không

Tư duy phản biện:

Đừng thêm thành phần "đề phòng hờ"
Mỗi thành phần = Thêm độ phức tạp
Chỉ thêm khi có lý do rõ ràng

Bước 3: Vạch Ra Luồng Dữ Liệu (Data Flow)

Lần theo yêu cầu (*request*) **từ máy khách** (*client*) **đến phản hồi** (*response*):

Người dùng tải ảnh lên:

1. Máy khách → Máy chủ API
2. API → Kiểm tra tính hợp lệ của tệp
3. API → Tải lên S3
4. API → Thêm tác vụ vào hàng đợi
5. API → Trả về phản hồi "đang xử lý"
6. Worker → Xử lý tác vụ thay đổi kích thước
7. Worker → Tải ảnh đã thay đổi kích thước lên S3
8. Worker → Cập nhật cơ sở dữ liệu
9. Người dùng hỏi vòng (poll) API → Lấy trạng thái

Xác định:

- Ranh giới giữa đồng bộ và bất đồng bộ
- Chỗ nào nên thêm bộ nhớ đệm
- Chỗ nào có thể xảy ra sự cố

Bước 4: Áp Dụng Tư Duy Đánh Đổi

Với mỗi thành phần, hãy hỏi:

1. Nó giải quyết vấn đề gì?
2. Chi phí là gì? (độ phức tạp, tiền bạc, bảo trì)
3. Có những phương án thay thế nào?
4. Có thể bắt đầu đơn giản hơn không?

Ví dụ cây quyết định (*decision tree*):

Cần xử lý ảnh:

Phương án A: Đồng bộ

- Người dùng chờ thay đổi kích thước xong
- Mã đơn giản
- Trải nghiệm người dùng tệ nếu chậm

Phương án B: Hàng đợi + Worker

- Người dùng nhận phản hồi ngay lập tức
- Hạ tầng phức tạp
- Trải nghiệm người dùng tốt

Với các ràng buộc đã cho:

- Thay đổi kích thước mất 5 giây
 - Người dùng có chờ được 5 giây không? KHÔNG
- Chọn Phương án B

Bước 5: Bắt Đầu Đơn Giản, Tiến Hoá Dần

Phiên bản 1 (MVP):

Một máy chủ duy nhất

PostgreSQL

Không bộ nhớ đệm

Không hàng đợi (nếu xử lý đủ nhanh)

Phát hành trong 2 tuần

Học hỏi từ việc sử dụng thực tế

Phiên bản 2 (Sau khi có dữ liệu):

Nếu chậm:

→ Thêm hàng đợi + worker

Nếu đọc chậm:

→ Thêm bộ nhớ đệm

Nếu máy chủ quá tải:

→ Thêm bộ cân bằng tải

Nguyên tắc: Xây những gì bạn cần BÂY GIỜ, không phải những gì bạn có thể cần SAU NÀY.

Ví Dụ Kiến Trúc Thực Tế 1: Nền Tảng Blog Đơn Giản

Hãy cùng thiết kế từ con số không.

Yêu Cầu

Chức năng:

- Người dùng đọc bài viết blog
- Quản trị viên viết/sửa bài viết
- Người dùng bình luận

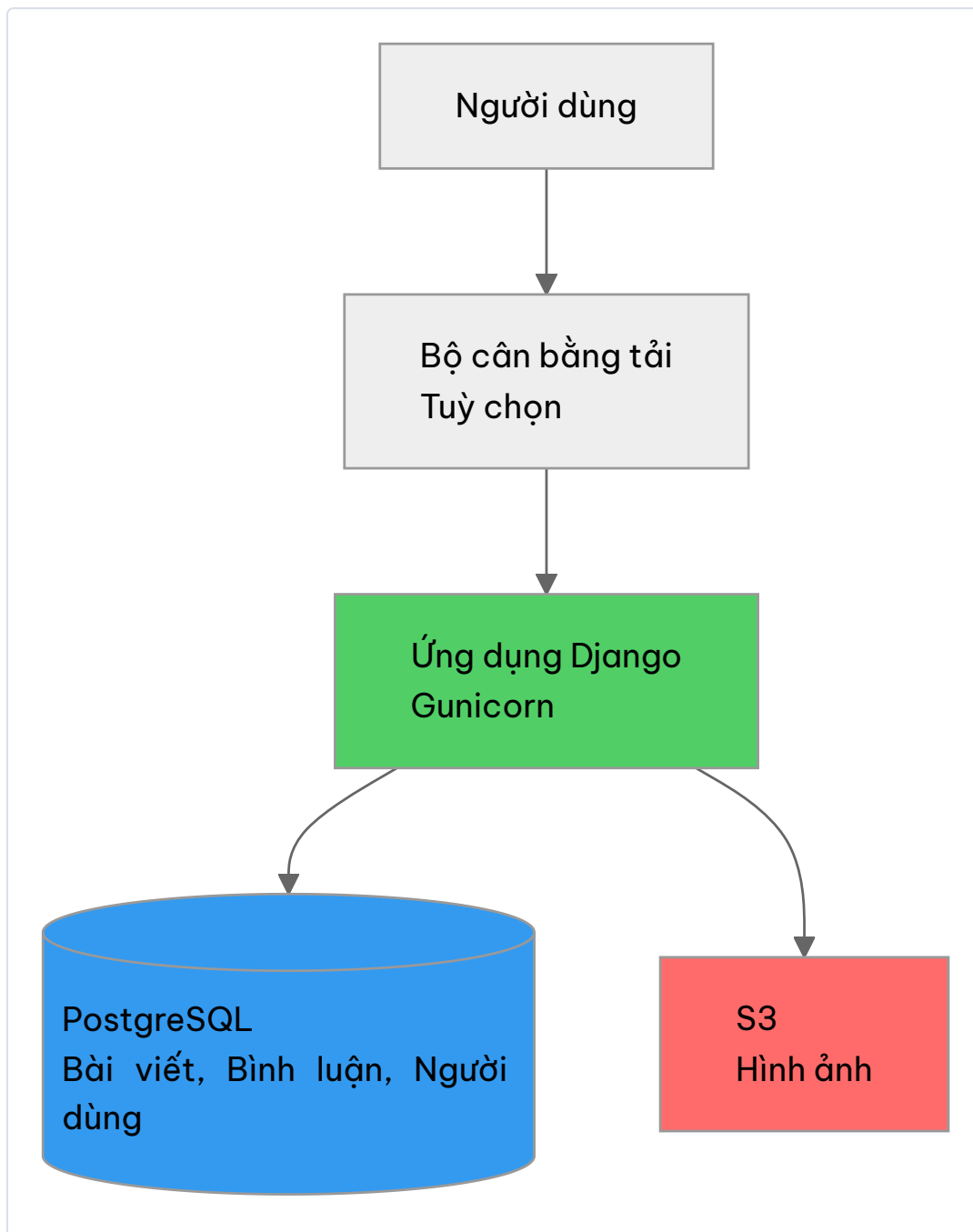
Phi chức năng:

- 10 nghìn người dùng hoạt động hằng ngày
- Tổng cộng 1 nghìn bài viết
- 10 nghìn bình luận/ngày
- Độ trễ < 1 giây

Ràng buộc:

- Ngân sách: \$200/tháng
- Nhóm: 2 lập trình viên
- Công nghệ: Python/Django, PostgreSQL

Kiến Trúc V1: Hệ Thống Nguyên Khối (Monolith) Đơn Giản



Kiến trúc V1: Hệ thống nguyên khối đơn giản, không có bộ nhớ đệm hay hàng đợi

Lý do đằng sau quyết định:

Không bộ nhớ đệm:

- ✓ 10 nghìn người dùng = Lưu lượng thấp
- ✓ 1 nghìn bài viết = CSDL xử lý dễ dàng
- ✓ PostgreSQL đủ nhanh khi có chỉ mục
- ✓ Tiết kiệm \$50/tháng chi phí Redis
- ✓ Ít phức tạp hơn

Không hàng đợi:

- ✓ Không có xử lý nặng
- ✓ Thêm bình luận nhanh (< 100ms)
- ✓ Tiết kiệm chi phí hạ tầng

Không CDN:

- ✓ Người dùng chỉ ở trong khu vực
- ✓ S3 vốn đã nhanh
- ✓ Tiết kiệm \$30/tháng chi phí CDN

Bộ cân bằng tải: Tùy chọn

- ✓ Có thì tốt để triển khai không gián đoạn
- ✓ 1 máy chủ xử lý tốt 10 nghìn người dùng
- ✓ Thêm sau nếu cần

Tổng chi phí: \$100/tháng (EC2 + RDS + S3)

Khi Nào Cần Tiến Hoá: Quyết Định Dựa Trên Dữ Liệu

Tín hiệu 1: Truy vấn (queries) cơ sở dữ liệu chậm

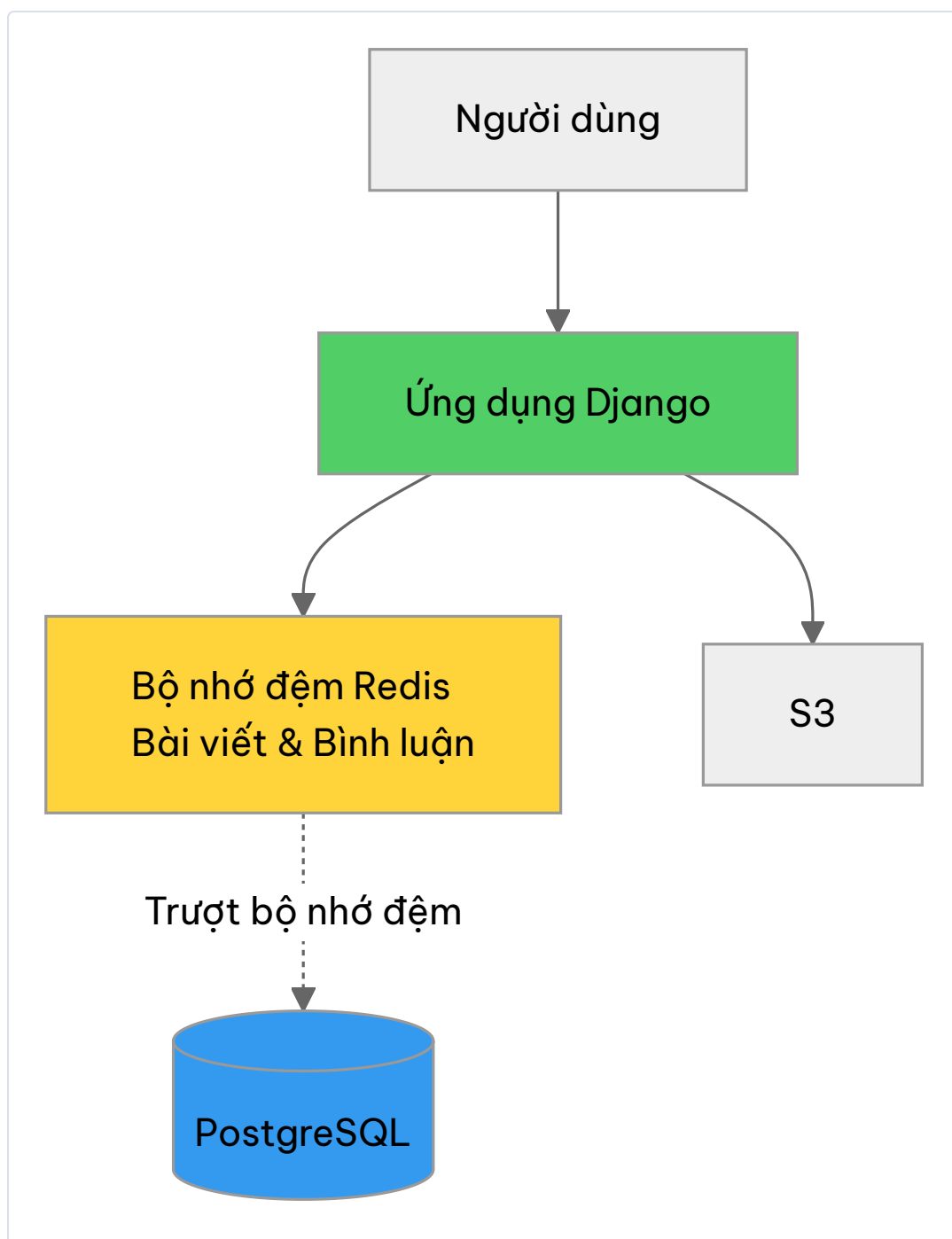
Triệu chứng:

- Tải trang mất 3 giây
- CPU của PostgreSQL ở mức 80%
- Log truy vấn chậm cho thấy các truy vấn lặp lại

Giải pháp:

- Thêm bộ nhớ đệm Redis

Kiến trúc V2: Thêm Tầng Bộ Nhớ Đệm (Cache Layer)



Kiến trúc V2: Thêm bộ nhớ đệm Redis khi truy vấn cơ sở dữ liệu chậm

Triển khai:


```
def get_post(post_id):
    # Try cache first
    cache_key = f"post:{post_id}"
    cached = redis.get(cache_key)

    if cached:
        return json.loads(cached) # Cache hit

    # Cache miss → Query database
    post = db.query("SELECT * FROM posts WHERE id = ?", post_id)

    # Store in cache (TTL 1 hour)
    redis.setex(cache_key, 3600, json.dumps(post))

    return post

def create_comment(post_id, comment_data):
    # Write to database
    db.insert("comments", comment_data)

    # Invalidate post cache (has new comment)
    redis.delete(f"post:{post_id}")
```

Chi phí mới: \$150/tháng (+\$50 cho Redis)

Tín hiệu 2: Lưu lượng (traffic) cao

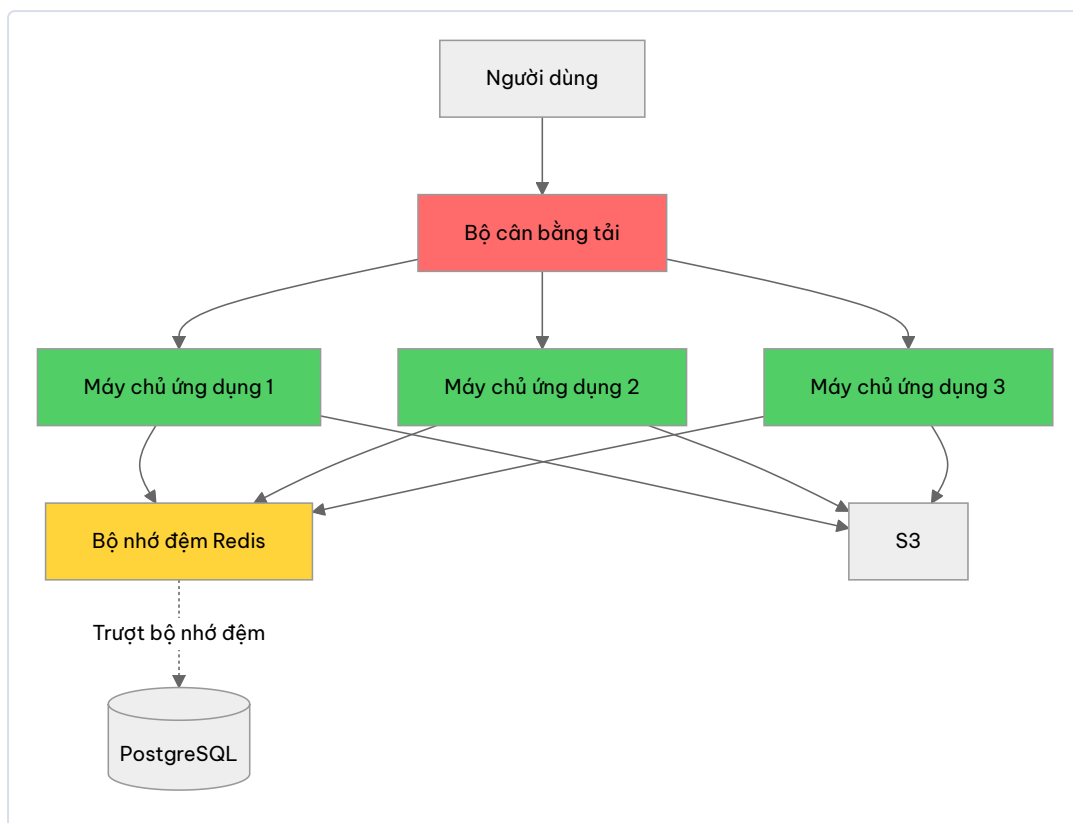
Triệu chứng:

- 100 nghìn người dùng hằng ngày (tăng 10 lần)
- CPU của máy chủ duy nhất ở mức 90%
- Thời gian phản hồi giảm sút

Giải pháp:

→ Thêm bộ cân bằng tải + Mở rộng theo chiều ngang

Kiến trúc V3: Mở Rộng Theo Chiều Ngang (Horizontal Scaling)



Kiến trúc V3: Bộ cân bằng tải + nhiều máy chủ (*servers*) khi lưu lượng cao

Chi phí mới: \$400/tháng (3 máy chủ + bộ cân bằng tải + Redis + CSDL)

Những Bài Học Chính

1. Bắt đầu đơn giản (V1)
 - Không tối ưu hoá sớm
 - Phát hành trong 2 tuần
 - Học hỏi từ dữ liệu thực tế
2. Tiến hoá dựa trên vấn đề thực sự (V2, V3)
 - Không dựa vào phỏng đoán
 - Không dựa vào "thực hành tốt nhất"
 - Số liệu đo thực tế dẫn dắt quyết định
3. Ý thức về chi phí
 - V1: \$100/tháng (10 nghìn người dùng)
 - V2: \$150/tháng (50 nghìn người dùng)
 - V3: \$400/tháng (100 nghìn người dùng)
 - Chi phí tăng tương ứng với giá trị
4. Kiểm soát độ phức tạp
 - Thêm từng thành phần một
 - Nhóm học hỏi dần dần
 - Duy trì sự ổn định trên môi trường thực tế

Ví Dụ Kiến Trúc Thực Tế 2: Dịch Vụ (Service) Xử Lý Ảnh

Một trường hợp phức tạp hơn với xử lý bất đồng bộ (*async processing*).

Yêu Cầu

Chức năng:

- Người dùng tải ảnh lên
- Hệ thống thay đổi kích thước thành 3 cỡ (ảnh thu nhỏ, vừa, lớn)
- Người dùng xem ảnh đã xử lý

Phi chức năng:

- 50 nghìn lượt tải lên/ngày
- Xử lý: 5-10 giây/ảnh
- Phản hồi khi tải lên: < 2 giây
- Độ trễ khi xem: < 500ms
- Tính sẵn sàng 99%

Ràng buộc:

- Ngân sách: \$2.000/tháng
- Nhóm: 4 lập trình viên
- Cần nhật ký kiểm toán (audit trail)

Kiến Trúc: Đầy Đủ Các Tầng



Kiến trúc hoàn chỉnh với tất cả các thành phần: CDN, bộ cân bằng tải, bộ nhớ đệm, hàng đợi, tiến trình xử lý (Workers), cơ sở dữ liệu

Tương Tác Giữa Các Thành Phần & Các Đánh Đổi

1. Luồng (Flow) Tải Lên (Ranh Giới Đồng Bộ (Sync) → Bất Đồng Bộ (Async))

```

# API Server
@app.post("/upload")
async def upload_image(file):
    # 1. Validate file (sync)
    if not is_valid_image(file):
        return {"error": "Invalid image"}, 400

    # 2. Upload original to S3 (sync, fast ~500ms)
    original_url = s3.upload(file, bucket="originals")

    # 3. Create database record (sync, fast ~50ms)
    image_id = db.insert("images", {
        "original_url": original_url,
        "status": "processing",
        "user_id": current_user.id
    })

    # 4. Add resize job to queue (async boundary)
    queue.send({
        "image_id": image_id,
        "original_url": original_url,
        "sizes": ["thumbnail", "medium", "large"]
    })

    # 5. Return immediately (< 2 seconds)
    return {
        "image_id": image_id,
        "status": "processing",
        "message": "Image is being processed"
    }, 202 # Accepted

# User không đợi resize (5-10 giây)

```

Đánh đổi (Trade-off):

Phản hồi người dùng nhanh (< 2 giây)
 Chịu được lưu lượng tăng đột biến (hàng đợi làm vùng đệm)
 Worker mở rộng độc lập
 Xử lý không tức thì (xử lý sau)
 Phức tạp hơn (hạ tầng hàng đợi)
 Cơ chế chờ vòng (polling) để lấy trạng thái

2. Xử Lý Tại Tiến Trình Xử Lý (Worker) (Bất Đồng Bộ)

```

# Worker
def process_resize_job(job):
    image_id = job["image_id"]
    original_url = job["original_url"]

    try:
        # 1. Download original
        original = s3.download(original_url)

        # 2. Resize to 3 sizes
        for size in ["thumbnail", "medium", "large"]:
            resized = resize_image(original, size)

            # Upload resized
            resized_url = s3.upload(resized, bucket=f"images-{size}")

            # Update database
            db.update("images", image_id, {
                f"{size}_url": resized_url
            })

        # 3. Update status
        db.update("images", image_id, {"status": "completed"})

        # 4. Invalidate cache
        cache.delete(f"image:{image_id}")

        # 5. Audit log
        db.insert("audit_logs", {
            "image_id": image_id,
            "action": "processed",
            "timestamp": now()
        })

    except Exception as e:
        # Error handling
        db.update("images", image_id, {"status": "failed"})

        # Retry logic (SQS auto-retry)
        if job.retry_count < 3:
            raise # Re-queue
        else:
            # Move to dead letter queue
            dlq.send(job)

```

Đánh đổi:

Chịu lỗi tốt (logic thu' lại)
Mở rộng được (thêm worker)
Có nhật ký kiểm toán (log trong cơ sở dữ liệu)
Xử lý lỗi phức tạp
Phải đảm bảo tính lũy đẳng
Chi phí phụ cho việc giám sát

3. Hở Vòng Trạng Thái (Đường Đọc (Read Path))

```
# API Server
@app.get("/image/{image_id}/status")
async def get_status(image_id):
    # 1. Check cache first
    cached = cache.get(f"image:{image_id}")
    if cached:
        return json.loads(cached)

    # 2. Cache miss → Query database
    image = db.query("SELECT * FROM images WHERE id = ?", image_id)

    # 3. Cache result (TTL 5 minutes)
    cache.setex(f"image:{image_id}", 300, json.dumps(image))

    return image
```

Đánh đổi:

Đọc nhanh (bộ nhớ đệm)
Giảm tải cho cơ sở dữ liệu
Độ phức tạp của việc vô hiệu hoá bộ nhớ đệm
Có thể có trạng thái cũ (tối đa 5 phút)

4. Xem Ảnh (Tầng CDN)

```
# Image URLs include CDN
image_url = f"https://cdn.example.com/images/large/{image_id}.jpg"

# CDN behavior:
- First request → CDN fetch từ S3 (origin) → Cache
- Subsequent requests → Serve từ CDN cache
- TTL: 1 year (images immutable)
```

Đánh đổi:

Cực nhanh (các điểm biên)
Giảm chi phí S3 (ít yêu cầu hơn)
Độ trễ thấp trên toàn cầu
Chi phí CDN (\$100-200/tháng)
Khó vô hiệu hoá bộ nhớ đệm (nếu ảnh thay đổi)

Tổng Hợp Các Đánh Đổi Của Kiến Trúc

Các thành phần:

- CDN: \$150/tháng
- Bộ cân bằng tải: \$50/tháng
- 2 máy chủ API: \$200/tháng
- Redis: \$100/tháng
- Hàng đợi SQS: \$20/tháng
- 3 worker: \$300/tháng
- PostgreSQL RDS: \$200/tháng
- S3: \$100/tháng

Tổng: \$1.120/tháng

Xử lý được:

- 50 nghìn lượt tải lên/ngày
- 150 nghìn ảnh được xử lý/ngày
- 1 triệu lượt xem/ngày
- Tính sẵn sàng 99,9%

Những đánh đổi chấp nhận:

Độ phức tạp → Đổi lấy khả năng mở rộng

Chi phí → Đổi lấy độ tin cậy

Xử lý không tức thì → Đổi lấy trải nghiệm người dùng

Nếu Ngân Sách Chỉ Là \$500/Tháng Thì Sao?

Kiến trúc thay thế:

Đơn giản hoá:

- Bỏ CDN → Truy cập S3 trực tiếp (tiết kiệm \$150)
- 1 máy chủ API → Xử lý 50 nghìn lượt tải lên (tiết kiệm \$100)
- Không có worker riêng → Dùng hàm Lambda (tiết kiệm \$250)
- Redis nhỏ hơn → ElastiCache t3.micro (tiết kiệm \$50)

Tổng: \$470/tháng

Đánh đổi:

Độ trễ toàn cầu chậm hơn (không có CDN)

Kém bền bỉ hơn (chỉ một máy chủ API)

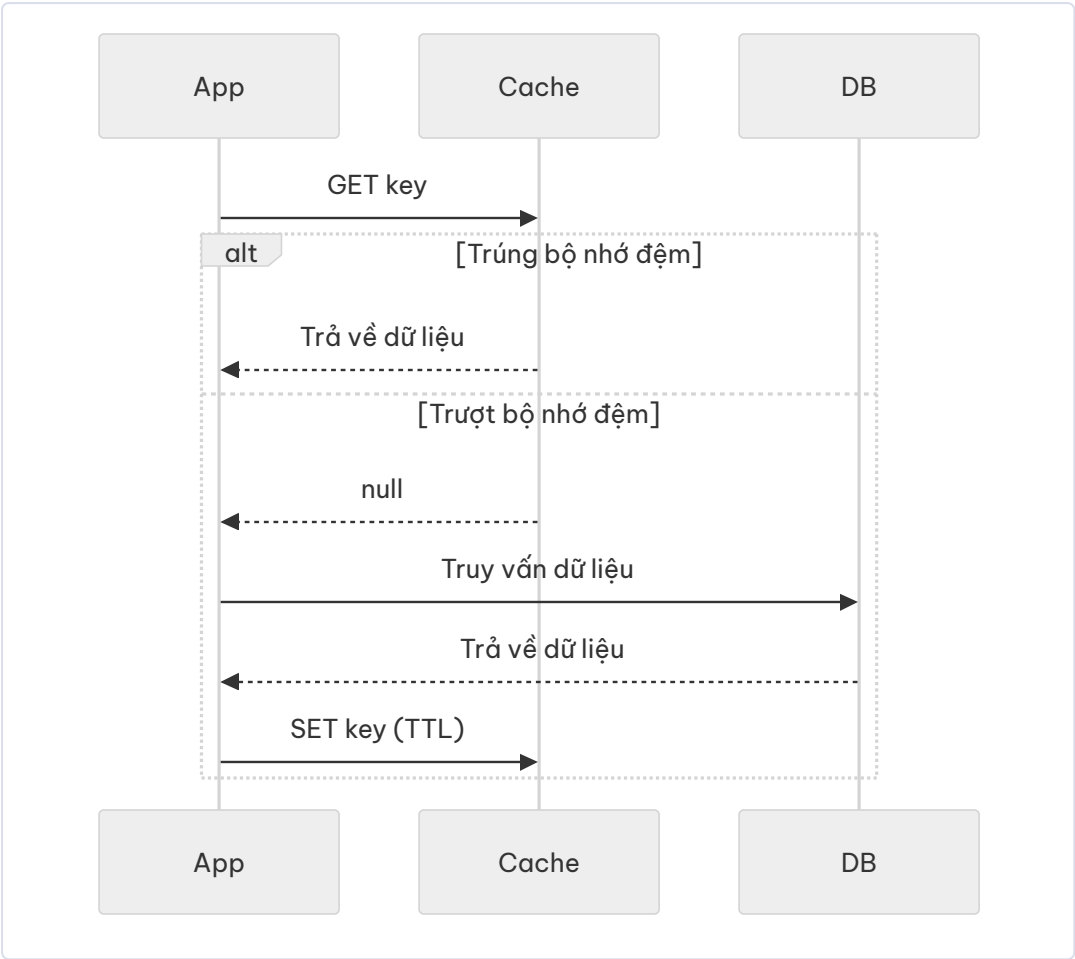
Vẫn mở rộng được (Lambda tự động mở rộng)

Vẫn hoạt động tốt

Bài học chính: Kiến trúc thích nghi theo các ràng buộc.

Các Mẫu (Patterns) Tương Tác Giữa Các Thành Phần (Component Interaction)

Mẫu (Pattern) 1: Cache-Aside (Bộ Nhớ Đặt Bên Cạnh)

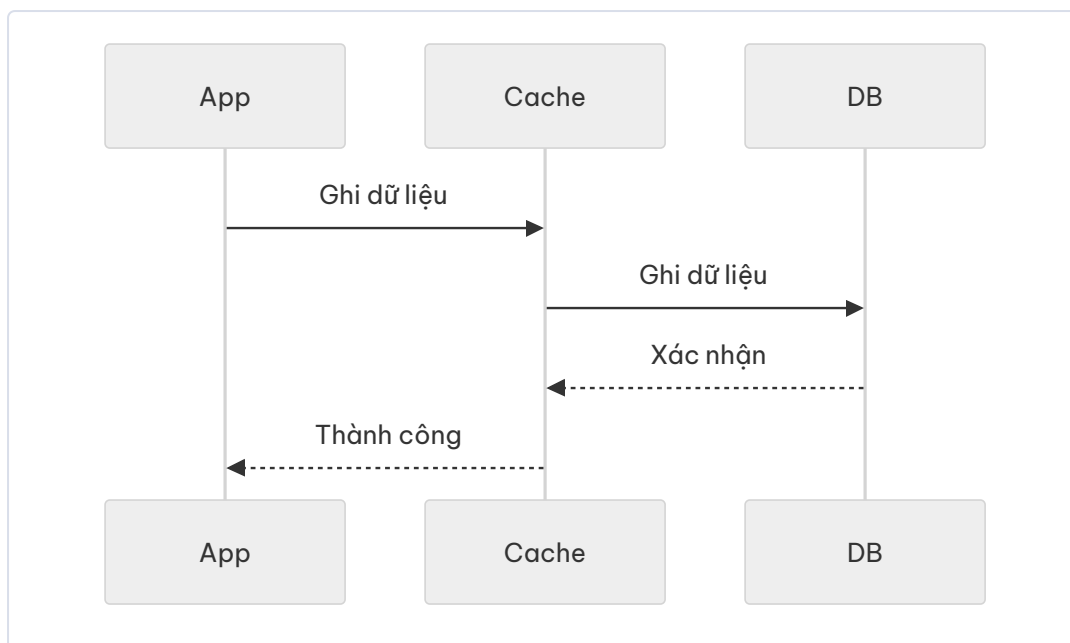


Mẫu cache-aside: Ứng dụng tự quản lý logic bộ nhớ đệm

Khi nào nên dùng:

- ✓ Khối lượng công việc thiên về đọc
- ✓ Dữ liệu không thay đổi thường xuyên
- ✓ Ứng dụng có thể tự xử lý logic bộ nhớ đệm

Mẫu 2: Write-Through Cache (Ghi Xuyên Qua Bộ Nhớ Đệm)

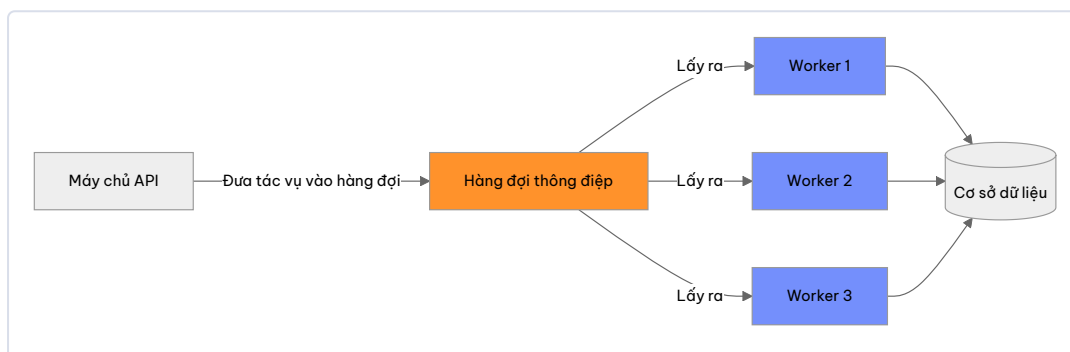


Write-through: Mọi thao tác ghi đều đi qua bộ nhớ đệm, bộ nhớ đệm cập nhật cơ sở dữ liệu (DB)

Khi nào nên dùng:

- ✓ Độ tươi mới của dữ liệu là cực kỳ quan trọng
- ✓ Đọc >> Ghi
- ✓ Chấp nhận được việc ghi chậm hơn

Mẫu 3: Mẫu Hàng Đợi – Worker (Queue-Worker Pattern)

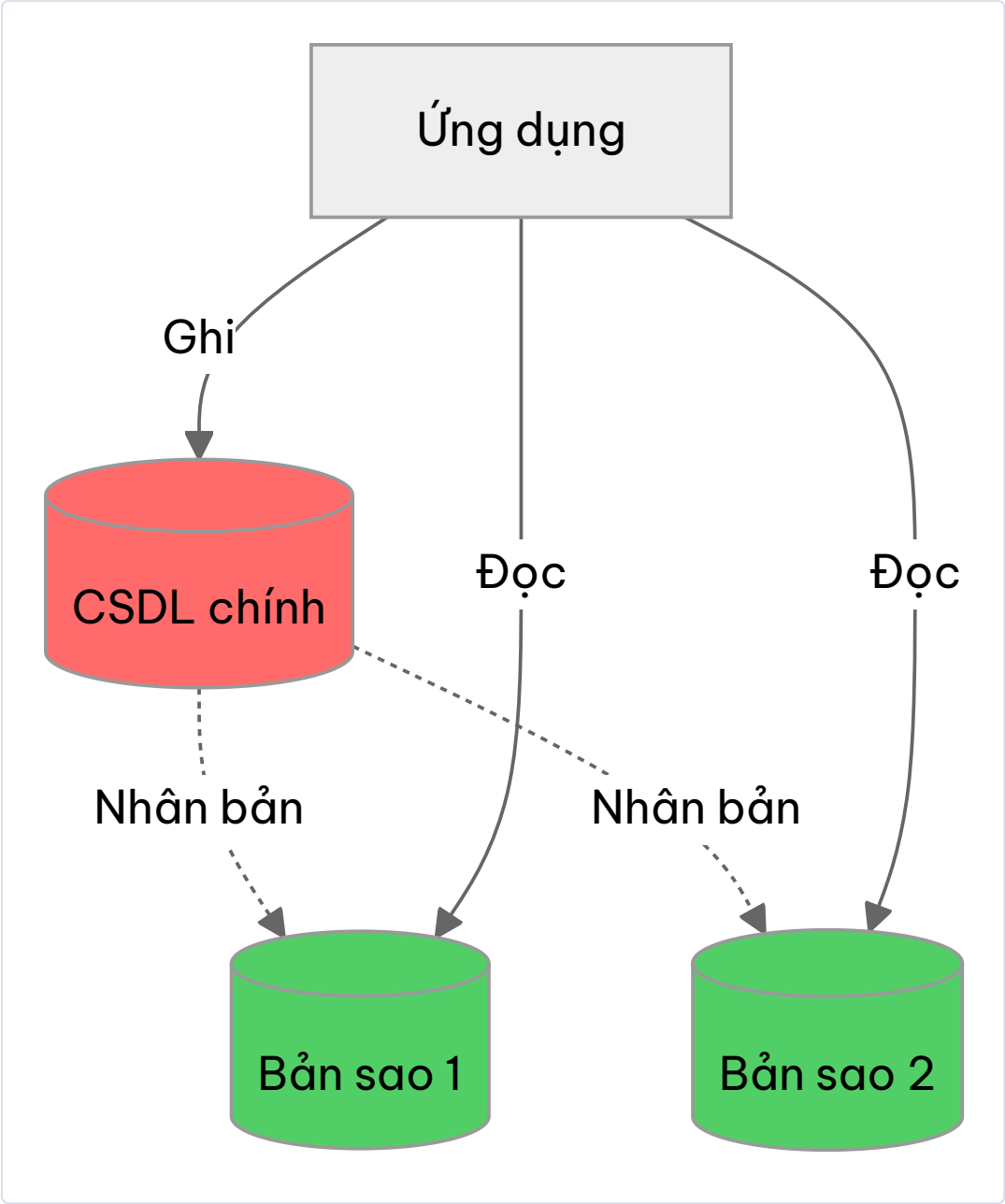


Hàng đợi – worker (Queue-worker): Xử lý bất đồng bộ kèm thử lại (retry) và mở rộng (scaling)

Khi nào nên dùng:

- ✓ Xử lý mất > 5 giây
- ✓ Người dùng không cần kết quả ngay lập tức
- ✓ Cần logic thư' lại
- ✓ Lưu lượng hay tăng đột biến

Mẫu 4: Mẫu Bản Sao Chỉ Đọc (*Read Replica Pattern*)



Bản sao chỉ đọc (*Read replicas*): Thao tác ghi đi vào máy chủ chính (*master*), thao tác đọc lấy từ các bản sao (*slaves*)

Khi nào nên dùng:

- ✓ Đọc >> Ghi (70%+ là đọc)
- ✓ Độ trễ nhận bản chấp nhận được (< 1 giây)
- ✓ Một máy chủ chính duy nhất xử lý được lượng ghi

Khung Ra Quyết Định (Decision Framework): Lựa Chọn Thành Phần

Ma Trận Khung Tư Duy

Với mỗi thành phần, hãy đánh giá:

- Vấn đề nó giải quyết:
 - Điểm nghẽn cụ thể
 - Tác động đo lường được
 - Không phải kiểu "có thì tốt"
- Chi phí (tổng thể):
 - Hạ tầng (\$)
 - Thời gian phát triển
 - Gánh nặng bảo trì
 - Độ phức tạp vận hành
- Phương án thay thế:
 - Có giải pháp đơn giản hơn không?
 - Có thể tối ưu cái đang có không?
 - Có bắt buộc phải thêm ngay không?
- Rủi ro:
 - Công nghệ mới?
 - Nhóm có chuyên môn không?
 - Các kiểu sự cố có thể xảy ra?

Quyết định: Chỉ thêm khi Vấn đề > Chi phí + Rủi ro

Các Quyết Định Ví Dụ

Quyết định 1: Có thêm bộ nhớ đệm hay không?

Vấn đề:

- Truy vấn cơ sở dữ liệu chậm (500ms)
- Thiên về đọc (90% là đọc)
- 1000 req/s

Đánh giá giải pháp:

Phương án A: Thêm bộ nhớ đệm Redis

Giảm thời gian truy vấn 500ms → 10ms

Giảm 90% tải cho CSDL

Chi phí \$100/tháng

Độ phức tạp của việc vô hiệu hoá bộ nhớ đệm

Thêm một thứ cần giám sát

Phương án B: Tối ưu truy vấn + chỉ mục

Miễn phí

Không thêm hạ tầng mới

Cải thiện lâu dài

Tốn thời gian đầu tư (1 tuần)

Có thể chưa đủ

Quyết định: Thử Phương án B trước

- Nếu sau khi tối ưu truy vấn vẫn > 100ms
- Khi đó mới thêm bộ nhớ đệm

Quyết định 2: Hàng đợi hay Lambda?

Vấn đề:

- Xử lý ảnh mất 10 giây
- 1000 lượt tải lên/ngày
- Người dùng không thể chờ

Đánh giá giải pháp:

Phương án A: SQS + worker trên EC2

Toàn quyền kiểm soát

Chi phí dự đoán được

Phải tự quản lý máy chủ

Chi phí nền \$150/tháng

Phương án B: Hàm Lambda

Tự động mở rộng

Trả tiền theo mức sử dụng (~\$20/tháng cho 1000 lượt tải lên)

Không phải quản lý máy chủ

Giới hạn thời gian chờ 15 phút

Độ trễ khởi động nguội (cold start)

Bối cảnh: 1000 lượt tải lên/ngày, xử lý mất 10 giây

Quyết định: Lambda

- Rẻ hơn 7,5 lần
- Có sẵn tự động mở rộng
- 10 giây thấp hơn nhiều so với giới hạn 15 phút
- Nhóm không phải quản lý máy chủ

Quyết định 3: Có dùng CDN hay không?

Vấn đề:

- Ảnh tải chậm với người dùng quốc tế
- 10 nghìn người dùng hằng ngày
- 50% ở Mỹ, 50% ở châu Á

Đánh giá giải pháp:

Phương án A: Thêm CloudFront CDN

Độ trễ thấp trên toàn cầu

Giảm chi phí S3

\$100/tháng

Thiết lập phức tạp

Phương án B: S3 Transfer Acceleration

Nhanh hơn S3 thông thường

Chỉ \$5/tháng

Không nhanh bằng CDN

Vẫn còn độ trễ từ máy chủ gốc

Phương án C: Không làm gì

Miễn phí

Trải nghiệm kém cho người dùng châu Á

Số liệu cần có:

- Đo độ trễ thực tế Mỹ so với châu Á
- Nếu châu Á > 2 giây → Thêm CDN
- Nếu châu Á 500ms-1 giây → Transfer Acceleration
- Nếu châu Á < 500ms → Không làm gì

Quyết định: Dựa trên dữ liệu, không dựa trên giả định

Những Sai Lầm Tích Hợp Thường Gặp

Sai Lầm 1: Thêm Thành Phần "Để Phòng Hờ"

Sai:

"Thêm bộ nhớ đệm Redis ngay từ ngày đầu đi"

"Sau này có thể mình sẽ cần"

Đúng:

"Truy vấn cơ sở dữ liệu trung bình mất 50ms"

"Với 1000 người dùng, thế là đủ nhanh"

"Thêm bộ nhớ đệm khi truy vấn > 200ms hoặc lưu lượng > 10 nghìn người dùng"

Nguyên tắc: YAGNI (You Aren't Gonna Need It - Bạn sẽ không cần đến nó đâu)

Sai Lầm 2: Sao Chép Nguyên Xi Kiến Trúc Của Netflix

Sai:

Công ty khởi nghiệp với 100 người dùng:

- 50 vi dịch vụ
- Cụm Kubernetes
- Service mesh
- Event sourcing
- CQRS

"Netflix làm thế mà!"

Đúng:

Công ty khởi nghiệp với 100 người dùng:

- Hệ thống nguyên khối
- PostgreSQL
- 1 máy chủ
- Phát hành nhanh

Bối cảnh rất quan trọng. Netflix có 200 triệu người dùng, 500 nhóm. Bạn không phải Netflix.

Sai Lầm 3: Tối Ưu Hoá Sớm (*Premature Optimization*)

Sai:

"Phân mảnh cơ sở dữ liệu ngay từ ngày đầu đi"

"Đăng nào sau này cũng phải mở rộng"

Đúng:

"Chúng ta có 1000 người dùng"

"Một PostgreSQL duy nhất xử lý được 100 nghìn người dùng"

"Chúng ta sẽ phân mảnh khi thực sự chạm giới hạn"

Nguyên tắc: Tối ưu cho HIỆN TẠI, không phải cho một tương lai giả định

Sai Lầm 4: Phớt Lờ Gánh Nặng Vận Hành

Tư duy sai:

"Kiến trúc này hoàn hảo về mặt kỹ thuật!"

Đối chiếu thực tế:

- 10 thành phần cần giám sát
 - 5 công nghệ khác nhau
 - Nhóm 3 người
 - Không ai có kinh nghiệm với một nửa số công nghệ
- Những cơn ác mộng trên môi trường thực tế

Tư duy đúng:

"Kiến trúc này đủ đơn giản để nhóm bảo trì"

"Chúng ta hiểu rõ mọi thành phần"

"Thực sự cố sẽ không phải là địa ngục"

Sai Lầm 5: Không Đo Lường

Sai:

Thêm bộ nhớ đệm → "Cảm giác nhanh hơn"

Thêm hàng đợi → "Chắc sẽ giúp chịu được lúc tăng đột biến"

Đúng:

Trước khi có bộ nhớ đệm:

- Thời gian truy vấn trung bình: 500ms
- P95: 2 giây
- CPU cơ sở dữ liệu: 80%

Sau khi có bộ nhớ đệm:

- Thời gian truy vấn trung bình: 50ms (nhanh hơn 10 lần)
- P95: 150ms (nhanh hơn 13 lần)
- CPU cơ sở dữ liệu: 20% (giảm 4 lần)
- Tỷ lệ trúng bộ nhớ đệm: 85%

Đo lường → Chứng minh tác động

Những Điểm Cốt Lõi

Tư duy tích hợp > Kiến thức về từng thành phần:

Biết từng thành phần: Điều kiện cần

Biết kết hợp các thành phần: Điều kiện đủ

Biết các đánh đổi: Thành thạo

Kiến trúc mang tính tiến hoá:

V1: Hệ thống nguyên khối đơn giản

↓ (khi chậm)

V2: Thêm bộ nhớ đệm

↓ (khi lưu lượng cao)

V3: Thêm bộ cân bằng tải

↓ (khi thao tác ghi quá tải)

V4: Thêm bản sao chỉ đọc

↓ (khi cân bất đồng bộ)

V5: Thêm hàng đợi + worker

Mỗi bước là phản ứng trước một vấn đề thực sự

Không phải phỏng đoán

Khung lựa chọn thành phần:

1. Vấn đề là gì? (cụ thể, đo lường được)
2. Chi phí là gì? (tiền bạc, độ phức tạp, bảo trì)
3. Phương án thay thế là gì? (giải pháp đơn giản hơn?)
4. Có thể chờ không? (thêm sau khi cân?)

Chỉ thêm thành phần khi: Vấn đề > Chi phí + Rủi ro

Những sự đánh đổi thực tế mới là điều quan trọng:

Đánh đổi kỹ thuật: Hiệu năng, tính nhất quán, tính sẵn sàng

Đánh đổi kinh doanh: Chi phí, thời gian đưa ra thị trường

Đánh đổi về nhóm: Chuyên môn, gánh nặng bảo trì

Đánh đổi về người dùng: Trải nghiệm, độ tin cậy

Kiến trúc sư giỏi cân bằng TẤT CẢ

Không chỉ riêng kỹ thuật

Bắt đầu đơn giản, tiến hoá dựa trên dữ liệu:

Đừng: Thiết kế cho 1 triệu người dùng khi bạn mới có 100

Hãy: Thiết kế cho quy mô hiện tại + mức tăng trưởng 2-3 lần

Hãy: Đo lường các điểm nghẽn thực tế

Hãy: Tiến hoá kiến trúc từng bước một

"Tôi ưu hoá sớm là cội rễ của mọi tội lỗi"

- Donald Knuth

Bối cảnh là tất cả:

Kiến trúc của Netflix $\neq$ Kiến trúc của bạn
Quy mô của Google $\neq$ Quy mô của bạn
Thực hành tốt nhất $\neq$ Tốt nhất cho BẠN

Hiểu bối cảnh:

- Quy mô hiện tại
- Tốc độ tăng trưởng
- Quy mô nhóm
- Ngân sách
- Thời hạn
- Chuyên môn

Rồi mới thiết kế cho phù hợp

Danh sách kiểm tra trước khi thêm thành phần:

- ☐ Có vấn đề cụ thể kèm số liệu đo không?
- ☐ Đã thu' tối ưu các thành phần hiện có chưa?
- ☐ Thành phần này có giải quyết được vấn đề đó không?
- ☐ Chi phí (thời gian + tiền bạc + độ phức tạp) có đáng không?
- ☐ Nhóm có đủ chuyên môn đề' bảo trì không?
- ☐ Có thể' thêm sau được không?

Nếu không vượt qua danh sách kiểm tra $\rightarrow$ Đừng thêm

Lời khuyên cuối cùng:

Thiết kế hệ thống không phải là dùng công nghệ mới nhất hay phức tạp nhất.

Nó là về **giải quyết các vấn đề thực sự bằng những giải pháp phù hợp.**

Giải pháp đơn giản cho vấn đề đơn giản.

Giải pháp phức tạp chỉ khi thực sự cần.

"Làm cho nó chạy, làm cho nó đúng, làm cho nó nhanh - theo đúng thứ tự đó."

Làm chủ tư duy tích hợp, và bạn sẽ thiết kế được những hệ thống mở rộng (*scale*) cho hàng triệu người dùng.

Bài tiếp theo: Phần 3 - Nền Tảng Hệ Thống Phân Tán (*Distributed Systems*): Tính Nhất Quán (*Consistency*), Đồng Thuận (*Consensus*) và Giao Dịch Phân Tán (*Distributed Transactions*).

PHẦN 3

Nền Tảng Hệ Thống Phân Tán

Đi sâu vào thực tế của hệ phân tán: sự cố, tính nhất quán, phối hợp, toàn vẹn dữ liệu và thứ tự sự kiện.

1) Giới Thiệu Phần

Hệ thống phân tán (*Distributed systems*) khác về bản chất so với hệ thống chạy trên một máy đơn lẻ.

Trong môi trường phân tán, sự cố (*failure*) là trạng thái bình thường: nút (*node*) có thể chết, mạng có thể bị phân vùng (*partition*), và dữ liệu có thể tạm thời lệch trạng thái.

Vì vậy, tính nhất quán (*consistency*) và sự phối hợp (*coordination*) trở thành bài toán trung tâm. Đây là cột mốc kỹ thuật quan trọng để bạn chuyển từ "xây dựng dịch vụ (*service*)" sang "thiết kế hệ thống phân tán".

2) Vì Sao Phần Này Tồn Tại

Tư duy một máy chủ (*single-server*) là không đủ khi hệ thống mở rộng.

Bạn cần hiểu rõ:

- Sự cố cục bộ (*Partial failure*) là thực tế không thể tránh khỏi.
- Mạng không bao giờ đáng tin cậy tuyệt đối.
- Mọi bảo đảm nhất quán (*consistency guarantee*) luôn đi kèm sự đánh đổi (*trade-off*).
- Dữ liệu trên nhiều nút cần cơ chế phối hợp rõ ràng.

Nếu không có tư duy phân tán (*distributed reasoning*), rất khó thiết kế được hệ thống hiện đại có độ tin cậy cao.

3) Mục Tiêu Học Tập

- Hiểu các mô hình sự cố (*failure model*) trong hệ thống phân tán.
- Hiểu tư duy CAP ở mức ra quyết định kiến trúc.
- Hiểu sự đánh đổi giữa tính nhất quán và tính sẵn sàng (*availability*).
- Hiểu các cơ chế phối hợp trong hệ phân tán (*distributed system*).

- Hiểu bài toán toàn vẹn dữ liệu (*data integrity*) khi một giao dịch đi qua nhiều thành phần.
- Hiểu vấn đề định danh (*identity*) và thứ tự (*ordering*) trong môi trường nhiều nút (*multi-node*).

4) Bạn Sẽ Học Gì Trong Phần Này

- **Xử lý sự cố** (*Failure handling*): cách tư duy và phản ứng với lỗi trong môi trường phân tán.
- **Đánh đổi** (*Trade-offs*) **về tính nhất quán**: khi nào ưu tiên nhất quán mạnh (*strong consistency*), khi nào chấp nhận nhất quán cuối cùng (*eventual*).
- **Phối hợp phân tán** (*Distributed coordination*): cơ chế để nhiều nút đạt đồng thuận (*consensus*) trong điều kiện không hoàn hảo.
- **Bầu chọn nút chủ** (*Leader election*): vai trò của nút chủ (*leader*)/nút theo (*follower*) và các giới hạn khi cuộc bầu chọn xảy ra.
- **Giao dịch phân tán** (*Distributed transactions*): cách giữ tính nhất quán dữ liệu xuyên suốt nhiều dịch vụ/kho dữ liệu.
- **Toàn vẹn toàn hệ thống** (*System-wide integrity*): bảo đảm dữ liệu đúng ở quy mô toàn hệ thống, không chỉ trong từng dịch vụ.
- **ID phân tán**: tạo định danh duy nhất mà không trở thành điểm nghẽn (*bottleneck*).
- **Thứ tự sự kiện** (*Event ordering*): xử lý thứ tự sự kiện trong hệ thống phân tán và bất đồng bộ.

5) Tổng Quan Các Bài Học

Bài 1 – Thực Tế Của Hệ Thống Phân Tán – Mô Hình Sự Cố (*Failure Models*) & Tư Duy CAP

- **Mô tả**: nhìn thẳng vào bản chất lỗi và giới hạn của hệ phân tán.
- **Bạn nhận được**: nền tư duy CAP và tư duy đặt sự cố lên đầu (*failure-first*) để ra quyết định thực tế.

Bài 2 – Mô Hình Nhất Quán (Consistency Models) – Nhất Quán Mạnh Và Nhất Quán Cuối Cùng (Eventual Consistency)

- **Mô tả:** so sánh các mức tính nhất quán trong bối cảnh sản phẩm thật.
- **Bạn nhận được:** khả năng chọn mô hình nhất quán (*consistency model*) phù hợp với yêu cầu nghiệp vụ.

Bài 3 – Đồng Thuận & Bầu Chọn Nút Chủ – Thoả Thuận Trong Hệ Phân Tán

- **Mô tả:** cơ chế đồng thuận và bầu chọn nút chủ để điều phối trạng thái hệ thống.
- **Bạn nhận được:** trực giác về thoả thuận phân tán (*distributed agreement*) và giới hạn của sự phối hợp.

Bài 4 – Giao Dịch Phân Tán & Toàn Vẹn Dữ Liệu – Bảo Đảm Tính Nhất Quán Dữ Liệu

- **Mô tả:** giải bài toán nhất quán dữ liệu khi giao dịch đi qua nhiều dịch vụ.
- **Bạn nhận được:** nền tảng để đánh giá chiến lược giao dịch (*transaction strategy*) và các bảo đảm toàn vẹn (*integrity guarantees*).

Bài 5 – ID Phân Tán & Thứ Tự – Giải Quyết Vấn Đề Định Danh Trong Hệ Thống Phân Tán

- **Mô tả:** xử lý việc sinh ID (*ID generation*) và thứ tự khi hệ thống không có một đồng hồ (*clock*) trung tâm.
- **Bạn nhận được:** khả năng thiết kế định danh và thứ tự sự kiện ở quy mô phân tán.

6) Kết Quả Sau Khi Hoàn Thành Phần 3

Bạn có thể:

- Lập luận về các sự đánh đổi cốt lõi trong hệ thống phân tán.
- Nhận diện rủi ro trong môi trường vận hành thực tế (*production*) liên quan đến sự cố và tính nhất quán.
- Đánh giá chiến lược phối hợp theo bối cảnh hệ thống.

- Thiết kế hệ thống với ý thức về sự cố (*failure awareness*) ngay từ đầu.

7) Chuẩn Bị Cho Phần 4

Phần 3 cho bạn nền tảng để hiểu vì sao hệ phân tán khó và đòi hỏi cân nhắc đánh đổi nghiêm túc.

Sang **Phần 4 — Khả Năng Mở Rộng & Hiệu Năng**, bạn sẽ tập trung vào:

- tối ưu hiệu năng (*performance*) ở quy mô lớn,
- các chiến lược mở rộng (*scaling*) nâng cao,
- tư duy tìm điểm nghẽn ở cấp hệ thống,
- và khả năng quan sát (*observability*) để vận hành ổn định trong môi trường thực tế.

Thực Tế Của Hệ Thống Phân Tán (Distributed Systems): Mô Hình Sự Cố (Failure Models) Và Tư Duy CAP

Tôi còn nhớ lần đầu tiên hệ thống vận hành thực tế (*production*) của tôi gặp phân vùng mạng (*network partition*).

2 giờ sáng. Cảnh báo (*Alert*) giám sát (*monitoring*) réo inh ỏi. Một phần người dùng (*users*) ở Mỹ không đăng nhập được. Một phần người dùng ở châu Âu vẫn đăng nhập bình thường. Cơ sở dữ liệu (*Database*) vẫn khoẻ mạnh. Các máy chủ ứng dụng (*application servers*) vẫn đang chạy.

Tôi hoàn toàn hoang mang. Kiểm tra log (*logs*), không thấy lỗi nào. Kiểm tra CPU, bộ nhớ, tất cả đều bình thường.

Một kiến trúc sư lâu năm (*senior architect*) vào trực tuyến: "Kiểm tra kết nối mạng giữa các trung tâm dữ liệu (*datacenters*) ở Mỹ và châu Âu đi."

Chuẩn rồi. AWS gặp sự cố mạng. Trung tâm dữ liệu (*Datacenter*) ở Mỹ không kết nối được với trung tâm dữ liệu ở châu Âu. Hệ thống đang ở trạng thái **phân vùng mạng**.

Đó là lúc tôi học được bài học đắt giá nhất: **Hệ thống phân tán thì luôn luôn gặp sự cố. Không phải "nếu", mà là "khi nào".**

Tại Sao Hệ Thống Phân Tán Lại Khác Biệt?

Một Máy Đơn Lẻ Và Hệ Thống Phân Tán: Những Giả Định (*Assumptions*) Bị Phá Vỡ

Một máy đơn lẻ (mã chạy cục bộ):

```
# Local function call
result = calculate_total(items)

Assumptions:
Function luôn return (hoặc throw exception)
Execution time predictable
Data consistent
Either success or fail (no in-between)
```

Hệ thống phân tán (*Distributed system*) (lời gọi qua mạng):

```
# Remote API call
result = api.calculate_total(items)

Reality:
Request có thể 'lost' (no response)
Response có thể 'timeout' (slow network)
Request có thể 'duplicate' (retry)
Partial success/failure possible
Data có thể 'inconsistent' across nodes
```

Điểm mấu chốt: Khi chuyển từ một máy đơn lẻ sang hệ thống phân tán, mọi giả định (*assumption*) bạn từng biết đều trở nên sai.

Những Ngộ Nhận Về Tính Toán Phân Tán (The Fallacies of Distributed Computing)

Peter Deutsch và James Gosling đã tổng kết 8 giả định sai lầm mà các lập trình viên (*developers*) thường mắc phải:

1. Mạng là đáng tin cậy
→ Thực tế: Gói tin bị rơi, kết nối bị lỗi
2. Độ trễ bằng không
→ Thực tế: Lờn gọi qua mạng mất 10-100ms
3. Băng thông là vô hạn
→ Thực tế: Mạng có giới hạn thông lượng
4. Mạng là an toàn
→ Thực tế: Dữ liệu có thể bị chặn bắt (intercept)
5. Cấu trúc mạng (topology) không thay đổi
→ Thực tế: Máy chủ chết, mạng được cấu hình lại
6. Chỉ có một người quản trị
→ Thực tế: Nhiều nhóm, nhiều cấu hình
7. Chi phí truyền tải bằng không
→ Thực tế: Tồn chi phí tuàn tự hoá và truyền qua mạng
8. Mạng là đồng nhất
→ Thực tế: Khác giao thức, khác phiên bản

Bài học: Thiết kế hệ thống phân tán = Thiết kế để đối phó với sự cố (failure).

Định Luật Murphy Trong Hệ Thống Phân Tán

Định luật Murphy: "Cái gì có thể hỏng thì sẽ hỏng."

Trong hệ thống phân tán, định luật này còn được khuếch đại lên:

"Mọi thứ có thể hỏng THÌ SẼ hỏng. Cùng một lúc. Lúc 3 giờ sáng. Vào đúng ngày Black Friday."

Những Sự Cố (Failures) Thực Tế Tôi Đã Trải Qua

Sự cố 1: Hỏng Dữ Liệu Thầm Lặng (Silent Data Corruption)

Kịch bản:

- Cơ sở dữ liệu nhân bản từ Master → Slave
- Gói tin mạng bị hỏng (1 trên 10 triệu)
- Slave nhận dữ liệu bị hỏng
- Ứng dụng đọc từ Slave
- Trả dữ liệu sai cho người dùng (lỗi khó phát hiện)

Tác động: 3 ngày đề'phát hiện, 1 tuàn đề'khắc phục

Sự cố 2: Hết Thời Gian Chờ Dây Chuyền (Cascading Timeout)

Kịch bản:

- Dịch vụ A gọi Dịch vụ B (timeout 30 giây)
- Dịch vụ B bị chậm (sự cố ổ đĩa)
- Dịch vụ A đợi 30 giây cho mỗi yêu cầu
- Các yêu cầu dồn ứ trong hàng đợi
- Dịch vụ A cạn kiệt bộ nhớ
- Dịch vụ A bị sập
- Các dịch vụ phụ thuộc vào A cũng sập theo

Tác động: Toàn bộ hệ thống ngừng hoạt động 2 giờ

Sự cố 3: Lệch Đồng Hồ (Clock Skew)

Kịch bản:

- Đồng hồ Máy chủ 1: 10:00:00
- Đồng hồ Máy chủ 2: 09:59:30 (chậm 30 giây)
- Sự kiện A xảy ra trên Máy chủ 1 lúc 10:00:00
- Sự kiện B xảy ra trên Máy chủ 2 lúc 09:59:45
- Hệ thống cho rằng B xảy ra trước A (sai!)

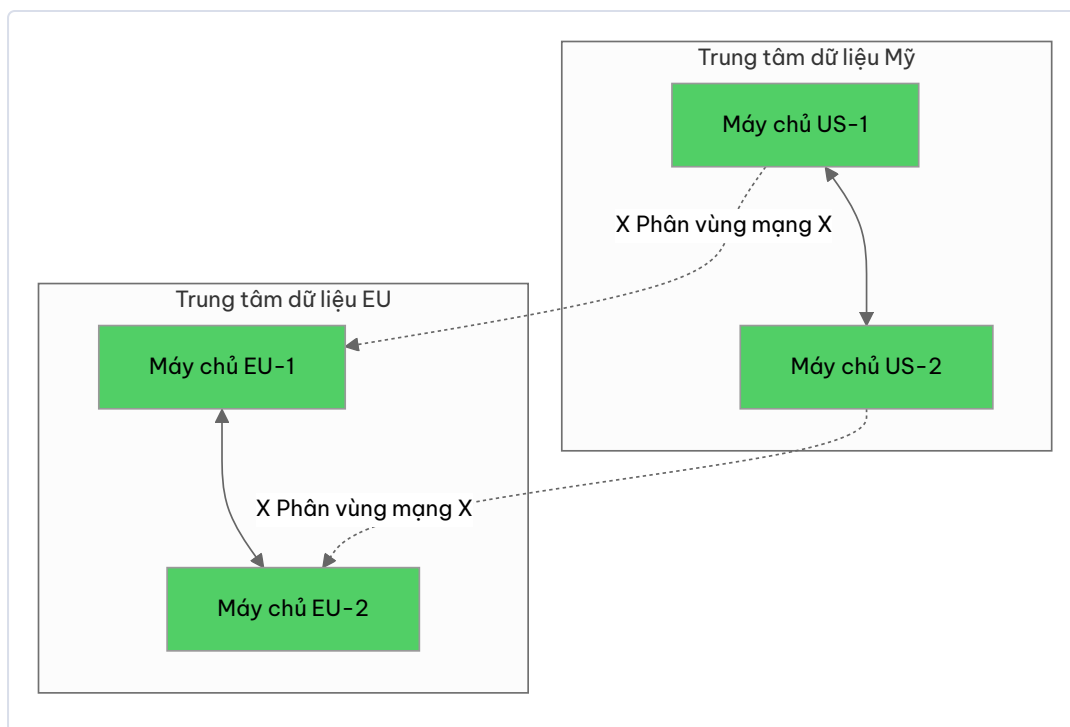
Tác động: Thứ tự giao dịch bị sai lệch

Bài học rút ra: Không có kịch bản (*scenario*) sự cố nào là "quá kỳ quặc". Tất cả đều xảy ra trong môi trường thực tế.

Các Loại Sự Cố Trong Hệ Thống Phân Tán

Loại Sự Cố 1: Phân Vùng Mạng

Định nghĩa: Một phần hệ thống không giao tiếp được với phần còn lại, nhưng cả hai vẫn đang chạy.



Phân vùng mạng: các trung tâm dữ liệu ở Mỹ và châu Âu không kết nối được với nhau, nhưng các máy chủ (servers) trong mỗi trung tâm dữ liệu vẫn hoạt động bình thường

Kịch bản thực tế:

Dòng thời gian:

10:00:00 - Phân vùng mạng xảy ra
10:00:01 - Người dùng A (Mỹ) cập nhật hồ sơ
10:00:02 - Người dùng B (EU) đọc hồ sơ → Thấy dữ liệu cũ
10:00:30 - Mạng được khôi phục
10:00:31 - Trung tâm dữ liệu EU nhận được bản cập nhật
10:00:32 - Người dùng B đọc lại → Thấy dữ liệu mới

Vấn đề: 30 giây không nhất quán

Tại sao phân vùng (partition) lại xảy ra?

Nguyên nhân phổ biến:

- Thiết bị mạng hỏng (switch, router)
- Đứt cáp quang
- Sự cố theo vùng của AWS/GCP
- Tấn công DDoS
- Cấu hình sai
- Quá nhiều lưu lượng (tắc nghẽn)

Không thể ngăn chặn, chỉ có thể xử lý:

```
# Assume network luôn available
def update_user(user_id, data):
    primary_db.write(data)
    replica_db.write(data) # Assume này luôn succeed

# Handle partition
def update_user(user_id, data):
    try:
        primary_db.write(data)
    except WriteError:
        log_error("Primary DB unavailable")
        queue_for_retry(data)

    try:
        replica_db.write(data)
    except WriteError:
        # Replica unavailable, but primary succeeded
        # System vẫn functional (eventual consistency)
        log_warning("Replica unavailable, will retry")
```

Loại Sự Cố 2: Độ Trễ (Latency) Cao

Định nghĩa: Lỗi gọi qua mạng chậm hơn dự kiến, nhưng cuối cùng vẫn hoàn thành.

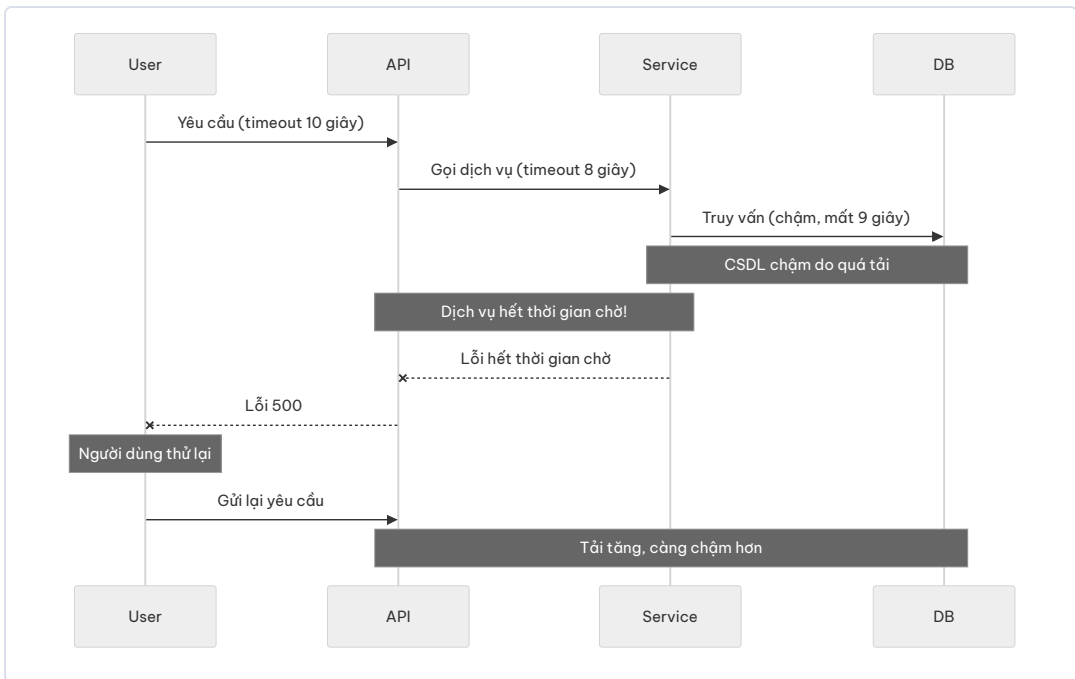
Vấn đề: Chậm ≠ Chết. Hệ thống vẫn hoạt động nhưng trải nghiệm người dùng (UX) tệ hại.

Độ trễ~bình thường: 50ms
Độ trễ~cao: 5000ms (chậm hơn 100 lần)

Trải nghiệm người dùng:

- Bấm nút
- Đợi 5 giây
- Vẫn đang tải...
- Bấm lại (yêu cầu trùng lặp!)
- Đợi thêm...
- Bỏ cuộc, đóng tab

Hiệu ứng dây chuyền:



Độ trễ cao gây hết thời gian chờ (timeout), hết thời gian chờ gây thử lại (retry), thử lại làm tăng tải (load), tải tăng làm hệ thống chậm hơn nữa → Vòng xoáy tử thần (Death spiral)

Giải pháp:

```

# Solution 1: Timeouts với reasonable values
response = requests.get(url, timeout=2) # Fail fast

# Solution 2: Circuit breaker
class CircuitBreaker:
    def call(self, func):
        if self.failure_rate > threshold:
            raise CircuitOpenError("Too many failures")

        try:
            return func()
        except TimeoutError:
            self.record_failure()

# Solution 3: Bulkhead pattern (isolate resources)
thread_pool_for_service_a = ThreadPool(10)
thread_pool_for_service_b = ThreadPool(10)
# Service A slow không ảnh hưởng Service B

```


Loại Sự Cố 3: Mất Gói Tin (*Packet Loss*)

Định nghĩa: Các gói tin mạng không đến được đích.

```
Gửi:      [Gói tin 1] [Gói tin 2] [Gói tin 3] [Gói tin 4]
Nhận:      [Gói tin 1] [          ] [Gói tin 3] [Gói tin 4]
                ↑ Mất!
```

Tỷ lệ mất gói tin điển hình:

LAN (mạng cục bộ):	0.01% - 0.1%
WiFi:	1% - 5%
Internet (tốt):	0.1% - 1%
Internet (kém):	5% - 20%

Tác động:

```
# Request bị lost
client.send(request)
# ... no response ...
# Client không biết: request lost? or response lost?

# TCP retransmit automatically (but slow)
# UDP không retry (faster but unreliable)
```

Xử lý ở tầng ứng dụng (*application-level*):

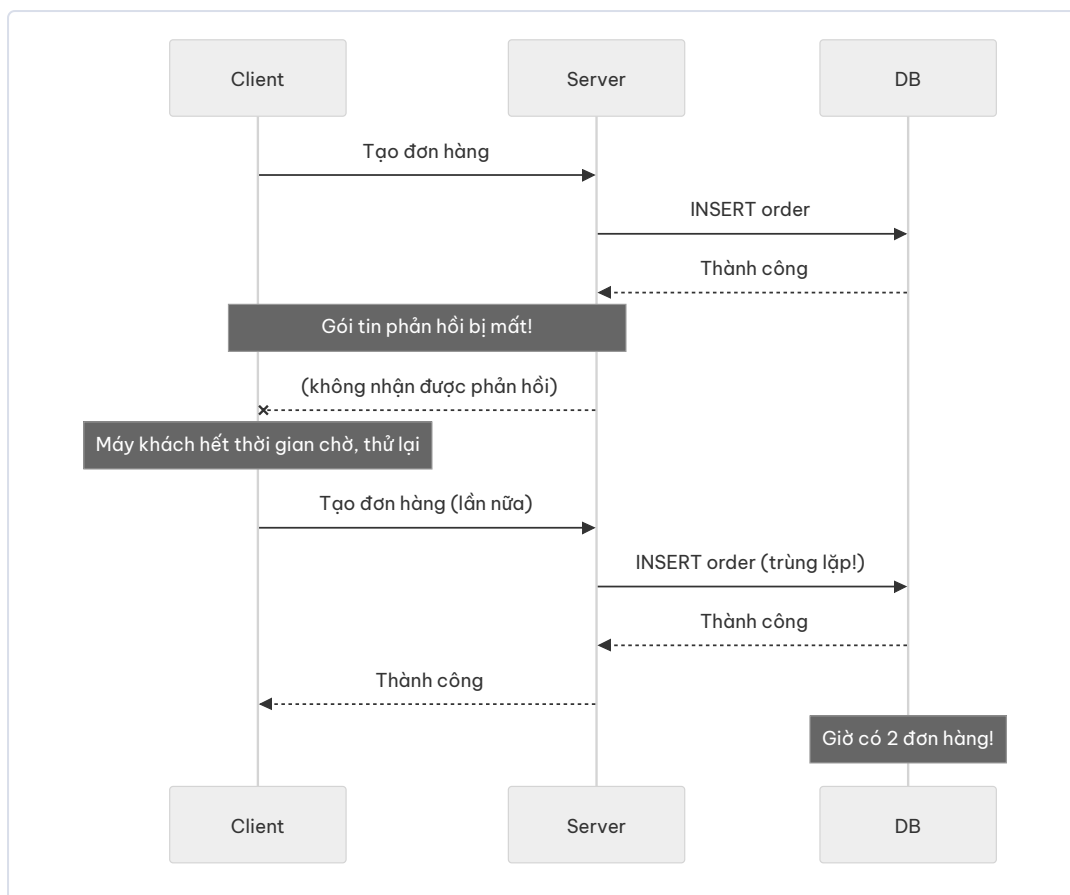
```
# Idempotent operations
def create_user(user_id, data):
    # Use user_id (client-generated UUID)
    # Retry-safe: create twice = same result
    db.insert_or_update(user_id, data)

# Non-idempotent (dangerous)
def increment_counter():
    counter += 1
    # Retry = wrong result!
```

Loại Sự Cố 4: Trùng Lặp Thông điệp (*Message Duplication*)

Định nghĩa: Cùng một thông điệp được gửi đến nhiều lần.

Kịch bản:



Mất gói tin trên mạng gây ra thông điệp trùng lặp khi máy khách (client) thử lại

Tác động trong thực tế:

Kịch bản: Xử lý thanh toán

1. Người dùng bấm "Thanh toán \$100"
2. Yêu cầu hết thời gian chờ
3. Người dùng bấm lại
4. Cả hai yêu cầu đều thành công
5. Người dùng bị trừ \$200!

🚩 Công ty bị kiện, phải hoàn tiền, mang tiếng xấu

Giải pháp:

```
# Solution 1: Idempotency key
def process_payment(amount, idempotency_key):
    if cache.exists(idempotency_key):
        return cache.get(idempotency_key) # Return cached result

    result = charge_card(amount)
    cache.set(idempotency_key, result, ttl=86400)
    return result

# Client
idempotency_key = uuid.generate()
process_payment(100, idempotency_key)
# Retry với same key → No duplicate charge

# Solution 2: Database constraints
CREATE TABLE orders (
    order_id UUID PRIMARY KEY, -- Client-generated
    user_id INT,
    amount DECIMAL
);
-- Duplicate INSERT fails due to PK constraint
```

Loại Sự Cố 5: Sự Cố Cục Bộ (*Partial Failure*)

Định nghĩa: Một phần hệ thống gặp sự cố (*fail*), phần khác vẫn hoạt động.

Đây là loại sự cố nguy hiểm nhất.

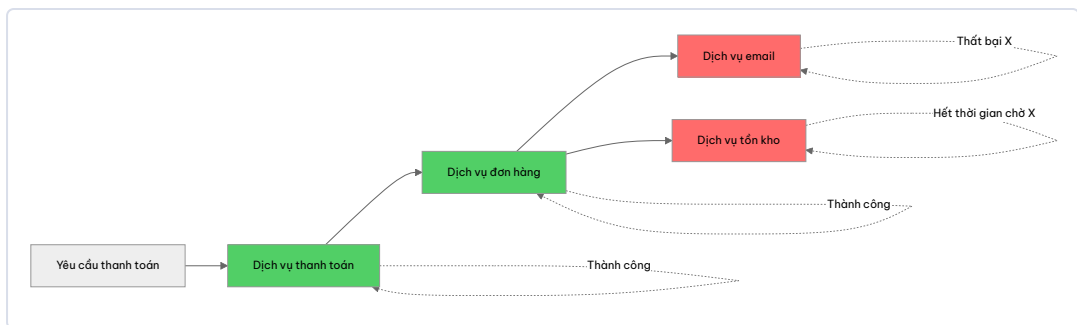
Kịch bản: Thanh toán trên trang thương mại điện tử

Bước 1: Trừ tiền thẻ tín dụng	→ Thành công
Bước 2: Tạo đơn hàng trong CSDL	→ Thành công
Bước 3: Gửi email xác nhận	→ Dịch vụ email ngừng hoạt động
Bước 4: Cập nhật tồn kho	→ Dịch vụ tồn kho chậm, hết thời gian chờ

Kết quả:

- Người dùng đã bị trừ tiền
- Đơn hàng đã được tạo
- Không có email (người dùng bối rối)
- Tồn kho chưa được cập nhật (nguy cơ bán vượt số lượng)

Thành công một phần = Trạng thái không nhất quán



Sự cố cục bộ: Một số dịch vụ (services) thành công, một số thất bại, dẫn đến trạng thái không nhất quán (inconsistent state)

Giải pháp:

```

# Solution 1: Saga pattern (compensating transactions)
def checkout_saga(order):
    try:
        # Step 1: Charge
        payment_id = payment_service.charge(order.amount)

        # Step 2: Create order
        order_id = order_service.create(order)

        # Step 3: Send email
        email_service.send(order.email)

        # Step 4: Update inventory
        inventory_service.decrease(order.items)

        return Success(order_id)

    except EmailServiceError:
        # Email failed, compensate
        order_service.mark_no_email(order_id)
        queue.retry_email(order) # Retry later
        return PartialSuccess(order_id)

    except InventoryServiceError:
        # Critical failure, rollback
        payment_service.refund(payment_id)
        order_service.cancel(order_id)
        return Failure("Inventory unavailable")

# Solution 2: Event sourcing
# Log all events, replay để achieve consistency
events = [
    {type: "payment_charged", amount: 100},
    {type: "order_created", order_id: 123},
    {type: "email_failed", retry_at: "10:00"},
    {type: "inventory_timeout", retry_at: "10:01"}
]
# Background job process failures

```

Định Lý CAP: Sự Đánh Đổi (*Trade-off*) Nền Tảng

Định Nghĩa CAP

Trong một cơ sở dữ liệu phân tán (*distributed database*) khi có phân vùng mạng, bạn chỉ có thể chọn 2 trong 3:

C (Consistency - Tính nhất quán):

Mọi thao tác đọc đều nhận được bản ghi mới nhất

Mọi nút thấy cùng một dữ liệu tại cùng thời điểm

A (Availability - Tính sẵn sàng):

Mọi yêu cầu đều nhận được phản hồi (không phải lỗi)

Hệ thống luôn hoạt động

P (Partition Tolerance - Khả năng chịu phân vùng):

Hệ thống tiếp tục hoạt động khi có phân vùng mạng

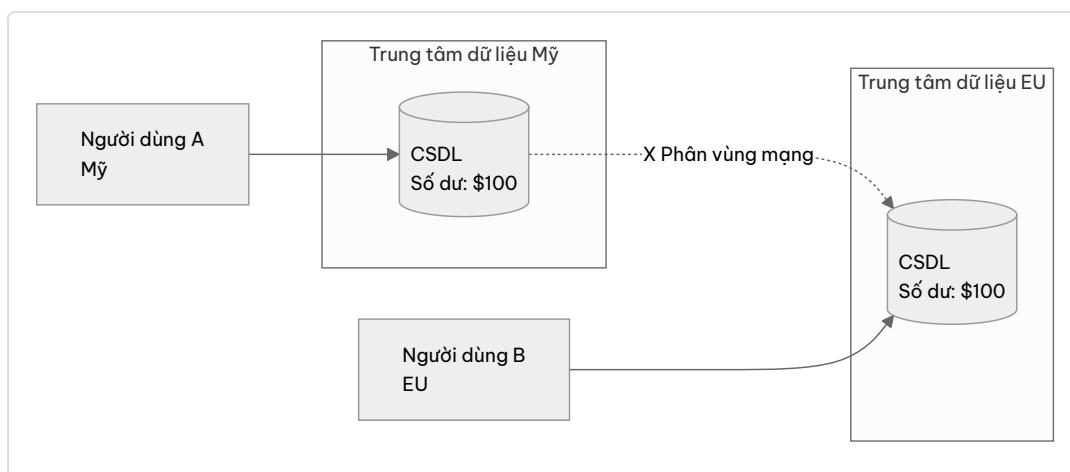
Các nút không giao tiếp được với nhau

Thực tế: Phân vùng mạng LUÔN xảy ra (P là bắt buộc).

→ **Phải chọn giữa C hoặc A.**

Trực Giác Về CAP Qua Ví Dụ Thực Tế

Kịch bản: Ứng dụng ngân hàng với 2 trung tâm dữ liệu



Phân vùng mạng xảy ra. Phải làm gì đây?

Lựa Chọn 1: CP (Tính Nhất Quán (Consistency) + Khả Năng Chịu Phân Vùng (Partition Tolerance))

Chiến lược: Hy sinh tính sẵn sàng (availability) để đảm bảo tính nhất quán.

```
# CP system behavior
def withdraw(amount):
    if not can_reach_all_replicas():
        raise ServiceUnavailableError("System temporarily unavailable")

    # Only proceed nếu có thể sync all replicas
    for replica in all_replicas:
        replica.withdraw(amount)

    return Success()
```

Timeline khi partition:

```
10:00:00 - Network partition
10:00:01 - User A (US) withdraw $50
           → US DB: "Cannot reach EU, refuse request"
           → User sees error (unavailable)
10:00:02 - User B (EU) withdraw $30
           → EU DB: "Cannot reach US, refuse request"
           → User sees error (unavailable)
```

Result:

System unavailable (bad UX)
No inconsistency (data safe)

Các sự đánh đổi (trade-offs):

Dữ liệu luôn nhất quán
Không có rủi ro cập nhật xung đột
Phù hợp cho: Ngân hàng, thanh toán, tiền kho

Hệ thống ngừng hoạt động khi có phân vùng
Trải nghiệm người dùng kém
Mất giao dịch

Lựa Chọn 2: AP (Tính Sẵn Sàng + Khả Năng Chịu Phân Vùng)

Chiến lược: Hy sinh tính nhất quán để đảm bảo tính sẵn sàng.

```
# AP system behavior
def withdraw(amount):
    # Proceed with local replica, sync later
    local_db.withdraw(amount)

    queue_sync_to_other_replicas(amount)

    return Success()
```

Timeline khi partition:

- 10:00:00 - Network partition
- 10:00:01 - User A (US) withdraw \$50
 - US DB: \$100 - \$50 = \$50
 - Success (local write)
- 10:00:02 - User B (EU) withdraw \$30
 - EU DB: \$100 - \$30 = \$70
 - Success (local write)
- 10:01:00 - Network heals
- 10:01:01 - Replicas sync
 - Conflict! US says \$50, EU says \$70

Result:

- System available (good UX)
- Data inconsistent (problem!)

Các chiến lược giải quyết xung đột (conflict resolution):

```
# Strategy 1: Last-Write-Wins (timestamp)
us_balance = {"value": 50, "timestamp": "10:00:01"}
eu_balance = {"value": 70, "timestamp": "10:00:02"}
# EU timestamp later → Balance = $70
# Lost US transaction!

# Strategy 2: Vector clocks (keep both)
us_balance = {"value": 50, "version": [1, 0]}
eu_balance = {"value": 70, "version": [0, 1]}
# Both are concurrent → Flag for manual resolution
# No data loss, but need human intervention

# Strategy 3: Application-specific merge
# Banking: Merge = apply both withdrawals
# Balance = $100 - $50 - $30 = $20
# Correct final state
# But requires domain knowledge
```

Các sự đánh đổi:

Hệ thống luôn sẵn sàng
Trải nghiệm người dùng tốt
Phù hợp cho: Mạng xã hội, gợi ý, phân tích dữ liệu

Dữ liệu có thể không nhất quán
Cần giải quyết xung đột
Nhất quán cuối cùng (không tức thời)

So Sánh CP Và AP

Ứng dụng ngân hàng:
Lựa chọn: CP
Tại sao? Độ chính xác dữ liệu > Tính sẵn sàng
Người dùng có thể đợi, nhưng không thể chấp nhận số dư sai

Bảng tin mạng xã hội:
Lựa chọn: AP
Tại sao? Tính sẵn sàng > Nhất quán tuyệt đối
Người dùng thấy số lượt thích hơi cũ một chút cũng không sao

Tồn kho thương mại điện tử?
Lựa chọn: CP
Tại sao? Không được bán vượt số lượng sản phẩm
Thà hiển thị "hết hàng" còn hơn bán món đã hết

Khung Tư Duy (*Framework*) Về Sự Đánh Đổi

Bước 1: Xác Định Yêu Cầu (*Requirements*)

- Tự hỏi:
1. Tính nhất quán có quan trọng sống còn không?
 - Dữ liệu sai = vấn đề lớn? (Ngân hàng: CÓ)
 - Dữ liệu cũ có chấp nhận được không? (Bảng tin mạng xã hội: CÓ)
 2. Tính sẵn sàng có quan trọng sống còn không?
 - Thời gian ngừng hoạt động có chấp nhận được không? (Công cụ nội bộ: CÓ)
 - Phải luôn hoạt động? (API công khai: CÓ)
 3. Tần suất phân vùng mạng?
 - Một trung tâm dữ liệu: Hiếm
 - Đa vùng: Thường xuyên

Bước 2: Phân Tích Các Sự Đánh Đổi

Hệ thống CP:

Khi nào: Giao dịch tài chính, tiền kho, đặt chỗ~

Chấp nhận: Ngừng hoạt động khi có phân vùng

Được: Độ chính xác dữ liệu

Hệ thống AP:

Khi nào: Tính năng mạng xã hội, phân tích dữ liệu, gợi ý

Chấp nhận: Nhất quán cuối cùng

Được: Luôn sẵn sàng

Hệ thống CA:

Không thể tồn tại trong hệ thống phân tán

(Phân vùng mạng luôn xảy ra)

Bước 3: Các Cách Tiếp Cận Lại

Thực tế: Không phải được ăn cả, ngã về không. Bạn có thể kết hợp.

```
# Different consistency cho different data
class UserService:
    def update_profile(self, user_id, data):
        # Profile updates: AP (eventual consistency OK)
        local_db.write(data)
        async_replicate(data)

    def update_balance(self, user_id, amount):
        # Balance updates: CP (strong consistency required)
        if not all_replicas_reachable():
            raise UnavailableError()

        for replica in all_replicas:
            replica.write(amount)

# Per-feature consistency choice
features = {
    "user_profile": "AP",      # Stale OK
    "account_balance": "CP",   # Must be accurate
    "friend_list": "AP",       # Stale OK
    "purchase_history": "CP"   # Must be accurate
}
```

Các Nguyên Tắc Thiết Kế Thực Tiễn

Nguyên Tắc 1: Thiết Kế Để Đối Phó Với Sự Cố

Hãy giả định mọi thứ đều sẽ hỏng.

```
# Optimistic (dangerous)
def call_service():
    return service.request()

# Pessimistic (safe)
def call_service():
    try:
        return service.request(timeout=2)
    except TimeoutError:
        log_error("Service timeout")
        return fallback_value()
    except ConnectionError:
        log_error("Service unreachable")
        return cached_value()
    except Exception as e:
        log_error(f"Unexpected error: {e}")
        return default_value()
```

Nguyên Tắc 2: Thất Bại Nhanh (Fail Fast)

Đừng chờ đợi mãi. Hãy thất bại nhanh rồi thử lại.

```
# Slow failure (30s timeout)
response = requests.get(url, timeout=30)

# Fast failure (2s timeout)
response = requests.get(url, timeout=2)

# Exponential backoff retry
for attempt in range(max_retries):
    try:
        return requests.get(url, timeout=2)
    except TimeoutError:
        if attempt < max_retries - 1:
            time.sleep(2 ** attempt) # 1s, 2s, 4s, 8s
        else:
            raise
```

Nguyên Tắc 3: Tính Lũy Đẳng (Idempotency)

Các thao tác phải an toàn khi thử lại.

```
# Not idempotent
def process_order(order):
    send_email(order.email) # Retry = duplicate emails!
    charge_card(order.amount) # Retry = double charge!

# Idempotent
def process_order(order, idempotency_key):
    if already_processed(idempotency_key):
        return get_cached_result(idempotency_key)

    result = {
        "email_sent": send_email_once(order.email, idempotency_key),
        "payment": charge_card_once(order.amount, idempotency_key)
    }

    cache_result(idempotency_key, result)
    return result
```

Nguyên Tắc 4: Nhất Quán Cuối Cùng (Eventual Consistency) Là Chấp Nhận Được

Không phải mọi thứ đều cần nhất quán tức thời (immediate consistency).

```
# Strong consistency (expensive)
def like_post(post_id):
    db.transaction:
        count = db.get_like_count(post_id)
        db.set_like_count(post_id, count + 1)
    # Every like requires DB write

# Eventual consistency (cheap)
def like_post(post_id):
    cache.increment(f"likes:{post_id}")
    queue.add({"action": "like", "post_id": post_id})
    # Batch update DB every 10 seconds

# User sees: 1000 likes
# Reality: 1002 likes (2 second lag)
# Acceptable? YES for social features
```

Những Điểm Cốt Lõi

Các sự cố trong hệ thống phân tán:

1. Phân vùng mạng: Các nút không giao tiếp được với nhau
2. Độ trễ cao: Chậm ≠ Chết
3. Mất gói tin: Thông điệp biến mất
4. Trùng lặp thông điệp: Cùng một thông điệp đến nhiều lần
5. Sự cố cục bộ: Một số thành công, một số thất bại

Định lý CAP:

Không thể có cả 3: Tính nhất quán, Tính sẵn sàng, Khả năng chịu phân vùng

Khả năng chịu phân vùng là bắt buộc (mạng luôn có lúc hỏng)

→ Chọn CP (nhất quán) hoặc AP (sẵn sàng)

CP: Ngân hàng, thanh toán, tiền kho

AP: Mạng xã hội, phân tích dữ liệu, gợi ý

Các nguyên tắc thiết kế:

1. Thiết kế để đối phó với sự cố (giả định mọi thứ đều hỏng)
2. Thất bại nhanh (timeout ngắn, thư lại nhanh)
3. Tính lũy đẳng (an toàn khi thư lại)
4. Nhất quán cuối cùng (chấp nhận được trong nhiều trường hợp)

Tư duy đánh đổi:

Không có giải pháp hoàn hảo.

Mọi lựa chọn đều hy sinh một thứ gì đó:

- CP hy sinh tính sẵn sàng
- AP hy sinh tính nhất quán
- Tính lũy đẳng hy sinh sự đơn giản
- Thư lại hy sinh độ trễ

Hãy chọn dựa trên yêu cầu nghiệp vụ, không phải sở thích kỹ thuật.

Định luật Murphy là có thật:

"Cái gì có thể hỏng thì sẽ hỏng."

Trong hệ thống phân tán:

"Mọi thứ đều sẽ hỏng. Cùng một lúc. Vào thời điểm tệ nhất."

Hãy thiết kế cho phù hợp.

Câu hỏi tự kiểm tra:

Khi thiết kế hệ thống phân tán, hãy tự hỏi:

1. Tính năng này cần tính nhất quán hay tính sẵn sàng hơn?
2. Các thao tác có lũy đẳng (*idempotent*) không?
3. Các giá trị thời gian chờ có hợp lý không?
4. Hệ thống xử lý phân vùng mạng thế nào?
5. Có phương án dự phòng (*fallback*) cho mọi lời gọi ra bên ngoài không?

Nếu không trả lời được → Chưa sẵn sàng cho môi trường thực tế.

Hệ thống phân tán rất khó. Nhưng với đúng tư duy (*mindset*) (thiết kế để đối phó với sự cố) và đúng mẫu thiết kế (*patterns*) (thời gian chờ (*timeouts*), thử lại (*retries*), tính lũy đẳng), bạn có thể xây dựng những hệ thống bền bỉ (*resilient*) và đáng tin cậy (*reliable*).

Thất bại một cách êm thấm > Không bao giờ thất bại.

Các Mô Hình Nhất Quán (Consistency Models): Nhất Quán Mạnh (Strong Consistency) và Nhất Quán Cuối Cùng (Eventual Consistency)

Tôi còn nhớ cái ngày tôi gây ra một lỗi (*bug*) nghiêm trọng vì không hiểu tính nhất quán (*consistency*).

Một hệ thống thương mại điện tử. Người dùng đặt mua sản phẩm cuối cùng trong kho. Nhưng vì nhất quán cuối cùng, 2 người dùng cùng thấy "còn hàng" và đặt hàng thành công.

Kết quả? Bán vượt số lượng tồn kho. Phải hoàn tiền cho một người. Bộ phận chăm sóc khách hàng gặp ác mộng. CEO gọi họp.

Tôi học được bài học đắt giá: **Tính nhất quán không phải sở thích kỹ thuật. Nó là quyết định kinh doanh.**

Bài học này sẽ giúp bạn tránh sai lầm đó.

Tại Sao Tính Nhất Quán Quan Trọng?

Trong một cơ sở dữ liệu (*database*) duy nhất, tính nhất quán là điều đương nhiên:

Người dùng A: Ghi $X = 10$
Người dùng B: Đọc $X \rightarrow$ Luôn thấy 10

Đơn giản. Luôn đúng.

Nhưng trong hệ thống phân tán (*distributed systems*) với nhiều cơ sở dữ liệu (*databases*):

Người dùng A: Ghi X = 10 vào CSDL 1
CSDL 1 → Nhân bản sang CSDL 2 (mất thời gian)
Người dùng B: Đọc X từ CSDL 2 → Có thể thấy giá trị cũ (X = 5)

Không đơn giản. Có thể sai.

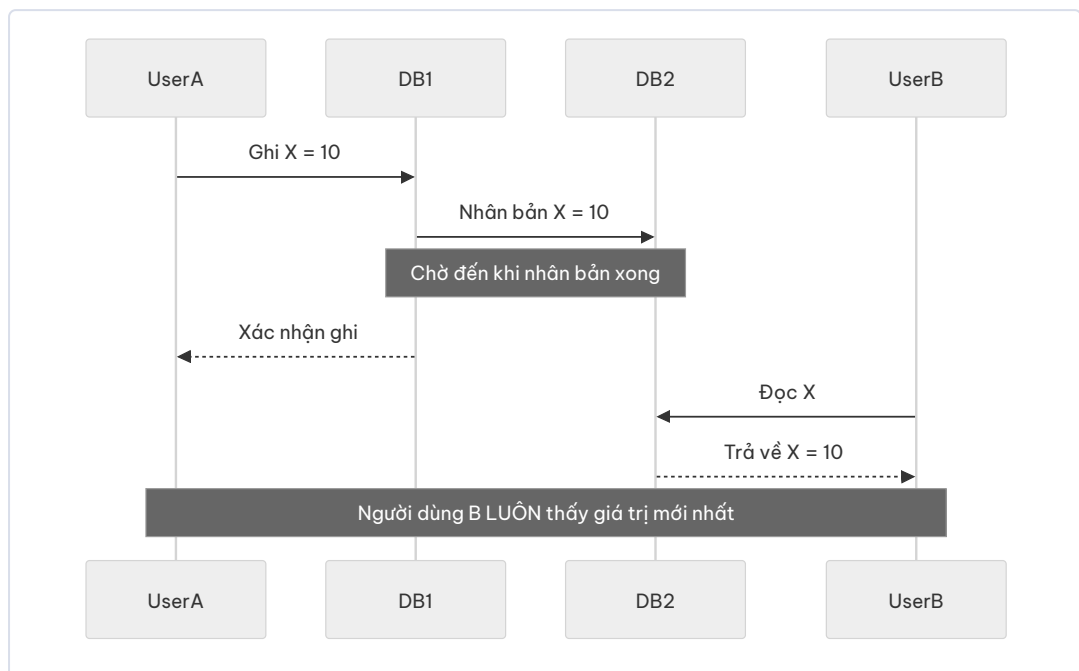
Câu hỏi: Người dùng B có nên thấy giá trị cũ không?

Trả lời: Còn tùy. Đó chính là các mô hình nhất quán.

Nhất Quán Mạnh: Luôn Đúng, Nhưng Chậm

Định Nghĩa

Nhất quán mạnh = Mọi thao tác đọc đều thấy thao tác ghi gần nhất



Nhất quán mạnh: Thao tác ghi chỉ được xác nhận sau khi nhân bản (replicate) xong

Hành vi:

Dòng thời gian:

10:00:00.000 - Người dùng A ghi X = 10
10:00:00.050 - Hệ thống nhân bản sang tất cả các nút (50ms)
10:00:00.051 - Xác nhận ghi cho Người dùng A
10:00:00.052 - Người dùng B đọc X → Thấy 10

Người dùng B LUÔN LUÔN thấy giá trị mới nhất
Giống như chỉ có 1 cơ sở dữ liệu duy nhất

Tính Tuyến Tính Hóa (*Linearizability*): Dạng Mạnh Nhất

Tính tuyến tính hóa = Các thao tác dường như xảy ra tức thời tại một thời điểm nào đó giữa lúc bắt đầu và lúc kết thúc

Nghe trừu tượng? Hãy xem ví dụ:

Dòng thời gian:

User A: Write X = 10 [===== 50ms =====]
User B: Read X → Phải thấy 10 (xảy ra sau khi ghi đã bắt đầu)
User C: Read X → Có thể thấy 5 hoặc 10 (đồng thời với thao tác ghi)
User D: Read X → Phải thấy 10

Quy tắc:

- Nếu đọc bắt đầu sau khi ghi hoàn tất → Phải thấy giá trị mới
- Nếu đọc đồng thời với ghi → Có thể thấy giá trị cũ hoặc mới
- Nếu ghi đã hoàn tất → Mọi thao tác đọc sau đó đều thấy giá trị mới

Ví dụ tương tự ngoài đời thực:

Tưởng tượng bạn đăng bài trên Facebook:

Nhất quán mạnh (tuyến tính hóa - Linearizable):

- Bạn đăng ảnh lúc 10:00
- Bạn bè tải lại trang lúc 10:01 → PHẢI thấy ảnh
- Không bao giờ có chuyện "bạn thấy ảnh nhưng bạn bè không thấy"

Giống như dán thông báo lên tường nhà
→ Ai đi ngang qua đều thấy

Chi Phí Triển Khai

Để đạt được nhất quán mạnh:

```
def write_with_strong_consistency(key, value):
    # 1. Write to primary
    primary.write(key, value)

    # 2. Replicate to ALL replicas
    for replica in replicas:
        replica.write(key, value)

    # 3. Wait for ALL confirmations
    wait_for_all_acks()

    # 4. Only then return success
    return "success"

# If ANY replica fails → Entire write fails
# If network slow → Write is slow
```

Tác động đến hiệu năng (*performance*):

Một cơ sở dữ liệu duy nhất:
Độ trễ ghi: 10ms

Nhất quán mạnh (3 bản sao):

Độ trễ ghi:

- Ghi cục bộ: 10ms
- Nhân bản sang Bản sao 1: 50ms (mạng)
- Nhân bản sang Bản sao 2: 50ms (mạng)
- Nhân bản sang Bản sao 3: 50ms (mạng)
- Chờ tất cả: $\max(50, 50, 50) = 50\text{ms}$

Tổng: $10 + 50 = 60\text{ms}$ (chậm hơn 6 lần!)

Nếu có bản sao ở xa:

- Bản sao Mỹ → châu Âu: 100ms
- Bản sao Mỹ → châu Á: 200ms

Tổng: $10 + 200 = 210\text{ms}$ (chậm hơn 21 lần!)

Đánh Đổi (Trade-offs)

Nhất quán mạnh:

Ưu điểm:

- Luôn đúng
- Không có dữ liệu cũ
- Dễ suy luận (như một cơ sở dữ liệu duy nhất)
- Không cần giải quyết xung đột ở tầng ứng dụng

Nhược điểm:

- Chậm (phải chờ nhân bản)
- Tính sẵn sàng thấp hơn (nếu bản sao bị sập, không thể ghi)
- Phân tán theo địa lý rất tốn kém
- Giới hạn khả năng mở rộng (càng nhiều bản sao = càng chậm)

Khi Nào Cần Nhất Quán Mạnh?

Cần nhất quán mạnh khi:

- ✓ Giao dịch tài chính
 - Chuyển khoản ngân hàng
 - Xử lý thanh toán
 - Kế toán
- ✓ Quản lý tồn kho
 - Món hàng cuối cùng trong kho
 - Đặt chỗ ngồi
 - Bán hàng phiên bản giới hạn
- ✓ Xác thực
 - Đổi mật khẩu
 - Cập nhật quyền
 - Token bảo mật
- ✓ Logic nghiệp vụ quan trọng
 - Chuyển trạng thái đơn hàng
 - Ký hợp đồng
 - Văn bản pháp lý

Ví dụ thực tế:

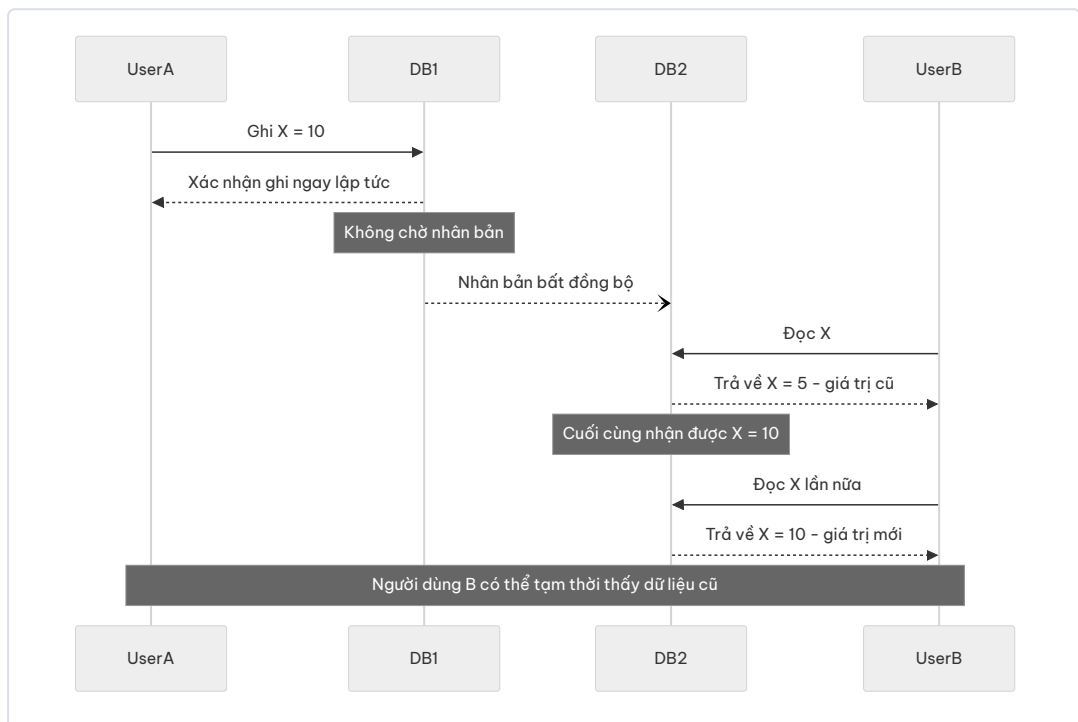
```
# Bank transfer PHẢI strong consistency
def transfer_money(from_account, to_account, amount):
    with transaction():
        # PHẢI atomic và immediate
        deduct(from_account, amount)
        add(to_account, amount)

# Không chấp nhận:
# - User thấy tiền đã trừ nhưng chưa vào account đích
# - Retry gây duplicate transfer
# - Stale balance data
```

Nhất Quán Cuối Cùng: Nhanh Nhưng Có Thể Sai Tạm Thời

Định Nghĩa

Nhất quán cuối cùng = Nếu không có thao tác ghi mới, cuối cùng tất cả các bản sao (replicas) sẽ hội tụ về cùng một giá trị



Nhất quán cuối cùng: Thao tác ghi được xác nhận ngay, nhân bản bất đồng bộ (async)

Hành vi:

Dòng thời gian:

10:00:00.000 - Người dùng A ghi X = 10 vào DB1
10:00:00.001 - Xác nhận ghi ngay lập tức (1ms)
10:00:00.010 - Người dùng B đọc từ DB2 → Thấy X = 5 (dữ liệu cũ!)
10:00:00.050 - DB1 nhân bản sang DB2
10:00:00.100 - Người dùng B đọc lại → Thấy X = 10

Người dùng B thấy dữ liệu cũ trong 50ms
Sau đó dữ liệu nhất quán cuối cùng

Ví Dụ Tương Tự Ngoài Đời Thực

Lượt thích trên Facebook:

- Bạn thích một bài đăng
- Lượt thích của bạn hiện ra ngay (cập nhật lạc quan - optimistic update)
- Phía backend nhân bản bất đồng bộ
- Bạn bè có thể thấy số đếm khác nhau
- Sau vài giây, mọi người đều thấy cùng số đếm

Chấp nhận được vì:

- Số lượt thích không quan trọng
- Trải nghiệm người dùng tốt hơn (nhanh)
- Đúng dần theo thời gian là đủ

Triển Khai

```
def write_with_eventual_consistency(key, value):  
    # 1. Write to primary  
    primary.write(key, value)  
  
    # 2. Return success IMMEDIATELY  
    return "success"  
  
    # 3. Replicate async in background  
    background_task.replicate_to_replicas(key, value)  
  
    # Fast! Không wait replication  
    # User không phải đợi  
  
def read_from_replica(key):  
    # Có thể đọc stale data  
    return replica.read(key)
```

Giải Quyết Xung Đột (Conflict Resolution)

Vấn đề với nhất quán cuối cùng:

Kịch bản:

10:00:00 - Người dùng A ghi X = 10 vào DB1

10:00:00 - Người dùng B ghi X = 20 vào DB2 (đồng thời!)

Xung đột! Cả hai thao tác ghi đều thành công, nhưng giá trị nào đúng?

DB1 cho rằng X = 10

DB2 cho rằng X = 20

Khi nhân bản → Xung đột!

Các chiến lược giải quyết:

Chiến lược 1: Ghi sau cùng thắng (*Last Write Wins*) (LWW)

```
def resolve_conflict(value1, value2):  
    # Use timestamp  
    if value1.timestamp > value2.timestamp:  
        return value1  
    else:  
        return value2  
  
# Simple nhưng có thể mất data  
# Nếu clocks không sync → Wrong decision
```

Chiến lược 2: Hợp nhất ở tầng ứng dụng (*Application-Level Merge*)

```
def resolve_conflict(value1, value2):  
    # Application decides  
    if feature_flag_enabled:  
        return merge_values(value1, value2)  
    else:  
        return value1 # Primary wins  
  
# Flexible nhưng complex logic
```

Chiến lược 3: Giữ cả hai (CRDTs)

```
# Conflict-free Replicated Data Types
# Example: Counter

# DB1: increment counter
counter_db1 = {node1: 5, node2: 3} # total = 8

# DB2: increment counter
counter_db2 = {node1: 5, node2: 4} # total = 9

# Merge: Take max per node
counter_merged = {node1: 5, node2: 4} # total = 9

# No conflict! Deterministic merge
```

Đánh Đổi

Nhất quán cuối cùng:

Ưu điểm:

- Ghi nhanh (không phải chờ)
- Tính sẵn sàng cao (các bản sao độc lập)
- Phù hợp với hệ thống phân tán theo địa lý
- Mở rộng tốt

Nhược điểm:

- Có thể đọc phải dữ liệu cũ
- Ứng dụng phải xử lý tình trạng thiếu nhất quán
- Độ phức tạp của việc giải quyết xung đột
- Khó suy luận hơn

Khi Nào Dừng Nhất Quán Cuối Cùng?

Nhất quán cuối cùng chấp nhận được khi:

- ✓ Tính năng mạng xã hội
 - Số lượt thích, lượt xem
 - Số người theo dõi
 - Bảng tin hoạt động
- ✓ Phân tích
 - Bảng điều khiển
 - Số liệu đo
 - Báo cáo
- ✓ Gợi ý
 - Gợi ý sản phẩm
 - Bảng tin nội dung
 - Kết quả tìm kiếm
- ✓ Cập nhật không quan trọng
 - Thay đổi hồ sơ
 - Tùy chọn cá nhân
 - Cài đặt giao diện

Ví dụ thực tế:

```
# Social media like KHÔNG CẦN strong consistency
def like_post(user_id, post_id):
    # Write immediately
    likes_db.increment(post_id)

    # User sees like immediately
    return {"likes": get_count_optimistic(post_id)}

# Background: Replicate to analytics DB
# Eventual consistency acceptable
# Không ai care nếu count sai 1-2 trong vài giây
```

Đánh Đổi Giữa Đọc và Ghi

Sự Đánh Đổi (Trade-off) Cốt Lõi

Tính nhất quán vs Hiệu năng vs Tính sẵn sàng

Chọn 2:

- Nhất quán mạnh + Tính sẵn sàng cao → Chậm
- Nhất quán mạnh + Nhanh → Tính sẵn sàng thấp hơn
- Nhanh + Tính sẵn sàng cao → Nhất quán cuối cùng

So sánh trực quan:

Nhất quán mạnh:
Ghi: ██████████ 60ms
Đọc: █████ 10ms
Tính sẵn sàng: 99.9%

Nhất quán cuối cùng:
Ghi: █ 5ms
Đọc: █ 5ms
Tính sẵn sàng: 99.99%

Tính Nhất Quán Có Thể Điều Chỉnh (*Tunable Consistency*)

Một số hệ thống cho phép điều chỉnh tính nhất quán cho từng thao tác:

```
# Cassandra example
def read_with_consistency_level(key, level):
    if level == "ALL":
        # Read từ TẤT CẢ replicas, đợi all responses
        # Strong consistency, slow
        return db.read(key, consistency="ALL")

    elif level == "QUORUM":
        # Read từ majority replicas
        # Good balance
        return db.read(key, consistency="QUORUM")

    elif level == "ONE":
        # Read từ 1 replica bất kỳ
        # Eventual consistency, fast
        return db.read(key, consistency="ONE")

# Ví dụ usage:
# Critical read → QUORUM
balance = read("account_balance", level="QUORUM")

# Non-critical read → ONE
recommendations = read("suggestions", level="ONE")
```

Ma trận đánh đổi:

Mức nhất quán	Độ trễ~	Tính sẵn sàng	Độ chính xác
ALL	Cao	Thấp	Hoàn hảo
QUORUM	Trung bình	Trung bình	Rất tốt
ONE	Thấp	Cao	Đúng dần theo thời gian

Ra Quyết Định Trong Thực Tế

Khung Tư Duy (Framework): 4 Câu Hỏi

Khi thiết kế một tính năng, hãy tự hỏi 4 câu hỏi này:

Câu hỏi 1: Người dùng có bị thiệt hại nếu thấy dữ liệu cũ (*stale data*) không?

Ví dụ:

- Số dư ngân hàng bị cũ → CÓ thiệt hại (người dùng bị thiếu chi)
- Số lượt thích bị cũ → KHÔNG thiệt hại (chỉ để hiển thị)

Nếu CÓ → Nhất quán mạnh

Nếu KHÔNG → Nhất quán cuối cùng là ổn

Câu hỏi 2: Doanh nghiệp có thiệt hại nếu dữ liệu thiếu nhất quán không?

Ví dụ:

- Bán vượt tồn kho → CÓ thiệt hại (hoàn tiền, uy tín)
- Thứ tự bảng tin sai → KHÔNG thiệt hại (chỉ ảnh hưởng trải nghiệm)

Nếu CÓ → Nhất quán mạnh

Nếu KHÔNG → Nhất quán cuối cùng là ổn

Câu hỏi 3: Có thể sửa sau không?

Ví dụ:

- Chuyển tiền sai → KHÔNG THỂ sửa (vấn đề pháp lý)
- Thông báo bị trùng lặp → CÓ THỂ sửa (chỉ gây khó chịu)

Nếu KHÔNG THỂ sửa → Nhất quán mạnh

Nếu CÓ THỂ sửa → Nhất quán cuối cùng là ổn

Câu hỏi 4: Người dùng có kỳ vọng thời gian thực (*real-time*) không?

Ví dụ:

- Tin nhắn chat → CÓ, kỳ vọng ngay lập tức
- Trạng thái đã đọc email → KHÔNG kỳ vọng (trễ cũng được)

Nếu CÓ → Nhất quán mạnh (hoặc trông như vậy)

Nếu KHÔNG → Nhất quán cuối cùng là ổn

Ma Trận Quyết Định

Loại tính năng	Nhất quán	Lý do
Xử lý thanh toán	Mạnh	Không được phép sai
Tồn kho (món cuối cùng)	Mạnh	Bán vượt tồn kho là tệ
Xác thực người dùng	Mạnh	Then chốt về bảo mật
Đổi mật khẩu	Mạnh	Then chốt về bảo mật
Trạng thái đơn hàng	Mạnh	Kỳ vọng của người dùng
Số lượt thích/xem	Cuối cùng	Xấp xỉ là ổn
Bảng tin hoạt động	Cuối cùng	Chấp nhận được độ trễ
Gợi ý	Cuối cùng	Vốn đã cá nhân hóa
Bảng điều khiển phân tích	Cuối cùng	Cập nhật theo lô là ổn
Cập nhật hồ sơ	Cuối cùng	Không gấp về thời gian
Kết quả tìm kiếm	Cuối cùng	Thứ hạng vốn thay đổi
Số thông báo	Cuối cùng	Lịch 1-2 vẫn ổn

Cách Tiếp Cận Kết Hợp

Trong thực tế, **hầu hết các hệ thống dùng cả hai:**

```
class EcommerceSystem:
    def checkout(self, cart_items):
        # STRONG consistency cho critical path
        with strong_consistency():
            inventory = check_inventory(cart_items) # Must be accurate
            payment = process_payment(amount)       # Must be exact
            order = create_order(cart_items)         # Must be atomic

        # EVENTUAL consistency cho non-critical
        with eventual_consistency():
            send_email_confirmation(order)           # Delay OK
            update_analytics(order)                   # Approximate OK
            generate_recommendations(user)            # Eventual OK
            update_activity_feed(user, order)         # Delay fine

# Critical = Strong
# Non-critical = Eventual
```

Quyết Định Tính Nhất Quán Theo Từng Tính Năng

Nghiên Cứu Tình Huống (Case Study) 1: Thanh Toán Trong Thương Mại Điện Tử

Tính năng: Thêm vào giỏ hàng

Quyết định: Nhất quán cuối cùng

Tại sao?

- Giỏ hàng là trạng thái tạm thời
- Người dùng có thể sửa trước khi thanh toán
- Trên vài giây là chấp nhận được
- Tính sẵn sàng cao quan trọng hơn

Triển khai:

- Ghi vào bộ nhớ đệm cục bộ ngay lập tức
- Nhân bản bất đồng bộ vào cơ sở dữ liệu
- Người dùng thấy giỏ hàng cập nhật tức thì

Tính năng: Đặt hàng

Quyết định: Nhất quán mạnh

Tại sao?

- Xử lý thanh toán là then chốt
- Kiểm tra tồn kho phải chính xác
- Không được phép bán vượt tồn kho
- Người dùng kỳ vọng được xác nhận ngay

Triển khai:

- Giao dịch trải qua tồn kho + thanh toán + đơn hàng
- Chờ tất cả xác nhận
- Người dùng đợi vài giây là ổn (đã được lường trước)

Nghiên Cứu Tình Hướng 2: Mạng Xã Hội

Tính năng: Đăng nội dung

Quyết định: Kết hợp

Tại sao?

- Người dùng kỳ vọng thấy bài của chính mình ngay (mạnh)
- Người theo dõi có thể thấy sau (cuối cùng)

Triển khai:

- Ghi với nhất quán mạnh
- Người dùng thấy bài đăng ngay
- Nhân bản bất đồng bộ đến người theo dõi
- Người theo dõi thấy trong vài giây

Tính năng: Số lượt thích

Quyết định: Nhất quán cuối cùng

Tại sao?

- Số đếm xấp xỉ là chấp nhận được
- Khối lượng lớn (hàng nghìn lượt thích/giây)
- Người dùng không quan tâm nếu lệch 10-20
- Phản hồi nhanh là then chốt

Triển khai:

- Tăng bộ đếm bất đồng bộ
- Cập nhật giao diện một cách lạc quan
- Đồng bộ nền vào cơ sở dữ liệu
- Số đếm nhất quán cuối cùng

Nghiên Cứu Tình Huống 3: Ứng Dụng Nhấn Tin

Tính năng: Gửi tin nhắn

Quyết định: Nhất quán mạnh

Tại sao?

- Thứ tự tin nhắn là then chốt
- Người dùng kỳ vọng người nhận thấy ngay
- Không được làm mất tin nhắn
- Xác nhận đã gửi rất quan trọng

Triển khai:

- Ghi vào CSDL của người gửi với nhất quán mạnh
- Nhân bản đồng bộ sang CSDL của người nhận
- Xác nhận đã gửi sau khi cả hai thao tác ghi xong
- Người dùng chờ vài ms (chấp nhận được)

Tính năng: Trạng thái "Hoạt động lần cuối"

Quyết định: Nhất quán cuối cùng

Tại sao?

- Thời gian xấp xỉ là ổn
- Cập nhật với tần suất cao
- Không phải dữ liệu quan trọng
- Trễ một chút là chấp nhận được

Triển khai:

- Cập nhật bất đồng bộ mỗi 30 giây
- Người dùng có thể thấy "đang trực tuyến" với độ trễ nhỏ
- Cuối cùng sẽ hội tụ
- Hiệu năng > Độ chính xác

Mô Hình Tư Duy (*Mental Model*): Tính Nhất Quán = Quyết Định Kinh Doanh

Điểm mấu chốt: Tính nhất quán không phải sở thích kỹ thuật.

Tư duy sai:

"Nhất quán cuối cùng tốt hơn vì nhanh"

"Nhất quán mạnh tốt hơn vì đúng"

Tư duy đúng:

"Tính năng này cần nhất quán mạnh vì yêu cầu kinh doanh X"

"Tính năng này ổn với nhất quán cuối cùng vì doanh nghiệp chấp nhận được Y"

Khung tư duy:

Với mỗi tính năng:

1. Xác định yêu cầu kinh doanh
→ Điều gì quan trọng nhất?
2. Đánh giá rủi ro
→ Trường hợp xấu nhất với dữ liệu cũ là gì?
3. Kiểm tra kỳ vọng của người dùng
→ Người dùng kỳ vọng tức thì hay trễ~cũng được?
4. Chọn mô hình nhất quán
→ Mạnh nếu then chốt
→ Cuối cùng nếu chấp nhận được
5. Triển khai tương ứng
→ Đừng nghi ngờ lại nhu cầu kinh doanh

Những Điểm Cốt Lỗi

Tóm tắt các mô hình nhất quán:

Nhất quán mạnh:

- Luôn đúng
- Chậm hơn, tính sẵn sàng thấp hơn
- Dùng cho: thanh toán, tồn kho, xác thực
- Đánh đổi: Hiệu năng lấy độ chính xác

Nhất quán cuối cùng:

- Nhanh, tính sẵn sàng cao
- Tạm thời không chính xác
- Dùng cho: lượt thích, bảng tin, phân tích
- Đánh đổi: Độ chính xác lấy hiệu năng

Khung ra quyết định (Decision framework):

Chọn nhất quán mạnh nếu:

- ✓ Dữ liệu then chốt với doanh nghiệp
- ✓ Không thể chấp nhận lỗi
- ✓ Người dùng bị thiệt hại nếu sai
- ✓ Có yêu cầu pháp lý/tuân thủ

Chọn nhất quán cuối cùng nếu:

- ✓ Dữ liệu xấp xỉ là ổn
- ✓ Có thể sửa lỗi sau
- ✓ Người dùng không quan tâm đến độ trễ
- ✓ Hiệu năng là then chốt

Cách tiếp cận kết hợp là tốt nhất:

Hầu hết các hệ thống thực tế:

- Nhất quán mạnh cho các luồng then chốt
- Nhất quán cuối cùng cho mọi thứ còn lại

Ví dụ:

- Thương mại điện tử: Mạnh cho thanh toán, Cuối cùng cho duyệt sản phẩm
- Mạng xã hội: Mạnh cho bài đăng, Cuối cùng cho lượt thích
- Ngân hàng: Mạnh cho giao dịch, Cuối cùng cho sao kê

Quy tắc vàng:

Tính nhất quán là quyết định kinh doanh, không phải sở thích kỹ thuật. Luôn bắt đầu bằng yêu cầu (*requirements*) kinh doanh, sau đó chọn mô hình nhất quán (*consistency model*) tương ứng.

Câu hỏi tự kiểm tra:

Trước khi triển khai một tính năng, hãy tự hỏi:

1. Người dùng có bị thiệt hại nếu dữ liệu cũ vài giây không?
2. Doanh nghiệp có thiệt hại nếu dữ liệu thiếu nhất quán không?
3. Người dùng có kỳ vọng cập nhật thời gian thực không?
4. Có thể bù đắp/sửa lỗi sau không?

Câu trả lời sẽ định hướng quyết định về tính nhất quán.

Hãy nhớ:

Nhất quán hoàn hảo ở mọi nơi = Hệ thống chậm

Không nhất quán ở đâu cả = Hệ thống sai

Kiến trúc sư thông minh:

- Nhất quán mạnh ở nơi cần thiết
- Nhất quán cuối cùng ở nơi chấp nhận được
- Lập luận rõ ràng cho mỗi quyết định

Các mô hình nhất quán không phải là chọn phương án "tốt hơn". Mà là chọn phương án **đúng** cho **bối cảnh** cụ thể.

Kinh doanh trước. Công nghệ sau. Luôn luôn.

Đồng Thuận (Consensus) & Bầu Chọn Nút Chủ (Leader Election): Thỏa Thuận Trong Hệ Thống Phân Tán (Distributed Agreement)

Tôi còn nhớ lần đầu tiên gỡ lỗi (debug) một lỗi (bug) não phân đôi (split brain).

Một hệ thống môi trường thực tế (production) với 5 máy chủ (servers). Mạng bị phân vùng (partition). Hình thành 2 nhóm máy chủ: Nhóm A (3 máy chủ) và Nhóm B (2 máy chủ).

Cả hai nhóm đều nghĩ mình là nút chủ (leader). Cả hai đều nhận thao tác ghi (writes). Dữ liệu xung đột khắp nơi.

Khôi phục mất 2 ngày. Đối soát dữ liệu (Data reconciliation) mất 1 tuần.

Một kiến trúc sư lâu năm (senior architect) nói: "Em chưa hiểu đồng thuận. Hệ thống phân tán (Distributed systems) phải THỐNG NHẤT trước khi HÀNH ĐỘNG."

Đó là bài học đắt giá về các thuật toán đồng thuận (consensus algorithms).

Tại Sao Cần Đồng Thuận?

Vấn Đề Cốt Lõi

Hệ thống một máy chủ (server): Đơn giản

Máy khách → Máy chủ (ra quyết định) → Thực thi

Chỉ có 1 bên ra quyết định. Không có bất đồng.

Hệ thống phân tán (Distributed system): Phức tạp

Máy khách → Máy chủ 1 (nghĩ mình là nút chủ)
→ Máy chủ 2 (nghĩ mình là nút chủ)
→ Máy chủ 3 (nghĩ mình là nút chủ)

Ai quyết định? Ai là nút chủ?

Nếu không có cơ chế thỏa thuận (*agreement mechanism*) → Hỗn loạn.

Các Kịch Bản Thực Tế Cần Đồng Thuận

Kịch bản 1: Bầu Chọn Máy Chủ Chính (*Master Election*) Cho Cơ Sở Dữ Liệu (*Database*)

- Cơ sở dữ liệu chính (master) chết
- Cần bầu master mới từ các bản sao (replica)
- Tất cả bản sao phải thống nhất master mới là ai
- Không được có 2 master (não phân đôi - split brain)

Kịch bản 2: Khóa Phân Tán (*Distributed Lock*)

- 2 dịch vụ cần truy cập cùng một tài nguyên
- Chỉ 1 dịch vụ được phép truy cập tại 1 thời điểm
- Cần thống nhất ai đang giữ khóa (lock)
- Không được để cả 2 đều nghĩ mình có khóa

Kịch bản 3: Quản Lý Cấu Hình (*Configuration Management*)

- Cập nhật cấu hình trong cụm (cluster)
- Tất cả các nút phải thấy cùng một cấu hình
- Không được có cấu hình thiếu nhất quán
- Cần đồng thuận trên các bản cập nhật cấu hình

Điểm mấu chốt: Hệ thống phân tán cần một cơ chế để **thống nhất về trạng thái** (*state*) trước khi hành động.

Bài Toán Thỏa Thuận Phân Tán (*Distributed Agreement Problem*)

Thách Thức

Trong hệ thống phân tán với mạng không đáng tin cậy:

Các vấn đề:

- Thông điệp có thể bị mất
- Thông điệp có thể bị trễ
- Máy chủ có thể sập
- Mạng có thể bị phân vùng

Câu hỏi: Làm sao để tất cả máy chủ thống nhất?

Ví Dụ: Điều Phối 3 Máy Chủ

Mục tiêu: Quyết định giá trị cho biến X

Máy chủ 1 đề xuất: $X = 10$

Máy chủ 2 đề xuất: $X = 20$

Máy chủ 3 đề xuất: $X = 10$

Làm sao để tất cả thống nhất trên 1 giá trị?

Cách tiếp cận ngây thơ (KHÔNG HOẠT ĐỘNG):

1. Máy chủ 1 phát tin: " $X = 10$ "
2. Chờ phản hồi từ tất cả máy chủ
3. Nếu đa số đồng ý $\rightarrow$ Commit (xác nhận)

Vấn đề:

- Máy chủ 2 sập trước khi phản hồi
- Máy chủ 1 không biết: Máy chủ 2 chết hay mạng chậm?
- Chờ mãi mãi? Bỏ cuộc sau bao lâu?

Cần một thuật toán đồng thuận (consensus algorithm) để giải quyết vấn đề này.

Các Yêu Cầu Của Đồng Thuận

Một thuật toán đồng thuận phải đảm bảo:

1. Thống nhất (Agreement): Tất cả máy chủ hoạt động đúng quyết định cùng một giá trị
2. Hợp lệ (Validity): Nếu máy chủ quyết định giá trị V, thì V phải được đề xuất bởi một máy chủ nào đó
3. Kết thúc (Termination): Tất cả máy chủ hoạt động đúng cuối cùng đều ra quyết định
4. Chịu lỗi (Fault tolerance): Vẫn hoạt động ngay cả khi một số máy chủ gặp sự cố

Raft: Thuật Toán Đồng Thuận Dễ Hiểu

Tại Sao Là Raft?

Trước Raft có Paxos (1989). Paxos nổi tiếng là cực kỳ khó hiểu.

Raft (2014) được thiết kế với mục tiêu: **Dễ hiểu**.

Tôi khuyên bạn học Raft trước. Nó đủ để hiểu những nền tảng của đồng thuận.

Các Ý Tưởng Cốt Lõi Của Raft

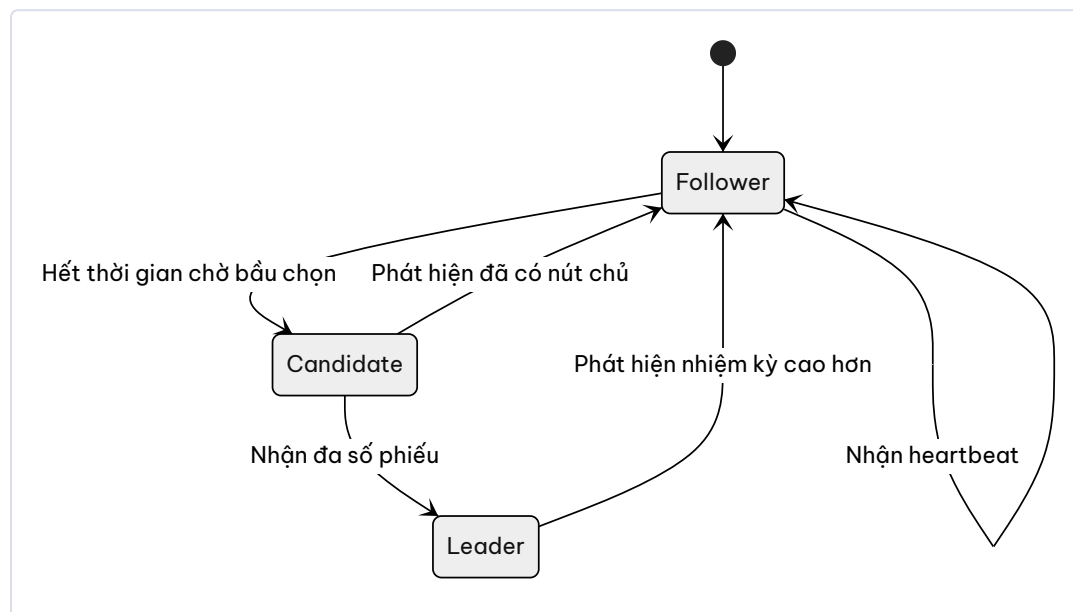
Raft chia bài toán thành 3 bài toán con:

1. Bầu chọn nút chủ (Leader Election): Chọn 1 máy chủ làm nút chủ
2. Nhân bản log (Log Replication): Nút chủ nhân bản các lệnh đến các nút theo (follower)
3. An toàn (Safety): Đảm bảo tính nhất quán khi có sự cố

Chúng ta tập trung vào Bầu Chọn Nút Chủ (quan trọng nhất).

Các Trạng Thái (States) Của Máy Chủ Trong Raft

Mỗi máy chủ có thể ở 1 trong 3 trạng thái:



Ba trạng thái trong Raft: Nút theo (Follower), Ứng viên (Candidate) và Nút chủ

Nút theo:

- Trạng thái thụ động
- Nhận và phản hồi các yêu cầu (*requests*)
- Không khởi tạo yêu cầu
- Hầu hết máy chủ ở trạng thái này

Ứng viên:

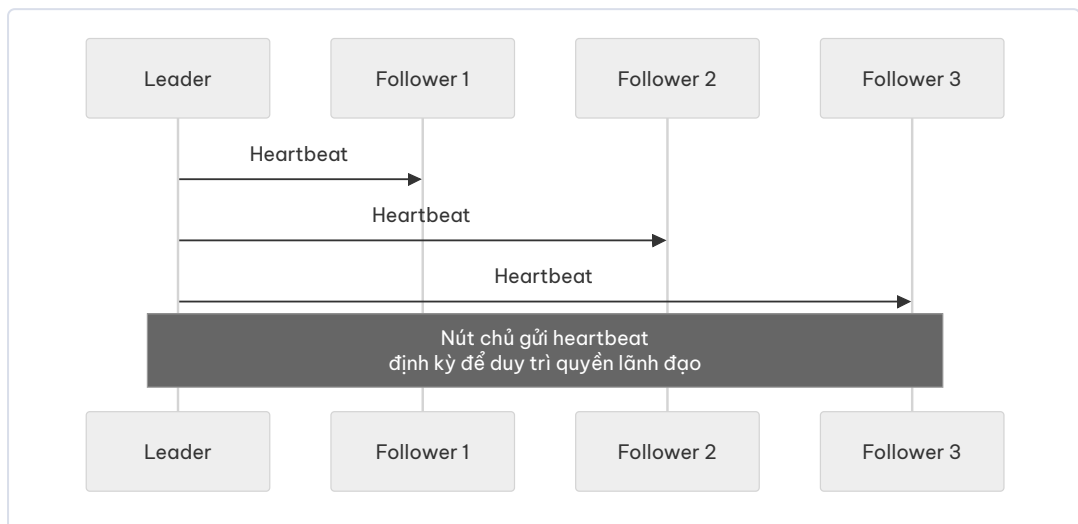
- Trạng thái chuyển tiếp
- Xin phiếu bầu từ các máy chủ khác
- Cố gắng trở thành nút chủ

Nút chủ:

- Xử lý tất cả yêu cầu từ máy khách (*client*)
- Nhân bản (*Replicate*) log đến các nút theo (*followers*)
- Chỉ có 1 nút chủ tại 1 thời điểm

Bầu Chọn Nút Chủ: Từng Bước Một

Hoạt Động Bình Thường

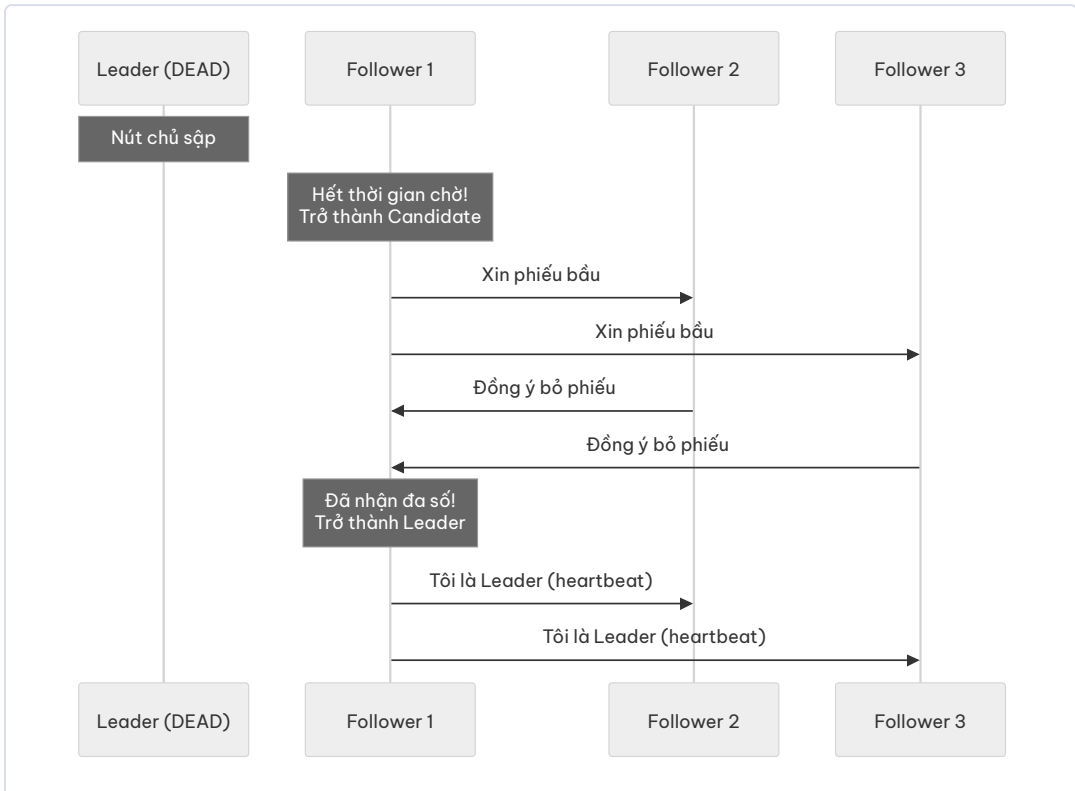


Nút chủ gửi tín hiệu nhịp tim (*heartbeats*) định kỳ để báo "tôi còn sống"

Quy trình:

1. Nút chủ gửi thông điệp heartbeat (nhịp tim) định kỳ (mỗi 50-100ms)
2. Các nút theo nhận heartbeat → Đặt lại bộ đếm thời gian bầu chọn
3. Mọi người đều vui vẻ, hệ thống ổn định

Nút Chủ Gặp Sự Cố (Failure) & Bầu Chọn



Khi nút chủ chết, quá trình bầu chọn (election process) bắt đầu

Quy Trình Chi Tiết:

Bước 1: Nút theo hết thời gian chờ (timeout)

- Nút theo 1 không nhận được heartbeat trong 150-300ms (ngẫu nhiên)
- Cho rằng nút chủ đã chết
- Chuyển sang trạng thái Ứng viên (Candidate)
- Tăng số nhiệm kỳ (term++)
- Tự bỏ phiếu cho chính mình

Bước 2: Xin phiếu bầu

Ứng viên gửi các RPC RequestVote đến tất cả máy chủ:

```
RequestVote {
    term: 5,           // Nhiệm kỳ hiện tại
    candidateId: 1,    // ID của tôi
    lastLogIndex: 100, // Mục log cuối cùng của tôi
    lastLogTerm: 4     // Nhiệm kỳ của mục log cuối cùng
}
```

Bước 3: Các nút theo bỏ phiếu

Nút theo nhận RequestVote:

```
if request.term < my_term:
    Từ chối (tôi biết nhiệm kỳ cao hơn)

if already_voted_this_term:
    Từ chối (đã bỏ phiếu rồi)

if my_log_is_more_updated:
    Từ chối (log của ứng viên đã lỗi thời)

else:
    Đồng ý bỏ phiếu
    Cập nhật my_term = request.term
```

Bước 4: Ứng viên đếm phiếu

Ứng viên thu thập phiếu bầu:

```
if votes_received >= (num_servers / 2) + 1:
    Trở thành LEADER
    Gửi heartbeat đến tất cả máy chủ

elif another_server_became_leader:
    Trở thành FOLLOWER

elif election_timeout:
    Bắt đầu cuộc bầu chọn mới (tăng nhiệm kỳ, thử lại)
```

Thời Gian Chờ Bầu Chọn (Election Timeout): Thiết Kế Then Chốt

Vấn đề: Nếu 2 ứng viên (*candidates*) bắt đầu bầu chọn cùng lúc thì sao?

Ứng viên 1 và Ứng viên 2 cùng xin phiếu bầu
→ Mỗi ứng viên nhận 2 phiếu (bao gồm phiếu của chính mình)
→ Không ai có đa số
→ Chia phiếu (split vote)!

Giải pháp: Thời gian chờ bầu chọn ngẫu nhiên (*Randomized election timeouts*)

```
# Mỗi server có random timeout
election_timeout = random(150ms, 300ms)

# Server 1: 173ms
# Server 2: 245ms
# Server 3: 198ms

→ Server 1 timeout trước
→ Start election trước
→ Likely win before others timeout
→ Avoid split votes
```

Xác suất:

Với thời gian chờ ngẫu nhiên:

- Hơn 90% cuộc bầu chọn chọn được nút chủ trong 1 vòng
- Hiếm khi chia phiếu
- Hệ thống hội tụ nhanh

Túc Số Đa Số (*Majority Quorum*): **Con Số Kỳ Diệu**

Tại Sao Là Đa Số?

Câu hỏi then chốt: Tại sao cần đa số phiếu? Tại sao không phải nhất trí (tất cả)?

Trả lời: Đa số đảm bảo an toàn.

Phần Toán Học

Tổng số máy chủ: N
Đa số: $\text{floor}(N/2) + 1$

Ví dụ:

$N = 3 \rightarrow$ Đa số = 2

$N = 5 \rightarrow$ Đa số = 3

$N = 7 \rightarrow$ Đa số = 4

Đảm Bảo: Chỉ Có 1 Nút Chủ

Kịch bản: Phân vùng mạng (Network partition)

Tổng cộng: 5 máy chủ
Bị phân vùng thành 2 nhóm:

Nhóm A: 3 máy chủ (đa số!)

Nhóm B: 2 máy chủ (thiếu số)

Nhóm A có thể bầu nút chủ (3 phiếu $\geq$ 3 cần thiết)

Nhóm B KHÔNG thể bầu nút chủ (2 phiếu $<$ 3 cần thiết)

→ Chỉ có 1 nút chủ!



Nhóm A có đa số, bầu được nút chủ. Nhóm B không đủ phiếu.

Tính Chất Giao Nhau

Nhận định then chốt:

Bất kỳ 2 tức số đa số (majority quorum) nào cũng PHẢI giao nhau

$N = 5$, Đa số = 3

Tức số 1: {Máy chủ 1, Máy chủ 2, Máy chủ 3}

Tức số 2: {Máy chủ 2, Máy chủ 4, Máy chủ 5}

Giao nhau: Máy chủ 2

- Không thể có 2 nhóm đa số tách rời nhau
- Không thể có 2 nút chủ trong cùng nhiệm kỳ

Chứng minh:

N máy chủ

Đa số = $\text{floor}(N/2) + 1$

2 nhóm đa số:

Tức số A: $\geq \text{floor}(N/2) + 1$ máy chủ

Tức số B: $\geq \text{floor}(N/2) + 1$ máy chủ

Tổng nếu tách rời:

$\geq 2 * (\text{floor}(N/2) + 1)$

$\geq N + 2$

Nhưng chỉ có N máy chủ!

- Mâu thuẫn
- Phải giao nhau

Tại Sao Lại Là Số Lẻ?

Khả năng chịu lỗi:

3 máy chủ: Chịu được 1 sự cố

4 máy chủ: Chịu được 1 sự cố (như nhau!)

5 máy chủ: Chịu được 2 sự cố

6 máy chủ: Chịu được 2 sự cố (như nhau!)

4 máy chủ không tốt hơn 3 máy chủ về khả năng chịu lỗi

6 máy chủ không tốt hơn 5 máy chủ

- Dùng số lẻ (3, 5, 7)
- Ít lãng phí tài nguyên hơn

Khuyến nghị:

Cụm nhỏ: 3 máy chủ (chịu được 1 sự cố)
Cụm vừa: 5 máy chủ (chịu được 2 sự cố)
Cụm lớn: 7 máy chủ (chịu được 3 sự cố)

> 7 máy chủ: Hiếm khi cần, chi phí phụ cho việc điều phối quá lớn

Vấn Đề Não Phân Đôi

Não Phân Đôi Là Gì?

Não phân đôi = Nhiều hơn 1 máy chủ nghĩ mình là nút chủ

Kịch bản thảm họa:
Nút chủ A xử lý thao tác ghi
Nút chủ B xử lý thao tác ghi (cùng lúc!)

- Dữ liệu phân kỳ
- Xung đột khắp nơi
- Khôi phục là một cơn ác mộng

Não Phân Đôi Xảy Ra Như Thế Nào

Kịch bản 1: Phân vùng mạng

Trước khi phân vùng:
[Máy chủ 1 (Nút chủ)] - [Máy chủ 2] - [Máy chủ 3]

Phân vùng mạng:
[Máy chủ 1 (Nút chủ)] | [Máy chủ 2] [Máy chủ 3]

Máy chủ 2 & 3 không thấy heartbeat từ Máy chủ 1
→ Bầu Máy chủ 2 làm nút chủ mới
→ Giờ có 2 nút chủ!

Kịch bản 2: Tín hiệu nhịp tim chậm

Máy chủ 1 (Nút chủ) vẫn còn sống nhưng chậm
→ Heartbeat bị trễ do nghẽn mạng
→ Các nút theo hết thời gian chờ, bầu nút chủ mới
→ Cuối cùng Máy chủ 1 vẫn gửi heartbeat đi
→ Tạm thời có 2 nút chủ

Raft Ngăn Chặn Nỗ Phân Đôi Như Thế Nào

Cơ chế 1: Túc số đa số

Phân vùng thành 2 nhóm:

Nhóm A: 3 máy chủ → Có thể bầu nút chủ ($3 \geq 3$)

Nhóm B: 2 máy chủ → Không thể bầu nút chủ ($2 < 3$)

→ Chỉ 1 nhóm có nút chủ

→ Không có nỗ phân đôi

Cơ chế 2: Số nhiệm kỳ (*Term numbers*)

Mỗi cuộc bầu chọn có một số nhiệm kỳ (term number)

Nút chủ cũ: nhiệm kỳ 5

Nút chủ mới được bầu: nhiệm kỳ 6

Khi nút chủ cũ hồi phục:

- Có gửi lệnh với nhiệm kỳ 5
- Các nút theo từ chối (nhiệm kỳ 5 < nhiệm kỳ hiện tại 6)
- Nút chủ cũ phát hiện nhiệm kỳ cao hơn
- Lùi xuống làm nút theo

→ Nút chủ cũ không còn hành động như nút chủ

Cơ chế 3: Hợp đồng thuê quyền nút chủ (*Leader lease*)

Nút chủ không chỉ cần đa số phiếu

Nút chủ phải duy trì "hợp đồng thuê" (lease) bằng heartbeat

Nếu nút chủ không gửi heartbeat:

→ Các nút theo thu hồi hợp đồng thuê ngầm định

→ Nút chủ không thể xử lý yêu cầu nữa

→ Nút chủ tự động dừng nếu bị cô lập

Khôi Phục Sau Nỗ Phân Đôi

Nếu vì lý do nào đó xảy ra nỗ phân đôi:

Phát hiện:

- 2 nút chủ phát hiện ra nhau
- So sánh số nhiệm kỳ

Nút chủ có nhiệm kỳ thấp hơn:

- Lập tức lùi xuống
- Trở thành nút theo
- Chấp nhận nút chủ có nhiệm kỳ cao hơn

Đối soát dữ liệu:

- Nút chủ có nhiệm kỳ cao hơn thắng
- Các thao tác ghi chưa commit từ nút chủ nhiệm kỳ thấp hơn = mất
- Đây là chủ ý thiết kế (an toàn > tính sống - safety > liveness)

Bầu Chọn Nút Chủ Trong Raft: Ví Dụ Đầy Đủ

Trạng Thái Ban Đầu

5 máy chủ, tất cả đều là nút theo
Nhiệm kỳ hiện tại: 3
Nút chủ: Máy chủ 1

Máy chủ 1 (Nút chủ, nhiệm kỳ 3)
Máy chủ 2 (Nút theo, nhiệm kỳ 3)
Máy chủ 3 (Nút theo, nhiệm kỳ 3)
Máy chủ 4 (Nút theo, nhiệm kỳ 3)
Máy chủ 5 (Nút theo, nhiệm kỳ 3)

Dòng Thời Gian (Timeline)

T=0: Nút chủ sập

Máy chủ 1 ĐÃ CHẾT
Máy chủ 2, 3, 4, 5 đang chờ heartbeat

T=200ms: Máy chủ 3 hết thời gian chờ

Máy chủ 3 hết thời gian chờ bầu chọn (ngẫu nhiên 150-300ms)
→ Trở thành Ứng viên (Candidate)
→ Tăng nhiệm kỳ: 3 → 4
→ Tự bỏ phiếu cho mình (1 phiếu)
→ Gửi RequestVote đến tất cả

T=205ms: Các máy chủ phản hồi

Máy chủ 2 nhận RequestVote:

- Chưa bỏ phiếu trong nhiệm kỳ 4
- Bỏ phiếu cho Máy chủ 3

Máy chủ 4 nhận RequestVote:

- Chưa bỏ phiếu trong nhiệm kỳ 4
- Bỏ phiếu cho Máy chủ 3

Máy chủ 5 nhận RequestVote:

- Chưa bỏ phiếu trong nhiệm kỳ 4
- Bỏ phiếu cho Máy chủ 3

T=210ms: Máy chủ 3 đếm phiếu

Máy chủ 3:

- Phiếu của chính mình: 1
- Máy chủ 2: 1
- Máy chủ 4: 1
- Máy chủ 5: 1

Tổng: 4 phiếu trên 5 máy chủ

Đa số: cần 3

4 >= 3 → THẮNG!

→ Trở thành NÚT CHỦ cho nhiệm kỳ 4

T=215ms: Nút chủ mới thông báo

Máy chủ 3 (Nút chủ):

- Gửi heartbeat đến tất cả máy chủ
- "Tôi là nút chủ của nhiệm kỳ 4"

Máy chủ 2, 4, 5:

- Nhận heartbeat
- Công nhận Máy chủ 3 là nút chủ
- Đặt lại bộ đếm thời gian bầu chọn
- Tiếp tục làm nút theo

T=220ms trở đi: Hoạt động bình thường

Máy chủ 3 tiếp tục gửi heartbeat mỗi 50ms

Hệ thống ổn định với Máy chủ 3 là nút chủ

Mô Hình Tư Duy (*Mental Model*): Thỏa Thuận Phân Tán

Nguyên Tắc Cốt Lõi

"Thông nhất trước khi Hành động"

Tư duy một máy chủ:

Thấy yêu cầu → Thực thi → Xong

Tư duy phân tán:

Thấy yêu cầu → Hỏi các nút khác → Đạt đồng thuận → Thực thi

Hệ Quả Đối Với Thiết Kế

Hệ quả 1: Độ trễ (*Latency*) tăng

Một máy chủ: 10ms

Đồng thuận (đa số trong 5): 50-100ms

Đánh đổi:

Tính nhất quán + khả năng chịu lỗi vs Tốc độ

Hệ quả 2: Cần số lẻ

3, 5, 7 máy chủ là tối ưu

Số chẵn lãng phí tài nguyên

Hệ quả 3: Phân vùng mạng (*Network partitions*) = mất tính sẵn sàng (*availability*)

Phân vùng mạng: Phân vùng thiểu số không thể tiếp tục xử lý

Đánh đổi: Ưu tiên tính nhất quán hơn tính sẵn sàng (CP trong CAP)

Khi Nào Nên Dùng Đồng Thuận

Dùng thuật toán đồng thuận khi:

- ✓ Cần nhất quán mạnh
- ✓ Không thể chấp nhận lỗi phân đôi
- ✓ Khả năng chịu lỗi là then chốt
- ✓ Bầu chọn máy chủ chính (master)
- ✓ Khóa phân tán
- ✓ Quản lý cấu hình

Tránh dùng đồng thuận khi:

- x Nhất quán cuối cùng là chấp nhận được
- x Không thể chấp nhận độ trễ tăng
- x Ưu tiên tính sẵn sàng hơn tính nhất quán (AP trong CAP)

Các Hệ Thống Sử Dụng Đồng Thuận

etcd:

Kho lưu trữ khóa-giá trị (key-value) phân tán
Dùng Raft để đạt đồng thuận
Kubernetes lưu trạng thái cụm trong etcd

Consul:

Khám phá dịch vụ (service discovery) và cấu hình
Dùng Raft cho bầu chọn nút chủ và nhân bản dữ liệu

Zookeeper:

Dịch vụ điều phối
Dùng Zab (tương tự Raft/Paxos)
Kafka, HBase phụ thuộc vào Zookeeper

CockroachDB:

Cơ sở dữ liệu SQL phân tán
Dùng Raft cho nhân bản và tính nhất quán

Những Sai Lầm Phổ Biến & Cách Tránh

Sai Lầm 1: Không Dùng Túc Số Đa Số

Tệ: Chờ TẤT CẢ máy chủ phản hồi
Vấn đề: 1 máy chủ chết → Không thể tiếp tục xử lý

Tốt: Chờ đa số
Lợi ích: Chịu được sự cố ở phần thiểu số

Sai Lầm 2: Quên Mất Nỗ Phân Đôi

Tệ: Giả định chỉ có 1 nút chủ
Thực tế: Phân vùng mạng vẫn xảy ra

Tốt: Thiết kế với giả định có thể có nhiều nút chủ
Dùng số nhiệm kỳ, ghi theo đa số để ngăn chặn

Sai Lầm 3: Số Máy Chủ Chẵn

Tệ: 4 máy chủ (chịu được 1 sự cố)
Chi phí: 4 máy chủ
Lợi ích: Như 3 máy chủ

Tốt: 3 máy chủ (chịu được 1 sự cố)
Chi phí: 3 máy chủ
Lợi ích: Cùng khả năng chịu lỗi, tiết kiệm 1 máy chủ

Sai Lầm 4: Không Hiểu Sự Đánh Đổi (*Trade-off*) Về Tính Nhất Quán (*Consistency*)

Tệ: Kỳ vọng có cả tính nhất quán lẫn tính sẵn sàng
Thực tế: Định lý CAP nói chỉ chọn được 2

Tốt: Chủ động chọn tính nhất quán (CP)
Chấp nhận mất tính sẵn sàng khi có phân vùng

Những Điểm Cốt Lõi

Thuật toán đồng thuận giải quyết bài toán thỏa thuận phân tán:

Vấn đề: Nhiều máy chủ cần thống nhất về trạng thái
Giải pháp: Bỏ phiếu theo đa số với số nhiệm kỳ
Kết quả: Nhất quán mạnh + khả năng chịu lỗi

Nền tảng của Raft:

1. Bầu chọn nút chủ: 1 nút chủ tại 1 thời điểm
2. Túc số đa số: Ngăn ngừa phân đôi
3. Số nhiệm kỳ: Phát hiện nút chủ lỗi thời
4. Thời gian chờ ngẫu nhiên: Tránh chia phiếu

Ngăn chặn não phân đôi:

- Túc số đa số (không có 2 nhóm đa số tách rời)
- Số nhiệm kỳ (nhiệm kỳ cao hơn thắng)
- Hợp đồng thuê quyền nút chủ (hết thời gian chờ heartbeat)

Nguyên tắc thiết kế:

- Số lẻ (3, 5, 7 máy chủ)
- Đa số = $\text{floor}(N/2) + 1$
- Thống nhất trước khi hành động
- Chấp nhận độ trễ tăng

Những sự đánh đổi (trade-offs) được chấp nhận:

Được: Nhất quán mạnh, khả năng chịu lỗi
Mất: Độ trễ, độ phức tạp, tính sẵn sàng khi có phân vùng

Mô hình tư duy:

"Hệ thống phân tán phải **THỐNG NHẤT** trước khi **HÀNH ĐỘNG**. Đồng thuận là cơ chế để thống nhất. Bỏ phiếu theo đa số ngăn chặn não phân đôi. Số nhiệm kỳ giúp phát hiện sự lỗi thời."

Câu hỏi tự kiểm tra:

Trước khi triển khai hệ thống phân tán với đồng thuận:

1. Tôi có thực sự cần nhất quán mạnh (strong consistency) không?
2. Tôi có thể chấp nhận độ trễ tăng không?
3. Tôi có thể đánh đổi tính sẵn sàng lấy tính nhất quán không?
4. Tôi đã hiểu não phân đôi và cách ngăn chặn nó chưa?

Đồng thuận rất mạnh mẽ nhưng đắt đỏ. Hãy dùng khi lợi ích vượt trội chi phí.

Câu nói tôi hay nhắc nhóm (team):

"Nếu bạn không cần nhất quán mạnh, đừng trả giá cho nó. Nhưng nếu bạn cần, hãy trả đủ giá - triển khai đồng thuận cho đúng."

Đồng thuận không phải phép màu. Nó là một cơ chế thỏa thuận được thiết kế cẩn thận với những sự đánh đổi rõ ràng.

Hiểu sự đánh đổi → Ra quyết định tốt hơn.

Giao Dịch Phân Tán (*Distributed Transactions*) & Toàn Vẹn Dữ Liệu (*Data Integrity*)

Tôi còn nhớ ngày hệ thống thanh toán (*payment*) đầu tiên tôi thiết kế gặp lỗi (*bug*) nghiêm trọng.

Người dùng bị trừ tiền thẻ thành công. Nhưng đơn hàng (*order*) không được tạo. Mất tiền mà không có đơn hàng.

Người dùng khiếu nại. Nhóm (*team*) tài chính hoảng loạn. CEO gọi họp khẩn cấp.

Lúc đó tôi nghĩ: "Sao giao dịch (*transaction*) không hoạt động? Tôi đã dùng `BEGIN TRANSACTION` rồi mà!"

Một kiến trúc sư lâu năm (*senior architect*) nhìn vào mã, lắc đầu: "Em đang làm hệ thống phân tán (*distributed system*). Giao dịch (*transactions*) ACID không còn tác dụng nữa đâu."

Đó là ngày tôi học được bài học đắt giá nhất về giao dịch phân tán.

Tại Sao ACID Bị Phá Vỡ Trong Hệ Thống Phân Tán (*Distributed Systems*)?

ACID Trong Một Cơ Sở Dữ Liệu (*Database*) Duy Nhất

Trong một ứng dụng nguyên khối (*monolithic*) với một cơ sở dữ liệu duy nhất, giao dịch ACID hoạt động hoàn hảo:

```
# Single database - ACID guaranteed
def transfer_money(from_account, to_account, amount):
    db.begin_transaction()
    try:
        db.execute("UPDATE accounts SET balance = balance - ? WHERE id = ?",
                    amount, from_account)
        db.execute("UPDATE accounts SET balance = balance + ? WHERE id = ?",
                    amount, to_account)
        db.commit()
    except:
        db.rollback() # All changes undone
```

Những đảm bảo của ACID:

Atomicity (Tính nguyên tử): Tất cả hoặc không gì cả (cả 2 cập nhật hoặc không cập nhật gì)

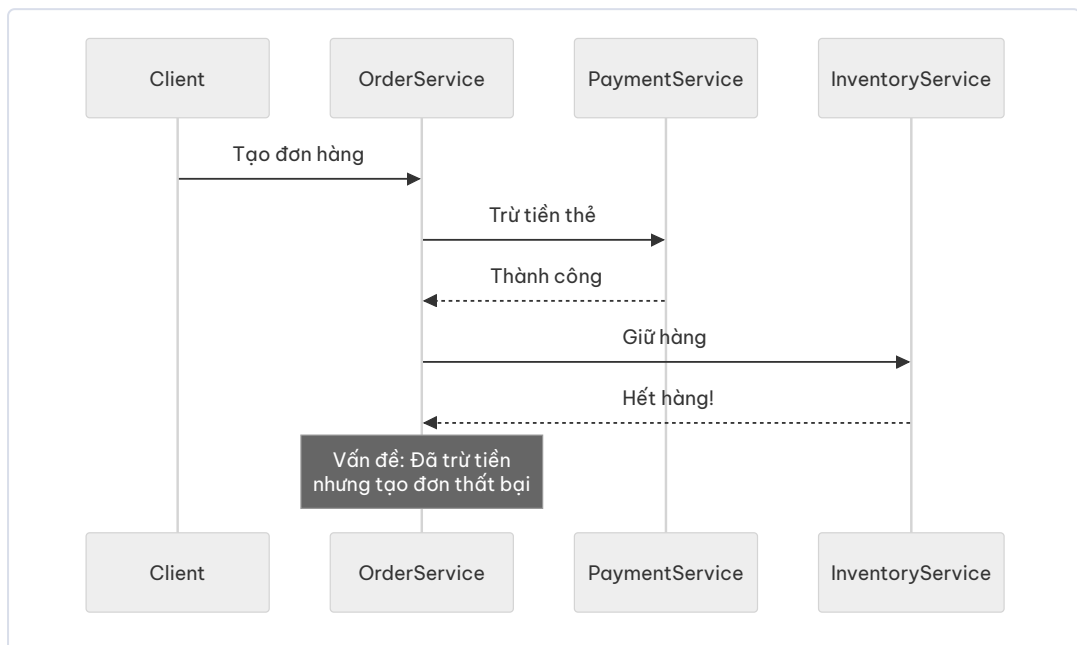
Consistency (Tính nhất quán): Dữ liệu hợp lệ (số dư không âm)

Isolation (Tính cô lập): Các giao dịch không thấy những thay đổi dở dang

Durability (Độ bền): Dữ liệu đã commit được lưu bền vững

ACID Bị Phá Vỡ Trong Hệ Thống Phân Tán

Giờ chuyển sang vi dịch vụ (*microservices*):



Luồng (Flow) tạo đơn hàng đi qua nhiều dịch vụ (services) - không thể dùng giao dịch của cơ sở dữ liệu

Vấn đề:

1. Dịch vụ Thanh toán trừ tiền thẻ → Thành công
2. Dịch vụ Kho kiểm tra hàng tồn → Hết hàng!
3. Dịch vụ Đơn hàng thất bại

Kết quả:

Đã trừ tiền (tiền đã bị lấy)

Đơn hàng không được tạo

Hàng trong kho không được giữ

Khách hàng: "Tại sao tiền bị trừ mà không có đơn hàng?"

Tại sao không thể hoàn tác (rollback)?

Không thể dùng giao dịch cơ sở dữ liệu:

- 3 dịch vụ = 3 cơ sở dữ liệu khác nhau
- Không thể hoàn tác xuyên qua nhiều cơ sở dữ liệu
- Dịch vụ Thanh toán đã commit rồi

Không thể huỷ thanh toán:

- Đã gửi sang công thanh toán
- Hệ thống bên ngoài, không kiểm soát được
- Không thể gọi "db.rollback()"

Nguyên nhân gốc rễ:

Các giả định của ACID bị phá vỡ khi:

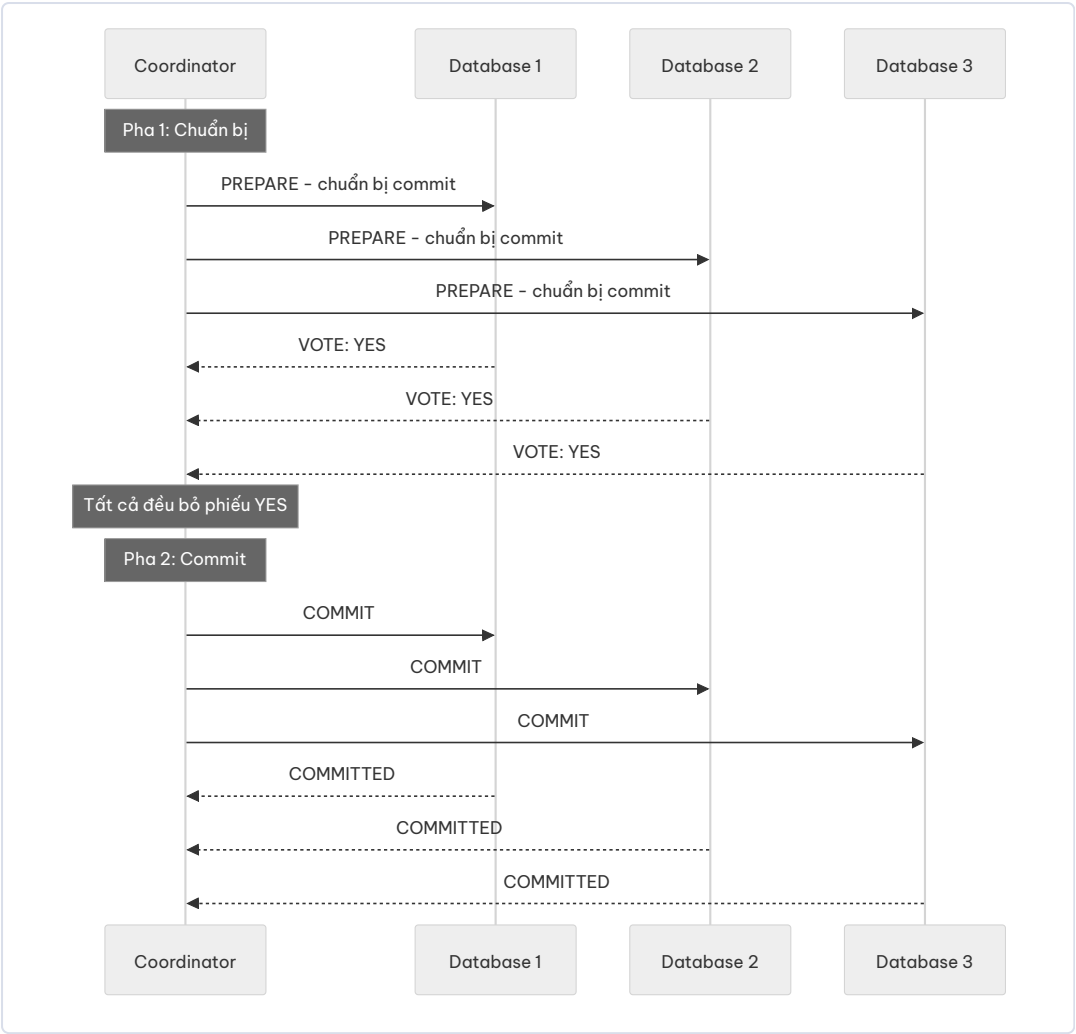
- Có nhiều cơ sở dữ liệu tham gia
- Có nhiều dịch vụ tham gia
- Có hệ thống bên ngoài tham gia
- Có các lời gọi qua mạng

Mạng không đáng tin cậy → Không thể đảm bảo ACID

2-Phase Commit (2PC - Cam Kết Hai Pha): Giải Pháp Truyền Thống

2PC Hoạt Động Như Thế Nào

2-Phase Commit = Giao thức (Protocol) để điều phối (coordinate) các giao dịch trên nhiều cơ sở dữ liệu (databases)



2PC: Pha 1 chuẩn bị (prepare), Pha 2 cam kết (commit) nếu tất cả đều đồng ý

Pha 1: Chuẩn bị

Bộ điều phối → Tất cả bên tham gia: "Chuẩn bị commit"

Mỗi bên tham gia:

1. Khoá tài nguyên
2. Ghi vào log giao dịch
3. Sẵn sàng commit NHƯNG chưa commit
4. Bỏ phiếu YES hoặc NO

Nếu bất kỳ bên nào bỏ phiếu NO → Huỷ bỏ (abort)

Nếu tất cả bỏ phiếu YES → Chuyển sang Pha 2

Pha 2: Cam kết

Nếu tất cả YES:

Bộ điều phối → Tất cả: "COMMIT"

Các bên tham gia commit thay đổi

Giải phóng khoá

Nếu có bất kỳ NO nào:

Bộ điều phối → Tất cả: "ABORT"

Các bên tham gia hoàn tác (rollback)

Giải phóng khoá

Ví Dụ Triển Khai 2PC

```
class TwoPhaseCommitCoordinator:
    def __init__(self, participants):
        self.participants = participants
        self.transaction_id = generate_id()

    def execute_transaction(self, operations):
        # Phase 1: Prepare
        print("Phase 1: Prepare")
        votes = []

        for participant in self.participants:
            try:
                vote = participant.prepare(self.transaction_id, operations)
                votes.append(vote)
                print(f"{participant.name}: {vote}")
            except Exception as e:
                votes.append("NO")
                print(f"{participant.name}: ERROR - {e}")

        # Decision
        if all(vote == "YES" for vote in votes):
            decision = "COMMIT"
        else:
            decision = "ABORT"

        print(f"\nDecision: {decision}")

        # Phase 2: Commit or Abort
        print("Phase 2: Execute decision")
        results = []

        for participant in self.participants:
            if decision == "COMMIT":
                result = participant.commit(self.transaction_id)
            else:
                result = participant.abort(self.transaction_id)
            results.append(result)
            print(f"{participant.name}: {result}")

        return decision == "COMMIT"

# Usage
coordinator = TwoPhaseCommitCoordinator([
    payment_service,
    inventory_service,
    order_service
])

success = coordinator.execute_transaction({
    'payment': {'amount': 100},
    'inventory': {'product_id': 123, 'quantity': 1},
```

```
'order': {'user_id': 456}
})
```

Điểm Yếu Chí Mạng: Chặn (Blocking)

Vấn đề: Bộ điều phối (Coordinator) sập sau Pha 1

Diễn biến:

1. Bộ điều phối gửi PREPARE tới tất cả bên tham gia
2. Tất cả bên tham gia bỏ phiếu YES, khoá tài nguyên, chờ...
3. Bộ điều phối SẬP (trước khi gửi COMMIT)
4. Các bên tham gia mắc kẹt chờ mãi mãi
5. Tài nguyên bị KHOÁ vô thời hạn

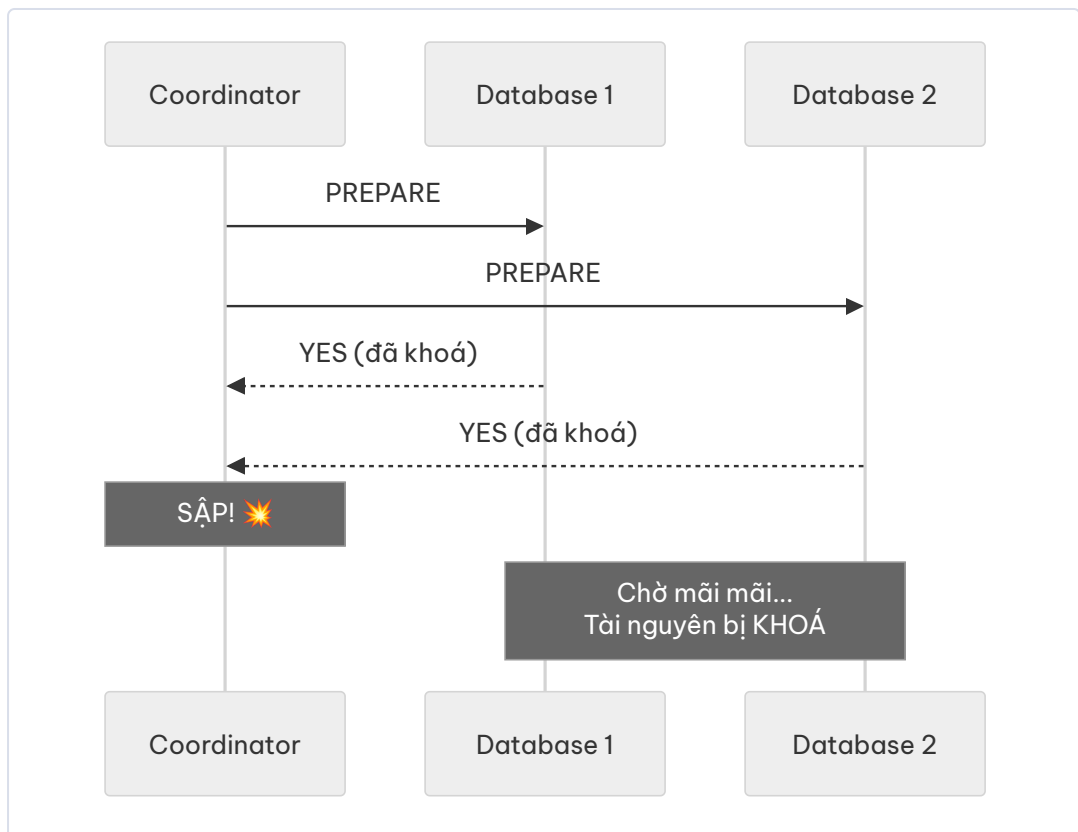
Kết quả:

Các cơ sở dữ liệu bị khoá

Không thể tiếp tục

Không thể dùng hết thời gian chờ (không biết bộ điều phối có hồi phục không)

Hệ thống bị CHẶN



Bộ điều phối sập sau pha chuẩn bị (*prepare phase*) → Các cơ sở dữ liệu bị khoá (*lock*) vô thời hạn

Tại sao không thể dùng hết thời gian chờ (*timeout*)?

Thế khó của bên tham gia:

- Không biết bộ điều phối đã gửi COMMIT chưa
- Không biết các bên tham gia khác đã commit chưa
- Không thể tự quyết định commit hay abort
- Hết thời gian chờ → Rủi ro mất nhất quán

→ Phải chờ bộ điều phối hồi phục

→ Giao thức CHẶN

Các Đánh Đổi (*Trade-offs*) Của 2PC

2-Phase Commit:

Nhất quán mạnh (giữ được ACID)

Đảm bảo tất cả hoặc không gì cả

Giao thức đã được kiểm chứng

Chặn (bộ điều phối gặp sự cố = thảm họa)

Chi phí phụ về hiệu năng (2 lượt khứ hồi)

Độ trễ cao (phải chờ bên tham gia chậm nhất)

Giảm tính sẵn sàng

Triển khai phức tạp

Khi Nào Dùng 2PC?

Sử dụng 2PC khi:

- ✓ Bắt buộc tuyệt đối phải có nhất quán mạnh
- ✓ Giao dịch tài chính (ngân hàng)
- ✓ Tất cả bên tham gia đều nằm trong tầm kiểm soát của bạn
- ✓ Chịu được việc bị chặn
- ✓ Độ trễ thấp không phải yêu tố then chốt

Tránh 2PC khi:

- x Cần tính sẵn sàng cao
- x Kiến trúc vi dịch vụ (có dịch vụ bên ngoài)
- x Yêu cầu thông lượng cao
- x Không chịu được việc bị chặn

Ý kiến cá nhân:

Tôi tránh 2PC trong vi dịch vụ. Rủi ro chặn quá cao. Độ phức tạp (*Complexity*) không đáng.

99% trường hợp có giải pháp tốt hơn: mẫu (pattern) Saga.

Mẫu Saga: Giải Pháp Thực Tế

Khái Niệm Saga

Saga = Chuỗi các giao dịch cục bộ (local transactions) + **bù trừ** (compensation)

Thay vì 1 giao dịch phân tán (distributed transaction):

Cách cũ (2PC):

BEGIN_DISTRIBUTED_TRANSACTION

Dịch vụ Thanh toán: trừ tiền

Dịch vụ Kho: giữ hàng

Dịch vụ Đơn hàng: tạo đơn

COMMIT_DISTRIBUTED_TRANSACTION

Dùng chuỗi các giao dịch cục bộ:

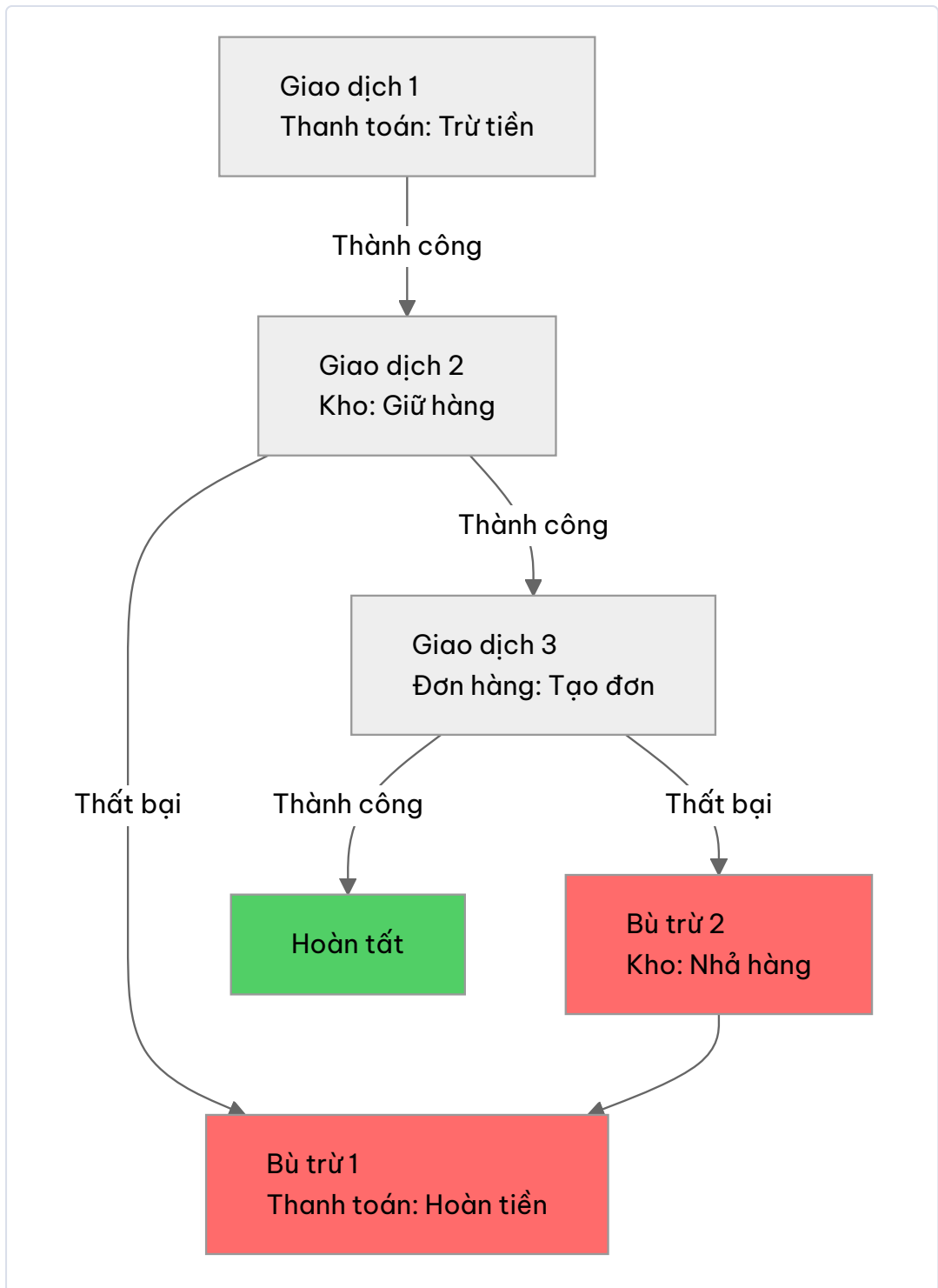
Cách Saga:

Giao dịch 1: Dịch vụ Thanh toán trừ tiền (commit cục bộ)

Giao dịch 2: Dịch vụ Kho giữ hàng (commit cục bộ)

Giao dịch 3: Dịch vụ Đơn hàng tạo đơn (commit cục bộ)

Nếu bất kỳ bước nào thất bại → Chạy các giao dịch bù trừ theo chiều ngược lại



Saga: Các giao dịch chạy xuôi + các bước bù trừ (compensations) chạy ngược nếu thất bại

Triển Khai Saga: Điều Phối Tập Trung (*Orchestration*)

Điều phối tập trung = Một bộ điều phối trung tâm điều khiển saga

```

class OrderSagaOrchestrator:
    def __init__(self):
        self.payment_service = PaymentService()
        self.inventory_service = InventoryService()
        self.order_service = OrderService()

    def create_order(self, order_data):
        saga_state = {
            'payment_id': None,
            'reservation_id': None,
            'order_id': None
        }

        try:
            # Step 1: Charge payment
            print("Step 1: Charging payment...")
            payment_id = self.payment_service.charge(
                order_data['amount']
            )
            saga_state['payment_id'] = payment_id
            print(f"✓ Payment charged: {payment_id}")

            # Step 2: Reserve inventory
            print("Step 2: Reserving inventory...")
            reservation_id = self.inventory_service.reserve(
                order_data['product_id'],
                order_data['quantity']
            )
            saga_state['reservation_id'] = reservation_id
            print(f"✓ Inventory reserved: {reservation_id}")

            # Step 3: Create order
            print("Step 3: Creating order...")
            order_id = self.order_service.create(order_data)
            saga_state['order_id'] = order_id
            print(f"✓ Order created: {order_id}")

            return {
                'status': 'success',
                'order_id': order_id
            }

        except Exception as e:
            print(f"\nSaga failed: {e}")
            print("Starting compensation...")

            # Compensate in reverse order
            self._compensate(saga_state)

            return {
                'status': 'failed',
                'error': str(e)
            }

```

```

def _compensate(self, saga_state):
    # Reverse order: Order → Inventory → Payment

    if saga_state['order_id']:
        print("Compensating: Cancel order")
        try:
            self.order_service.cancel(saga_state['order_id'])
            print("✓ Order cancelled")
        except Exception as e:
            print(f"△ Order compensation failed: {e}")

    if saga_state['reservation_id']:
        print("Compensating: Release inventory")
        try:
            self.inventory_service.release(saga_state['reservation_id'])
            print("✓ Inventory released")
        except Exception as e:
            print(f"△ Inventory compensation failed: {e}")

    if saga_state['payment_id']:
        print("Compensating: Refund payment")
        try:
            self.payment_service.refund(saga_state['payment_id'])
            print("✓ Payment refunded")
        except Exception as e:
            print(f"△ Payment compensation failed: {e}")

# Usage
orchestrator = OrderSagaOrchestrator()
result = orchestrator.create_order({
    'user_id': 123,
    'product_id': 456,
    'quantity': 2,
    'amount': 50.00
})

```

Kết quả khi thành công:

Bước 1: Đang trừ tiền thanh toán...
 ✓ Đã trừ tiền: payment_789

Bước 2: Đang giữ hàng trong kho...
 ✓ Đã giữ hàng: reservation_321

Bước 3: Đang tạo đơn hàng...
 ✓ Đã tạo đơn hàng: order_654

Kết quả: Thành công

Kết quả khi thất bại:

Bước 1: Đang trừ tiền thanh toán...
✓ Đã trừ tiền: payment_789

Bước 2: Đang giữ hàng trong kho...
Hết hàng!

Saga thất bại: Hết hàng

Bắt đầu bù trừ...
Đang bù trừ: Hoàn tiền thanh toán
✓ Đã hoàn tiền

Kết quả: Thất bại - Hết hàng

Triển Khai Saga: Phối Hợp Phân Tán (*Choreography*)

Phối hợp phân tán = Các dịch vụ tự phối hợp với nhau thông qua sự kiện (events)

```

# Payment Service
class PaymentService:
    def charge(self, amount):
        # Process payment
        payment_id = process_payment(amount)

        # Publish event
        event_bus.publish('payment.charged', {
            'payment_id': payment_id,
            'amount': amount
        })

        return payment_id

    @event_bus.subscribe('order.failed')
    def on_order_failed(self, event):
        # Compensate: Refund
        payment_id = event['payment_id']
        self.refund(payment_id)

        event_bus.publish('payment.refunded', {
            'payment_id': payment_id
        })

# Inventory Service
class InventoryService:
    @event_bus.subscribe('payment.charged')
    def on_payment_charged(self, event):
        try:
            # Reserve inventory
            reservation_id = self.reserve(event['product_id'])

            event_bus.publish('inventory.reserved', {
                'reservation_id': reservation_id,
                'payment_id': event['payment_id']
            })
        except OutOfStockError:
            # Trigger compensation
            event_bus.publish('order.failed', {
                'reason': 'out_of_stock',
                'payment_id': event['payment_id']
            })

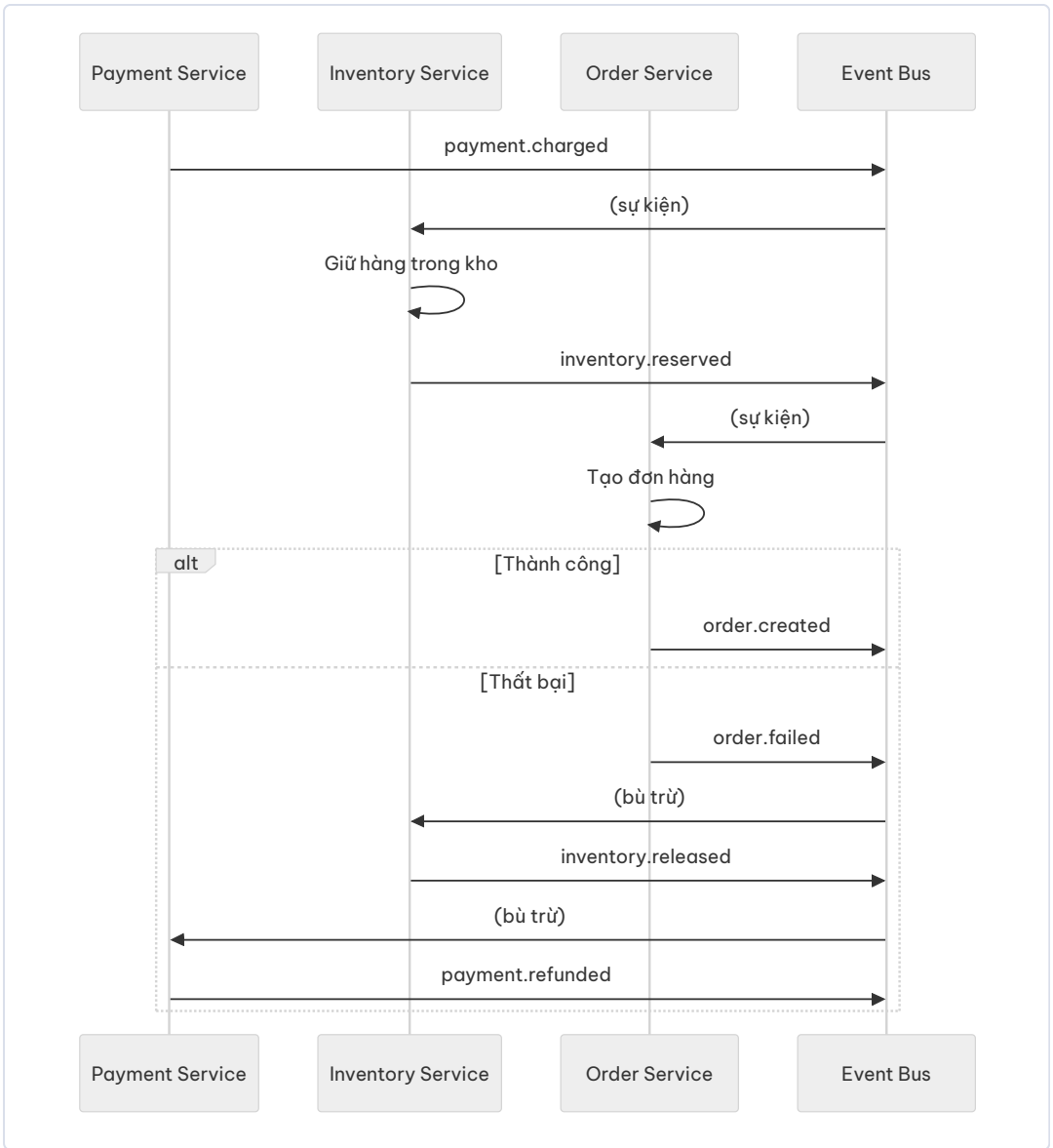
    @event_bus.subscribe('order.failed')
    def on_order_failed(self, event):
        # Compensate: Release inventory
        if event.get('reservation_id'):
            self.release(event['reservation_id'])

        event_bus.publish('inventory.released', {
            'reservation_id': event['reservation_id']
        })

```

```
# Order Service
class OrderService:
    @event_bus.subscribe('inventory.reserved')
    def on_inventory_reserved(self, event):
        try:
            # Create order
            order_id = self.create_order(event)

            event_bus.publish('order.created', {
                'order_id': order_id
            })
        except Exception as e:
            # Trigger compensation
            event_bus.publish('order.failed', {
                'reason': str(e),
                'payment_id': event['payment_id'],
                'reservation_id': event['reservation_id']
            })
```



Phối hợp phân tán: Các dịch vụ phản ứng với sự kiện, tự động bù trừ

Điều Phối Tập Trung So Với Phối Hợp Phân Tán

Orchestration (Bộ điều phối trung tâm):

Luồng rõ ràng, dễ gỡ lỗi

Logic tập trung

Thứ tự bù trừ tường minh

Điểm lỗi duy nhất

Liên kết chặt với bộ điều phối

Choreography (Hướng sự kiện):

Liên kết lỏng

Không có điểm lỗi duy nhất

Mở rộng được

Khó hiểu được luồng

Khó gỡ lỗi

Xử lý lỗi phức tạp

Khuyến nghị của tôi:

Bắt đầu với Orchestration:

- Dễ hiểu hơn
- Dễ gỡ lỗi hơn
- Trách nhiệm rõ ràng

Chuyển sang Choreography khi:

- Cần tách rời ở mức tối đa
- Đã có hạ tầng hướng sự kiện
- Nhóm có kinh nghiệm với các mẫu bất đồng bộ

Tư Duy Bù Trừ: Tư Duy (Mindset) Cốt Lõi

Bù Trừ ≠ Hoàn Tác

Hoàn tác trong cơ sở dữ liệu:

```
# Database rollback: Undo changes
db.begin_transaction()
db.execute("INSERT INTO orders ...")
db.rollback() # Changes disappear completely
# As if nothing happened
```

Bù trừ trong Saga:

```
# Saga compensation: Reverse effect
payment_id = payment_service.charge(100) # Money charged
# Later...
payment_service.refund(payment_id) # Money returned

# NOT same as rollback:
- Payment happened (visible in logs)
- Refund is separate transaction
- Both visible in audit trail
```

Khác biệt then chốt:

Hoàn tác (rollback): Huỷ bỏ → như chưa có gì xảy ra
 Bù trừ (compensation): Đảo ngược tác động → có nhật ký kiểm toán đầy đủ

Thiết Kế Các Giao Dịch Bù Trừ (Compensating Transactions)

Quy tắc 1: Bù trừ phải lũy đẳng (idempotent)

```
# BAD: Not idempotent
def refund_payment(payment_id):
    payment = get_payment(payment_id)
    payment.amount = 0 # If called twice → problem
    save(payment)

# GOOD: Idempotent
def refund_payment(payment_id):
    payment = get_payment(payment_id)

    if payment.status == 'refunded':
        return # Already refunded, skip

    payment.status = 'refunded'
    payment.refunded_at = now()
    save(payment)
```

Quy tắc 2: Bù trừ cũng có thể thất bại

```

def _compensate(self, saga_state):
    compensation_errors = []

    # Try each compensation
    for step in reversed(saga_state['completed_steps']):
        try:
            step.compensate()
        except Exception as e:
            # Log but continue
            compensation_errors.append({
                'step': step.name,
                'error': str(e)
            })
            logger.error(f"Compensation failed: {step.name} - {e}")

    # If any compensation failed → Manual intervention needed
    if compensation_errors:
        alert_ops_team({
            'saga_id': saga_state['id'],
            'failures': compensation_errors,
            'action_needed': 'manual_compensation'
        })

```

Quy tắc 3: Một số hành động không thể bù trừ (*compensated*)

```

# Cannot compensate:
Email sent (cannot "unsubscribe")
SMS sent
External API called (no control)
Physical shipment initiated

# Solutions:
Put compensable actions first
Non-compensable actions last (after all risks)
Make them idempotent (safe to retry)

```

Thứ Tự Bù Trừ Rất Quan Trọng

Thứ tự xuôi:

1. Trừ tiền thanh toán
2. Giữ hàng trong kho
3. Gửi email xác nhận ← Không thể bù trừ

Thứ tự bù trừ (NGƯỢC LẠI):

3. (Không thể thu hồi email - chấp nhận điều đó)
2. Nhả hàng trong kho
1. Hoàn tiền thanh toán

Tại sao lại ngược? Thay đổi mới nhất được hủy trước.

Tính Lũy Đẳng (*Idempotency*): Tấm Lưới An Toàn

Định Nghĩa Tính Lũy Đẳng

Thao tác lũy đẳng (*Idempotent operation*) = **Gọi nhiều lần = gọi 1 lần**

$f(x) = y$

$f(f(x)) = y$ ← Cùng kết quả

$f(f(f(x))) = y$ ← Vẫn cùng kết quả

Ví dụ:

- Đặt nhiệt độ thành 20°C → Lũy đẳng
- Tăng nhiệt độ thêm 1°C → KHÔNG lũy đẳng

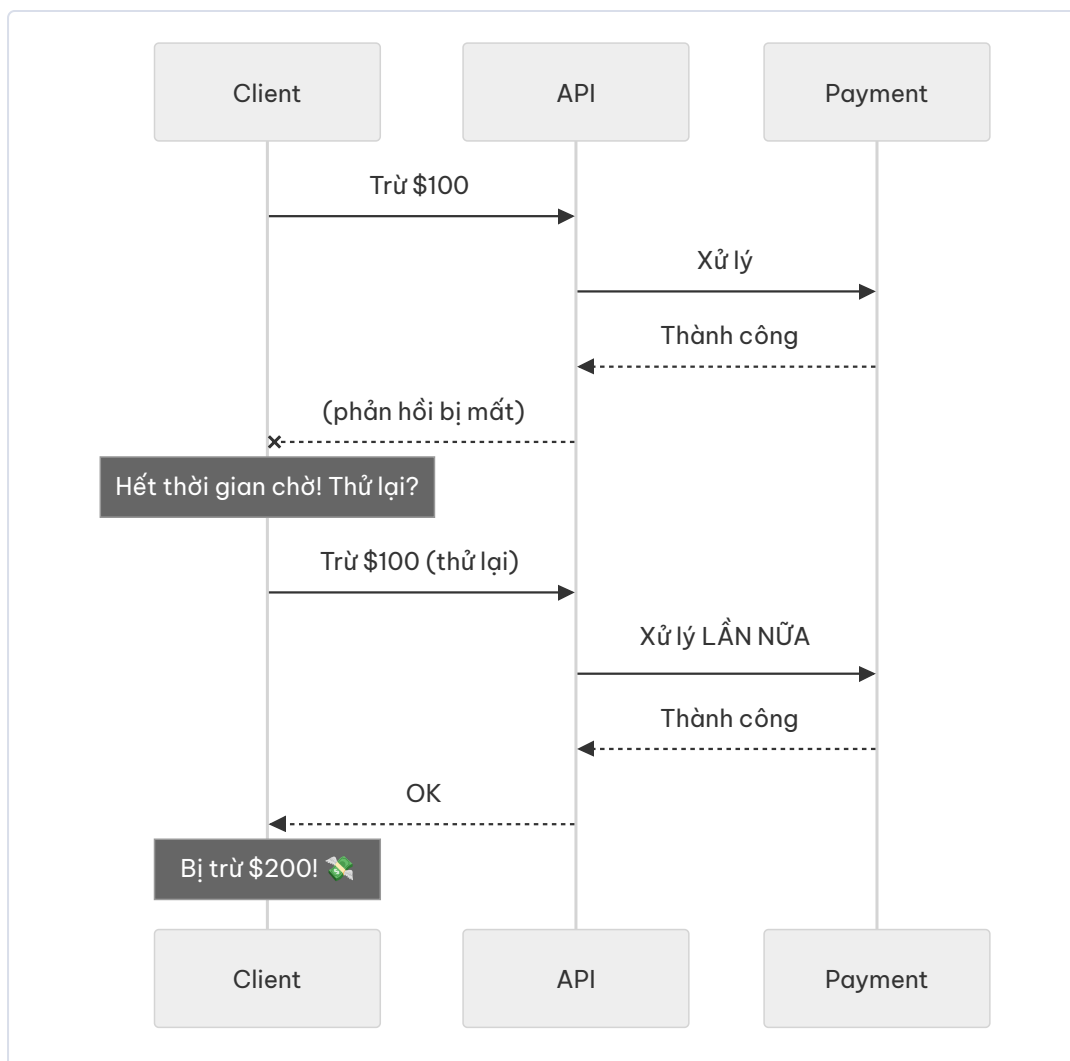
Tại Sao Cần Tính Lũy Đẳng?

Vấn đề: Mạng không đáng tin cậy

Diễn biến:

1. Máy khách gọi API: "Trừ \$100"
2. Máy chủ xử lý, trừ \$100
3. Máy chủ gửi phản hồi
4. Mạng lỗi trước khi máy khách nhận được phản hồi
5. Máy khách không biết đã thành công hay chưa
6. Máy khách thử lại: "Trừ \$100" lần nữa
7. Máy chủ trừ THÊM \$100 nữa

Kết quả: Khách hàng bị trừ \$200 thay vì \$100!



Mạng gặp sự cố (failure) → Thử lại (Retry) → Trừ tiền hai lần

Triển Khai Tính Lũy Đắc

Giải pháp 1: Khoá Lũy Đắc (Idempotency Key)

```

class PaymentService:
    def __init__(self):
        self.processed_requests = {} # In practice: Redis/DB

    def charge(self, amount, idempotency_key):
        # Check if already processed
        if idempotency_key in self.processed_requests:
            # Return cached result
            print(f"Duplicate request detected: {idempotency_key}")
            return self.processed_requests[idempotency_key]

        # First time seeing this request
        print(f"Processing new request: {idempotency_key}")

        # Process payment
        payment_id = self._process_payment(amount)

        # Cache result
        self.processed_requests[idempotency_key] = {
            'payment_id': payment_id,
            'amount': amount,
            'status': 'success'
        }

        return self.processed_requests[idempotency_key]

# Client usage
payment_service = PaymentService()

# Client generates unique key
idempotency_key = "order_123_payment_attempt_1"

# First call
result1 = payment_service.charge(100, idempotency_key)
# Processing new request: order_123_payment_attempt_1
# Result: payment_id=pay_789

# Retry (network failed, client retries)
result2 = payment_service.charge(100, idempotency_key)
# Duplicate request detected: order_123_payment_attempt_1
# Result: payment_id=pay_789 (SAME!)

# Charged only once ✓

```

Chi tiết triển khai:

```

# Production-grade implementation
class IdempotentPaymentService:
    def charge(self, amount, idempotency_key):
        # Check cache (Redis với TTL 24h)
        cached = redis.get(f"idempotency:{idempotency_key}")
        if cached:
            return json.loads(cached)

        # Acquire lock để prevent concurrent duplicates
        lock_key = f"lock:idempotency:{idempotency_key}"
        with redis.lock(lock_key, timeout=30):
            # Double-check sau khi có lock
            cached = redis.get(f"idempotency:{idempotency_key}")
            if cached:
                return json.loads(cached)

            # Process payment
            result = self._process_payment(amount)

            # Cache result với TTL 24h
            redis.setex(
                f"idempotency:{idempotency_key}",
                86400, # 24 hours
                json.dumps(result)
            )

        return result

```

Giải pháp 2: Lũy Đẳng Tự Nhiên (Natural Idempotency)

```

# NOT idempotent
def add_item_to_cart(user_id, product_id):
    cart = get_cart(user_id)
    cart.items.append(product_id) # Duplicate if called twice
    save(cart)

# Idempotent
def add_item_to_cart(user_id, product_id):
    cart = get_cart(user_id)
    if product_id not in cart.items: # Check first
        cart.items.append(product_id)
    save(cart)

# Even better: Use Set
def add_item_to_cart(user_id, product_id):
    cart = get_cart(user_id)
    cart.items.add(product_id) # Set naturally idempotent
    save(cart)

```

Giải pháp 3: Ràng Buộc (Constraints) Của Cơ Sở Dữ Liệu

```
-- Allows duplicates
CREATE TABLE payments (
  id SERIAL PRIMARY KEY,
  order_id INT,
  amount DECIMAL
);

-- Can insert same order_id multiple times!

-- Prevent duplicates
CREATE TABLE payments (
  id SERIAL PRIMARY KEY,
  order_id INT UNIQUE, -- Unique constraint
  amount DECIMAL
);

-- Second insert with same order_id → Error
-- Application catches error → Return cached result
```

Thực Hành Tốt Nhất (Best Practices) Về Tính Lũy Đăng

1. Sinh khoá lũy đăng (idempotency keys) ở phía máy khách (client)

```
// Client generates key
const idempotencyKey = `${userId}_${orderId}_${Date.now()}_${uuidv4()}`;

fetch('/api/payment/charge', {
  method: 'POST',
  headers: {
    'Idempotency-Key': idempotencyKey
  },
  body: JSON.stringify({ amount: 100 })
});
```

2. Lưu kết quả lũy đăng kèm TTL

Đừng lưu mãi mãi → Lãng phí dung lượng lưu trữ
TTL 24-48 giờ thường là đủ
Sau TTL, yêu cầu mới = thao tác mới (chấp nhận được)

3. Trả về cùng một mã trạng thái cho các yêu cầu (requests) lũy đăng

```

# BAD
if duplicate_request:
    return 200, cached_result

# GOOD
if duplicate_request:
    return 200, cached_result # Same as original
else:
    result = process()
    return 200, result # Same status code

```

4. Tính lũy đẳng cho cả bù trừ

```

# Compensation cũng phải idempotent
def compensate_payment(payment_id, idempotency_key):
    # Check if already compensated
    if already_refunded(payment_id):
        return cached_result

    # Refund
    refund_payment(payment_id)

    # Cache compensation result
    cache_result(idempotency_key, result)

```

Ví Dụ Thực Tế: Thanh Toán Đơn Hàng Thương Mại Điện Tử

Hãy xem một ví dụ hoàn chỉnh áp dụng tất cả các khái niệm:

```

class CheckoutSaga:
    def __init__(self):
        self.payment_service = PaymentService()
        self.inventory_service = InventoryService()
        self.order_service = OrderService()
        self.shipping_service = ShippingService()

    def execute(self, checkout_data, idempotency_key):
        # Check idempotency
        cached = self._check_cache(idempotency_key)
        if cached:
            return cached

        saga_state = {
            'steps_completed': [],
            'checkout_id': checkout_data['checkout_id']
        }

        try:
            # Step 1: Reserve inventory (compensable)
            reservation = self.inventory_service.reserve(
                checkout_data['items'],
                idempotency_key=f"{idempotency_key}_inventory"
            )
            saga_state['steps_completed'].append({
                'name': 'inventory',
                'data': reservation,
                'compensate': lambda:
self.inventory_service.release(reservation['id'])
            })

            # Step 2: Charge payment (compensable)
            payment = self.payment_service.charge(
                checkout_data['amount'],
                idempotency_key=f"{idempotency_key}_payment"
            )
            saga_state['steps_completed'].append({
                'name': 'payment',
                'data': payment,
                'compensate': lambda:
self.payment_service.refund(payment['id'])
            })

            # Step 3: Create order (compensable)
            order = self.order_service.create(
                checkout_data,
                idempotency_key=f"{idempotency_key}_order"
            )
            saga_state['steps_completed'].append({
                'name': 'order',
                'data': order,
                'compensate': lambda: self.order_service.cancel(order['id'])
            })

```

```

# Step 4: Create shipment (NOT compensable - put last)
shipment = self.shipping_service.create(
    order['id'],
    idempotency_key=f"{idempotency_key}_shipment"
)
# No compensation - cannot undo physical shipment

result = {
    'status': 'success',
    'order_id': order['id'],
    'shipment_id': shipment['id']
}

# Cache result
self._cache_result(idempotency_key, result)

return result

except Exception as e:
    logger.error(f"Saga failed: {e}")

# Compensate in reverse order
self._compensate(saga_state)

return {
    'status': 'failed',
    'error': str(e)
}

def _compensate(self, saga_state):
    # Reverse order
    for step in reversed(saga_state['steps_completed']):
        try:
            logger.info(f"Compensating: {step['name']}")
            step['compensate']()
            logger.info(f"✓ Compensated: {step['name']}")
        except Exception as e:
            logger.error(f"Compensation failed: {step['name']} - {e}")
            # Alert ops team
            alert_ops_team({
                'saga_id': saga_state['checkout_id'],
                'step': step['name'],
                'error': str(e)
            })

```

Những Điểm Cốt Lỗi

ACID bị phá vỡ trong hệ thống phân tán:

Một cơ sở dữ liệu: ACID hoạt động
Nhiều cơ sở dữ liệu/dịch vụ: ACID là bất khả thi
Mạng không đáng tin cậy → Không thể đảm bảo tính nguyên tử

2-Phase Commit (cam kết hai pha):

Nhất quán mạnh
Giữ được ACID
Chặn (điểm yếu chí mạng)
Bộ điều phối gặp sự cố = thảm họa
Tránh dùng trong vi dịch vụ

Mẫu Saga:

Không chặn
Tính sẵn sàng cao
Thực tế cho vi dịch vụ
Nhất quán cuối cùng
Độ phức tạp của việc bù trừ

Điều phối tập trung so với phối hợp phân tán:

Orchestration: Bộ điều phối trung tâm (đơn giản hơn, dễ gỡ lỗi hơn)
Choreography: Hướng sự kiện (tách rời, phức tạp)

Bắt đầu với Orchestration

Tư duy bù trừ:

- Bù trừ ≠ Hoàn tác
- Đảo ngược tác động, không phải huỷ bỏ
- Phải lũy đẳng
- Một số hành động không thể bù trừ (đặt chúng cuối cùng)
- Thứ tự rất quan trọng (thứ tự ngược)

Tính lũy đẳng:

- Thiết yếu cho hệ thống phân tán
- Ngăn chặn thao tác trùng lặp
- Dùng khoá lũy đẳng
- Áp dụng cho MỌI thao tác (kể cả bù trừ)
- Lưu kết quả kèm TTL

Khung ra quyết định (Decision framework):

Cần ACID + Chịu được việc bị chặn:
→ 2-Phase Commit (hiếm)

Cần tính sẵn sàng cao + Chấp nhận nhất quán cuối cùng:
→ Mẫu Saga (phổ biến)

Triển khai Saga:

- Bắt đầu đơn giản: Orchestration
- Mọi thao tác: Lũy đẳng
- Hành động không thể bù trừ: Đặt cuối cùng
- Giám sát các lần bù trừ thất bại

Câu hỏi tự kiểm tra:

Khi thiết kế giao dịch phân tán:

1. Có thực sự cần nhất quán mạnh (*strong consistency*) không?
2. Có thể chấp nhận nhất quán cuối cùng (*eventual consistency*) không?
3. Thao tác nào bù trừ (*compensable*) được, thao tác nào không?
4. Đã triển khai tính lũy đẳng chưa?
5. Thứ tự bù trừ đã đúng chưa (ngược lại)?

Giao dịch phân tán rất khó. Nhưng với Saga + tính lũy đẳng, bạn có đủ công cụ để xử lý chúng một cách đúng đắn.

Hãy nhớ: Không thể đạt được tính nhất quán (*consistency*) hoàn hảo trong hệ thống phân tán. Mục tiêu là đảm bảo nhất quán cuối cùng với các lộ trình bù trừ rõ ràng.

ID Phân Tán Và Thứ Tự (*Ordering*): Vấn Đề Định Danh Trong Hệ Thống Phân Tán

Tôi còn nhớ lỗi (*bug*) đầu tiên khiến tôi phải thức trắng đêm vì hệ thống phân tán (*distributed systems*).

Chúng tôi vừa chuyển đổi (*migrate*) từ hệ thống nguyên khối (*monolith*) sang vi dịch vụ (*microservices*). Mọi thứ chạy ngon lành trên máy cá nhân. Triển khai (*Deploy*) lên môi trường thực tế (*production*)... hỗn loạn.

Các đơn hàng bị trùng ID. Người dùng (*Users*) phàn nàn "tôi đặt 1 món nhưng thấy 2 đơn hàng". Các ràng buộc (*constraints*) của cơ sở dữ liệu (*database*) liên tục báo lỗi.

Nguyên nhân gốc rễ? ID tự tăng (*auto-increment*).

Một kiến trúc sư lâu năm (*senior architect*) nhìn tôi: "Em biết tại sao tự tăng không hoạt động trong hệ thống phân tán chưa?"

Tôi: "... Chưa ạ."

Đó là đêm tôi học về việc sinh ID phân tán (*distributed ID generation*). Và đó là một trong những vấn đề tinh vi nhất nhưng cũng quan trọng nhất trong hệ thống phân tán.

Tại Sao Tự Tăng Thất Bại?

Hệ Thống Nguyên Khối: Tự Tăng Hoạt Động Hoàn Hảo

```
-- Single database
CREATE TABLE orders (
  id SERIAL PRIMARY KEY, -- Auto-increment
  user_id INT,
  amount DECIMAL
);

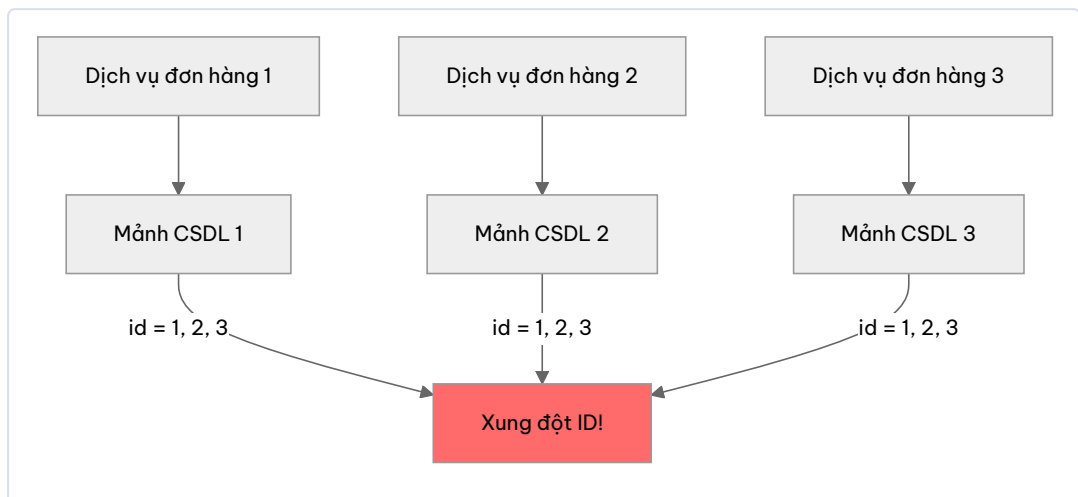
-- Insert order 1 → id = 1
-- Insert order 2 → id = 2
-- Insert order 3 → id = 3

Perfect! Đơn giản, sequential, unique.
```

Tại sao nó hoạt động:

Một cơ sở dữ liệu duy nhất = Một nguồn sự thật duy nhất
Bộ máy cơ sở dữ liệu đảm bảo tính duy nhất
Cơ chế khoá ngăn chặn trùng lặp
Đơn giản vô cùng

Hệ Thống Phân Tán: Tự Tăng Thất Bại



Ba mảnh (shards) cơ sở dữ liệu độc lập, mỗi mảnh tự sinh ID tự tăng riêng → Xung đột!

Vấn đề:

Mảnh CSDL 1: id = 1, 2, 3, 4, ...
Mảnh CSDL 2: id = 1, 2, 3, 4, ... (TRÙNG LẶP!)
Mảnh CSDL 3: id = 1, 2, 3, 4, ... (TRÙNG LẶP!)

- Cùng một ID trên các mảnh khác nhau
- Gộp dữ liệu = thảm hoạ
- Khoá ngoại bị hỏng
- Khoá bộ nhớ đệm xung đột

Tác động trong thực tế:

Tôi từng thấy một trang thương mại điện tử dính lỗi này:

- Người dùng A đặt sản phẩm X → id = 123 (mảnh (*shard*) 1)
- Người dùng B đặt sản phẩm Y → id = 123 (mảnh 2)
- Hệ thống gộp các đơn hàng → Xung đột
- Người dùng A nhận sản phẩm của Người dùng B
- Thảm hoạ!

Bài học rút ra: Tự tăng chỉ hoạt động với một cơ sở dữ liệu duy nhất. Hệ thống phân tán cần cách tiếp cận khác.

Các Yêu Cầu (*Requirements*) Đối Với ID Phân Tán

Trước khi đi vào các giải pháp, hãy hiểu các yêu cầu.

Bắt buộc phải có:

1. Duy nhất trên toàn cục
 - ID phải là duy nhất trên TẤT CẢ các dịch vụ/mảnh
 - Không xung đột dù được sinh ra ở đâu
2. Tính sẵn sàng cao
 - Sinh ID không cần phối hợp giữa các nút
 - Không có điểm lỗi duy nhất
 - Vẫn hoạt động khi xảy ra phân vùng mạng
3. Hiệu năng cao
 - Sinh nhanh (< 1ms)
 - Không chặn
 - Hàng triệu ID/giây

Nên có:

4. Gần đúng theo thứ tự thời gian
 - ID mới > ID cũ
 - Hữu ích cho việc sắp xếp, gỡ lỗi
 - Không cần thứ tự hoàn hảo
5. Gọn nhẹ
 - 64 bit thay vì 128 bit
 - Tiết kiệm lưu trữ, bộ nhớ
 - So sánh nhanh hơn
6. Con người đọc được
 - Có thể gỡ lỗi
 - Trích xuất siêu dữ liệu (dấu thời gian, machine ID)

Thực tế của sự đánh đổi:

Không có giải pháp nào thoả mãn được tất cả. Phải lựa chọn dựa trên thứ tự ưu tiên.

Giải Pháp 1: UUID (Universally Unique Identifier - Định Danh Duy Nhất Toàn Cục)

Khái Niệm

UUID = số ngẫu nhiên 128 bit, được đảm bảo duy nhất trên toàn cục

Định dạng: 8-4-4-4-12 chữ số thập lục phân
Ví dụ: 550e8400-e29b-41d4-a716-446655440000

Tổng số UUID có thể có: $2^{128} \approx 3.4 \times 10^{38}$
(Nhiều hơn số nguyên tử trong vũ trụ!)

Các phiên bản:

UUID v1: Dấu thời gian + địa chỉ MAC
UUID v4: Ngẫu nhiên (phổ biến nhất)
UUID v7: Sắp theo dấu thời gian (mới nhất, tốt nhất cho cơ sở dữ liệu)

Cài Đặt

```
import uuid

# Generate UUID v4 (random)
id = uuid.uuid4()
print(id) # 550e8400-e29b-41d4-a716-446655440000

# In database
CREATE TABLE orders (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id INT,
    amount DECIMAL
);

# Application code
def create_order(user_id, amount):
    order_id = uuid.uuid4()
    db.execute("""
        INSERT INTO orders (id, user_id, amount)
        VALUES (?, ?, ?)
    """, [order_id, user_id, amount])
    return order_id
```

Không cần phối hợp:

Dịch vụ 1 sinh ra: 550e8400-e29b-41d4-a716-446655440000
Dịch vụ 2 sinh ra: 7c9e6679-7425-40de-944b-e07fc1f90ae7
Dịch vụ 3 sinh ra: 9e107d9d-372b-4d16-a86a-3a0f8e4f0c3d

Tất cả đều duy nhất, không xung đột!

Sự Đánh Đổi (Trade-offs)

Ưu điểm:

- Cực kỳ đơn giản để cài đặt
- Không cần phối hợp
- Thực sự duy nhất trên toàn cục
- Hoạt động cả khi ngoại tuyến
- Là tiêu chuẩn của ngành

Nhược điểm:

- 128 bit (lớn gấp đôi 64 bit)
- Không sắp xếp được (thứ tự ngẫu nhiên)
- Hiệu năng cơ sở dữ liệu kém (chèn ngẫu nhiên)
- Con người khó đọc
- Khó gỡ lỗi

Tác Động Đến Hiệu Năng (*Performance*) Cơ Sở Dữ Liệu

Vấn đề với UUID ngẫu nhiên:

Cấu trúc chỉ mục B-tree:

```
      [middle]
     /      \
  [left]    [right]
```

Chèn tuần tự (tự tăng):

- Luôn chèn vào lá ngoài cùng bên phải
- Không phải tách trang
- Nhanh!

UUID ngẫu nhiên:

- Chèn ngẫu nhiên khắp cây
- Tách trang thường xuyên
- Phân mảnh dữ liệu trên đĩa
- Chèn chậm hơn 2-3 lần

Đo hiệu năng (PostgreSQL):

```
Auto-increment INT: 100,000 lượt chèn = 2.3 giây
UUID v4:             100,000 lượt chèn = 7.1 giây (chậm hơn 3 lần)
UUID v7:             100,000 lượt chèn = 2.8 giây (gần bằng INT!)
```

Giải pháp: UUID v7 (sắp theo dấu thời gian)

```
# UUID v7: Time-ordered, better for databases
import uuid_utils as uuid

id = uuid.uuid7()
# 018e8e7a-0000-7000-8000-000000000000
# ^^^^^^^^ - timestamp (sortable!)
```

Khi Nào Dùng UUID?

Sử dụng UUID khi:

- ✓ Sự đơn giản quan trọng nhất
- ✓ Không cần sắp xếp theo ID
- ✓ Lưu trữ/bộ nhớ không phải là ràng buộc
- ✓ Muốn dùng tiêu chuẩn của ngành
- ✓ Cần ID thực sự ngẫu nhiên (bảo mật)

Tránh UUID khi:

- ✗ Hiệu năng cơ sở dữ liệu là yếu tố sống còn
- ✗ Cần ID sắp theo thời gian
- ✗ Bị ràng buộc về lưu trữ (di động, IoT)
- ✗ Cần ID con người đọc được

Quan điểm cá nhân:

UUID là lựa chọn mặc định an toàn. Khi không chắc nên dùng gì, hãy dùng UUID (tốt nhất là v7).

Chỉ tối ưu theo hướng khác khi có những yêu cầu cụ thể.

Giải Pháp 2: Twitter Snowflake

Khái Niệm

Snowflake = ID 64 bit, sắp theo thời gian, duy nhất trên toàn cục

Cấu trúc 64 bit:

1	41 bit	10 bit	12 bit
	dấu thời gian	ID	số thứ tự
	(mili giây)	máy	bộ đếm
0			

Phân tích chi tiết:

Bit 0 (1 bit):

- Luôn là 0 (dự trữ, bit dấu)

Bit 1-41 (41 bit): Dấu thời gian

- Số mili giây kể từ mốc epoch (epoch tùy chỉnh)
- $2^{41} \text{ ms} \approx 69 \text{ năm}$
- Sắp xếp được theo thời gian!

Bit 42-51 (10 bit): Machine ID

- Xác định máy nào đã sinh ra ID
- Hỗ trợ $2^{10} = 1,024$ máy
- Được cấu hình khi khởi động

Bit 52-63 (12 bit): Số thứ tự

- Bộ đếm trong cùng một mili giây
- $2^{12} = 4,096$ ID mỗi mili giây trên mỗi máy
- Đặt lại mỗi khi sang mili giây mới

Tính toán năng lực:

Mỗi máy:

- 4,096 ID/mili giây
- 4,096,000 ID/giây
- ~350 tỷ ID/ngày

1,024 máy:

- 4.2 triệu ID/giây trên toàn hệ thống
- 360 nghìn tỷ ID/ngày

Quá đủ!

Cài Đặt

```
import time
import threading

class SnowflakeGenerator:
    def __init__(self, machine_id, datacenter_id=0):
        # Custom epoch (Jan 1, 2020)
        self.epoch = 1577836800000 # milliseconds

        # Bit lengths
        self.machine_id_bits = 5
        self.datacenter_id_bits = 5
        self.sequence_bits = 12

        # Max values
        self.max_machine_id = (1 << self.machine_id_bits) - 1
        self.max_datacenter_id = (1 << self.datacenter_id_bits) - 1
        self.max_sequence = (1 << self.sequence_bits) - 1

        # Current state
        self.machine_id = machine_id
        self.datacenter_id = datacenter_id
        self.sequence = 0
        self.last_timestamp = -1

        # Thread safety
        self.lock = threading.Lock()

        # Validate
        if machine_id > self.max_machine_id or machine_id < 0:
            raise ValueError(f"machine_id must be 0-{self.max_machine_id}")

    def _current_millis(self):
        return int(time.time()) * 1000

    def _wait_next_millis(self, last_timestamp):
        timestamp = self._current_millis()
        while timestamp <= last_timestamp:
            timestamp = self._current_millis()
        return timestamp

    def generate_id(self):
        with self.lock:
            timestamp = self._current_millis()

            # Same millisecond
            if timestamp == self.last_timestamp:
                self.sequence = (self.sequence + 1) & self.max_sequence

            # Sequence overflow, wait next millisecond
            if self.sequence == 0:
                timestamp = self._wait_next_millis(self.last_timestamp)
```

```

        else:
            # New millisecond, reset sequence
            self.sequence = 0

            # Clock moved backwards (rare but possible)
            if timestamp < self.last_timestamp:
                raise Exception(f"Clock moved backwards. Refusing to generate
id")

            self.last_timestamp = timestamp

            # Combine all parts
            id = ((timestamp - self.epoch) << (self.datacenter_id_bits +
                                                self.machine_id_bits +
                                                self.sequence_bits))

            id |= (self.datacenter_id << (self.machine_id_bits +
self.sequence_bits))
            id |= (self.machine_id << self.sequence_bits)
            id |= self.sequence

            return id

# Usage
generator = SnowflakeGenerator(machine_id=1, datacenter_id=1)

id1 = generator.generate_id() # 1234567890123456789
id2 = generator.generate_id() # 1234567890123456790
id3 = generator.generate_id() # 1234567890123456791

# Properties
print(id1 < id2 < id3) # True - sortable!
print(len(str(id1)))   # 19 digits (64-bit)

```

Trích Xuất Siêu Dữ Liệu

```
def parse_snowflake(id):
    """Extract timestamp, machine_id, sequence từ Snowflake ID"""

    # Constants
    epoch = 1577836800000
    sequence_bits = 12
    machine_id_bits = 10

    # Extract sequence
    sequence = id & ((1 << sequence_bits) - 1)

    # Extract machine ID
    id >>= sequence_bits
    machine_id = id & ((1 << machine_id_bits) - 1)

    # Extract timestamp
    id >>= machine_id_bits
    timestamp_ms = id + epoch

    return {
        'timestamp': timestamp_ms,
        'datetime': datetime.fromtimestamp(timestamp_ms / 1000),
        'machine_id': machine_id,
        'sequence': sequence
    }

# Parse ID
info = parse_snowflake(1234567890123456789)
print(info)
# {
#   'timestamp': 1709567890123,
#   'datetime': '2024-03-04 12:34:50',
#   'machine_id': 1,
#   'sequence': 789
# }
```

Sự Đánh Đổi

Ưu điểm:

- 64 bit (gọn, hiệu quả)
- Sắp theo thời gian (sắp xếp được, tốt cho CSDL)
- Hiệu năng cao (không cần phối hợp)
- Phi tập trung (không có điểm lỗi duy nhất)
- Con người gỡ lỗi được (trích xuất dấu thời gian)
- Đã được ngành kiểm chứng (Twitter, Discord, Instagram)

Nhược điểm:

- Phải quản lý Machine ID (cần cấu hình)
- Bắt buộc đồng bộ đồng hồ (NTP)
- Giới hạn 69 năm (cần đặt lại epoch)
- Cận số thứ tự trong cùng một mili giây (hiếm gặp)

Vấn Đề Lệch Đồng Hồ (Clock Skew)

Vấn đề:

Đồng hồ máy A: 10:00:00.000

Đồng hồ máy B: 10:00:05.000 (nhanh hơn 5 giây)

Máy B sinh ra: ID = 1000005000...

Máy A sinh ra: ID = 1000000000... (nhỏ hơn!)

ID từ máy A < ID từ máy B về giá trị
nhưng lại được sinh ra SAU về thời gian!

→ Thứ tự bị phá vỡ

Giải pháp:

1. Đồng bộ NTP (khuyến nghị)
 - Giữ các đồng hồ đồng bộ (độ lệch < 100ms)
 - Giám sát độ lệch đồng hồ
 - Cảnh báo nếu độ lệch > ngưỡng
2. Từ chối sinh ID nếu đồng hồ chạy lùi
 - Kiểm tra: `current_time < last_time`
 - Báo lỗi hoặc chờ
 - Đánh đổi: Tính sẵn sàng giảm
3. Dùng đồng hồ logic (Lamport, Vector)
 - Không phụ thuộc vào đồng hồ thực
 - Cài đặt phức tạp

Khi Nào Dùng Snowflake?

Sử dụng Snowflake khi:

- ✓ Cần ID sắp theo thời gian
- ✓ Hiệu năng cơ sở dữ liệu là yếu tố sống còn
- ✓ Cần ID 64 bit gọn nhẹ
- ✓ Cần ID con người gỡ lỗi được
- ✓ Thông lượng cao (hàng triệu ID/giây)
- ✓ Có thể quản lý Machine ID

Tránh Snowflake khi:

- ✗ Không thể đồng bộ đồng hồ (NTP không tin cậy)
- ✗ Sự đơn giản > Hiệu năng
- ✗ Không muốn quản lý Machine ID
- ✗ Cần ID thực sự ngẫu nhiên

Trải nghiệm cá nhân:

Snowflake là lựa chọn hàng đầu cho các hệ thống quy mô (scale) lớn. Twitter, Discord, Instagram đều dùng nó.

Tôi khuyên dùng Snowflake cho:

- Các API có lưu lượng (traffic) cao
- Dữ liệu chuỗi thời gian (time-series)
- Hệ thống ghi log (logging)
- Lưu trữ dựa trên sự kiện (Event sourcing)

So Sánh: UUID Và Snowflake

	UUID	Snowflake
Kích thước	128-bit	64-bit
Tính duy nhất	Đảm bảo	Đảm bảo
Sắp xếp được	Không	Có
Hiệu năng CSDL	Trung bình	Cao
Triển khai	Đơn giản	Trung bình
Cấu hình	Không cần	Machine ID
Phụ thuộc	Không có	Đồng bộ NTP
Đễ đọc	Kém	Tốt
Mức phổ biến	Rất cao	Cao

Khung ra quyết định (Decision framework):

```
def choose_id_strategy(requirements):
    if requirements.simplicity_critical:
        return "UUID"

    if requirements.need_ordering:
        return "Snowflake"

    if requirements.db_performance_critical:
        return "Snowflake"

    if requirements.cannot_manage_machine_ids:
        return "UUID"

    if requirements.cannot_sync_clocks:
        return "UUID"

    # Default
    return "UUID" # Safest choice
```

Vấn Đề Thứ Tự Sự Kiện (*Event Ordering*)

Việc sinh ID giải quyết được tính duy nhất. Nhưng còn thứ tự thì sao?

Phát Biểu Vấn Đề

Dịch vụ A (Máy 1):

- Sự kiện 1 xảy ra lúc 10:00:00.000 → Sinh ID = 100
- Sự kiện 2 xảy ra lúc 10:00:00.001 → Sinh ID = 101

Dịch vụ B (Máy 2, đồng hồ nhanh hơn 5ms):

- Sự kiện 3 xảy ra lúc 10:00:00.003 → Sinh ID = 108

Dòng thời gian:

Sự kiện 1 (10:00:00.000) → ID = 100

Sự kiện 2 (10:00:00.001) → ID = 101

Sự kiện 3 (10:00:00.003) → ID = 108

Sắp xếp theo ID:

100 < 101 < 108 ✓ Đúng thứ tự

Nhưng nếu đồng hồ Máy 2 bị lệch:

Sự kiện 1 (10:00:00.000) → ID = 100

Sự kiện 3 (10:00:00.003) → ID = 95 (đồng hồ Máy 2 chạy chậm!)

Sự kiện 2 (10:00:00.001) → ID = 101

Sắp xếp theo ID:

95 < 100 < 101 ✗ Sai thứ tự! (Sự kiện 3 phải ở cuối)

Tại Sao Thứ Tự Quan Trọng

Ví dụ 1: Giao dịch ngân hàng

Sự kiện 1: Nạp \$100 (số dư = \$100)

Sự kiện 2: Rút \$50 (số dư = \$50)

Sự kiện 3: Rút \$40 (số dư = \$10)

Đúng thứ tự: \$0 → \$100 → \$50 → \$10 ✓

Sai thứ tự:

Sự kiện 2 trước → Không đủ số dư! ✗

Sự kiện 3 trước → Không đủ số dư! ✗

Ví dụ 2: Mạng xã hội

Bài đăng của người dùng:

Bài 1: "Giới thiệu dự án"

Bài 2: "Cập nhật tiến độ"

Bài 3: "Kết quả cuối cùng"

Đúng thứ tự: Mạch nội dung hợp lý ✓

Sai thứ tự: "Kết quả" xuất hiện trước "Giới thiệu" ✗

→ Người đọc bối rối

Các Giải Pháp Cho Bài Toán Thứ Tự

Giải pháp 1: Lamport Timestamps (dấu thời gian Lamport)


```

class LamportClock:
    def __init__(self):
        self.time = 0
        self.lock = threading.Lock()

    def tick(self):
        """Increment trước mỗi event"""
        with self.lock:
            self.time += 1
            return self.time

    def update(self, received_time):
        """Update khi nhận message từ process khác"""
        with self.lock:
            self.time = max(self.time, received_time) + 1
            return self.time

# Process A
clock_a = LamportClock()
time_1 = clock_a.tick() # 1
time_2 = clock_a.tick() # 2

# Process B nhận message từ A với time = 2
clock_b = LamportClock()
time_3 = clock_b.update(2) # max(0, 2) + 1 = 3
time_4 = clock_b.tick()    # 4

# Guarantee: time_1 < time_2 < time_3 < time_4

```

Đặc tính:

Bảo toàn quan hệ xảy-ra-trước (happened-before)
 Không cần đồng hồ thực
 Thuật toán đơn giản

Không biết thời gian tuyệt đối
 Không phát hiện được các sự kiện đồng thời
 Cần truyền thông điệp

Giải pháp 2: Vector Clocks (đồng hồ vector)

```

class VectorClock:
    def __init__(self, process_id, num_processes):
        self.process_id = process_id
        self.clock = [0] * num_processes
        self.lock = threading.Lock()

    def tick(self):
        """Increment own counter"""
        with self.lock:
            self.clock[self.process_id] += 1
            return self.clock.copy()

    def update(self, received_clock):
        """Merge với received vector clock"""
        with self.lock:
            for i in range(len(self.clock)):
                self.clock[i] = max(self.clock[i], received_clock[i])
            self.clock[self.process_id] += 1
            return self.clock.copy()

    def compare(self, other_clock):
        """So sánh 2 vector clocks"""
        less = all(self.clock[i] <= other_clock[i] for i in
range(len(self.clock)))
        greater = all(self.clock[i] >= other_clock[i] for i in
range(len(self.clock)))

        if less and not greater:
            return "BEFORE" # self happened before other
        elif greater and not less:
            return "AFTER" # self happened after other
        elif self.clock == other_clock:
            return "EQUAL"
        else:
            return "CONCURRENT" # Cannot determine order

# 3 processes
clock_a = VectorClock(0, 3) # Process A
clock_b = VectorClock(1, 3) # Process B
clock_c = VectorClock(2, 3) # Process C

# Process A events
event_1 = clock_a.tick() # [1, 0, 0]
event_2 = clock_a.tick() # [2, 0, 0]

# Process B nhận message từ A
event_3 = clock_b.update(event_2) # [2, 1, 0]

# Process C độc lập
event_4 = clock_c.tick() # [0, 0, 1]

```

```
# Compare  
clock_b.compare(event_4) # "CONCURRENT" - cannot order!
```

Giải pháp 3: Hybrid Logical Clocks - HLC (đồng hồ logic lai)

Kết hợp ưu điểm của cả hai: Thời gian vật lý + Bộ đếm logic

```

class HybridLogicalClock:
    def __init__(self):
        self.pt = 0 # Physical time
        self.lc = 0 # Logical counter
        self.lock = threading.Lock()

    def now(self):
        """Generate timestamp cho event"""
        with self.lock:
            pt_new = self._physical_time()

            if pt_new > self.pt:
                self.pt = pt_new
                self.lc = 0
            else:
                self.lc += 1

            return (self.pt, self.lc)

    def update(self, received_pt, received_lc):
        """Update khi nhận message"""
        with self.lock:
            pt_new = self._physical_time()

            if received_pt > self.pt and received_pt > pt_new:
                self.pt = received_pt
                self.lc = received_lc + 1
            elif received_pt == pt_new and received_pt == self.pt:
                self.lc = max(self.lc, received_lc) + 1
            elif pt_new > self.pt and pt_new > received_pt:
                self.pt = pt_new
                self.lc = 0
            else:
                self.lc += 1

            return (self.pt, self.lc)

    def _physical_time(self):
        return int(time.time() * 1000) # milliseconds

# Usage
hlc = HybridLogicalClock()

event_1 = hlc.now() # (1709567890123, 0)
event_2 = hlc.now() # (1709567890124, 0)
event_3 = hlc.now() # (1709567890124, 1) - same ms, increment lc

# Compare
event_1 < event_2 < event_3 # ✓ Total order preserved

```

Sự đánh đổi:

Lamport:
Đơn giản
Thứ tự từng phần
Không phát hiện được tính đồng thời

Vector:
Phát hiện được tính đồng thời
Đầy đủ quan hệ nhân quả
Lưu trữ $O(N)$ (N = số tiến trình)
Phức tạp

HLC:
Gần với thời gian vật lý
Thứ tự toàn phần
Gọn nhẹ (2 con số)
Vẫn cần đồng bộ đồng hồ (ở mức lỏng)

Đánh Đối Giữa Thứ Tự Và Tính Duy Nhất

Mâu thuẫn cốt lõi:

Tính duy nhất: Các ID không xung đột
Thứ tự: Các ID phản ánh đúng trình tự thời gian thực
Không thể hoàn hảo cả hai với cách sinh ID phi tập trung

Các phương án:

1. Duy nhất hoàn hảo, không có thứ tự
 - UUID v4
 - Hoàn toàn ngẫu nhiên
 - Không cần phối hợp
2. Duy nhất tốt, thứ tự gần đúng
 - Snowflake
 - Sắp theo thời gian (khi đồng hồ được đồng bộ)
 - Machine ID ngăn xung đột
3. Thứ tự hoàn hảo, bắt buộc phải phối hợp
 - Máy chủ sinh ID tập trung
 - Điểm lỗi duy nhất
 - Điểm nghẽn
 - Không mở rộng được

Lựa chọn dựa trên thứ tự ưu tiên

Khuyến Nghị Thực Tế

Cho Hầu Hết Các Ứng Dụng

Bắt đầu với UUID v7 (UUID sắp theo dấu thời gian)

```
import uuid

# UUID v7: Time-ordered, globally unique
id = uuid.uuid7()

Simple as UUID v4
Time-ordered like Snowflake
No configuration needed
Standard library support
Good database performance
```

Cho Các Hệ Thống Quy Mô Lớn

Dùng Snowflake nếu:

- ✓ Sinh > 100K ID/giây
- ✓ Có thể quản lý Machine ID
- ✓ Có thể đồng bộ đồng hồ (NTP)
- ✓ Cần ID 64 bit gọn nhẹ
- ✓ Cần trích xuất siêu dữ liệu (dấu thời gian, máy)

Cho Các Yêu Cầu Nghiêm Ngặt Về Thứ Tự

Dùng Hybrid Logical Clocks nếu:

- ✓ Thứ tự là yếu tố sống còn (tài chính, log kiểm toán)
- ✓ Chấp nhận được độ phức tạp
- ✓ Cần phát hiện quan hệ nhân quả
- ✓ Sự kiện phân tán trên nhiều dịch vụ

Các Phản Mẫu (Anti-Patterns) Cần Tránh

ĐỪNG dùng tự tăng trong hệ thống phân tán
ĐỪNG dựa vào đồng hồ thực để xác định thứ tự
ĐỪNG sinh ID tập trung (điểm nghẽn)
ĐỪNG bỏ qua vấn đề lệch đồng hồ
ĐỪNG chỉ dùng dấu thời gian làm ID (không duy nhất)

Ví Dụ Thực Tế

Twitter:

Dùng Snowflake cho ID của tweet

- 64 bit
- Sắp theo thời gian
- Hơn 200,000 tweet/giây
- Có thể trích xuất dấu thời gian từ ID

Instagram:

Snowflake đã được tùy biến

- 41 bit: dấu thời gian
- 13 bit: shard ID
- 10 bit: số thứ tự
- Lưu ảnh trên hơn 1000 máy

Discord:

Dựa trên Snowflake

- 64 bit
- Trích xuất dấu thời gian để hiển thị "X phút trước"
- Giữ lỗi hiệu năng theo các khoảng ID

MongoDB:

ObjectID (96 bit)

- 4 byte: dấu thời gian
- 5 byte: giá trị ngẫu nhiên
- 3 byte: bộ đếm
- Ý tưởng tương tự Snowflake

Những Điểm Cốt Lõi

Tự tăng thất bại trong hệ thống phân tán:

Một cơ sở dữ liệu: ✓ Hoạt động

Nhiều cơ sở dữ liệu: x Xung đột

Nhiều dịch vụ: x Xung đột

Cần chiến lược sinh ID phân tán

Các chiến lược sinh ID:

UUID:

- Đơn giản nhất
- 128 bit
- Ngẫu nhiên
- Không có thứ tự
- Lựa chọn mặc định

Snowflake:

- 64 bit
- Sắp theo thời gian
- Cần Machine ID
- Hiệu năng cao
- Tiêu chuẩn của ngành

HLC:

- Thứ tự tốt nhất
- Theo dõi quan hệ nhân quả
- Phức tạp nhất
- Dành cho tình huống chuyên biệt

Đánh đổi về thứ tự:

Không phối hợp: Thứ tự gần đúng
Phối hợp một phần: Thứ tự tốt
Phối hợp hoàn toàn: Thứ tự hoàn hảo (không mở rộng được)

Lựa chọn dựa trên yêu cầu

Khung ra quyết định:

```
if simple_is_priority:
    use UUID_v7
elif high_scale and need_ordering:
    use Snowflake
elif strict_ordering_required:
    use HLC
else:
    use UUID_v7 # Safe default
```

Các quy tắc vàng:

1. Không bao giờ dùng tự tăng trong hệ thống phân tán
2. UUID v7 là lựa chọn mặc định an toàn cho hầu hết các trường hợp
3. Snowflake cho các nhu cầu quy mô lớn, cần sắp theo thời gian

4. Thứ tự hoàn hảo đòi hỏi phải phối hợp (không miễn phí)

5. Luôn kiểm thử các kịch bản (*scenarios*) lệch đồng hồ

Sinh ID phân tán là một trong những vấn đề trông thì đơn giản nhưng ẩn chứa độ phức tạp tinh vi.

Hiểu rõ các sự đánh đổi. Lựa chọn phù hợp. Kiểm thử kỹ lưỡng.

ID là nền móng của hệ thống. Làm sai = khổ về sau.

Thời Gian & Thứ Tự Sự Kiện (Event Ordering) - Bài Toán Đồng Hồ (Clock Problem) Trong Hệ Thống Phân Tán (Distributed Systems)

Có một câu hỏi tưởng chừng đơn giản:

"Sự kiện (Event) A xảy ra trước hay sau sự kiện B?"

Trong một ứng dụng chạy trên một máy duy nhất (*single-machine*), câu trả lời rất dễ: kiểm tra dấu thời gian (*timestamp*). Đồng hồ (*clock*) CPU cho biết thứ tự chính xác.

Nhưng trong một hệ thống phân tán (*distributed system*) với 3 máy chủ (*servers*) đặt ở 3 nơi khác nhau, câu hỏi này trở nên cực kỳ khó.

Máy chủ 1 (Singapore): Dấu thời gian của sự kiện A = 10:00:00.123

Máy chủ 2 (Tokyo): Dấu thời gian của sự kiện B = 10:00:00.120

Sự kiện B nhỏ hơn → xảy ra trước? **Không chắc.**

Vì đồng hồ của 2 máy chủ không đồng bộ (*sync*) hoàn hảo. Độ trôi đồng hồ (*Clock drift*) có thể làm dấu thời gian sai lệch vài mili giây, thậm chí vài giây.

Đây chính là Bài Toán Đồng Hồ - một trong những vấn đề khó nhất trong hệ thống phân tán.

Bài học này dạy bạn:

- Tại sao đồng hồ vật lý (*physical clocks*) không đáng tin cậy (*reliable*)
- Đồng hồ logic (*Logical clocks*) giải quyết vấn đề như thế nào
- Dấu thời gian Lamport (*Lamport timestamps*) và đồng hồ vector (*Vector clocks*)
- Quan hệ nhân quả (*Causality*) và tính đồng thời (*concurrency*)
- Khi nào cần thứ tự (*ordering*), khi nào không

Tại Sao Thời Gian Quan Trọng?

Trước khi đi sâu vào giải pháp, hãy hiểu tại sao việc sắp xếp thứ tự các sự kiện (events) lại quan trọng đến vậy.

Tình Huống Sử Dụng (Use Case) 1: Nhân Bản (Replication) Cơ Sở Dữ Liệu (Database)

Người dùng cập nhật hồ sơ ở Máy chủ A
dấu thời gian: 10:00:00.100
tên: "Alice" → "Bob"

Người dùng cập nhật hồ sơ ở Máy chủ B (đồng thời)
dấu thời gian: 10:00:00.095
tên: "Alice" → "Charlie"

Khi nhân bản (replicate), áp dụng theo thứ tự nào?

Nếu dựa vào dấu thời gian → "Charlie" thắng (dấu thời gian nhỏ hơn).

Nhưng có thể đồng hồ của Máy chủ B chậm 10ms → thực tế "Bob" mới là bản cập nhật cuối cùng.

Sai thứ tự = sai trạng thái cuối cùng.

Tình Huống Sử Dụng 2: Giao Dịch Phân Tán (Distributed Transaction)

Giao dịch 1: Chuyển \$100 (A → B)
Giao dịch 2: Kiểm tra số dư của A

Nếu Giao dịch (Transaction) 2 đọc trước khi Giao dịch 1 hoàn tất → dữ liệu cũ (stale data).

Phải đảm bảo quan hệ nhân quả: việc kiểm tra số dư phải thấy được giao dịch chuyển tiền đã xong.

Tình Huống Sử Dụng 3: Event Sourcing (lưu trữ theo chuỗi sự kiện)

Sự kiện 1: Đơn hàng được tạo
Sự kiện 2: Thanh toán được xử lý
Sự kiện 3: Đơn hàng được giao

Các sự kiện phải được phát lại đúng thứ tự.

Nếu giao hàng trước khi thanh toán → logic sai.

Mô hình tư duy (*Mental model*): Hệ thống phân tán cần thứ tự để duy trì tính đúng đắn và tính nhất quán (*consistency*).

Đồng Hồ Vật Lý & Vấn Đề Độ Trôi Đồng Hồ

Đồng Hồ Vật Lý (*Physical Clock*) Là Gì?

Đồng hồ vật lý = **đồng hồ hệ thống** (*system clock*) của máy chủ, đo thời gian thực.

Mỗi máy chủ có một bộ dao động thạch anh (*quartz crystal oscillator*) để đếm thời gian.

```
Máy chủ 1: 2025-02-15 10:00:00.000
Máy chủ 2: 2025-02-15 10:00:00.003 (trôi +3ms)
Máy chủ 3: 2025-02-15 09:59:59.998 (trôi -2ms)
```

Vấn đề: Các đồng hồ (*clocks*) không bao giờ đồng bộ hoàn hảo.

Độ Trôi Đồng Hồ

Độ trôi đồng hồ = sự sai lệch giữa đồng hồ của các máy chủ.

Nguyên nhân:

- Bộ dao động thạch anh (*Crystal oscillator*) không hoàn hảo (nhiệt độ, sai số trong sản xuất)
- Độ trễ mạng (*Network delay*) khi đồng bộ thời gian
- Hiệu ứng tương đối tính (các máy chủ ở độ cao khác nhau - đùa thôi, đây không phải lý do chính 😊)

Tốc độ trôi (*Drift rate*) **trong thực tế:**

- Thông thường: ± 50 ppm (phần triệu)
- Máy chủ tốt: ± 10 ppm
- Trường hợp xấu nhất: ± 100 ppm

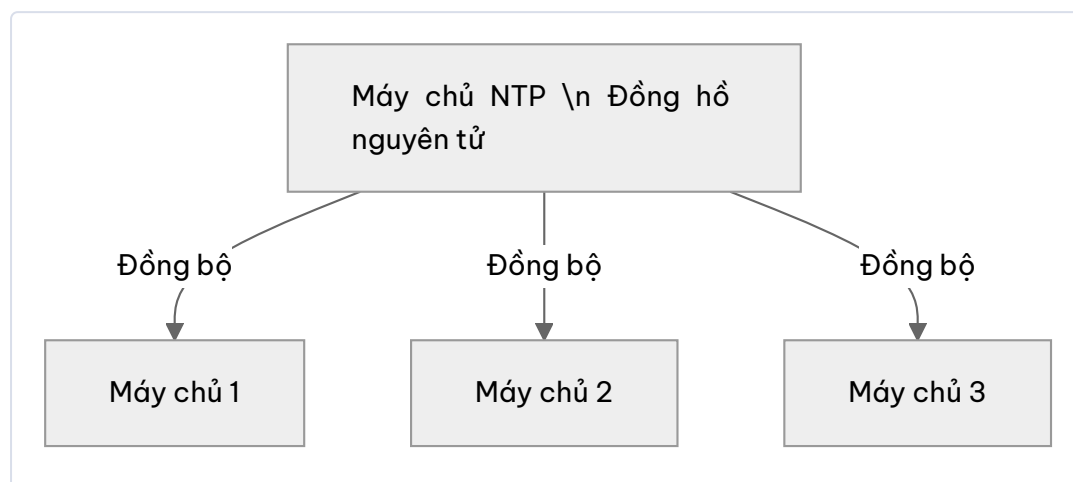
Ý nghĩa:

$\pm 50 \text{ ppm} = \pm 50 \text{ micro giây mỗi giây}$
 $= \pm 4,3 \text{ giây mỗi ngày}$
 $= \pm 26 \text{ phút mỗi năm}$

Chỉ sau 1 ngày không đồng bộ, 2 máy chủ có thể sai lệch vài giây.

NTP (Network Time Protocol - giao thức thời gian mạng)

Giải pháp: Đồng bộ đồng hồ với một nguồn thời gian trung tâm.



Cách NTP hoạt động:

1. Máy chủ gửi yêu cầu (*request*) đến máy chủ NTP
2. Máy chủ NTP trả lời kèm dấu thời gian
3. Máy chủ tính độ trễ mạng (*network latency*)
4. Điều chỉnh đồng hồ cục bộ (*local clock*)

Độ chính xác:

- Internet: $\pm 10\text{-}100\text{ms}$
- LAN: $\pm 1\text{-}10\text{ms}$
- GPS/Đồng hồ nguyên tử: $\pm 1\mu\text{s}$

Các vấn đề vẫn còn:

1. Dao động độ trễ mạng (*Network jitter*)

- Độ trễ mạng không ổn định
- Yêu cầu đồng bộ có thể bị chậm trễ

2. Lệnh đồng hồ (Clock skew)

- Việc điều chỉnh đồng hồ có thể khiến thời gian nhảy lùi
- Ứng dụng thấy thời gian quay ngược → lỗi (*bugs*)

3. Không hoàn hảo

- Vẫn có độ trôi (*drift*) giữa các khoảng đồng bộ
- Độ chính xác $\pm 10\text{ms}$ vẫn là vấn đề với các sự kiện có tần suất cao

Điểm mấu chốt: Đồng hồ vật lý không đáng tin cậy cho việc xác định thứ tự trong hệ thống phân tán.

Đồng Hồ Logic - Xác Định Thứ Tự Mà Không Cần Thời Gian Vật Lý

Tư duy đột phá: Chúng ta không cần biết thời gian chính xác, chỉ cần biết **thứ tự** (*order*) của các sự kiện.

Đồng hồ logic (*Logical clock*) = **một bộ đếm** (*counter*) để theo dõi thứ tự, không phải thời gian thực.

Dấu Thời Gian Lamport - Nền Tảng Của Việc Sắp Thứ Tự Logic

Leslie Lamport (1978) đưa ra một thuật toán đồng hồ logic đơn giản nhưng mạnh mẽ.

Ý Tưởng Cốt Lõi

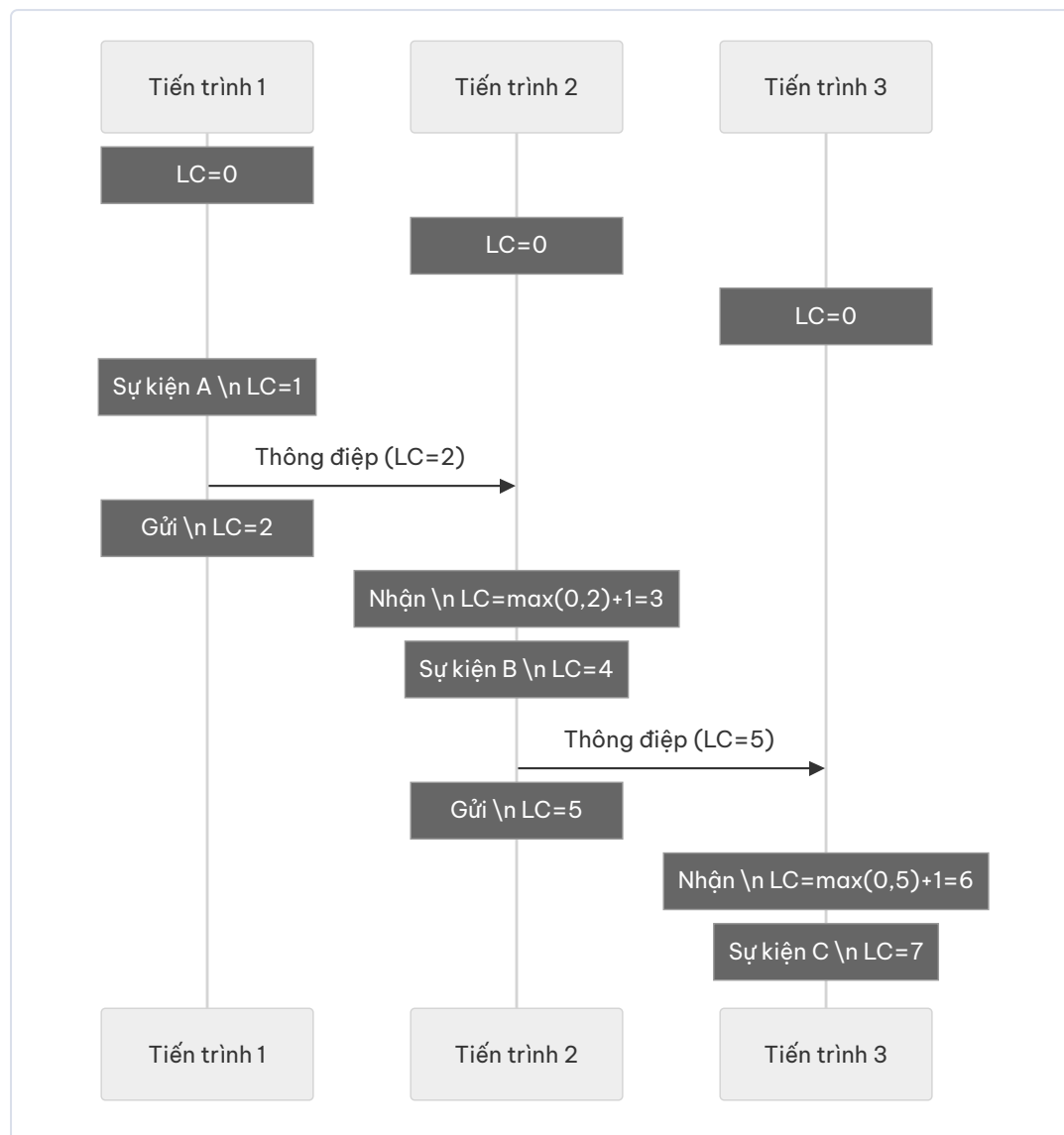
Mỗi tiến trình (*process*) có một bộ đếm (đồng hồ Lamport):

- Bắt đầu từ 0
- Tăng lên trước mỗi sự kiện
- Khi gửi thông điệp (*message*): đính kèm giá trị bộ đếm
- Khi nhận thông điệp: cập nhật bộ đếm = $\max(\text{cục bộ}, \text{nhận được}) + 1$

Quy Tắc

1. Sự kiện nội bộ: $LC = LC + 1$
2. Gửi thông điệp: $LC = LC + 1$, đính kèm LC vào thông điệp
3. Nhận thông điệp: $LC = \max(LC, \text{message.LC}) + 1$

Ví Dụ Thực Tế



Từng bước một:

Tiến trình 1:

LC=1: Sự kiện A (nội bộ)

LC=2: Gửi thông điệp đến P2

Tiến trình 2:

LC=3: Nhận từ P1 ($\max(0,2)+1$)

LC=4: Sự kiện B (nội bộ)

LC=5: Gửi thông điệp đến P3

Tiến trình 3:

LC=6: Nhận từ P2 ($\max(0,5)+1$)

LC=7: Sự kiện C (nội bộ)

Thứ tự:

- Sự kiện A (LC=1) < Gửi P1 → P2 (LC=2)
- Gửi P1 → P2 (LC=2) < Nhận tại P2 (LC=3)
- Sự kiện B (LC=4) < Gửi P2 → P3 (LC=5)
- Gửi P2 → P3 (LC=5) < Nhận tại P3 (LC=6)

Lamport đảm bảo: Nếu $A \rightarrow B$ (A xảy ra trước B), thì $LC(A) < LC(B)$.

Quan Hệ Xảy-Ra-Trước (*Happened-Before Relationship*) ($\rightarrow$)

$A \rightarrow B$ có nghĩa là:

1. A và B ở cùng một tiến trình, A trước B, hoặc
2. A là thao tác gửi, B là thao tác nhận của cùng một thông điệp, hoặc
3. Tồn tại C sao cho $A \rightarrow C$ và $C \rightarrow B$ (tính bắc cầu (*transitivity*))

Nếu không có $A \rightarrow B$ và cũng không có $B \rightarrow A$ → A và B đồng thời (*concurrent*).

Hạn Chế Của Dấu Thời Gian Lamport

Vấn đề: Lamport chỉ đảm bảo theo 1 chiều.

Nếu $A \rightarrow B$ thì $LC(A) < LC(B)$	ĐÚNG
Nếu $LC(A) < LC(B)$ thì $A \rightarrow B$	SAI

Ví dụ:

Tiến trình 1: Sự kiện A (LC=5)
Tiến trình 2: Sự kiện B (LC=8)

$LC(A) < LC(B)$, nhưng không có quan hệ nhân quả (*causal relationship*) $\rightarrow$ A và B đồng thời.

Lamport không phân biệt được quan hệ nhân quả với tính đồng thời.

Đồng Hồ Vector - Phát Hiện Quan Hệ Nhân Quả

Đồng hồ vector giải quyết hạn chế của Lamport.

Ý Tưởng Cốt Lõi

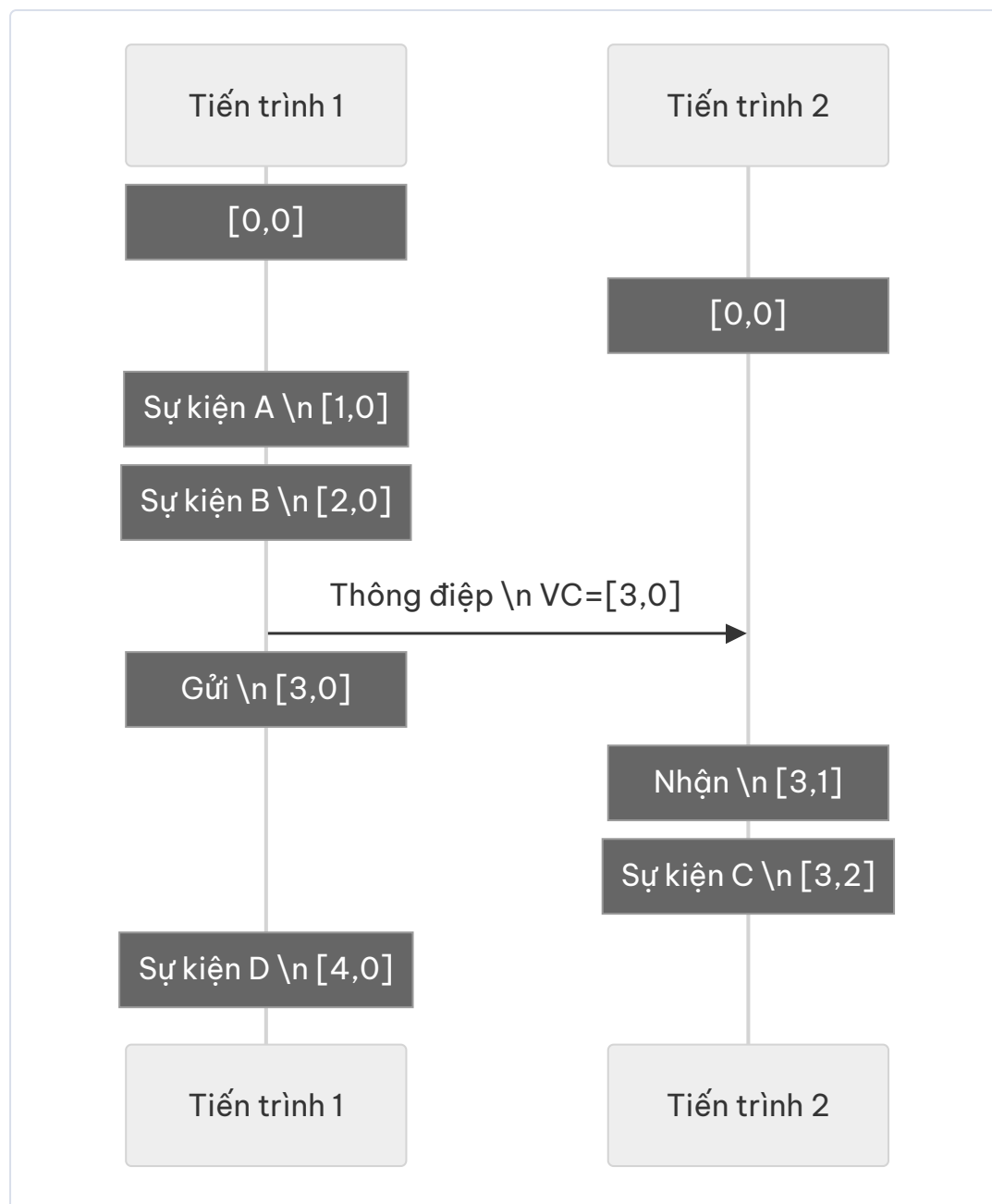
Mỗi tiến trình duy trì một vector các bộ đếm (*counters*) - mỗi tiến trình ứng với 1 bộ đếm.

Tiến trình 1: VC = [1, 0, 0] // [P1, P2, P3]
Tiến trình 2: VC = [0, 1, 0]
Tiến trình 3: VC = [0, 0, 1]

Quy Tắc

1. Sự kiện nội bộ: $VC[i] = VC[i] + 1$ (i = tiến trình của chính mình)
2. Gửi thông điệp:
 - $VC[i] = VC[i] + 1$
 - Đính kèm VC vào thông điệp
3. Nhận thông điệp:
 - $VC[i] = \max(VC[i], \text{message.VC}[i])$ với mọi i
 - $VC[\text{own}] = VC[\text{own}] + 1$

Ví Dụ Đồng Hồ Vector



So sánh:

Sự kiện A: [1,0]
Sự kiện B: [2,0]
Sự kiện C: [3,2]
Sự kiện D: [4,0]

A → B?

[1,0] < [2,0] (mọi thành phần đều ≤) → **CÓ, có quan hệ nhân quả** (causal)

B → C?

[2,0] < [3,2] → **CÓ, có quan hệ nhân quả** (B xảy ra trước C)

C → D?

[3,2] vs [4,0] : 3<4 nhưng 2>0 → **KHÔNG, đồng thời**

D → C?

[4,0] vs [3,2] : 4>3 → **KHÔNG, đồng thời**

C và D đồng thời - không có quan hệ nhân quả.

Quy Tắc So Sánh Đồng Hồ Vector (Vector Clock)

VC1 < VC2 (VC1 đứng trước VC2 về nhân quả) nếu:

- VC1[i] ≤ VC2[i] với mọi i
- ∃j sao cho VC1[j] < VC2[j]

VC1 || VC2 (đồng thời) nếu:

- ∃i sao cho VC1[i] < VC2[i]
- ∃j sao cho VC1[j] > VC2[j]

Đồng hồ vector phát hiện tính đồng thời một cách chính xác.

Sự Đánh Đổi (Trade-off): Lamport và Đồng Hồ Vector

Khía cạnh	Lamport	Đồng hồ vector
Kích thước	O(1) - một số duy nhất	O(N) - N tiến trình (processes)
Phát hiện quan hệ nhân quả	Chỉ một chiều	Cả hai chiều

Khía cạnh	Lamport	Đồng hồ vector
Phát hiện tính đồng thời	Không	Có
Tình huống sử dụng	Sắp thứ tự đơn giản	Phát hiện xung đột (conflict)

Lamport: Nhẹ, đủ dùng để sắp thứ tự log và sự kiện.

Đồng hồ vector: Phát hiện xung đột (conflicts), cần thiết cho nhân bản và quản lý phiên bản (version control).

Quan Hệ Nhân Quả - Quan Hệ Xảy-Ra-Trước (Happens-Before Relationship)

Quan hệ nhân quả = mối quan hệ giữa nguyên nhân và kết quả.

Các Sự Kiện Có Quan Hệ Nhân Quả (Causal Events)

Sự kiện A: Người dùng nhấn "Đặt hàng"
Sự kiện B: Hệ thống lưu đơn hàng vào cơ sở dữ liệu
Sự kiện C: Hệ thống gửi email xác nhận

A → B → C: Một chuỗi nhân quả rõ ràng.

Thứ tự này phải được duy trì. Không thể gửi email trước khi lưu vào cơ sở dữ liệu.

Các Sự Kiện Đồng Thời (Concurrent Events)

Tiến trình 1: Người dùng A cập nhật hồ sơ
Tiến trình 2: Người dùng B cập nhật hồ sơ (người dùng khác)

Không có quan hệ nhân quả → đồng thời.

Thứ tự không quan trọng. Có thể áp dụng các bản cập nhật theo bất kỳ thứ tự nào.

Tại Sao Quan Hệ Nhân Quả Quan Trọng

1. Tính nhất quán dữ liệu

Ghi X=1 trên Máy chủ A
Ghi X=2 trên Máy chủ B (xảy ra sau về nhân quả)

Khi nhân bản phải duy trì thứ tự: giá trị cuối cùng = 2.

2. Logic ứng dụng

Đơn hàng được tạo → Thanh toán được xử lý → Đơn hàng được giao

Vi phạm quan hệ nhân quả = lỗi logic.

3. Phát hiện xung đột

Các thao tác ghi đồng thời:
Người dùng A: X=1
Người dùng B: X=2

Không có thứ tự nhân quả → xung đột → cần chiến lược giải quyết (*resolution strategy*).

Ứng Dụng Thực Tế

1. Quản Lý Phiên Bản (Git)

Các commit trong Git dùng đồ thị có hướng không chu trình (*Directed Acyclic Graph*) (DAG) - một đồ thị nhân quả.

```
commit A → commit B → commit C
                ↓
            commit D (nhánh)
```

Xung đột khi hợp nhất (*Merge conflicts*) = các chỉnh sửa đồng thời ($B \rightarrow C$ và $B \rightarrow D$).

2. Cơ Sở Dữ Liệu Phân Tán (*Distributed Databases*) (DynamoDB, Cassandra)

Đồng hồ vector phát hiện các thao tác ghi xung đột (*conflicting*).

Ghi 1: VC=[1,0], value="Alice"
Ghi 2: VC=[0,1], value="Bob" (đồng thời)

Hệ thống phát hiện các thao tác ghi đồng thời → ứng dụng tự giải quyết xung đột.

3. CRDT (Conflict-free Replicated Data Types - kiểu dữ liệu nhân bản không xung đột)

Nhất quán cuối cùng (*Eventual consistency*) với cơ chế tự động giải quyết xung đột.

Việc theo dõi quan hệ nhân quả đảm bảo các thao tác có tính giao hoán (thứ tự không quan trọng).

4. Event Sourcing (lưu trữ theo chuỗi sự kiện)

Các sự kiện được lưu kèm siêu dữ liệu (*metadata*) nhân quả (đồng hồ Lamport/vector).

Phát lại sự kiện theo đúng quan hệ nhân quả → trạng thái nhất quán.

Đồng Hồ Logic Lai (*Hybrid Logical Clocks*) (HLC) - Kết Hợp Ưu Điểm Của Cả Hai

Cách tiếp cận hiện đại: Kết hợp thời gian vật lý + đồng hồ logic.

Tại Sao Cần HLC?

Vấn đề với đồng hồ logic thuần túy:

- Không có khái niệm về thời gian thực
- Khó gỡ lỗi (*debug*) (không biết sự kiện xảy ra vào lúc nào trong thực tế)
- Khó tích hợp với bên ngoài (các API cần dấu thời gian (*timestamps*) thực)

Vấn đề với đồng hồ vật lý:

- Độ trôi đồng hồ
- Không phát hiện được quan hệ nhân quả

HLC = Thời gian vật lý + bộ đếm kiểu Lamport.

Cấu Trúc HLC

```
HLC = (physical_time, logical_counter)
```

Quy tắc:

```
1. Sự kiện nội bộ:
  pt = max(physical_clock, last_hlc.pt)
  if pt == last_hlc.pt:
    lc = last_hlc.lc + 1
  else:
    lc = 0
  hlc = (pt, lc)

2. Gửi/Nhận: logic tương tự
```

Lợi ích:

- Đảm bảo quan hệ nhân quả (như Lamport)
- Gần với thời gian vật lý (hữu ích cho gỡ lỗi (*debugging*), giám sát (*monitoring*))
- Độ trôi có giới hạn (nhờ bộ đếm logic (*logical counter*) bị chặn)

Được dùng bởi: CockroachDB, MongoDB (ở mức độ nào đó), Google Spanner (TrueTime).

Khi Nào Bạn Thực Sự Cần Thứ Tự Sự Kiện?

Đối chiếu thực tế: Không phải lúc nào cũng cần thứ tự chính xác.

Cần Thứ Tự Khi:

1. Duy trì tính nhất quán dữ liệu

- Cơ sở dữ liệu (*databases*) có nhân bản (*replicated*)
- Giao dịch phân tán (*Distributed transactions*)
- Event Sourcing

2. Phụ thuộc nhân quả (*Causal dependencies*)

- Đặt hàng → Thanh toán → Giao hàng
- Tạo tài khoản → Xác minh email → Kích hoạt

3. Phát hiện xung đột

- Các thao tác ghi đồng thời vào cùng một dữ liệu
- Hợp nhất trong quản lý phiên bản

Không Cần (Hoặc Nói Lỏng) Thứ Tự:

1. Các sự kiện độc lập

- Hành động của Người dùng A không ảnh hưởng đến Người dùng B
- Tài nguyên khác nhau, không dùng chung

2. Hệ thống nhất quán cuối cùng (*eventually consistent*)

- Lượt thích trên mạng xã hội (số đếm cuối cùng chính xác là được)
- Bộ đếm lượt xem
- Dữ liệu phân tích

3. Thao tác lũy đẳng (*idempotent*)

- Set X=5 (thứ tự không quan trọng, trạng thái cuối cùng như nhau)

Mô hình tư duy: Thứ tự có chi phí (*cost*) (độ phức tạp (*complexity*), hiệu năng (*performance*)). Chỉ áp đặt khi thật sự cần thiết.

Những Cân Nhắc Thực Tế

1. Chi Phí Phụ (*Overhead*)

Lamport: Tối thiểu - một số nguyên duy nhất.

Đồng hồ vector: $O(N)$ cho mỗi tiến trình - cao khi có nhiều máy chủ.

HLC: Tương tự Lamport + đồng bộ đồng hồ vật lý.

Sự đánh đổi: Độ chính xác và chi phí phụ.

2. Lưu Trữ (*Storage*)

Đồng hồ vector phình to theo quy mô hệ thống.

10 máy chủ: 10 số nguyên cho mỗi sự kiện
1000 máy chủ: 1000 số nguyên cho mỗi sự kiện

Giải pháp:

- Loại bỏ các mục cũ
- Tập máy chủ thay đổi → thay đổi kích thước vector
- Dùng các cách tiếp cận lai

3. Gỡ Lỗi

Đồng hồ logic khó gỡ lỗi vì không có thời gian thực.

Thực hành tốt nhất (*Best practice*): Ghi log cả dấu thời gian logic lẫn vật lý.

```
{
  "event": "order_created",
  "lamport_clock": 12345,
  "wall_clock": "2025-02-15T10:00:00Z",
  "vector_clock": [5, 3, 8]
}
```

Mô Hình Tư Duy: Thời Gian Trong Hệ Thống Phân Tán ≠ Thời Gian Trong Đời Thật

Điểm cốt lõi của bài học này:

Trong hệ thống phân tán, thời gian không phải là một trục tuyến tính toàn cục.

Một máy duy nhất:
Sự kiện A (10:00:00) → Sự kiện B (10:00:01)
Dòng thời gian rõ ràng

Hệ thống phân tán:
Máy chủ 1: Sự kiện A (10:00:00.000)
Máy chủ 2: Sự kiện B (10:00:00.001)
→ Không chắc A trước B

Thay vì dựa vào thời gian vật lý, hãy dùng các quan hệ nhân quả (*causal relationships*):

- $A \rightarrow B$ (xảy ra trước (*happened-before*))
- $A || B$ (đồng thời)

Đồng hồ vật lý: Hữu ích cho con người, không đáng tin cậy cho việc sắp thứ tự.

Đồng hồ logic: Đáng tin cậy cho việc sắp thứ tự, nhưng không mang ý nghĩa thời gian thực.

Đồng hồ lai (*Hybrid clocks*): Kết hợp ưu điểm của cả hai.

Quyết định của kiến trúc sư (*Architect*): Chọn chiến lược đồng hồ dựa trên yêu cầu (*requirements*) - mức độ nhất quán, nhu cầu gỡ lỗi, các ràng buộc (*constraints*) về hiệu năng.

Những Điểm Cốt Lõi

1. Đồng hồ vật lý bị trôi - không đáng tin cậy cho việc sắp thứ tự sự kiện

Sai số $\pm 10-100\text{ms}$ là bình thường. NTP có giúp nhưng không hoàn hảo.

2. Đồng hồ logic sắp thứ tự sự kiện mà không cần thời gian vật lý

Dấu thời gian Lamport đảm bảo thứ tự nhân quả. Nhẹ, $O(1)$.

3. Đồng hồ vector phát hiện quan hệ nhân quả và tính đồng thời

Phân biệt được sự kiện có quan hệ nhân quả và sự kiện đồng thời. Chi phí: $O(N)$ bộ nhớ.

4. Quan hệ xảy-ra-trước ($\rightarrow$) định nghĩa quan hệ nhân quả

$A \rightarrow B$ hoặc $A || B$ (đồng thời). Không có vùng trung gian.

5. HLC kết hợp thời gian vật lý + sắp thứ tự logic

Kết hợp ưu điểm của cả hai. Được dùng trong các cơ sở dữ liệu hiện đại.

6. Thứ tự có chi phí - chỉ áp đặt khi thật sự cần thiết

Các sự kiện độc lập không cần thứ tự nghiêm ngặt. Đánh đổi giữa độ phức tạp và tính đúng đắn.

7. Mô hình tư duy: Thời gian $\neq$ dòng thời gian toàn cục trong hệ thống phân tán

Hãy tư duy theo quan hệ nhân quả, không phải theo giờ đồng hồ thực (wall clock).

Hoàn thành Phần 3. Bạn đã học những nền tảng của hệ thống phân tán:

- Các mô hình sự cố (failure) & CAP
- Các mô hình nhất quán
- Đồng thuận (Consensus) & bầu chọn nút chủ (leader election)
- Giao dịch phân tán
- Sinh ID phân tán
- Thời gian & thứ tự sự kiện

Phần 4 tiếp theo sẽ tập trung vào khả năng mở rộng (scalability) & hiệu năng - tư duy về điểm nghẽn (bottleneck), các chiến lược lưu đệm (caching), CDN, các mẫu (patterns) mở rộng (scaling) cơ sở dữ liệu.

Hãy nhớ: Trong hệ thống phân tán, "điều gì xảy ra trước?" thường là câu hỏi sai. Câu hỏi đúng là "A có gây ra B không?" Hãy hiểu quan hệ nhân quả, chứ không chỉ dấu thời gian.

PHẦN 4

Khả Năng Mở Rộng & Hiệu Năng

Tư duy và kỹ thuật tối ưu hiệu năng ở quy mô vận hành thực tế: điểm nghẽn, bộ nhớ đệm, kiểm soát tải, khả năng quan sát.

1) Giới Thiệu Phần

Khi lưu lượng (*traffic*) của hệ thống bắt đầu tăng, hiệu năng (*performance*) trở thành yếu tố sống còn.

Ở quy mô vận hành thực tế (*production*), hệ thống không sụp đổ vì một lỗi lớn duy nhất. Nó sụp đổ vì nhiều điểm nghẽn (*bottleneck*) nhỏ tích tụ lại: truy vấn (*query*) chậm, tỷ lệ trượt bộ nhớ đệm (*cache miss*) cao, hàng đợi tồn đọng (*queue backlog*), hoặc thiếu kiểm soát tải (*load control*).

Mở rộng (*Scale*) không chỉ là thêm máy chủ (*server*). Mở rộng là đo đúng, tối ưu đúng và chấp nhận đúng sự đánh đổi (*trade-off*).

Đây là cột mốc kỹ thuật giúp bạn chuyển từ tư duy phân tán (*distributed thinking*) sang thiết kế hệ thống đặt hiệu năng lên đầu (*performance-first*).

2) Vì Sao Phần Này Tồn Tại

Mở rộng ngây thơ (*Naive scaling*) chỉ giải quyết phần ngọn.

Thực tế vận hành đòi hỏi bạn hiểu rõ:

- Giới hạn tài nguyên: CPU, bộ nhớ, mạng, disk I/O.
- Điểm nghẽn dịch chuyển theo tải và theo kiến trúc.
- Hệ thống cần lớp bảo vệ (*protection layer*) để tránh quá tải (*overload*) dây chuyền.
- Không có khả năng quan sát (*observability*) thì không thể tối ưu một cách có cơ sở.

Phần này tồn tại để bạn học cách thiết kế hệ thống cho lưu lượng thật, chứ không chỉ cho môi trường lý tưởng.

3) Mục Tiêu Học Tập

- Nhận diện và phân tích điểm nghẽn ở cấp hệ thống.
- Thiết kế chiến lược lưu đệm (*caching strategy*) dựa trên sự đánh đổi giữa tính nhất quán (*consistency*) và tốc độ.

- Thiết kế kiến trúc cơ sở dữ liệu (*database*) có thể mở rộng theo mức tăng trưởng dữ liệu.
 - Xử lý lưu lượng cao (*high traffic*) một cách an toàn bằng kiểm soát tải.
 - Bảo vệ hệ thống khỏi quá tải và sự cố dây chuyền (*cascading failure*).
 - Tối ưu độ trễ (*latency*) và thông lượng (*throughput*) theo mục tiêu rõ ràng.
 - Thiết kế hệ thống có khả năng quan sát tốt.
 - Gỡ lỗi (*Debug*) các vấn đề hiệu năng bằng dữ liệu đo lường.
-

4) Bạn Sẽ Học Gì Trong Phần Này

- **Phân tích điểm nghẽn:** tìm đúng điểm nghẽn thay vì tối ưu theo cảm tính.
 - **Đánh đổi (Trade-offs) khi lưu đệm (caching):** cân bằng giữa tốc độ, độ mới của dữ liệu và độ phức tạp của việc vô hiệu hoá (*invalidation*).
 - **CDN & phân phối tại điểm biên (edge):** giảm độ trễ bằng cách đưa dữ liệu đến gần người dùng.
 - **Tối ưu cơ sở dữ liệu & phân mảnh dữ liệu (sharding):** từ tinh chỉnh truy vấn (*query tuning*) đến phân mảnh dữ liệu.
 - **Giới hạn tần suất (Rate limiting) & kiểm soát tải:** kiểm soát tải đầu vào để giữ hệ thống ổn định.
 - **Xử lý bất đồng bộ (Async processing):** tách các luồng xử lý nặng để tăng thông lượng.
 - **Giám sát (Monitoring) & khả năng quan sát:** dùng số liệu đo (*metrics*), log (*logs*), truy vết (*tracing*) để nhìn đúng trạng thái hệ thống.
 - **Quy trình gỡ lỗi hiệu năng (performance debugging):** đo -> khoanh vùng -> xác minh -> tối ưu.
-

5) Tổng Quan Các Bài Học

Bài 1 – Hiệu Năng & Tư Duy Điểm Nghẽn – Tư Duy Tìm Và Giải Quyết Điểm Nghẽn

- **Mô tả:** xây dựng tư duy hiệu năng dựa trên điểm nghẽn thực tế.
- **Bạn nhận được:** khả năng ưu tiên đúng chỗ cần tối ưu trước.

Bài 2 – Chiến Lược Lưu Đệm – Đánh Đổi Tính Nhất Quán Để Lấy Tốc Độ

- **Mô tả:** thiết kế lớp bộ nhớ đệm (*cache layer*) theo khối lượng công việc (*workload*) và ràng buộc (*constraint*) của dữ liệu.
- **Bạn nhận được:** năng lực chọn chiến lược lưu đệm (*cache strategy*) dựa trên sự đánh đổi rõ ràng.

Bài 3 – CDN & Điện Toán Biên (*Edge Computing*)

- **Mô tả:** giảm độ trễ và giảm tải cho backend bằng cách phân phối nội dung gần người dùng.
- **Bạn nhận được:** tư duy ưu tiên điểm biên (*edge-first*) cho lưu lượng thiên về đọc (*read-heavy*).

Bài 4 – Mở Rộng (*Scaling*) & Tối Ưu Cơ Sở Dữ Liệu – Từ Tinh Chỉnh Truy Vấn Đến Phân Mảnh Dữ Liệu

- **Mô tả:** tối ưu cơ sở dữ liệu từ lớp truy vấn đến kiến trúc dữ liệu phân tán.
- **Bạn nhận được:** cách mở rộng dữ liệu bền vững khi khối lượng (*volume*) tăng mạnh.

Bài 5 – Giới Hạn Tần Suất & Kiểm Soát Tải – Bảo Vệ Hệ Thống Khỏi Quá Tải

- **Mô tả:** áp dụng các cơ chế bảo vệ để hệ thống không bị sập dây chuyền.
- **Bạn nhận được:** chiến lược giữ ổn định khi lưu lượng vượt ngưỡng.

Bài 6 – Xử Lý Bất Đồng Bộ (*Asynchronous Processing*) & Hàng Đợi (*Queues*)

- **Mô tả:** dùng hàng đợi (*queue*) để tách luồng xử lý, giảm chặn (*blocking*) và tăng thông lượng.
- **Bạn nhận được:** nền tảng thiết kế đường ống xử lý (*processing pipeline*) cho khối lượng công việc lớn.

Bài 7 – Giám Sát, Khả Năng Quan Sát & Gỡ Lỗi Hiệu Năng – Đo Lường Để Mở Rộng

- **Mô tả:** xây dựng khả năng quan sát và gỡ lỗi hiệu năng dựa trên tín hiệu của hệ thống.
- **Bạn nhận được:** quy trình cải thiện hệ thống có thể lặp lại và kiểm chứng được.

6) Kết Quả Sau Khi Hoàn Thành Phần 4

Bạn có thể:

- Thiết kế hệ thống chịu được lưu lượng lớn hơn một cách có chủ đích.
- Tối ưu hiệu năng dựa trên đo lường thay vì phỏng đoán.
- Lập luận về sự đánh đổi hiệu năng ở cấp kiến trúc.
- Bảo vệ hệ thống khỏi quá tải và sự cố (*failure*) lan rộng.
- Nhanh chóng xác định điểm nghẽn khi hệ thống chậm.
- Dùng khả năng quan sát để cải thiện độ tin cậy (*reliability*) liên tục.
- Thiết kế kiến trúc sẵn sàng cho vận hành thực tế (*production-ready*) hơn về hiệu năng và vận hành.

7) Chuẩn Bị Cho Phần 5

Phần 4 cung cấp nền tảng về khả năng mở rộng và hiệu năng ở cấp hệ thống.

Sang **Phần 5 — Các Mẫu Kiến Trúc Thực Tế**, bạn sẽ áp dụng trực tiếp các nguyên tắc này vào kiến trúc thực tế theo từng lĩnh vực (*domain*):

- tiến hoá kiến trúc theo đà tăng trưởng của sản phẩm,
- các mẫu (*pattern*) hệ thống trong môi trường vận hành thực tế,
- và thiết kế theo ngữ cảnh nghiệp vụ cụ thể.

Hiệu Năng (Performance) & Tư Duy Điểm Nghẽn (Bottleneck Thinking) - Tư Duy Tìm Và Giải Quyết Điểm Nghẽn

Khi hệ thống bắt đầu chậm, nhiều lập trình viên (*developer*) nghĩ ngay: "Thêm máy chủ (*server*)!". Nhưng đây là sai lầm phổ biến nhất trong thiết kế hệ thống (*system design*).

Mở rộng (Scaling) không phải là thêm tài nguyên. Mở rộng là tìm và loại bỏ điểm nghẽn (bottleneck).

Nếu bạn không biết đâu là điểm nghẽn, việc thêm máy chủ chỉ là lãng phí tiền. Đôi khi thêm 10 máy chủ vẫn không giải quyết được vấn đề vì điểm nghẽn nằm ở cơ sở dữ liệu (*database*), không phải ở máy chủ ứng dụng (*application server*).

Bài này dạy bạn **tư duy về hiệu năng từ góc nhìn của kiến trúc sư (architect)** - cách nhìn hệ thống như một đường ống (*pipeline*), tìm điểm nghẽn, và quyết định đúng cách mở rộng.

Tại Sao Phải Học Tư Duy Điểm Nghẽn Trước?

Trước khi học CDN, lưu đệm (*caching*), phân mảnh dữ liệu (*sharding*) cơ sở dữ liệu... bạn cần hiểu:

Mỗi kỹ thuật mở rộng đều giải quyết một loại điểm nghẽn cụ thể.

- CDN giải quyết điểm nghẽn về **độ trễ (latency)** và **băng thông (bandwidth) mạng**
- Lưu đệm giải quyết điểm nghẽn về **tải (load) đọc của cơ sở dữ liệu**
- Phân mảnh dữ liệu giải quyết điểm nghẽn về **thông lượng (throughput) ghi của cơ sở dữ liệu**
- Hàng đợi thông điệp (*Message queue*) giải quyết điểm nghẽn về **thời gian xử lý đồng bộ (sync)**

Nếu không biết cách tìm điểm nghẽn, bạn sẽ áp dụng sai giải pháp → lãng phí công sức + tiền bạc.

Mô hình tư duy (*Mental model*) **đúng:**

1. Đo lường → tìm điểm nghẽn
2. Hiểu nguyên nhân gốc (*root cause*)
3. Chọn giải pháp phù hợp
4. Đo lường lại

Không phải: nghe người ta dùng Redis → mình cũng dùng Redis.

Các Chiều Khả Năng Mở Rộng (*Scalability*) - 3 Chiều Của Hiệu Năng

Khi nói về hiệu năng, hệ thống có 3 chiều bạn phải quan tâm:

1. Tính Toán (*Compute*) (CPU/Năng Lực Xử Lý)

Đây là khả năng xử lý của máy chủ - tính toán, logic, thuật toán.

Điểm nghẽn ở tính toán khi:

- Mức sử dụng CPU cao (>80%)
- Máy chủ ứng dụng chậm vì logic nghiệp vụ (*business logic*) phức tạp
- Xử lý ảnh, mã hoá video, suy luận ML (*ML inference*) chạy lâu

Giải pháp:

- Mở rộng theo chiều dọc (*Vertical scaling*): nâng cấp CPU mạnh hơn
- Mở rộng theo chiều ngang (*Horizontal scaling*): thêm máy chủ ứng dụng + bộ cân bằng tải (*load balancer*)
- Xử lý bất đồng bộ (*Async processing*): đẩy tác vụ nặng sang hàng đợi (*queue*)

2. Lưu Trữ (*Storage*) (Disk I/O & Khối Lượng Dữ Liệu)

Khả năng đọc/ghi dữ liệu và lượng dữ liệu bạn cần lưu.

Điểm nghẽn ở lưu trữ khi:

- Disk I/O chậm (HDD và SSD khác biệt rất lớn)
- Truy vấn (*Query*) cơ sở dữ liệu chậm vì kích thước dữ liệu lớn

- Quét toàn bộ bảng (Full table scan) trên bảng có hàng triệu bản ghi

Giải pháp:

- Nâng cấp lưu trữ: HDD → SSD → NVMe
- Đánh chỉ mục (index) cho cơ sở dữ liệu
- Phân mảnh dữ liệu cơ sở dữ liệu (chia dữ liệu ra nhiều máy chủ)
- Lưu trữ lâu dài (Archiving) dữ liệu cũ

3. Mạng (Băng Thông & Độ Trễ)

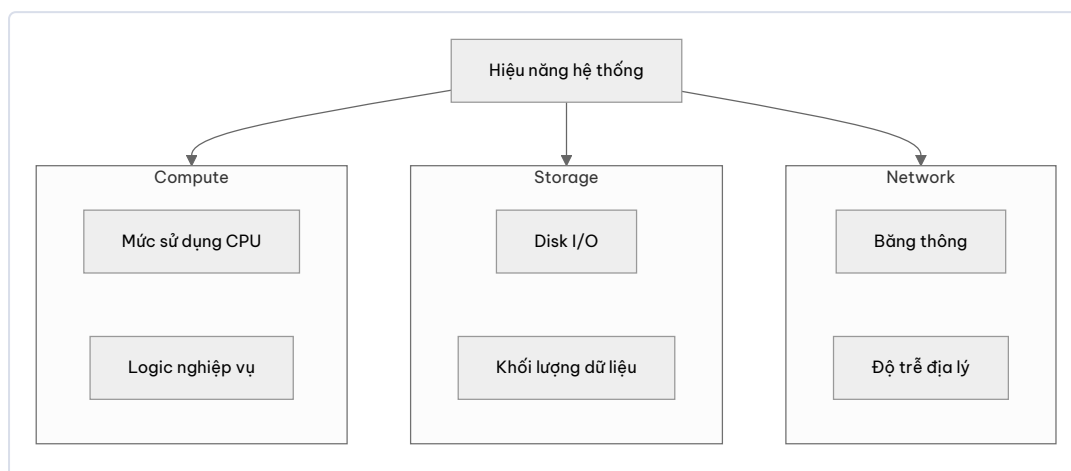
Tốc độ truyền tải dữ liệu giữa các thành phần (components).

Điểm nghẽn ở mạng khi:

- Truyền tải file lớn (video, hình ảnh)
- Người dùng ở xa máy chủ (độ trễ địa lý (geographic latency))
- Băng thông bị giới hạn (nhiều yêu cầu (requests) cùng lúc)

Giải pháp:

- CDN: lưu đệm nội dung tĩnh ở gần người dùng
- Nén: giảm kích thước dữ liệu
- Giảm dữ liệu gửi kèm (payload): chỉ trả về dữ liệu cần thiết
- Nhiều vùng (regions): triển khai (deploy) máy chủ gần người dùng



Điểm nghẽn có thể nằm ở bất kỳ chiều nào. Nhiệm vụ của kiến trúc sư là tìm ra chiều nào đang giới hạn hệ thống.

Thông Lượng Và Độ Trễ - Hai Chỉ Số (Metrics) Quan Trọng Nhất

Khi đo lường hiệu năng, có 2 chỉ số bạn phải hiểu rõ:

Thông Lượng (Số Yêu Cầu Mỗi Giây)

Thông lượng = số lượng yêu cầu mà hệ thống xử lý được trong 1 giây.

Ví dụ:

- API xử lý 1000 req/s
- Cơ sở dữ liệu xử lý 500 lượt ghi/s
- Hàng đợi tiêu thụ 10.000 thông điệp/s

Thông lượng cao = hệ thống xử lý được nhiều lưu lượng (traffic).

Độ Trễ (Thời Gian Phản Hồi)

Độ trễ = thời gian để hoàn thành 1 yêu cầu.

Ví dụ:

- Thời gian phản hồi (*response*) của API: 100ms
- Thời gian truy vấn cơ sở dữ liệu: 50ms
- Thời gian khứ hồi (*round trip*) mạng: 200ms

Độ trễ thấp = trải nghiệm người dùng tốt.

Sự Đánh Đổi (Trade-off) Giữa Thông Lượng Và Độ Trễ

Đây là điểm khó: **không phải lúc nào cũng tối ưu (optimize) được cả hai.**

Trường hợp 1: Tăng thông lượng → có thể làm tăng độ trễ

Khi bạn cho phép API xử lý nhiều yêu cầu cùng lúc (tăng mức độ đồng thời (*concurrency*)), mỗi yêu cầu có thể phải chờ lâu hơn vì tài nguyên bị chia sẻ.

Trước: 100 req/s, độ trễ ~50ms
Sau: 500 req/s, độ trễ ~200ms

Thông lượng tăng gấp 5 lần, nhưng độ trễ tăng gấp 4 lần.

Trường hợp 2: Giảm độ trễ → có thể làm giảm thông lượng

Khi bạn tối ưu để từng yêu cầu chạy nhanh hơn (giảm độ trễ), có thể phải giảm số yêu cầu xử lý đồng thời.

Quyết định của kiến trúc sư:

- Thương mại điện tử: độ trễ quan trọng (trải nghiệm người dùng)
- Hệ thống phân tích: thông lượng quan trọng (xử lý dữ liệu khối lớn)
- Hệ thống thanh toán: cả hai đều quan trọng

Bạn phải biết nghiệp vụ cần gì để quyết định hướng tối ưu.

Mở Rộng Theo Chiều Dọc Và Theo Chiều Ngang - Hai Cách Mở Rộng Cơ Bản

Mở Rộng Theo Chiều Dọc (Scale Up)

Nâng cấp máy chủ hiện tại: CPU mạnh hơn, RAM nhiều hơn, ổ đĩa nhanh hơn.

Ưu điểm:

- Đơn giản: không cần thay đổi mã
- Không có độ phức tạp (*complexity*) của hệ thống phân tán (*distributed system*)
- Phù hợp với ứng dụng nguyên khối (*monolithic*)

Nhược điểm:

- Có giới hạn vật lý (không thể mua CPU vô hạn)
- Đắt đỏ: máy chủ càng mạnh thì giá càng tăng theo cấp số nhân
- Điểm lỗi duy nhất (*Single point of failure*): máy chủ chết → toàn bộ hệ thống sập

Khi nào dùng mở rộng theo chiều dọc:

- Hệ thống nhỏ, lưu lượng chưa cao
- Cơ sở dữ liệu cần tính nhất quán (*consistency*) cao (khó mở rộng theo chiều ngang)
- Giải pháp sửa nhanh tạm thời

Mở Rộng Theo Chiều Ngang (Scale Out)

Thêm nhiều máy chủ cùng xử lý: 1 máy chủ → 5 máy chủ → 50 máy chủ.

Ưu điểm:

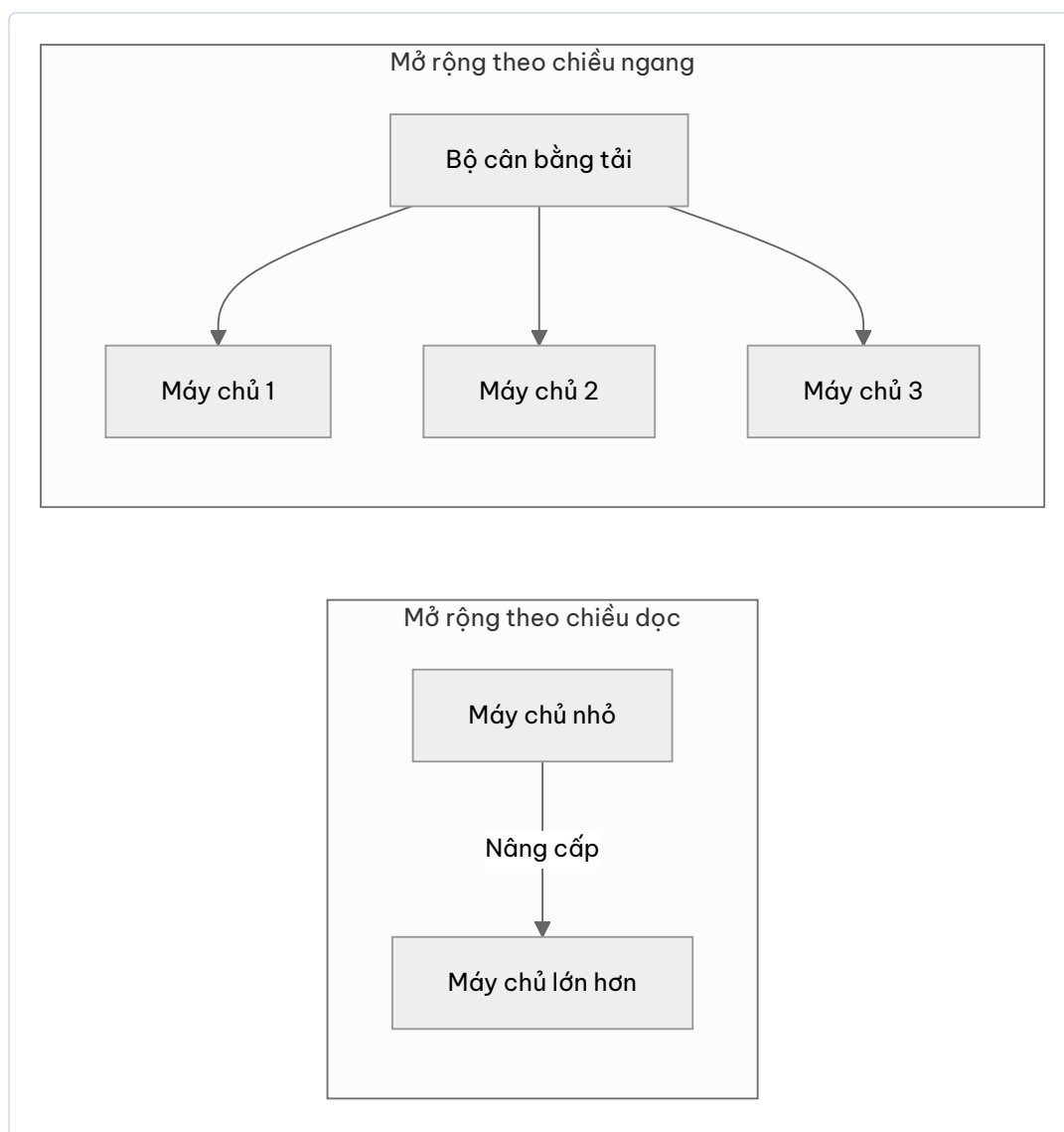
- Không có giới hạn về lý thuyết (có thể thêm máy chủ liên tục)
- Hiệu quả về chi phí (*cost*): nhiều máy chủ nhỏ rẻ hơn 1 máy chủ lớn
- Tính sẵn sàng cao (*High availability*): 1 máy chủ chết, các máy còn lại vẫn hoạt động

Nhược điểm:

- Phức tạp: cân bộ cân bằng tải, quản lý phiên (*session*), tính nhất quán dữ liệu (*data consistency*)
- Không phải thành phần (*component*) nào cũng mở rộng theo chiều ngang được (cơ sở dữ liệu thì khó)
- Công sức phát triển cao

Khi nào dùng mở rộng theo chiều ngang:

- Hệ thống lớn, lưu lượng cao
- Cần tính sẵn sàng cao
- Ứng dụng phi trạng thái (*stateless*) (API, vi dịch vụ (*microservices*))



Thực tế: Hầu hết hệ thống lớn dùng **cả hai** - chiều dọc cho cơ sở dữ liệu, chiều ngang cho tầng ứng dụng (*application layer*).

Định Luật Amdahl (*Amdahl's Law*) - Tại Sao Không Thể Mở Rộng Vô Hạn

Định luật Amdahl nói rằng: Mức tăng tốc (*speedup*) của hệ thống bị giới hạn bởi phần không thể song song hoá (*parallelize*).

Giả sử một yêu cầu có luồng xử lý:

1. Phân tích cú pháp yêu cầu: 10ms (có thể song song)
2. Truy vấn cơ sở dữ liệu: 80ms (có thể song song)
3. Tuần tự hoá (*Serialize*) phản hồi: 10ms (**KHÔNG thể song song - điểm nghẽn**)

Tổng thời gian: 100ms

Khi bạn mở rộng theo chiều ngang:

- Phân tích cú pháp: 10ms → 5ms (nhanh gấp 2 lần)
- Cơ sở dữ liệu: 80ms → 40ms (nhanh gấp 2 lần)
- Tuần tự hoá: 10ms → **vẫn 10ms** (không thể song song)

Tổng thời gian: 55ms (chỉ cải thiện 1,8 lần, không phải 2 lần)

Bước tuần tự hoá phản hồi chính là điểm nghẽn. Dù bạn thêm bao nhiêu máy chủ, bước này vẫn giữ nguyên.

Nhận định thực tế:

Khi bạn mở rộng mà không thấy cải thiện tuyến tính → hãy tìm phần không song song hoá được (tuần tự hoá (*serialization*), vùng găng (*critical section*), logic đơn luồng (*single-threaded*)).

Tư duy kiến trúc sư: Không phải cứ thêm máy chủ là hệ thống nhanh hơn. Bạn phải tìm ra phần nào đang chặn việc thực thi song song.

Quy Trình Xác Định Điểm Nghẽn (*Bottleneck Identification*) - Quy Trình Tìm Điểm Nghẽn

Đây là quy trình thực tế mà kiến trúc sư dùng để tìm điểm nghẽn:

Bước 1: Xác Định Mục Tiêu Hiệu Năng

Trước khi tìm điểm nghẽn, phải biết mục tiêu (*target*) là gì.

Không rõ ràng:

- "Hệ thống phải nhanh"

Rõ ràng:

- Thời gian phản hồi của API < 200ms ở bách phân vị (*percentile*) thứ 95

- Duy trì ổn định 10.000 req/s
- Truy vấn cơ sở dữ liệu trung bình < 50ms

Không có mục tiêu = không biết đâu là điểm nghẽn.

Bước 2: Đo Lường Trạng Thái Hiện Tại

Đo tất cả các chỉ số quan trọng:

Tầng ứng dụng:

- Tần suất yêu cầu (req/s)
- Thời gian phản hồi (p50, p95, p99)
- Tỷ lệ lỗi (*Error rate*)
- Mức sử dụng CPU & bộ nhớ

Tầng cơ sở dữ liệu:

- Thời gian truy vấn
- Mức sử dụng nhóm kết nối (*connection pool*)
- Log truy vấn chậm (*slow query*)
- Disk I/O

Mạng:

- Mức sử dụng băng thông
- Độ trễ giữa các dịch vụ (*services*)
- Mất gói tin (*Packet loss*)

Công cụ: Prometheus, Grafana, DataDog, New Relic, CloudWatch.

Bước 3: Xác Định Ràng Buộc (*Constraint*)

Nhìn vào các chỉ số → tìm thành phần nào đang bị dùng kịch trần.

Ví dụ thực tế:

Máy chủ API: CPU 40%, Bộ nhớ 50% → OK
 Bộ cân bằng tải: CPU 10% → OK
 CSDL: CPU 95%, thời gian truy vấn 500ms → ĐIỂM NGHẼN!

→ Điểm nghẽn nằm ở cơ sở dữ liệu, không phải ở máy chủ API.

Thêm máy chủ API sẽ không giải quyết được vấn đề.

Bước 4: Hiểu Nguyên Nhân Gốc

Tìm được điểm nghẽn là chưa đủ. Phải hiểu **tại sao** nó trở thành điểm nghẽn.

CPU của cơ sở dữ liệu cao có thể vì:

- Thiếu chỉ mục → quét toàn bộ bảng
- Truy vấn tồi (vấn đề N+1 (*N+1 problem*))
- Khối lượng dữ liệu quá lớn
- Quá nhiều kết nối

Phải đào sâu hơn để biết giải pháp đúng.

Bước 5: Chọn Đúng Giải Pháp

Mỗi nguyên nhân gốc có giải pháp khác nhau:

Nguyên Nhân Gốc	Giải Pháp
Thiếu chỉ mục	Thêm chỉ mục
Truy vấn N+1 (<i>N+1 query</i>)	Tối ưu truy vấn (<i>Query optimization</i>), nạp sớm (<i>eager loading</i>)
Tải đọc cao	Bản sao chỉ đọc (<i>Read replica</i>), lưu đệm
Tải ghi cao	Phân mảnh dữ liệu, ghi bất đồng bộ (<i>async</i>) qua hàng đợi
Tập dữ liệu lớn	Lưu trữ lâu dài, phân vùng (<i>partitioning</i>)

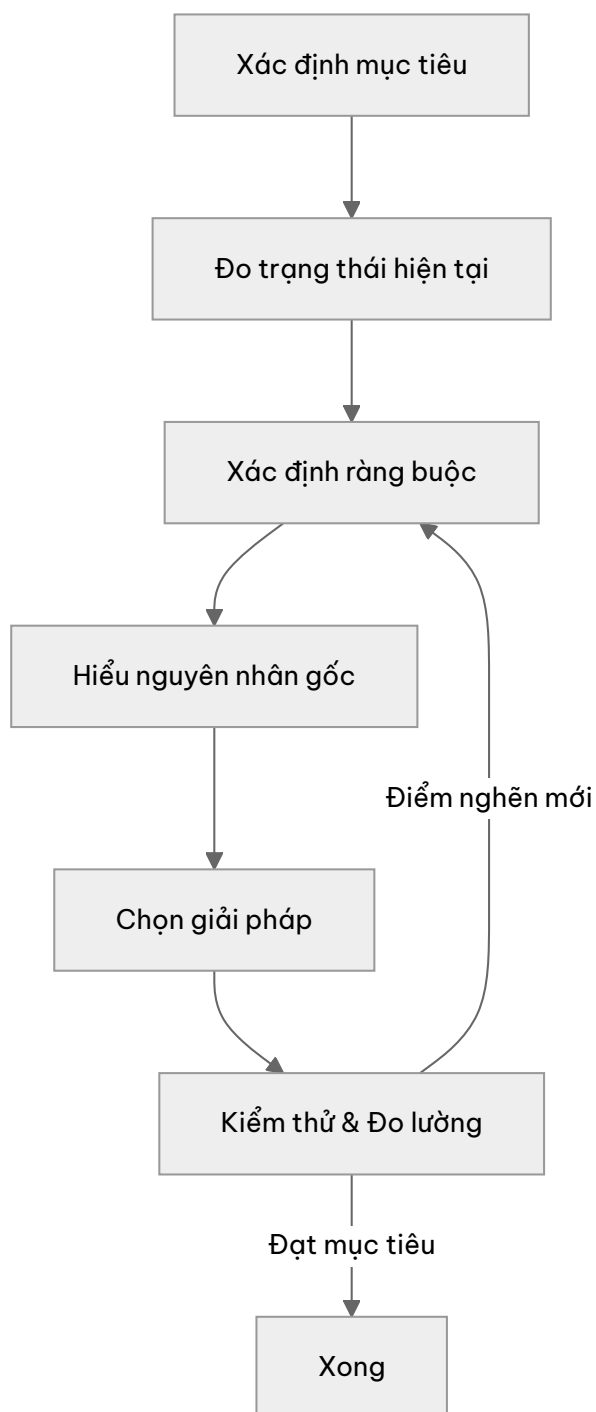
Không có giải pháp một cỡ vừa cho tất cả (*one-size-fits-all*).

Bước 6: Kiểm Thử & Đo Lường Lại

Sau khi áp dụng giải pháp:

- Kiểm thử tải (*Load test*) lại
- Đo lường các chỉ số
- Xác minh sự cải thiện
- Tìm điểm nghẽn mới (thường sẽ có điểm nghẽn tiếp theo)

Tối ưu hiệu năng (*Performance optimization*) là một quá trình lặp đi lặp lại.



Tư Duy Kiểm Thử Tải (*Load Testing Thinking*) - Tìm Điểm Nghẽn Trước Khi Lên Môi Trường Thực Tế (*Production*)

Đừng đợi đến khi môi trường thực tế sập mới đi tìm điểm nghẽn.

Kiểm thử tải (*Load testing*) = mô phỏng lưu lượng để phơi bày điểm nghẽn từ sớm.

Các Loại Kiểm Thử Tải

1. Kiểm thử cơ sở (*Baseline test*)

- Chạy với lưu lượng bình thường
- Đo các chỉ số cơ sở
- Hiểu hành vi bình thường của hệ thống

2. Kiểm thử áp lực (*Stress test*)

- Tăng lưu lượng từ từ
- Tìm điểm gãy (*breaking point*)
- Xem thành phần nào gặp sự cố trước

3. Kiểm thử đột biến (*Spike test*)

- Lưu lượng tăng đột biến (*spike*) (flash sale, bài đăng lan truyền)
- Kiểm thử tự động mở rộng (*auto-scaling*)
- Xác minh khả năng phục hồi

4. Kiểm thử ngâm (*Soak test*) (**kiểm thử độ bền** (*Endurance test*))

- Chạy tải ổn định liên tục trong nhiều giờ
- Tìm rò rỉ bộ nhớ (*memory leak*), tình trạng cạn kiệt tài nguyên
- Xác minh độ ổn định trong dài hạn

Công Cụ: k6, JMeter, Gatling, Locust

Ví dụ script k6:

```
import http from 'k6/http';
import { check, sleep } from 'k6';

export let options = {
  stages: [
    { duration: '2m', target: 100 }, // ramp up
    { duration: '5m', target: 100 }, // stay at 100 users
    { duration: '2m', target: 200 }, // ramp to 200
    { duration: '5m', target: 200 },
    { duration: '2m', target: 0 }, // ramp down
  ],
};

export default function () {
  let res = http.get('https://api.example.com/users');
  check(res, { 'status was 200': (r) => r.status == 200 });
  sleep(1);
}
```

Quan sát các chỉ số khi tải tăng:

- Thời gian phản hồi có tăng không?
- Tỷ lệ lỗi có tăng đột biến không?
- Thành phần nào có CPU/bộ nhớ tăng nhanh?

Đó chính là điểm nghẽn.

Mô Hình Tư Duy: Mở Rộng = Loại Bỏ Điểm Nghẽn

Hãy ghi nhớ mô hình tư duy này:

Mở rộng không phải là thêm tài nguyên một cách ngẫu nhiên.

Mở rộng = xác định điểm nghẽn → loại bỏ điểm nghẽn → lặp lại.

Hệ thống giống như một đường ống. Dòng nước chảy bị giới hạn bởi **đoạn ống nhỏ nhất**.

```
[Ông rộng] → [ÔNG HẸP] → [Ông rộng]
           ↑
        Điểm nghẽn
```

Dù bạn có nối rộng 2 đoạn ống hai bên, dòng nước vẫn bị giới hạn bởi đoạn ở giữa.

Áp dụng vào hệ thống:

```

[API: 10k req/s] → [DB: 1k req/s] → [Phản hồi]
                    ↑
                Điểm nghẽn

```

Thêm máy chủ API chẳng giúp được gì. Phải mở rộng cơ sở dữ liệu hoặc thêm bộ nhớ đệm (*cache*).

Tư duy (Mindset) của kiến trúc sư:

1. Hệ thống luôn có điểm nghẽn
2. Sửa điểm nghẽn hiện tại → điểm nghẽn mới xuất hiện
3. Tối ưu là một quá trình liên tục
4. Đo lường → Sửa → Đo lường lại

Những Điểm Cốt Lỗi

1. Khả năng mở rộng có 3 chiều: Tính toán, Lưu trữ, Mạng

Điểm nghẽn có thể nằm ở bất kỳ chiều nào. Phải xác định cho đúng.

2. Thông lượng $\neq$ Độ trễ

Hai chỉ số khác nhau, có thể xung đột nhau. Phải biết nghiệp vụ cần gì để tối ưu đúng hướng.

3. Mở rộng theo chiều dọc: đơn giản nhưng có giới hạn. Mở rộng theo chiều ngang: không giới hạn nhưng phức tạp

Hầu hết hệ thống lớn dùng cách tiếp cận kết hợp.

4. Định luật Amdahl: Mức tăng tốc bị giới hạn bởi phần không song song hoá được

Tìm ra vùng găng, điểm tuần tự hoá là then chốt.

5. Quy trình xác định điểm nghẽn:

Xác định mục tiêu → Đo lường → Xác định ràng buộc → Nguyên nhân gốc → Giải pháp → Kiểm thử lại.

6. Kiểm thử tải giúp tìm điểm nghẽn trước khi lên môi trường thực tế

Kiểm thử áp lực, kiểm thử đột biến, kiểm thử ngâm đều quan trọng.

7. Mô hình tư duy: Mở rộng = Loại bỏ điểm nghẽn

Không phải thêm tài nguyên. Mà là tìm và loại bỏ ràng buộc.

Hãy nhớ: **Kiến trúc sư giỏi không phải người biết nhiều công nghệ. Mà là người biết tìm đúng điểm nghẽn và chọn đúng giải pháp.**

Chiến Lược Lưu Đệm (Caching) - Đánh Đổi Tính Nhất Quán (Consistency) Để Lấy Tốc Độ

Có một câu nói nổi tiếng trong thiết kế hệ thống (system design):

"Trong Khoa học Máy tính chỉ có hai việc khó: vô hiệu hoá bộ nhớ đệm (cache invalidation) và đặt tên." - Phil Karlton

Lưu đệm nghe thì đơn giản: lưu dữ liệu vào bộ nhớ để đọc nhanh hơn. Nhưng khi triển khai thực tế, bạn sẽ gặp hàng tá câu hỏi khó:

- Đặt bộ nhớ đệm (cache) ở đâu? Ở ứng dụng? Ở cơ sở dữ liệu? Ở CDN?
- Khi nào cập nhật bộ nhớ đệm? Ngay lập tức hay để sau (lazy)?
- Khi nào xoá bộ nhớ đệm? Làm sao biết dữ liệu đã cũ (stale)?
- Lưu đệm bao nhiêu dữ liệu? RAM có giới hạn mà
- Chuyện gì xảy ra khi bộ nhớ đệm và cơ sở dữ liệu không đồng bộ (sync) với nhau?

Đây chính là lý do chiến lược lưu đệm lại quan trọng.

Bài học này dạy bạn tư duy về lưu đệm như một kiến trúc sư (architect) - hiểu các mẫu thiết kế (patterns), biết khi nào dùng mẫu nào, và quan trọng nhất: **hiểu sự đánh đổi (trade-off) giữa hiệu năng (performance) và tính nhất quán.**

Tại Sao Lưu Đệm Lại Hiệu Quả?

Trước khi học chiến lược, phải hiểu **tại sao** lưu đệm lại hiệu quả đến vậy.

Tính Cục Bộ Của Tham Chiếu (Locality of Reference)

Dữ liệu được truy cập gần đây có khả năng cao sẽ được truy cập lại.

Ví dụ thực tế:

- Facebook: Bạn làm mới bảng tin (*newsfeed*) 10 lần/phút → vẫn là cùng các bài viết đó
- Thương mại điện tử: Sản phẩm "hot" (iPhone mới) được xem hàng triệu lần
- Trang tin tức: Tin nóng được hàng triệu người đọc

Thay vì truy vấn (*query*) cơ sở dữ liệu 1 triệu lần, hãy lưu đệm ở lần đầu → phục vụ 999.999 lần còn lại từ bộ nhớ.

Chênh Lệch Tốc Độ Giữa Các Tầng Lưu Trữ (*Storage*)

RAM (bộ nhớ đệm):	~100 nano giây
SSD (CSDL):	~100 micro giây (chậm hơn 1000 lần)
HDD:	~10 mili giây (chậm hơn 100.000 lần)
Mạng (từ xa):	~100 mili giây (chậm hơn 1.000.000 lần)

Trúng bộ nhớ đệm (*Cache hit*) = nhanh hơn từ 1000 đến 1.000.000 lần.

Đây là lý do tại sao lưu đệm có tác động khổng lồ lên hiệu năng.

Quy Tắc 80/20 (Nguyên Lý Pareto)

80% yêu cầu (*requests*) truy cập vào 20% dữ liệu.

- YouTube: 20% video chiếm 80% lượt xem (các video lan truyền)
- Reddit: 20% bài viết trên trang nhất chiếm 80% lưu lượng (*traffic*)
- API: 20% điểm cuối (*endpoints*) nhận 80% yêu cầu

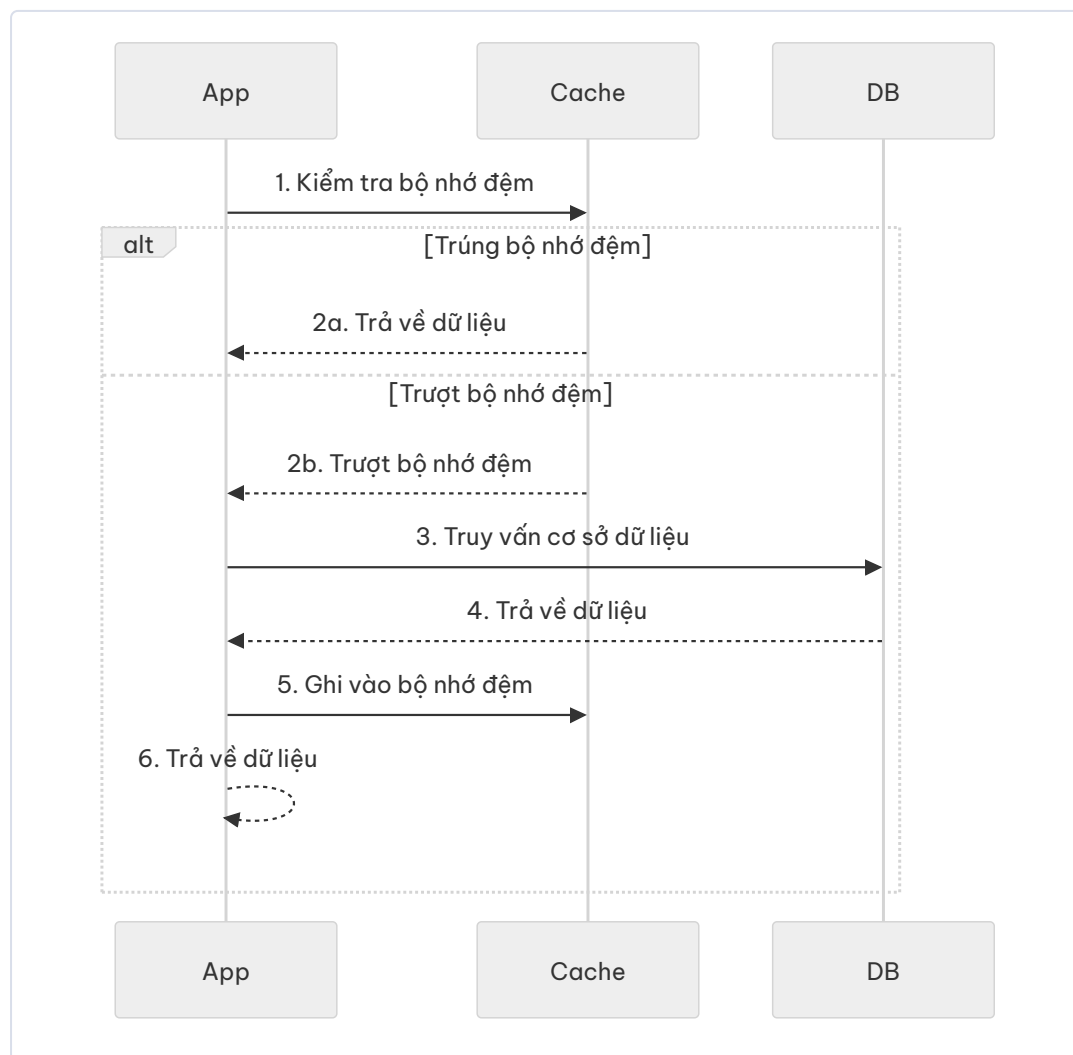
Chỉ cần lưu đệm 20% dữ liệu "nóng" (*data hot*), bạn đã giải quyết được 80% lưu lượng.

Mô hình tư duy (*Mental model*): Lưu đệm không phải là lưu đệm tất cả. Mà là lưu đệm đúng dữ liệu.

Mẫu Cache-Aside (Tải Lười - Lazy Loading)

Đây là mẫu thiết kế (*pattern*) phổ biến nhất và đơn giản nhất.

Luồng Hoạt Động



Từng bước:

1. Ứng dụng kiểm tra bộ nhớ đệm trước
2. **Trúng bộ nhớ đệm:** Trả về dữ liệu từ bộ nhớ đệm (đường nhanh)
3. **Trượt bộ nhớ đệm** (*Cache miss*): Truy vấn cơ sở dữ liệu
4. Ghi dữ liệu vào bộ nhớ đệm
5. Trả dữ liệu về cho người dùng

Ví Dụ Mã Nguồn

```
def get_user(user_id):
    # 1. Check cache
    cache_key = f"user:{user_id}"
    user = redis.get(cache_key)

    if user is not None:
        # 2. Cache hit
        return json.loads(user)

    # 3. Cache miss - query database
    user = db.query("SELECT * FROM users WHERE id = ?", user_id)

    # 4. Write to cache (TTL = 1 hour)
    redis.setex(cache_key, 3600, json.dumps(user))

    # 5. Return data
    return user
```

Ưu Điểm

Đơn giản và dễ triển khai

- Logic rõ ràng
- Dễ gỡ lỗi (*debug*)
- Ứng dụng kiểm soát hoàn toàn

Bộ nhớ đệm chỉ chứa dữ liệu được yêu cầu (*request*)

- Không lãng phí bộ nhớ cho dữ liệu không dùng đến
- Bộ nhớ đệm tự nhiên được "làm nóng" (*warm up*) theo đặc điểm lưu lượng

Chống chịu tốt khi bộ nhớ đệm gặp sự cố (*failure*)

- Bộ nhớ đệm sập → ứng dụng vẫn hoạt động (chỉ chậm hơn thôi)
- Không có điểm lỗi duy nhất (*single point of failure*)

Nhược Điểm

Cái giá của trượt bộ nhớ đệm cao

- Yêu cầu đầu tiên luôn chậm (truy vấn CSDL + ghi vào bộ nhớ đệm)

- Vấn đề dồn dập (*stampede*): nhiều yêu cầu cùng trượt bộ nhớ đệm → truy vấn CSDL nhiều lần

Dữ liệu có thể bị cũ

- Bộ nhớ đệm không tự động cập nhật khi CSDL thay đổi
- Phải dựa vào TTL hoặc vô hiệu hoá thủ công (*manual invalidation*)

Khi Nào Dùng Cache-Aside

Phù hợp với:

- Khối lượng công việc (*Workload*) thiên về đọc (*read-heavy*)
- Dữ liệu không thay đổi thường xuyên
- Ứng dụng cần kiểm soát logic bộ nhớ đệm
- Chấp nhận được nhất quán cuối cùng (*eventual consistency*)

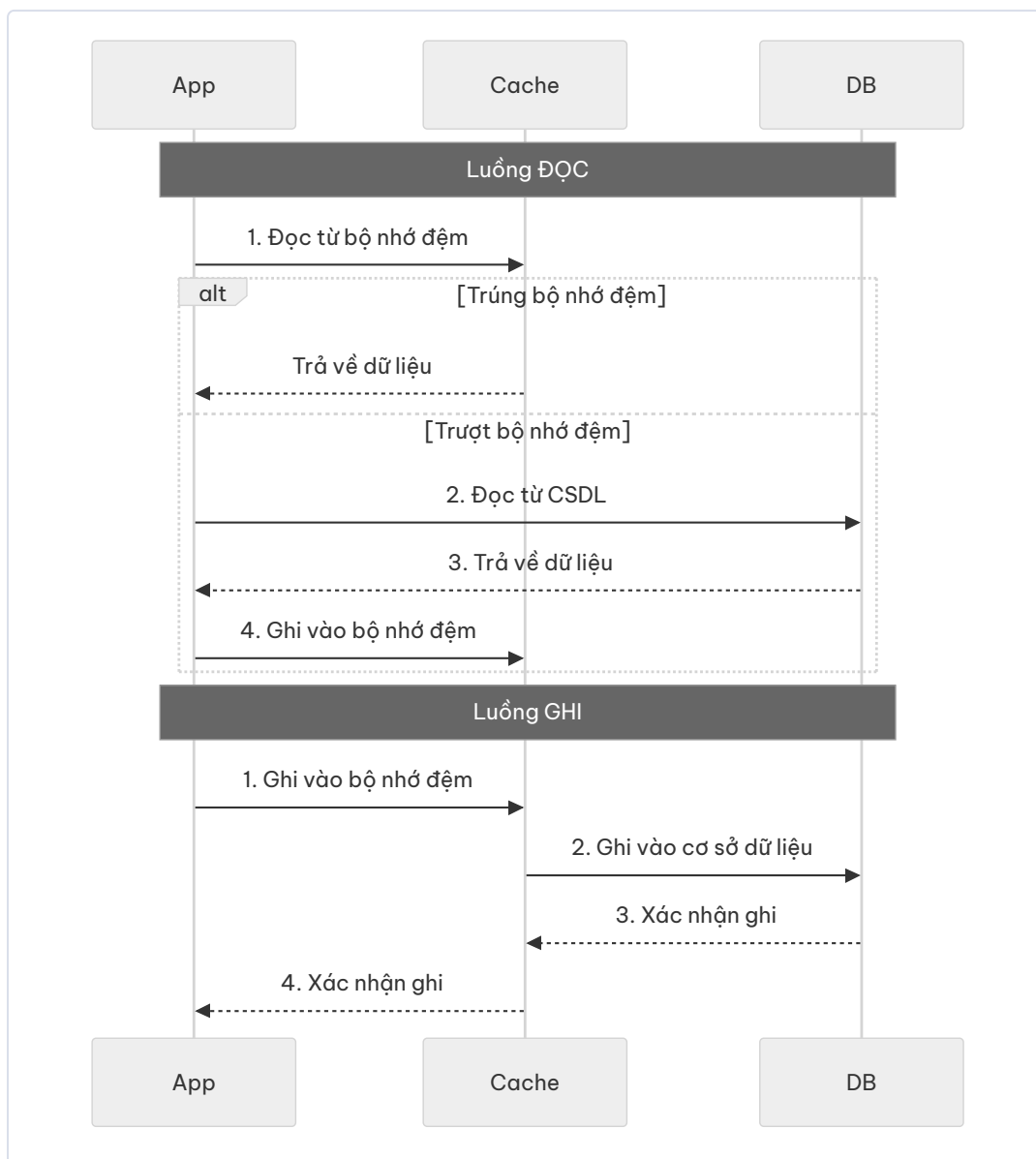
Ví dụ tình huống sử dụng (*use case*):

- Hồ sơ người dùng (ít thay đổi)
- Danh mục sản phẩm
- Dữ liệu cấu hình
- Bài viết blog

Mẫu Write-Through (Ghi Xuyên)

Write-through giải quyết vấn đề tính nhất quán của cache-aside.

Luồng Hoạt Động



Khác biệt then chốt: Khi ghi dữ liệu, ứng dụng ghi vào bộ nhớ đệm → bộ nhớ đệm ghi vào cơ sở dữ liệu.

Bộ nhớ đệm luôn đồng bộ với cơ sở dữ liệu.

Ví Dụ Mã Nguồn

```
def update_user(user_id, data):
    cache_key = f"user:{user_id}"

    # 1. Write to cache first
    redis.set(cache_key, json.dumps(data))

    # 2. Cache writes to database
    db.execute("UPDATE users SET ... WHERE id = ?", user_id)

    # Cache and DB are now in sync
    return data
```

Ưu Điểm

Tính nhất quán dữ liệu cao

- Bộ nhớ đệm và cơ sở dữ liệu luôn đồng bộ
- Không có vấn đề dữ liệu cũ (*stale data*)

Hiệu năng đọc tốt

- Dữ liệu luôn có sẵn trong bộ nhớ đệm
- Không bị trượt bộ nhớ đệm với dữ liệu vừa ghi

Nhược Điểm

Độ trễ (*Latency*) ghi cao

- Mỗi lần ghi phải đợi cả bộ nhớ đệm lẫn cơ sở dữ liệu
- Đường chậm cho các thao tác ghi

Không hiệu quả với khối lượng công việc thiên về ghi (*write-heavy*)

- Bộ nhớ đệm được cập nhật nhưng có thể không bao giờ được đọc
- Lãng phí công sức nếu dữ liệu không "nóng"

Bộ nhớ đệm lỗi = ghi lỗi

- Bộ nhớ đệm sập → không thể ghi
- Tăng sự phụ thuộc và rủi ro

Khi Nào Dùng Write-Through

Phù hợp với:

- Cần nhất quán mạnh (*strong consistency*)
- Mẫu đọc ngay sau khi ghi (*read-after-write*) (ghi rồi đọc ngay)
- Dữ liệu quan trọng không được phép cũ

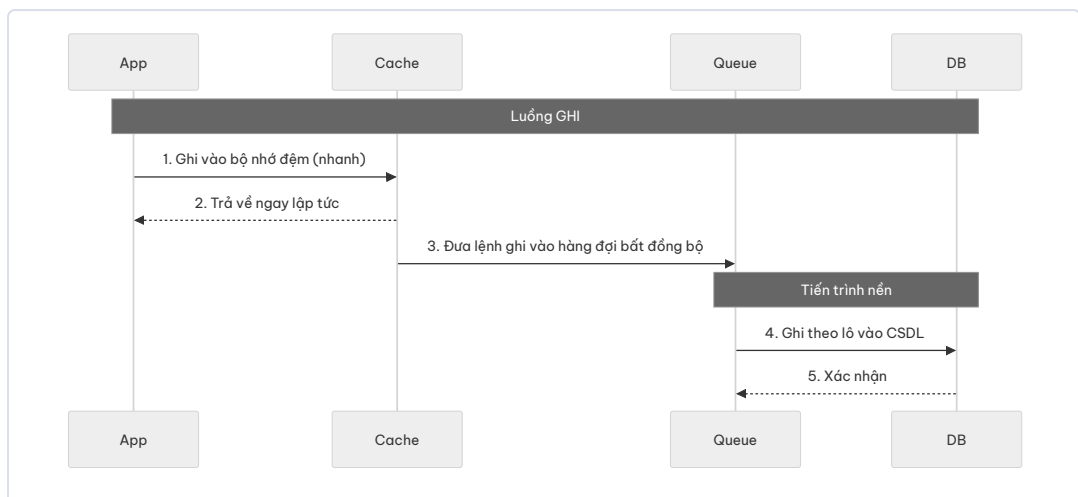
Ví dụ tình huống sử dụng:

- Giao dịch (*transactions*) tài chính
- Quản lý tồn kho
- Dữ liệu phiên (*session*)
- Giỏ hàng

Mẫu Write-Behind (Ghi Sau - Write-Back)

Write-behind tối ưu hiệu năng ghi bằng cách ghi bất đồng bộ (*async*).

Luồng Hoạt Động



Khác biệt then chốt: Ứng dụng chỉ ghi vào bộ nhớ đệm → trả về ngay. Bộ nhớ đệm tự đồng bộ xuống cơ sở dữ liệu sau đó (bất đồng bộ).

Ưu Điểm

Hiệu năng ghi cực cao

- Ứng dụng chỉ ghi vào bộ nhớ
- Độ trễ thấp nhất có thể

Ghi theo lô (*Batch*)

- Bộ nhớ đệm có thể gộp nhiều lần ghi lại với nhau
- Giảm tải (*load*) cơ sở dữ liệu đáng kể

Thông lượng (*Throughput*) cao

- Hệ thống chịu được khối lượng công việc thiên về ghi
- Cơ sở dữ liệu không bị quá tải

Nhược Điểm

Rủi ro mất dữ liệu

- Bộ nhớ đệm sập trước khi đồng bộ → mất dữ liệu
- Cần nhân bản (*replication*) cho tầng bộ nhớ đệm

Tính nhất quán phức tạp

- Bộ nhớ đệm có dữ liệu mới, cơ sở dữ liệu có dữ liệu cũ
- Đọc trực tiếp từ cơ sở dữ liệu sẽ nhận dữ liệu cũ

Triển khai phức tạp

- Cần hệ thống hàng đợi (*queue*)
- Cần xử lý thử lại (*retry*), sự cố
- Cần giám sát (*monitoring*)

Khi Nào Dùng Write-Behind

Phù hợp với:

- Ứng dụng thiên về ghi
- Chấp nhận được nhất quán cuối cùng
- Có hạ tầng (*infra*) để xử lý độ phức tạp (*complexity*)

Ví dụ tình huống sử dụng:

- Thu thập dữ liệu phân tích
- Hệ thống ghi log (*logging*)
- Bộ đếm lượt thích/lượt xem trên mạng xã hội
- Bảng xếp hạng trong game

Vô Hiệu Hoá Bộ Nhớ Đệm - Bài Toán Khó Của Lưu Đệm

"Vô hiệu hoá bộ nhớ đệm là một trong những bài toán khó nhất của khoa học máy tính."

Vấn đề: Làm sao biết khi nào bộ nhớ đệm đã cũ và cần làm mới?

Chiến Lược 1: Dựa Trên Thời Gian (TTL)

Đặt thời gian hết hạn cho mỗi mục trong bộ nhớ đệm.

```
# Cache expires after 1 hour
redis.setex("user:123", 3600, user_data)
```

Ưu điểm:

- Đơn giản
- Tự động dọn dẹp
- Hành vi dễ dự đoán

Nhược điểm:

- Dữ liệu có thể cũ trong khoảng thời gian TTL
- Khó chọn được giá trị TTL phù hợp
- Trượt bộ nhớ đệm tăng vọt khi nhiều khoá (*keys*) hết hạn cùng lúc

Thực hành tốt nhất (*Best practice*):

- Thêm độ lệch ngẫu nhiên (*random jitter*) để tránh hiện tượng đàn bò dồn dập (*thundering herd*)

```
ttl = 3600 + random.randint(0, 300) # 1h ± 5 min
```

Chiến lược 2: Vô Hiệu Hoá (*Invalidation*) Dựa Trên Sự Kiện (*Event*)

Vô hiệu hoá (*Invalidate*) bộ nhớ đệm khi dữ liệu thay đổi.

```
def update_user(user_id, data):  
    # Update database  
    db.execute("UPDATE users SET ... WHERE id = ?", user_id)  
  
    # Invalidate cache  
    redis.delete(f"user:{user_id}")
```

Ưu điểm:

- Dữ liệu luôn mới
- Không có khoảng thời gian dữ liệu cũ

Nhược điểm:

- Phức tạp khi có nhiều nơi cùng cập nhật dữ liệu
- Trượt bộ nhớ đệm sau khi vô hiệu hoá (khởi động lạnh)
- Khó theo dõi hết mọi phụ thuộc

Chiến lược 3: Vô Hiệu Hoá Dựa Trên Phiên Bản

Dùng số phiên bản trong khoá bộ nhớ đệm (*cache key*).

```
def get_user(user_id):
    version = get_user_version(user_id) # from DB or separate cache
    cache_key = f"user:{user_id}:v{version}"

    user = redis.get(cache_key)
    if user is None:
        user = db.query(...)
        redis.set(cache_key, user)

    return user

def update_user(user_id, data):
    db.execute("UPDATE users SET ... WHERE id = ?", user_id)
    increment_user_version(user_id) # Bump version
```

Ưu điểm:

- Bộ nhớ đệm cũ tự nhiên mất hiệu lực (không khớp phiên bản)
- Không cần xoá một cách tường minh
- Dễ hoàn tác (*rollback*)

Nhược điểm:

- Cần duy trì siêu dữ liệu về phiên bản
- Nhiều khoá bộ nhớ đệm (*cache keys*) hơn (tốn bộ nhớ)

Đối Chiếu Thực Tế: Kết Hợp Các Chiến Lược

Trong thực tế, kiến trúc sử dụng kết hợp:

- **TTL làm lưới an toàn:** Đảm bảo dữ liệu không bị cũ quá lâu
- **Dựa trên sự kiện cho các đường đi then chốt:** Người dùng cập nhật hồ sơ → vô hiệu hoá ngay
- **Phiên bản cho các phụ thuộc phức tạp:** Sản phẩm có nhiều dữ liệu liên quan

Không có giải pháp hoàn hảo. Hãy chọn sự đánh đổi phù hợp.

Chính Sách Loại Bỏ (*Eviction*) Khi Bộ Nhớ Đệm - Khi Bộ Nhớ Đệm Đầy

RAM có giới hạn. Khi bộ nhớ đệm đầy, phải xoá bớt dữ liệu cũ.

Câu hỏi: Xóa mục nào trong bộ nhớ đệm?

LRU (Least Recently Used - Ít Được Dùng Gần Đây Nhất)

Xóa mục lâu nhất không được truy cập.

Logic:

- Mỗi lần trúng bộ nhớ đệm → đưa mục đó lên đầu
- Khi bộ nhớ đệm đầy → xóa mục ở cuối (lâu nhất không dùng)

```
from collections import OrderedDict

class LRUCache:
    def __init__(self, capacity):
        self.cache = OrderedDict()
        self.capacity = capacity

    def get(self, key):
        if key not in self.cache:
            return None
        # Move to end (most recent)
        self.cache.move_to_end(key)
        return self.cache[key]

    def put(self, key, value):
        if key in self.cache:
            self.cache.move_to_end(key)
        self.cache[key] = value
        if len(self.cache) > self.capacity:
            # Remove least recent (first item)
            self.cache.popitem(last=False)
```

Khi nào dùng:

- Kiểu truy cập theo thời gian (dữ liệu gần đây được dùng lại)
- Lưu đệm cho mục đích chung
- Lựa chọn mặc định cho hầu hết các trường hợp

Ví dụ: Trang tin tức - Tin nóng (gần đây) được đọc nhiều.

LFU (Least Frequently Used - Ít Được Dùng Thường Xuyên Nhất)

Xóa mục có tần suất truy cập thấp nhất.

Logic:

- Theo dõi số lần mỗi mục được truy cập
- Khi bộ nhớ đệm đầy → xoá mục ít được dùng nhất

```
class LFUCache:
    def __init__(self, capacity):
        self.capacity = capacity
        self.cache = {}
        self.freq = {} # Track access frequency

    def get(self, key):
        if key not in self.cache:
            return None
        self.freq[key] += 1
        return self.cache[key]

    def put(self, key, value):
        if len(self.cache) >= self.capacity:
            # Find least frequent key
            min_key = min(self.freq, key=self.freq.get)
            del self.cache[min_key]
            del self.freq[min_key]

        self.cache[key] = value
        self.freq[key] = 1
```

Khi nào dùng:

- Dữ liệu "nóng" (*Hot data*) không thay đổi theo thời gian (luôn được ưa chuộng)
- Bộ nhớ đệm tồn tại lâu dài

Ví dụ: Thương mại điện tử - Sản phẩm bán chạy luôn "hot", không phụ thuộc vào thời điểm.

FIFO (First In First Out - Vào Trước Ra Trước)

Xoá mục cũ nhất (theo thứ tự thêm vào).

Đơn giản nhưng ít được dùng vì không tính đến kiểu truy cập.

So Sánh

Chính sách	Loại bỏ	Phù hợp nhất cho	Chi phí phụ (<i>Overhead</i>)
LRU	Lâu nhất không dùng	Truy cập theo thời gian	Trung bình
LFU	Ít được dùng nhất	Phổ biến lâu dài	Cao
FIFO	Cũ nhất	Trường hợp đơn giản	Thấp

Thực tế: Redis mặc định dùng **LRU xấp xỉ** (nhanh + hiệu quả).

Lưu Đệm Nhiều Tầng (*Multi-Layer Caching*) - Phòng Thủ Nhiều Lớp (*Defense In Depth*)

Lưu đệm không chỉ có một tầng. Hệ thống lớn có nhiều tầng bộ nhớ đệm.

Tầng 1: Bộ Nhớ Đệm Trình Duyệt (*Browser Cache*)

Bộ nhớ đệm nằm ở trình duyệt phía máy khách (*client*).

```
HTTP/1.1 200 OK
Cache-Control: public, max-age=31536000
ETag: "abc123"
```

Ưu điểm:

- Không phát sinh yêu cầu qua mạng
- Nhanh nhất có thể
- Giảm tải cho máy chủ (*server*)

Dùng cho:

- Tài nguyên tĩnh (CSS, JS, hình ảnh)
- Nội dung bất biến

Tầng 2: Bộ Nhớ Đệm CDN

Bộ nhớ đệm đặt tại các điểm biên (*edge locations*) **gần người dùng**.

```
Người dùng (Việt Nam) → CDN (Singapore) → Máy chủ gốc (Mỹ)
```

Ưu điểm:

- Giảm độ trễ (theo vị trí địa lý)
- Giảm tải cho máy chủ gốc (*origin server*)
- Chịu được lưu lượng tăng đột biến (*spikes*)

Dùng cho:

- Hình ảnh, video
- Các trang HTML tĩnh
- Nội dung công khai

Tầng 3: Bộ Nhớ Đệm Ứng Dụng (*Application Cache*) (Redis/Memcached)

Bộ nhớ đệm ở tầng ứng dụng.

```
# Redis cache
user = redis.get(f"user:{user_id}")
```

Ưu điểm:

- Linh hoạt (logic tùy biến)
- Dùng chung giữa các máy chủ (*servers*) ứng dụng
- Tra cứu nhanh

Dùng cho:

- Phản hồi (*responses*) của API
- Kết quả truy vấn cơ sở dữ liệu
- Dữ liệu phiên
- Dữ liệu đã tính toán sẵn

Tầng 4: Bộ Nhớ Đệm Cơ Sở Dữ Liệu (Bộ Nhớ Đệm Truy Vấn (*Query Cache*))

Bộ nhớ đệm ở tầng cơ sở dữ liệu.

Query cache của MySQL, shared buffers của PostgreSQL, v.v.

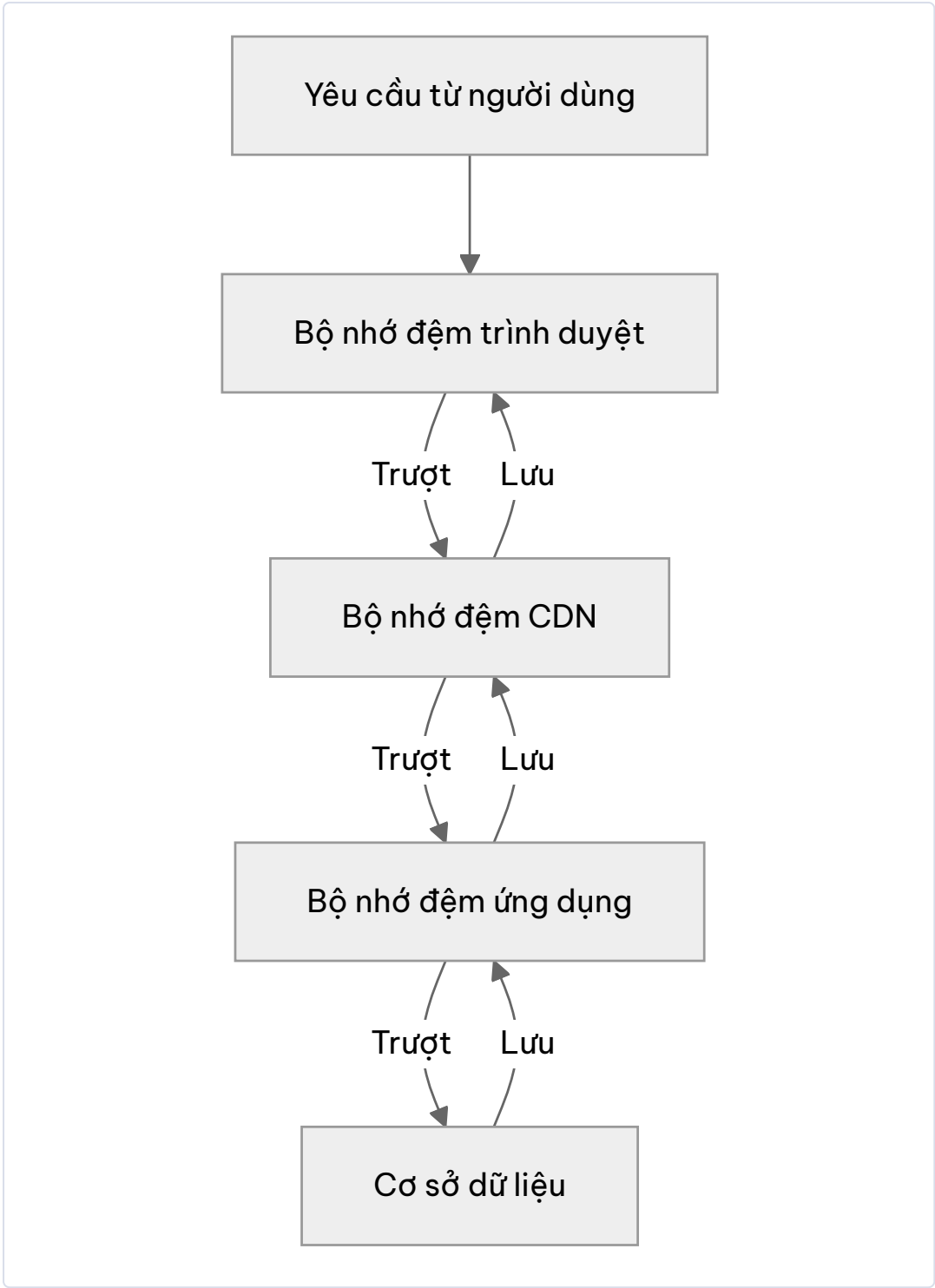
Ưu điểm:

- Trong suốt (ứng dụng không cần biết)
- Tối ưu các truy vấn (*queries*) lặp lại

Dùng cho:

- Các truy vấn tồn kém
- Các phép tổng hợp
- Kết quả phép join

Chiến Lược Nhiều Tầng



Luồng yêu cầu:

1. Kiểm tra bộ nhớ đệm trình duyệt → trúng = tức thì
2. Trượt → kiểm tra CDN → trúng = nhanh
3. Trượt → kiểm tra bộ nhớ đệm ứng dụng → trúng = tốt
4. Trượt → truy vấn cơ sở dữ liệu → chậm nhưng chấp nhận được

Mỗi tầng đều giảm tải cho tầng phía sau.

So Sánh Các Mẫu Lưu Đệm

Mẫu	Hiệu năng (<i>Perf</i>) ghi	Hiệu năng đọc	Tính nhất quán	Độ phức tạp
Cache-Aside	Nhanh	Tốt	Cuối cùng	Thấp
Write-Through	Chậm	Tốt nhất	Mạnh	Trung bình
Write-Behind	Tốt nhất	Tốt nhất	Cuối cùng	Cao

Không có mẫu tốt nhất. Chỉ có mẫu đúng cho từng tình huống sử dụng.

Mô Hình Tư Duy: Hiệu Năng = Đánh Đổi Tính Nhất Quán Lấy Tốc Độ

Điểm mấu chốt của lưu đệm:

Bạn đang đánh đổi tính nhất quán để lấy tốc độ.

- Cache-aside: Chấp nhận dữ liệu cũ trong khoảng thời gian TTL
- Write-through: Hy sinh tốc độ ghi để có tính nhất quán
- Write-behind: Chấp nhận rủi ro mất dữ liệu để có tốc độ cực cao

Không có bữa trưa nào miễn phí.

Kiến trúc sư giỏi biết:

- Tình huống sử dụng cần mức nhất quán nào?
- Sự đánh đổi nào chấp nhận được?
- Rủi ro nào có thể giảm thiểu?

Ví dụ về các quyết định:

Giá sản phẩm trong thương mại điện tử:

- Cần nhất quán mạnh → Write-through
- Giá cũ = đơn hàng sai = mất tiền

Số lượt thích trên mạng xã hội:

- Chấp nhận nhất quán cuối cùng → Write-behind
- Số đếm cũ vài giây = chấp nhận được
- Thông lượng ghi cực cao = then chốt

Bài viết blog:

- Chấp nhận dữ liệu cũ → Cache-aside với TTL
- Nội dung ít thay đổi
- Hiệu năng quan trọng hơn tính thời gian thực

Những Điểm Cốt Lõi

1. Lưu đệm hiệu quả nhờ tính cục bộ của tham chiếu

80% lưu lượng rơi vào 20% dữ liệu. Hãy lưu đệm đúng 20% đó.

2. Ba mẫu chính: Cache-Aside, Write-Through, Write-Behind

Mỗi mẫu có sự đánh đổi khác nhau. Hãy chọn dựa trên tình huống sử dụng.

3. Vô hiệu hoá bộ nhớ đệm rất khó

Kết hợp TTL, dựa trên sự kiện, dựa trên phiên bản. Không có giải pháp hoàn hảo.

4. Loại bỏ khỏi bộ nhớ đệm: LRU cho truy cập theo thời gian, LFU cho dữ liệu phổ biến lâu dài

LRU mặc định của Redis đáp ứng được hầu hết các trường hợp.

5. Lưu đệm nhiều tầng: Trình duyệt → CDN → Ứng dụng → Cơ sở dữ liệu

Mỗi tầng giảm tải cho tầng phía sau. Phòng thủ nhiều lớp.

6. Mô hình tư duy: Đánh đổi tính nhất quán lấy tốc độ

Hiểu sự đánh đổi, chọn đúng mức độ cho từng tình huống sử dụng.

7. Bộ nhớ đệm là sự tối ưu, không phải yêu cầu (*requirement*) bắt buộc

Bộ nhớ đệm gặp sự cố không được làm cả hệ thống sập. Luôn phải có phương án dự phòng.

Hãy nhớ: Kiến trúc sư (*architects*) giỏi không chỉ thêm bộ nhớ đệm vào. Họ hiểu nên lưu đệm dữ liệu nào, lưu đệm ở đâu, và họ đang từ bỏ những đảm bảo nhất quán nào.

CDN & Điện Toán Biên (Edge Computing) - Bộ Nhớ Đệm Phân Tán (Distributed Cache) Gần Người Dùng

Bạn có một máy chủ (*server*) API ở Singapore. Người dùng ở Brazil truy cập website → độ trễ (*latency*) 300ms chỉ riêng cho một vòng khứ hồi mạng (*network round trip*).

Mã của bạn tối ưu đến đâu cũng không quan trọng. **Không thể ăn gian các định luật vật lý.**

Ánh sáng đi qua cáp quang có tốc độ giới hạn. Khoảng cách từ Singapore → Brazil = ~18.000km. Một vòng khứ hồi (*Round trip*) tối thiểu mất ~240ms, thực tế là 300-400ms do định tuyến (*routing*).

Giải pháp duy nhất: Đưa nội dung đến gần người dùng hơn.

Đây chính là lý do CDN (Mạng phân phối nội dung (*Content Delivery Network*)) tồn tại.

Bài này dạy bạn:

- Kiến trúc CDN trong thực tế
- Cách tối ưu nội dung tĩnh (*static content*) và nội dung động (*dynamic content*)
- Vô hiệu hoá bộ nhớ đệm (*Cache invalidation*) ở điểm biên (*edge*)
- Điện toán biên - chạy logic gần người dùng
- Khi nào CDN thực sự cần thiết

CDN Là Gì? (Không Chỉ Là Bộ Nhớ Đệm (*Cache*))

CDN = Hệ thống bộ nhớ đệm phân tán được triển khai trên toàn cầu tại các vị trí biên (*edge locations*).

Nhưng CDN không chỉ là bộ nhớ đệm đơn thuần. CDN là:

1. Phân bố theo địa lý

- Có máy chủ (*servers*) ở nhiều địa điểm trên thế giới

- Người dùng được tự động định tuyến (*route*) đến máy chủ gần nhất

2. Lưu đệm (*Caching*) chuyên biệt

- Tối ưu cho nội dung tĩnh (hình ảnh, video, CSS, JS)
- Xử lý lượng yêu cầu (*requests*) đồng thời khổng lồ
- Kiến trúc bộ nhớ đệm phân tầng (*tiered cache*)

3. Tối ưu mạng

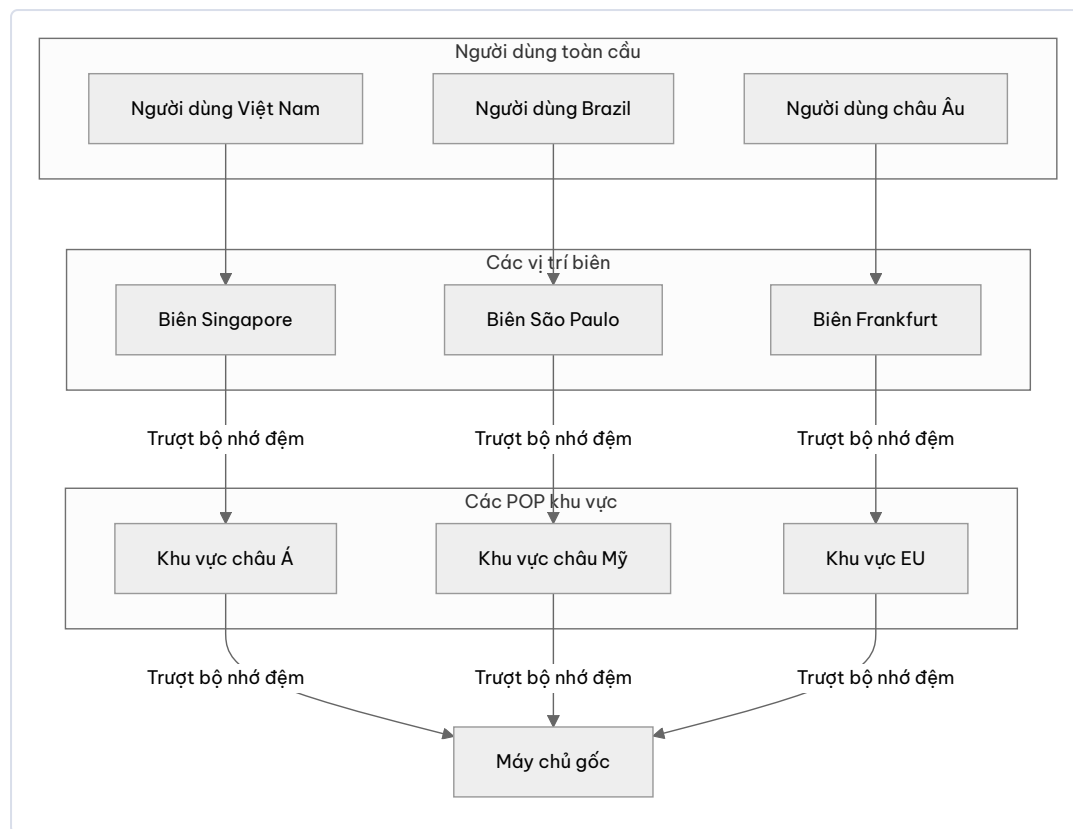
- Định tuyến tốt hơn internet công cộng
- Tối ưu kết nối TCP
- Hỗ trợ HTTP/2, HTTP/3

4. Lớp bảo mật

- Chống DDoS
- WAF (Tường lửa ứng dụng web (*Web Application Firewall*))
- Kết thúc (*termination*) SSL/TLS

Mô hình tư duy (*Mental model*): CDN = tầng bộ nhớ đệm (*cache layer*) + tầng mạng + tầng bảo mật ở điểm biên.

Kiến Trúc CDN - Từ Máy Chủ Gốc (Origin) Đến Điểm Biên



Kiến trúc CDN phân tầng (tiered) (ví dụ Cloudflare, Akamai):

Tầng 1: Vị Trí Biên (200+ địa điểm)

Ở gần người dùng nhất.

- Yêu cầu (Request) của người dùng đến điểm biên gần nhất (độ trễ thấp nhất)
- Lưu đệm nội dung tĩnh
- Xử lý phần lớn yêu cầu

Tầng 2: Các POP Khu Vực (Điểm Hiện Diện (Point of Presence))

Tầng tổng hợp.

- Điểm biên trượt → kiểm tra tầng khu vực
- Giảm tải (load) cho máy chủ gốc

- Che chắn máy chủ gốc khỏi lưu lượng (traffic) tăng đột biến (spike)

Tầng 3: Máy Chủ Gốc (Origin Server)

Máy chủ thực sự của bạn.

- Chỉ phục vụ khi cả điểm biên lẫn tầng khu vực đều trượt
- Được bảo vệ khỏi lưu lượng truy cập trực tiếp
- Xử lý việc tạo nội dung động

Luồng thực tế:

```

Người dùng (Việt Nam)
  → Biên (Singapore) [TRÚNG - 10ms] ✓

Người dùng (Thái Lan)
  → Biên (Bangkok) [TRƯỢT]
  → Khu vực (châu Á) [TRÚNG - 50ms] ✓

Người dùng (Myanmar - địa điểm mới)
  → Biên (Myanmar) [TRƯỢT]
  → Khu vực (châu Á) [TRƯỢT]
  → Máy chủ gốc (Singapore) [200ms] ⚠
  
```

Hơn 90% yêu cầu được phục vụ từ điểm biên/khu vực. Máy chủ gốc chỉ xử lý <10%.

Tiêu Đề Bộ Nhớ Đệm (Cache Headers) - Điều Khiển Hành Vi CDN

Hành vi lưu đệm của CDN được điều khiển bởi các tiêu đề (headers) HTTP.

Tiêu Đề (Header) Cache-Control

Quan trọng nhất để điều khiển việc lưu đệm.

```
Cache-Control: public, max-age=31536000, immutable
```

Các chỉ thị (Directives):

public

- Nội dung có thể được lưu đệm bởi bất kỳ ai (CDN, trình duyệt, proxy)
- Dùng cho nội dung công khai

private

- Chỉ trình duyệt lưu đệm, CDN không lưu đệm
- Dùng cho nội dung riêng của từng người dùng

max-age=<số giây>

- Bộ nhớ đệm còn hiệu lực trong bao lâu
- `max-age=3600` = lưu đệm 1 giờ

s-maxage=<số giây>

- Ghi đè max-age cho bộ nhớ đệm dùng chung (*shared cache*) (CDN)
- Trình duyệt dùng max-age, CDN dùng s-maxage

immutable

- Nội dung sẽ không bao giờ thay đổi
- CDN không cần xác thực lại (*revalidate*)

no-cache

- Phải xác thực lại với máy chủ gốc ở mỗi yêu cầu
- Bộ nhớ đệm có thể lưu trữ nhưng bắt buộc phải kiểm tra

no-store

- Không lưu đệm gì cả
- Dữ liệu nhạy cảm

Ví Dụ Thực Tế

Tài nguyên tĩnh (Static assets) (CSS/JS có đánh phiên bản (versioning)):

```
Cache-Control: public, max-age=31536000, immutable
```

- Lưu đệm 1 năm
- Không bao giờ thay đổi (vì có phiên bản trong URL)

- `style.abc123.css` → thay đổi mã → `style.def456.css`

Hình ảnh:

```
Cache-Control: public, max-age=86400
```

- Lưu đệm 1 ngày
- Xác thực lại sau đó

Phản hồi (Responses) API (có thể lưu đệm):

```
Cache-Control: public, max-age=300, s-maxage=600
```

- Trình duyệt lưu đệm 5 phút
- CDN lưu đệm 10 phút
- Nhất quán cuối cùng (*Eventual consistency*) là chấp nhận được

Dữ liệu riêng của người dùng:

```
Cache-Control: private, max-age=0, must-revalidate
```

- Chỉ trình duyệt lưu đệm
- Bắt buộc xác minh ở mỗi yêu cầu

Dữ liệu nhạy cảm:

```
Cache-Control: no-store
```

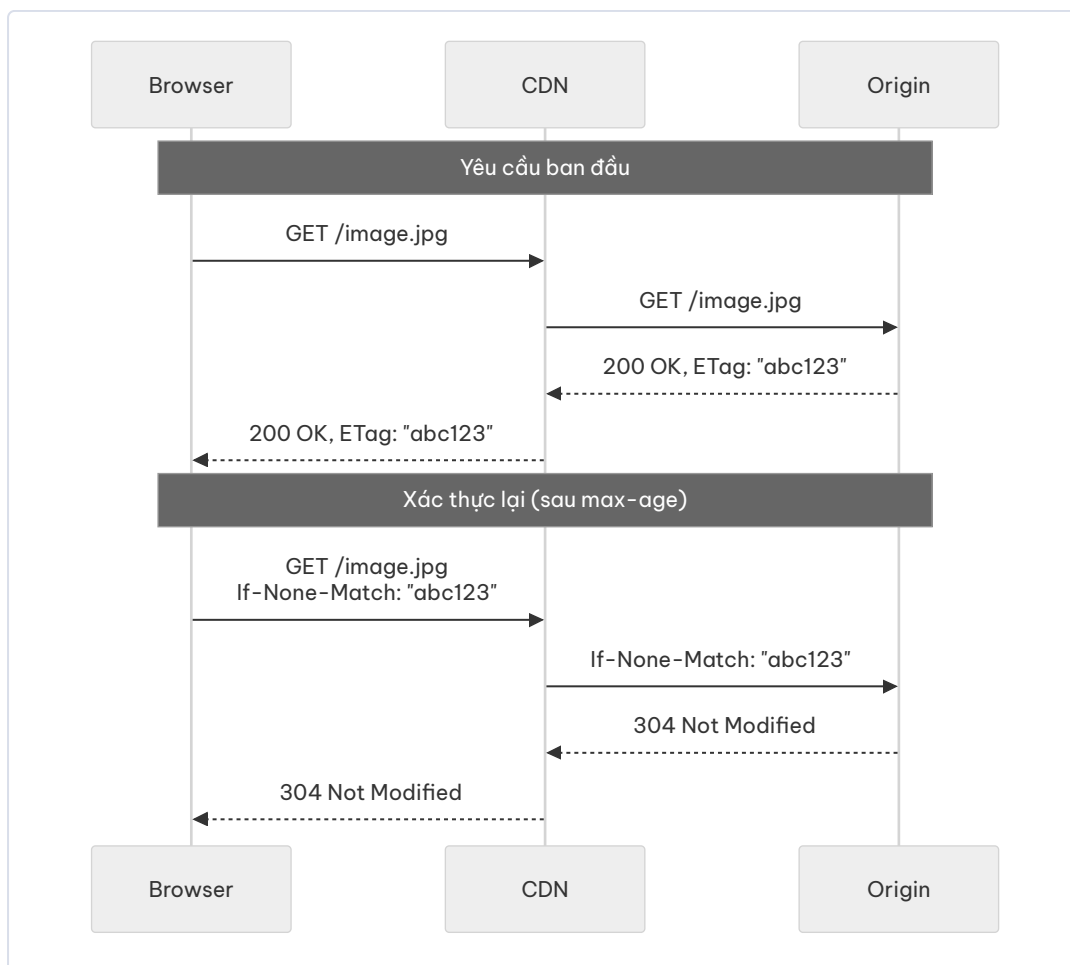
- Không lưu đệm ở bất kỳ đâu

ETag & Yêu Cầu Có Điều Kiện (*Conditional Requests*)

ETag = dấu vân tay của nội dung.

```
HTTP/1.1 200 OK
ETag: "abc123def456"
Cache-Control: public, max-age=3600
```

Luồng xác thực lại (*revalidation*):



304 Not Modified = nội dung chưa thay đổi, dùng lại bộ nhớ đệm.

Tiết kiệm băng thông (*Bandwidth*): Không phải truyền lại toàn bộ nội dung.

Nội Dung Tĩnh và Nội Dung Động - Chiến Lược Lưu Đệm (*Caching Strategy*)

Nội Dung Tĩnh

Nội dung không thay đổi hoặc ít thay đổi.

Ví dụ:

- Hình ảnh, video
- Tập CSS, JavaScript

- Phong chữ
- PDF, tài liệu

Chiến lược CDN:

```
Cache-Control: public, max-age=31536000, immutable
```

Tỷ lệ trúng bộ nhớ đệm (*cache hit*): **95%+**

Thực hành tốt nhất (*Best practices*):

- **URL có phiên bản:** `app.v123.js` thay vì `app.js`
- **Băm nội dung:** `bundle.abc123.js`
- **Tên miền riêng:** `static.example.com` (không có cookie)

Nội Dung Động

Nội dung được cá nhân hoá (*personalized*) **hoặc thay đổi thường xuyên.**

Ví dụ:

- Bảng điều khiển (*Dashboard*) của người dùng
- Phản hồi API
- Gợi ý được cá nhân hoá
- Dữ liệu thời gian thực (*real-time*)

Thách thức: Mỗi người dùng một khác, lưu đệm không hiệu quả?

Giải pháp: Các chiến lược lưu đệm (*caching strategies*) **thông minh.**

Chiến Lược 1: Lưu Đệm Phần Dùng Chung

```
<!-- Cached HTML template -->
<html>
  <body>
    <header>...</header>

    <!-- Dynamic part loaded via JS -->
    <div></div>

    <footer>...</footer>
  </body>
</html>
```

Lưu đệm khuôn mẫu (*template*), dữ liệu người dùng được lấy riêng.

Chiến Lược 2: Tiêu Đề Vary

```
Vary: Accept-Encoding, User-Agent
Cache-Control: public, max-age=300
```

CDN lưu đệm các bản sao riêng biệt dựa trên tiêu đề Vary.

Ví dụ:

- Máy tính và di động: HTML khác nhau
- Đã đăng nhập và chưa đăng nhập: nội dung khác nhau

Chiến Lược 3: Edge Side Includes (ESI - ghép trang ở điểm biên)

```
<html>
  <body>
    <esi:include src="/user/profile" />
    <esi:include src="/recommended-products" />
  </body>
</html>
```

CDN ghép trang từ nhiều phần:

- Phần tĩnh: lưu đệm lâu
- Phần động: lưu đệm ngắn hoặc bỏ qua bộ nhớ đệm

Vô Hiệu Hoá Bộ Nhớ Đệm Ở CDN - Bài Toán Khó

Vấn đề: Nội dung đã thay đổi ở máy chủ gốc, nhưng CDN vẫn phục vụ dữ liệu cũ (*stale data*).

Giải Pháp 1: API Xóa Bộ Nhớ Đệm (*Purge API*)

Ra lệnh rõ ràng cho CDN xóa bộ nhớ đệm.

```
# Cloudflare purge API
curl -X POST "https://api.cloudflare.com/client/v4/zones/{zone_id}/purge_cache" \
  -H "Authorization: Bearer {api_token}" \
  -d '{"files":["https://example.com/image.jpg"]}'
```

Khi nào dùng:

- Cập nhật nội dung quan trọng
- Người dùng tải lên ảnh đại diện mới
- Giá sản phẩm thay đổi

Nhược điểm:

- Không tức thì (có độ trễ lan truyền (*propagation delay*))
- Giới hạn tần suất (*Rate limits*) của API
- Độ phức tạp (*Complexity*) trong ứng dụng

Giải Pháp 2: Phá Bộ Nhớ Đệm (*Cache Busting*) (URL Có Phiên Bản)

Thay đổi URL khi nội dung thay đổi.

```
<!-- Old version -->
<script src="/app.js?v=1"></script>

<!-- New version -->
<script src="/app.js?v=2"></script>
```

Tốt hơn: Mã băm nội dung (*Content hash*)

```
<script src="/app.abc123.js"></script>
<!-- After change -->
<script src="/app.def456.js"></script>
```

Ưu điểm:

- Không cần API xoá bộ nhớ đệm
- Cập nhật tức thì
- Phiên bản cũ vẫn truy cập được (để hoàn tác (*rollback*))

Đây là thực hành tốt nhất (*best practice*) cho tài nguyên tĩnh.

Giải Pháp 3: TTL Ngắn + Stale-While-Revalidate

```
Cache-Control: max-age=60, stale-while-revalidate=86400
```

Hành vi:

- Còn mới trong 60 giây
- Cũ từ 60 giây đến 1 ngày: phục vụ bản cũ, xác thực lại bất đồng bộ (*async*)
- Sau 1 ngày: bắt buộc phải xác thực lại

Đánh đổi (*Trade-off*): Chấp nhận nhất quán cuối cùng để có hiệu năng (*performance*).

Giải Pháp 4: Khoá Thay Thế (*Surrogate Keys*) (Nâng Cao)

Gắn thẻ các nội dung liên quan bằng các khoá.

```
Surrogate-Key: product-123 category-shoes
```

Xoá theo thẻ:

```
# Update product 123 → purge all related
curl -X PURGE "https://example.com/" \
  -H "Surrogate-Key: product-123"
```

Xoá nhiều URL cùng lúc mà không cần biết chính xác từng URL.

Điện Toán Biên - Chạy Logic Gần Người Dùng

Sự tiến hoá của CDN: Từ lưu đệm → tính toán.

Điện toán biên = chạy mã tại các vị trí biên, không chỉ lưu đệm.

Tình Huống Sử Dụng (*Use Cases*)

1. Kiểm Thử A/B (*A/B Testing*) Ở Điểm Biên

```
// Cloudflare Workers example
addEventListener('fetch', event => {
  event.respondWith(handleRequest(event.request))
})

async function handleRequest(request) {
  const variant = Math.random() < 0.5 ? 'A' : 'B'

  // Modify response based on variant
  const response = await fetch(request)

  return new Response(response.body, {
    ...response,
    headers: {
      ...response.headers,
      'X-Variant': variant
    }
  })
}
```

Lợi ích: Không cần phía máy chủ (*backend*) xử lý logic A/B.

2. Xác Thực (*Authentication*)/Phân Quyền (*Authorization*) Ở Điểm Biên

```
async function handleRequest(request) {
  const token = request.headers.get('Authorization')

  // Verify JWT ở edge
  if (!verifyToken(token)) {
    return new Response('Unauthorized', { status: 401 })
  }

  // Pass to origin only if valid
  return fetch(request)
}
```


Lợi ích: Chặn các yêu cầu trái phép ngay ở điểm biên, bảo vệ máy chủ gốc.

3. Định Tuyến & Biến Đổi Yêu Cầu

```
async function handleRequest(request) {
  const url = new URL(request.url)

  // Route based on path
  if (url.pathname.startsWith('/api/v2')) {
    // Route to new backend
    return fetch('https://new-api.example.com' + url.pathname)
  }

  // Route to old backend
  return fetch('https://old-api.example.com' + url.pathname)
}
```

Triển khai blue-green (*Blue-green deployment*), **phát hành canary** (*canary release*) ngay ở điểm biên.

4. Biến Đổi Phản Hồi (*Response*) API

```
async function handleRequest(request) {
  const response = await fetch(request)
  const data = await response.json()

  // Transform response ở edge
  const transformed = {
    ...data,
    cached_at: Date.now(),
    edge_location: 'singapore'
  }

  return new Response(JSON.stringify(transformed))
}
```

Lợi ích: Máy chủ gốc trả dữ liệu thô, điểm biên tùy biến cho từng khu vực.

5. Nội Dung Dựa Trên Vị Trí Địa Lý (Geolocation)

```
async function handleRequest(request) {
  const country = request.cf.country // Cloudflare provides

  if (country === 'VN') {
    return fetch('https://api.example.com/content/vn')
  } else if (country === 'US') {
    return fetch('https://api.example.com/content/us')
  }

  return fetch('https://api.example.com/content/global')
}
```

Thực vụ nội dung bản địa hoá mà không cần gọi tới máy chủ gốc.

Các Nền Tảng Điện Toán Biên

Cloudflare Workers

- V8 isolates (khởi động nguội (*cold start*) nhanh)
- Mạng lưới toàn cầu
- Gói miễn phí: 100 nghìn yêu cầu/ngày

AWS Lambda@Edge

- Chạy cùng CloudFront
- Đầy đủ khả năng của Lambda
- Đắt hơn

Fastly Compute@Edge

- WebAssembly
- Độ trễ thấp

Vercel Edge Functions

- Tích hợp sẵn với Next.js
- Triển khai (*Deployment*) đơn giản

Hạn Chế Của Điện Toán Biên

1. Thời gian tính toán hạn chế

- Workers: 50ms thời gian CPU (miễn phí), 50-200ms (trả phí)
- Không thể chạy các tác vụ kéo dài

2. Bộ nhớ hạn chế

- Workers: giới hạn bộ nhớ 128MB
- Không thể nạp các tập dữ liệu lớn

3. Không có lưu trữ (storage) bền vững

- Các hàm phi trạng thái (stateless)
- Không thể lưu dữ liệu cục bộ

4. Khởi động nguội (tuỳ nền tảng)

- Lambda@Edge có khởi động nguội
- Workers thì không (V8 isolates)

Điện toán biên phù hợp cho các phép biến đổi nhẹ, không phải xử lý nặng.

Định Tuyến Theo Địa Lý (Geo Routing) - Phân Phối Lưu Lượng Thông Minh

Định tuyến theo địa lý = định tuyến người dùng đến máy chủ gần nhất dựa trên vị trí.

Định Tuyến Theo Địa Lý Dựa Trên DNS

Route 53, Cloudflare DNS:

Người dùng Việt Nam → truy vấn DNS
→ Trả về IP máy chủ Singapore (35.240.x.x)

Người dùng Mỹ → truy vấn DNS
→ Trả về IP máy chủ Mỹ (34.120.x.x)

Định tuyến theo độ trễ (Latency-based routing):

- Đo độ trễ từ vị trí người dùng đến các máy chủ
- Trả về máy chủ nhanh nhất

Định tuyến theo vị trí địa lý (Geolocation-based routing):

- Dựa trên vị trí IP của người dùng
- Trả về máy chủ gần nhất về mặt địa lý

Định Tuyến Anycast

Một địa chỉ IP, nhiều địa điểm.

IP của CDN: 104.16.0.0

Người dùng Việt Nam → tới POP Singapore

Người dùng Brazil → tới POP São Paulo

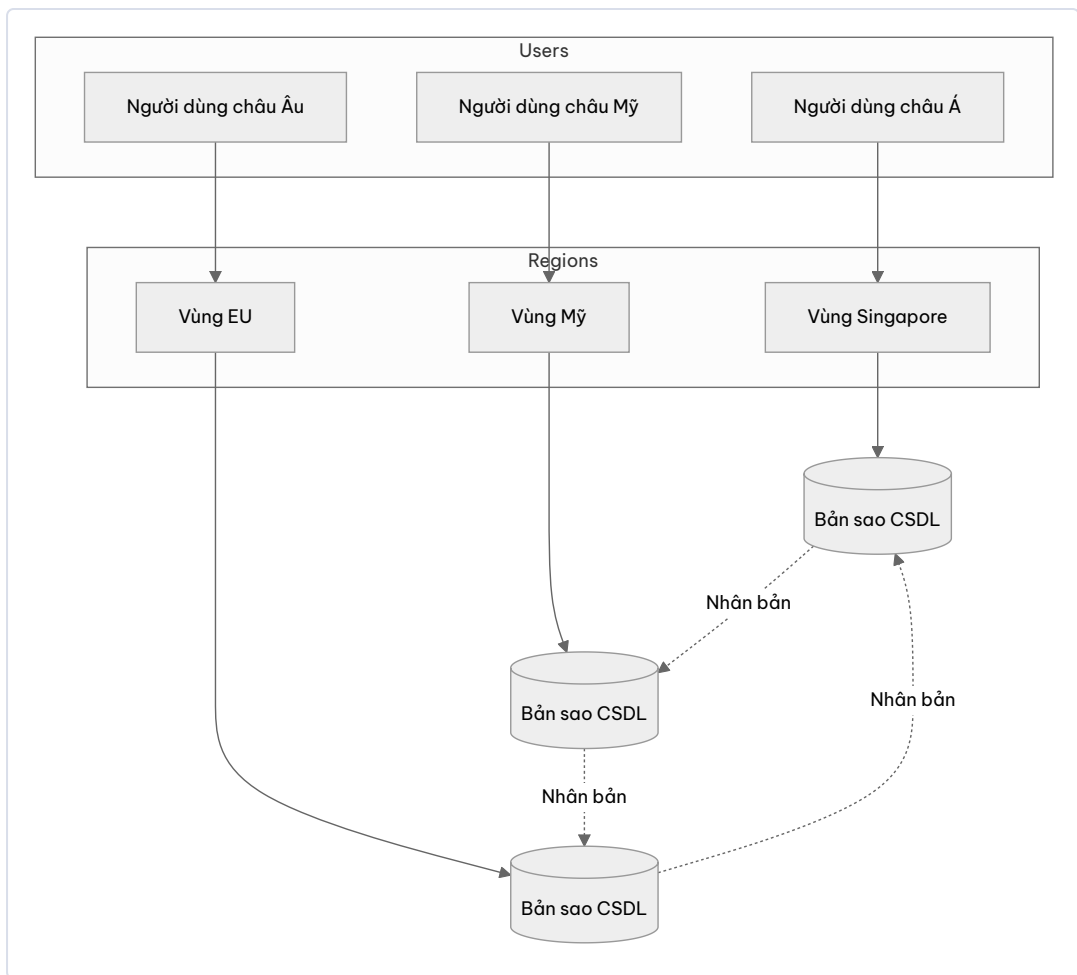
Mạng tự động định tuyến (routes) lưu lượng đến POP gần nhất.

Lợi ích:

- Đơn giản (1 IP cho mọi địa điểm)
- Tự động chuyển đổi dự phòng (failover)
- Giảm thiểu DDoS

Đa Vùng Chủ Động – Chủ Động (Active-Active Multi-Region)

Triển khai ứng dụng ở nhiều vùng, định tuyến một cách thông minh.



Lợi ích:

- Độ trễ thấp trên toàn thế giới
- Tính sẵn sàng cao (*High availability*) (chuyển đổi dự phòng sang vùng khác)

Thách thức:

- Tính nhất quán dữ liệu (*Data consistency*) (độ trễ nhân bản (*replication lag*))
- Chi phí (*Cost*) (nhiều vùng)
- Độ phức tạp

Khi Nào Cần CDN? (Khung Ra Quyết Định (Decision Framework))

CDN không phải lúc nào cũng cần thiết. Đây là khung ra quyết định:

Bạn CẦN CDN khi:

1. Người dùng trải khắp toàn cầu

- Người dùng ở nhiều quốc gia/châu lục
- Độ trễ ảnh hưởng đến trải nghiệm người dùng (UX)

2. Chủ yếu là nội dung tĩnh

- Hình ảnh, video chiếm phần lớn lưu lượng
- Sử dụng nhiều băng thông

3. Lưu lượng tăng đột biến (spikes)

- Flash sale, nội dung lan truyền mạnh
- Cần chống DDoS

4. Hiệu năng là yếu tố sống còn

- Thương mại điện tử (tỷ lệ chuyển đổi (conversion rate) phụ thuộc vào tốc độ)
- Trang media (phát video trực tuyến (video streaming))

Bạn KHÔNG CẦN CDN khi:

1. Người dùng chỉ ở trong nước

- Tất cả người dùng ở 1 quốc gia
- Một vùng là đủ

2. Chỉ có nội dung động

- Phía máy chủ chỉ cung cấp API
- Không có tài nguyên tĩnh

3. Lưu lượng thấp

- <1.000 người dùng/ngày

- Chi phí không xứng đáng

4. Công cụ nội bộ B2B

- Số người dùng hạn chế
- Hiệu năng không quá quan trọng

Giải Pháp Trung Gian: CDN Một Phần

CDN chỉ dành cho tài nguyên tĩnh, nội dung động đi thẳng tới máy chủ gốc.

Tĩnh (hình ảnh/CSS/JS) → CDN
Yêu cầu API → Đi thẳng tới máy chủ gốc

Đơn giản, tiết kiệm chi phí, lợi ích đáng kể.

So Sánh Các Nhà Cung Cấp CDN

Nhà cung cấp	Số POP toàn cầu	Thế mạnh	Giá
Cloudflare	300+	Chống DDoS, Workers	Có gói miễn phí
AWS CloudFront	450+	Tích hợp với AWS	Trả theo mức sử dụng
Fastly	70+	Xoá bộ nhớ đệm (<i>Purge</i>) theo thời gian thực, VCL	Giá cao cấp
Akamai	4100+	Doanh nghiệp lớn, nhiều POP nhất	Đắt
Bunny CDN	100+	Giá phải chăng	\$0.01/GB

Cloudflare = lựa chọn tốt nhất cho hầu hết công ty khởi nghiệp (*startups*) (gói miễn phí hào phóng, Workers mạnh mẽ).

AWS CloudFront = tốt nhất nếu đã dùng AWS (tích hợp chặt chẽ).

Fastly = tốt nhất cho doanh nghiệp lớn có nhu cầu thời gian thực.

Thực Hành Tốt Nhất - Triển Khai CDN

1. Đánh phiên bản cho tài nguyên tĩnh

```
<script src="/app.abc123.js"></script>
```

Lưu đệm bất biến, cập nhật tức thì.

2. Tên miền riêng cho nội dung tĩnh

```
www.example.com (động)  
static.example.com (CDN)
```

Tên miền không có cookie, lưu đệm tốt hơn.

3. Tối ưu hình ảnh

- Định dạng WebP
- Hình ảnh đáp ứng (*Responsive images*)
- Tải lười (*Lazy loading*)

4. Dùng tiêu đề bộ nhớ đệm đúng cách

```
Cache-Control: public, max-age=31536000, immutable # Static  
Cache-Control: public, max-age=300 # Dynamic
```

5. Giám sát (*Monitor*) tỷ lệ trùng bộ nhớ đệm

- Mục tiêu: 90%+ cho nội dung tĩnh
- <80% = cần xem lại cấu hình bộ nhớ đệm

6. Tự động hoá việc xoá bộ nhớ đệm

```
// Deploy hook  
await cdn.purge(['/index.html', '/api/latest'])
```

7. Kiểm thử từ nhiều địa điểm

- Dùng công cụ: Pingdom, GTmetrix

- Xác minh hiệu năng tại điểm biên

8. Bật HTTP/2 hoặc HTTP/3

- Ghép kênh (Multiplexing)
- Nén tiêu đề
- Hiệu năng tốt hơn

Những Điểm Cốt Lõi

1. CDN = bộ nhớ đệm phân tán + tối ưu mạng ở điểm biên

Không chỉ lưu đệm, còn có định tuyến, bảo mật, tính toán.

2. Kiến trúc phân tầng: Điểm biên → Khu vực → Máy chủ gốc

Phần lớn yêu cầu được phục vụ từ điểm biên, máy chủ gốc được bảo vệ.

3. Tiêu đề bộ nhớ đệm điều khiển hành vi của CDN

Cache-Control, ETag, Vary - hãy hiểu và dùng cho đúng.

4. Nội dung tĩnh: TTL dài + URL có phiên bản

Thực hành tốt nhất cho lưu đệm bất biến.

5. Nội dung động: các chiến lược lưu đệm thông minh

ESI, tiêu đề Vary, stale-while-revalidate.

6. Vô hiệu hoá bộ nhớ đệm: API xoá bộ nhớ đệm hoặc phá bộ nhớ đệm

URL có phiên bản > API xoá bộ nhớ đệm (đơn giản hơn, tức thì).

7. Điện toán biên: logic nhẹ chạy gần người dùng

Kiểm thử A/B, xác thực (auth), định tuyến - không phải xử lý nặng.

8. Định tuyến theo địa lý: dựa trên độ trễ hoặc dựa trên vị trí địa lý

Người dùng được tự động định tuyến đến máy chủ gần nhất.

9. Khung ra quyết định: Toàn cầu + nội dung tĩnh = cần CDN

Trong nước + chỉ có nội dung động = có thể bỏ qua.

Hãy nhớ: CDN không phải viên đạn bạc. Nó là sự đánh đổi giữa hiệu năng, tính nhất quán (*consistency*) và chi phí. Những kiến trúc sư (*architects*) giỏi biết khi nào cần, khi nào không.

Mở Rộng Và Tối Ưu Cơ Sở Dữ Liệu (Database) - Từ Tinh Chỉnh Truy Vấn (Query Tuning) Đến Phân Mảnh Dữ Liệu (Sharding)

Trong hầu hết các hệ thống, cơ sở dữ liệu là **điểm nghẽn** (bottleneck) **lớn nhất**.

Máy chủ ứng dụng (Application server) mở rộng theo chiều ngang (scale horizontal) rất dễ - thêm máy chủ (server), gắn thêm bộ cân bằng tải (load balancer), xong. Nhưng cơ sở dữ liệu thì sao? Đây là một bài toán khó.

Tại sao?

Cơ sở dữ liệu là **có trạng thái** (stateful). Không thể đơn giản thêm một máy chủ cơ sở dữ liệu rồi kỳ vọng nó tự động hoạt động. Dữ liệu phải được đồng bộ (sync), giao dịch (transactions) phải nhất quán (consistent), truy vấn (queries) phải được điều phối.

Khi cơ sở dữ liệu chậm, toàn bộ hệ thống chậm. Khi cơ sở dữ liệu sập, toàn bộ hệ thống sập.

Mô hình tư duy (Mental model) **quan trọng nhất**:

Mở rộng cơ sở dữ liệu không phải là một giải pháp duy nhất. Đó là một lộ trình tiến dần từ tối ưu (optimization) **đơn giản** → **các mẫu phân tán** (distributed patterns) **phức tạp**.

Bài này dạy bạn:

- Tối ưu truy vấn (Query optimization) và đánh chỉ mục (indexing) (tuyển phòng thủ đầu tiên)
- Bản sao chỉ đọc (Read replicas) (mở rộng việc đọc)
- Phân mảnh dữ liệu (mở rộng việc ghi và lưu trữ (storage))
- Khi nào dùng kỹ thuật nào
- Sự đánh đổi (Trade-offs) của từng cách tiếp cận

Tại Sao Cơ Sở Dữ Liệu Là Điểm Nghẽn?

Trước khi học cách mở rộng, hãy hiểu tại sao cơ sở dữ liệu lại giới hạn hiệu năng (*performance*).

1. Điểm Nghẽn Disk I/O

Cơ sở dữ liệu lưu dữ liệu trên đĩa. Đĩa chậm hơn RAM 1000 lần.

Đọc RAM: ~100 nano giây
Đọc SSD: ~100 micro giây (chậm hơn 1000 lần)
Đọc HDD: ~10 mili giây (chậm hơn 100.000 lần)

Mỗi truy vấn = các lần đọc đĩa.

Truy vấn càng phức tạp → càng nhiều disk I/O → càng chậm.

2. Các Đảm Bảo ACID

Cơ sở dữ liệu phải duy trì tính nhất quán (*consistency*).

- **Tính nguyên tử** (*Atomicity*): Giao dịch hoặc thực hiện toàn bộ, hoặc không thực hiện gì
- **Tính nhất quán**: Dữ liệu hợp lệ theo các ràng buộc (*constraints*)
- **Tính cô lập** (*Isolation*): Các giao dịch đồng thời (*concurrent transactions*) không xung đột nhau
- **Độ bền dữ liệu** (*Durability*): Dữ liệu đã commit (*committed*) không bị mất

ACID có chi phí (*cost*):

- Khoá (*Locking*) (chặn truy cập đồng thời)
- Nhật ký ghi trước (*Write-ahead log*) (tốn thêm lượt ghi)
- Điều phối giao dịch (chi phí phụ (*overhead*))

Sự đánh đổi (*Trade-off*): Tính đúng đắn và hiệu năng.

3. Vấn Đề Chỉ Có Một Nút Ghi

Hầu hết RDBMS có kiến trúc một nút ghi (*single-writer*).

Chỉ 1 nút (*node*) xử lý việc ghi để duy trì tính nhất quán.

Việc đọc có thể mở rộng (bản sao (*replicas*)), nhưng việc ghi là điểm nghẽn.

4. Dữ Liệu Tăng Trưởng

Dữ liệu tăng → hiệu năng giảm.

Bảng 1K dòng: Truy vấn 10ms
Bảng 100K dòng: Truy vấn 50ms
Bảng 10M dòng: Truy vấn 5000ms

Không có chỉ mục phù hợp, truy vấn sẽ chậm đi theo cấp số nhân.

Điểm mấu chốt: Phải tối ưu trước khi mở rộng. Mở rộng một hệ thống tồn kém thì vẫn tồn kém.

Chiến Lược Chỉ Mục (*Index*) - Tuyển Phòng Thủ Đầu Tiên

Chỉ mục là cấu trúc dữ liệu giúp cơ sở dữ liệu tìm dữ liệu nhanh hơn.

Không có chỉ mục = quét toàn bảng (*full table scan*) (đọc toàn bộ các dòng).

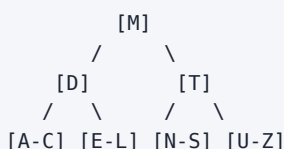
Có chỉ mục = tra cứu trực tiếp (*direct lookup*) (giống mục lục của một cuốn sách).

Chỉ Mục B-Tree (Mặc Định)

Loại chỉ mục phổ biến nhất.

```
CREATE INDEX idx_users_email ON users(email);
```

Cấu trúc:



Thời gian tra cứu: $O(\log N)$ - cực nhanh với các tập dữ liệu lớn.

Khi nào dùng:

- Các cột trong mệnh đề WHERE
- Các khoá JOIN
- Các cột ORDER BY
- Khoá ngoại (*Foreign keys*)

Chỉ Mục Kết Hợp (*Composite Index*)

Chỉ mục trên nhiều cột.

```
CREATE INDEX idx_orders_user_date
ON orders(user_id, created_at);
```

Hiệu quả cho:

```
-- Uses index (leftmost column)
WHERE user_id = 123

-- Uses full index
WHERE user_id = 123 AND created_at > '2025-01-01'

-- Không dùng index (missing leftmost column)
WHERE created_at > '2025-01-01'
```

Quy tắc: Chỉ mục được khớp từ trái sang phải.

Chỉ Mục Bao Phủ (*Covering Index*)

Chỉ mục chứa tất cả các cột mà truy vấn cần.

```
-- Query này
SELECT user_id, created_at, total
FROM orders
WHERE user_id = 123;

-- Covering index
CREATE INDEX idx_orders_covering
ON orders(user_id, created_at, total);
```

Lợi ích: Cơ sở dữ liệu không cần truy cập bảng, chỉ đọc chỉ mục → cực nhanh.

Chỉ Mục Một Phần (Partial Index)

Chỉ mục trên một tập con các dòng.

```
-- Chỉ index active users
CREATE INDEX idx_active_users
ON users(email)
WHERE status = 'active';
```

Chỉ mục nhỏ hơn → tra cứu nhanh hơn, tốn ít dung lượng lưu trữ hơn.

Đánh Đổi Khi Dùng Chỉ Mục

Quá nhiều chỉ mục:

- Ghi chậm (phải cập nhật tất cả chỉ mục)
- Tồn thêm dung lượng lưu trữ
- Bộ lập kế hoạch truy vấn (*Query planner*) bị "rối"

Quá ít chỉ mục:

- Đọc chậm (quét toàn bảng)

Cân bằng: Đánh chỉ mục cho các truy vấn có lưu lượng (*traffic*) cao, theo dõi mức độ sử dụng.

```
-- PostgreSQL: Find unused indexes
SELECT schemaname, tablename, indexname
FROM pg_stat_user_indexes
WHERE idx_scan = 0;
```

Tối Ưu Truy Vấn - Làm Truy Vấn Nhanh Hơn

Kể cả có chỉ mục, truy vấn viết tệ vẫn chậm.

EXPLAIN ANALYZE - Người Bạn Thân Nhất Của Bạn

Xem kế hoạch thực thi truy vấn (*query execution plan*).

```
EXPLAIN ANALYZE
SELECT * FROM orders
WHERE user_id = 123
ORDER BY created_at DESC
LIMIT 10;
```

Kết quả:

```
Limit (cost=0.42..8.44 rows=10)
-> Index Scan using idx_orders_user_date on orders
    Index Cond: (user_id = 123)
    Rows Removed by Filter: 0
Planning Time: 0.123 ms
Execution Time: 0.456 ms
```

Cần để ý:

- **Seq Scan** (quét tuần tự toàn bảng) → thêm chỉ mục
- **Cost cao** → tối ưu truy vấn
- **Rows removed by filter** (số dòng bị bộ lọc loại bỏ) → cải thiện mệnh đề WHERE

SELECT * Là Tai Hoạ

Tệ:

```
SELECT * FROM users WHERE id = 123;
```

Lấy về tất cả các cột (có thể tới 50 cột, nhiều KB dữ liệu).

Tốt:

```
SELECT id, name, email FROM users WHERE id = 123;
```

Chỉ lấy những cột cần thiết.

Tác động:

- Ít disk I/O hơn

- Ít dữ liệu truyền qua mạng hơn
- Tận dụng bộ nhớ đệm (*cache*) tốt hơn

Tránh Dùng OR Trong Mệnh Đề WHERE

Tệ:

```
SELECT * FROM users
WHERE email = 'test@example.com'
OR username = 'testuser';
```

Cơ sở dữ liệu phải quét 2 chỉ mục hoặc quét toàn bảng.

Tốt:

```
SELECT * FROM users WHERE email = 'test@example.com'
UNION
SELECT * FROM users WHERE username = 'testuser';
```

2 lượt quét chỉ mục riêng biệt, sau đó gộp lại.

Dùng LIMIT

Tệ:

```
SELECT * FROM orders ORDER BY created_at DESC;
```

Lấy về hàng triệu dòng, sắp xếp toàn bộ.

Tốt:

```
SELECT * FROM orders
ORDER BY created_at DESC
LIMIT 20;
```

Dừng sau 20 dòng, không phải sắp xếp toàn bộ.

Tránh Dùng Hàm Trong WHERE

Tệ:

```
WHERE YEAR(created_at) = 2025
```

Không thể dùng chỉ mục (vì hàm được áp lên cột).

Tốt:

```
WHERE created_at >= '2025-01-01'
AND created_at < '2026-01-01'
```

Chỉ mục có thể dùng được.

Vấn Đề N+1 - Sát Thủ Thầm Lặng Của Hiệu Năng

N+1 là một trong những lỗi (*bugs*) phổ biến và nguy hiểm nhất.

Vấn Đề

```
# Fetch 10 users (1 query)
users = db.query("SELECT * FROM users LIMIT 10")

# For each user, fetch orders (10 queries)
for user in users:
    orders = db.query("SELECT * FROM orders WHERE user_id = ?", user.id)
    print(f"{user.name}: {len(orders)} orders")
```

Tổng cộng: 1 + 10 = 11 truy vấn.

Với 100 người dùng (*users*) → 101 truy vấn.

Với 1000 người dùng → 1001 truy vấn.

Mỗi truy vấn có độ trễ (*latency*) 10ms → 1001 truy vấn = 10 giây!

Giải Pháp 1: Tải Sớm (*Eager Loading*)

```
# Single query with JOIN
query = """
SELECT u.*, o.id as order_id, o.total
FROM users u
LEFT JOIN orders o ON u.id = o.user_id
LIMIT 10
"""

results = db.query(query)
```

1 truy vấn thay vì N+1.

Giải Pháp 2: Tải Theo Lô (*Batch Loading*)

```
# Fetch users
users = db.query("SELECT * FROM users LIMIT 10")
user_ids = [u.id for u in users]

# Batch fetch all orders
orders = db.query(
    "SELECT * FROM orders WHERE user_id IN (?)",
    user_ids
)

# Group orders by user
orders_by_user = defaultdict(list)
for order in orders:
    orders_by_user[order.user_id].append(order)
```

Tổng cộng 2 truy vấn.

Giải Pháp Từ Các Framework

Rails:

```
User.includes(:orders).limit(10)
```

Django:

```
User.objects.prefetch_related('orders')[:10]
```

TypeORM:

```
userRepository.find({ relations: ['orders'], take: 10 })
```

Luôn kiểm tra câu SQL được sinh ra để tránh N+1.

Phi Chuẩn Hoá (*Denormalization*) - Đánh Đổi Tính Nhất Quán Lấy Hiệu Năng

Chuẩn hoá (*Normalization*) = tránh trùng lặp dữ liệu.

Phi chuẩn hoá = chủ động nhân đôi dữ liệu để đạt hiệu năng.

Ví Dụ: Tổng Giá Trị Đơn Hàng

Đã chuẩn hoá (chậm):

```
-- Calculate total mỗi lần query
SELECT o.id, SUM(oi.price * oi.quantity) as total
FROM orders o
JOIN order_items oi ON o.id = oi.order_id
GROUP BY o.id;
```

Phép tổng hợp (*aggregation*) tốn kém ở mỗi yêu cầu (*request*).

Đã phi chuẩn hoá (nhanh):

```
-- Store pre-computed total
ALTER TABLE orders ADD COLUMN total DECIMAL(10,2);

-- Update when order items change
UPDATE orders SET total = ? WHERE id = ?;

-- Query simple
SELECT id, total FROM orders;
```

Sự đánh đổi: Tồn thêm dung lượng lưu trữ + cập nhật phức tạp hơn để đổi lấy hiệu năng đọc.

Khi Nào Nên Phi Chuẩn Hoá (Denormalize)

Phi chuẩn hoá khi:

- Các phép tổng hợp (*aggregations*) tồn kém
- Dữ liệu ít thay đổi
- Khối lượng công việc thiên về đọc (*Read-heavy workload*)

Không phi chuẩn hoá khi:

- Dữ liệu thay đổi thường xuyên
- Việc đồng bộ hoá (*synchronization*) phức tạp
- Yêu cầu nhất quán mạnh (*strong consistency*)

Khung Nhìn Cụ Thể Hoá (Materialized Views)

Phi chuẩn hoá ở cấp cơ sở dữ liệu.

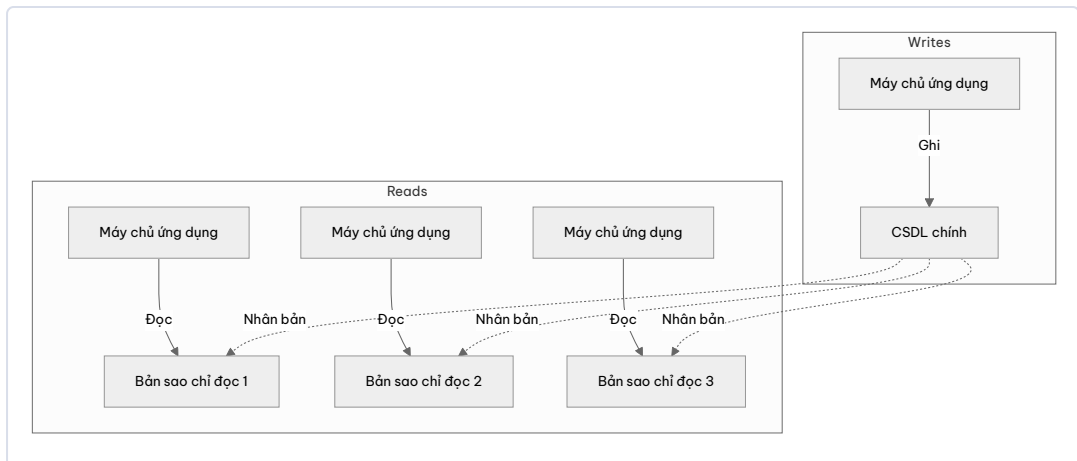
```
-- PostgreSQL
CREATE MATERIALIZED VIEW user_stats AS
SELECT
    user_id,
    COUNT(*) as order_count,
    SUM(total) as total_spent
FROM orders
GROUP BY user_id;

-- Refresh periodically
REFRESH MATERIALIZED VIEW user_stats;
```

Kết quả được tính toán trước, truy vấn nhanh.

Bản Sao Chỉ Đọc - Mở Rộng Việc Đọc Theo Chiều Ngang

Bản sao chỉ đọc (*Read replica*) = bản sao của cơ sở dữ liệu chính (*primary database*), xử lý lưu lượng đọc.



Cách Hoạt Động

1. Cơ sở dữ liệu chính xử lý việc ghi

- Mọi lệnh INSERT, UPDATE, DELETE → máy chủ chính (*primary*)

2. Thay đổi được nhân bản (*replicate*) sang các bản sao

- Nhân bản bất đồng bộ (*Async replication*) (thường là vậy)
- Chuyển binary log (*Binary log shipping*) (MySQL) hoặc chuyển WAL (*WAL shipping*) (PostgreSQL)

3. Các bản sao xử lý việc đọc

- Truy vấn SELECT → các bản sao
- Phân bổ tải (*load*) cho nhiều bản sao

Lợi Ích

Mở rộng lưu lượng đọc:

- Thêm bản sao khi lưu lượng đọc tăng
- Máy chủ chính không bị quá tải

Phân bổ theo địa lý:

- Bản sao (*Replica*) ở Singapore cho người dùng châu Á
- Bản sao ở Mỹ cho người dùng Mỹ
- Độ trễ thấp hơn

Truy vấn phân tích:

- Chạy các báo cáo tồn kém trên bản sao
- Không ảnh hưởng tới lưu lượng của môi trường thực tế (*production*)

Độ Trễ Nhân Bản (*Replication Lag*)

Vấn đề: Các bản sao không được cập nhật ngay lập tức.

```
t=0: Ghi vào máy chủ chính (user.status = 'active')
t=1: Đọc từ bản sao (user.status = 'pending') ← dữ liệu cũ!
t=2: Nhân bản bắt kịp (user.status = 'active')
```

Độ trễ thường gặp: 10-100ms, có thể cao hơn.

Xử Lý Độ Trễ Nhân Bản

1. Nhất quán đọc-sau-ghi (*Read-after-write consistency*)

```
def update_profile(user_id, data):
    # Write to primary
    db_primary.execute("UPDATE users SET ... WHERE id = ?", user_id)

    # Read from primary (not replica)
    return db_primary.query("SELECT * FROM users WHERE id = ?", user_id)
```

2. Gắn phiên (*Session stickiness*)

- Phiên (*session*) của người dùng luôn đọc từ cùng một bản sao
- Trải nghiệm nhất quán

3. Các lượt đọc quan trọng lấy từ máy chủ chính

```
# Payment - critical
balance = db_primary.query("SELECT balance FROM accounts")

# Profile view - không critical
profile = db_replica.query("SELECT * FROM users")
```

Các Kiểu Nhân Bản (Replication)

Bất đồng bộ (Asynchronous) (mặc định):

- Máy chủ chính không đợi bản sao xác nhận
- Ghi nhanh
- Rủi ro: bản sao bị trễ, có thể mất dữ liệu

Đồng bộ (Synchronous):

- Máy chủ chính đợi ít nhất 1 bản sao xác nhận
- Ghi chậm hơn
- Nhất quán mạnh

Bán đồng bộ (Semi-synchronous):

- Đợi 1 bản sao, các bản sao còn lại bất đồng bộ (async)
- Cân bằng giữa hiệu năng và độ bền dữ liệu

Phân Mảnh Dữ Liệu - Mở Rộng Việc Ghi Và Lưu Trữ

Phân mảnh dữ liệu = chia cơ sở dữ liệu thành nhiều cơ sở dữ liệu, mỗi cơ sở dữ liệu chứa một tập con của dữ liệu.

Đây là giải pháp mở rộng tối thượng, nhưng cũng phức tạp nhất.

Tại Sao Cần Phân Mảnh Dữ Liệu?

Bản sao chỉ đọc chỉ mở rộng được việc đọc. Việc ghi vẫn bị nghẽn ở máy chủ chính.

Phân mảnh dữ liệu mở rộng được cả đọc lẫn ghi:

- Lưu lượng ghi được phân bổ cho các mảnh (shards)
- Dung lượng lưu trữ được phân bổ cho các mảnh
- Mỗi mảnh (shard) nhỏ hơn → truy vấn nhanh hơn

Các Chiến Lược Phân Mảnh Dữ Liệu

1. Phân Mảnh Theo Khoảng (Range-Based Sharding)

Chia dữ liệu theo khoảng giá trị.

```
Shard 1: user_id 1-1000000  
Shard 2: user_id 1000001-2000000  
Shard 3: user_id 2000001-3000000
```

```
def get_shard(user_id):  
    if user_id <= 1000000:  
        return shard_1  
    elif user_id <= 2000000:  
        return shard_2  
    else:  
        return shard_3
```

Ưu điểm:

- Logic đơn giản
- Truy vấn theo khoảng (Range queries) hiệu quả

Nhược điểm:

- Phân bố không đều (điểm nóng (hotspots))
- Khó tái cân bằng (rebalance)

2. Phân Mảnh Theo Hàm Băm (Hash-Based Sharding)

Băm (Hash) khoá để xác định mảnh.

```
def get_shard(user_id):  
    shard_num = hash(user_id) % num_shards  
    return shards[shard_num]
```

Ưu điểm:

- Phân bố đồng đều
- Không có điểm nóng

Nhược điểm:

- Truy vấn theo khoảng phải truy vấn tất cả các mảnh
- Thêm mảnh = phải băm lại (*re-hash*) (phức tạp)

3. Phân Mảnh Theo Địa Lý (*Geographic Sharding*)

Chia theo vị trí địa lý.

Shard 1: Người dùng châu Á
Shard 2: Người dùng Mỹ
Shard 3: Người dùng châu Âu

Ưu điểm:

- Độ trễ thấp (dữ liệu ở gần người dùng)
- Tuân thủ quy định (luật về nơi lưu trữ dữ liệu (*data residency*))

Nhược điểm:

- Tải không đồng đều
- Truy vấn xuyên vùng tốn kém

4. Phân Mảnh Theo Thực Thể (*Entity-Based Sharding*) (Thực Tế)

Chia theo thực thể nghiệp vụ.

Shard 1: Dữ liệu của Tô'chức A
Shard 2: Dữ liệu của Tô'chức B
Shard 3: Dữ liệu của Tô'chức C

Phù hợp nhất cho các ứng dụng đa khách thuê (*multi-tenant*).

Những Thách Thức Của Phân Mảnh Dữ Liệu

1. Truy vấn xuyên mảnh (*Cross-shard queries*)

```
-- Query users across all shards
SELECT * FROM users WHERE created_at > '2025-01-01'
```

Phải truy vấn tất cả các mảnh → tổng hợp kết quả → chậm.

Giải pháp: Tránh truy vấn xuyên mảnh bằng cách thiết kế khoá phân mảnh (*shard key*) cẩn thận.

2. Giao dịch xuyên mảnh (Cross-shard transactions)

```
-- Transfer between users on different shards
BEGIN TRANSACTION;
  UPDATE accounts SET balance = balance - 100 WHERE user_id = 123; -- Shard 1
  UPDATE accounts SET balance = balance + 100 WHERE user_id = 456; -- Shard 2
COMMIT;
```

Giao dịch phân tán (Distributed transaction) **rất phức tạp**. Thường được tránh bằng cách:

- Nhất quán cuối cùng (Eventual consistency)
- Mẫu Saga
- Giao dịch bù trừ (Compensating transactions)

3. Tái cân bằng (Rebalancing)

Dữ liệu tăng trưởng không đều → một mảnh đầy trước.

Tái cân bằng = di chuyển dữ liệu giữa các mảnh.

Phức tạp, có rủi ro ngừng hoạt động (downtime), cần lên kế hoạch cẩn thận.

4. Phép JOIN

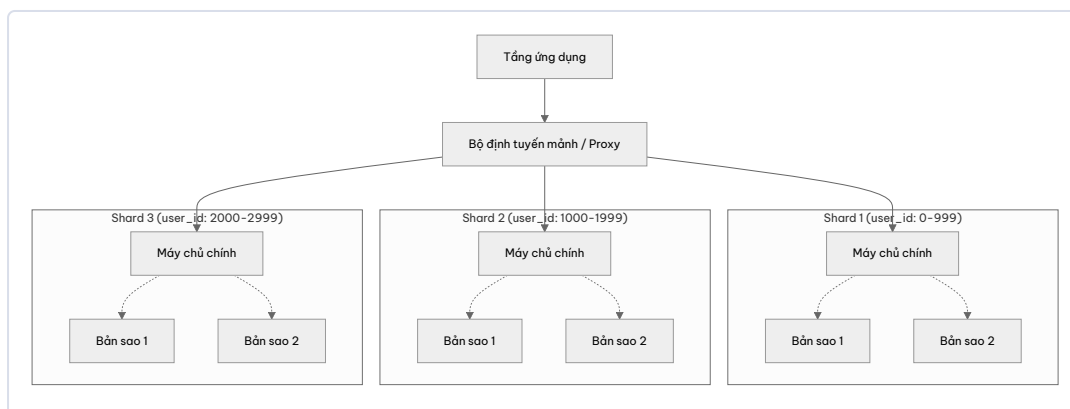
```
-- Join users + orders across shards
SELECT u.name, o.total
FROM users u
JOIN orders o ON u.id = o.user_id
```

Bảng users ở mảnh A, bảng orders ở mảnh B → không thể JOIN.

Giải pháp:

- Phi chuẩn hoá dữ liệu
- JOIN ở cấp ứng dụng
- Thiết kế lược đồ (schema) để tránh JOIN xuyên mảnh

Ví Dụ Kiến Trúc Phân Mảnh Dữ Liệu



Bộ định tuyến mảnh (*Shard router*) quyết định truy vấn đi đến mảnh nào dựa trên khoá phân mảnh.

Khi Nào Nên Phân Mảnh

Phân mảnh khi:

- Thông lượng (*throughput*) ghi vượt quá khả năng của một cơ sở dữ liệu đơn lẻ
- Dung lượng lưu trữ vượt quá năng lực của một máy chủ đơn lẻ
- Đã tối ưu truy vấn, chỉ mục (*indexes*), bản sao chỉ đọc
- Nhu cầu kinh doanh xứng đáng với độ phức tạp (*complexity*) bỏ ra

Chưa nên phân mảnh khi:

- Một cơ sở dữ liệu đơn lẻ vẫn đủ sức xử lý lưu lượng
- Chưa tối ưu truy vấn
- Chưa dùng bản sao chỉ đọc
- Nhóm (*Team*) chưa có chuyên môn

Nguyên tắc kinh nghiệm: Phân mảnh dữ liệu là phương án cuối cùng. Hãy dùng hết các giải pháp đơn giản hơn trước.

Phân Vùng (*Partitioning*) - Phân Mảnh Logic Bên Trong Một Cơ Sở Dữ Liệu

Phân vùng = chia một bảng thành nhiều phân vùng (*partitions*) vật lý, nhưng về logic vẫn là 1 bảng.

Tương tự phân mảnh dữ liệu, nhưng do cơ sở dữ liệu quản lý, không phải ứng dụng.

Các Kiểu Phân Vùng

Phân Vùng Theo Khoảng (*Range Partitioning*)

```
-- PostgreSQL
CREATE TABLE orders (
    id BIGINT,
    user_id BIGINT,
    created_at TIMESTAMP
) PARTITION BY RANGE (created_at);

CREATE TABLE orders_2024
PARTITION OF orders
FOR VALUES FROM ('2024-01-01') TO ('2025-01-01');

CREATE TABLE orders_2025
PARTITION OF orders
FOR VALUES FROM ('2025-01-01') TO ('2026-01-01');
```

Truy vấn được tự động định tuyến đến đúng phân vùng (*partition*).

```
-- Chỉ scan orders_2025 partition
SELECT * FROM orders
WHERE created_at >= '2025-02-01';
```

Phân Vùng Theo Hàm Băm (Hash Partitioning)

```
CREATE TABLE users (  
    id BIGINT,  
    name VARCHAR(100)  
) PARTITION BY HASH (id);  
  
CREATE TABLE users_p0 PARTITION OF users  
FOR VALUES WITH (MODULUS 4, REMAINDER 0);  
  
CREATE TABLE users_p1 PARTITION OF users  
FOR VALUES WITH (MODULUS 4, REMAINDER 1);  
-- ... p2, p3
```

Phân bố đồng đều giữa các phân vùng.

Phân Vùng Theo Danh Sách (List Partitioning)

```
CREATE TABLE orders (  
    country VARCHAR(2)  
) PARTITION BY LIST (country);  
  
CREATE TABLE orders_us PARTITION OF orders  
FOR VALUES IN ('US');  
  
CREATE TABLE orders_eu PARTITION OF orders  
FOR VALUES IN ('DE', 'FR', 'UK');
```

Lợi Ích Của Phân Vùng

1. Hiệu năng truy vấn

- Cắt tĩa phân vùng (*Partition pruning*): chỉ quét các phân vùng liên quan
- Chỉ mục trên mỗi phân vùng nhỏ hơn

2. Bảo trì

- Xoá (DROP) các phân vùng cũ thay vì dùng DELETE (nhẹ)
- Chạy VACUUM/ANALYZE theo từng phân vùng

3. Lưu trữ lâu dài

- Chuyển các phân vùng cũ sang bộ lưu trữ chậm hơn

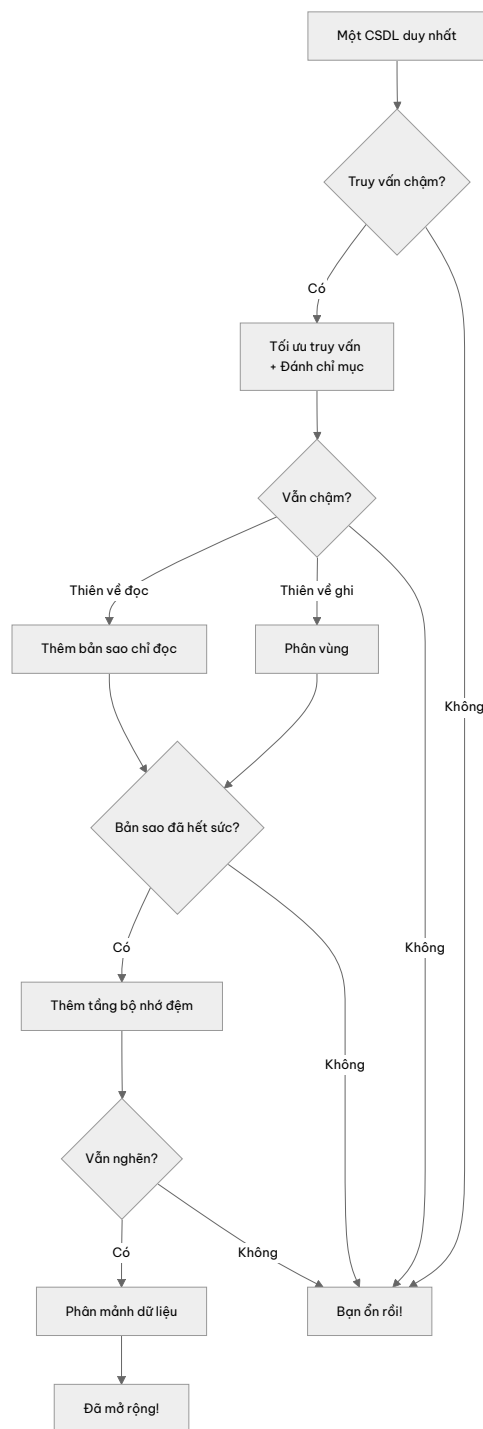
Phân Vùng So Với Phân Mảnh

Khía cạnh	Phân vùng	Phân mảnh
Phạm vi	Một cơ sở dữ liệu	Nhiều cơ sở dữ liệu
Độ phức tạp	Thấp	Cao
Quy mô (Scale)	Có giới hạn	Không giới hạn
Quản lý	Do cơ sở dữ liệu quản lý	Do ứng dụng quản lý
Giao dịch	Dễ	Khó (phân tán)

Phân vùng = phân mảnh đơn giản hơn cho một cơ sở dữ liệu đơn lẻ.

Lộ Trình Mở Rộng Cơ Sở Dữ Liệu

Đừng nhảy thẳng vào phân mảnh dữ liệu. Hãy đi theo lộ trình:



Mỗi bước đều làm tăng độ phức tạp. Chỉ tiến lên khi thật sự cần thiết.

Khuyến Nghị Thực Tế

1. Đo Lường Trước

Đừng đoán. Hãy đo.

```
-- Slow query log
SET global slow_query_log = 'ON';
SET global long_query_time = 1; -- queries > 1s

-- Monitor metrics
- Query latency (p50, p95, p99)
- Queries per second
- Connection pool usage
- Disk I/O
- CPU usage
```

2. Ưu Tiên Những Việc Dễ Làm Trước

Trước khi phân mảnh dữ liệu:

1. Thêm các chỉ mục còn thiếu
2. Tối ưu truy vấn
3. Thêm bản sao chỉ đọc
4. Thêm lưu đệm (*caching*)
5. Phân vùng các bảng

Chỉ phân mảnh dữ liệu khi đã dùng hết các phương án đơn giản hơn.

3. Thiết Kế Theo Tỷ Lệ Đọc/Ghi Của Bạn

Thiên về đọc (*Read-heavy*) (90% là đọc):

- Bản sao chỉ đọc + lưu đệm = giải quyết được phần lớn vấn đề
- Máy chủ chính chỉ xử lý 10% lượt ghi

Thiên về ghi (*Write-heavy*) (từ 50% trở lên là ghi):

- Phân vùng → phân mảnh dữ liệu
- Ghi bất đồng bộ thông qua hàng đợi (*queue*)
- Nhất quán cuối cùng

4. Gom Nhóm Kết Nối (*Connection Pooling*)

Kết nối cơ sở dữ liệu rất tốn kém.

```
# Bad: New connection mỗi query
def get_user(id):
    conn = create_connection()
    user = conn.query(...)
    conn.close()

# Good: Connection pool
pool = ConnectionPool(max_connections=20)

def get_user(id):
    with pool.get_connection() as conn:
        user = conn.query(...)
```

Tái sử dụng kết nối, giảm chi phí phụ.

5. Tinh Chỉnh Riêng Cho Từng Cơ Sở Dữ Liệu

PostgreSQL:

```
-- Increase shared buffers
shared_buffers = 4GB

-- Work mem for sorting
work_mem = 256MB

-- Effective cache size
effective_cache_size = 12GB
```

MySQL:

```
-- InnoDB buffer pool
innodb_buffer_pool_size = 8GB

-- Query cache
query_cache_size = 256MB
```

Tinh chỉnh dựa trên khối lượng công việc (*workload*) + phần cứng.

Mô Hình Tư Duy: Cơ Sở Dữ Liệu Thường Là Điểm Nghẽn Lớn Nhất

Hiểu biết cốt lõi:

Máy chủ ứng dụng (*Application servers*) **phi trạng thái** (*stateless*) → mở rộng dễ.

Cơ sở dữ liệu có trạng thái → mở rộng khó.

Thêm máy chủ API: 1 lệnh, 5 phút
Thêm một mảnh CSDL: Hàng tuần lên kế hoạch + triển khai

Thứ tự ưu tiên của kiến trúc sư (*architect*):

1. Giảm tải cho cơ sở dữ liệu:

- Lưu đệm (90% trúng bộ nhớ đệm (*cache hit*) = giảm 10 lần số truy vấn vào CSDL)
- Xử lý bất đồng bộ (ghi → hàng đợi)
- CDN (nội dung tĩnh)

2. Tối ưu truy vấn:

- Chỉ mục phù hợp
- Tránh N+1
- Chạy EXPLAIN ANALYZE cho mọi thứ

3. Mở rộng việc đọc:

- Bản sao chỉ đọc
- Phân bố theo địa lý

4. Phương án cuối cùng - mở rộng việc ghi:

- Phân vùng
- Phân mảnh dữ liệu

Những kiến trúc sư (*architects*) **giỏi luôn giảm thiểu áp lực lên cơ sở dữ liệu, không vội lao vào các kỹ thuật mở rộng phức tạp.**

Những Điểm Cốt Lõi

1. Cơ sở dữ liệu là điểm nghẽn lớn nhất trong hầu hết các hệ thống

Có trạng thái, các đảm bảo ACID, giới hạn của disk I/O.

2. Chiến lược chỉ mục = tuyên phòng thủ đầu tiên

Chỉ mục phù hợp có thể cải thiện hiệu năng 100-1000 lần.

3. Tối ưu truy vấn trước khi mở rộng

Truy vấn tẹt trên một hệ thống đã mở rộng thì vẫn tẹt. Hãy tối ưu trước.

4. Vấn đề N+1 = sát thủ thầm lặng

1000 truy vấn thay vì 2. Luôn kiểm tra câu SQL được sinh ra.

5. Phi chuẩn hoá = đánh đổi tính nhất quán lấy tốc độ

Tính toán trước các phép tổng hợp tốn kém. Khung nhìn cụ thể hoá (*Material views*) rất hữu ích.

6. Bản sao chỉ đọc mở rộng việc đọc theo chiều ngang

Thêm bản sao khi lưu lượng tăng. Xử lý độ trễ nhân bản đúng cách.

7. Phân mảnh dữ liệu = mở rộng việc ghi + lưu trữ, nhưng phức tạp

Truy vấn và giao dịch xuyên mảnh đều khó. Là phương án cuối cùng.

8. Phân vùng = lựa chọn đơn giản hơn thay cho phân mảnh dữ liệu

Do cơ sở dữ liệu quản lý, chạy trên một máy chủ, phù hợp với hầu hết các tình huống sử dụng (*use cases*).

9. Đi theo lộ trình: tối ưu → bản sao → phân vùng → phân mảnh

Đừng nhảy thẳng vào phân mảnh dữ liệu. Hãy dùng hết các giải pháp đơn giản hơn trước.

10. Hãy đo, đừng đoán

Nhật ký truy vấn chậm (*Slow query log*), số liệu giám sát (*monitoring metrics*), EXPLAIN ANALYZE.

Hãy nhớ: Chiến lược mở rộng cơ sở dữ liệu tốt nhất là bắt phải cần đến cơ sở dữ liệu. Hãy lưu đệm thật tích cực, tối ưu truy vấn, và chỉ mở rộng khi bạn đã dùng hết các phương án đơn giản hơn.

Giới Hạn Tần Suất (*Rate Limiting*) & Kiểm Soát Tải (*Load Control*) - Bảo Vệ Hệ Thống Khỏi Quá Tải (*Overload*)

Bạn xây dựng một API hoàn hảo. Mã sạch, cơ sở dữ liệu được tối ưu, hạ tầng vững chắc.

Rồi một ngày, lưu lượng (*traffic*) bất ngờ tăng đột biến (*spike*) gấp 10 lần. Nguyên nhân có thể là:

- Một bài đăng lan truyền chóng mặt trên mạng xã hội
- Tấn công bằng bot
- Lỗi ở phía máy khách (vòng lặp thử lại (*retry*) vô hạn)
- Đợt giảm giá chớp nhoáng (*flash sale*) trong thương mại điện tử

Hệ thống sụp đổ. Cơ sở dữ liệu bị quá tải. Máy chủ sập. Toàn bộ dịch vụ ngừng hoạt động.

Không phải vì kiến trúc (*architecture*) tồi. Mà vì **không có các cơ chế bảo vệ**.

Mô hình tư duy (*Mental model*) **quan trọng**: Hệ thống phải biết tự bảo vệ mình. Không thể tin rằng lưu lượng lúc nào cũng ở mức hợp lý.

Bài học này dạy bạn:

- Các thuật toán giới hạn tần suất (*token bucket*, *leaky bucket*)
- Giới hạn tần suất phân tán (*Distributed rate limiting*)
- Áp lực ngược (*Backpressure*) & cắt giảm tải (*load shedding*)
- Mẫu bộ ngắt mạch (*circuit breaker*)
- Khi nào dùng kỹ thuật nào

Tại Sao Cần Giới Hạn Tần Suất?

Vấn Đề 1: Cạn Kiệt Tài Nguyên

Mọi hệ thống đều có giới hạn.

Cơ sở dữ liệu: 1000 kết nối
Máy chủ API: 10,000 yêu cầu đồng thời
Mạng: băng thông 1 Gbps

Vượt quá giới hạn → hệ thống sập.

Không có giới hạn tần suất, một máy khách có thể chiếm dụng toàn bộ tài nguyên.

Vấn Đề 2: Kiểm Soát Chi Phí (Cost)

Các dịch vụ đám mây tính phí theo mức sử dụng.

API Gateway: \$3.50 cho mỗi triệu yêu cầu
Lambda: \$0.20 cho mỗi triệu lượt gọi
Cơ sở dữ liệu: \$0.10 cho mỗi GB truyền tải

Lưu lượng không được kiểm soát = chi phí không giới hạn.

Một đợt tấn công bằng bot có thể tạo ra hàng triệu yêu cầu (*requests*) → hoá đơn \$10.000 chỉ trong 1 ngày.

Vấn Đề 3: Lạm Dụng & Bảo Mật

Kẻ tấn công khai thác hệ thống:

- Dò mật khẩu đăng nhập bằng vét cạn (*brute force*) (thử hàng nghìn mật khẩu)
- Cào dữ liệu (thu thập toàn bộ cơ sở dữ liệu)
- DDoS (làm quá tải bằng lưu lượng rác)

Giới hạn tần suất = tuyến phòng thủ đầu tiên.

Vấn Đề 4: Sử Dụng Công Bằng

Một người dùng không nên độc chiếm tài nguyên.

Trong hệ thống đa khách thuê (*multi-tenant*), chỉ 1 khách hàng gửi yêu cầu ồ ạt → ảnh hưởng tới tất cả khách hàng.

Giới hạn tần suất đảm bảo sự công bằng.

Tình Huống Sử Dụng (*Use Cases*)

Hạn mức API:

- Gói miễn phí: 1000 yêu cầu/ngày
- Gói Pro: 100.000 yêu cầu/ngày

Số lần đăng nhập:

- 5 lần thất bại → khoá tài khoản 15 phút

Truy vấn tìm kiếm:

- 10 lượt tìm kiếm/phút cho mỗi người dùng

Tải tệp lên:

- 100 MB/giờ cho mỗi người dùng

Các Thuật Toán Giới Hạn Tần Suất

Có nhiều thuật toán, mỗi thuật toán phù hợp với một tình huống sử dụng (*use case*) khác nhau.

1. Fixed Window Counter (Bộ Đếm Cửa Sổ Cố Định)

Thuật toán đơn giản nhất.


```

import time

class FixedWindowRateLimiter:
    def __init__(self, limit, window_seconds):
        self.limit = limit
        self.window = window_seconds
        self.counter = 0
        self.window_start = time.time()

    def allow_request(self):
        now = time.time()

        # New window
        if now - self.window_start >= self.window:
            self.counter = 0
            self.window_start = now

        # Check limit
        if self.counter < self.limit:
            self.counter += 1
            return True

        return False

```

Ví dụ:

Cửa sổ: 1 phút
 Giới hạn: 100 yêu cầu

00:00-00:59: 100 yêu cầu 
 01:00-01:59: Đặt lại bộ đếm, 100 yêu cầu 

Vấn đề: Hiện tượng ở ranh giới cửa sổ

00:59: 100 yêu cầu (chạm mức tối đa)
 01:00: Đặt lại
 01:01: 100 yêu cầu

Tổng cộng 200 yêu cầu trong 2 giây! 

Lưu lượng bùng nổ (*Burst traffic*) tại ranh giới cửa sổ vượt qua được giới hạn.

2. Sliding Window Log (Nhật Ký Cửa Sổ Trượt)

Theo dõi dấu thời gian (*timestamp*) của mỗi yêu cầu.

```

from collections import deque
import time

class SlidingWindowLog:
    def __init__(self, limit, window_seconds):
        self.limit = limit
        self.window = window_seconds
        self.log = deque()

    def allow_request(self):
        now = time.time()

        # Remove old requests outside window
        while self.log and self.log[0] <= now - self.window:
            self.log.popleft()

        # Check limit
        if len(self.log) < self.limit:
            self.log.append(now)
            return True

        return False

```

Ví dụ:

Cửa sổ: 60 giây
 Giới hạn: 100 yêu cầu

Yêu cầu lúc 10:00:30
 → Đếm số yêu cầu từ 09:59:30 đến 10:00:30
 → Nếu < 100, cho phép

Ưu điểm:

- Chính xác (không có vấn đề ở ranh giới cửa sổ)
- Giới hạn tần suất mượn mà

Nhược điểm:

- Tốn bộ nhớ (lưu tất cả dấu thời gian (*timestamps*))
- Thời gian tra cứu $O(N)$

3. Sliding Window Counter (Bộ Đếm Cửa Sổ Trượt – Dạng Lai (*Hybrid*))

Kết hợp cửa sổ cố định + phép tính trượt.

```

import time

class SlidingWindowCounter:
    def __init__(self, limit, window_seconds):
        self.limit = limit
        self.window = window_seconds
        self.current_window = {"start": time.time(), "count": 0}
        self.prev_window_count = 0

    def allow_request(self):
        now = time.time()

        # Calculate sliding count
        elapsed = now - self.current_window["start"]

        if elapsed >= self.window:
            # New window
            self.prev_window_count = self.current_window["count"]
            self.current_window = {"start": now, "count": 0}
            elapsed = 0

        # Weighted count from previous window
        prev_weight = (self.window - elapsed) / self.window
        estimated_count = (
            self.prev_window_count * prev_weight +
            self.current_window["count"]
        )

        if estimated_count < self.limit:
            self.current_window["count"] += 1
            return True

        return False

```

Làm mượt các đợt tăng đột biến (spikes) của cửa sổ cố định với bộ nhớ $O(1)$.

4. Token Bucket (Thùng Token – PHỔ BIẾN NHẤT)

Thùng chứa các token. Mỗi yêu cầu tiêu thụ 1 token.

```

import time

class TokenBucket:
    def __init__(self, capacity, refill_rate):
        self.capacity = capacity          # Max tokens
        self.tokens = capacity            # Current tokens
        self.refill_rate = refill_rate    # Tokens per second
        self.last_refill = time.time()

    def allow_request(self, tokens_needed=1):
        now = time.time()

        # Refill tokens
        elapsed = now - self.last_refill
        tokens_to_add = elapsed * self.refill_rate
        self.tokens = min(self.capacity, self.tokens + tokens_to_add)
        self.last_refill = now

        # Check if enough tokens
        if self.tokens >= tokens_needed:
            self.tokens -= tokens_needed
            return True

        return False

```

Ví dụ:

Sức chứa: 100 token
 Tốc độ nạp lại: 10 token/giây

Bắt đầu: 100 token
 Yêu cầu 1: còn 99 token (cho phép)
 Yêu cầu 2: còn 98 token (cho phép)
 ... (100 yêu cầu)
 Yêu cầu 101: còn 0 token (từ chối) ❌

Chờ 1 giây: +10 token
 Yêu cầu 102: còn 9 token (cho phép) ✅

Ưu điểm:

- Cho phép bùng nổ (*bursts*) (tận dụng số token đã tích lũy)
- Tốc độ dài hạn ổn định
- Triển khai đơn giản

Nhược điểm:

- Các đợt bùng nổ có thể làm quá tải các dịch vụ hạ nguồn (*downstream*)

Được dùng bởi: AWS API Gateway, Kong, Nginx.

5. Leaky Bucket (Thùng Rò Rỉ)

Thùng rò rỉ token với tốc độ không đổi.

```
import time
from collections import deque

class LeakyBucket:
    def __init__(self, capacity, leak_rate):
        self.capacity = capacity      # Max queue size
        self.leak_rate = leak_rate    # Requests processed per second
        self.queue = deque()
        self.last_leak = time.time()

    def allow_request(self):
        now = time.time()

        # Leak (process) requests
        elapsed = now - self.last_leak
        leaks = int(elapsed * self.leak_rate)

        for _ in range(min(leaks, len(self.queue))):
            self.queue.popleft()

        self.last_leak = now

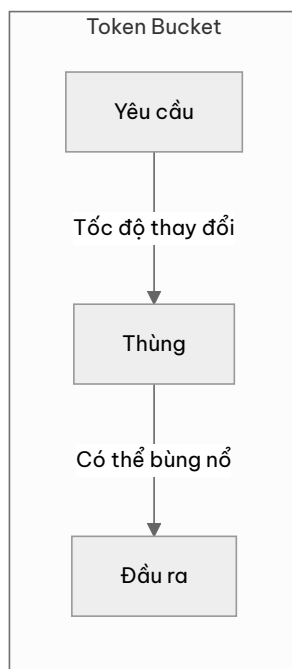
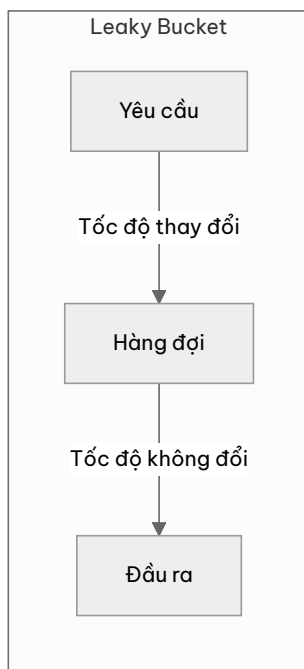
        # Add request to queue
        if len(self.queue) < self.capacity:
            self.queue.append(now)
            return True

        return False
```

Khác biệt so với Token Bucket:

Token Bucket: Đầu ra theo đợt bùng nổ (*bursty*) (có thể gửi nhiều yêu cầu cùng lúc nếu còn token)

Leaky Bucket: Đầu ra đều đặn (tốc độ không đổi, không có bùng nổ)



Token Bucket = giới hạn tần suất đầu vào.

Leaky Bucket = giới hạn tần suất đầu ra.

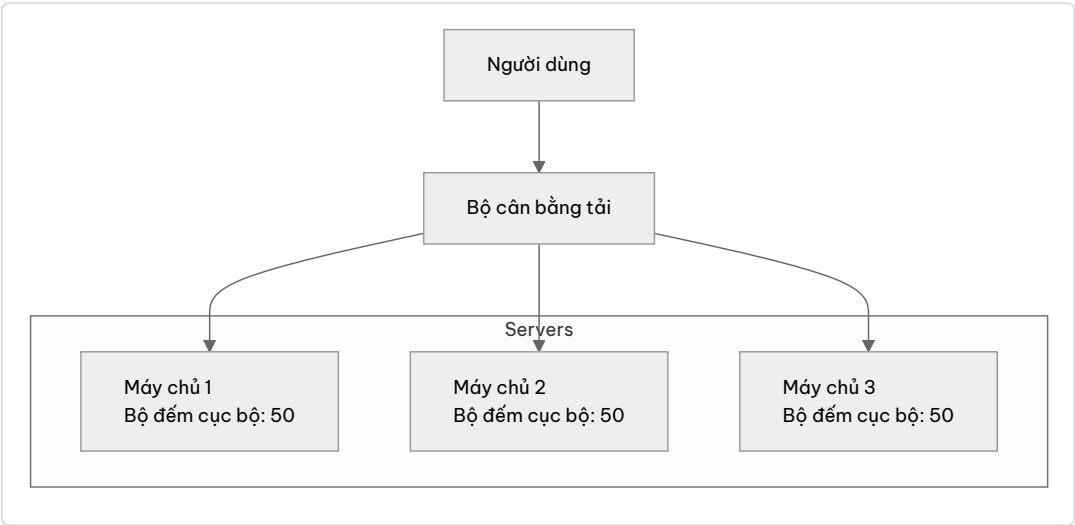
So Sánh Các Thuật Toán

Thuật toán	Bộ nhớ	Độ chính xác	Bùng nổ	Tình huống sử dụng
Cửa sổ cố định	O(1)	Thấp	Có	Hạn mức đơn giản
Nhật ký trượt	O(N)	Cao	Không	Giới hạn chính xác
Bộ đếm trượt	O(1)	Trung bình	Không	Cân bằng tốt
Token Bucket	O(1)	Tốt	Có	Phần lớn API
Leaky Bucket	O(N)	Tốt	Không	Định hình lưu lượng (<i>Traffic shaping</i>)

Token Bucket = phổ biến nhất vì cân bằng giữa sự đơn giản + hiệu quả.

Giới Hạn Tần Suất Phân Tán

Vấn đề: Có nhiều máy chủ API, làm sao đồng bộ các giới hạn tần suất?



Giới hạn: 100 req/phút cho mỗi người dùng.

Nếu mỗi máy chủ tự theo dõi riêng → người dùng có thể gửi 50 yêu cầu tới mỗi máy chủ = tổng cộng 150! ⚠️

Giải Pháp 1: Trạng Thái Tập Trung (Redis)

Lưu các bộ đếm trong Redis dùng chung.

```
import redis
import time

class DistributedRateLimiter:
    def __init__(self, redis_client, limit, window):
        self.redis = redis_client
        self.limit = limit
        self.window = window

    def allow_request(self, user_id):
        key = f"rate_limit:{user_id}"
        current = self.redis.get(key)

        if current is None:
            # First request
            self.redis.setex(key, self.window, 1)
            return True

        if int(current) < self.limit:
            self.redis.incr(key)
            return True

        return False
```

Token bucket trong Redis:


```

def allow_request_token_bucket(self, user_id):
    key = f"token_bucket:{user_id}"
    now = time.time()

    # Lua script for atomic operation
    lua_script = """
    local key = KEYS[1]
    local capacity = tonumber(ARGV[1])
    local refill_rate = tonumber(ARGV[2])
    local now = tonumber(ARGV[3])

    local bucket = redis.call('HMGET', key, 'tokens', 'last_refill')
    local tokens = tonumber(bucket[1]) or capacity
    local last_refill = tonumber(bucket[2]) or now

    -- Refill tokens
    local elapsed = now - last_refill
    tokens = math.min(capacity, tokens + elapsed * refill_rate)

    if tokens >= 1 then
        tokens = tokens - 1
        redis.call('HMSET', key, 'tokens', tokens, 'last_refill', now)
        redis.call('EXPIRE', key, 3600)
        return 1
    else
        return 0
    end
    """

    return self.redis.eval(lua_script, 1, key,
                           self.capacity, self.refill_rate, now)

```

Ưu điểm:

- Chính xác trên toàn bộ các máy chủ
- Logic tập trung

Nhược điểm:

- Redis là điểm lỗi duy nhất (*single point of failure*)
- Độ trễ (*latency*) mạng cho mỗi yêu cầu
- Redis có thể trở thành điểm nghẽn (*bottleneck*)

Giải Pháp 2: Nhất Quán Cuối Cùng (*Eventually Consistent*) (Gossip)

Mỗi máy chủ duy trì trạng thái cục bộ, đồng bộ định kỳ.

```

class EventuallyConsistentRateLimiter:
    def __init__(self, limit, num_servers):
        self.limit = limit
        self.local_limit = limit // num_servers # Divide among servers
        self.local_counter = 0
        self.global_counters = {} # From other servers

    def allow_request(self, user_id):
        # Check local limit first
        if self.local_counter < self.local_limit:
            self.local_counter += 1
            return True

        # Check global limit (eventual)
        total = sum(self.global_counters.values()) + self.local_counter
        return total < self.limit

    def sync_with_peers(self):
        # Gossip protocol: exchange counters with other servers
        pass

```

Ưu điểm:

- Không có điểm lỗi duy nhất
- Độ trễ thấp (kiểm tra cục bộ)

Nhược điểm:

- Không chính xác (nhất quán cuối cùng (*eventual consistency*))
- Triển khai phức tạp

Cách Tiếp Cận Lại (Thực Tế)

Bộ nhớ đệm (*cache*) **cục bộ** + **Redis**.

```

class HybridRateLimiter:
    def __init__(self, redis_client, limit, window):
        self.redis = redis_client
        self.limit = limit
        self.window = window
        self.local_cache = {}
        self.cache_ttl = 1 # 1 second cache

    def allow_request(self, user_id):
        # Check local cache first
        cache_entry = self.local_cache.get(user_id)
        now = time.time()

        if cache_entry and now - cache_entry['time'] < self.cache_ttl:
            if cache_entry['count'] < self.limit:
                cache_entry['count'] += 1
                return True
            return False

        # Cache miss or expired - check Redis
        key = f"rate_limit:{user_id}"
        current = self.redis.get(key)

        if current is None or int(current) < self.limit:
            count = self.redis.incr(key)
            self.redis.expire(key, self.window)

            # Update local cache
            self.local_cache[user_id] = {
                'count': count,
                'time': now
            }
            return True

        return False

```

Cân bằng giữa độ chính xác + hiệu năng (*performance*).

Các Tiêu Đề (*Headers*) Phản Hồi Về Giới Hạn Tần Suất

Máy khách cần biết trạng thái giới hạn tần suất.

Các tiêu đề chuẩn:

```

HTTP/1.1 200 OK
X-RateLimit-Limit: 1000          # Total limit
X-RateLimit-Remaining: 950      # Requests remaining
X-RateLimit-Reset: 1643990400   # Unix timestamp when resets

HTTP/1.1 429 Too Many Requests
Retry-After: 60                  # Seconds until retry
X-RateLimit-Limit: 1000
X-RateLimit-Remaining: 0
X-RateLimit-Reset: 1643990400

```

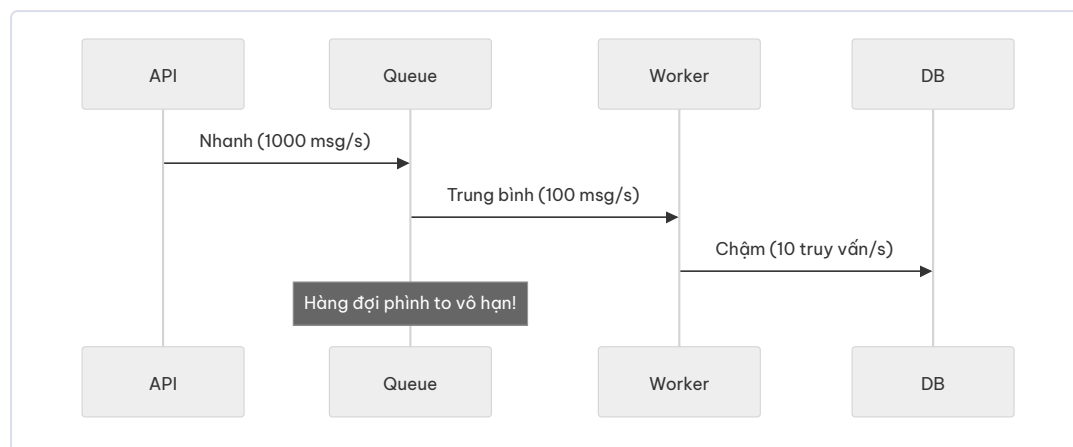
Máy khách có thể:

- Hiện thị hạn mức cho người dùng
- Triển khai lùi thời gian theo cấp số nhân (*exponential backoff*)
- Lên lịch gửi yêu cầu một cách hiệu quả

Áp Lực Ngược - Đẩy Ngược Lại Thương Nguồn (*Upstream*)

Giới hạn tần suất bảo vệ hệ thống khỏi lưu lượng bên ngoài. Áp lực ngược bảo vệ khỏi tình trạng quá tải bên trong.

Vấn Đề



Bên phát (Producer) nhanh hơn bên tiêu thụ (consumer) → hàng đợi (queue) bùng nổ.

Giải Pháp: Áp Lực Ngược

Làm chậm bên phát khi bên tiêu thụ bị quá tải.

```
class BackpressureQueue:
    def __init__(self, max_size):
        self.queue = []
        self.max_size = max_size

    def push(self, item):
        if len(self.queue) >= self.max_size:
            # Backpressure: Block or reject
            raise QueueFullError("Queue full, slow down!")

        self.queue.append(item)

    def pop(self):
        if self.queue:
            return self.queue.pop(0)
        return None
```

API kiểm tra kích thước hàng đợi:

```
@app.route('/api/task', methods=['POST'])
def create_task():
    try:
        queue.push(request.data)
        return {"status": "queued"}, 202
    except QueueFullError:
        return {"error": "System overloaded, try later"}, 503
```

503 Service Unavailable = tín hiệu áp lực ngược.

Áp Lực Ngược Trong Reactive Streams

Cách tiếp cận hiện đại: kéo dữ liệu theo yêu cầu.

```

class ReactiveStream:
    def __init__(self, producer, consumer):
        self.producer = producer
        self.consumer = consumer
        self.buffer = []
        self.requested = 0

    def request(self, n):
        # Consumer requests n items
        self.requested += n
        self.try_send()

    def try_send(self):
        while self.requested > 0 and self.buffer:
            item = self.buffer.pop(0)
            self.consumer.on_next(item)
            self.requested -= 1

    def on_data(self, item):
        # Producer sends data
        if self.requested > 0:
            self.consumer.on_next(item)
            self.requested -= 1
        else:
            # Buffer or drop
            if len(self.buffer) < MAX_BUFFER:
                self.buffer.append(item)
            else:
                self.consumer.on_error("Backpressure limit")

```

Bên tiêu thụ tự kéo dữ liệu theo nhịp độ của mình, thay vì bị đẩy tới.

Cắt Giảm Tải - Loại Bỏ Bớt Lưu Lượng Để Sống Sót

Khi hệ thống quá tải, từ chối bớt yêu cầu để bảo vệ chức năng cốt lõi.

Cắt Giảm Tải (*Shedding*) Theo Mức Ưu Tiên

Không phải mọi yêu cầu đều quan trọng như nhau.

```

class LoadShedder:
    def __init__(self, max_load):
        self.max_load = max_load
        self.current_load = 0

    def allow_request(self, priority):
        if self.current_load < self.max_load:
            return True

        # Overload: only allow high priority
        if priority == "critical":
            return True
        elif priority == "high" and self.current_load < self.max_load * 1.1:
            return True
        else:
            return False

```

Ví dụ về mức ưu tiên:

```

# Critical: payment, checkout
if allow_request("critical"):
    process_payment()

# High: user profile update
if allow_request("high"):
    update_profile()

# Low: analytics, logging
if allow_request("low"):
    track_analytics()

```

Loại bỏ lưu lượng có mức ưu tiên thấp trước.

Cắt Giảm Tải Thích Ứng

Cắt giảm dựa trên các chỉ số (*metrics*) sức khỏe của hệ thống.

```

class AdaptiveLoadShedder:
    def __init__(self):
        self.error_rate = 0.0
        self.latency_p99 = 0.0

    def should_shed_load(self):
        # Shed if error rate high
        if self.error_rate > 0.05: # 5% errors
            return True

        # Shed if latency high
        if self.latency_p99 > 1000: # 1 second
            return True

        # Shed if CPU high
        cpu_usage = psutil.cpu_percent()
        if cpu_usage > 90:
            return True

        return False

    def calculate_shed_probability(self):
        # Progressive shedding
        if self.error_rate < 0.01:
            return 0.0 # No shedding
        elif self.error_rate < 0.05:
            return 0.5 # Shed 50%
        else:
            return 0.9 # Shed 90%

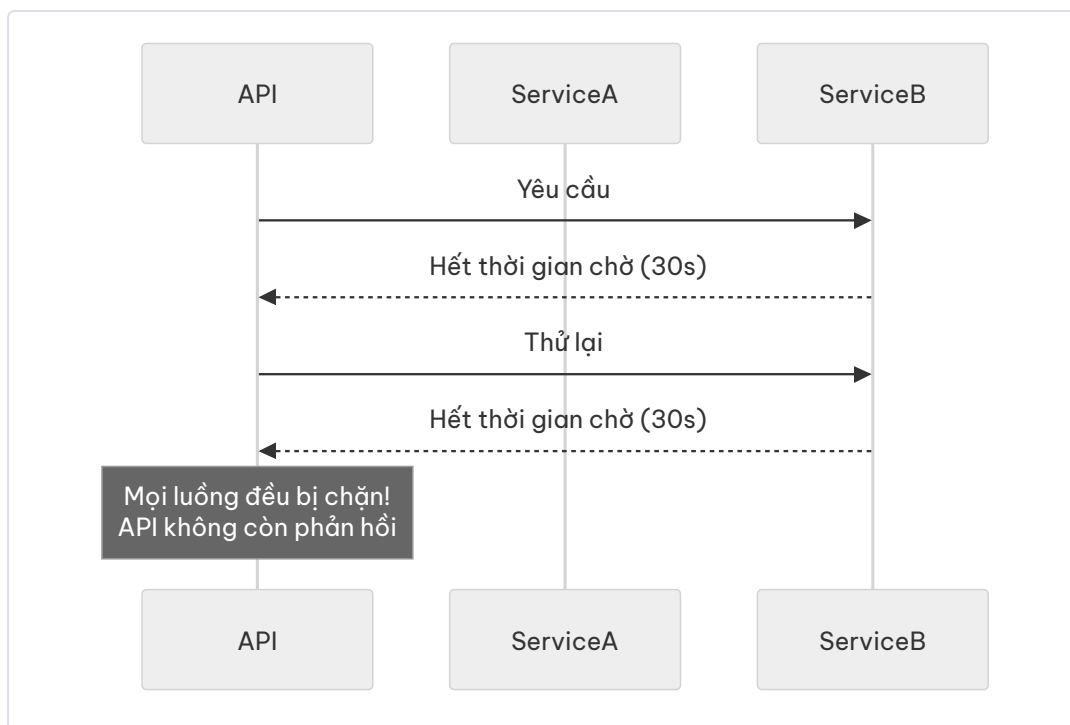
```

Suy giảm dần dần thay vì cắt đứt đột ngột.

Bộ Ngắt Mạch - Ngừng Gọi Các Dịch Vụ Đang Lỗi

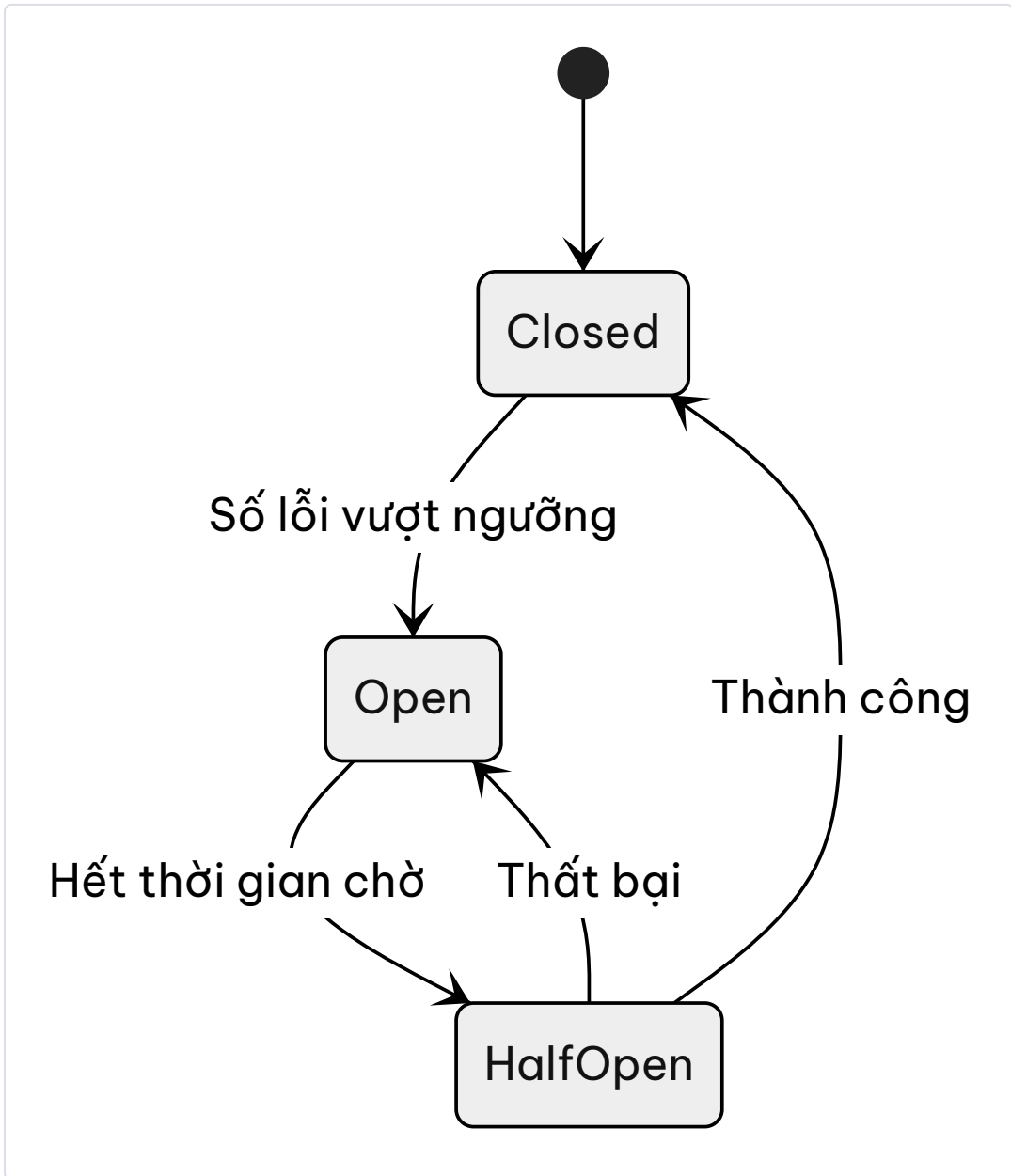
Mẫu (Pattern) để ngăn chặn lỗi dây chuyền (*cascading failures*).

Vấn Đề



Dịch vụ B sập → API cứ tiếp tục thử → API cũng sập theo.

Các Trạng Thái Của Bộ Ngắt Mạch



Các trạng thái:

Closed – Đóng (Bình thường):

- Các yêu cầu đi qua bình thường
- Theo dõi số lỗi

Open – Mở (Đã lỗi):

- Từ chối yêu cầu ngay lập tức (thất bại nhanh (*fail fast*))
- Không gọi dịch vụ đang lỗi
- Trả về dữ liệu từ bộ nhớ đệm hoặc lỗi

Half-Open – Nửa mở (Đang thử nghiệm):

- Cho phép một số lượng yêu cầu giới hạn
- Kiểm tra xem dịch vụ đã phục hồi chưa
- Đóng nếu thành công, mở nếu thất bại

Triển Khai

```
import time
from enum import Enum

class CircuitState(Enum):
    CLOSED = "closed"
    OPEN = "open"
    HALF_OPEN = "half_open"

class CircuitBreaker:
    def __init__(self, failure_threshold=5, timeout=60, half_open_max=3):
        self.failure_threshold = failure_threshold
        self.timeout = timeout # Seconds before trying again
        self.half_open_max = half_open_max

        self.state = CircuitState.CLOSED
        self.failure_count = 0
        self.last_failure_time = None
        self.half_open_attempts = 0

    def call(self, func, *args, **kwargs):
        if self.state == CircuitState.OPEN:
            # Check if timeout expired
            if time.time() - self.last_failure_time >= self.timeout:
                self.state = CircuitState.HALF_OPEN
                self.half_open_attempts = 0
            else:
                raise CircuitBreakerOpenError("Circuit breaker is open")

        try:
            result = func(*args, **kwargs)
            self.on_success()
            return result
        except Exception as e:
            self.on_failure()
            raise e

    def on_success(self):
        if self.state == CircuitState.HALF_OPEN:
            self.state = CircuitState.CLOSED

        self.failure_count = 0

    def on_failure(self):
        self.failure_count += 1
        self.last_failure_time = time.time()

        if self.state == CircuitState.HALF_OPEN:
            self.state = CircuitState.OPEN
        elif self.failure_count >= self.failure_threshold:
            self.state = CircuitState.OPEN
```

Cách dùng:

```
breaker = CircuitBreaker(failure_threshold=5, timeout=60)

def call_external_service():
    return breaker.call(external_api.get_data)

try:
    data = call_external_service()
except CircuitBreakerOpenError:
    # Service is down, use fallback
    data = get_cached_data()
```

Lợi Ích

1. Thất bại nhanh

- Không lãng phí thời gian thử lại với dịch vụ đang lỗi
- Giải phóng luồng/tài nguyên

2. Cho dịch vụ thời gian phục hồi

- Ngừng dồn dập gọi vào dịch vụ đã lỗi
- Cho phép phục hồi một cách êm ái

3. Ngăn chặn lỗi dây chuyền

- Một dịch vụ sập ≠ toàn bộ hệ thống sập

4. Cung cấp phương án dự phòng (fallbacks)

```
try:
    data = call_external_service()
except CircuitBreakerOpenError:
    # Fallback strategies
    data = get_cached_data()           # Use cache
    data = get_default_data()         # Use defaults
    data = call_backup_service()       # Alternate service
```

Mẫu Vách Ngăn (Bulkhead Pattern) - Cô Lập Sự Cố

Đừng để một thành phần (component) chiếm hết toàn bộ tài nguyên.

Vách Ngăn (Bulkheads) Bằng Nhóm Luồng (Thread Pool)

```
from concurrent.futures import ThreadPoolExecutor

class BulkheadPattern:
    def __init__(self):
        # Separate thread pools for different services
        self.service_a_pool = ThreadPoolExecutor(max_workers=10)
        self.service_b_pool = ThreadPoolExecutor(max_workers=10)
        self.service_c_pool = ThreadPoolExecutor(max_workers=10)

    def call_service_a(self, request):
        return self.service_a_pool.submit(service_a.process, request)

    def call_service_b(self, request):
        return self.service_b_pool.submit(service_b.process, request)
```

Dịch vụ B chậm/treo → chỉ chiếm dụng 10 luồng trong nhóm luồng của B.

Dịch vụ A và C vẫn hoạt động bình thường.

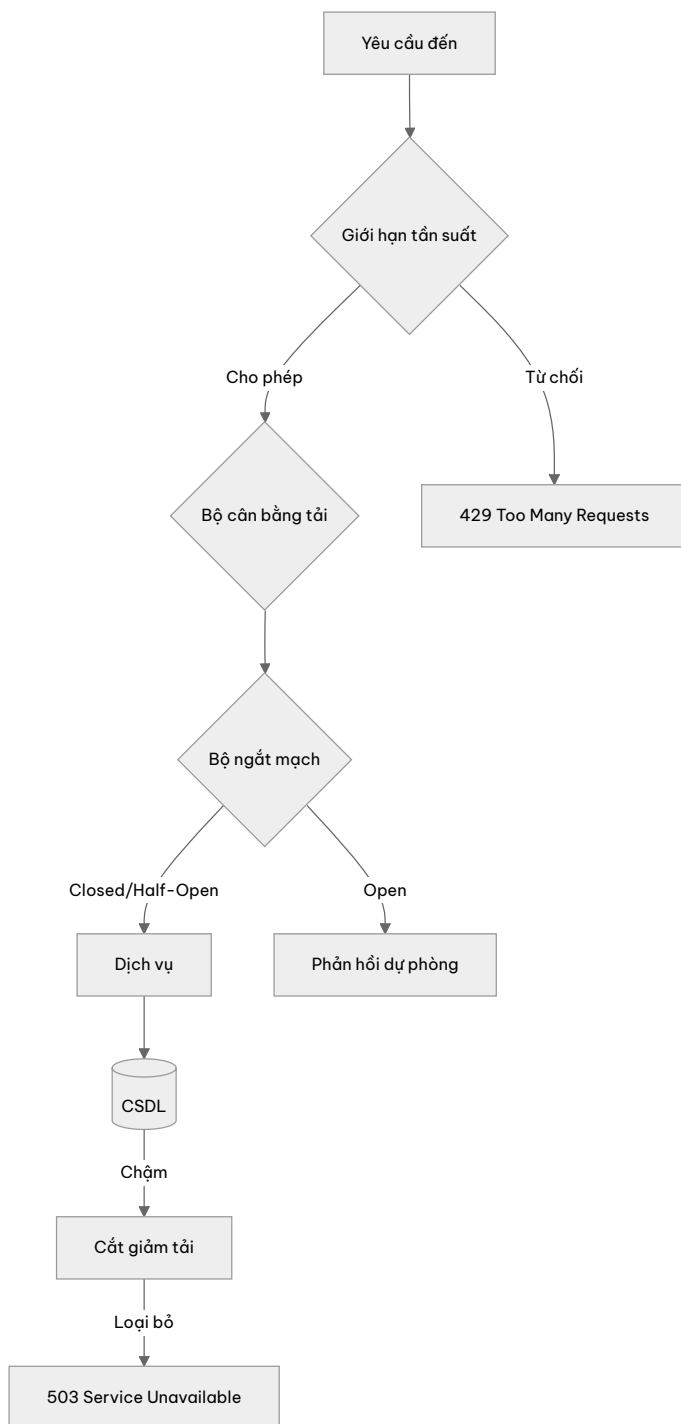
Vách Ngăn Bằng Nhóm Kết Nối (Connection Pool)

```
# Separate database connection pools
db_pool_users = create_pool(max_connections=20)
db_pool_orders = create_pool(max_connections=20)
db_pool_analytics = create_pool(max_connections=10)

# Analytics queries won't starve user queries
```

Kết Hợp Tất Cả Lại

Chiến lược bảo vệ toàn diện:



Các lớp bảo vệ:

1. **Giới hạn tần suất:** Ngăn chặn lạm dụng
2. **Bộ ngắt mạch:** Thất bại nhanh khi các dịch vụ hạ nguồn gặp sự cố
3. **Áp lực ngược:** Làm chậm lại khi hàng đợi (*queues*) đầy
4. **Cắt giảm tải:** Loại bỏ bớt lưu lượng khi quá tải
5. **Vách ngăn** (*Bulkhead*): Cô lập sự cố

Phòng thủ nhiều lớp (*Defense in depth*).

Khuyến Nghị Thực Tế

1. Bắt Đầu Với Giới Hạn Tần Suất

Mọi API công khai đều cần giới hạn tần suất.

```
# Simple Flask example
from flask_limiter import Limiter

limiter = Limiter(
    app,
    key_func=lambda: request.headers.get('X-API-Key'),
    default_limits=["1000 per day", "100 per hour"]
)

@app.route('/api/data')
@limiter.limit("10 per minute")
def get_data():
    return jsonify(data)
```

Ngay cả các API nội bộ cũng được hưởng lợi từ giới hạn tần suất.

2. Triển Khai Bộ Ngắt Mạch (Circuit Breakers) Cho Các Phụ Thuộc (Dependencies) Bên Ngoài

```
# Wrap external API calls
payment_breaker = CircuitBreaker()
email_breaker = CircuitBreaker()

@payment_breaker.call
def charge_payment(amount):
    return stripe_api.charge(amount)

@email_breaker.call
def send_email(to, body):
    return sendgrid_api.send(to, body)
```

3. Giám Sát (Monitor) & Cảnh Báo (Alert)

Theo dõi các chỉ số:

- Số lần chạm giới hạn tần suất (theo từng endpoint, từng người dùng)
- Các lần chuyển trạng thái của bộ ngắt mạch
- Độ sâu hàng đợi
- Độ trễ yêu cầu (p50, p95, p99)
- Tỷ lệ lỗi

Cảnh báo khi:

- Thường xuyên chạm giới hạn tần suất (có thể cần nâng giới hạn)
- Bộ ngắt mạch mở (có vấn đề ở phụ thuộc (*dependency*))
- Hàng đợi đang phình to (cần áp lực ngược)

4. Thiết Kế Để Suy Giảm Có Kiểm Soát

Suy giảm có kiểm soát (*Graceful degradation*) > **sập hoàn toàn**.

```
def get_user_recommendations():
    try:
        # Try ML service
        return ml_service.get_recommendations()
    except CircuitBreakerOpenError:
        # Fallback to simple algorithm
        return simple_recommendation()
    except Exception:
        # Last resort: popular items
        return get_popular_items()
```

5. Kiểm Thử Dưới Tải

Kiểm thử tải (*Load testing*) **nhận kích hoạt được các cơ chế bảo vệ.**

```
# k6 load test
k6 run --vus 1000 --duration 30s load_test.js
```

Xác minh:

- Giới hạn tần suất được kích hoạt
- Bộ ngắt mạch mở khi có lỗi
- Hệ thống vẫn ổn định khi quá tải

Mô Hình Tư Duy: Hệ Thống Phải Tự Bảo Vệ Mình

Nhận thức cốt lõi:

Hệ thống tốt không giả định mọi điều kiện đều hoàn hảo. Chúng lường trước và xử lý được lạm dụng, sự cố và quá tải.

Hệ thống tồi:

Giả định: Lưu lượng hợp lý, các phụ thuộc đáng tin cậy
Thực tế: Lưu lượng tăng đột biến → sập

Hệ thống tốt:

Giả định: Chuyện gì cũng có thể xảy ra
Thiết kế: Cơ chế bảo vệ ở mọi tầng
Thực tế: Lưu lượng tăng đột biến → suy giảm có kiểm soát

Tư duy (*Mindset*) **của kiến trúc sư** (*architect*):

1. **Lưu lượng bên ngoài:** Giới hạn tần suất
2. **Sự cố ở các phụ thuộc:** Bộ ngắt mạch
3. **Quá tải bên trong:** Áp lực ngược, cắt giảm tải
4. **Cô lập tài nguyên:** Mẫu vách ngăn

Hệ thống tự bảo vệ = độ tin cậy (*reliability*).

Những Điểm Cốt Lõi

1. Giới hạn tần suất ngăn chặn lạm dụng và cạn kiệt tài nguyên

Thuật toán token bucket phổ biến nhất. Giới hạn tần suất phân tán cần Redis hoặc gossip.

2. Bộ ngắt mạch ngăn chặn lỗi dây chuyền

Thất bại nhanh, cho dịch vụ thời gian phục hồi, cung cấp phương án dự phòng.

3. Áp lực ngược đẩy ngược lại các bên phát (*producers*) ở thượng nguồn

Ngăn hàng đợi bùng nổ, duy trì sự ổn định của hệ thống.

4. Cắt giảm tải loại bỏ bớt lưu lượng để sống sót

Theo mức ưu tiên hoặc thích ứng. Thà suy giảm có kiểm soát còn hơn sập hoàn toàn.

5. Mẫu vách ngăn cô lập sự cố

Tách riêng nhóm luồng (*thread pools*)/nhóm kết nối (*connection pools*) cho từng phụ thuộc.

6. Phòng thủ nhiều lớp

Nhiều lớp bảo vệ. Giới hạn tần suất + bộ ngắt mạch + cắt giảm tải.

7. Giám sát và cảnh báo khi cơ chế bảo vệ được kích hoạt

Cơ chế bảo vệ được kích hoạt = tín hiệu cảnh báo sớm.

8. Thiết kế để suy giảm có kiểm soát

Các chiến lược dự phòng (*fallback*) tốt hơn là sập hoàn toàn.

9. Kiểm thử khi quá tải

Kiểm thử tải nên vận hành thử các cơ chế bảo vệ.

Hãy nhớ: Hệ thống tốt nhất là hệ thống sống sót được trước lạm dụng, xử lý sự cố một cách êm ái và tự động bảo vệ chính mình. Đừng đợi đến khi xảy ra sự cố trong môi trường thực tế (*production*) mới bổ sung các cơ chế bảo vệ này - hãy xây dựng chúng ngay từ đầu.

Xử Lý Bất Đồng Bộ (Asynchronous Processing) & Hàng Đợi (Queues) - Trì Hoãn Công Việc Để Giảm Độ Trễ (Latency)

Người dùng bấm "Đặt hàng" trên một trang thương mại điện tử.

Cách tiếp cận đồng bộ (synchronous):

- | | |
|-------------------------------|---------|
| 1. Xác thực đơn hàng | (100ms) |
| 2. Kiểm tra tồn kho | (200ms) |
| 3. Xử lý thanh toán | (500ms) |
| 4. Cập nhật cơ sở dữ liệu | (100ms) |
| 5. Gửi email xác nhận | (300ms) |
| 6. Tạo hoá đơn PDF | (400ms) |
| 7. Thông báo cho kho hàng | (200ms) |
| 8. Cập nhật số liệu phân tích | (100ms) |

Tổng: 1900ms (gần 2 giây!)

Người dùng phải đợi 2 giây mới thấy "Đặt hàng thành công". **Trải nghiệm tệ.**

Cách tiếp cận bất đồng bộ (asynchronous):

- | | |
|--|---------|
| 1. Xác thực đơn hàng | (100ms) |
| 2. Kiểm tra tồn kho | (200ms) |
| 3. Xử lý thanh toán | (500ms) |
| 4. Cập nhật cơ sở dữ liệu | (100ms) |
| 5. Đưa các tác vụ còn lại vào hàng đợi | (10ms) |

Trả về "Đặt hàng thành công": 910ms

Chạy nền (bất đồng bộ):

- Gửi email
- Tạo PDF
- Thông báo cho kho hàng
- Cập nhật số liệu phân tích

Người dùng chỉ đợi khoảng 1 giây. **Trải nghiệm tốt hơn gấp đôi.**

Đây chính là sức mạnh của xử lý bất đồng bộ (async processing).

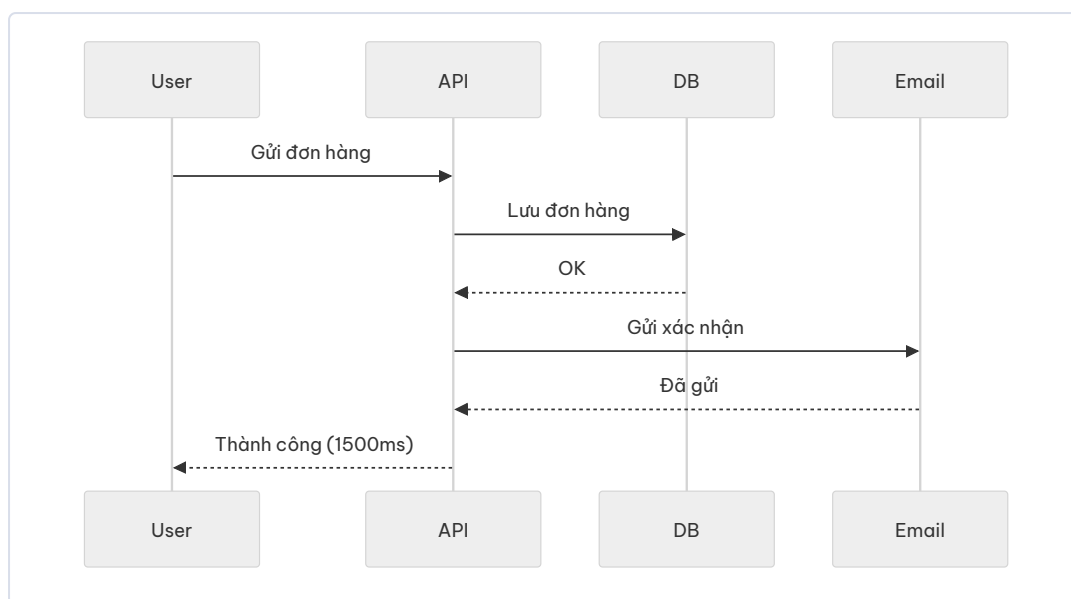
Bài này sẽ dạy bạn:

- Khi nào dùng đồng bộ (*sync*), khi nào dùng bất đồng bộ (*async*)
- Kiến trúc (*Architecture*) hàng đợi thông điệp (*message queues*)
- Các chiến lược thử lại (*retry*) và xử lý lỗi
- Hàng đợi thư chết (*Dead letter queues*)
- Tính lũy đẳng (*Idempotency*) (yếu tố then chốt cho độ tin cậy (*reliability*))
- Tư duy hướng sự kiện (*event-driven*)

Đồng Bộ Và Bất Đồng Bộ - Sự Đánh Đổi Cơ Bản

Xử Lý Đồng Bộ (*Synchronous Processing*)

Yêu cầu (*Request*) → **Xử lý** → **Phản hồi** (*Response*). Người dùng phải đợi toàn bộ quá trình.



Đặc điểm:

Ưu điểm:

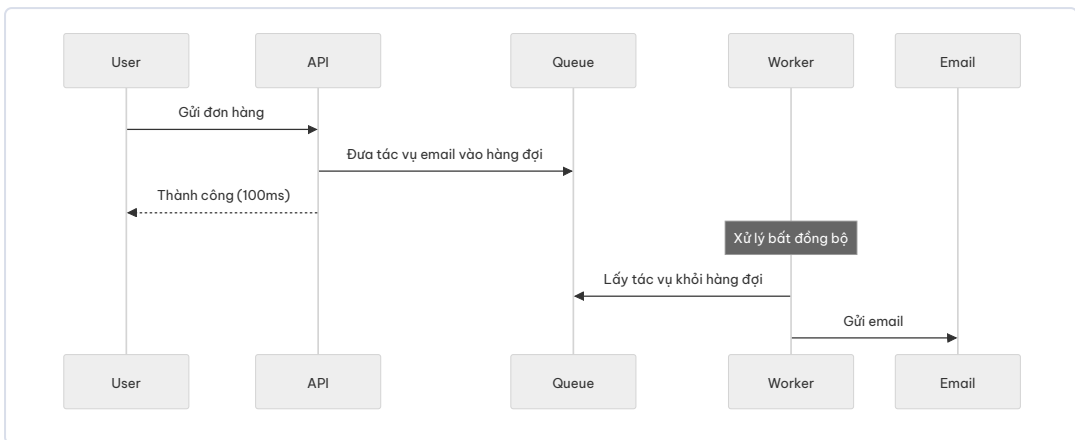
- Logic đơn giản
- Có kết quả phản hồi ngay lập tức
- Dễ gỡ lỗi (*debug*)
- Nhất quán mạnh (*Strong consistency*)

Nhược điểm:

- Độ trễ cao (người dùng phải đợi lâu)
- Trải nghiệm người dùng kém
- Lãng phí tài nguyên (người dùng chờ = kết nối bị giữ)
- Lỗi dây chuyền (*Cascade failures*) (1 bước chậm = cả yêu cầu chậm)

Xử Lý Bất Đồng Bộ

Yêu cầu → Phản hồi nhanh → Xử lý ở chế độ nền.



Đặc điểm:

Ưu điểm:

- Độ trễ thấp (người dùng không phải đợi)
- Trải nghiệm người dùng tốt hơn
- Bền bỉ (sự cố (*failures*) không chặn người dùng)
- Dễ mở rộng (*scalable*) (xử lý theo nhịp độ riêng)

Nhược điểm:

- Nhất quán cuối cùng (*Eventual consistency*)
- Xử lý lỗi phức tạp
- Khó gỡ lỗi
- Cần hạ tầng (*infrastructure*) hàng đợi (*queue*)

Khung Ra Quyết Định (Decision Framework)

Dùng ĐỒNG BỘ khi:

1. Người dùng cần kết quả ngay lập tức

```
# Login - must know if success/fail
def login(username, password):
    user = authenticate(username, password) # Must wait
    return user
```

2. Logic nghiệp vụ bắt buộc như vậy

```
# Payment - must confirm before proceeding
def checkout(cart):
    payment_result = charge_card(cart.total) # Must wait
    if payment_result.success:
        create_order()
```

3. Các thao tác đơn giản, nhanh

```
# Get user profile - fast query
def get_profile(user_id):
    return db.get_user(user_id) # 10ms, không cần async
```

Dùng BẤT ĐỒNG BỘ khi:

1. Các thao tác không quan trọng

```
# Send email - user doesn't need to wait
def register(email):
    user = create_user(email)
    send_welcome_email.delay(email) # Async
    return user
```

2. Các thao tác nặng/chậm


```
# Video processing - can take minutes
def upload_video(file):
    video = save_video(file)
    process_video.delay(video.id) # Async
    return video
```

3. Các lời gọi tới dịch vụ (service) bên ngoài

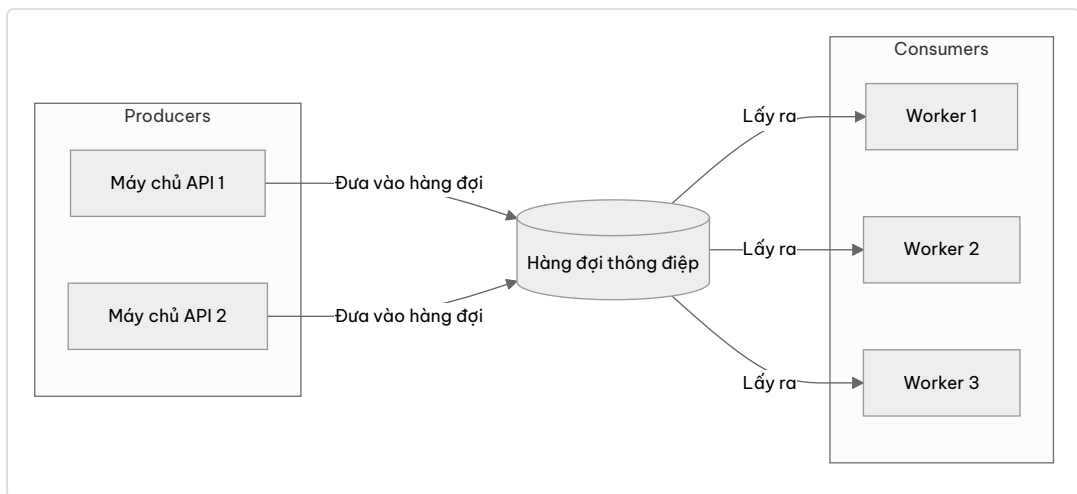
```
# 3rd party APIs - unpredictable latency
def create_order(order):
    save_order(order)
    notify_warehouse.delay(order.id) # Async
    update_analytics.delay(order)    # Async
    return order
```

4. Các thao tác phân phát (fan-out)

```
# Notify many users - can be slow
def post_update(post):
    save_post(post)
    notify_followers.delay(post.id) # Async (may be 10k users)
    return post
```

Hàng Đợi Thông Điệp - Hạ Tầng Cho Xử Lý Bất Đồng Bộ

Hàng đợi thông điệp (*Message queue*) = vùng đệm giữa bên phát (*producer*) và bên tiêu thụ (*consumer*).



Các Khái Niệm Cốt Lõi

Bên phát:

- Ứng dụng tạo ra các tác vụ
- Đưa thông điệp (*messages*) vào hàng đợi (*enqueue*)

Hàng đợi:

- Lưu trữ thông điệp
- Đảm bảo giao thông điệp
- Xử lý việc phân phối

Bên tiêu thụ/Tiến trình xử lý (*Worker*):

- Lấy thông điệp ra khỏi hàng đợi (*dequeue*)
- Xử lý tác vụ
- Xác nhận hoàn thành (*acknowledge*)

Các Hệ Thống Hàng Đợi Phổ Biến

RabbitMQ:

- Môi giới thông điệp (*Message broker*) đầy đủ tính năng
- Giao thức AMQP
- Định tuyến (*Routing*) phức tạp, *exchange*
- Phù hợp với các quy trình công việc phức tạp

Redis (cùng với Sidekiq/Bull):

- Đơn giản, nhanh
- Lưu trong bộ nhớ (lưu bền là tùy chọn)
- Phù hợp với các tác vụ đơn giản cần thông lượng (*throughput*) cao

AWS SQS:

- Dịch vụ được quản lý (*Managed service*)
- Khả năng mở rộng hoàn toàn
- Trả tiền theo mức sử dụng
- Phù hợp với hệ sinh thái AWS

Kafka:

- Nền tảng truyền dữ liệu luồng (*Streaming platform*) phân tán
- Thông lượng cao (hàng triệu thông điệp/giây)
- Event sourcing (lưu trữ theo sự kiện), log
- Phù hợp với kiến trúc hướng sự kiện (*event-driven architecture*)

So sánh:

Tính năng	RabbitMQ	Redis	SQS	Kafka
Thông lượng	Cao	Rất cao	Cao	Cực cao
Độ trễ	Thấp	Rất thấp	Trung bình	Thấp
Lưu bền (<i>Persistence</i>)	Có	Tùy chọn	Có	Có
Thứ tự (<i>Ordering</i>)	Có	Hạn chế	Hàng đợi FIFO	Có
Độ phức tạp (<i>Complexity</i>)	Trung bình	Thấp	Thấp	Cao
Tình huống sử dụng (<i>Use Case</i>)	Hàng đợi tác vụ (<i>Task queues</i>)	Tác vụ đơn giản	Ứng dụng trên AWS	Luồng sự kiện (<i>Event streams</i>)

Các Thao Tác Cơ Bản Với Hàng Đợi

Đưa vào hàng đợi - Enqueue (Bên phát):

```
# Python with Celery + Redis
from celery import Celery

app = Celery('tasks', broker='redis://localhost:6379')

@app.task
def send_email(to, subject, body):
    # Email sending logic
    smtp.send(to, subject, body)

# Enqueue task
send_email.delay('user@example.com', 'Welcome', 'Hello!')
```

Lấy ra khỏi hàng đợi - Dequeue (Bên tiêu thụ):

```
# Worker processes tasks automatically
# Start worker:
# celery -A tasks worker --loglevel=info

# Celery handles:
# - Dequeue messages
# - Call task function
# - Retry on failure
# - Acknowledge on success
```

Lấy ra thủ công (ví dụ với Redis):

```
import redis

r = redis.Redis()

while True:
    # Blocking pop (wait for messages)
    task = r.brpop('email_queue', timeout=5)

    if task:
        queue_name, message = task
        process_email(message)
```

Tác Vụ Nền (Background Jobs) - Các Mẫu (Patterns) Phổ Biến

Mẫu (Pattern) 1: Thực Thi Trì Hoãn (Deferred Execution)

Thực thi tác vụ sau một khoảng thời gian.

```
# Send reminder email after 24 hours
send_reminder_email.apply_async(
    args=[user.id],
    countdown=86400 # 24 hours in seconds
)
```

Tình huống sử dụng (Use cases):

- Thông báo nhắc nhở
- Cảnh báo sắp hết thời gian dùng thử
- Gia hạn gói đăng ký

Mẫu 2: Tác Vụ Định Kỳ (Scheduled Jobs) (Cron)

Thực thi tác vụ theo lịch định kỳ.

```
# Celery beat - periodic tasks
from celery.schedules import crontab

app.conf.beat_schedule = {
    'cleanup-old-data': {
        'task': 'tasks.cleanup_old_data',
        'schedule': crontab(hour=2, minute=0), # 2 AM daily
    },
    'send-daily-report': {
        'task': 'tasks.send_daily_report',
        'schedule': crontab(hour=9, minute=0, day_of_week=1), # Monday 9 AM
    },
}
```

Tình huống sử dụng:

- Báo cáo hằng ngày
- Dọn dẹp dữ liệu
- Làm nóng bộ nhớ đệm (Cache warming)

- Tác vụ sao lưu

Mẫu 3: Xử Lý Theo Lô (*Batch Processing*)

Xử lý nhiều mục cùng một lúc.

```
# Process batch of emails
@app.task
def send_batch_emails(email_ids):
    emails = Email.objects.filter(id__in=email_ids)
    for email in emails:
        send_email(email)

# Split into batches
email_ids = [1, 2, 3, ..., 10000]
batch_size = 100

for i in range(0, len(email_ids), batch_size):
    batch = email_ids[i:i+batch_size]
    send_batch_emails.delay(batch)
```

Lợi ích:

- Sử dụng tài nguyên hiệu quả
- Thông lượng tốt hơn
- Tối ưu truy vấn (*query*) cơ sở dữ liệu

Mẫu 4: Đường Ống (*Pipeline*)/Chuỗi (*Chain*)

Các tác vụ được thực thi theo trình tự.

```
from celery import chain

# Task chain
workflow = chain(
    resize_image.s(image_id),
    apply_watermark.s(),
    upload_to_cdn.s(),
    update_database.s()
)

workflow.apply_async()
```

Đầu ra của mỗi tác vụ = đầu vào của tác vụ kế tiếp.

Mẫu 5: Phân Tán – Gom Về (*Fan-out/Fan-in*)

Thực thi song song, sau đó tổng hợp kết quả.

```
from celery import group, chord

# Process multiple images in parallel
job = chord(
    group(process_image.s(img_id) for img_id in image_ids),
    aggregate_results.s() # Called after all finish
)

job.apply_async()
```

Tình huống sử dụng:

- Xử lý nhiều tệp
- Gọi API song song
- Các thao tác map-reduce

Chiến Lược Thử Lại - Xử Lý Sự Cố Một Cách Êm Thấm

Sự cố là điều không tránh khỏi. Hết thời gian chờ (*Timeout*) mạng, dịch vụ sập, bị giới hạn tần suất (*rate limits*).

Thử Lại Tự Động

```
@app.task(
    bind=True,
    max_retries=3,
    default_retry_delay=60 # 60 seconds
)
def send_email(self, to, subject, body):
    try:
        smtp.send(to, subject, body)
    except SMTPException as exc:
        # Retry with exponential backoff
        raise self.retry(exc=exc, countdown=2 ** self.request.retries)
```

Lùi thời gian theo cấp số nhân (*Exponential backoff*):

Lần thử lại 1: chờ $2^1 = 2$ giây
Lần thử lại 2: chờ $2^2 = 4$ giây
Lần thử lại 3: chờ $2^3 = 8$ giây

Tránh dồn quá tải lên dịch vụ đang gặp sự cố.

Thử Lại Có Điều Kiện

Chỉ thử lại với một số lỗi cụ thể.

```
@app.task(bind=True, max_retries=5)
def call_external_api(self, url):
    try:
        response = requests.get(url, timeout=10)
        response.raise_for_status()
        return response.json()
    except requests.Timeout as exc:
        # Retry on timeout
        raise self.retry(exc=exc, countdown=30)
    except requests.HTTPError as exc:
        if exc.response.status_code == 429: # Rate limited
            raise self.retry(exc=exc, countdown=120)
        elif exc.response.status_code >= 500: # Server error
            raise self.retry(exc=exc)
        else:
            # 4xx errors - don't retry
            raise
```

Logic:

- Hết thời gian chờ → thử lại (lỗi tạm thời)
- 429 Giới hạn tần suất (*Rate limit*) → thử lại với khoảng chờ dài hơn
- 5xx Lỗi phía máy chủ → thử lại (có thể tự phục hồi)
- 4xx Lỗi phía máy khách → không thử lại (sẽ không thành công)

Giới Hạn Số Lần Thử Lại

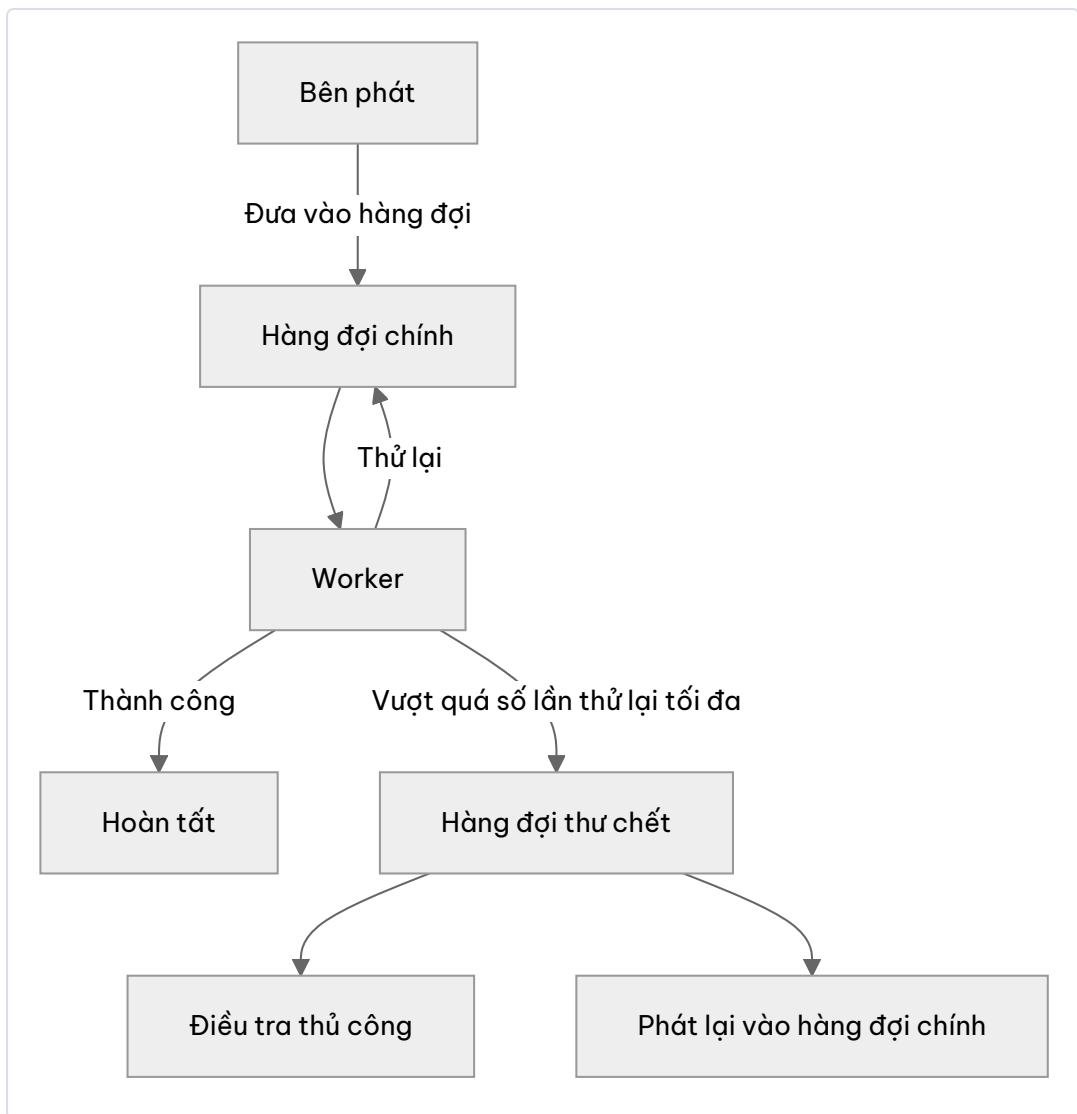
Đừng thử lại mãi mãi.


```
@app.task(
    max_retries=10,
    retry_backoff=True,
    retry_backoff_max=600, # Max 10 minutes
    retry_jitter=True      # Add randomness
)
def risky_task():
    # Task logic
    pass
```

Sau số lần thử lại tối đa → tác vụ thất bại → chuyển vào DLQ.

Hàng Đợi Thư Chết (*Dead Letter Queue*) (DLQ) - Xử Lý Các Lỗi Vĩnh Viễn

DLQ = hàng đợi dành cho các thông điệp đã thất bại vĩnh viễn.



Vì Sao DLQ Quan Trọng

1. Không làm mất thông điệp

Các tác vụ thất bại không biến mất. Chúng được lưu trong DLQ để điều tra.

2. Tránh làm tắc hàng đợi

Thông điệp (*Message*) bị lỗi không chặn hàng đợi. Các thông điệp khác vẫn được xử lý bình thường.

3. Hỗ trợ gỡ lỗi (*debugging*)

Phân tích các thông điệp thất bại, hiểu quy luật, sửa lỗi (*bugs*).

4. Can thiệp thủ công

Quản trị viên có thể phát lại thông điệp sau khi đã khắc phục sự cố.

Triển Khai DLQ

Celery + RabbitMQ:

```
from celery import Celery

app = Celery('tasks')

app.conf.task_reject_on_worker_lost = True
app.conf.task_acks_late = True

@app.task(
    bind=True,
    max_retries=3,
    dead_letter_exchange='dlx',
    dead_letter_routing_key='dlq'
)
def problematic_task(self):
    # Task logic
    pass
```

AWS SQS:

```
import boto3

sqs = boto3.client('sqs')

# Create main queue with DLQ
response = sqs.create_queue(
    QueueName='main-queue',
    Attributes={
        'RedrivePolicy': json.dumps({
            'deadLetterTargetArn': dlq_arn,
            'maxReceiveCount': '3' # After 3 attempts → DLQ
        })
    }
)
```

Giám Sát (Monitoring) DLQ

Theo dõi các chỉ số (metrics):

- Kích thước DLQ (số lượng thông điệp)
- Tuổi của thông điệp trong DLQ
- Phân bố nguyên nhân thất bại
- Xu hướng tỷ lệ thất bại

Cảnh báo (Alert) khi:

- Kích thước DLQ > ngưỡng
- Số lỗi tăng đột biến (spike)
- Thông điệp nằm lâu trong DLQ

Rà soát định kỳ:

```
def review_dlq():
    messages = dlq.get_messages(limit=100)

    for msg in messages:
        # Analyze failure reason
        if is_fixable(msg):
            fix_and_replay(msg)
        else:
            log_permanent_failure(msg)
```

Tính Lũy Đẳng - Yếu Tố Then Chốt Cho Độ Tin Cậy

Lũy đẳng (Idempotent) = thực thi nhiều lần cho ra cùng kết quả như thực thi một lần.

Vấn Đề

Kịch bản:

1. Người dùng gửi đơn hàng
2. API đưa tác vụ "charge_payment" vào hàng đợi
3. Worker xử lý tác vụ, trừ tiền thẻ
4. Hết thời gian chờ mạng trước khi kịp xác nhận (ack)
5. Hàng đợi giao lại tác vụ
6. Worker xử lý lần nữa → trừ tiền thẻ HAI LẦN! 💎

Người dùng bị trừ tiền 2 lần. Rất tệ.

Giải Pháp: Các Thao Tác Lũy Đăng

Cách 1: Khoá lũy đăng (*Idempotency Keys*)

```
@app.task
def charge_payment(order_id, amount, idempotency_key):
    # Check if already processed
    if Payment.objects.filter(idempotency_key=idempotency_key).exists():
        print("Already processed, skipping")
        return

    # Process payment
    result = stripe.charge(amount, idempotency_key=idempotency_key)

    # Save with idempotency key
    Payment.objects.create(
        order_id=order_id,
        amount=amount,
        idempotency_key=idempotency_key,
        transaction_id=result.id
    )
```

Stripe API cũng hỗ trợ khoá lũy đăng - bảo vệ hai lớp.

Cách 2: Ràng buộc (*Constraints*) **Của Cơ Sở Dữ Liệu**

```
# Unique constraint on database
class Payment(models.Model):
    order_id = models.IntegerField()
    transaction_id = models.CharField(max_length=100, unique=True)

@app.task
def charge_payment(order_id):
    try:
        result = stripe.charge(...)
        Payment.objects.create(
            order_id=order_id,
            transaction_id=result.id # Unique - prevents duplicates
        )
    except IntegrityError:
        # Duplicate transaction - already processed
        print("Already processed")
```

Cách 3: Thao tác nguyên tử (*Atomic Operations*)

```
# Use database atomic operations
@app.task
def increment_view_count(post_id):
    # Idempotent - can run multiple times safely
    Post.objects.filter(id=post_id).update(
        view_count=F('view_count') + 1
    )
```

Cách 4: Máy trạng thái (State Machines)

```
class Order(models.Model):
    status = models.CharField(
        choices=[
            ('pending', 'Pending'),
            ('processing', 'Processing'),
            ('completed', 'Completed'),
        ]
    )

@app.task
def process_order(order_id):
    order = Order.objects.get(id=order_id)

    # Only process if pending
    if order.status != 'pending':
        print("Already processed")
        return

    # Update status atomically
    updated = Order.objects.filter(
        id=order_id,
        status='pending'
    ).update(status='processing')

    if updated == 0:
        print("Another worker already processing")
        return

    # Process order...

    order.status = 'completed'
    order.save()
```

Các bước chuyển trạng thái đảm bảo tính lũy đẳng.

Thực Hành Tốt Nhất (Best Practices)

1. Luôn dùng khoá lũy đẳng cho các thao tác tài chính

```
import uuid

idempotency_key = str(uuid.uuid4())
charge_payment.delay(order_id, amount, idempotency_key)
```

2. Tạo khoá lũy đẳng (idempotency key) ở phía bên phát, không phải ở bên tiêu thụ

Bên phát đảm bảo dùng cùng một khoá cho các lần thử lại (retries).

3. Lưu các khoá đã xử lý kèm TTL

```
# Redis
redis.setex(f"processed:{idempotency_key}", 86400, "1") # 24h TTL
```

4. Làm cho mọi lời gọi API bên ngoài đều lũy đẳng

Stripe, các cổng thanh toán, dịch vụ email đều hỗ trợ khoá lũy đẳng.

Giới Thiệu Kiến Trúc Hướng Sự Kiện

Xử lý bất đồng bộ tự nhiên dẫn tới tư duy hướng sự kiện.

Từ Hướng Yêu Cầu (Request-Driven) Sang Hướng Sự Kiện

Hướng yêu cầu (truyền thống):

```
def create_order(order_data):
    order = save_order(order_data)
    charge_payment(order)
    send_confirmation(order)
    notify_warehouse(order)
    update_analytics(order)
    return order
```

Liên kết chặt (*Tight coupling*). Phần tạo đơn hàng phải biết về tất cả các hệ thống phía sau.

Hướng sự kiện:

```
def create_order(order_data):
    order = save_order(order_data)

    # Publish event
    publish_event('order.created', {
        'order_id': order.id,
        'user_id': order.user_id,
        'total': order.total
    })

    return order

# Separate subscribers
@subscribe('order.created')
def handle_payment(event):
    charge_payment(event['order_id'])

@subscribe('order.created')
def send_email(event):
    send_confirmation(event['order_id'])

@subscribe('order.created')
def notify_warehouse(event):
    notify_warehouse(event['order_id'])
```

Lợi ích:

1. Tách rời (*Decoupling*)

Dịch vụ đơn hàng không cần biết về dịch vụ email hay dịch vụ kho hàng.

2. Khả năng mở rộng (*Scalability*)

Mỗi bên đăng ký (*subscriber*) mở rộng (*scale*) một cách độc lập.

3. Tính linh hoạt

Thêm bên đăng ký (*subscribers*) mới mà không phải thay đổi dịch vụ đơn hàng.

4. Khả năng phục hồi

1 bên đăng ký gặp lỗi không ảnh hưởng tới các bên khác.

Cấu Trúc Sự Kiện (Event)

```
{
  "event_id": "evt_abc123",
  "event_type": "order.created",
  "timestamp": "2025-02-15T10:00:00Z",
  "version": "1.0",
  "data": {
    "order_id": 12345,
    "user_id": 789,
    "total": 99.99
  }
}
```

Bao gồm:

- ID sự kiện duy nhất (phục vụ tính lũy đẳng)
- Loại sự kiện (phục vụ định tuyến)
- Dấu thời gian (*Timestamp*) (phục vụ thứ tự)
- Phiên bản (phục vụ tiến hoá lược đồ (*schema*))
- Dữ liệu gửi kèm (*Data payload*)

Các Mẫu Sự Kiện

1. Thông Báo Sự Kiện (Event Notification)

Thông báo rằng có điều gì đó đã xảy ra. Các bên đăng ký tự lấy chi tiết nếu cần.

```
publish_event('user.registered', {'user_id': 123})
```

2. Sự Kiện Mang Theo Trạng Thái (Event-Carried State Transfer)

Sự kiện chứa toàn bộ dữ liệu mà các bên đăng ký cần.

```
publish_event('user.registered', {
  'user_id': 123,
  'email': 'user@example.com',
  'name': 'John Doe',
  'signup_date': '2025-02-15'
})
```

3. Event Sourcing (Lưu Trữ Theo Sự Kiện)

Lưu mọi sự kiện (*events*) làm nguồn sự thật duy nhất. Trạng thái hiện tại = phát lại các sự kiện.

```
events = [
    {'type': 'account.created', 'balance': 1000},
    {'type': 'transaction.deposit', 'amount': 500},
    {'type': 'transaction.withdraw', 'amount': 200},
]

# Current balance = 1000 + 500 - 200 = 1300
```

Khuyến Nghị Thực Tế

1. Bắt Đầu Đơn Giản

Đừng thiết kế thừa (*over-engineer*). Hãy bắt đầu với các tác vụ nền cơ bản.

```
# Simple Celery task
@app.task
def send_email(to, subject, body):
    smtp.send(to, subject, body)

# Usage
send_email.delay('user@example.com', 'Welcome', 'Hello!')
```

Chỉ thêm độ phức tạp khi cần:

- Thử lại → khi có phụ thuộc bên ngoài
- DLQ → khi cần giám sát các sự cố
- Tính lũy đẳng → khi có các thao tác quan trọng

2. Giám Sát (*Monitor*) Sức Khỏe Hàng Đợi

Theo dõi các chỉ số:

```

# Queue depth
queue_size = celery.control.inspect().stats()

# Task success/failure rate
from celery.events import EventReceiver

def monitor_events():
    receiver = EventReceiver(app.connection())

    for event in receiver.events():
        if event['type'] == 'task-succeeded':
            metrics.increment('task.success')
        elif event['type'] == 'task-failed':
            metrics.increment('task.failure')

```

Cảnh báo khi:

- Độ sâu hàng đợi (*Queue depth*) > ngưỡng (các tiến trình xử lý (*workers*) không theo kịp)
- Tỷ lệ thất bại cao (vấn đề mang tính hệ thống)
- Thời gian xử lý > SLA

3. Thiết Kế Cho Sự Cố (*Failure*)

Hãy giả định rằng các tác vụ sẽ thất bại. Lên kế hoạch tương ứng.

```

@app.task(
    bind=True,
    max_retries=3,
    default_retry_delay=60
)
def resilient_task(self, data):
    try:
        # Task logic with timeout
        result = process_with_timeout(data, timeout=30)
        return result
    except TimeoutError as exc:
        # Retry on timeout
        raise self.retry(exc=exc)
    except Exception as exc:
        # Log error for investigation
        logger.error(f"Task failed: {exc}", extra={'data': data})
        raise

```

4. Kiểm Thử Các Luồng Bất Đồng Bộ

Kiểm thử mã bất đồng bộ khó hơn mã đồng bộ.

```
# Test with eager mode (synchronous for testing)
app.conf.task_always_eager = True

def test_order_creation():
    order = create_order(order_data)

    # Tasks execute synchronously in tests
    assert email_was_sent(order.user.email)
    assert payment_was_charged(order.id)
```

Kiểm thử tích hợp (*Integration tests*):

```
def test_queue_integration():
    # Send task to real queue
    result = send_email.delay('test@example.com', 'Test', 'Body')

    # Wait for completion
    assert result.get(timeout=10) == 'sent'
```

5. Suy Giảm Có Kiểm Soát (*Graceful Degradation*)

Hàng đợi sập không nên kéo theo cả hệ thống sập.

```
def create_order(order_data):
    order = save_order(order_data)

    try:
        send_confirmation.delay(order.id)
    except ConnectionError:
        # Queue unavailable - fallback to sync
        logger.warning("Queue unavailable, sending email synchronously")
        send_confirmation_sync(order.id)

    return order
```

Mô Hình Tư Duy (*Mental Model*): Giảm Độ Trễ Bằng Cách Trì Hoãn Công Việc

Điểm mấu chốt:

Người dùng không cần đợi những công việc không quan trọng. Hãy trì hoãn chúng để cải thiện trải nghiệm.

Tư duy đồng bộ:

Hoàn tất mọi thứ trước khi phản hồi
Trải nghiệm người dùng = chậm

Tư duy bất đồng bộ:

Chỉ hoàn tất công việc quan trọng
Trì hoãn phần còn lại sang chạy nền
Trải nghiệm người dùng = nhanh

Cây quyết định:

Người dùng có cần kết quả ngay không?

CÓ → Đồng bộ (ví dụ: xác thực đăng nhập)

KHÔNG → Có thể trì hoãn không?

CÓ → Bất đồng bộ (ví dụ: gửi email)

KHÔNG → Đồng bộ (ví dụ: xác thực thanh toán)

Tư duy kiến trúc sư (*architect*):

1. Xác định đường găng (*critical path*) (thứ người dùng thực sự cần)
2. Trì hoãn mọi thứ còn lại (email, số liệu phân tích, v.v.)
3. Dùng hàng đợi để đảm bảo độ tin cậy (thử lại, DLQ)
4. Thiết kế hướng tới tính lũy đẳng (an toàn)
5. Giám sát sức khỏe hàng đợi (vận hành xuất sắc)

Hệ thống tuyệt vời phản hồi nhanh, còn công việc thì diễn ra sau.

Những Điểm Cốt Lõi

1. Xử lý bất đồng bộ giảm độ trễ mà người dùng cảm nhận

Người dùng nhận phản hồi nhanh, công việc diễn ra ở chế độ nền.

2. Hàng đợi thông điệp tách rời (decouple) bên phát (producers) khỏi bên tiêu thụ (consumers)

Mở rộng độc lập, chịu được tăng đột biến (spikes), tăng khả năng phục hồi.

3. Chọn hệ thống hàng đợi dựa trên yêu cầu (requirements)

Redis = đơn giản/nhanh, RabbitMQ = nhiều tính năng, SQS = được quản lý sẵn, Kafka = truyền dữ liệu luồng.

4. Các chiến lược thử lại xử lý các sự cố tạm thời

Lùi thời gian theo cấp số nhân, thử lại có điều kiện, giới hạn số lần thử lại.

5. DLQ thu giữ các sự cố vĩnh viễn

Không làm mất thông điệp, hỗ trợ điều tra, cho phép phát lại.

6. Tính lũy đẳng là yếu tố then chốt cho độ tin cậy

Dùng khoá lũy đẳng, ràng buộc cơ sở dữ liệu, thao tác nguyên tử.

7. Kiến trúc hướng sự kiện = tách rời ở mức tối đa

Các dịch vụ (services) giao tiếp qua sự kiện, không gọi trực tiếp lẫn nhau.

8. Giám sát sức khỏe hàng đợi liên tục

Độ sâu hàng đợi, tỷ lệ tác vụ thành công, thời gian xử lý.

9. Thiết kế cho sự cố

Tác vụ sẽ thất bại. Hãy chuẩn bị logic thử lại, DLQ, phương án dự phòng.

10. Kiểm thử kỹ các luồng bất đồng bộ

Dùng chế độ eager cho kiểm thử đơn vị (unit tests), dùng hàng đợi thật cho kiểm thử tích hợp (integration).

Hãy nhớ: Trải nghiệm người dùng tốt nhất là phản hồi nhanh. Hãy trì hoãn công việc không quan trọng sang chạy nền. Người dùng không cần đợi email được gửi đi hay số liệu phân tích được cập nhật - họ cần thấy "Đặt hàng thành công" ngay lập tức.

Giám Sát (Monitoring), Khả Năng Quan Sát (Observability) & Gỡ Lỗi Hiệu Năng (Performance Debugging) - Đo Lường Để Mở Rộng

Bạn triển khai (*deploy*) hệ thống lên môi trường thực tế (*production*). Mọi thứ chạy tốt... hoặc ít nhất bạn nghĩ vậy.

Rồi người dùng phàn nàn: "Trang chậm quá". Bạn kiểm tra:

- CPU máy chủ: 30% 
- Bộ nhớ: 50%
- Ổ đĩa: 40% 

Các chỉ số (*metrics*) trông đều ổn. Vậy tại sao người dùng lại nói chậm?

Vấn đề: Bạn đang đo sai thứ.

CPU 30% chẳng nói lên điều gì về trải nghiệm người dùng. Có thể là:

- Thời gian phản hồi (*response*) của API là 5 giây (người dùng phải chờ lâu)
- Các truy vấn (*queries*) cơ sở dữ liệu (*database*) bị hết thời gian chờ (*timeout*)
- Các lời gọi API bên thứ ba thất bại ở 50% yêu cầu (*requests*)

Không đo đúng = không biết hệ thống thực sự hoạt động thế nào.

Mô hình tư duy (*Mental model*) quan trọng nhất: Nếu bạn không đo được, bạn không thể mở rộng.

Bài này dạy bạn:

- Những chỉ số nào thực sự quan trọng
- Bách phân vị (*Percentiles*) (p50, p95, p99) và tại sao chúng quan trọng
- Chiến lược cảnh báo (*Alert strategy*) (không gửi tràn lan, không bỏ sót sự cố nghiêm trọng)
- Truy vết phân tán (*Distributed tracing*) cho vi dịch vụ (*microservices*)

- Thực hành tốt nhất (*Best practices*) khi ghi log (*logging*)
- SLO/SLA/Ngân sách lỗi (*Error budgets*)

Tại Sao Giám Sát Lại Quan Trọng?

1. Phát Hiện Vấn Đề Trước Người Dùng

Chủ động > Bị động.

Không có giám sát:

Người dùng phàn nàn → điều tra → tìm ra vấn đề → sửa

Thời gian trung bình để phát hiện (MTTD): Vài giờ đến vài ngày

Có giám sát:

Cảnh báo kích hoạt → điều tra → sửa → người dùng không hề nhận ra

MTTD: Vài giây đến vài phút

Giám sát tốt = người dùng không biết là đã có sự cố.

2. Hiểu Hành Vi Của Hệ Thống

Hoạch định năng lực (*Capacity planning*):

Hiện tại: 1000 req/s, CPU 40%

Câu hỏi: Có chịu được 5000 req/s không?

Trả lời: Cần dữ liệu để ngoại suy

Không có số liệu đo thì chỉ là đoán mò. Có số liệu đo thì quyết định dựa trên thông tin.

3. Gỡ Lỗi (*Debug*) Các Vấn Đề Hiệu Năng (*Performance*)

Người dùng: "Trang chậm vào lúc 3 giờ chiều mỗi ngày."

Không có giám sát: Không biết tại sao.

Có giám sát:

Kiểm tra số liệu lúc 3 giờ chiều:

- Thời gian truy vấn CSDL tăng đột biến gấp 10 lần

- Log truy vấn chậm: tác vụ tạo báo cáo hằng ngày

→ Dời việc tạo báo cáo sang giờ thấp điểm

4. Kiểm Chứng Các Tối Ưu

Bạn: "Đã thêm lưu đệm (*caching*), chắc là nhanh hơn rồi."

Đối chiếu thực tế:

Trước: độ trễ p95 là 800ms
Sau: độ trễ p95 là 150ms
→ Xác nhận cải thiện gấp 5 lần ✓

Đo lường để chứng minh tác động.

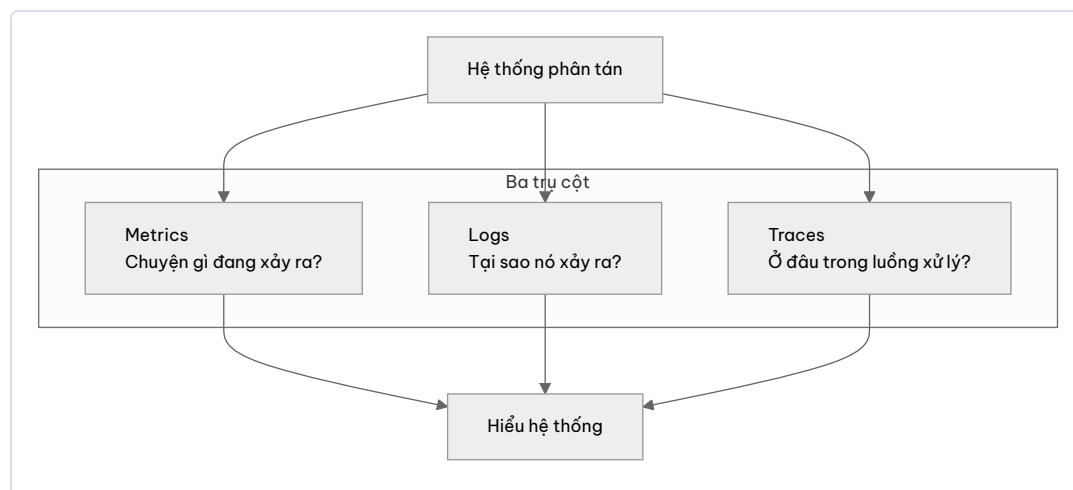
5. Đáp Ứng SLA

Hợp đồng: "Thời gian hoạt động (*Uptime*) 99,9%, độ trễ (*latency*) p99 < 500ms"

Làm sao biết mình có đáp ứng được không? Giám sát 24/7.

Ba Trụ Cột Của Khả Năng Quan Sát

Khả năng quan sát = khả năng hiểu trạng thái bên trong của hệ thống từ các đầu ra bên ngoài.



Số liệu đo: Các con số tổng hợp (CPU, độ trễ, tỷ lệ lỗi)

Log (Logs): Các sự kiện (*events*) rời rạc (lỗi, cảnh báo, thông tin gỡ lỗi)

Vết (Traces): Luồng đi của yêu cầu qua các dịch vụ (*services*)

Cần cả ba thì mới có được tầm nhìn toàn diện.

Số Liệu Đo - Nên Đo Những Gì

Tín Hiệu Vàng (*Golden Signals*) (Google SRE)

4 chỉ số quan trọng nhất:

1. Độ Trễ

Thời gian để phục vụ một yêu cầu.

```
import time

def track_latency(func):
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        duration = time.time() - start

        metrics.histogram('api.latency', duration, tags=['endpoint:users'])
        return result
    return wrapper

@track_latency
def get_users():
    return db.query("SELECT * FROM users")
```

Theo dõi:

- Độ trễ tổng thể của API
- Độ trễ theo từng điểm cuối (*endpoint*)
- Thời gian truy vấn cơ sở dữ liệu
- Thời gian gọi API bên ngoài

Tại sao: Ảnh hưởng trực tiếp đến trải nghiệm người dùng.

2. Lưu Lượng (*Traffic*)

Số yêu cầu mỗi giây (thông lượng (*throughput*)).

```
@app.before_request
def track_request():
    metrics.increment('api.requests', tags=[
        f'endpoint:{request.endpoint}',
        f'method:{request.method}'
    ])

```

Theo dõi:

- Tổng số yêu cầu/giây
- Số yêu cầu theo từng điểm cuối
- Tỷ lệ đọc so với ghi

Tại sao: Hiểu các mẫu tải (*load*), phục vụ hoạch định năng lực.

3. Lỗi

Tỷ lệ yêu cầu thất bại.

```
@app.after_request
def track_response(response):
    if response.status_code >= 500:
        metrics.increment('api.errors.5xx')
    elif response.status_code >= 400:
        metrics.increment('api.errors.4xx')

    return response

```

Theo dõi:

- Lỗi 4xx (lỗi phía máy khách (*client*))
- Lỗi 5xx (lỗi phía máy chủ (*server*))
- Tỷ lệ lỗi (*Error rate*) (%)

Tại sao: Chỉ báo độ tin cậy (*reliability*), là điều kiện kích hoạt cảnh báo.

4. Độ Bão Hòa (*Saturation*)

Dịch vụ của bạn đang "đầy" đến mức nào.

```
# System metrics
metrics.gauge('system.cpu_percent', psutil.cpu_percent())
metrics.gauge('system.memory_percent', psutil.virtual_memory().percent)
metrics.gauge('system.disk_percent', psutil.disk_usage('/').percent)

# Application metrics
metrics.gauge('db.connection_pool.used', db.pool.size())
metrics.gauge('queue.depth', redis.llen('task_queue'))
```

Theo dõi:

- Mức sử dụng CPU
- Mức sử dụng bộ nhớ
- Mức sử dụng ổ đĩa
- Mức sử dụng nhóm kết nối (*connection pool*)
- Độ sâu hàng đợi (*Queue depth*)

Tại sao: Dự đoán khi nào hệ thống sẽ chạm giới hạn.

Phương Pháp RED (Cho Dịch Vụ)

Rate (tần suất), Errors (lỗi), Duration (thời lượng).

```
class ServiceMetrics:
    def record_request(self, duration, success):
        # Rate
        self.metrics.increment('service.requests')

        # Errors
        if not success:
            self.metrics.increment('service.errors')

        # Duration
        self.metrics.histogram('service.duration', duration)
```

Đơn giản, bao quát những gì thiết yếu.

Phương Pháp USE (Cho Tài Nguyên)

Utilization (mức sử dụng), Saturation (độ bão hòa), Errors (lỗi).

```
# Utilization: % time resource busy
cpu_utilization = psutil.cpu_percent()

# Saturation: work queued
load_average = os.getloadavg()[0] # 1-minute load

# Errors: error count
disk_errors = read_disk_error_count()
```

Dùng cho giám sát hạ tầng (infrastructure).

Bách Phân Vị - Tại Sao Chúng Quan Trọng Hơn Giá Trị Trung Bình

Vấn Đề Của Giá Trị Trung Bình

Kịch bản:

```
Độ trễ API:
99 yêu cầu: 100ms
1 yêu cầu: 10,000ms (10 giây!)

Trung bình: (99 * 100 + 10,000) / 100 = 199ms
```

Trung bình nói 199ms - trông ổn!

Thực tế: 1% người dùng phải chờ 10 giây - trải nghiệm tệ hại.

Giá trị trung bình che giấu các giá trị ngoại lai (outliers).

Bách Phân Vị Kể Câu Chuyện Thật

p50 (trung vị (median)): 50% yêu cầu nhanh hơn, 50% chậm hơn

p95: 95% yêu cầu nhanh hơn mức này

p99: 99% yêu cầu nhanh hơn mức này

p99.9: 99,9% yêu cầu nhanh hơn mức này

Ví dụ về phân phối:

p50: 100ms (trải nghiệm điển hình)
p95: 200ms (đa số người dùng)
p99: 500ms (trường hợp biên)
p99.9: 2000ms (ngoại lai hiếm gặp)

p95, p99 cho thấy trải nghiệm người dùng trong trường hợp xấu nhất.

Tại Sao p99 Quan Trọng

Nghiên cứu của Amazon: độ trễ p99 tương quan trực tiếp với doanh thu.

Độ trễ p99 = 1 giây
→ Chậm với 1% người dùng
→ Với 1 triệu yêu cầu/ngày = 10,000 trải nghiệm tồi
→ Đánh giá xấu, mất doanh số

Tối ưu cho p99, không phải cho giá trị trung bình.

Theo Dõi Bách Phân Vị

```
from prometheus_client import Histogram

# Histogram automatically calculates percentiles
latency_histogram = Histogram(
    'api_latency_seconds',
    'API request latency',
    buckets=[0.1, 0.25, 0.5, 1.0, 2.5, 5.0]
)

@app.route('/api/data')
def get_data():
    with latency_histogram.time():
        return process_request()
```

Truy vấn Prometheus:

```
# p95 latency over 5 minutes
histogram_quantile(0.95,
    rate(api_latency_seconds_bucket[5m])
)
```

Chỉ Số Mức Dịch Vụ (Service Level Indicators) (SLI)

Các bách phân vị đo được sẽ trở thành SLI.

SLI: độ trễ p99 < 500ms
SLI: tính sẵn sàng p95 > 99.9%
SLI: tỷ lệ lỗi p99 < 0.1%

Các mục tiêu khách quan, đo lường được.

Chiến Lược Cảnh Báo - Tín Hiệu Và Nhiều

Vấn Đề Mệt Mỏi Vì Cảnh Báo (Alert Fatigue)

Cảnh báo tồi:

03:00 - Cảnh báo: CPU > 80%
03:05 - Cảnh báo: CPU > 80%
03:10 - Cảnh báo: CPU > 80%
...
→ 100 cảnh báo/đêm
→ Kỹ sư phớt lờ cảnh báo
→ Bỏ sót sự cố thật

Mệt mỏi vì cảnh báo giết chết độ tin cậy.

Các Nguyên Tắc Cảnh Báo Tốt

1. Cảnh báo dựa trên triệu chứng, không phải nguyên nhân

Tồi:

Cảnh báo: CPU của CSDL > 80%

Tốt:

Cảnh báo: độ trễ p99 của API > 1 giây

Tại sao: CPU cao có thể vẫn ổn nếu độ trễ bình thường. Độ trễ cao = người dùng bị ảnh hưởng = vấn đề thật.

2. Cảnh báo dựa trên tác động tới người dùng

Tồi:

Cảnh báo: 1 máy chủ ngừng hoạt động (trên tổng số 10)

Tốt:

Cảnh báo: Tỷ lệ lỗi > 5%
(Chỉ khi trải nghiệm người dùng bị ảnh hưởng)

3. Chỉ dùng cảnh báo có thể hành động được

Mỗi cảnh báo (*alert*) phải đi kèm một hành động rõ ràng.

Cảnh báo: Phát hiện truy vấn chậm trên CSDL

Hành động:

1. Kiểm tra log truy vấn chậm
2. Kiểm tra tải của CSDL
3. Hủy các truy vấn chạy lâu nếu cần
4. Gọi nhóm CSDL nếu tình trạng kéo dài > 15 phút

Không có hành động = không nên cảnh báo.

4. Dùng ngưỡng (*thresholds*) một cách khôn ngoan

Ngưỡng tĩnh (*Static threshold*):

```
alert: CPU > 90%
```

Vấn đề: Tải bình thường dao động. 90% lúc 2 giờ sáng = có vấn đề. 90% lúc cao điểm (*peak*) = bình thường.

Ngưỡng động (*Dynamic threshold*):

```
alert: CPU > (baseline + 2 * stddev)
```

Tự thích ứng với các mẫu hành vi bình thường.

5. Các mức độ ưu tiên của cảnh báo

P0 (Ng nghiêm trọng): Hệ thống sập, ảnh hưởng doanh thu

→ Gọi ngay người trực sự cố (on-call)

P1 (Cao): Hiệu năng suy giảm, ảnh hưởng người dùng

→ Cảnh báo người trực sự cố, yêu cầu phản hồi trong vòng 30 phút

P2 (Trung bình): Vấn đề không nghiêm trọng

→ Tạo phiếu xử lý, giải quyết trong giờ hành chính

P3 (Thấp): Mang tính thông tin

→ Chỉ ghi log, không cảnh báo

Sổ Tay Xử Lý (Runbooks) Cho Cảnh Báo

Mỗi cảnh báo cần một sổ tay xử lý (runbook).

Cảnh báo: Tỷ lệ lỗi API cao

Triệu chứng

Tỷ lệ lỗi API > 5% trong 5 phút

Tác động

Người dùng gặp lỗi, có thể mất doanh thu

Các bước điều tra

1. Kiểm tra bằng điều khiển lỗi: `${dashboard_link}`
2. Kiểm tra log lỗi: ``kubectl logs -l app=api --tail=100``
3. Kiểm tra trạng thái các dịch vụ phụ thuộc
4. Kiểm tra các lần triển khai gần đây

Khắc phục

1. Nếu do triển khai lỗi: hoàn tác (rollback) ngay lập tức
2. Nếu do CSDL: kiểm tra sức khỏe CSDL, cân nhắc dùng bản sao chỉ đọc
3. Nếu do bên thứ ba: bật bộ ngắt mạch (circuit breaker), dùng phương án dự phòng

Leo thang

Nếu chưa giải quyết sau 15 phút: gọi @backend-lead

Giảm MTTR (thời gian trung bình để khắc phục (Mean Time To Resolve)).

Truy Vết Phân Tán - Gỡ Lỗi Xuyên Suốt Các Dịch Vụ

Vấn Đề Của Vi Dịch Vụ

Hệ thống nguyên khối (Monolith):

Yêu cầu người dùng → Một codebase duy nhất → Dễ gỡ lỗi

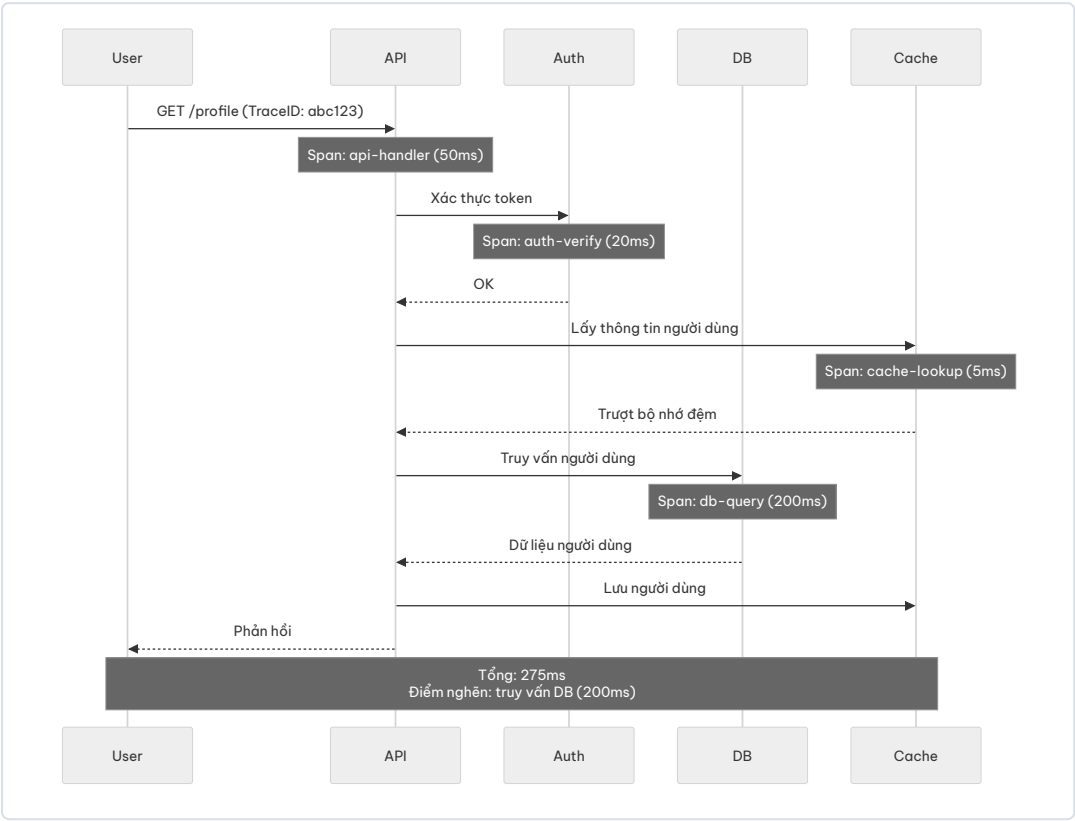
Vi dịch vụ:

Yêu cầu người dùng → Service A → Service B → Service C
↓
Service D

Điểm nghẽn (Bottleneck) nằm ở đâu? Khó mà biết được.

Giải Pháp Truy Vết Phân Tán

Vết (Trace) = toàn bộ hành trình của yêu cầu qua các dịch vụ.



Mỗi thao tác = một span (đoạn). Tập hợp các span = một vết.

Triển Khai Truy Vết (*Tracing*)

OpenTelemetry (chuẩn chung):

```
from opentelemetry import trace
from opentelemetry.instrumentation.flask import FlaskInstrumentor

# Initialize tracer
tracer = trace.get_tracer(__name__)

# Instrument Flask automatically
FlaskInstrumentor().instrument_app(app)

# Manual spans for custom operations
@app.route('/api/user/<user_id>')
def get_user(user_id):
    with tracer.start_as_current_span("get_user") as span:
        span.set_attribute("user.id", user_id)

        # Database query
        with tracer.start_as_current_span("db.query.user"):
            user = db.query_user(user_id)

        # Cache operation
        with tracer.start_as_current_span("cache.set"):
            cache.set(f"user:{user_id}", user)

    return jsonify(user)
```

Lan truyền ngữ cảnh truy vết (trace context) qua các dịch vụ:

```
import requests
from opentelemetry.propagate import inject

def call_downstream_service(url):
    headers = {}
    # Inject trace context into headers
    inject(headers)

    response = requests.get(url, headers=headers)
    return response.json()
```

Dịch vụ phía sau tiếp tục vết truy vết:

```
from opentelemetry.propagate import extract

@app.before_request
def extract_trace_context():
    # Extract trace context from headers
    ctx = extract(request.headers)
    # Automatic - OpenTelemetry uses context
```

Các Công Cụ Truy Vết

Jaeger:

- Mã nguồn mở
- Do Uber phát triển
- Dễ triển khai
- Phù hợp với hệ thống vừa và nhỏ

Zipkin:

- Mã nguồn mở
- Do Twitter phát triển
- Tương tự Jaeger
- Cộng đồng lớn

Datadog APM:

- Thương mại
- Đầy đủ tính năng
- Gắn đo đặc (*Instrumentation*) tự động
- Tích hợp với số liệu đo/log

AWS X-Ray:

- Gốc AWS
- Dễ tích hợp với AWS
- Trả tiền theo mức sử dụng

Truy Vết Tiết Lộ Điều Gì

1. Các phụ thuộc (*dependencies*) chậm

```
Service A: 10ms
Service B: 500ms ← Điểm nghẽn!
Service C: 5ms

Tổng: 515ms
```

2. Truy vấn N+1 (*N+1 queries*)

```
get_users: 10ms
  ↳ get_orders (người dùng 1): 50ms
  ↳ get_orders (người dùng 2): 50ms
  ↳ get_orders (người dùng 3): 50ms
  ...
```

3. Các lời gọi không cần thiết

```
get_profile:
  ↳ get_user
  ↳ get_preferences (không dùng tới!)
  ↳ get_settings (không dùng tới!)
```

4. Lỗi dây chuyền (*Cascade failures*)

```
Service A hết thời gian chờ (30s)
  ↳ Service B hết thời gian chờ (30s)
    ↳ Service C hết thời gian chờ (30s)

Tổng: thất bại sau 90s!
```

Chiến Lược Ghi Log (*Logging Strategy*) - Gỡ Lỗi Đến Từng Chi Tiết

Các Mức Log (*Log Levels*)

FATAL: Hệ thống không thể sử dụng được

ERROR: Các sự kiện lỗi, cần được chú ý

WARN: Các tình huống có khả năng gây hại

INFO: Các thông điệp mang tính thông tin

DEBUG: Thông tin chẩn đoán chi tiết

TRACE: Cực kỳ chi tiết (quá nhiều cho môi trường thực tế)

```
import logging

logging.fatal("Database connection lost!")
logging.error("Payment processing failed", extra={
    'user_id': user.id,
    'amount': payment.amount
})
logging.warning("Cache miss rate high: 40%")
logging.info("User logged in", extra={'user_id': user.id})
logging.debug("Query executed", extra={'query': sql, 'duration': 0.5})
```

Môi trường thực tế: INFO trở lên.

Môi trường thử nghiệm (Staging): DEBUG.

Môi trường phát triển (Development): TRACE.

Log Có Cấu Trúc (Structured Logging)

Tồi (không có cấu trúc):

```
logging.info(f"User {user_id} purchased {item_name} for ${price}")
```

Khó phân tích cú pháp, truy vấn, tổng hợp.

Tốt (có cấu trúc):

```
logging.info("purchase_completed", extra={
    'event': 'purchase',
    'user_id': user_id,
    'item_name': item_name,
    'price': price,
    'currency': 'USD'
})
```

Đầu ra JSON:

```
{
  "timestamp": "2025-02-15T10:00:00Z",
  "level": "INFO",
  "message": "purchase_completed",
  "user_id": 12345,
  "item_name": "iPhone",
  "price": 999,
  "currency": "USD"
}
```

Dễ truy vấn:

```
SELECT COUNT(*) FROM logs
WHERE event = 'purchase'
      AND price > 500
      AND timestamp > now() - interval '1 hour'
```

Nên Ghi Log Những Gì

Nên ghi log:

- Lỗi và ngoại lệ (*exceptions*)
- Các thay đổi trạng thái quan trọng
- Các sự kiện bảo mật (đăng nhập, thay đổi quyền)
- Các lời gọi API bên ngoài (yêu cầu (*request*)/phản hồi)
- Các sự kiện nghiệp vụ (mua hàng, đăng ký)

Không nên ghi log:

- Mật khẩu, token, thông tin định danh cá nhân (PII)
- Log gỡ lỗi tần suất cao (mọi truy vấn)

- Dữ liệu gửi kèm (Payloads) lớn (toàn bộ phản hồi)

ID Tương Quan (Correlation IDs)

Theo dõi yêu cầu xuyên suốt các dịch vụ.

```
import uuid

@app.before_request
def inject_request_id():
    request_id = request.headers.get('X-Request-ID') or str(uuid.uuid4())
    g.request_id = request_id

@app.after_request
def add_request_id(response):
    response.headers['X-Request-ID'] = g.request_id
    return response

# Log with request ID
logging.info("Processing request", extra={
    'request_id': g.request_id,
    'endpoint': request.endpoint
})
```

Tìm log theo request_id = thấy toàn bộ luồng của yêu cầu.

Tổng Hợp Log (Log Aggregation)

Hệ thống ghi log tập trung.

ELK Stack:

- Elasticsearch (lưu trữ (storage) + tìm kiếm)
- Logstash (thu thập + xử lý)
- Kibana (trực quan hoá)

Grafana Loki:

- Gọn nhẹ
- Tích hợp với Prometheus
- Phù hợp cho môi trường đám mây gốc (cloud-native)

Datadog Logs:

- Thương mại
- Nền tảng tích hợp
- Tự động phân tích cú pháp

CloudWatch Logs:

- Gốc AWS
- Thân thiện với kiến trúc phi máy chủ (*serverless*)
- Trả tiền theo GB

Đẩy log đi:

```
# Filebeat config
filebeat.inputs:
  - type: log
    paths:
      - /var/log/app/*.log
    fields:
      app: myapp
      env: production

output.elasticsearch:
  hosts: ["elasticsearch:9200"]
```

SLO, SLA, Ngân Sách Lỗi - Khung Tư Duy (*Framework*) Về Độ Tin Cậy

Định Nghĩa

SLI (Chỉ số mức dịch vụ (*Service Level Indicator*)):

Chỉ số đo được.

Ví dụ: độ trễ~p99 của API = 450ms

SLO (Mục tiêu mức dịch vụ (*Service Level Objective*)):

Mục tiêu đặt ra cho SLI.

Ví dụ: độ trễ~p99 của API < 500ms

SLA (Thoả thuận mức dịch vụ (Service Level Agreement)):

Hợp đồng với khách hàng, có hậu quả nếu không đạt.

Ví dụ: thời gian hoạt động 99.9%, nếu không sẽ hoàn lại 10% phí hàng tháng

Ngân sách lỗi (Error Budget):

Thời gian ngừng hoạt động (Downtime) được phép mà vẫn đạt SLO.

SLO: thời gian hoạt động 99.9%

Ngân sách lỗi: 0.1% thời gian ngừng hoạt động = 43 phút/tháng

Đặt SLO Tốt

Tiêu chí SMART:

Specific (Cụ thể): "độ trễ p99 < 500ms"

(không phải "thời gian phản hồi nhanh")

Measurable (Đo lường được): Có thể theo dõi bằng số liệu đo

Achievable (Khả thi): Không phải "100% thời gian hoạt động" (bất khả thi)

Relevant (Liên quan): Ảnh hưởng tới trải nghiệm người dùng

Time-bound (Có thời hạn): "Trong cửa sổ 30 ngày"

Ví dụ về SLO:

```
# Availability SLO
availability:
  sli: successful_requests / total_requests
  target: 99.9%
  window: 30 days

# Latency SLO
latency:
  sli: p99_latency
  target: 500ms
  window: 5 minutes

# Throughput SLO
throughput:
  sli: requests_per_second
  target: "> 1000 req/s"
  window: 1 minute
```

Ngân Sách Lỗi Trong Thực Tế

Ngân sách lỗi = số lỗi được phép.

SLO: tỷ lệ thành công 99.9%
Tổng số yêu cầu/tháng: 10M
Ngân sách lỗi: $0.1\% * 10M = 10,000$ yêu cầu thất bại

Chi tiêu ngân sách lỗi cho:

- Các lần triển khai (*deployments*) (rủi ro có kiểm soát)
- Các thử nghiệm (tính năng mới)
- Các sự cố (*incidents*) (không thể tránh)

Theo dõi tốc độ tiêu hao (*burn rate*):

```
def calculate_error_budget():
    total_requests = get_total_requests(window='30d')
    failed_requests = get_failed_requests(window='30d')

    error_rate = failed_requests / total_requests
    error_budget = 0.001 # 99.9% SLO

    budget_remaining = error_budget - error_rate
    budget_consumed_percent = (error_rate / error_budget) * 100

    return {
        'remaining': budget_remaining,
        'consumed_percent': budget_consumed_percent
    }
```

Cảnh báo dựa trên tốc độ tiêu hao ngân sách:

```
alert: ErrorBudgetBurnRateHigh
expr: |
    error_rate > (error_budget * 0.1) # Burning 10% of budget/hour
for: 1h
annotations:
    summary: Burning error budget too fast
    action: Stop deployments, investigate issues
```

Khung ra quyết định (Decision framework):

Ngân sách lỗi còn lại: 80% 

→ An toàn để triển khai tính năng mới

Ngân sách lỗi còn lại: 20% 

→ Tạm dừng các thay đổi rủi ro, tập trung vào sự ổn định

Ngân sách lỗi đã cạn: 0% 

→ Đóng băng mọi lần triển khai

→ Chỉ sửa các vấn đề về độ tin cậy

Lợi Ích

1. Định lượng độ tin cậy

Thước đo khách quan, không phải cảm giác chủ quan kiểu "thấy có vẻ ổn định".

2. Cân bằng giữa đổi mới và ổn định

Ngân sách lỗi cho phép chấp nhận rủi ro có kiểm soát.

3. Ưu tiên công việc

Ngân sách thấp? Tập trung vào độ tin cậy.

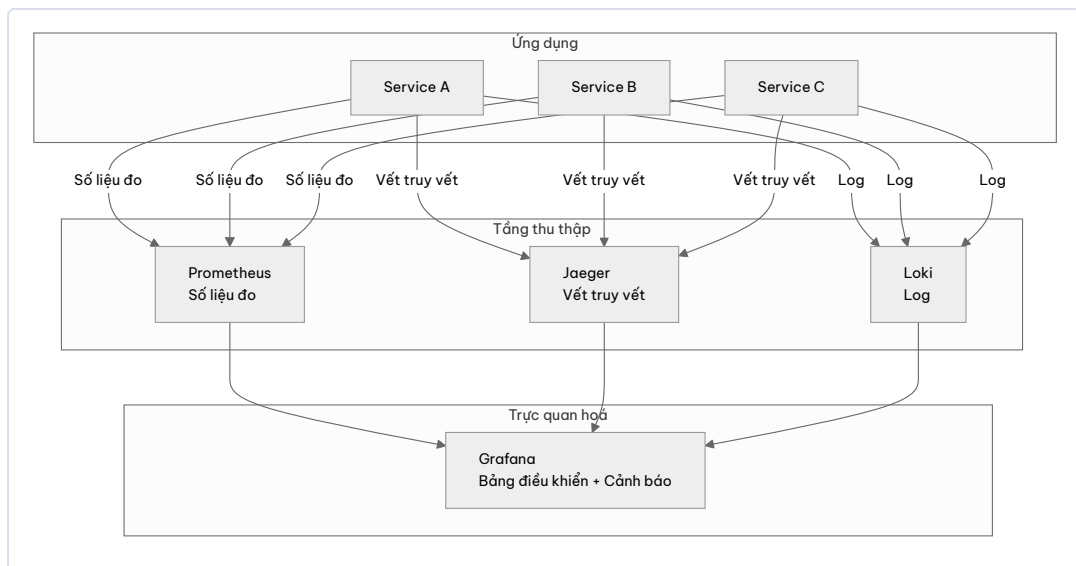
Ngân sách cao? Tập trung vào tính năng.

4. Gắn kết các nhóm (teams)

SRE và nhóm phát triển thống nhất về các mục tiêu độ tin cậy.

Ví Dụ Về Bộ Công Cụ Giám Sát

Thiết lập khả năng quan sát hoàn chỉnh:



Góc nhìn hợp nhất trong Grafana:

- Biểu đồ số liệu đo
- Trình khám phá log
- Trình xem vết truy vết
- Trình quản lý cảnh báo

Khuyến Nghị Thực Tế

1. Bắt Đầu Với Tín Hiệu Vàng

Đừng cố giám sát mọi thứ.

```
# Minimum viable monitoring
from prometheus_client import Counter, Histogram

requests_total = Counter('api_requests_total', 'Total requests')
request_duration = Histogram('api_request_duration_seconds', 'Request
duration')
errors_total = Counter('api_errors_total', 'Total errors')

@app.route('/api/endpoint')
def endpoint():
    requests_total.inc()

    with request_duration.time():
        try:
            result = process()
            return result
        except Exception as e:
            errors_total.inc()
            raise
```

4 chỉ số này bao quát phần lớn các vấn đề.

2. Xây Dựng Bảng Điều Khiển (Dashboards)

Mỗi dịch vụ (service) một bảng điều khiển (dashboard):

```
Bảng điều khiển dịch vụ
├─ Tín hiệu vàng (hàng trên cùng)
│  ├─ Tần suất yêu cầu
│  ├─ Tỷ lệ lỗi
│  ├─ Độ trễ~p99
│  └─ Độ bão hòa
├─ Số liệu chi tiết (ở giữa)
│  ├─ Độ trễ~theo từng endpoint
│  ├─ Thời gian truy vấn CSDL
│  ├─ Tỷ lệ trúng bộ nhớ đệm
│  └─ Độ sâu hàng đợi
└─ Tài nguyên hệ thống (dưới cùng)
   ├─ Mức sử dụng CPU
   ├─ Mức sử dụng bộ nhớ
   └─ Disk I/O
```

Chuẩn hoá giữa các dịch vụ - dễ so sánh hơn.

3. Dùng Thư Viện Gắn Đo Đặc (*Instrument*) Sẵn Có

Đừng phát minh lại bánh xe.

```
# Auto-instrument Flask
from prometheus_flask_exporter import PrometheusMetrics
metrics = PrometheusMetrics(app)

# Auto-instrument SQLAlchemy
from prometheus_sqlalchemy import PrometheusMetrics
PrometheusMetrics(engine)

# Auto-instrument Redis
from prometheus_redis import RedisMetrics
RedisMetrics(redis_client)
```

Ít mã hơn, độ bao phủ nhiều hơn.

4. Thiết Lập Giám Sát Tổng Hợp (*Synthetic Monitoring*)

Kiểm tra chủ động.

```
# Cron job - check critical paths
def health_check():
    checks = [
        ('api', check_api_health),
        ('database', check_db_connection),
        ('cache', check_redis_connection),
        ('payments', check_payment_gateway),
    ]

    for name, check_func in checks:
        try:
            check_func()
            metrics.gauge(f'health.{name}', 1)
        except Exception:
            metrics.gauge(f'health.{name}', 0)
            alert(f'{name} health check failed')
```

Phát hiện vấn đề trước khi người dùng gặp phải.

5. Luyện Tập Gỡ Lỗi (Debugging)

Mô phỏng sự cố (incident):

```
# Deliberately inject latency
@app.route('/api/slow')
def slow_endpoint():
    if random.random() < 0.05: # 5% requests
        time.sleep(5) # 5 second delay
    return process_request()
```

Luyện tập:

1. Phát hiện độ trễ tăng đột biến (*spike*) trên bảng điều khiển
2. Kiểm tra các vết - tìm ra điểm cuối chậm
3. Kiểm tra log - hiểu nguyên nhân
4. Sửa vấn đề
5. Xác minh sự cải thiện

Tạo phản xạ cơ bắp cho các sự cố thật.

Mô Hình Tư Duy: Nếu Không Đo Được, Bạn Không Thể Mở Rộng

Nhận thức cốt lõi:

Các quyết định mở rộng (scaling) đòi hỏi dữ liệu. Không đo lường, bạn đang bay mù.

Không có số liệu đo:

"Hệ thống chậm" → Đoán giải pháp → Có thể chẳng giúp gì

Có số liệu đo:

"Độ trễ~p99 là 2s" → Kiểm tra vết truy vết → Điểm nghẽn ở CSDL
→ Thêm bản sao chỉ đọc → Độ trễ~p99 còn 200ms 

Đo lường → Hiểu → Tối ưu → Đo lại.

Quyết định của kiến trúc sư (architect) dựa trên dữ liệu:

Có nên thêm bộ nhớ đệm?

→ Kiểm tra: Tỷ lệ trùng bộ nhớ đệm sẽ là X%

→ Tác động: Giảm tải CSDL Y%

→ Quyết định: Có/Không dựa trên con số

Có nên phân mảnh CSDL?

→ Kiểm tra: Tải hiện tại Z req/s

→ Năng lực: Một CSDL chịu được W req/s

→ Quyết định: Phân mảnh khi $Z > 0.8W$

Kiến trúc sư giỏi đo lường trước, tối ưu sau.

Những Điểm Cốt Lõi

1. Giám sát Tín hiệu vàng: Độ trễ, Lưu lượng, Lỗi, Độ bão hòa

4 chỉ số này bao quát phần lớn các vấn đề trong môi trường thực tế.

2. Bách phân vị > Giá trị trung bình

p95, p99 hé lộ trải nghiệm người dùng trong trường hợp xấu nhất. Hãy tối ưu cho độ trễ đuôi (*tail latency*).

3. Cảnh báo dựa trên triệu chứng, không phải nguyên nhân

Cảnh báo dựa trên tác động tới người dùng, không phải mức sử dụng tài nguyên.

4. Truy vết phân tán là tối quan trọng với vi dịch vụ

Hiểu luồng đi của yêu cầu, tìm ra điểm nghẽn (*bottlenecks*) xuyên suốt các dịch vụ.

5. Log có cấu trúc giúp phân tích được

Log dạng JSON = dữ liệu truy vấn được, không chỉ là văn bản.

6. Đặt SLO dựa trên kỳ vọng của người dùng

Xác định các mục tiêu độ tin cậy, theo dõi ngân sách lỗi.

7. ID tương quan giúp theo dõi yêu cầu

Gỡ lỗi toàn bộ luồng của yêu cầu xuyên suốt các dịch vụ.

8. Ngân sách lỗi cân bằng giữa độ tin cậy và đổi mới

Chấp nhận rủi ro có kiểm soát dựa trên dữ liệu.

9. Bắt đầu đơn giản, chỉ thêm độ phức tạp (*complexity*) khi cần

Tín hiệu vàng trước, giám sát nâng cao sau.

10. Luyện tập gỡ lỗi trước khi có sự cố

Mô phỏng sự cố (*failures*), xây dựng phản xạ cơ bắp.

Hãy nhớ: Bạn không thể cải thiện thứ mà bạn không đo được. Giám sát tốt không phải là tùy chọn - nó là nền tảng của những hệ thống đáng tin cậy và có khả năng mở rộng. Hãy đầu tư vào khả năng quan sát từ sớm, đừng đợi đến khi gặp sự cố trên môi trường thực tế.

PHẦN 5

Các Mẫu Kiến Trúc Thực Tế

Áp dụng kiến thức hệ phân tán và hiệu năng vào kiến trúc vận hành thực tế theo từng bối cảnh.

1) Giới Thiệu Phần

Hệ thống thực tế khác với hệ thống trong sách giáo khoa.

Kiến trúc vận hành thực tế (*production*) không đứng yên: nó phải tiến hoá theo lưu lượng (*traffic*), theo sản phẩm và theo các giới hạn vận hành. Không có một kiến trúc duy nhất đúng cho mọi bài toán.

Phần này tập trung vào tư duy thiết kế theo ngữ cảnh: chọn mẫu (*pattern*) phù hợp, chấp nhận sự đánh đổi (*trade-off*) một cách rõ ràng, và xây dựng hệ thống có thể sống lâu trong môi trường thực tế.

2) Vì Sao Phần Này Tồn Tại

Sau khi nắm vững hệ thống phân tán (*distributed systems*) và khả năng mở rộng (*scalability*), bạn cần bước tiếp theo: biến kiến thức đó thành quyết định kiến trúc thực tế.

Phần này tồn tại để lấp khoảng cách giữa lý thuyết và thực tế vận hành:

- Từ hiểu từng thành phần (*component*) -> phối hợp thành kiến trúc theo vòng đời hệ thống.
- Từ tối ưu cục bộ -> thiết kế theo mẫu lưu lượng (*traffic pattern*) thật.
- Từ mô hình chung chung -> quyết định theo lĩnh vực (*domain*), chi phí, tốc độ tăng trưởng và hành vi người dùng.

3) Mục Tiêu Học Tập

- Hiểu cách kiến trúc tiến hoá theo từng giai đoạn của sản phẩm.
- Thiết kế hệ thống theo mẫu truy cập (*access pattern*) thay vì theo sở thích công nghệ.
- Lập luận quyết định kiến trúc theo phân bố lưu lượng (*traffic distribution*) thực tế.
- Chọn mẫu phù hợp với hành vi hệ thống và ràng buộc (*constraint*) vận hành.
- Áp dụng kiến thức hệ thống phân tán vào các bài toán thực tế.

- Thiết kế kiến trúc thực dụng, sẵn sàng cho tăng trưởng.

4) Bạn Sẽ Học Gì Trong Phần Này

- **Chiến lược tiến hoá kiến trúc:** bắt đầu đơn giản, nâng cấp có kiểm soát.
- **Tư duy thiết kế hệ thống thiên về đọc** (*read-heavy*): tối ưu cho khối lượng công việc (*workload*) mà thao tác đọc chiếm đa số.
- **Kiến trúc phân phát** (*fanout*): mẫu cốt lõi cho các hệ thống bảng tin (*feed*)/mạng xã hội.
- **Thiết kế dẫn dắt bởi mẫu lưu lượng:** kiến trúc theo hành vi truy cập thực tế.
- **Đánh đổi** (*Trade-offs*) **trong vận hành thực tế:** cân bằng độ trễ (*latency*), chi phí, độ phức tạp và khả năng vận hành (*operability*).
- **Lập luận trên hệ thống thật:** ra quyết định dựa trên bối cảnh sản phẩm và dữ liệu vận hành.

5) Tổng Quan Các Bài Học

Bài 1 – Tư Duy Tiến Hoá Kiến Trúc – Bắt Đầu Đơn Giản, Mở Rộng Dần Dần

- **Mô tả:** học cách phát triển kiến trúc theo từng giai đoạn thay vì thiết kế quá mức ngay từ đầu.
- **Bạn nhận được:** tư duy tránh phức tạp hoá sớm (*premature complexity*) và biết khi nào cần nâng cấp kiến trúc.

Bài 2 – Hệ Thống Thiên Về Đọc – Thiết Kế Cho 99% Thao Tác Đọc

- **Mô tả:** thiết kế hệ thống cho khối lượng công việc mà thao tác đọc áp đảo, nơi độ trễ và chiến lược lưu đệm (*cache strategy*) quyết định trải nghiệm.
- **Bạn nhận được:** khả năng tối ưu kiến trúc theo mẫu truy cập chủ đạo (*dominant access pattern*).

Bài 3 – Hệ Thống Mạng Xã Hội / Bảng Tin – Kiến Trúc Phân Phát

- **Mô tả:** phân tích kiến trúc bảng tin trong môi trường lưu lượng lớn và cập nhật liên tục.

- **Bạn nhận được:** nền tảng về sự đánh đổi giữa phân phát khi ghi (*fanout-on-write*) và phân phát khi đọc (*fanout-on-read*), cùng các thách thức mở rộng thực tế.

6) Kết Quả Sau Khi Hoàn Thành Phần 5

Bạn có thể:

- Thiết kế kiến trúc vận hành thực tế theo bối cảnh cụ thể thay vì theo khuôn mẫu cố định.
- Lập luận về lộ trình tiến hoá kiến trúc theo đà tăng trưởng của sản phẩm.
- Chọn kiến trúc theo khối lượng công việc và mẫu lưu lượng.
- Đánh giá sự đánh đổi thực tế giữa hiệu năng, chi phí và độ phức tạp vận hành.
- Áp dụng kiến thức hệ thống phân tán vào thiết kế hệ thống thật.
- Tư duy như một kiến trúc sư hệ thống vận hành thực tế khi ra quyết định.

7) Chuẩn Bị Cho Phần 6

Phần 5 giúp bạn chuyển từ "biết các mẫu" sang "ra quyết định theo bối cảnh".

Sang **Phần 6 — Làm Chủ Thiết Kế Hệ Thống**, bạn sẽ đi từ cấp độ mẫu lên cấp độ khung tư duy (*framework*):

- tư duy ra quyết định kiến trúc ở mức nâng cao,
- kỹ năng thiết kế hệ thống ở mức phỏng vấn chuyên sâu,
- tư duy kiến trúc sư (*architect mindset*) cho các bài toán mở,
- và năng lực giải quyết vấn đề thực tế với lập luận rõ ràng.

Tư Duy Tiến Hoá Kiến Trúc (*Architecture Evolution*) - Bắt Đầu Đơn Giản, Mở Rộng Dần Dần

Có một sai lầm phổ biến mà các lập trình viên (*developer*) hay mắc phải khi học thiết kế hệ thống (*system design*):

Học về vi dịch vụ (*microservices*), **Kubernetes**, **kiến trúc hướng sự kiện** (*event-driven architecture*) → **Nghĩ rằng mọi dự án đều cần những thứ này ngay từ đầu.**

Đây là thiết kế thừa (*over-engineering*) - và nó giết chết các công ty khởi nghiệp (*startups*).

Bạn có biết:

- **Instagram:** Một ứng dụng Django nguyên khối (*monolith*) khi được Facebook mua lại với giá 1 tỷ đô la
- **Shopify:** Một ứng dụng Rails nguyên khối cho đến khi có hàng triệu nhà bán hàng
- **Stack Overflow:** Một ứng dụng .NET nguyên khối phục vụ hàng tỷ yêu cầu (*requests*) mỗi tháng
- **GitHub:** Một ứng dụng Rails nguyên khối cho đến khi có hơn 40 triệu người dùng (*users*)

Họ không bắt đầu với vi dịch vụ. Họ đã tiến hoá từ đơn giản đến phức tạp khi THỰC SỰ CẦN THIẾT.

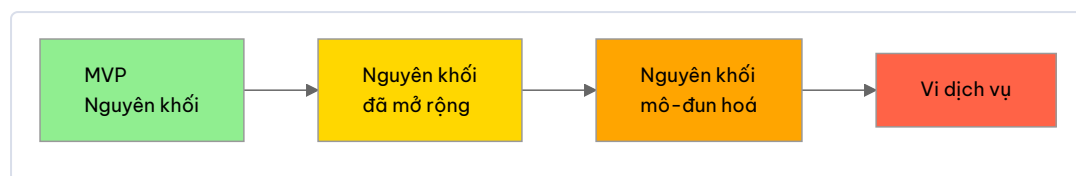
Bài này - có lẽ là bài quan trọng nhất trong Phần 5 - dạy bạn **tư duy tiến hoá kiến trúc**:

- Khi nào dùng hệ thống nguyên khối, khi nào chuyển đổi (*migrate*)
- Các tín hiệu cần mở rộng (*scale triggers*) trong thực tế
- Các phản mẫu (*anti-patterns*) cần tránh
- Tư duy đánh đổi (*trade-off*)

Mô hình tư duy (*Mental model*): "Bắt đầu đơn giản, tiến hoá dần dần" - chứ không phải "Bắt đầu phức tạp vì sợ sau này phải mở rộng (*scale*)".

Các Giai Đoạn Tiến Hoá Kiến Trúc

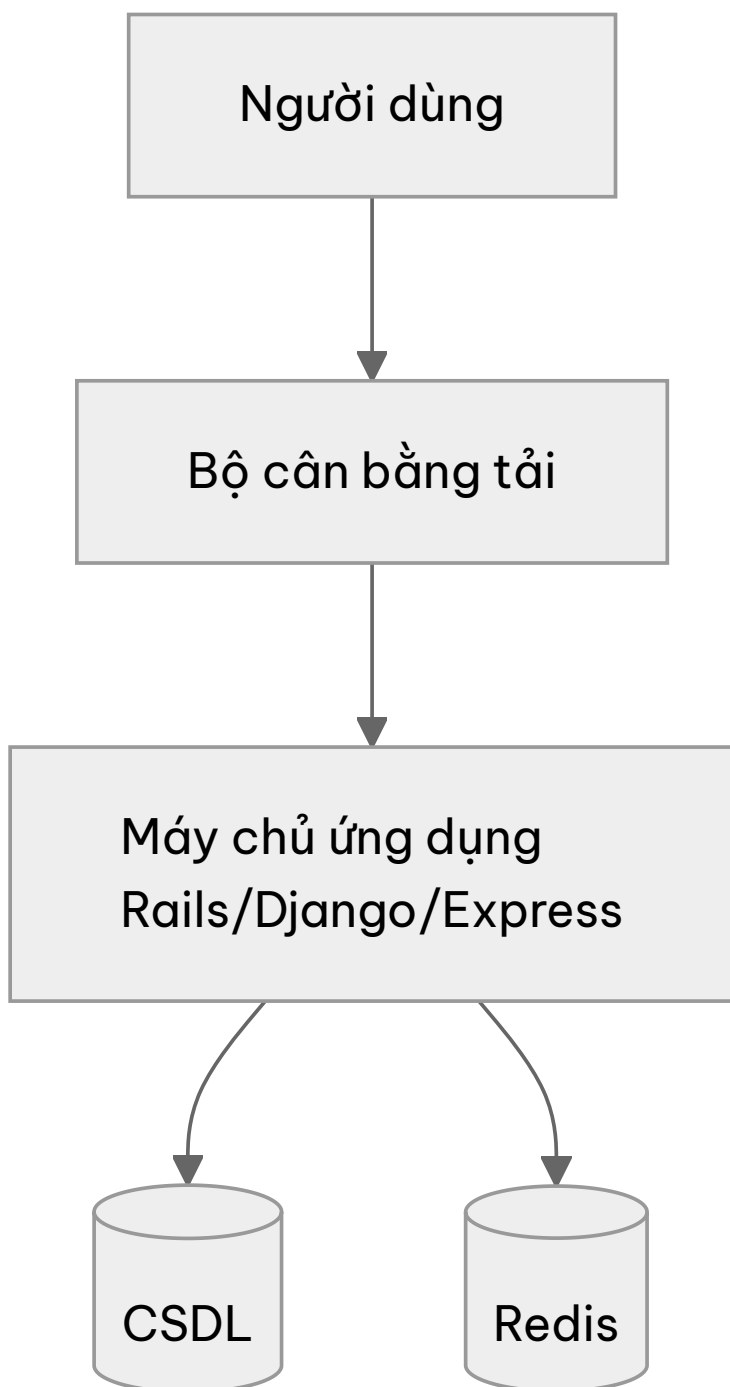
Kiến trúc (*Architecture*) không phải là lựa chọn nhị phân (nguyên khối hay vi dịch vụ). Nó là một dải liên tục.



Mỗi giai đoạn phù hợp với một quy mô và quy mô nhóm khác nhau.

Giai Đoạn 1: MVP Nguyên Khối (0-10 Nghìn Người Dùng)

Một mã nguồn (*codebase*), một cơ sở dữ liệu (*database*), một lần triển khai (*deployment*).



Đặc điểm:

Ưu điểm:

- Tốc độ phát triển nhanh nhất
- Triển khai đơn giản (git push)
- Dễ gỡ lỗi (*debugging*) (chỉ 1 mã nguồn)
- Không có độ phức tạp (*complexity*) của hệ thống phân tán (*distributed system*)
- Chi phí phụ (*overhead*) vận hành thấp

Nhược điểm:

- Điểm lỗi duy nhất (*Single point of failure*)
- Toàn bộ mã liên kết chặt (*couples*) với nhau
- Khó mở rộng riêng từng phần
- Bị giới hạn trong một ngôn ngữ/framework

Ví dụ bộ công nghệ (*tech stack*):

Frontend: React/Vue (được phục vụ bởi cùng máy chủ)
Backend: Rails/Django/Laravel/Express
Cơ sở dữ liệu: PostgreSQL/MySQL
Bộ nhớ đệm: Redis
Triển khai: Heroku/Railway/một VPS duy nhất

Khi nào dùng:

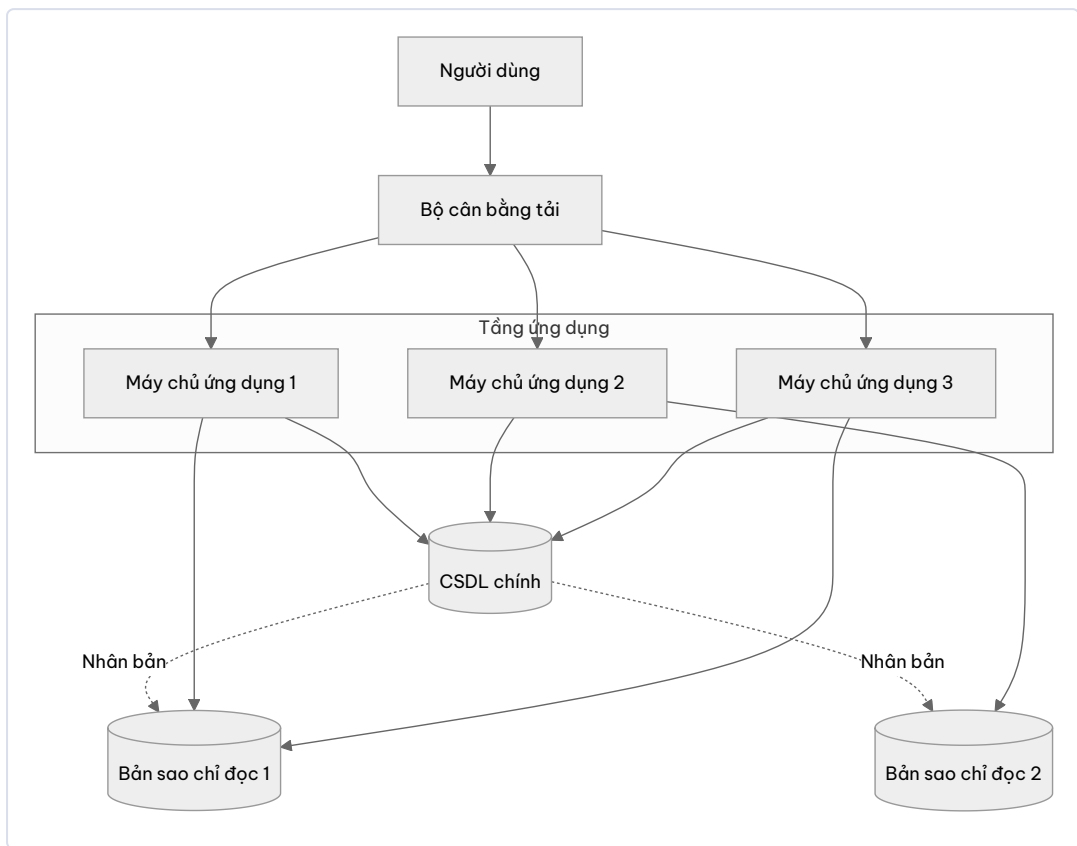
- Trước khi đạt độ phù hợp sản phẩm – thị trường (*product-market-fit*)
- Công ty khởi nghiệp dưới 10 kỹ sư (*engineers*)
- Lưu lượng (*Traffic*) dưới 100 req/s
- Đang chứng minh ý tưởng

Ví dụ thực tế:

- Basecamp (vẫn là nguyên khối ở quy mô khổng lồ)
- Shopify (bắt đầu với nguyên khối)
- GitHub (bắt đầu với nguyên khối)

Giai Đoạn 2: Nguyên Khối Đã Mở Rộng (10 Nghìn – 100 Nghìn Người Dùng)

Cùng một mã nguồn, thêm máy chủ (*servers*) + tối ưu cơ sở dữ liệu.



Thay đổi so với Giai đoạn 1:

Hạ tầng (Infrastructure):

- Mở rộng theo chiều ngang (*Horizontal scaling*) (nhiều máy chủ ứng dụng)
- Bản sao chỉ đọc (*read replicas*) cho cơ sở dữ liệu
- CDN cho tài nguyên tĩnh (*static assets*)
- Cụm (*cluster*) Redis cho phiên (*sessions*)

Tối ưu mã:

- Tối ưu truy vấn (*query*)
- Sửa lỗi N+1
- Tầng lưu đệm (*caching*)
- Tác vụ nền (*Background jobs*) (Sidekiq/Celery)

Vẫn là nguyên khối, nhưng đã được mở rộng.

Khi nào chuyển sang giai đoạn này:

- Lưu lượng 100-1000 req/s
- CPU của một máy chủ liên tục trên 70%
- Thời gian phản hồi (*Response time*) ngày càng xấu đi
- Nhóm 10-30 kỹ sư

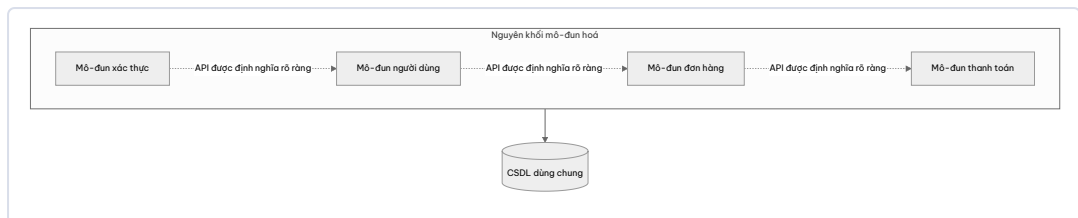
Chi phí (Cost) so với độ phức tạp:

Giai đoạn 1: \$50-200/tháng
Giai đoạn 2: \$500-2000/tháng
Độ phức tạp: +20%

Một sự đánh đổi hợp lý cho tăng trưởng.

Giai Đoạn 3: Nguyên Khối Mô-đun Hoá (*Modular Monolith*) (100 Nghìn - 1 Triệu Người Dùng)

Một lần triển khai duy nhất, nhưng mã được tổ chức thành các mô-đun (*modules*).



Cấu trúc mô-đun:

```
/app
  /modules
    /auth
      - controllers/
      - services/
      - models/
      - api.ts (giao diện công khai)
    /users
      - controllers/
      - services/
      - models/
      - api.ts
    /orders
      - controllers/
      - services/
      - models/
      - api.ts
```

Quy tắc:

- Các mô-đun giao tiếp với nhau qua các API được định nghĩa rõ ràng
- Không truy cập trực tiếp cơ sở dữ liệu của mô-đun khác
- Ranh giới rõ ràng
- Chuẩn bị cho vi dịch vụ (nếu cần)

Lợi ích:

- Quyền sở hữu theo nhóm (*Team ownership*) (nhóm A sở hữu mô-đun xác thực)
- Kiểm thử dễ hơn (các mô-đun được cô lập)
- Có thể tách thành vi dịch vụ (*microservice*) về sau
- Vẫn giữ được sự đơn giản của nguyên khối

Khi nào dùng:

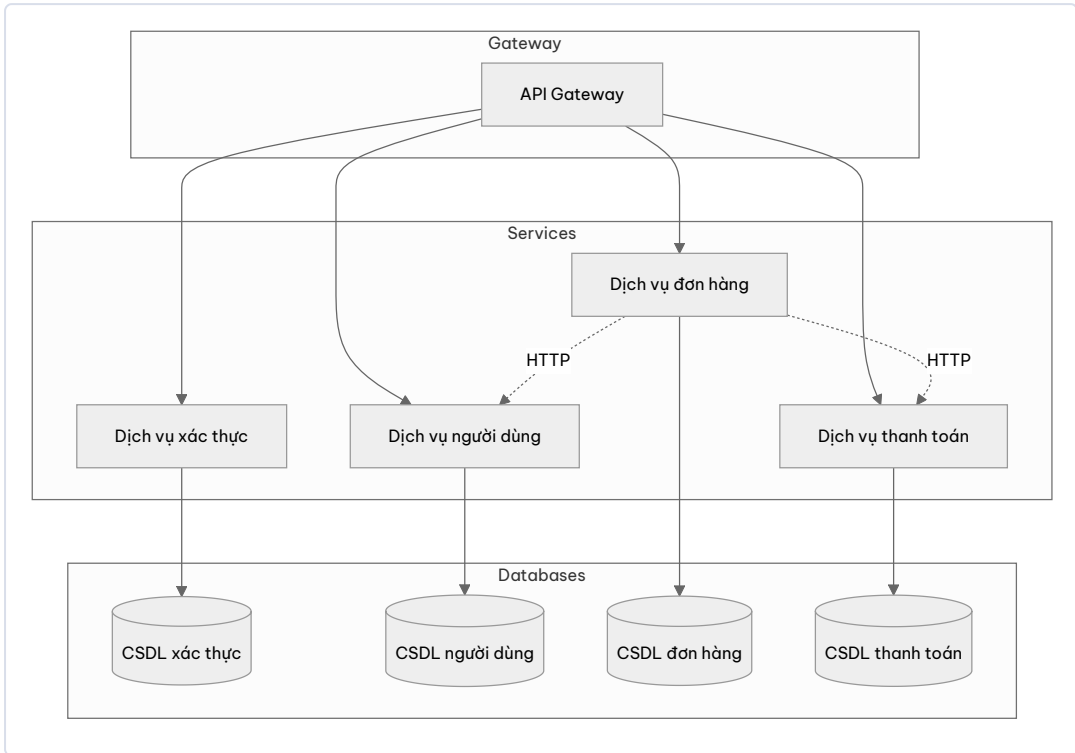
- Nhóm 30-100 kỹ sư
- Logic nghiệp vụ (*domain logic*) phức tạp
- Chuẩn bị cho khả năng chuyển sang vi dịch vụ
- Muốn các nhóm tự chủ nhưng chưa sẵn sàng cho vi dịch vụ

Ví dụ thực tế:

- Shopify (nguyên khối mô-đun hoá trước khi chuyển sang vi dịch vụ)
- Basecamp (kiến trúc hiện tại)

Giai Đoạn 4: Vi Dịch Vụ (Trên 1 Triệu Người Dùng)

Nhiều dịch vụ (*services*) độc lập, mỗi dịch vụ có cơ sở dữ liệu riêng.



Đặc điểm:

Ưu điểm:

- Mở rộng độc lập
- Triển khai độc lập
- Linh hoạt về công nghệ
- Các nhóm tự chủ
- Cô lập lỗi (*Fault isolation*)

Nhược điểm:

- Độ phức tạp của hệ thống phân tán
- Độ trễ (*latency*) mạng
- Thách thức về tính nhất quán (*consistency*) dữ liệu
- Chi phí phụ vận hành

- Kiểm thử phức tạp
- Triển khai phức tạp

Khi nào chuyển đổi:

Chuyển đổi khi:

- Nhóm trên 100 kỹ sư
- Ranh giới miền nghiệp vụ (*domain*) rõ ràng
- Mỗi mô-đun có nhu cầu mở rộng khác nhau
- Hệ thống nguyên khối trở thành điểm nghẽn (*bottleneck*) về mặt tổ chức
- Có đủ ngân sách + chuyên môn để gánh độ phức tạp

KHÔNG chuyển đổi khi:

- Nhóm dưới 30 kỹ sư
- Ranh giới miền nghiệp vụ còn mơ hồ
- Tối ưu quá sớm (*Premature optimization*)
- Chưa có chuyên môn DevOps
- Ngân sách hạn chế

Bước nhảy chi phí:

Nguyên khối mô-đun hoá: \$2k-5k/tháng

Vi dịch vụ: \$10k-50k+/tháng

Hạ tầng + nhóm DevOps + giám sát + điều phối (*orchestration*)

Bước nhảy độ phức tạp: gấp 10 lần.

Tín Hiệu Cần Mở Rộng - Khi Nào Nên Tiến Hoá?

Đừng tiến hoá vì xu hướng. Hãy tiến hoá vì những điểm đau (*pain points*) **thực sự.**

Tín Hiệu 1: Điểm Nghẽn Hiệu Năng (*Performance Bottleneck*)

Triệu chứng:

Một thành phần duy nhất gây nghẽn:

- Xử lý ảnh: 80% CPU
- Phần còn lại của ứng dụng: 20% CPU

Giải pháp: Tách riêng dịch vụ xử lý ảnh.

Trước: Hệ thống nguyên khối xử lý ảnh

Sau: Dịch vụ ảnh riêng biệt

- Mở rộng độc lập
- Dùng máy chủ có GPU
- Không lãng phí tài nguyên cho các phần khác

Tiêu chí quyết định:

- Điểm nghẽn được cô lập rõ ràng
- Nhu cầu tài nguyên khác nhau (CPU hay bộ nhớ hay GPU)
- Việc tách riêng giải quyết được vấn đề

Tín Hiệu 2: Điểm Nghẽn Khi Mở Rộng (Scaling) Nhóm

Triệu chứng:

Quy mô nhóm: 50 kỹ sư

Tần suất triển khai: 1 lần/tuần

Xung đột khi merge: Hàng ngày

Chờ triển khai: Hàng giờ liền

Mọi người giẫm chân lên nhau.

Giải pháp: Chia tách mã nguồn theo quyền sở hữu của từng nhóm.

Nhóm Xác thực: Dịch vụ xác thực

Nhóm Thanh toán: Dịch vụ thanh toán

Nhóm Đơn hàng: Dịch vụ đơn hàng

Mỗi nhóm tự triển khai độc lập

Tiêu chí quyết định:

- Nhóm trên 30-50 kỹ sư
- Cơ cấu tổ chức rõ ràng
- Các nhóm phụ trách những miền nghiệp vụ (*domains*) tách biệt

Tín Hiệu 3: Yêu Cầu (Requirements) Mở Rộng Khác Nhau

Triệu chứng:

Dịch vụ đơn hàng: 10 req/s
Xử lý ảnh: 1000 req/s
Thanh toán: 50 req/s

Mở rộng toàn bộ hệ thống nguyên khối sẽ lãng phí tài nguyên.

Giải pháp: Tách riêng dịch vụ có lưu lượng cao.

Xử lý ảnh → Dịch vụ riêng
→ Mở rộng lên 20 phiên bản (instance)
Phần còn lại của ứng dụng → 3 phiên bản

Tiết kiệm chi phí + hiệu năng tốt hơn

Tín Hiệu 4: Giới Hạn Về Công Nghệ

Triệu chứng:

Ứng dụng chính: Ruby (tốt cho CRUD)
Mô hình ML: Cần Python
Thời gian thực: Cần Node.js/Go

Giải pháp: Kiến trúc đa ngôn ngữ (Polyglot architecture).

Ứng dụng chính: Ruby on Rails
Dịch vụ ML: Python + TensorFlow
Dịch vụ WebSocket: Node.js

Công cụ khác nhau cho những công việc khác nhau.

Những Thứ Không Phải Tín Hiệu (Không Nên Chuyển Đổi)

"Có thể sau này chúng ta sẽ cần mở rộng"

Tối ưu quá sớm. YAGNI (You Aren't Gonna Need It - Bạn sẽ không cần đến nó đâu).

"Vi dịch vụ là thực hành tốt nhất" (best practice)"

Thực hành tốt nhất cho ai? Netflix? Bạn đâu phải Netflix.

"Phát triển hướng CV (Resume-driven development)"

Muốn học Kubernetes không phải là lý do chính đáng để làm dự án phức tạp lên.

"Hệ thống nguyên khối trông lộn xộn quá"

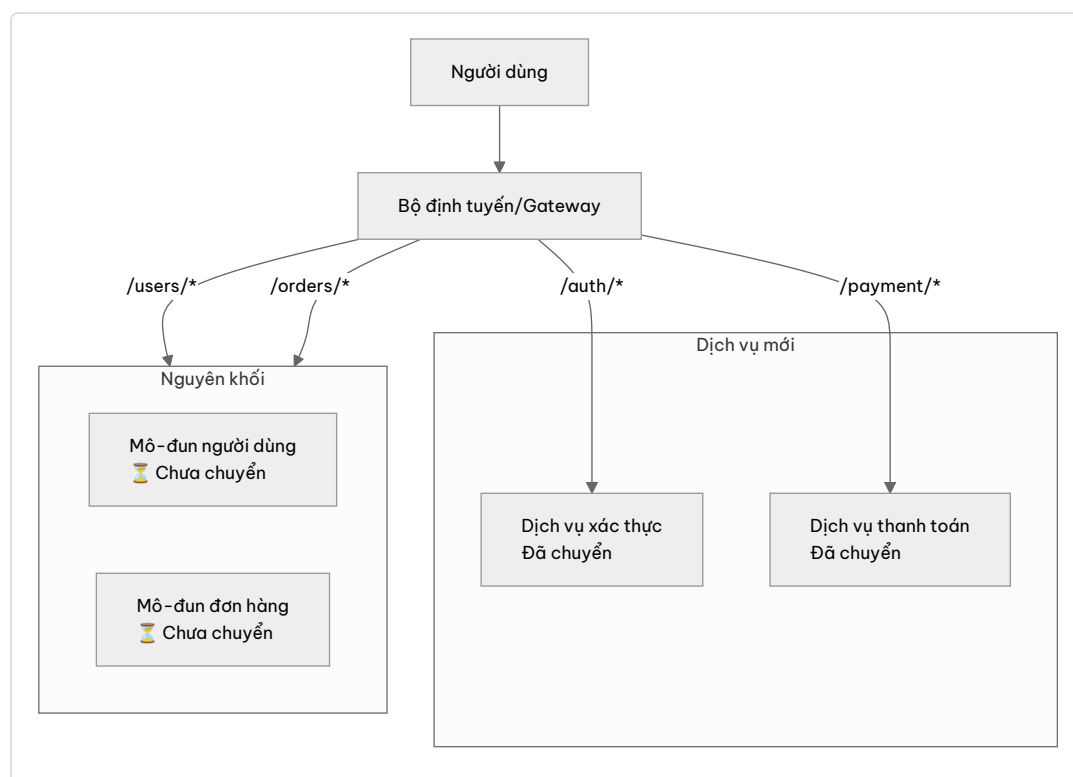
Hãy dọn dẹp cấu trúc mã. Dùng nguyên khối mô-đun hoá. Đừng nhảy ngay sang vi dịch vụ.

Chiến Lược Chuyển Đổi (Migration) - Tiến Hoá Dần Dần

KHÔNG BAO GIỜ viết lại toàn bộ hệ thống.

Strangler Fig Pattern (Mẫu Cây Đa Bóp Nhẹt)

Thay thế dần hệ thống nguyên khối, từng phần một.



Quy trình:

Giai đoạn 1: Chọn mô-đun để tách ra (mô-đun ít liên kết (coupled) nhất)

Ứng viên: Dịch vụ thanh toán
Phụ thuộc: Ít
Tác động: Giá trị cao

Giai đoạn 2: Xây dựng dịch vụ mới

Tái hiện chức năng
Thêm kiểm thử
Đạt tương đương về tính năng

Giai đoạn 3: Chuyển hướng lưu lượng dần dần

Tuần 1: 5% lưu lượng sang dịch vụ mới
Tuần 2: 25%
Tuần 3: 50%
Tuần 4: 100%

Giai đoạn 4: Loại bỏ dần mã cũ

Giám sát lỗi
Gỡ khỏi hệ thống nguyên khối
Ăn mừng 🎉

Giai đoạn 5: Lặp lại với mô-đun tiếp theo

Lợi ích:

- Rủi ro thấp (triển khai dần (*gradual rollout*))
- Dễ dàng hoàn tác (*rollback*)
- Rút kinh nghiệm sau mỗi lần chuyển đổi
- Hoạt động kinh doanh không bị gián đoạn

Phản Mẫu (Anti-Pattern): Viết Lại Toàn Bộ Một Lần (*Big Bang Rewrite*)

KHÔNG LÀM:

1. Ngừng phát triển tính năng
2. Viết lại mọi thứ thành vi dịch vụ
3. Triển khai tất cả một lần (*big bang*)
4. Cầu mong nó chạy được

Tại sao thất bại:

- Đóng băng phát triển 6-12 tháng
- Doanh nghiệp đánh mất lợi thế cạnh tranh
- Yêu cầu thay đổi trong lúc đang viết lại
- Lỗi (*Bugs*) trong hệ thống mới
- Không thể hoàn tác dễ dàng

Những thất bại nổi tiếng:

- Netscape (viết lại trình duyệt, thua Internet Explorer)
- Friendster (viết lại, thua Facebook)

Quy tắc: Hãy tiến hoá, đừng viết lại.

Các Phản Mẫu – Tránh Những Sai Lầm Này

Phản Mẫu 1: Vi Dịch Vụ Quá Sớm

Kịch bản:

Công ty khởi nghiệp: 3 kỹ sư, 0 người dùng
 Kiến trúc: 15 vi dịch vụ, Kubernetes, Kafka

Vấn đề:

- Tốc độ phát triển: chậm như rùa
- Gỡ lỗi: cơn ác mộng
- Chi phí phụ vận hành: quá tải
- Độ phù hợp sản phẩm – thị trường: không bao giờ tìm thấy

Cái giá phải trả:

- 3 tháng dựng hạ tầng
- 6 tháng vật lộn với độ phức tạp
- Công ty khởi nghiệp cạn tiền
- Hết game

Cách tiếp cận đúng:

Công ty khởi nghiệp: 3 kỹ sư, 0 người dùng
Kiến trúc: Nguyên khối, Heroku, PostgreSQL

Độ phù hợp sản phẩm – thị trường: 6 tháng
Tăng trưởng: Thêm tính năng thật nhanh
Mở rộng: Vấn đề của sau này

Phản Mẫu 2: Kubernetes Cho Mọi Thứ

Kịch bản:

Lưu lượng: 10 req/s
Nhóm: 5 kỹ sư
Công nghệ: Cụm Kubernetes

Vấn đề:

- Đường cong học tập Kubernetes: rất dốc
- Độ phức tạp vận hành: cao
- Chi phí: \$500+/tháng (so với \$50 trên Heroku)
- Giá trị mang lại: bằng không

Khi nào Kubernetes hợp lý:

- Lưu lượng trên 1000 req/s
- Nhóm có chuyên môn DevOps
- Yêu cầu triển khai phức tạp
- Chiến lược đa đám mây (*multi-cloud*)

Trước khi dùng Kubernetes, hãy thử:

- Heroku/Railway/Render
- AWS Elastic Beanstalk
- Google App Engine
- Một VPS đơn giản + Docker Compose

Phản Mẫu 3: Dùng Chung Cơ Sở Dữ Liệu Giữa Các Dịch Vụ

Kịch bản:

Kiến trúc vi dịch vụ
Mọi dịch vụ → Cùng một cơ sở dữ liệu

Vấn đề:

- Liên kết chặt (*Tight coupling*) (đi ngược mục đích của vi dịch vụ)
- Thay đổi lược đồ (*schema*) làm hỏng mọi thứ
- Không thể mở rộng từng cơ sở dữ liệu độc lập
- Độ phức tạp của giao dịch (*transactions*)

Cách tiếp cận đúng:

Mỗi dịch vụ → Cơ sở dữ liệu riêng
Giao tiếp qua API/sự kiện

Nếu dùng chung cơ sở dữ liệu → bạn không có vi dịch vụ, mà có một hệ thống nguyên khối phân tán (*distributed monolith*).

Phản Mẫu 4: Dịch Vụ Siêu Nhỏ (*Nano-Services*)

Kịch bản:

Dịch vụ User
Dịch vụ UserAuth
Dịch vụ UserProfile
Dịch vụ UserSettings
Dịch vụ UserPreferences

Quá nhiều dịch vụ tí hon.

Vấn đề:

- Chi phí phụ về mạng
- Độ phức tạp vận hành
- Khó hiểu luồng xử lý
- Triển khai là cơn ác mộng

Độ chi tiết hợp lý:

Dịch vụ User (xử lý toàn bộ logic về người dùng)

- ↳ Xác thực
- ↳ Hồ sơ
- ↳ Cài đặt
- ↳ Tùy chọn

Ranh giới dịch vụ = năng lực nghiệp vụ, không phải chức năng.

Phản Mẫu 5: Tối Ưu Sai Chỗ

Kịch bản:

Tối ưu: Phân mảnh cơ sở dữ liệu

Vấn đề thực sự: Truy vấn chậm (thiếu chỉ mục)

Tối ưu: Thêm tầng lưu đệm

Vấn đề thực sự: Truy vấn N+1

Tối ưu: Vi dịch vụ

Vấn đề thực sự: Mã nguồn nguyên khối lộn xộn

Đo lường trước, tối ưu sau.

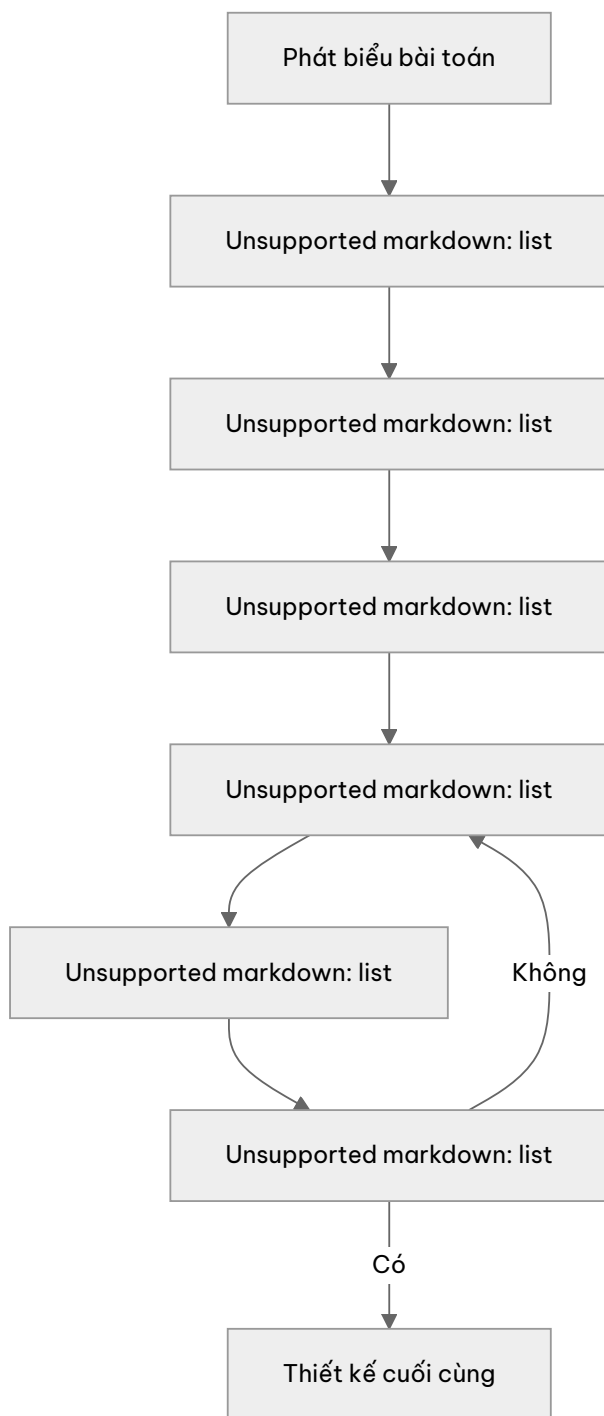
Quy trình:

1. Xác định điểm nghẽn (số liệu đo (*metrics*))
2. Hiểu nguyên nhân gốc (phân tích hiệu năng (*profiling*))
3. Chọn giải pháp đơn giản nhất
4. Đo lường mức cải thiện
5. Lặp lại

KHÔNG bỏ qua bước 1 và 2.

Luồng Tư Duy Thiết Kế Hệ Thống

Cách tiếp cận có cấu trúc cho các quyết định kiến trúc.



Bước 1: Yêu Cầu

Yêu cầu chức năng (Functional requirements):

Hệ thống làm gì:

- Người dùng có thể tải ảnh lên
- Người dùng có thể xem bảng tin
- Người dùng có thể theo dõi người khác

Yêu cầu phi chức năng (Non-functional requirements):

Hệ thống làm tốt đến mức nào:

- 10 triệu người dùng hoạt động hằng ngày
- Độ trễ p99 < 200ms
- Tính sẵn sàng 99,9%
- Tải ảnh lên < 5 giây

Bước 2: Ước Lượng (Estimation)

Các phép ước tính nhanh (Back-of-envelope calculations):

Lưu lượng:

- 10 triệu DAU (người dùng hoạt động hằng ngày)
- Mỗi người dùng: 20 yêu cầu/ngày
- Tổng: 200 triệu yêu cầu/ngày
- Cao điểm: $200M / 86400 * 3 = \sim 7000 \text{ req/s}$

Lưu trữ:

- 1 triệu ảnh được tải lên/ngày
- Ảnh trung bình: 2MB
- Lưu trữ mỗi ngày: 2TB/ngày
- Mỗi năm: 730TB

Băng thông:

- Tải lên: $2TB/ngày = 23 \text{ MB/s}$
- Tỷ lệ đọc:ghi: 10:1
- Tải xuống: 230 MB/s

Là cơ sở cho các quyết định kiến trúc.

Bước 3: Điểm Nghẽn (Bottlenecks)

Dựa trên các ước lượng (estimations):

7000 req/s → Cân cân bằng tải
2TB/ngày tải lên → Cân lưu trữ đối tượng (S3)
Lưu lượng đọc cao → Cân CDN + lưu đệm

Lường trước vấn đề từ sớm.

Bước 4: Đánh Đổi (*Trade-offs*)

Mọi quyết định = một sự đánh đổi.

Lưu đệm ảnh?

Ưu điểm: Truy cập nhanh, giảm tải

Nhược điểm: Chi phí lưu trữ, vô hiệu hoá bộ nhớ đệm

Quyết định: Có, lưu đệm ảnh "nóng" (20% ảnh = 80% lưu lượng)

Nhất quán mạnh?

Ưu điểm: Dữ liệu luôn chính xác

Nhược điểm: Độ trễ cao hơn, phức tạp

Quyết định: Nhất quán cuối cùng là đủ (bằng tin mạng xã hội)

Vi dịch vụ?

Ưu điểm: Mở rộng độc lập

Nhược điểm: Phức tạp, tốn kém

Quyết định: Bắt đầu với nguyên khối, tách ra sau nếu cần

Tư duy đánh đổi một cách tường minh.

Bước 5: Thiết Kế

Bắt đầu đơn giản:

MVP (0-100 nghìn người dùng):

- Nguyên khối (Rails/Django)
- PostgreSQL
- Bộ nhớ đệm Redis
- S3 để lưu ảnh
- CDN

Tín hiệu cần tiến hoá:

- >1 triệu người dùng → Thêm bản sao chỉ đọc
- >5 triệu người dùng → Cân nhắc nguyên khối mô-đun hoá
- >10 triệu người dùng → Đánh giá vi dịch vụ

Lên kế hoạch tiến hoá, đừng xây cho 10 triệu người dùng ngay từ ngày đầu.

Bước 6: Kiểm Chứng

Những câu hỏi cần đặt ra:

Có đáp ứng yêu cầu không?

- Chức năng: ☒
- Phi chức năng: (ở quy mô hiện tại)

Có mở rộng được không?

- Lên gấp 10 lần lưu lượng: Có (thêm máy chủ)
- Lên gấp 100 lần lưu lượng: Cần thiết kế lại kiến trúc

Có quá phức tạp không?

- Nhóm có thể xây dựng: ☒
- Nhóm có thể vận hành: ☒

Chi phí có hợp lý không?

- MVP: \$200/tháng ☒
- Khi đã mở rộng: \$5k/tháng ☒

Tư Duy Đánh Đổi

Mọi quyết định kiến trúc = một sự đánh đổi. Không có giải pháp hoàn hảo.

Đánh Đổi 1: Hiệu Năng (*Performance*) và Độ Phức Tạp

Ví dụ: Lưu đệm

Không có bộ nhớ đệm:

- Mã đơn giản
- Đọc chậm (mọi yêu cầu → cơ sở dữ liệu)
- Tải cơ sở dữ liệu cao

Có bộ nhớ đệm:

- Phức tạp (vô hiệu hoá bộ nhớ đệm)
- Đọc nhanh (truy cập bộ nhớ)
- Tải cơ sở dữ liệu thấp hơn

Khung ra quyết định (*Decision framework*):

- Thiên về đọc (*Read-heavy*)? → Bộ nhớ đệm (*Cache*) rất đáng giá
- Thiên về ghi (*Write-heavy*)? → Bộ nhớ đệm ít giá trị hơn
- Có ngân sách cho độ phức tạp? → Dùng bộ nhớ đệm được
- Chuyên môn của nhóm? → Ưu tiên đơn giản trước

Đánh Đổi 2: Tính Nhất Quán và Tính Sẵn Sàng (*Availability*)

Nhắc lại định lý CAP:

Nhất quán mạnh:

- Dữ liệu luôn chính xác
- Có thể không sẵn sàng (phải chờ đồng bộ)
- Thông lượng thấp hơn

Nhất quán cuối cùng:

- Tính sẵn sàng cao
- Dữ liệu tạm thời bị cũ
- Thông lượng cao hơn

Quyết định:

- Tài khoản ngân hàng: Nhất quán mạnh (*Strong consistency*)
- Bảng tin mạng xã hội: Nhất quán cuối cùng (*Eventual consistency*)
- Giỏ hàng: Dựa trên phiên (*session*) (giải pháp dung hoà)

Đánh Đổi 3: Chi Phí và Quy Mô

Ví dụ: Cơ sở dữ liệu

Một PostgreSQL duy nhất:

- Chi phí: \$50/tháng
- Quy mô: ~1000 req/s
- Đơn giản

PostgreSQL phân mảnh:

- Chi phí: \$500/tháng
- Quy mô: ~10,000 req/s
- Phức tạp

Dịch vụ được quản lý (Aurora):

- Chi phí: \$2000/tháng
- Quy mô: ~50,000 req/s
- Dễ hơn tự phân mảnh

Lựa chọn dựa trên nhu cầu hiện tại + ngân sách.

Đánh Đổi 4: Tốc Độ Phát Triển và Tính Linh Hoạt

Lựa chọn framework:

Rails/Django:

- Phát triển nhanh
- Có quy ước sẵn (ít phải quyết định)
- Thân thiện với nguyên khối

Vi dịch vụ Go/Node:

- Phát triển ban đầu chậm hơn
- Linh hoạt hơn
- Thân thiện với vi dịch vụ

Giai đoạn đầu: Tốc độ chiến thắng.

Sản phẩm đã trưởng thành: Tính linh hoạt mới quan trọng.

Ma Trận Đánh Đổi

Những quyết định thường gặp:

Quyết định	Đơn giản	Phức tạp	Khi Nào Phức Tạp Mới Đáng
Nguyên khối hay Vi dịch vụ	Nguyên khối	Vi dịch vụ	Nhóm trên 50 người, miền nghiệp vụ rõ ràng
SQL hay NoSQL	SQL	NoSQL	Quy mô khổng lồ, lược đồ linh hoạt
Đồng bộ (<i>Sync</i>) hay Bất đồng bộ (<i>Async</i>)	Đồng bộ	Bất đồng bộ	Các thao tác có độ trễ cao
Một CSDL hay Phân mảnh (<i>Sharding</i>)	Một CSDL	Phân mảnh	>10 nghìn lượt ghi/giấy
Tự vận hành hay Dịch vụ được quản lý	Tự vận hành	Được quản lý	Nhóm dưới 10 người, tập trung vào sản phẩm

Mô Hình Tư Duy: Bắt Đầu Đơn Giản, Tiến Hoá Dần Dần

Nguyên tắc cốt lõi của một kiến trúc tốt.

Nguyên Tắc

"Tôi ưu quá sớm là cội nguồn của mọi tội lỗi." - Donald Knuth

Áp dụng vào kiến trúc:

Đừng xây cho quy mô mà bạn chưa có
Đừng giải quyết những vấn đề bạn chưa gặp
Đừng thêm độ phức tạp cho đến khi nỗi đau là thật

Tại sao?

1. Yêu cầu thay đổi

Tuần 1: Xây bản sao Instagram
Tuần 10: Chuyển hướng sang phân tích dữ liệu B2B
Tuần 20: Chuyển hướng sang chatbot AI

Kiến trúc phức tạp → công sức đô'sông đô'biển

2. Giả định về quy mô bị sai

Kế'hoạch: 1 triệu người dùng trong năm đầu
Thực tế: 10 nghìn người dùng

Thiết kế' thừa cho một thị trấn ma

3. Nhóm cần học hỏi

Nguyên khối:

- Ra mắt tính năng nhanh
- Hiểu nhu cầu người dùng
- Cải tiến nhanh

Vi dịch vụ:

- Vật lộn với hạ tầng
- Phát triển tính năng chậm
- Lỡ mất thời cơ thị trường

Lộ Trình Tiến Hoá

Tiến trình lành mạnh:

Tháng 1-3: MVP nguyên khối

- Kiểm chứng ý tưởng
- Có người dùng

Tháng 3-12: Nguyên khối đã mở rộng

- Tối ưu hiệu năng
- Bổ sung hạ tầng (CDN, bộ nhớ đệm, bản sao)

Năm 1-2: Nguyên khối mô-đun hoá

- Dọn dẹp kiến trúc
- Chuẩn bị cho việc mở rộng nhóm

Năm 2 trở đi: Vi dịch vụ có chọn lọc

- Tách các điểm nghẽn ra
- Mở rộng độc lập ở những chỗ cần

Tiến trình không lành mạnh:

Tháng 1: 15 vi dịch vụ, Kubernetes

Tháng 2: Vật lộn với hạ tầng

Tháng 3: Hết tiền

Khung Ra Quyết Định

Khi đứng trước một lựa chọn kiến trúc:

Câu hỏi: Có nên dùng [giải pháp phức tạp] không?

Hãy hỏi:

1. Chúng ta có gặp vấn đề mà nó giải quyết không? (CÓ/KHÔNG)
2. Đã thử các giải pháp đơn giản hơn chưa? (CÓ/KHÔNG)
3. Chúng ta có chuyên môn không? (CÓ/KHÔNG)
4. Chúng ta có ngân sách không? (CÓ/KHÔNG)

Nếu BẤT KỲ câu trả lời nào là KHÔNG → Đừng làm vội

Ví dụ:

"Có nên dùng Kubernetes không?"

- Có vấn đề về mở rộng? KHÔNG → Chờ đã
- Đã thuê các nền tảng được quản lý? KHÔNG → Thuê Heroku trước
- Có chuyên môn DevOps? KHÔNG → Học trên nền tảng đơn giản hơn
- Có ngân sách (\$500+/tháng)? KHÔNG → Dùng phương án rẻ hơn

"Có nên tách thành vi dịch vụ không?"

- Có vấn đề khi mở rộng nhóm? KHÔNG → Chờ đã
- Đã thuê nguyên khối mô-đun hoá? KHÔNG → Tái cấu trúc nguyên khối trước
- Ranh giới dịch vụ đã rõ ràng? KHÔNG → Chưa sẵn sàng
- Có ngân sách cho độ phức tạp? KHÔNG → Giữ nguyên khối

Những Điểm Cốt Lỗi

1. Kiến trúc tiến hoá: Nguyên khối → Đã mở rộng → Mô-đun hoá → Vi dịch vụ

Mỗi giai đoạn phù hợp với một quy mô và quy mô nhóm khác nhau.

2. Nguyên khối không phải là từ xấu

Shopify, GitHub, Basecamp đã mở rộng ở quy mô khổng lồ với hệ thống nguyên khối. Hãy bắt đầu đơn giản.

3. Tín hiệu chuyển đổi phải là nỗi đau thật, không phải phỏng đoán

Điểm nghẽn hiệu năng, mở rộng nhóm, nhu cầu tài nguyên khác nhau.

4. Strangler fig pattern > viết lại toàn bộ một lần (big bang rewrite)

Chuyển đổi dần dần, từng phần một. Rủi ro thấp, dễ hoàn tác.

5. Các phản mẫu: Vi dịch vụ quá sớm, Kubernetes quá mức cần thiết, dùng chung cơ sở dữ liệu

Tránh độ phức tạp không cần thiết. Đo lường vấn đề trước khi giải quyết.

6. Luồng thiết kế hệ thống: Yêu cầu → Ước lượng → Điểm nghẽn → Đánh đổi → Thiết kế

Quy trình tư duy có cấu trúc cho các quyết định kiến trúc.

7. Mọi quyết định = một sự đánh đổi

Hiệu năng và độ phức tạp, tính nhất quán và tính sẵn sàng, chi phí và quy mô.

8. Bắt đầu đơn giản, tiến hoá dần dần

Đừng xây cho quy mô mà bạn chưa có. Nguyên tắc YAGNI.

9. Kiểm chứng giả định bằng dữ liệu

Đo lưu lượng, phân tích hiệu năng (*profile performance*), hiểu rõ những điểm nghẽn thực sự.

10. Chuyên môn của nhóm quan trọng hơn công nghệ

Một bộ công nghệ đơn giản mà nhóm thành thạo > một bộ công nghệ phức tạp mà nhóm phải vật lộn.

Nhưng luôn nhớ: **các mẫu thiết kế** (*patterns*) là công cụ, không phải yêu cầu bắt buộc. Dùng khi cần, không chạy theo xu hướng.

Hãy nhớ: Instagram được mua lại với giá 1 tỷ đô la khi vẫn là một hệ thống nguyên khối. Độ phức tạp không tạo ra giá trị - giải quyết vấn đề của người dùng mới tạo ra giá trị. Hãy bắt đầu đơn giản, tiến hoá khi nỗi đau là thật, và luôn tự hỏi liệu độ phức tạp thêm vào có đáng hay không.

Hệ Thống Thiên Về Đọc (*Read-Heavy Systems*) - Thiết Kế Cho 99% Lướt Đọc

Hầu hết các hệ thống trong thế giới thực đều có chung một đặc điểm:

Lướt đọc >> Lướt ghi (thường là 100:1 hoặc 1000:1).

Ví dụ:

- **Dịch vụ rút gọn URL** (*URL shortener*): 1 lượt ghi (tạo URL ngắn), hơn 1.000 lượt đọc (chuyển hướng)
- **YouTube**: 1 lượt tải lên, hơn 10.000 lượt xem
- **Twitter**: 1 tweet, hơn 100 lượt đọc
- **Trang tin tức**: 1 bài viết được đăng, 1.000.000 lượt đọc

Đây là hệ thống thiên về đọc (*read-heavy system*) - **mẫu** (*pattern*) **phổ biến nhất mà bạn sẽ gặp**.

Hệ thống thiên về đọc có những đặc điểm rất khác so với hệ thống thiên về ghi (*write-heavy*):

- Thao tác ghi vào cơ sở dữ liệu (*database*) không phải là điểm nghẽn (*bottleneck*)
- Tỷ lệ trùng bộ nhớ đệm (*Cache hit rate*) là yếu tố then chốt (90%+ = thành công)
- Nhất quán cuối cùng (*Eventual consistency*) là chấp nhận được
- Xử lý bất đồng bộ (*async*) cho phân phân tích dữ liệu

Bài học này dạy bạn thiết kế hệ thống thiên về đọc qua 2 nghiên cứu tình huống (*case studies*) thực tế:

1. Dịch vụ rút gọn URL (mẫu TinyURL/Bitly)

- Chiến lược sinh ID
- Tối ưu việc chuyển hướng (*redirect*)
- Kiến trúc ưu tiên bộ nhớ đệm (*Cache-first architecture*)
- Đường ống phân tích dữ liệu (*Analytics pipeline*)

2. Hệ Thống Phân Phối Nội Dung (*Content Delivery System*)

- Lưu đệm nhiều tầng (Multi-layer caching)
- Phân phối toàn cầu
- Vô hiệu hoá bộ nhớ đệm (Cache invalidation)

Mô hình tư duy (Mental model): Bộ nhớ đệm (Cache) → Cơ sở dữ liệu → Đường ống bất đồng bộ (Async pipeline).

Đặc Điểm Của Hệ Thống Thiên Về Đọc (Read-Heavy)

Mô Hình Lưu Lượng (Traffic)

Hệ thống thiên về ghi:

Ghi: 1000/s
 Đọc: 1000/s
 Tỷ lệ: 1:1

CSDL: Điềm nghìn ở thao tác ghi
 Mở rộng: Phức tạp (phân mảnh dữ liệu, nhân bản)

Hệ thống thiên về đọc:

Ghi: 10/s
 Đọc: 10.000/s
 Tỷ lệ: 1:1000

CSDL: Không phải điềm nghìn (ít thao tác ghi)
 Mở rộng: Đơn giản (lưu đệm thật mạnh tay)

Ưu Tiên Trong Thiết Kế

Các ưu tiên của hệ thống thiên về đọc:

1. Tỷ lệ trúng bộ nhớ đệm

- Mục tiêu: 90-99%
- Mỗi lần trượt bộ nhớ đệm (cache miss) = một truy vấn (query) cơ sở dữ liệu tốn kém
- Tối ưu cho dữ liệu "nóng" (hot data)

2. Độ trễ (latency) khi đọc

- Người dùng cảm nhận ngay độ trễ khi đọc
- Độ trễ khi ghi ít quan trọng hơn (có thể xử lý bất đồng bộ)

3. Tính sẵn sàng (*Availability*)

- Thao tác đọc phải hoạt động kể cả khi thao tác ghi bị sập
- Bản sao chỉ đọc (*Read replicas*) để dự phòng (*redundancy*)

4. Hiệu quả chi phí (*cost*)

- Bộ nhớ đệm rẻ (RAM)
- Cơ sở dữ liệu đắt (ổ đĩa + tính toán)
- 90% trùng bộ nhớ đệm = tiết kiệm chi phí gấp 10 lần

Mẫu Kiến Trúc



Syntax error in text
mermaid version 11.4.1

Luồng đọc được tối ưu cho tốc độ.

Luồng ghi được tối ưu cho độ tin cậy (*reliability*).

Phân phân tích được tách rời (*decoupled*) (**bất đồng bộ**).

Nghiên Cứu Tình Huống (*Case Study*) 1: Dịch Vụ Rút Gọn URL (TinyURL/Bitly)

Yêu cầu (*requirements*) **hệ thống:**

Chức năng:

- Sinh URL ngắn từ URL dài
- Chuyển hướng URL ngắn sang URL dài
- Theo dõi số liệu phân tích lượt nhấp

Phi chức năng:

- 100 triệu URL được tạo/tháng
- 10 tỷ lượt chuyển hướng/tháng (tỷ lệ 100:1)
- Độ trễ < 50ms cho chuyển hướng
- Tính sẵn sàng: 99,99%

Ước Lượng (Estimation) Năng Lực (Capacity)

Lưu lượng:

URL được tạo: 100 triệu/tháng
= ~40 URL/giây

Chuyển hướng: 10 tỷ/tháng
= ~4000 lượt chuyển hướng/giây

Lưu trữ (Storage):

Mỗi URL:

- short_id: 7 byte
- long_url: 200 byte (trung bình)
- metadata: 50 byte

Tổng: ~250 byte

100 triệu URL/tháng * 250 byte = 25 GB/tháng

5 năm: 1,5 TB

Băng thông (Bandwidth):

Chuyển hướng: 4000/s * 250 byte = 1 MB/s

Kèm biên an toàn: 5 MB/s

Kích thước bộ nhớ đệm:

URL "nóng" (20% dữ liệu):

1,5 TB * 0,2 = 300 GB

Kèm nhân bản (3x): 1 TB bộ nhớ đệm

Chiến Lược Sinh ID

Yêu cầu:

- Ngắn (6-7 ký tự)
- Duy nhất
- Ngẫu nhiên (không tuần tự - vì lý do bảo mật)

Cách Tiếp Cận 1: Hàm Băm (Hash) + Phát Hiện Xung Đột (Collision)

```
import hashlib
import base62

def generate_short_id(long_url):
    # Hash long URL
    hash_bytes = hashlib.md5(long_url.encode()).digest()

    # Take first 8 bytes
    hash_int = int.from_bytes(hash_bytes[:8], 'big')

    # Convert to base62
    short_id = base62.encode(hash_int)[:7]

    # Check collision in database
    if db.exists(short_id):
        # Handle collision (add counter, retry)
        short_id = handle_collision(long_url)

    return short_id
```

Ưu điểm:

- Tất định (Deterministic) (cùng URL → cùng ID ngắn)
- Sinh nhanh

Nhược điểm:

- Có thể xảy ra xung đột (cần phát hiện)
- Phải kiểm tra cơ sở dữ liệu mỗi lần sinh

Cách Tiếp Cận 2: ID Tự Tăng (Auto-increment) + Base62

```
import base62

def generate_short_id():
    # Get next ID from database
    id = db.get_next_id() # Auto-increment

    # Convert to base62
    short_id = base62.encode(id)

    return short_id

# Example:
# ID 1 → "1"
# ID 62 → "10"
# ID 3844 → "100"
# ID 56800235584 → "zzzzzzz" (7 chars)
```

Bộ ký tự Base62:

$[0-9][a-z][A-Z] = 62$ ký tự

Năng lực:

6 ký tự: $62^6 = 56$ tỷ URL
7 ký tự: $62^7 = 3,5$ nghìn tỷ URL

Ưu điểm:

- Không có xung đột (collisions)
- Logic đơn giản
- Năng lực có thể dự đoán

Nhược điểm:

- Tuần tự (rủi ro bảo mật)
- Điểm lỗi duy nhất (Single point of failure) (bộ sinh ID)

Cách Tiếp Cận 3: Sinh ID Phân Tán (Dùng Trong Môi Trường Thực Tế (Production))

```
# Use Twitter Snowflake or similar
def generate_short_id():
    # Snowflake generates 64-bit ID
    id = snowflake.generate()

    # Convert to base62
    short_id = base62.encode(id)

    return short_id

# Snowflake structure:
# [timestamp: 41 bits][datacenter: 5 bits][machine: 5 bits][sequence: 12 bits]
```

Ưu điểm:

- Không có xung đột
- Không có điểm lỗi duy nhất
- Có thứ tự theo thời gian (hữu ích cho phân tích dữ liệu)
- Thông lượng (*throughput*) cao

Được dùng bởi: Twitter, Instagram, Discord

Lược Đồ (Schema) Cơ Sở Dữ Liệu

```
-- URLs table
CREATE TABLE urls (
  short_id VARCHAR(7) PRIMARY KEY,
  long_url VARCHAR(2048) NOT NULL,
  user_id BIGINT,
  created_at TIMESTAMP DEFAULT NOW(),
  expires_at TIMESTAMP NULL,

  INDEX idx_user_id (user_id),
  INDEX idx_created_at (created_at)
);

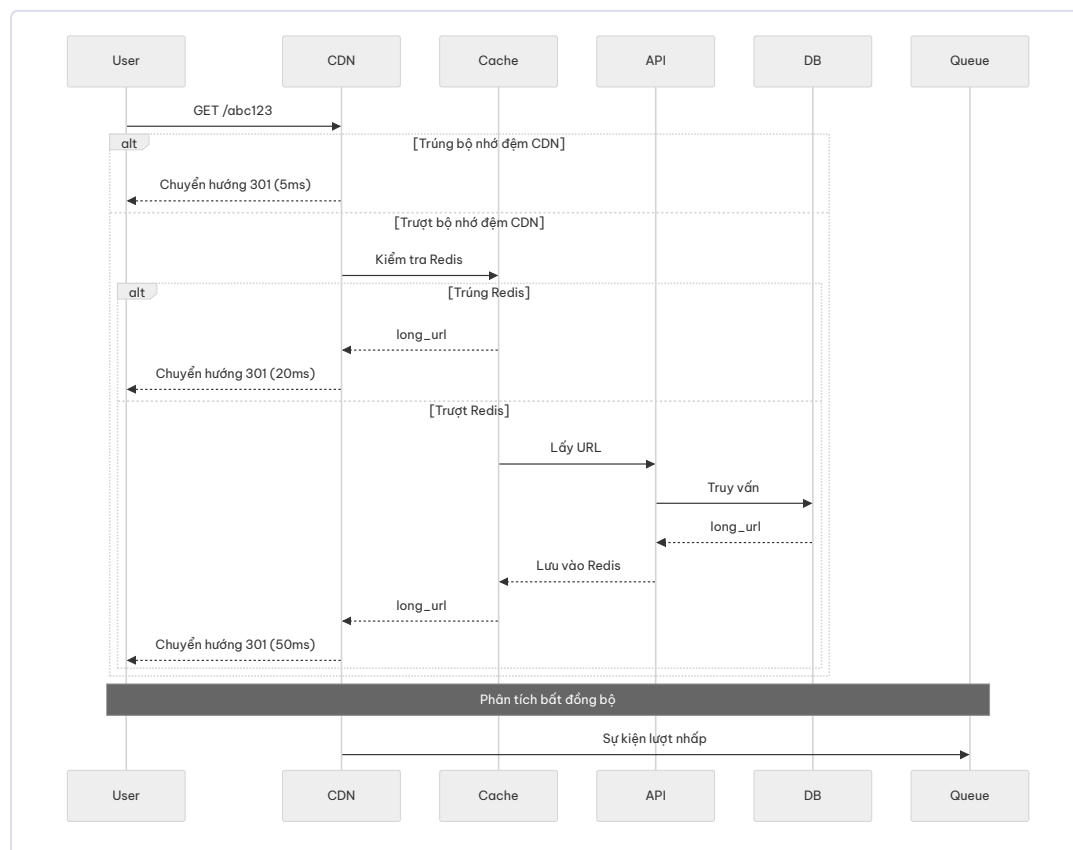
-- Analytics table (separate for performance)
CREATE TABLE url_clicks (
  short_id VARCHAR(7),
  clicked_at TIMESTAMP,
  user_agent TEXT,
  ip_address VARCHAR(45),
  country VARCHAR(2),

  INDEX idx_short_id_time (short_id, clicked_at)
) PARTITION BY RANGE (clicked_at);
```

Tại sao tách bảng riêng:

- URL: Nhỏ, thiên về đọc, có thể lưu đệm
- Lượt nhấp: Lớn, thiên về ghi, chỉ ghi nối thêm (*append-only*)

Luồng Chuyển Hướng (Đã Tối Ưu)



Các tầng bộ nhớ đệm:

1. **Điểm biên (edge) CDN:** tỷ lệ trúng (hit rate) 80-90%, <5ms
2. **Redis:** tỷ lệ trúng 95-99%, <20ms
3. **Cơ sở dữ liệu:** 1-5% trượt (miss), <50ms

Mục tiêu: độ trễ p99 <50ms.

Hiện Thực API

```
from flask import Flask, redirect, request
import redis
import hashlib

app = Flask(__name__)
cache = redis.Redis(host='redis', decode_responses=True)

@app.route('/<short_id>')
def redirect_url(short_id):
    # Check cache first
    long_url = cache.get(f"url:{short_id}")

    if not long_url:
        # Cache miss - query database
        url = db.query(
            "SELECT long_url FROM urls WHERE short_id = ?",
            short_id
        )

        if not url:
            return "URL not found", 404

        long_url = url['long_url']

        # Store in cache (24h TTL)
        cache.setex(f"url:{short_id}", 86400, long_url)

    # Track analytics (async)
    track_click.delay(short_id, request)

    # Redirect (301 permanent)
    return redirect(long_url, code=301)

@app.route('/api/shorten', methods=['POST'])
def create_short_url():
    long_url = request.json['url']

    # Validate URL
    if not is_valid_url(long_url):
        return {"error": "Invalid URL"}, 400

    # Generate short ID
    short_id = generate_short_id(long_url)

    # Store in database
    db.execute(
        "INSERT INTO urls (short_id, long_url, user_id) VALUES (?, ?, ?)",
        short_id, long_url, current_user.id
    )

    # Warm cache
```

```

cache.setex(f"url:{short_id}", 86400, long_url)

return {
    "short_url": f"https://short.ly/{short_id}",
    "long_url": long_url
}

```

Chiến Lược Lưu Đệm

Lưu đệm mạnh tay các URL "nóng":

```

# Cache tiers
class URLCache:
    def __init__(self):
        self.l1 = {} # In-memory (application)
        self.l2 = redis.Redis() # Redis cluster
        self.l3 = None # CDN (edge)

    def get(self, short_id):
        # L1: Application memory (microseconds)
        if short_id in self.l1:
            metrics.increment('cache.l1.hit')
            return self.l1[short_id]

        # L2: Redis (milliseconds)
        long_url = self.l2.get(f"url:{short_id}")
        if long_url:
            metrics.increment('cache.l2.hit')
            self.l1[short_id] = long_url # Populate L1
            return long_url

        # L3: Database (tens of milliseconds)
        metrics.increment('cache.miss')
        long_url = db.get_url(short_id)

        if long_url:
            # Populate all cache layers
            self.l1[short_id] = long_url
            self.l2.setex(f"url:{short_id}", 86400, long_url)

        return long_url

```

Loại bỏ khỏi bộ nhớ đệm (Cache eviction):

```
# LRU eviction in Redis
redis_config = {
    'maxmemory': '10gb',
    'maxmemory-policy': 'allkeys-lru'
}

# Hot URLs stay cached
# Cold URLs evicted automatically
```

Làm ấm bộ nhớ đệm (Cache warming):

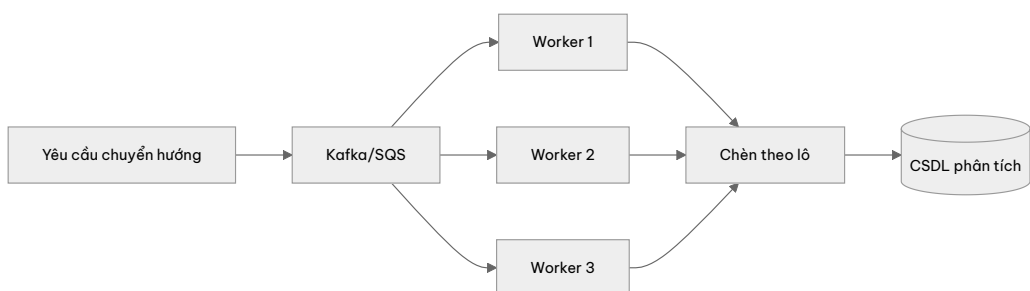
```
# Warm cache with popular URLs
def warm_cache():
    popular_urls = db.query("""
        SELECT short_id, long_url
        FROM urls
        WHERE created_at > NOW() - INTERVAL '7 days'
        ORDER BY click_count DESC
        LIMIT 10000
    """)

    for url in popular_urls:
        cache.setex(f"url:{url['short_id']}", 86400, url['long_url'])
```

Đường Ống Phân Tích Dữ Liệu (Bất Đồng Bộ)

Vấn đề: Đừng để thao tác ghi dữ liệu phân tích làm chậm việc chuyển hướng (redirects).

Giải pháp: Xử lý bất đồng bộ.



Hiện thực:

```

from celery import Celery

app = Celery('tasks', broker='redis://localhost')

@app.task
def track_click(short_id, request_data):
    # Extract analytics data
    analytics = {
        'short_id': short_id,
        'clicked_at': datetime.now(),
        'ip_address': request_data.remote_addr,
        'user_agent': request_data.user_agent.string,
        'referrer': request_data.referrer,
        'country': geoip.lookup(request_data.remote_addr)
    }

    # Batch insert (better performance)
    analytics_buffer.append(analytics)

    if len(analytics_buffer) >= 1000:
        db.bulk_insert('url_clicks', analytics_buffer)
        analytics_buffer.clear()

```

Xử lý theo lô (*batch*):

```

# Write clicks in batches
def batch_write_clicks():
    while True:
        clicks = queue.get_batch(size=5000)

        if clicks:
            # Bulk insert (much faster)
            db.bulk_insert("""
                INSERT INTO url_clicks
                (short_id, clicked_at, ip_address, user_agent, country)
                VALUES %s
            """, clicks)

            # Update click counts
            update_click_counts(clicks)

        time.sleep(1)

```

Chiến lược Phân Mảnh Dữ Liệu (*Sharding*)

Khi nào cần phân mảnh (*shard*): >10 triệu URL, thông lượng ghi trở thành điểm nghẽn.

Phân mảnh theo short_id:

```
def get_shard(short_id):  
    # Hash short_id to determine shard  
    shard_num = hash(short_id) % NUM_SHARDS  
    return shards[shard_num]  
  
# Example: 4 shards  
# short_id "abc123" → hash → shard 2  
# short_id "xyz789" → hash → shard 3
```

Lợi ích:

- Phân bố đồng đều
- Không có điểm nóng (*hotspots*)
- Truy vấn (*Queries*) luôn chỉ nhắm đến 1 mảnh

Hạn chế:

- Truy vấn theo khoảng (*Range queries*) phải quét qua tất cả các mảnh (*shards*)
- Các URL của một người dùng bị phân tán trên nhiều mảnh

Phương án khác: Phân mảnh theo user_id

```
def get_shard(user_id):  
    return shards[user_id % NUM_SHARDS]
```

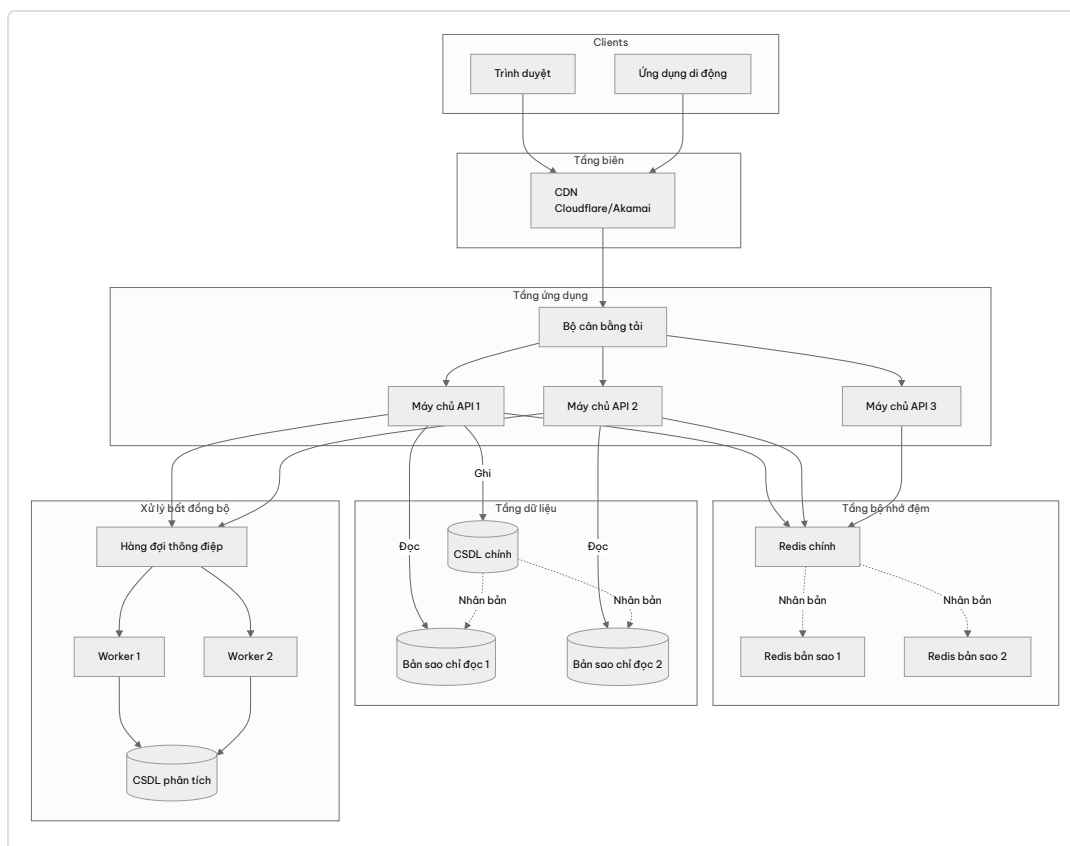
Lợi ích:

- Các URL của một người dùng nằm trên cùng một mảnh
- Truy vấn theo user_id hiệu quả

Hạn chế:

- Việc chuyển hướng cân bằng tra cứu (*short_id* → mảnh)

Kiến Trúc Hoàn Chính



Kiến trúc hoàn chỉnh cho một dịch vụ rút gọn URL ở quy mô (scale) lớn.

Các Tối Ưu Then Chốt

1. Chuyển hướng vĩnh viễn (Permanent redirects) (301)

```
# 301 = browser caches redirect
return redirect(long_url, code=301)

# Users won't hit server on subsequent visits
```

2. Lưu đệm (caching) trên CDN

```
# Cache-Control header
response.headers['Cache-Control'] = 'public, max-age=86400'
```


3. Tải trước DNS (*DNS prefetch*)

```
<!-- In landing page -->  
<link rel="dns-prefetch" href="//short.ly">
```

4. Gộp kết nối (*Connection pooling*)

```
# Reuse database connections  
db_pool = ConnectionPool(max_connections=100)
```

5. Bản sao chỉ đọc

```
# Route reads to replicas  
if operation == 'read':  
    db = read_replica  
else:  
    db = primary_db
```

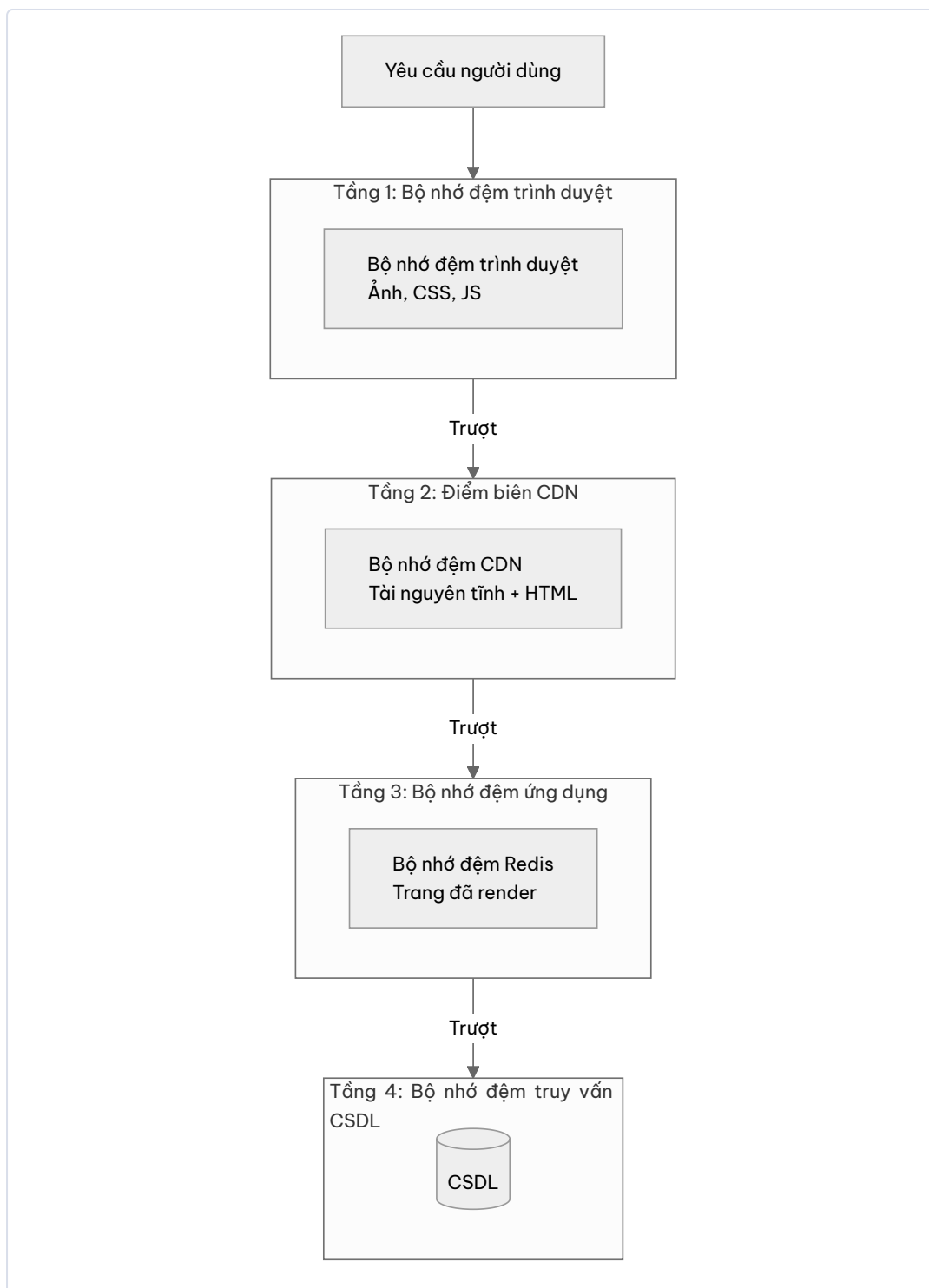
Nghiên Cứu Tình Huống 2: Hệ Thống Phân Phối Nội Dung

Ví dụ: WordPress, Medium, các trang tin tức

Đặc điểm:

- Ghi một lần, đọc hàng triệu lần
- Nhiều nội dung đa phương tiện nặng (ảnh, video)
- Lượng độc giả toàn cầu
- SEO là yếu tố then chốt

Lưu Đệm Nhiều Tầng



Tỷ lệ trúng bộ nhớ đệm (*Cache hit rates*):

- Trình duyệt: 50-60% (khách truy cập quay lại)
- CDN: 80-90% (nội dung phổ biến)
- Redis: 95-99% (dữ liệu "nóng")
- Cơ sở dữ liệu: 1-5% (các lần trượt bộ nhớ đệm (*cache misses*))

Chiến Lược Theo Loại Nội Dung

Tài nguyên tĩnh (*Static assets*) (ảnh, CSS, JS):

```
Cache-Control: public, max-age=31536000, immutable
```

Lưu đệm mạnh tay, URL có phiên bản (*versioned URLs*):

```
/static/app.v123.js
/images/logo.abc123.png
```

Trang HTML:

```
Cache-Control: public, max-age=300, s-maxage=3600
```

Trình duyệt: 5 phút, CDN: 1 giờ.

Phản hồi (*responses*) API:

```
Cache-Control: public, max-age=60
```

TTL ngắn, cập nhật thường xuyên.

Vô Hiệu Hoá Bộ Nhớ Đệm

Vấn đề: Nội dung đã cập nhật, nhưng bộ nhớ đệm vẫn là bản cũ.

Chiến lược 1: Dựa trên TTL

```
# Short TTL for frequently updated content
cache.setex('article:123', 300, article_html) # 5 minutes
```

Ưu điểm: Đơn giản, nhất quán cuối cùng

Nhược điểm: Có độ trễ trước khi có nội dung mới

Chiến lược 2: Dựa trên sự kiện (event)

```
# Invalidate on update
@app.route('/admin/article/<id>', methods=['PUT'])
def update_article(id):
    # Update database
    db.update_article(id, data)

    # Invalidate all caches
    cache.delete(f'article:{id}')
    cdn.purge(f'/article/{id}')

    return {"status": "updated"}
```

Ưu điểm: Nhất quán ngay lập tức

Nhược điểm: Phức tạp, bị giới hạn tần suất (rate limits) của API

Chiến lược 3: URL có phiên bản

```
# Include version in URL
def article_url(article):
    return f"/article/{article.id}?v={article.updated_at.timestamp()}"

# Old URL cached, new URL = cache miss
```

Ưu điểm: Đơn giản, không cần vô hiệu hoá (invalidation)

Nhược điểm: URL rối mắt

Cách tiếp cận lại (trong môi trường thực tế):

```
# Static assets: versioned URLs
/static/app.v123.js

# HTML pages: short TTL + event invalidation
cache.setex('page:home', 300, html) # 5 min TTL
on_update: cache.delete('page:home') # Immediate

# API: very short TTL
cache.setex('api:posts', 60, json) # 1 min TTL
```

Mô Hình Hoá Cơ Sở Dữ Liệu Cho Thao Tác Đọc

Phi chuẩn hoá (Denormalization) để tăng hiệu năng (performance):

```
-- Normalized (slow - requires JOIN)
SELECT a.*, u.name, u.avatar
FROM articles a
JOIN users u ON a.author_id = u.id
WHERE a.id = 123;

-- Denormalized (fast - single query)
CREATE TABLE articles (
  id BIGINT PRIMARY KEY,
  title VARCHAR(200),
  content TEXT,
  author_id BIGINT,
  author_name VARCHAR(100), -- Denormalized
  author_avatar VARCHAR(255), -- Denormalized
  created_at TIMESTAMP
);

SELECT * FROM articles WHERE id = 123;
```

Sự đánh đổi (Trade-off): Lưu trữ + độ phức tạp (complexity) khi cập nhật để đổi lấy hiệu năng đọc.

Khung nhìn cụ thể hoá (Materialized views):

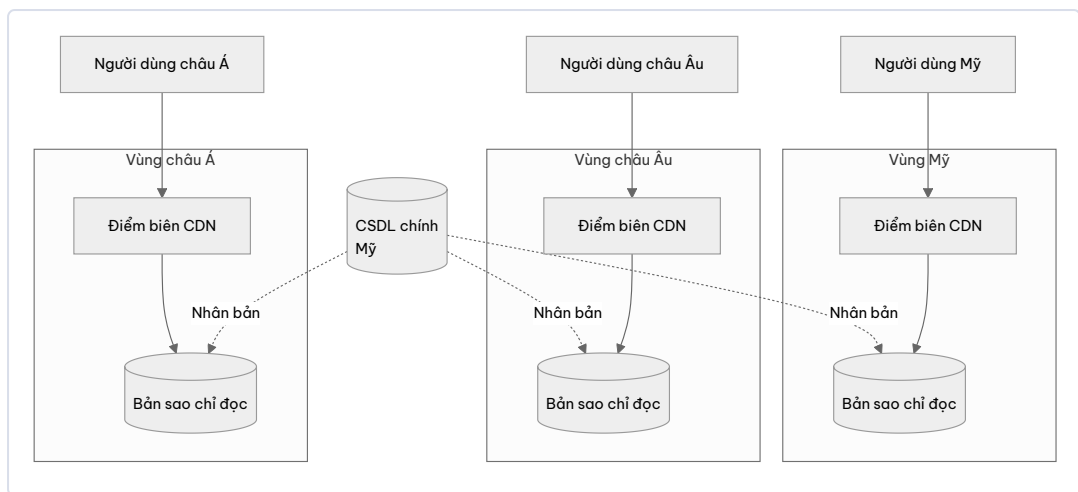
```
-- Pre-computed popular articles
CREATE MATERIALIZED VIEW popular_articles AS
SELECT
  a.*,
  COUNT(v.id) as view_count,
  COUNT(c.id) as comment_count
FROM articles a
LEFT JOIN views v ON a.id = v.article_id
LEFT JOIN comments c ON a.id = c.article_id
WHERE a.published_at > NOW() - INTERVAL '7 days'
GROUP BY a.id
ORDER BY view_count DESC
LIMIT 100;

-- Refresh periodically
REFRESH MATERIALIZED VIEW popular_articles;
```

Truy vấn trực tiếp từ khung nhìn - có kết quả tức thì.

Triển Khai (Deployment) Toàn Cầu

Kiến trúc đa vùng (multi-region):



Lợi ích:

- Độ trễ thấp trên toàn thế giới
- Dự phòng (khi cả một vùng gặp sự cố (failure))

Thách thức:

- Độ trễ nhân bản (Replication lag) (nhất quán cuối cùng)

- Chi phí cao hơn

Tối Ưu Hình Ảnh

Yếu tố then chốt với các trang nội dung.

Ảnh đáp ứng (*Responsive images*):

```

```

Trình duyệt tải kích thước phù hợp.

Tối ưu định dạng:

```
# Serve WebP to supporting browsers
def serve_image(image_id):
    accepts_webp = 'image/webp' in request.headers.get('Accept', '')

    if accepts_webp:
        return f"/images/{image_id}.webp"
    else:
        return f"/images/{image_id}.jpg"
```

WebP: nhỏ hơn JPEG 30%.

Tải lười (*Lazy loading*):

```

```

Chỉ tải ảnh khi cuộn đến vùng hiển thị.

Xử lý ảnh trên CDN:

Góc: /images/photo.jpg
Đôi kích thước: /images/photo.jpg?w=800&h=600
Tối ưu: /images/photo.jpg?format=webp&quality=80

Cloudflare, Cloudinary, Imgix đều hỗ trợ việc này.

Tư Duy Ưu Tiên Bộ Nhớ Đệm (Cache-First)

Sự thay đổi tư duy (*mindset*) cho hệ thống thiên về đọc.

Luồng Truyền Thống

Yêu cầu người dùng → API → CSDL → Phản hồi

Mọi yêu cầu (*request*) đều chạm tới cơ sở dữ liệu - chậm, tốn kém.

Luồng Ưu Tiên Bộ Nhớ Đệm

Yêu cầu người dùng → Bộ nhớ đệm (trúng) → Phản hồi (nhanh)
→ Bộ nhớ đệm (trượt) → CSDL → Bộ nhớ đệm → Phản hồi

99% yêu cầu (*requests*) được phục vụ từ bộ nhớ đệm - nhanh, rẻ.

Các Câu Hỏi Thiết Kế

Khi thiết kế một hệ thống thiên về đọc, hãy hỏi:

1. Tỷ lệ đọc/ghi là bao nhiêu?

Nếu từ 100:1 trở lên → Kiến trúc ưu tiên bộ nhớ đệm

2. Tập dữ liệu "nóng" là gì?

Xác định 20% dữ liệu = 80% lưu lượng
Lưu đệm phần này thật mạnh tay

3. Dữ liệu cần mới đến mức nào?

Thời gian thực: TTL ngắn (vài giây)
Gần thời gian thực: TTL trung bình (vài phút)
Nhất quán cuối cùng: TTL dài (vài giờ)

4. Tỷ lệ trượt bộ nhớ đệm (*cache miss rate*) chấp nhận được là bao nhiêu?

Mục tiêu: 90-99% trúng bộ nhớ đệm
Đo lường và tối ưu

5. Lưu đệm ở đâu?

Trình duyệt → CDN → Ứng dụng → CSDL
Nhiều tầng = tỷ lệ trúng cao hơn

Làm Ấm Bộ Nhớ Đệm

Đừng chờ đến khi trượt bộ nhớ đệm.

```
# Warm cache on deploy
def warm_cache():
    # Popular articles
    articles = db.query("""
        SELECT id, rendered_html
        FROM articles
        WHERE view_count > 1000
        ORDER BY view_count DESC
        LIMIT 1000
    """)

    for article in articles:
        cache.setex(
            f'article:{article.id}',
            3600,
            article.rendered_html
        )

    # User sessions
    sessions = db.query("SELECT * FROM active_sessions")
    for session in sessions:
        cache.setex(f'session:{session.id}', 1800, session.data)
```

Lưu đệm chủ động = hiệu năng ổn định.

Tóm Tắt Các Mẫu (Patterns) Then Chốt

Mẫu 1: Bộ Nhớ Đệm Nhiều Tầng (Multi-Layer Cache)

Trình duyệt → CDN → Redis → CSDL

Mỗi tầng hứng một phần lưu lượng khác nhau.

Mẫu 2: Phân Tích Bất Đồng Bộ

Yêu cầu → Phản hồi nhanh
→ Đưa sự kiện vào hàng đợi
→ Worker xử lý
→ CSDL phân tích

Đừng để thao tác ghi dữ liệu phân tích làm chậm thao tác đọc.

Mẫu 3: Phi Chuẩn Hoá

JOIN 3 bảng → Chậm
Bảng phi chuẩn hoá → Nhanh

Đánh đổi lưu trữ để lấy hiệu năng đọc.

Mẫu 4: Bản Sao Chỉ Đọc

Ghi → Máy chủ chính
Đọc → Các bản sao (3-5x)

Mở rộng thao tác đọc theo chiều ngang.

Mẫu 5: Chuyển Hướng Vĩnh Viễn

HTTP 301 → Trình duyệt lưu đệm
HTTP 302 → Trình duyệt không lưu đệm

Dùng 301 cho các URL ổn định.

Những Điểm Cốt Lõi

1. Hệ thống thiên về đọc = kiến trúc ưu tiên bộ nhớ đệm

Tỷ lệ trùng bộ nhớ đệm 90-99% là then chốt. Nhiều tầng bộ nhớ đệm.

2. Mẫu dịch vụ rút gọn URL: sinh ID + lưu đệm mạnh tay

Mã hoá Base62, bộ nhớ đệm nhiều tầng, phân tích bất đồng bộ.

3. Phân phối nội dung: Lưu đệm nhiều tầng + CDN

Trình duyệt → CDN → Ứng dụng → Cơ sở dữ liệu.

4. Các chiến lược sinh ID: Hàm băm, tự tăng, Snowflake

Snowflake là tốt nhất cho hệ thống phân tán (*distributed systems*).

5. Đường ống phân tích dữ liệu bất đồng bộ

Đừng làm chậm thao tác đọc. Hàng đợi (*Queue*) → Worker → CSDL phân tích.

6. Các chiến lược vô hiệu hoá bộ nhớ đệm: TTL, dựa trên sự kiện, URL có phiên bản

Dùng cách tiếp cận lại trong môi trường thực tế.

7. Phi chuẩn hoá cơ sở dữ liệu cho thao tác đọc

Đánh đổi lưu trữ + độ phức tạp khi cập nhật để lấy hiệu năng truy vấn.

8. Phân mảnh dữ liệu theo ID để phân bố đồng đều

Tránh điểm nóng, mở rộng thao tác ghi.

9. Triển khai toàn cầu với bản sao chỉ đọc

Độ trễ thấp trên toàn thế giới, nhất quán cuối cùng là chấp nhận được.

10. Giám sát (*Monitor*) tỷ lệ trùng bộ nhớ đệm liên tục

Mục tiêu 90-99%. Tối ưu việc lưu đệm dữ liệu "nóng".

Hãy nhớ: Với hệ thống thiên về đọc, bộ nhớ đệm không phải là một phép tối ưu - nó chính là kiến trúc. Hãy thiết kế theo hướng ưu tiên bộ nhớ đệm, đừng coi bộ nhớ đệm là thứ tính sau. 99% người dùng của bạn không bao giờ

nên chạm tới cơ sở dữ liệu.

Hệ Thống Bảng Tin (*Feed*) Mạng Xã Hội: Kiến Trúc Phức Tạp Nhất

Tôi còn nhớ lần đầu được giao nhiệm vụ: "Thiết kế bảng tin (*news feed*) cho một mạng xã hội".

Tôi nghĩ đơn giản: "Truy vấn các bài đăng của những người mà người dùng theo dõi, sắp xếp theo thời gian, xong."

Một kiến trúc sư lâu năm (*senior architect*) nhìn tôi: "Với 1 triệu người dùng, mỗi người theo dõi 500 người, mỗi người đăng bài 10 lần/ngày. Em tính sao?"

Tôi tính nhanh:

- Tải bảng tin = Truy vấn 500 người dùng $\times$ 10 bài đăng = 5.000 bài đăng
- Sắp xếp 5.000 bài đăng
- 1 triệu người dùng làm việc này $\rightarrow$ 1 triệu $\times$ 5.000 truy vấn (*queries*) = 5 tỷ truy vấn
- Cơ sở dữ liệu (*Database*) sập ngay lập tức

Anh ấy cười: "Chào mừng đến với bài toán phân phát (*fanout*). Đây là lý do Facebook phải tự xây dựng hạ tầng (*infrastructure*) riêng."

Đó là lúc tôi học được: **Hệ thống bảng tin là kiến trúc phức tạp nhất trong các ứng dụng hướng người tiêu dùng.**

Tại Sao Hệ Thống Bảng Tin Lại Khó?

Thách Thức Cốt Lõi

Phát biểu bài toán:

Người dùng mở ứng dụng → Thấy bảng tin được cá nhân hoá

- Bài đăng từ những người họ theo dõi
- Sắp xếp theo mức độ liên quan/thời gian
- Tải < 500ms
- Hàng triệu người dùng đồng thời

Nghe thì đơn giản. Thực tế: Một cơn ác mộng.

Tính toán quy mô (scale):

Nền tảng: 100M người dùng

Số người theo dõi trung bình: 200 người

Số bài đăng trung bình: 2 bài/ngày mỗi người dùng

Cách tiếp cận ngây thơ (Truy vấn khi đọc):

Người dùng tải bảng tin:

- Truy vấn 200 người được theo dõi
- Lấy các bài đăng mới nhất của từng người
- Gộp 400 bài đăng
- Sắp xếp theo thời gian
- Áp dụng xếp hạng

Với 100M người dùng:

- 100M người dùng × 200 truy vấn = 20 tỷ truy vấn cơ sở dữ liệu/ngày
- Cơ sở dữ liệu tan chảy

Tại sao nó khó:

1. Khuếch đại đọc (read amplification)

- Mỗi lần tải bảng tin = hàng trăm lượt đọc
- Không thể lưu đệm (cá nhân hoá theo từng người dùng)

2. Khuếch đại ghi (write amplification)

- 1 bài đăng → hàng nghìn người theo dõi nhìn thấy
- Làm sao lan truyền hiệu quả?

3. Cá nhân hoá

- Mỗi người dùng thấy một bảng tin khác nhau
- Xếp hạng phụ thuộc vào lịch sử người dùng
- Không thể lưu đệm đơn giản

4. Đánh đổi về tính nhất quán

- Người dùng đăng bài → Bao lâu thì người theo dõi thấy?
- Nhất quán cuối cùng có chấp nhận được không?

5. Bài toán người nổi tiếng

- Người dùng có 100M người theo dõi
- 1 bài đăng = 100M lượt cập nhật bảng tin?

Điểm mấu chốt: Hệ thống bảng tin buộc bạn đối mặt với mọi thách thức của hệ thống phân tán (*distributed systems*) cùng một lúc.

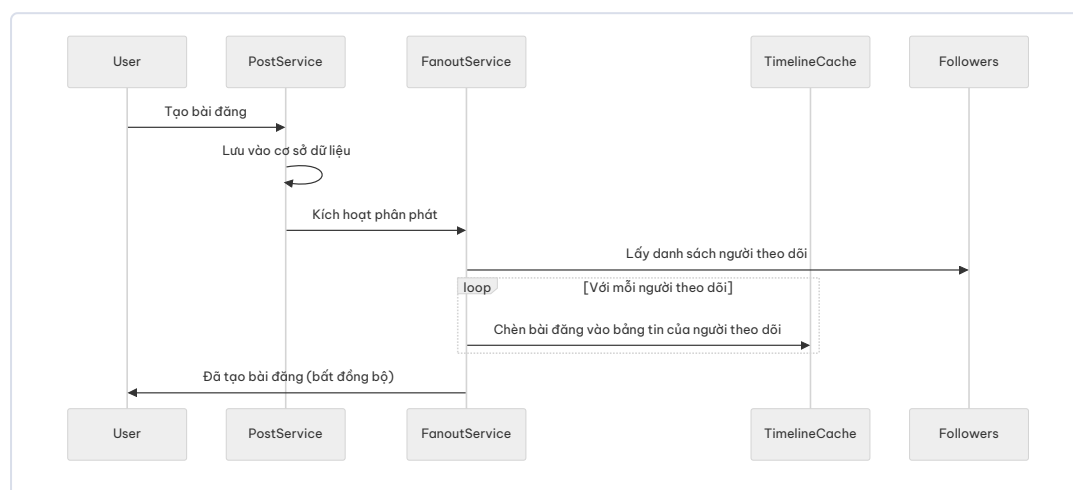
Các Chiến Lược Phân Phát: Quyết Định Kiến Trúc Cốt Lõi

Phân phát = Cách đưa 1 bài đăng đến bảng tin (*feeds*) của những người theo dõi (*followers*)

Có 3 cách tiếp cận khác nhau về căn bản. Mỗi cách tiếp cận có những sự đánh đổi (*trade-offs*) sâu sắc.

Cách Tiếp Cận 1: Phân Phát Khi Ghi (*Fanout on Write*) (Mô Hình Đẩy (*Push Model*))

Ý tưởng: Tính trước (*Pre-compute*) bảng tin khi người dùng đăng bài



Phân phát khi ghi: Khi người dùng đăng bài, lập tức đẩy vào bảng tin của tất cả người theo dõi

Triển khai:

```

class FanoutOnWriteService:
    def create_post(self, user_id, content):
        # 1. Save post to database
        post = self.db.create_post({
            'user_id': user_id,
            'content': content,
            'created_at': datetime.now()
        })

        # 2. Get followers
        follower_ids = self.db.get_followers(user_id)

        # 3. Fanout to all followers (async via queue)
        for follower_id in follower_ids:
            self.queue.publish({
                'task': 'insert_into_timeline',
                'follower_id': follower_id,
                'post_id': post.id
            })

        return {'status': 'success', 'post_id': post.id}

class FanoutWorker:
    def process_fanout(self, message):
        follower_id = message['follower_id']
        post_id = message['post_id']

        # Insert into follower's timeline cache (Redis sorted set)
        self.redis.zadd(
            f"timeline:{follower_id}",
            {post_id: timestamp}
        )

        # Keep only recent 1000 posts
        self.redis.zremrangebyrank(f"timeline:{follower_id}", 0, -1001)

# Read feed (extremely fast!)
class FeedService:
    def get_feed(self, user_id, page=1, size=20):
        # Read from pre-computed cache
        post_ids = self.redis.zrevrange(
            f"timeline:{user_id}",
            (page-1)*size,
            page*size-1
        )

        # Fetch post details
        posts = self.db.get_posts(post_ids)

        return posts

```


Sự đánh đổi:

Ưu điểm:

- Đọc cực nhanh ($< 10\text{ms}$)
- Bảng tin đã được tính trước
- Logic đọc đơn giản
- Việc đọc dễ mở rộng

Nhược điểm:

- Khuếch đại ghi (1 bài đăng $\rightarrow$ N tác vụ phân phát)
- Bài toán người nổi tiếng (1 bài đăng $\rightarrow$ 100M lượt cập nhật)
- Lãng phí công sức (người dùng không hoạt động vẫn nhận cập nhật)
- Chi phí lưu trữ ($N \times$ số bài đăng trung bình mỗi người dùng)
- Độ trễ phân phát (nhất quán cuối cùng)

Tính toán năng lực (capacity):

Người dùng thường đăng bài:

- Có 1.000 người theo dõi
- 1 bài đăng $\rightarrow$ 1.000 lượt ghi phân phát

Người nổi tiếng đăng bài:

- Có 10M người theo dõi
- 1 bài đăng $\rightarrow$ 10M lượt ghi phân phát
- Mất vài phút mới hoàn tất

Toàn nền tảng:

- 100M người dùng đăng 2 lần/ngày = 200M bài đăng/ngày
- Trung bình 200 người theo dõi
- Tổng phân phát: $200\text{M} \times 200 = 40$ tỷ lượt ghi/ngày
- $40\text{B} / 86400\text{s} \approx 460\text{K}$ lượt ghi/giây

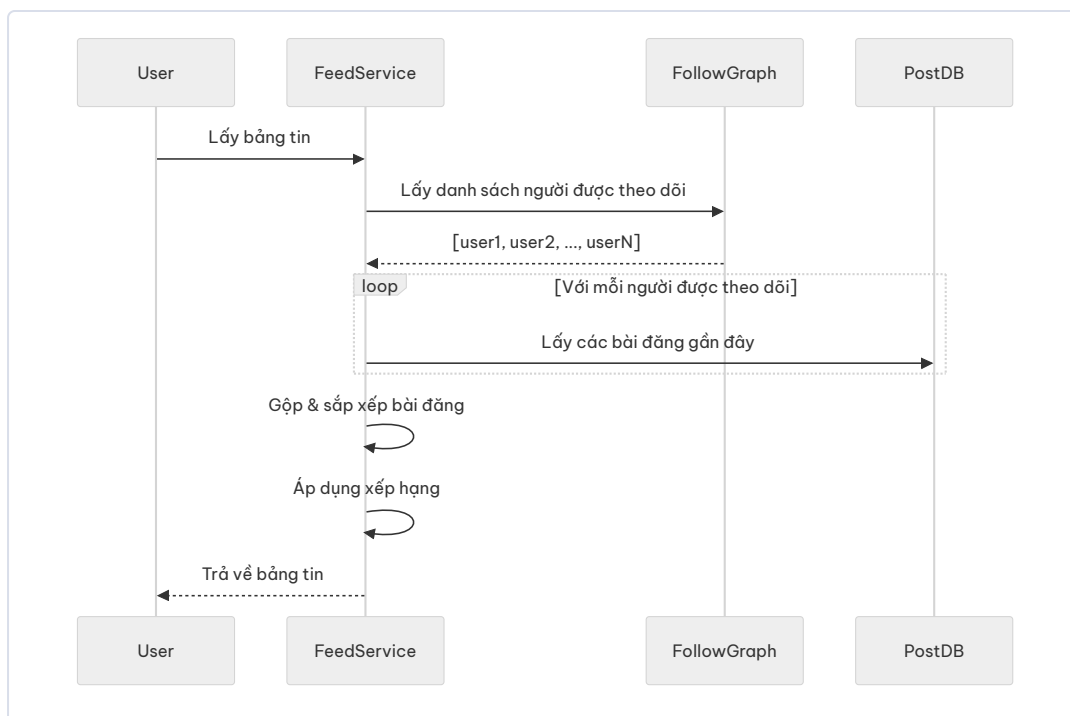
Cần năng lực ghi cực lớn!

Khi nào dùng:

- ✓ Đa số người dùng có < 5.000 người theo dõi
- ✓ Hiệu năng đọc là then chốt ($< 100\text{ms}$)
- ✓ Chịu được khuếch đại ghi
- ✓ Lưu trữ không phải là ràng buộc
- ✓ Nhất quán cuối cùng là chấp nhận được

Cách Tiếp Cận 2: Phân Phát Khi Đọc (Fanout on Read) (Mô Hình Kéo (Pull Model))

Ý tưởng: Tạo bảng tin khi người dùng yêu cầu (requests)



Phân phát khi đọc: Khi người dùng mở bảng tin, truy vấn bài đăng từ tất cả những người được theo dõi theo thời gian thực (*real-time*)

Triển khai:

```

class FanoutOnReadService:
    def create_post(self, user_id, content):
        # Simply save post, no fanout
        post = self.db.create_post({
            'user_id': user_id,
            'content': content,
            'created_at': datetime.now()
        })

        return {'status': 'success', 'post_id': post.id}

    def get_feed(self, user_id, page=1, size=20):
        # 1. Get followees
        followee_ids = self.db.get_followees(user_id)

        # 2. Get recent posts from each followee
        all_posts = []
        for followee_id in followee_ids:
            posts = self.cache.get(f"user_posts:{followee_id}")
            if not posts:
                # Cache miss, query DB
                posts = self.db.get_user_posts(
                    followee_id,
                    limit=100 # Recent 100 posts
                )
                self.cache.set(f"user_posts:{followee_id}", posts, ttl=300)

            all_posts.extend(posts)

        # 3. Merge and sort
        all_posts.sort(key=lambda p: p['created_at'], reverse=True)

        # 4. Paginate
        start = (page-1) * size
        end = start + size

        return all_posts[start:end]

```

Sự đánh đổi:

Ưu điểm:

- Không khuếch đại ghi (chỉ lưu bài đăng)
- Ghi nhanh
- Không có độ trễ~phân phát (luôn mới nhất)
- Không lãng phí công sức
- Không có bài toán người nổi tiếng
- Chi phí lưu trữ thấp

Nhược điểm:

- Khuếch đại đọc (truy vấn nhiều người dùng)
- Đọc chậm (100-500ms)
- Khó mở rộng việc đọc
- Chiến lược lưu đệm phức tạp
- Tải lên cơ sở dữ liệu cao

Tính toán hiệu năng (performance):

Người dùng tải bảng tin:

- Theo dõi 200 người
- Truy vấn các bài đăng gần đây của từng người
- 200 truy vấn (kể cả khi gộp lô: 10-20 truy vấn)
- Gộp 4.000 bài đăng
- Sắp xếp
- Tổng: 200-500ms

1M người dùng đồng thời:

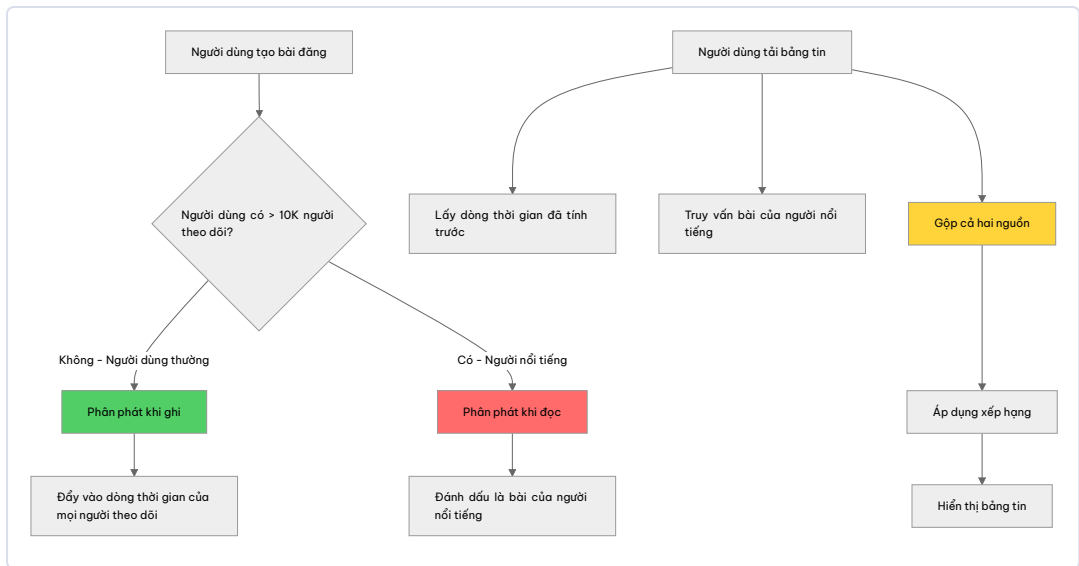
- $1M \times 200$ truy vấn = 200M truy vấn
- Cơ sở dữ liệu không chịu nổi

Khi nào dùng:

- ✓ Quy mô nhỏ (< 100K người dùng)
- ✓ Người dùng theo dõi ít người (< 50)
- ✓ Hiệu năng ghi là then chốt
- ✓ Yêu cầu nhất quán mạnh
- ✓ Hạ tầng hạn chế

Cách Tiếp Cận 3: Phân Phát Lai (Hybrid Fanout) (Kết Hợp Ưu Điểm Của Cả Hai)

Ý tưởng: Phân phát khi ghi cho người dùng thông thường, phân phát khi đọc cho người nổi tiếng



Phân phát lại: Kết hợp cả hai chiến lược dựa trên số lượng người theo dõi (follower)

Triển khai:

```

class HybridFanoutService:
    CELEBRITY_THRESHOLD = 10_000 # 10K followers

    def create_post(self, user_id, content):
        # Save post
        post = self.db.create_post({
            'user_id': user_id,
            'content': content,
            'created_at': datetime.now()
        })

        # Check if celebrity
        follower_count = self.db.get_follower_count(user_id)

        if follower_count < self.CELEBRITY_THRESHOLD:
            # Normal user: Fanout on write
            self._fanout_to_followers(user_id, post.id)
        else:
            # Celebrity: Mark for fanout on read
            self.redis.sadd('celebrity_users', user_id)
            # Store in celebrity posts cache
            self.redis.zadd(
                f"celebrity_posts:{user_id}",
                {post.id: post.created_at.timestamp()}
            )

        return {'status': 'success', 'post_id': post.id}

    def get_feed(self, user_id, page=1, size=20):
        # 1. Get pre-computed timeline (normal users)
        timeline_posts = self.redis.zrevrange(
            f"timeline:{user_id}",
            0,
            100 # Get more than needed
        )

        # 2. Get celebrity posts (users this user follows)
        celebrity_followees = self._get_celebrity_followees(user_id)
        celebrity_posts = []

        for celebrity_id in celebrity_followees:
            posts = self.redis.zrevrange(
                f"celebrity_posts:{celebrity_id}",
                0,
                20 # Recent 20 posts
            )
            celebrity_posts.extend(posts)

        # 3. Merge both sources
        all_post_ids = timeline_posts + celebrity_posts

        # 4. Fetch post details
        posts = self.db.get_posts(all_post_ids)

```

```

# 5. Sort by timestamp
posts.sort(key=lambda p: p['created_at'], reverse=True)

# 6. Apply ranking algorithm
ranked_posts = self.ranking_service.rank(posts, user_id)

# 7. Paginate
start = (page-1) * size
end = start + size

return ranked_posts[start:end]

def _get_celebrity_followees(self, user_id):
    """Get list of celebrities this user follows"""
    followees = self.db.get_followees(user_id)
    celebrities = self.redis.smembers('celebrity_users')
    return [f for f in followees if f in celebrities]

```

Sự đánh đổi:

Ưu điểm:

- Đọc nhanh (tính trước cho đa số)
- Không bùng nổ phân phát vì người nổi tiếng
- Tải ghi cân bằng
- Linh hoạt (tính chỉnh ngưỡng)
- Kết hợp ưu điểm của cả hai

Nhược điểm:

- Triển khai phức tạp nhất
- Cần duy trì danh sách người nổi tiếng
- Bài của người nổi tiếng bị trễ một chút
- Nhiều thành phần chuyển động hơn
- Khó gỡ lỗi hơn

Các ngưỡng trong thực tế:

Twitter (2015):

- Ngưỡng phân phát: 2.000 người theo dõi
- Trên ngưỡng → Phân phát khi đọc

Facebook (ước tính):

- Ngưỡng phân phát: 5.000 người theo dõi
- Trên ngưỡng → Cách tiếp cận lại

Instagram (ước tính):

- Ngưỡng phân phát: 10.000 người theo dõi
- Tài khoản đã xác minh → Luôn phân phát khi đọc

Khi nào dùng:

- ✓ Quy mô lớn (> 1M người dùng)
- ✓ Có cả người dùng thường và người nổi tiếng
- ✓ Cần tối ưu cả đọc và ghi
- ✓ Chịu được độ phức tạp
- ✓ Có đủ nguồn lực kỹ thuật

Đây là cách Facebook, Twitter, Instagram đang làm!

Thiết Kế Bộ Nhớ Đệm Dòng Thời Gian (*Timeline Cache*)

Bộ nhớ đệm dòng thời gian là trái tim của hệ thống bảng tin.

Cấu Trúc Bộ Nhớ Đệm (*Cache*)

Redis Sorted Set (tập hợp có thứ tự) = Hoàn toàn phù hợp

```
# Structure
Key: "timeline:{user_id}"
Score: post_timestamp (for sorting)
Member: post_id

# Example
timeline:user123 = {
  post_456: 1709567890000, # Score = timestamp in ms
  post_457: 1709567891000,
  post_458: 1709567892000,
  ...
}
```

Tại sao lại là Sorted Set?

Tự động sắp xếp theo dấu thời gian
Chèn với độ phức tạp $O(\log N)$
Truy vấn theo khoảng (phân trang)
Dễ dàng cắt bớt bài đăng cũ
Sử dụng bộ nhớ hiệu quả

Các thao tác:


```

# Insert post into timeline
redis.zadd(
    "timeline:user123",
    {post_id: timestamp}
)

# Get paginated feed (20 posts per page)
post_ids = redis.zrevrange(
    "timeline:user123",
    start=0,
    end=19,
    withscores=True
)

# Trim to keep only recent 1000 posts
redis.zremrangebyrank(
    "timeline:user123",
    0,
    -1001 # Keep rank 1000 to end
)

# Get posts after certain timestamp (pull to refresh)
post_ids = redis.zrevrangebyscore(
    "timeline:user123",
    max="+inf",
    min=last_seen_timestamp
)

```

Định Cỡ Bộ Nhớ Đệm

Tính toán bộ nhớ (*memory*):

Mỗi người dùng:

- Lưu 1.000 bài đăng gần nhất trong dòng thời gian
- Mỗi mục: 8 byte (post_id) + 8 byte (timestamp) = 16 byte
- Mỗi người dùng: $1.000 \times 16 = 16 \text{ KB}$

100M người dùng:

- $100M \times 16 \text{ KB} = 1,6 \text{ TB RAM}$

Với cụm Redis (đã phân mảnh):

- 100 nút $\times 16 \text{ GB RAM}$ mỗi nút = 1,6 TB
- Chi phí: ~\$10.000/tháng (AWS ElastiCache)

Đắt đỏ nhưng cần thiết!

Vô Hiệu Hoá Bộ Nhớ Đệm (Cache Invalidation)

Vấn đề:

Người dùng đăng bài
→ Phân phát tới 1.000 người theo dõi
→ Mất 5 giây
→ Một số người theo dõi thấy bài ngay lập tức
→ Một số phải chờ 5 giây

Nhất quán cuối cùng!

Chiến lược:

```
class TimelineCacheManager:
    def invalidate_after_post(self, user_id, post_id):
        # Don't invalidate, just ensure freshness
        # Timeline cache is append-only
        pass

    def invalidate_after_delete(self, user_id, post_id):
        # Post deleted → Remove from all followers' timelines
        followers = self.db.get_followers(user_id)

        for follower_id in followers:
            self.redis.zrem(
                f"timeline:{follower_id}",
                post_id
            )

    def invalidate_after_unfollow(self, user_id, unfollowed_id):
        # User unfollows someone → Remove their posts from timeline
        posts = self.redis.zrange(f"celebrity_posts:{unfollowed_id}", 0, -1)

        for post_id in posts:
            self.redis.zrem(
                f"timeline:{user_id}",
                post_id
            )
```

Các Thuật Toán Xếp Hạng (Ranking)

Sắp xếp theo thứ tự thời gian là không đủ. Cần có xếp hạng.

Tại Sao Cần Xếp Hạng?

Vấn đề khi chỉ sắp xếp theo thời gian:

- Người dùng theo dõi 500 người
- Có người đăng 50 lần/ngày
- Có người đăng 1 lần/tuần
- Bảng tin bị những kẻ spam chiếm lĩnh
- Bỏ lỡ bài đăng quan trọng từ bạn thân

Câu: Hiển thị bài đăng liên quan nhất trước

Xếp Hạng Đơn Giản: Dựa Trên Mức Độ Tương Tác (*Engagement*)

```
def calculate_engagement_score(post):
    """Simple engagement-based ranking"""

    # Age decay
    age_hours = (datetime.now() - post.created_at).total_seconds() / 3600
    age_factor = 1 / (age_hours + 2) ** 1.5 # Exponential decay

    # Engagement signals
    engagement = (
        post.likes * 1.0 +
        post.comments * 2.0 + # Comments more valuable
        post.shares * 3.0      # Shares most valuable
    )

    # Combine
    score = engagement * age_factor

    return score

# Apply ranking
def rank_posts(posts):
    for post in posts:
        post.score = calculate_engagement_score(post)

    # Sort by score descending
    posts.sort(key=lambda p: p.score, reverse=True)

    return posts
```

Sự đánh đổi:

Dễ triển khai

Khuyến khích nội dung có tính tương tác

Hệ số suy giảm theo thời gian ngắn bài cũ nổi lên

Giàu càng giàu thêm (bài phổ biến cứ nằm trên đầu)

Không tính đến sở thích của người dùng

Có thể bị thao túng (tương tác giả)

Xếp Hạng Nâng Cao: Cá Nhân Hoá

```
class PersonalizedRanking:
    def rank(self, posts, user_id):
        """ML-based personalized ranking"""

        # Get user features
        user_profile = self.get_user_profile(user_id)

        scored_posts = []
        for post in posts:
            # Extract post features
            features = self._extract_features(post, user_profile)

            # ML model prediction
            score = self.model.predict(features)

            scored_posts.append((post, score))

        # Sort by predicted score
        scored_posts.sort(key=lambda x: x[1], reverse=True)

        return [post for post, score in scored_posts]

    def _extract_features(self, post, user_profile):
        """Extract features for ML model"""
        return {
            # Time features
            'age_hours': (datetime.now() - post.created_at).hours,
            'hour_of_day': post.created_at.hour,

            # Engagement features
            'like_count': post.likes,
            'comment_count': post.comments,
            'share_count': post.shares,
            'engagement_rate': post.likes / max(post.impressions, 1),

            # Author features
            'author_follower_count': post.author.followers,
            'user_follows_author': user_profile.follows(post.author_id),
            'past_engagement_with_author':
                user_profile.engagement_with(post.author_id),

            # Content features
            'has_image': post.has_image,
            'has_video': post.has_video,
            'text_length': len(post.content),
            'contains_hashtags': '#' in post.content,

            # User preference features
            'topic_match': self._topic_similarity(post.topics,
                user_profile.interests),
```

```

    'language_match': post.language == user_profile.language
}

```

Quy trình học máy (ML Pipeline) (mức tổng quan):



Quy trình xếp hạng bằng học máy (ML ranking pipeline): Học liên tục từ các tương tác của người dùng (*user interactions*)

Độ phức tạp trong thực tế:

News Feed của Facebook sử dụng:

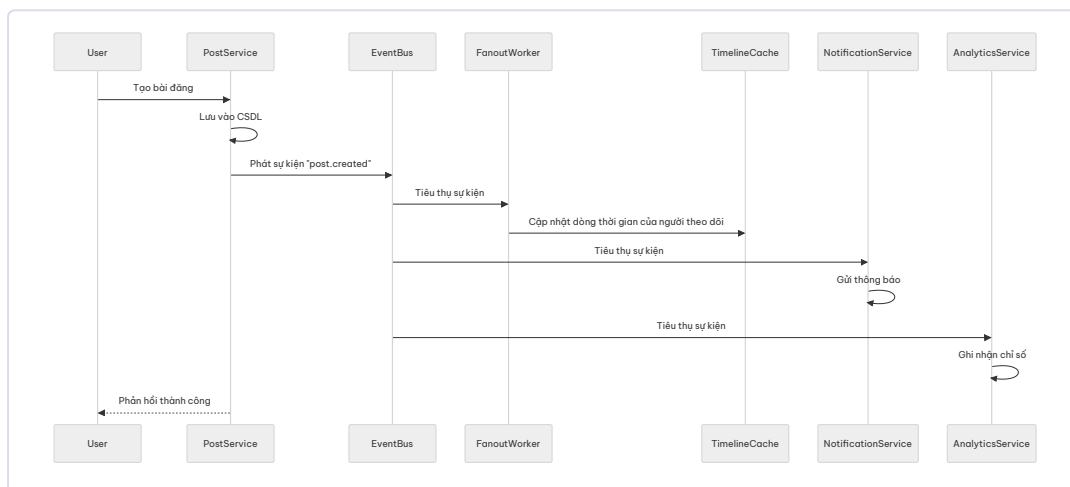
- Hơn 100.000 đặc trưng
- Tổ hợp nhiều mô hình
- Cá nhân hoá theo thời gian thực
- Khung thử nghiệm A/B
- Liên tục cải tiến

Không hề đơn giản!

Kiến Trúc Hướng Sự Kiện (*Event-Driven*)

Hệ thống băng tin thực chất là các hệ thống hướng sự kiện.

Lưuồng Sự Kiện (*Event*)



Hướng sự kiện: Một hành động kích hoạt nhiều tiến trình bất đồng bộ (async)

Cấu Trúc Sự Kiện

```
# Post created event
{
  "event_type": "post.created",
  "event_id": "evt_123456",
  "timestamp": 1709567890000,
  "data": {
    "post_id": "post_456",
    "author_id": "user_123",
    "content": "Hello world!",
    "media_urls": ["https://..."],
    "created_at": 1709567890000
  },
  "metadata": {
    "source": "mobile_app",
    "version": "v2.1"
  }
}

# Like event
{
  "event_type": "post.liked",
  "event_id": "evt_123457",
  "timestamp": 1709567891000,
  "data": {
    "post_id": "post_456",
    "user_id": "user_789",
    "liked_at": 1709567891000
  }
}

# Comment event
{
  "event_type": "post.commentated",
  "event_id": "evt_123458",
  "timestamp": 1709567892000,
  "data": {
    "post_id": "post_456",
    "comment_id": "cmt_789",
    "author_id": "user_999",
    "content": "Nice post!",
    "created_at": 1709567892000
  }
}
```

Các Bên Tiêu Thụ (Consumers) Sự Kiện

```
# Fanout consumer
class FanoutEventConsumer:
    def handle_post_created(self, event):
        post_id = event['data']['post_id']
        author_id = event['data']['author_id']
        timestamp = event['data']['created_at']

        # Get followers
        followers = self.db.get_followers(author_id)

        # Fanout
        for follower_id in followers:
            self.redis.zadd(
                f"timeline:{follower_id}",
                {post_id: timestamp}
            )

# Notification consumer
class NotificationEventConsumer:
    def handle_post_liked(self, event):
        post_id = event['data']['post_id']
        liker_id = event['data']['user_id']

        # Get post author
        post = self.db.get_post(post_id)
        author_id = post.author_id

        # Send notification
        if author_id != liker_id: # Don't notify self
            self.notification_service.send({
                'user_id': author_id,
                'type': 'like',
                'actor_id': liker_id,
                'post_id': post_id
            })

# Analytics consumer
class AnalyticsEventConsumer:
    def handle_all_events(self, event):
        # Stream to analytics pipeline
        self.kinesis.put_record(event)

        # Update real-time counters
        if event['event_type'] == 'post.created':
            self.redis.incr('metrics:posts_created:today')
        elif event['event_type'] == 'post.liked':
            self.redis.incr('metrics:likes:today')
```


Lợi Ích Của Kiến Trúc Hướng Sự Kiện

Tách rời

- Các dịch vụ không cần biết về nhau
- Dễ thêm bên tiêu thụ mới
- Triển khai độc lập

Khả năng mở rộng

- Xử lý bất đồng bộ
- Có thể mở rộng từng bên tiêu thụ một cách độc lập
- Hàng đợi hấp thụ lưu lượng tăng đột biến

Tính linh hoạt

- Thêm tính năng mà không đụng vào phần lõi
- Thử nghiệm A/B dễ dàng hơn
- Hoàn tác (rollback) đơn giản hơn

Độ tin cậy

- Thử lại các sự kiện bị lỗi
- Hàng đợi thư chết (dead letter queue)
- Gửi ít nhất một lần (at-least-once)

Độ phức tạp

- Nhiều thành phần chuyển động hơn
- Nhất quán cuối cùng
- Gỡ lỗi khó hơn
- Thách thức về thứ tự

Chiến Lược Phân Trang (Pagination)

Cuộn vô hạn (*Infinite scroll*) = **phân trang đầy mọo mực**.

Phân Trang Dựa Trên Offset (Offset-Based Pagination) (Đừng Dùng!)

```
# BAD: Offset-based
def get_feed_offset(user_id, page, size=20):
    offset = (page - 1) * size

    post_ids = redis.zrevrange(
        f"timeline:{user_id}",
        offset,
        offset + size - 1
    )

    return post_ids

# Problems:
# 1. Duplicates if new posts added while paginating
# 2. Missing posts if posts deleted
# 3. Inefficient for large offsets (O(N))
```

Phân Trang Dựa Trên Con Trỏ (Cursor-Based Pagination) (Hãy Dùng Cách Này!)

```
# GOOD: Cursor-based
def get_feed_cursor(user_id, cursor=None, size=20):
    if cursor is None:
        # First page: Start from highest score
        cursor = "+inf"

    # Get posts with score < cursor
    post_ids = redis.zrevrangebyscore(
        f"timeline:{user_id}",
        max=cursor,
        min="-inf",
        start=0,
        num=size + 1, # Fetch one extra to check if more
        withscores=True
    )

    has_more = len(post_ids) > size
    if has_more:
        post_ids = post_ids[:size]

    # Next cursor = smallest score in this page
    next_cursor = post_ids[-1][1] if post_ids else None

    return {
        'posts': [pid for pid, score in post_ids],
        'next_cursor': next_cursor,
        'has_more': has_more
    }

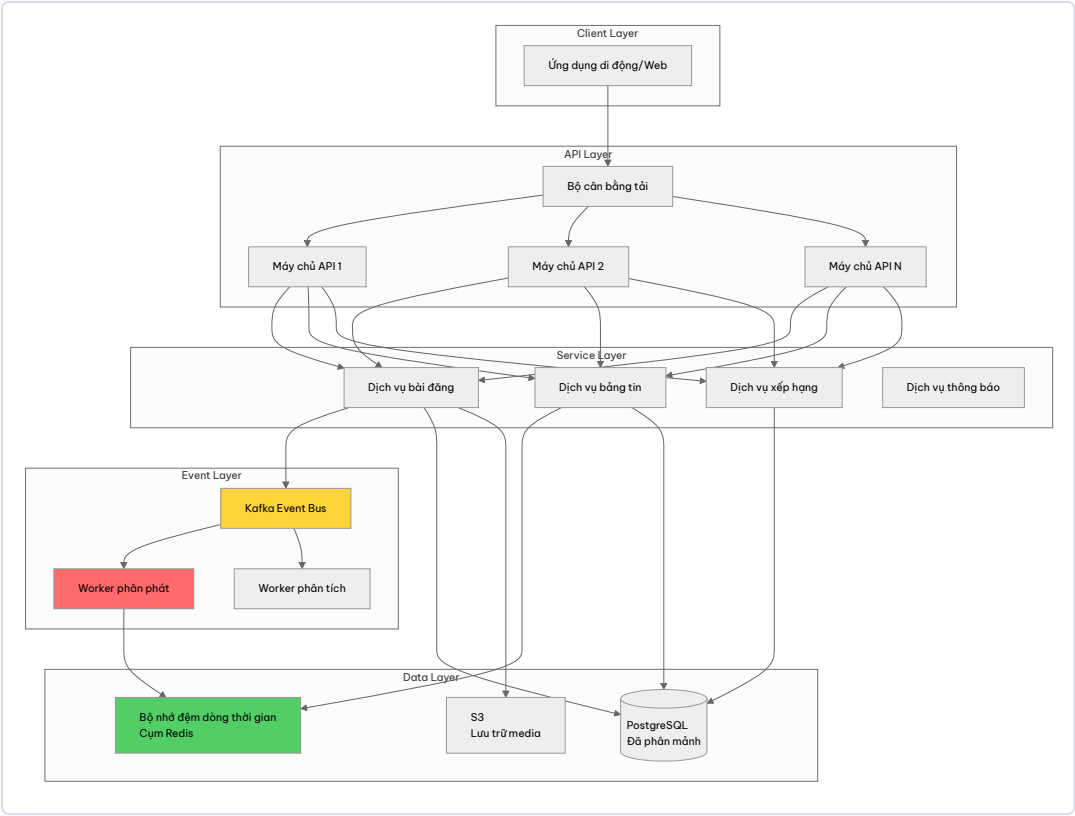
# Client usage:
# Page 1: GET /feed?size=20
# Response: {posts: [...], next_cursor: 1709567890000}
#
# Page 2: GET /feed?cursor=1709567890000&size=20
# Response: {posts: [...], next_cursor: 1709567850000}
```

Tại sao phân trang dựa trên con trỏ (cursor-based) chiến thắng:

- Không trùng lặp
- Không sót bài đăng
- Hiệu quả (luôn $O(\log N)$)
- Hoạt động tốt với cập nhật thời gian thực
- Phi trạng thái (con trỏ chứa mọi thông tin)

Tích Hợp: Ghép Tất Cả Lại Với Nhau

Kiến trúc thực tế cho 100 triệu người dùng:



Kiến trúc hoàn chỉnh của hệ thống bảng tin với thiết kế hướng sự kiện

Hoạch Định Năng Lực (*Capacity Planning*)

Nền tảng: 100M người dùng hoạt động hằng tháng
Hoạt động hằng ngày: 30M người dùng
Bài đăng/ngày: 60M bài (trung bình 2 bài/người dùng)
Số người theo dõi trung bình: 200

Tải ghi:

- 60M bài đăng/ngày
- Phân phát: $60M \times 200 = 12B$ lượt ghi dòng thời gian/ngày
- $12B / 86400s \approx 140K$ lượt ghi/giây

Tải đọc:

- 30M người dùng mở bảng tin 10 lần/ngày = $300M$ lượt tải bảng tin/ngày
- $300M / 86400s \approx 3.500$ lượt đọc/giây

Hạ tầng:

- Máy chủ API: 200 instance
- Worker phân phát: 500 instance
- Cụm Redis: 100 nút (tổng 1,6TB RAM)
- PostgreSQL: 50 mảnh
- Kafka: 20 broker
- S3: Không giới hạn (lưu trữ media)

Chi phí (hàng tháng, AWS):

- Tính toán (EC2): \$50.000
- Redis (ElastiCache): \$10.000
- RDS (PostgreSQL): \$30.000
- S3: \$5.000
- Kafka (MSK): \$8.000
- Truyền tải dữ liệu: \$15.000

Tổng: ~\$120.000/tháng

Không rẻ, nhưng cần thiết ở quy mô lớn!

Những Điểm Cốt Lõi

Độ phức tạp của hệ thống bảng tin:

Kiến trúc phức tạp nhất trong các ứng dụng tiêu dùng vì:

- Khuếch đại đọc và ghi
- Cá nhân hoá ở quy mô lớn
- Yêu cầu thời gian thực
- Bài toán người nổi tiếng
- Nhất quán cuối cùng

Các chiến lược phân phát:

Phân phát khi ghi:

- Tính trước bảng tin
- Đọc nhanh
- Khuếch đại ghi
- Phù hợp cho người dùng thường

Phân phát khi đọc:

- Truy vấn khi cần
- Ghi nhanh
- Khuếch đại đọc
- Phù hợp cho người nổi tiếng

Lai (Tốt nhất):

- Kết hợp cả hai
- Tối ưu dựa trên số người theo dõi
- Facebook, Twitter, Instagram dùng cách này

Các mẫu kiến trúc (*architecture patterns*):

- ✓ Hướng sự kiện (tách rời các dịch vụ)
- ✓ Bộ nhớ đệm dòng thời gian (Redis sorted set)
- ✓ Phân trang dựa trên con trỏ (tránh offset)
- ✓ Phân phát bất đồng bộ (dựa trên hàng đợi)
- ✓ Xếp hạng cá nhân hoá (quy trình học máy)

Luôn luôn là sự đánh đổi:

Tính nhất quán và Độ trễ:

- Nhất quán cuối cùng là chấp nhận được
- Độ trễ phân phát là ổn (vài giây)

Lưu trữ và Tính toán:

- Lưu đệm dòng thời gian (chi phí lưu trữ)
- Hoặc truy vấn khi cần (chi phí tính toán)

Đơn giản và Tối ưu:

- Bắt đầu đơn giản (phân phát khi ghi)
- Thêm độ phức tạp khi đã chứng minh là cần (lai)

Bài học rút ra từ kinh nghiệm:

1. **Bắt đầu đơn giản:** Phân phát khi ghi, cải tiến dần về sau
2. **Đo lường mọi thứ:** Biết điểm nghẽn (*bottleneck*) nằm ở đâu
3. **Nhất quán cuối cùng** (*Eventual consistency*) **là ổn:** Người dùng không nhận ra độ trễ 2 giây
4. **Bộ nhớ đệm là vua:** Tính trước càng nhiều càng tốt

5. Kiến trúc hướng sự kiện để mở rộng: Tách rời các dịch vụ (*services*) từ sớm

Hệ thống bảng tin buộc bạn đối mặt với mọi thách thức của hệ thống phân tán.
Làm chủ bảng tin = Làm chủ tư duy về hệ thống phân tán.

Đây là lý do Kỹ sư bảng tin (*Feed Engineer*) là một vị trí chuyên biệt tại các công ty FAANG.

PHẦN 6

Làm Chủ Thiết Kế Hệ Thống

Giai đoạn làm chủ: tư duy kiến trúc sư, ra quyết định kiến trúc, chiến lược phỏng vấn và làm chủ hệ thống vận hành thực tế.

1) Giới Thiệu Phần

Phần 6 là giai đoạn chuyển từ "biết các mẫu (*pattern*)" sang "ra quyết định kiến trúc".

Làm chủ thiết kế hệ thống không nằm ở việc nhớ nhiều mô hình, mà nằm ở khả năng chọn đúng giải pháp theo bối cảnh thực tế.

Ở cấp độ lâu năm (*senior*), kiến trúc là bài toán về sự đánh đổi (*trade-off*), ràng buộc (*constraint*) và trách nhiệm trong môi trường vận hành thực tế (*production*). Đây là nơi bạn hình thành tư duy ở tầm kiến trúc sư.

2) Vì Sao Phần Này Tồn Tại

Khoảng cách lớn nhất ở cấp độ cao không phải là thiếu kiến thức, mà là thiếu năng lực ra quyết định có cấu trúc.

Phần này tồn tại để lấp khoảng cách đó:

- Từ "biết nhiều mẫu" -> "chọn đúng mẫu theo bài toán".
- Từ "trả lời rời rạc" -> "tư duy theo khung (*framework*) rõ ràng".
- Từ "thiết kế trên giấy" -> "làm chủ (*ownership*) trong môi trường vận hành thực tế".
- Từ "kỹ sư giỏi cài đặt (*implementation*)" -> "kỹ sư lâu năm có khả năng dẫn dắt kiến trúc".

3) Mục Tiêu Học Tập

- Áp dụng khung tư duy thiết kế hệ thống có cấu trúc.
- Ra quyết định kiến trúc dựa trên sự đánh đổi và ràng buộc.
- Thiết kế hệ thống trong điều kiện thiếu thông tin và nhiều bất định.
- Nâng cao năng lực xử lý phỏng vấn thiết kế hệ thống (*system design interview*) một cách tự tin.
- Lý giải các quyết định kỹ thuật ở cấp độ kỹ sư lâu năm/kỹ sư cấp cao (*staff*).

- Thiết kế hệ thống sẵn sàng cho vận hành thực tế (*production-ready*) với tư duy làm chủ.
- Giao tiếp về kiến trúc rõ ràng với nhóm và các bên liên quan (*stakeholder*).

4) Bạn Sẽ Học Gì Trong Phần Này

- **Khung tư duy thiết kế hệ thống** (*System design thinking framework*): chuyển từ cấp độ mẫu sang tư duy giải quyết vấn đề.
- **Phương pháp có cấu trúc**: quy trình giải bài theo từng bước rõ ràng.
- **Ra quyết định dựa trên đánh đổi**: cách so sánh và chốt quyết định kiến trúc ở cấp hệ thống.
- **Chiến lược thực hiện buổi phỏng vấn**: cách dẫn dắt buổi phỏng vấn có nhịp độ, có chiều sâu.
- **Các chủ đề nâng cao**: mở rộng kiến thức cho các tình huống khó và các lĩnh vực (*domain*) đặc thù.
- **Tư duy kiến trúc sư** (*Architect mindset*): tư duy chịu trách nhiệm về độ tin cậy (*reliability*), khả năng vận hành (*operability*), chi phí và sự tiến hoá.
- **Lập luận dưới ràng buộc** (*constraints*): ra quyết định trong giới hạn về thời gian, dữ liệu và nguồn lực.
- **Giao tiếp về kiến trúc**: trình bày và bảo vệ kiến trúc một cách rõ ràng, thuyết phục.

5) Tổng Quan Các Bài Học

Bài 1 – Khung Tư Duy Thiết Kế Hệ Thống – Từ Mẫu Đến Tư Duy Giải Quyết Vấn Đề

- **Mô tả**: xây dựng khung tư duy để giải bài toán hệ thống theo ngữ cảnh thay vì theo khuôn mẫu.
- **Bạn nhận được**: khả năng lập luận có cấu trúc, đi từ yêu cầu đến quyết định.

Bài 2 – Khung SNAKE – Phương Pháp Để Chinh Phục Phỏng Vấn Thiết Kế Hệ Thống

- **Mô tả:** khung tư duy thực hành giúp điều hướng buổi phỏng vấn rõ ràng và mạch lạc.
- **Bạn nhận được:** phương pháp trả lời ổn định, bớt rối khi gặp bài toán mới.

Bài 3 – Đánh Đổi (Trade-offs) & Ra Quyết Định Kiến Trúc – Tư Duy Của Kiến Trúc Sư Lâu Năm (Senior Architect)

- **Mô tả:** đi sâu vào cách cân bằng độ trễ (latency), tính nhất quán (consistency), chi phí, độ phức tạp và tốc độ phát triển.
- **Bạn nhận được:** năng lực chốt quyết định kiến trúc ở cấp độ lâu năm.

Bài 4 – Chiến Lược Phỏng Vấn Thiết Kế Hệ Thống Thực Chiến

- **Mô tả:** chiến lược thực chiến cho phỏng vấn thiết kế hệ thống: phạm vi (scope), nhịp trình bày, đào sâu (deep dive), cách gỡ lại khi bí.
- **Bạn nhận được:** khả năng triển khai buổi phỏng vấn có kiểm soát và tự tin hơn.

Bài 5 – Các Chủ Đề Kiến Trúc Nâng Cao (Kiến Thức Chuyên Sâu Tùy Chọn)

- **Mô tả:** mở rộng kiến thức nâng cao cho các bài toán lớn và các lĩnh vực chuyên biệt.
- **Bạn nhận được:** chiều sâu bổ sung để xử lý các bài toán phức tạp hơn mức nền tảng.

Bài 6 – Tư Duy Kiến Trúc Sư & Tư Duy Vận Hành Thực Tế

- **Mô tả:** chuyển từ tư duy cài đặt sang làm chủ toàn bộ hệ thống.
- **Bạn nhận được:** tư duy chịu trách nhiệm về vận hành thực tế và tiến hoá kiến trúc dài hạn.

6) Kết Quả Sau Khi Hoàn Thành Phần 6

Bạn có thể:

- Tư duy như một kiến trúc sư hệ thống khi phân tích bài toán.
- Tự tin thiết kế hệ thống dưới các ràng buộc thực tế.
- Ra quyết định kiến trúc với lập luận đánh đổi rõ ràng.
- Trình bày và bảo vệ thiết kế một cách mạch lạc trước người nghe chuyên môn kỹ thuật.
- Xử lý phỏng vấn thiết kế hệ thống ở mức chuyên sâu hơn.
- Thiết kế hệ thống đạt chuẩn vận hành thực tế (*production-grade*), có thể vận hành và tiến hoá.
- Chủ động nhận trách nhiệm làm chủ các hệ thống lớn và phức tạp.

7) Góc Nhìn Khi Hoàn Thành Chương Trình

Phần 6 không phải là điểm kết thúc, mà là điểm khởi đầu của **hành trình làm chủ không ngừng**.

Thiết kế hệ thống là một hành trình liên tục:

- liên tục cập nhật mô hình tư duy (*mental model*),
- liên tục nâng chất lượng các quyết định kiến trúc,
- liên tục học hỏi từ các hệ thống vận hành thực tế.

Sau phần này, bạn đi từ "người xây hệ thống" thành "người định hướng hệ thống".

Đây là bước tiến dài hạn trên lộ trình nghề nghiệp từ kỹ sư lên kiến trúc sư.

Khung Tư Duy Thiết Kế Hệ Thống (System Design Thinking Framework): Từ Biết Mẫu Thiết Kế (Pattern) Đến Biết Tư Duy

Chào mừng đến với Phần 6 - Làm Chủ Thiết Kế Hệ Thống (System Design).

Bạn đã đi qua 5 phần, học về các thành phần (components), hệ thống phân tán (distributed systems), các mẫu mở rộng (scalability patterns), các kiến trúc (architectures) ngoài thực tế. Bạn biết rất nhiều mẫu thiết kế (patterns).

Nhưng có một sự thật tôi phải nói thẳng:

Biết mẫu thiết kế không đồng nghĩa với giỏi thiết kế hệ thống.

Tôi từng phỏng vấn (interview) một ứng viên rất giỏi. Anh ta thuộc lòng tất cả các mẫu thiết kế: định lý CAP (CAP theorem), băm nhất quán (consistent hashing), CQRS, lưu trữ theo sự kiện (event sourcing)...

Tôi hỏi: "Hãy thiết kế một dịch vụ rút gọn URL."

Anh ta trả lời ngay lập tức: "Em sẽ dùng NoSQL, bộ nhớ đệm (cache) Redis, CDN, vi dịch vụ (microservices), kiến trúc hướng sự kiện (event-driven architecture)..."

Tôi: "Tại sao?"

Anh ta: "Vì... đó là thực hành tốt nhất (best practices) ạ?"

Anh ta trượt buổi phỏng vấn.

Không phải vì thiếu kiến thức. Mà vì **thiếu tư duy thiết kế hệ thống** (system design thinking).

Bài học này sẽ dạy bạn điều quan trọng nhất: **Cách tư duy khi tiếp cận một vấn đề.**

Tư Duy Thiết Kế Hệ Thống Là Gì?

Định Nghĩa

Tư duy thiết kế hệ thống = Khung tư duy (*Mental framework*) **để tiếp cận và giải quyết các bài toán kiến trúc** (*architecture*)

Không phải là:

- Thuộc lòng mẫu thiết kế
- Biết nhiều công nghệ
- Sao chép giải pháp

Mà là:

- Hiểu vấn đề một cách sâu sắc
- Lập luận từ nguyên lý gốc (*first principles*)
- Đưa ra các quyết định đánh đổi (*trade-off*)
- Lý giải được mọi lựa chọn

Ví Dụ So Sánh

Tư duy theo mẫu thiết kế (Tệ):

Bài toán: Xây dựng mạng xã hội

Cách tiếp cận:

1. Mạng xã hội = cân bằng tin
2. Bảng tin = cân phân phát (*fan-out*)
3. Dùng mẫu *fan-out* khi ghi (*fan-out on write*)
4. Xong!

Kết quả: Thiết kế thừa cho một công ty khởi nghiệp với 100 người dùng

Tư duy thiết kế hệ thống (Tốt):

Bài toán: Xây dựng mạng xã hội

Câu hỏi:

1. Quy mô? (100 người dùng hay 100 triệu người dùng?)
2. Đặc điểm lưu lượng? (thiên về đọc hay thiên về ghi?)
3. Yêu cầu nhất quán? (thời gian thực hay nhất quán cuối cùng là được?)
4. Quy mô nhóm? (2 lập trình viên hay 50?)
5. Thời hạn? (MVP trong 1 tháng hay 1 năm?)
6. Ngân sách? (hạn chế hay không giới hạn?)

Với 100 người dùng, 2 lập trình viên, thời hạn 1 tháng:

- Một cơ sở dữ liệu SQL đơn giản
- Truy vấn bài viết của những người mình theo dõi khi cần
- Sắp xếp đơn giản theo thời gian
- Chưa cần bộ nhớ đệm

Mở rộng sau khi đã chứng minh là cần thiết.

Kết quả: Ra mắt đúng hạn, cải tiến nhanh

Thấy sự khác biệt chưa?

Tư duy theo mẫu thiết kế = Áp dụng giải pháp có sẵn

Tư duy thiết kế hệ thống = **Hiểu vấn đề** → **Chọn giải pháp phù hợp**

Chuyển Đổi Tư Duy: Từ "Dùng Công Nghệ Gì" Sang "Giải Quyết Vấn Đề Gì"

Tư Duy Cũ (Dùng Công Nghệ Gì)

Kỹ sư lâu năm: "Thiết kế hệ thống thông báo đi"

Kỹ sư mới vào nghề nghĩ:

- "Dùng công nghệ gì nhỉ?"
- "Kafka hay RabbitMQ?"
- "WebSocket hay Server-Sent Events?"
- "PostgreSQL hay MongoDB?"

→ Tư duy ưu tiên công nghệ

→ Đi "mua sắm" giải pháp

Vấn đề:

Chưa hiểu vấn đề`
Không biết các ràng buộc
Không lý giải được các đánh đổi
Có thể thiết kế thừa
Có thể thiết kế thiếu

Tư Duy Mới (Giải Quyết Vấn Đề Gì)

Kỹ sư lâu năm: "Thiết kế hệ thống thông báo đi"

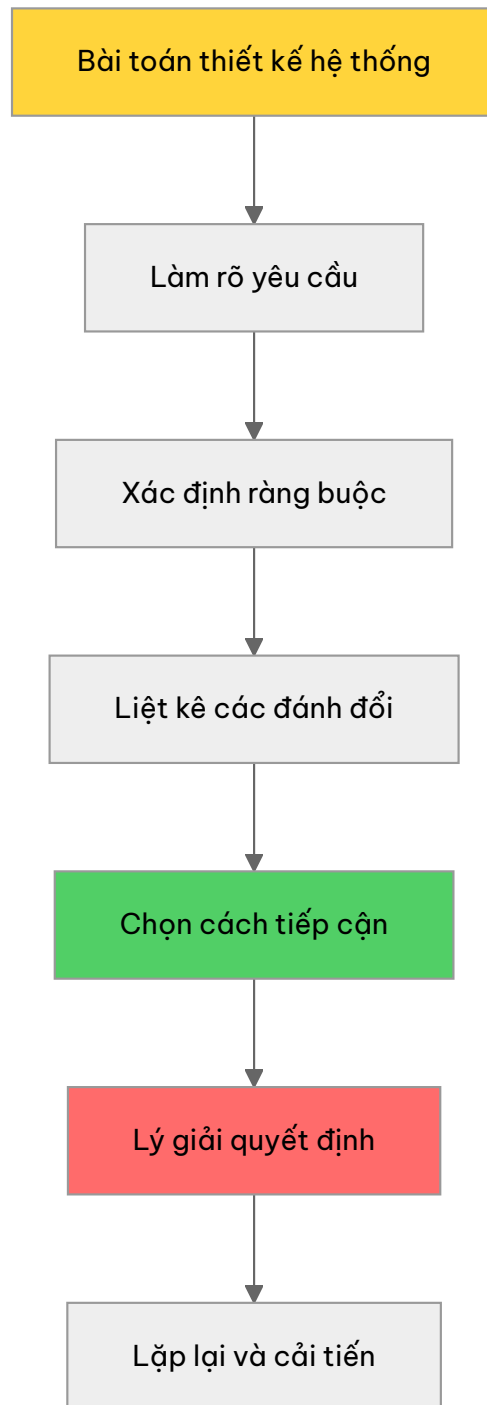
Kỹ sư lâu năm nghĩ:

- "Đang giải quyết vấn đề gì?"
- "Quy mô bao nhiêu? (1 nghìn hay 1 tỷ thông báo/ngày?)"
- "Yêu cầu độ trễ? (thời gian thực hay nhất quán cuối cùng là được?)"
- "Đảm bảo gửi? (ít nhất một lần hay đúng một lần?)"
- "Các loại thông báo? (push, email, SMS?)"
- "Tuỳ chọn của người dùng? (tắt tiếng, tần suất?)"

→ Tư duy ưu tiên vấn đề`

→ Thiết kế dựa trên yêu cầu

Khung tư duy (Framework):



Quy trình tư duy thiết kế hệ thống: Yêu cầu → Ràng buộc → Đánh đổi → Quyết định → Lý giải

Thiết Kế Dựa Trên Yêu Cầu (Requirement)

Bắt đầu mọi thiết kế bằng yêu cầu (requirements). Luôn luôn như vậy.

Yêu Cầu Chức Năng (Functional Requirements)

Định nghĩa: Hệ thống phải làm gì?

Example: URL Shortener

Functional Requirements:

1. Shorten URL
 - Input: long URL
 - Output: short URL
 - Example: `https://example.com/long-path` → `ex.co/abc123`
2. Redirect
 - Input: short URL
 - Output: redirect to original URL
 - Must work consistently
3. Custom aliases (optional)
 - User can choose short code
 - Example: `ex.co/my-link`
4. Analytics (optional)
 - Track clicks
 - Show statistics
5. Expiration (optional)
 - URL expires after X days
 - Auto-cleanup

Kỹ thuật làm rõ yêu cầu:

Người phỏng vấn: "Thiết kế dịch vụ rút gọn URL"

Bạn: "Cho phép tôi làm rõ các yêu cầu chức năng:

1. Có hỗ trợ URL rút gọn tùy chỉnh không?
2. Có cần thông kê (theo dõi lượt nhấp) không?
3. URL có hết hạn không?
4. Có cần tài khoản người dùng không?
5. Người dùng có thể sửa/xóa URL của họ không?
6. Có cần giới hạn tần suất gọi API không?"

Mỗi câu trả lời đều thay đổi thiết kế đáng kể!

Yêu Cầu Phi Chức Năng (Non-Functional Requirements) (Cực Kỳ Quan Trọng!)

Định nghĩa: Hệ thống phải vận hành với hiệu năng (*perform*) như thế nào?

Non-Functional Requirements

1. Scale

- Users: 1M, 10M, hay 100M?
- Requests: 100 req/s hay 100K req/s?
- Data: 1GB hay 10TB?
- Growth rate: 2x/year hay 10x/year?

2. Performance

- Latency: < 100ms? < 1s?
- Throughput: 1K req/s? 1M req/s?
- Availability: 99%? 99.99%?

3. Consistency

- Strong consistency needed?
- Eventual consistency OK?
- Tolerance for stale data?

4. Durability

- Data loss acceptable? (NO for banking, maybe OK for analytics)
- Backup requirements?
- Recovery time objective (RTO)?

5. Security

- Authentication needed?
- Authorization model?
- Data encryption?
- Rate limiting?

6. Cost

- Budget constraints?
- Optimize for cost vs performance?

Các phép ước tính nhanh (back-of-envelope):

Example: URL Shortener

Given:

- 100M URLs shortened per month
- 10B redirects per month
- 5 years retention

Calculate:

Write traffic

$\text{writes_per_second} = 100\text{M} / (30 * 24 * 3600)$
 $\approx 40 \text{ writes/second}$

Read traffic

$\text{reads_per_second} = 10\text{B} / (30 * 24 * 3600)$
 $\approx 4,000 \text{ reads/second}$

Read:Write ratio = 100:1

→ Read-heavy system → Caching critical

Storage

$\text{urls_total} = 100\text{M/month} \times 12 \text{ months} \times 5 \text{ years}$
 $= 6 \text{ billion URLs}$

$\text{storage_per_url} = 500 \text{ bytes (URL + metadata)}$

$\text{total_storage} = 6\text{B} \times 500 \text{ bytes}$
 $= 3\text{TB}$

→ Single database can handle (scale vertically first)

Bandwidth

$\text{write_bandwidth} = 40 \text{ req/s} \times 500 \text{ bytes}$
 $= 20 \text{ KB/s (negligible)}$

$\text{read_bandwidth} = 4,000 \text{ req/s} \times 500 \text{ bytes}$
 $= 2 \text{ MB/s (manageable)}$

→ Network **not** bottleneck

Conclusion:

- Database: Start **with** single PostgreSQL
- Caching: Critical (Redis **for** hot URLs)
- CDN: Not needed (no static assets)
- Sharding: Not needed yet (3TB manageable)

Tại sao tính toán lại quan trọng:

Không tính toán:

"Cần xử lý hàng triệu người dùng!"

→ Thiết kế thừa với vi dịch vụ, Kafka, phân mảnh dữ liệu

Có tính toán:

"40 lượt ghi/giây, 4 nghìn lượt đọc/giây, 3TB dữ liệu"

→ Một máy chủ + bộ nhớ đệm Redis là đủ

→ Ra mắt trong 2 tuần, không phải 6 tháng

Tư Duy Ưu Tiên Ràng Buộc (*Constraint*)

Ràng buộc (*Constraints*) **định hình thiết kế. Hãy xác định ràng buộc TRƯỚC KHI thiết kế.**

Các Loại Ràng Buộc

1. Ràng buộc kỹ thuật

```
# Database constraints
- Single database max: ~10K writes/second
- PostgreSQL max connections: ~500
- Redis max memory: Depends on instance (16GB typical)
- Network latency: Speed of light (can't beat physics)

# CAP theorem constraint
- Can't have perfect Consistency + Availability với Partition tolerance
- Must choose trade-off

# Eventual consistency constraint
- Distributed caches → replication lag
- Async processing → delay
```

2. Ràng buộc kinh doanh

```
# Timeline
- MVP in 1 month → Simple architecture
- No deadline pressure → Can optimize

# Budget
- Limited ($1000/month) → Optimize for cost
- Unlimited → Optimize for performance

# Team
- 2 developers → Choose simple stack
- 20 developers → Can handle complexity

# Compliance
- GDPR → Data residency requirements
- PCI DSS → Strict security requirements
- HIPAA → Healthcare data protection
```

3. Ràng buộc về quy mô (*scale*)

```
# Current scale
- 1K users → Monolith OK
- 1M users → Need horizontal scaling
- 100M users → Need distributed architecture

# Growth rate
- Stable growth (2x/year) → Scale gradually
- Hypergrowth (10x/year) → Plan for scale early

# Traffic pattern
- Uniform → Simple load balancing
- Spiky (Black Friday) → Need burst capacity
- Predictable → Can optimize
- Unpredictable → Need flexibility
```

Khung Phân Tích Ràng Buộc (Constraint Analysis Framework)

```
def analyze_constraints(problem):
    """Framework để identify constraints"""

    constraints = {
        # Hard constraints (cannot violate)
        'hard': [
            'Budget: $5000/month maximum',
            'Timeline: 3 months to launch',
            'Availability: Must be 99.9%',
            'Compliance: GDPR required'
        ],

        # Soft constraints (prefer but flexible)
        'soft': [
            'Team familiar with Python',
            'Prefer open-source solutions',
            'Easy to maintain'
        ],

        # Technical limits
        'technical': [
            'Database writes: < 1000/second',
            'Read latency: < 200ms',
            'Network: Public internet (not dedicated)'
        ],

        # Scale limits
        'scale': [
            'Current: 10K users',
            'Target: 100K users in 1 year',
            'Data: Currently 100GB, growing 10GB/month'
        ]
    }

    return constraints

# Design decisions từ constraints
def make_decisions(constraints):
    """Derive design từ constraints"""

    decisions = []

    # Budget constraint → Choose managed services
    if constraints['hard']['budget'] < 10000:
        decisions.append('Use managed services (RDS, ElastiCache)')
        decisions.append('Avoid self-managed Kubernetes')

    # Timeline constraint → Choose familiar tech
    if constraints['hard']['timeline'] < 6_months:
        decisions.append('Use team\'s existing stack')
        decisions.append('Avoid learning new paradigms')
```

```
# Scale constraint → Start simple
if constraints['scale']['current'] < 100K:
    decisions.append('Monolith architecture')
    decisions.append('Vertical scaling first')

return decisions
```

Ví dụ thực tế:

Kịch bản công ty khởi nghiệp:

- Ngân sách: \$2000/tháng
- Nhóm: 3 lập trình viên (Python)
- Thời hạn: 2 tháng để ra MVP
- Quy mô: Dự kiến 5 nghìn người dùng ban đầu

Các ràng buộc quyết định:

Không dùng: Kubernetes, Kafka, Cassandra

→ Quá đắt, quá phức tạp, quá mức cần thiết

Nên dùng:

- Heroku hoặc AWS Elastic Beanstalk (dịch vụ được quản lý)
- PostgreSQL (quen thuộc, đủ mạnh)
- Redis (lưu đệm đơn giản)
- Hệ thống nguyên khối (phát triển nhanh)

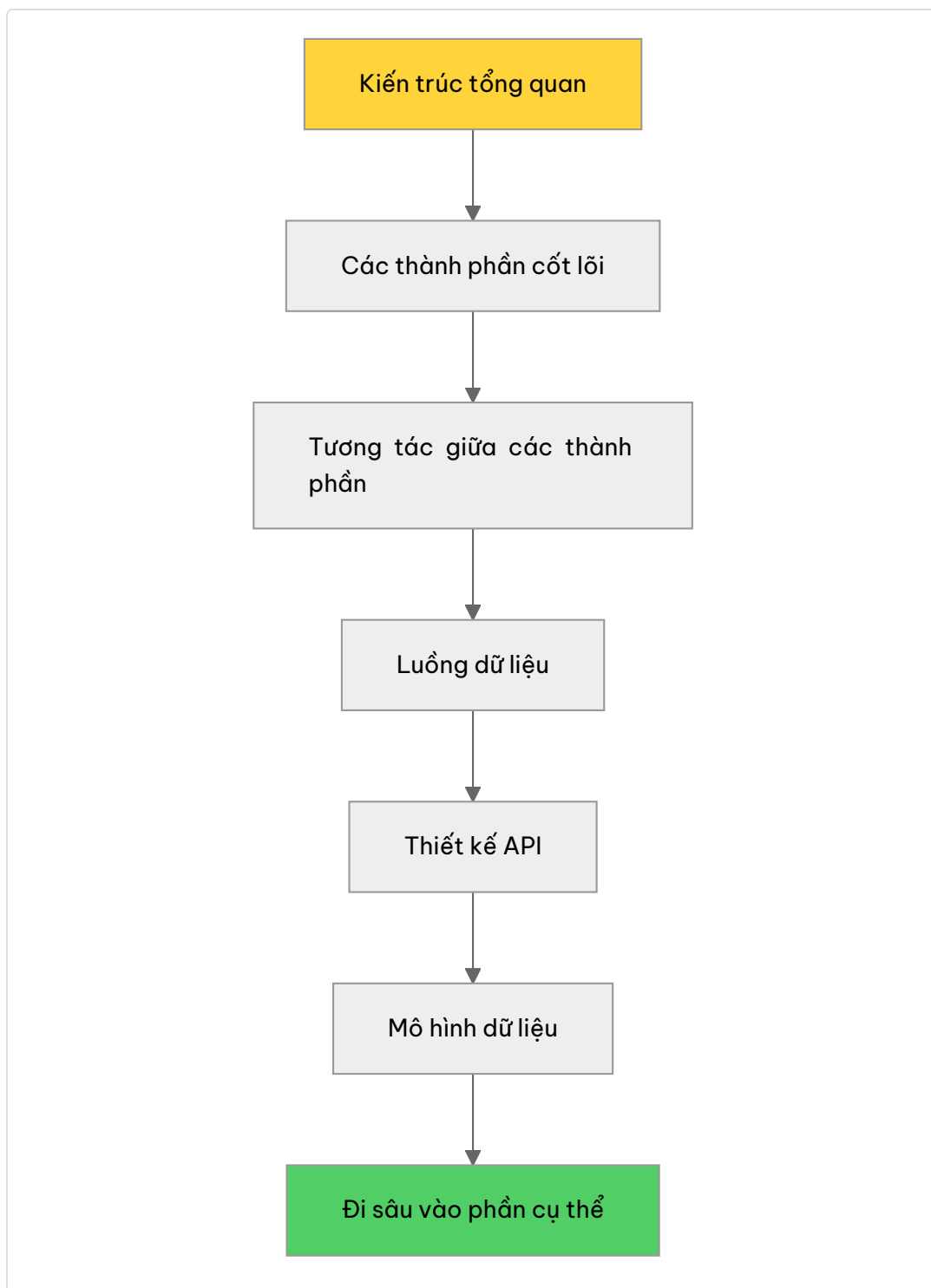
Quyết định được lý giải bằng ràng buộc, không phải bằng "thực hành tốt nhất"

Thiết Kế Từ Trên Xuống (Top-Down) và Từ Dưới Lên (Bottom-Up)

Có 2 cách tiếp cận để thiết kế hệ thống. Hãy biết khi nào dùng cách nào.

Thiết Kế Từ Trên Xuống (Khuyên Dùng Khi Phỏng Vấn)

Cách tiếp cận: Bắt đầu từ tổng quan, rồi đi sâu vào chi tiết



Từ trên xuống: Từ tổng quan đến chi tiết

Quy trình:

```

# Step 1: High-level boxes
"""
[Client] → [Load Balancer] → [API Servers] → [Database]
                        ↓
                        [Cache]
"""

# Step 2: Define components
"""
Load Balancer: Nginx
API Servers: Python/FastAPI (stateless)
Cache: Redis
Database: PostgreSQL
"""

# Step 3: Detail interactions
"""
Client:
- HTTPS requests
- JWT authentication
- Rate limited

API Servers:
- RESTful endpoints
- Horizontal scaling
- Health checks

Cache:
- Store hot data
- TTL: 5 minutes
- Cache-aside pattern
"""

# Step 4: API design
"""
POST /shorten
Body: {"url": "https://..."}
Response: {"short_url": "ex.co/abc123"}

GET /{shortCode}
Response: 302 Redirect
"""

# Step 5: Data models
"""
CREATE TABLE urls (
    id BIGSERIAL PRIMARY KEY,
    short_code VARCHAR(10) UNIQUE,
    original_url TEXT,
    created_at TIMESTAMP,
    expires_at TIMESTAMP
);
"""

```

```
# Step 6: Deep dive critical parts
"""
Short code generation:
- Base62 encoding (a-z, A-Z, 0-9)
- 7 characters =  $62^7 = 3.5$  trillion combinations
- Collision handling: retry với new code
"""
```

Ưu điểm:

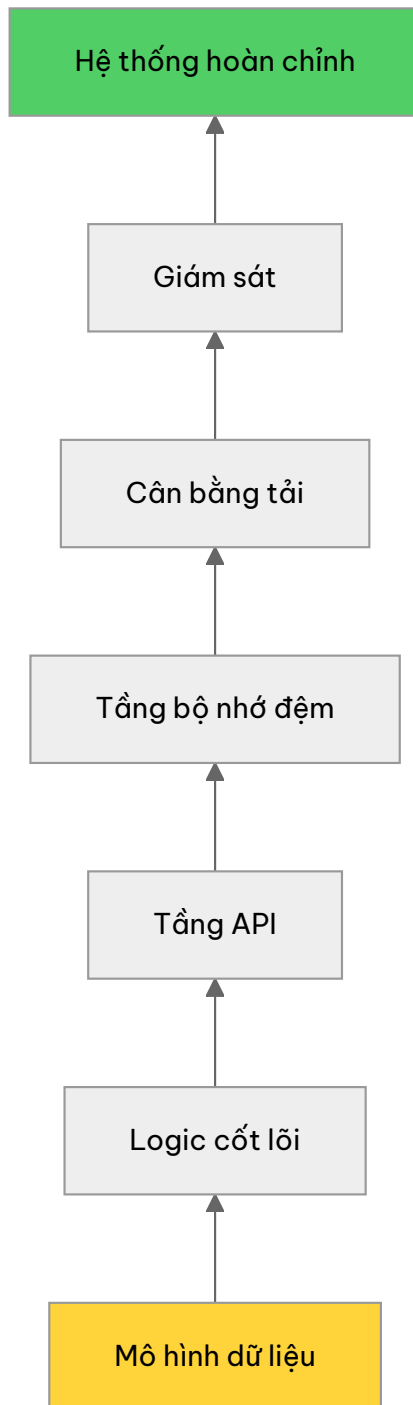
Có cái nhìn toàn diện từ sớm
Phát hiện vấn đề kiến trúc nhanh
Dễ trao đổi
Phù hợp khi phỏng vấn (thể hiện quá trình tư duy)
Có thể điều chỉnh trước khi đi sâu vào triển khai

Nhược điểm:

Có thể bỏ sót chi tiết
Các giả định có thể sai
Khó hơn nếu chưa có kinh nghiệm

Thiết Kế Từ Dưới Lên (Phù Hợp Khi Hiện Thực Hoá)

Cách tiếp cận: Bắt đầu từ bài toán cốt lõi, rồi xây dần lên



Từ dưới lên: Xây dần lên từ nền móng

Quy trình:

```
# Step 1: Core data model
"""
What data do we store?
- URL mapping
- Metadata
- Analytics

CREATE TABLE urls (...);
"""

# Step 2: Core logic
"""
How do we generate short codes?
- Base62 encoding
- Collision handling
- Validation
"""

# Step 3: API layer
"""
What endpoints do we need?
POST /shorten
GET /{code}
"""

# Step 4: Caching
"""
Cache hot URLs
Redis sorted set by access frequency
"""

# Step 5: Scaling
"""
Add load balancer
Multiple API servers
Database read replicas
"""

# Step 6: Monitoring
"""
Metrics: latency, error rate
Alerts: high latency, failures
Dashboards
"""
```

Ưu điểm:

Nền móng vững chắc
Ít phải làm lại
Không bỏ sót chi tiết
Phù hợp khi hiện thực hoá

Nhược điểm:

Không thấy bức tranh tổng thể từ sớm
Có thể tối ưu chi tiết quá mức
Khó chuyển hướng hơn
Kém hiệu quả khi phỏng vấn

Khi Nào Dùng Cách Tiếp Cận Nào?

Từ trên xuống:
✓ Phỏng vấn thiết kế hệ thống
✓ Các buổi họp lên kế hoạch kiến trúc
✓ Dự án mới hoàn toàn (greenfield)
✓ Trao đổi với các bên liên quan
✓ Khi cần nhanh chóng kiểm chứng ý tưởng (proof of concept)

Từ dưới lên:
✓ Giai đoạn hiện thực hoá
✓ Tái cấu trúc hệ thống hiện có
✓ Khi chi tiết là quan trọng (bảo mật, tuân thủ)
✓ Các bài toán thuật toán phức tạp
✓ Thiết kế lược đồ cơ sở dữ liệu

Cách tiếp cận kết hợp (Tốt nhất trong thực tế):

1. Từ trên xuống: Vẽ kiến trúc tổng quan
2. Xác định các thành phần then chốt
3. Từ dưới lên: Chi tiết hoá các phần then chốt
4. Từ trên xuống: Kiểm tra các phần khớp với nhau
5. Lặp lại

Tư Duy Về Khả Năng Mở Rộng (Scalability)

Tư duy về khả năng mở rộng $\neq$ "Làm cho nó chịu được hàng triệu người dùng"

Tư duy về khả năng mở rộng = Thiết kế để có thể mở rộng khi cần, mà không phải viết lại

Phổ Khả Năng Mở Rộng

Thiết kế thừa ↔ Vừa vặn ↔ Thiết kế thiếu

[Microservices]	[Monolith	[Mã
[Kubernetes]	mô-đun hoá]	rối rắm]
[Kafka]	[PostgreSQL]	[CSDL không
[Sharding]	[Redis]	có chỉ mục]

Trái: Tối ưu hoá quá sớm

Giữa: Vùng Goldilocks (vừa đủ) ✓

Phải: Nợ kỹ thuật

Tìm vùng Goldilocks:

```
def choose_architecture(current_scale, target_scale, timeline):
    """Choose architecture dựa trên scale"""

    if current_scale < 10_000 and target_scale < 100_000:
        return {
            'architecture': 'Monolith',
            'database': 'Single PostgreSQL',
            'cache': 'Redis (optional)',
            'reasoning': 'Simple, ship fast, iterate'
        }

    elif current_scale < 1_000_000 and target_scale < 10_000_000:
        return {
            'architecture': 'Modular Monolith',
            'database': 'PostgreSQL with read replicas',
            'cache': 'Redis cluster',
            'reasoning': 'Balance simplicity with scalability'
        }

    else: # > 10M users
        return {
            'architecture': 'Microservices',
            'database': 'Sharded PostgreSQL or NoSQL',
            'cache': 'Distributed Redis',
            'messaging': 'Kafka for async',
            'reasoning': 'Need independent scaling, team autonomy'
        }
```

Thiết Kế Để Mở Rộng Mà Không Thiết Kế Thừa (Over-Engineering)

Nguyên tắc:

1. Máy chủ ứng dụng phi trạng thái (stateless)


```

# Bad: Stateful
sessions = {} # In-memory state

def handle_request(user_id):
    session = sessions[user_id] # Tied to this server
    # ...

# Good: Stateless
def handle_request(user_id, session_token):
    session = redis.get(f"session:{session_token}") # Any server can handle
    # ...

```

2. Chỉ mục (*indexes*) cơ sở dữ liệu ngay từ ngày đầu

```

-- Don't wait until slow to add indexes

-- Add early
CREATE INDEX idx_users_email ON users(email);
CREATE INDEX idx_posts_created_at ON posts(created_at);
CREATE INDEX idx_posts_user_id ON posts(user_id);

-- Small cost now, huge benefit later

```

3. Giám sát (*Monitoring*) ngay từ ngày đầu

```

# Track metrics early
metrics.increment('api.requests')
metrics.timing('api.latency', duration)
metrics.gauge('active_users', count)

# When scale issues hit, you have data
# Without metrics = flying blind

```

4. Bất đồng bộ (*Async*) ở những chỗ phù hợp

```

# Long tasks → Async

# Synchronous
def create_user(email):
    user = db.create(email)
    send_welcome_email(email) # Block 5 seconds
    generate_thumbnail(user)   # Block 10 seconds
    return user # User waits 15 seconds

# Asynchronous
def create_user(email):
    user = db.create(email)
    queue.publish('user.created', user.id) # Fire and forget
    return user # User waits 100ms

# Workers handle async
@worker.task
def on_user_created(user_id):
    send_welcome_email(user_id)
    generate_thumbnail(user_id)

```

5. Cấu trúc mã theo mô-đun (*modular*)

```

# Even trong monolith, structure well

# Bad: Everything in one file
# app.py (5000 lines)

# Good: Clear boundaries
/services
  /user_service.py
  /post_service.py
  /notification_service.py
/models
/api

# Khi cần split to microservices, boundaries already clear

```

Tư Duy Thiết Kế và Tư Duy Hiện Thực Hoá

Điểm phân biệt then chốt: Thiết kế ≠ Hiện thực hoá

Tư Duy Thiết Kế (Phỏng Vấn, Lên Kế Hoạch)

Trọng tâm: Các quyết định kiến trúc, đánh đổi (*trade-offs*), lý giải

Các câu hỏi cần trả lời:

- Cần những thành phần nào?
- Chúng tương tác với nhau ra sao?
- Điểm nghẽn nằm ở đâu?
- Cái gì có thể hỏng và xử lý thế nào?
- Mỗi lựa chọn có những đánh đổi gì?
- Hệ thống mở rộng ra sao?

Đầu ra:

- Sơ đồ kiến trúc
- Trách nhiệm của từng thành phần
- Hợp đồng API
- Mô hình dữ liệu (mức tổng quan)
- Phân tích đánh đổi

Ví dụ hội thoại:

Người phỏng vấn: "Thiết kế Twitter"

Bạn (tư duy thiết kế):

"Tôi xin bắt đầu với các yêu cầu..."

- 500 triệu người dùng mỗi ngày
- 200 triệu tweet mỗi ngày
- Thiên về đọc (95% là đọc)

Kiến trúc:

- Dịch vụ phân phát (fan-out) để phân phối tweet
- Bộ nhớ đệm dòng thời gian trong Redis
- CDN cho nội dung đa phương tiện
- PostgreSQL phân mảnh để lưu trữ lâu dài

Đánh đổi then chốt: Fan-out khi ghi và fan-out khi đọc

- Với người dùng bình thường: Fan-out khi ghi (tính trước dòng thời gian)
- Với người nổi tiếng: Fan-out khi đọc (quá nhiều người theo dõi)
- Cách tiếp cận kết hợp cân bằng hiệu năng ghi và đọc

Điểm nghẽn:

- Tạo dòng thời gian trong quá trình fan-out
- Phân phối nội dung đa phương tiện
- Vấn đề người nổi tiếng "nóng"

Xử lý sự cố:

- Hàng đợi cho fan-out (nếu dịch vụ sập thì thư lại)
 - Xử lý bất đồng bộ (nhất quán cuối cùng là được)
 - Bộ ngắt mạch cho các dịch vụ bên ngoài
- "

Tư Duy Hiện Thực Hoá (Xây Dựng)

Trọng tâm: Mã nguồn, thuật toán, công nghệ cụ thể

Các câu hỏi cần trả lời:

- Dùng framework nào?
- Tổ chức mã ra sao?
- Dùng thư viện nào?
- Xử lý các trường hợp biên thế nào?
- Kiểm thử thế nào?
- Triển khai thế nào?

Đầu ra:

- Mã chạy được
- Kiểm thử đơn vị
- Kiểm thử tích hợp
- Script triển khai
- Tài liệu

Ví dụ:

```
# Implementation details

# Short code generation algorithm
def generate_short_code(url_id):
    """Convert numeric ID to base62 string"""
    BASE62 = "0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ"

    if url_id == 0:
        return BASE62[0]

    result = []
    while url_id > 0:
        result.append(BASE62[url_id % 62])
        url_id //= 62

    return ''.join(reversed(result))

# Collision handling
def create_short_url(long_url, max_retries=3):
    for attempt in range(max_retries):
        url_id = get_next_id()
        short_code = generate_short_code(url_id)

        try:
            db.insert(short_code, long_url)
            return short_code
        except UniqueViolation:
            # Collision, retry
            continue

    raise Exception("Failed to generate unique short code")
```

Cân Bằng Cả Hai

Kỹ sư giỏi biết cân bằng giữa tư duy thiết kế và tư duy hiện thực hoá:

Giai đoạn thiết kế:

- Nghĩ ở mức tổng quan
- Tập trung vào kiến trúc
- Lý giải các đánh đổi
- Đừng sa lầy vào chi tiết hiện thực hoá

Giai đoạn hiện thực hoá:

- Nghĩ ở mức chi tiết
- Tập trung vào chất lượng mã
- Xử lý các trường hợp biên
- Giữ vững tầm nhìn kiến trúc

Đừng trộn lẫn các giai đoạn:

Phỏng vấn thiết kế: "Tôi sẽ dùng FastAPI với async/await..."

→ Quá thiên về hiện thực hoá

Lúc viết mã: "Đề'tôi thiết kế'lại toàn bộ kiến trúc..."

→ Sai thời điểm

Thiết kế: Tập trung vào cái gì và tại sao

Hiện thực hoá: Tập trung vào làm thế nào

Khung Tư Duy Hoàn Chỉnh: Kết Hợp Tất Cả Lại

Khi tiếp cận một bài toán thiết kế hệ thống:

Khung Tư Duy Từng Bước

1. LÀM RÕ (5 phút)
 - └ Yêu cầu chức năng
 - └ Yêu cầu phi chức năng
 - └ Quy mô (người dùng, lưu lượng, dữ liệu)
 - └ Ràng buộc (ngân sách, thời hạn, nhóm)
2. ƯỚC LƯỢNG (5 phút)
 - └ Các phép ước tính nhanh
 - └ Tỷ lệ đọc/ghi
 - └ Nhu cầu lưu trữ
 - └ Nhu cầu băng thông
3. THIẾT KẾ TỔNG QUAN (10 phút)
 - └ Vẽ các thành phần chính
 - └ Thể hiện luồng dữ liệu
 - └ Xác định đường đi then chốt
 - └ Nêu rõ các quyết định quan trọng
4. ĐI SÂU (15 phút)
 - └ Thiết kế API
 - └ Mô hình dữ liệu
 - └ Chi tiết thuật toán
 - └ Chiến lược lưu đệm
 - └ Cách tiếp cận mở rộng
5. ĐÁNH ĐỔI (10 phút)
 - └ Thảo luận các phương án khác
 - └ Lý giải các lựa chọn
 - └ Nêu các đánh đổi
 - └ Xử lý các trường hợp biên
6. TỔNG KẾT (5 phút)
 - └ Điểm nghẽn & giải pháp
 - └ Các kịch bản sự cố
 - └ Chiến lược giám sát
 - └ Cải tiến trong tương lai

Danh Sách Kiểm Tra Trong Đầu

Trước khi chốt thiết kế, hãy kiểm tra:

- ☑ Yêu cầu đã rõ ràng?
- ☑ Đã xác định ràng buộc?
- ☑ Đã tính toán quy mô?
- ☑ Đã biết tỷ lệ đọc/ghi?
- ☑ Đã xác định điểm nghẽn?
- ☑ Đã xem xét các kiểu sự cố?
- ☑ Các đánh đổi đã được lý giải?
- ☑ Lộ trình mở rộng đã rõ ràng?
- ☑ Đã có kế hoạch giám sát?
- ☑ Đã ước lượng chi phí?

Những Sai Lầm Thường Gặp Cần Tránh

Nhảy ngay vào giải pháp

→ Hãy làm rõ yêu cầu trước

Thiết kế thừa cho một tương lai chưa biết

→ Thiết kế cho quy mô gấp 2 lần, không phải 100 lần

Bỏ qua ràng buộc

→ Ngân sách, thời hạn, nhóm đều ảnh hưởng đến thiết kế

Sao chép kiến trúc của các công ty công nghệ lớn

→ Quy mô của họ $\neq$ quy mô của bạn

Xem nhẹ các kịch bản sự cố

→ Hệ thống sẽ hỏng, hãy lên kế hoạch cho điều đó

Bỏ qua giám sát

→ Không thể cải thiện thứ bạn không đo lường

Hoàn hảo hay ra mắt được

→ MVP trước, cải tiến sau

Ứng Dụng Thực Tế

Ví dụ: Thiết kế một ứng dụng giống Instagram

Áp Dụng Khung Tư Duy

1. Làm rõ yêu cầu:

Chức năng:

- Tải ảnh lên
- Theo dõi người dùng khác
- Xem bảng tin (ảnh từ những người mình theo dõi)
- Thích, bình luận
- Tìm kiếm người dùng

Phi chức năng:

- 100 triệu người dùng
- 50 triệu người dùng hoạt động mỗi ngày
- 10 triệu ảnh được tải lên/ngày
- Thiên về`đọc (xem:tải lên = 100:1)
- Độ trễ: Bảng tin < 500ms
- Tính sẵn sàng: 99,9%

2. Ước lượng (Estimate):

Lưu trữ:

- 10 triệu ảnh/ngày × trung bình 2MB = 20TB/ngày
- 20TB × 365 = 7,3PB/năm
- Cần lưu trữ đối tượng (S3)

Lưu lượng:

- Tải lên: 10 triệu/ngày / 86400 giây ≈ 115 lượt tải lên/giây
- Lượt xem: 115 × 100 = 11.500 lượt xem/giây
- Thiên về`đọc, lưu đệm là then chốt

Cơ sở dữ liệu:

- 100 triệu người dùng × 500 byte = 50GB (bảng users)
- 3,6 tỷ ảnh/năm × 200 byte siêu dữ liệu = 720GB/năm
- SQL vẫn xử lý được, phân mảnh sau 2-3 năm

3. Thiết kế tổng quan:

[Di động/Web]

↓

[CDN] (hình ảnh)

↓

[Bộ cân bằng tải]

↓

[Máy chủ API] (phi trạng thái)

↓

[Bộ nhớ đệm Redis] ← Dòng thời gian bảng tin

↓

[PostgreSQL] ← Siêu dữ liệu

↓

[S3] ← Lưu trữ ảnh

4. Các quyết định then chốt:

Lưu trữ ảnh: S3

- Bền vững, dễ mở rộng
- Tích hợp với CDN
- Đánh đổi: Chi phí và độ tin cậy

Tạo bảng tin: Fan-out khi ghi

- Tính trước bảng tin
- Đọc nhanh (< 50ms từ bộ nhớ đệm)
- Đánh đổi: Khuếch đại ghi (write amplification)

Lưu đệm: Dòng thời gian trong Redis

- Lưu 1000 bài viết gần nhất cho mỗi người dùng
- TTL: Không hết hạn (vô hiệu hoá chủ động)
- Đánh đổi: Chi phí bộ nhớ và hiệu năng đọc

Cơ sở dữ liệu: PostgreSQL có phân mảnh

- Nhất quán mạnh cho dữ liệu quan trọng
- Phân mảnh theo user_id sau 2 năm
- Đánh đổi: Độ phức tạp và quy mô

5. Các đánh đổi đã thảo luận:

Fan-out khi ghi hay khi đọc:

- Chọn khi ghi vì hệ thống thiên về đọc
- Vấn đề người nổi tiếng: Chuyển sang kéo (pull) với tài khoản > 10 nghìn người theo dõi

SQL hay NoSQL:

- Chọn SQL vì tính nhất quán và quan hệ dữ liệu
- Có thể thêm bản sao chỉ đọc, rồi phân mảnh

Đồng bộ hay bất đồng bộ:

- Tải ảnh lên: Đồng bộ (người dùng chờ)
- Fan-out bảng tin: Bất đồng bộ (nhất quán cuối cùng là được)

6. Tổng kết:

Điểm nhấn:

- Tạo bảng tin (giải quyết bằng fan-out bất đồng bộ)
- Phân phối ảnh (giải quyết bằng CDN)
- Đọc cơ sở dữ liệu (giải quyết bằng lưu đệm)

Giám sát:

- Tỷ lệ tải lên thành công
- Độ trễ tải bảng tin
- Tỷ lệ trúng bộ nhớ đệm
- Độ trễ fan-out

Cải tiến trong tương lai:

- Xếp hạng bảng tin bằng ML
- Tính năng Stories
- Video trực tiếp
- Gợi ý nội dung

Những Điểm Cốt Lõi

Tư duy thiết kế hệ thống $\neq$ Biết mẫu thiết kế

Mẫu thiết kế là công cụ.

Tư duy là biết khi nào dùng công cụ nào.

Chuyển đổi tư duy:

Từ: "Dùng công nghệ gì?"

Sang: "Giải quyết vấn đề gì?"

Từ: "Thực hành tốt nhất"

Sang: "Phù hợp nhất với bối cảnh"

Từ: "Sao chép công ty công nghệ lớn"

Sang: "Thiết kế cho quy mô của mình"

Khung tư duy:

1. Yêu cầu trước tiên (luôn luôn)
2. Ràng buộc định hình thiết kế
3. Tính toán quy mô (con số rất quan trọng)
4. Từ trên xuống khi lên kế hoạch
5. Thiết kế cho gấp 2 lần, không phải 100 lần
6. Lý giải mọi quyết định
7. Đánh đổi quan trọng hơn sự hoàn hảo

Phép thử tối thượng:

Tư duy thiết kế tốt = Có thể giải thích "TẠI SAO" cho mọi quyết định

Nếu bạn không giải thích được tại sao:

- Tại sao chọn cơ sở dữ liệu này?
- Tại sao chọn chiến lược lưu đệm này?
- Tại sao chọn kiến trúc này?

→ Bạn chưa hiểu chính thiết kế của mình

→ Hãy quay lại và suy nghĩ sâu hơn

Hãy nhớ:

Kỹ sư mới vào nghề: Biết các mẫu thiết kế

Kỹ sư trung cấp: Áp dụng các mẫu thiết kế

Kỹ sư lâu năm: Biết khi nào KHÔNG áp dụng mẫu thiết kế

Làm chủ thiết kế hệ thống = Tư duy, không phải học thuộc lòng

Bạn đã học đủ các mẫu thiết kế qua 5 phần. Phần 6 là để mài giũa quá trình tư duy.

Hãy luyện tập khung tư duy (*thinking framework*), không chỉ học thuộc mẫu thiết kế.

Đó là điều phân biệt kỹ sư lâu năm (*Senior Engineer*) với kỹ sư mới vào nghề (*Junior Engineer*).

Khung Tư Duy (Framework) SNAKE: Phương Pháp Để Chinh Phục Buổi Phỏng Vấn Thiết Kế Hệ Thống (System Design Interview)

Tôi còn nhớ lần đầu tiên tham gia một buổi phỏng vấn thiết kế hệ thống.

Người phỏng vấn (Interviewer): "Thiết kế YouTube."

Tôi: "Ừm... YouTube... video... cần cơ sở dữ liệu (database)... cần CDN... cần..."

Tôi vẽ vài cái hộp, nói về vi dịch vụ (microservices), nhắc đến Kafka. 30 phút trôi qua trong mơ hồ.

Người phỏng vấn: "Em có cách tiếp cận nào có cấu trúc không?"

Tôi: "... Không ạ."

Tôi trượt buổi phỏng vấn đó.

Không phải vì thiếu kiến thức. Mà vì **không có phương pháp rõ ràng**.

Sau đó, tôi học khung tư duy SNAKE từ một kiến trúc sư lâu năm (senior architect). Nó đã thay đổi cuộc chơi.

SNAKE = **S**cope (Phạm vi) → **N**umbers (Con số) → **A**PI → **K**ey components (Thành phần chính) → **E**laborate (Đào sâu)

Khung tư duy này đã thay đổi cách tôi tiếp cận các buổi phỏng vấn thiết kế hệ thống. Và nó cũng sẽ thay đổi cách bạn tiếp cận.

Tại Sao Cần Khung Tư Duy?

Vấn Đề Khi Không Có Cấu Trúc

Khi không có khung tư duy:

Người phỏng vấn: "Thiết kế Twitter"

Trong đầu ứng viên:

"Twitter... tweet... người dùng... người theo dõi... bảng tin...
cơ sở dữ liệu... NoSQL? SQL? Bộ nhớ đệm? Ví dịch vụ?
Kafka? Redis? Khoan, có nên bắt đầu từ cơ sở dữ liệu không?
Hay API? Hay kiến trúc? Aaah..."

- Nhảy lung tung không theo thứ tự
- Bỏ sót các yêu cầu quan trọng
- Lãng phí thời gian vào chi tiết không liên quan
- Không có mạch trình bày rõ ràng
- Người phỏng vấn bối rối

Khi có khung tư duy (SNAKE):

Người phỏng vấn: "Thiết kế Twitter"

Ứng viên:

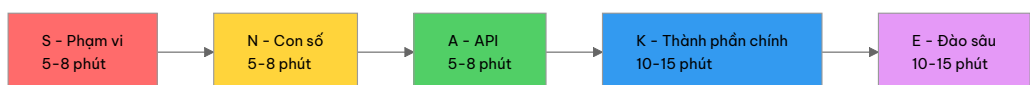
"Trước tiên em xin làm rõ các yêu cầu (Scope)
Sau đó ước lượng quy mô (Numbers)
Định nghĩa các hợp đồng API (API)
Thiết kế kiến trúc tổng thể (Key components)
Đào sâu vào các phân quan trọng (Elaborate)"

- Tiến trình rõ ràng
- Bao quát mọi khía cạnh
- Tư duy có cấu trúc
- Dễ theo dõi
- Người phỏng vấn ấn tượng

Điểm mấu chốt: Phỏng vấn thiết kế hệ thống kiểm tra **quá trình tư duy**, không chỉ kiến thức.

Khung tư duy cho thấy bạn có cách tiếp cận bài toán một cách bài bản.

Tổng Quan Về Khung Tư Duy SNAKE



Khung tư duy SNAKE: 5 bước với cách phân bổ thời gian rõ ràng cho một buổi phỏng vấn 45 phút

Tổng quỹ thời gian: 45 phút

Scope:	5-8 phút	(Làm rõ yêu cầu)
Numbers:	5-8 phút	(Ước lượng quy mô)
API:	5-8 phút	(Định nghĩa giao diện)
Key Components:	10-15 phút	(Thiết kế tổng thể)
Elaborate:	10-15 phút	(Đào sâu)
Dự phòng:	5 phút	(Câu hỏi bất ngờ)

Mỗi bước có mục đích rõ ràng. Đừng bỏ qua bước nào.

S - Scope: Làm Rõ Yêu Cầu (*Requirements*)

Mục tiêu: Hiểu sâu bài toán trước khi thiết kế

Cần Làm Rõ Những Gì

1. Yêu cầu chức năng (*Functional Requirements*) (Hệ thống làm gì)

Template questions

"Let me clarify the functional requirements:

Core features:

1. What are the must-have features?

Example: For Twitter

- Post tweets ✓
- Follow users ✓
- View feed ✓
- Like/retweet ✓

2. What are nice-to-have features?

- Direct messages?
- Notifications?
- Trending topics?
- Search?

3. Out of scope (explicitly state):

- Video tweets
- Analytics dashboard
- Ads system"

2. Yêu cầu phi chức năng (*Non-Functional Requirements*) (Hệ thống vận hành ra sao)

```
# Template questions
```

```
"Non-functional requirements:
```

```
Scale:
```

- How many users? (1M, 10M, 100M?)
- Daily active users?
- Tweets per day?
- Peak traffic pattern?

```
Performance:
```

- Latency requirement? (< 100ms? < 1s?)
- Availability requirement? (99%? 99.99%?)

```
Consistency:
```

- Strong consistency needed?
- Eventual consistency acceptable?

```
Other:
```

- Geographic distribution?
- Mobile vs web ratio?
- Read vs write ratio?"

3. Ràng buộc (Constraints)

```
# Template questions
```

```
"Any constraints I should know?
```

```
Technical:
```

- Existing infrastructure to integrate with?
- Technology preferences?
- Legacy system compatibility?

```
Business:
```

- Timeline to launch?
- Budget constraints?
- Team size?
- Compliance requirements (GDPR, etc.)?"

Cách Đặt Câu Hỏi (Kỹ Năng Giao Tiếp Rất Quan Trọng!)

Cách tiếp cận tôi:

Ứng viên: "Có bao nhiêu người dùng ạ?"
Người phỏng vấn: "Em cho tôi biết đi."
Ứng viên: "Ừm... 1 triệu?"
Người phỏng vấn: "..."

→ Thụ động, không thể hiện được tư duy

Cách tiếp cận tốt:

Ứng viên: "Em xin làm rõ quy mô. Với một hệ thống kiểu Twitter, em nghĩ chúng ta đang thiết kế cho:

- 200 triệu người dùng hoạt động hằng tháng
- 100 triệu người dùng hoạt động hằng ngày
- 500 triệu tweet mỗi ngày
- Khối lượng công việc thiên về đọc (95% đọc, 5% ghi)

Như vậy có khớp với kỳ vọng của anh không, hay em nên điều chỉnh?"

Người phỏng vấn: "Nghe hợp lý đấy."

- Nêu rõ giả định, mời phản hồi
- Thể hiện quá trình tư duy

Danh Sách Kiểm Tra Cho Phạm Vi

Kiểm tra xem bạn đã làm rõ:

- ☒ Đã xác định các tính năng cốt lõi
- ☒ Đã ghi nhận các tính năng "có thì tốt"
- ☒ Đã nêu rõ những gì nằm ngoài phạm vi
- ☒ Đã thống nhất các con số về quy mô
- ☒ Yêu cầu hiệu năng đã rõ ràng
- ☒ Đã hiểu yêu cầu về tính nhất quán
- ☒ Đã ghi nhận các ràng buộc

Thời gian dành ra: 5-8 phút

Đừng vội vàng ở bước này. Nền móng vững chắc = thiết kế tốt hơn.

N - Numbers: Ước Tính Nhanh (*Back-of-Envelope Calculations*)

Mục tiêu: Định lượng quy mô (*scale*) để làm cơ sở cho các quyết định thiết kế

Cần Tính Toán Những Gì

1. Ước lượng lưu lượng (Traffic)

Example: Twitter

Given:

- 100M daily active users (DAU)
- Each user views feed 10 times/day
- Each user posts 2 tweets/day

Calculate:

Read traffic

$\text{read_requests} = 100\text{M users} \times 10 \text{ views/day}$
 $= 1\text{B requests/day}$
 $= 1\text{B} / 86400 \text{ seconds}$
 $\approx 12\text{K requests/second}$

Peak (3x average):

$= 36\text{K requests/second}$

Write traffic

$\text{write_requests} = 100\text{M users} \times 2 \text{ tweets/day}$
 $= 200\text{M tweets/day}$
 $= 200\text{M} / 86400$
 $\approx 2.3\text{K tweets/second}$

Peak: $\approx 7\text{K tweets/second}$

Read:Write ratio

$\text{ratio} = 12\text{K} / 2.3\text{K} \approx 5:1$ (read-heavy)

→ Caching critical for reads

2. Ước lượng lưu trữ (Storage)

```
# Storage calculation

Per tweet:
- tweet_id: 8 bytes (bigint)
- user_id: 8 bytes
- content: 280 chars × 1 byte = 280 bytes
- metadata: 100 bytes (timestamps, etc.)
- media URL: 200 bytes
Total: ~600 bytes per tweet

Daily storage:
200M tweets/day × 600 bytes = 120 GB/day

Yearly:
120 GB × 365 = 43.8 TB/year

5 years:
43.8 TB × 5 = 219 TB

→ Need distributed storage strategy
→ Sharding after 2-3 years
```

3. Ước lượng băng thông (Bandwidth)

```
# Bandwidth calculation

# Incoming (writes)
write_bandwidth = 2.3K tweets/s × 600 bytes
                = 1.4 MB/s (negligible)

# Outgoing (reads)
# Assume each feed load fetches 20 tweets
read_bandwidth = 12K requests/s × 20 tweets × 600 bytes
               = 144 MB/s

Peak: 144 MB/s × 3 = 432 MB/s

# Media (images/videos)
# 30% of tweets have media, average 2MB
media_requests = 12K × 0.3 = 3.6K/s
media_bandwidth = 3.6K × 2MB = 7.2 GB/s

→ CDN absolutely required for media
```

4. Ước lượng bộ nhớ (Lưu đệm (Caching))

```
# Cache hot data

# Cache user feeds (top 1000 tweets per active user)
active_users = 100M DAU
tweets_cached = 1000 per user
size_per_tweet = 600 bytes

cache_memory = 100M × 1000 × 600 bytes
               = 60 TB

# Only cache 20% most active users
cache_memory = 60 TB × 0.2 = 12 TB

# Redis cluster: 100 nodes × 128 GB = 12.8 TB
→ Feasible with distributed cache
```

Tại Sao Các Con Số Quan Trọng

Các con số định hướng quyết định thiết kế:

Kết quả tính toán cho thấy:

- Đọc:Ghi = 5:1 → Chiến lược bộ nhớ đệm là then chốt
- 7,2 GB/s dữ liệu đa phương tiện → Bắt buộc dùng CDN
- 219 TB/5 năm → Sớm muộn cũng cần phân mảnh dữ liệu (sharding)
- 36K req/s → Cần cân bằng tải
- 12 TB bộ nhớ đệm → Cụm Redis phân tán

Không có tính toán:

- Đoán mò
- Thiết kế thừa hoặc thiết kế thiếu
- Không có lý lẽ bảo vệ quyết định

Danh Sách Kiểm Tra Cho Các Con Số

- ☑ Lưu lượng (đọc, ghi, cao điểm)
- ☑ Lưu trữ (hiện tại, tăng trưởng)
- ☑ Băng thông (vào, ra, đa phương tiện)
- ☑ Bộ nhớ (yêu cầu bộ nhớ đệm)
- ☑ Đã xác định tỷ lệ Đọc:Ghi
- ☑ Đã ghi nhận dự báo tăng trưởng

Thời gian dành ra: 5-8 phút

Hãy trình bày phép tính! Viết lên bảng trắng.

A - API: Định Nghĩa Giao Diện

Mục tiêu: Thiết kế các hợp đồng API trước khi triển khai phần bên trong

Tại Sao Bắt Đầu Từ API?

Thiết kế API buộc mọi thứ phải rõ ràng:

Mơ hồ: "Hệ thống cho phép người dùng đăng và xem tweet"

API rõ ràng:

POST /tweets

GET /feed

GET /tweets/{id}

POST /tweets/{id}/like

→ Cụ thể, kiểm thử được, thảo luận được

Thiết Kế API Theo Chuẩn REST

Ví dụ: Twitter

```

# Core APIs

# 1. Create tweet
POST /v1/tweets
Request:
{
  "content": "Hello world!",
  "media_urls": ["https://..."],
  "reply_to": "tweet_id" // optional
}

Response: 201 Created
{
  "tweet_id": "123456",
  "user_id": "789",
  "content": "Hello world!",
  "created_at": "2024-03-15T10:30:00Z"
}

# 2. Get user feed
GET /v1/feed?cursor={cursor}&limit=20
Response: 200 OK
{
  "tweets": [
    {
      "tweet_id": "123456",
      "user": {...},
      "content": "...",
      "likes": 100,
      "retweets": 50
    }
  ],
  "next_cursor": "abc123",
  "has_more": true
}

# 3. Like tweet
POST /v1/tweets/{tweet_id}/like
Response: 204 No Content

# 4. Follow user
POST /v1/users/{user_id}/follow
Response: 204 No Content

# 5. Get tweet
GET /v1/tweets/{tweet_id}
Response: 200 OK
{
  "tweet_id": "123456",
  "user": {...},
  "content": "...",
  "created_at": "...",
  "stats": {

```

```
    "likes": 100,  
    "retweets": 50,  
    "replies": 20  
  }  
}
```

Các Nguyên Tắc Thiết Kế API

1. Đặt tên nhất quán

```
# Good: Consistent  
POST /tweets           # Create  
GET  /tweets/{id}      # Read  
PUT   /tweets/{id}      # Update  
DELETE /tweets/{id}     # Delete  
  
# Bad: Inconsistent  
POST /createTweet  
GET  /getTweetById  
POST /tweet_update  
POST /remove_tweet
```

2. Quản lý phiên bản (Versioning)

```
# Good: Versioned  
GET /v1/tweets  
GET /v2/tweets # Breaking changes in v2  
  
# Can support both versions simultaneously
```

3. Phân trang (Pagination)

```
# Good: Cursor-based (for real-time data)  
GET /feed?cursor=xyz&limit=20  
  
# Also good: Offset-based (for stable data)  
GET /users?offset=100&limit=50
```

4. Phản hồi lỗi

```
# Standard error format
{
  "error": {
    "code": "RATE_LIMIT_EXCEEDED",
    "message": "Rate limit exceeded. Try again in 60 seconds.",
    "details": {
      "retry_after": 60
    }
  }
}
```

Danh Sách Kiểm Tra Cho API

- ☑ Đã định nghĩa các điểm cuối (endpoint) cốt lõi
- ☑ Đã chỉ rõ định dạng yêu cầu/phản hồi
- ☑ Phương thức HTTP phù hợp
- ☑ Có nhắc đến xác thực
- ☑ Đã chọn chiến lược phân trang
- ☑ Đã ghi nhận cách xử lý lỗi
- ☑ Đã cân nhắc giới hạn tần suất

Thời gian dành ra: 5-8 phút

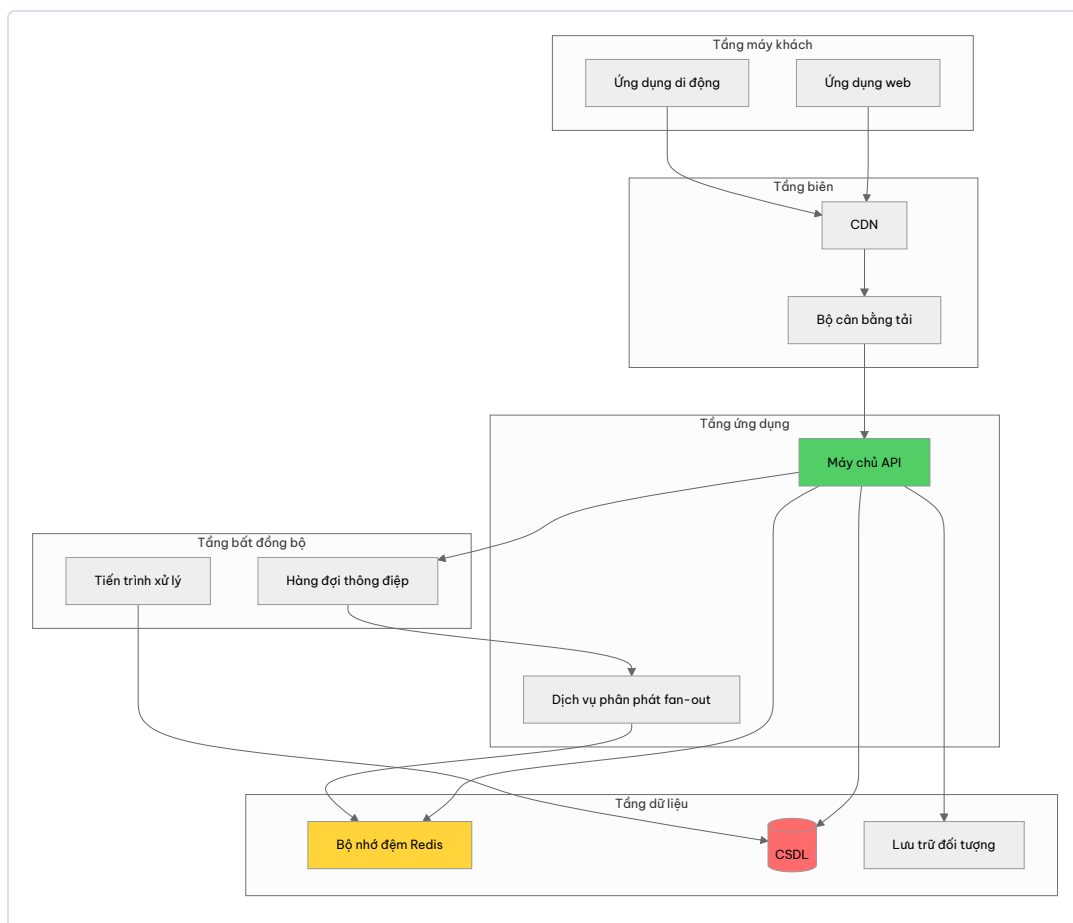
Đừng đi quá chi tiết. Hợp đồng ở mức tổng quan là đủ.

K - Key Components: Kiến Trúc Tổng Thể (*High-Level Architecture*)

Mục tiêu: Vẽ các hộp và mũi tên

Các Thành Phần Cần Có

Các thành phần tiêu chuẩn của một hệ thống web:



Kiến trúc tổng thể cho một hệ thống kiểu Twitter với các thành phần (components) chính

Mô Tả Các Thành Phần

Mỗi thành phần (component) cần được giải thích ngắn gọn:

Template

Component: Load Balancer

Purpose: Distribute traffic across API servers

Technology: AWS ALB / Nginx

Why: Handle 36K req/s, need multiple servers

Component: API Servers

Purpose: Handle business logic

Technology: Python/FastAPI (stateless)

Scale: 50+ servers (horizontal scaling)

Why: Stateless = easy to scale

Component: Redis Cache

Purpose: Cache user feeds, hot tweets

Technology: Redis Cluster

Capacity: 12 TB (100 nodes × 128 GB)

Why: Sub-10ms latency for reads

Component: Database

Purpose: Persistent storage (tweets, users, relationships)

Technology: PostgreSQL (sharded)

Scale: 50 shards by user_id

Why: Strong consistency for critical data

Component: Message Queue

Purpose: Async fanout of tweets to followers

Technology: Kafka

Why: Decouple write from fanout, handle spikes

Component: CDN

Purpose: Serve media files (images, videos)

Technology: CloudFront

Why: 7.2 GB/s bandwidth, global distribution

Luồng Dữ Liệu (*Data Flow*)

Chỉ ra các luồng xử lý quan trọng:

Luồng ghi (Đăng tweet):

1. Người dùng → Máy chủ API
2. API → Lưu vào cơ sở dữ liệu
3. API → Xuất bản sự kiện lên Kafka
4. Tiễn trình fan-out → Cập nhật bảng tin của những người theo dõi trong bộ nhớ đệm
5. Trả kết quả thành công cho người dùng

Luồng đọc (Tải bảng tin):

1. Người dùng → Máy chủ API
2. API → Kiểm tra bộ nhớ đệm (Redis)
3. Trúng bộ nhớ đệm → Trả về bảng tin đã lưu đệm
4. Trượt bộ nhớ đệm → Truy vấn cơ sở dữ liệu → Cập nhật bộ nhớ đệm
5. Trả bảng tin về cho người dùng

Nêu Bật Các Quyết Định Then Chốt

Làm nổi bật những lựa chọn quan trọng:

Quyết định 1: Fan-out khi ghi (kết hợp)

- Người dùng bình thường: Tính trước bảng tin (đọc nhanh)
- Người nổi tiếng: Truy vấn khi cần (tránh bùng nổ fan-out)

Đánh đổi: Khuếch đại ghi và hiệu năng đọc

Quyết định 2: PostgreSQL phân mảnh

- Phân mảnh theo user_id
- Ban đầu 50 mảnh

Đánh đổi: Độ phức tạp và quy mô

Quyết định 3: Redis làm bộ nhớ đệm dòng thời gian

- Lưu 1000 tweet gần nhất cho mỗi người dùng
- Không đặt thời hạn hết hạn (vô hiệu hoá chủ động)

Đánh đổi: Chi phí bộ nhớ và hiệu năng

Danh Sách Kiểm Tra Cho Các Thành Phần Chính

- ☑ Đã vẽ tất cả các thành phần chính
- ☑ Các kết nối được thể hiện bằng mũi tên
- ☑ Mỗi thành phần đều có mục đích
- ☑ Có nhắc đến lựa chọn công nghệ
- ☑ Đã giải thích luồng dữ liệu
- ☑ Đã làm nổi bật các luồng quan trọng
- ☑ Đã nêu rõ các quyết định then chốt

Thời gian dành ra: 10-15 phút

Đây là phần cốt lõi của buổi phỏng vấn. Hãy dành thời gian ở đây.

E - Elaborate: Đào Sâu

Mục tiêu: Thể hiện chuyên môn ở những phần cụ thể

Đào Sâu Vào Đầu

Người phỏng vấn thường sẽ dẫn dắt:

Người phỏng vấn: "Em có thể nói rõ hơn về dịch vụ fan-out không?"

Người phỏng vấn: "Em xử lý vấn đề người nổi tiếng thế nào?"

Người phỏng vấn: "Còn lược đồ cơ sở dữ liệu thì sao?"

Người phỏng vấn: "Em đảm bảo tính nhất quán của bảng tin bằng cách nào?"

Nếu không được hỏi, bạn tự chọn:

"Em xin đào sâu vào một vài phân quan trọng:

1. Chiến lược fan-out
2. Lược đồ cơ sở dữ liệu
3. Cách tiếp cận lưu đệm"

Ví Dụ Về Đào Sâu

Ví dụ 1: Chiến lược phân phát (*Fanout*)

```
# Hybrid fanout implementation

class FanoutService:
    CELEBRITY_THRESHOLD = 10_000 # followers

    def handle_new_tweet(self, tweet):
        author = self.get_user(tweet.user_id)

        if author.followers_count < self.CELEBRITY_THRESHOLD:
            # Normal user: Fanout on write
            self.fanout_to_followers(tweet)
        else:
            # Celebrity: Mark for fanout on read
            self.mark_celebrity_tweet(tweet)

    def fanout_to_followers(self, tweet):
        """Async fanout to all followers"""
        followers = self.db.get_followers(tweet.user_id)

        for follower_id in followers:
            self.queue.publish({
                'task': 'insert_into_feed',
                'follower_id': follower_id,
                'tweet_id': tweet.id,
                'timestamp': tweet.created_at
            })

    def get_feed(self, user_id):
        """Merge pre-computed + celebrity tweets"""
        # Get pre-computed feed
        feed = self.cache.get_feed(user_id)

        # Get celebrity tweets
        celebrities = self.get_celebrity_followees(user_id)
        celebrity_tweets = self.query_celebrity_tweets(celebrities)

        # Merge and sort
        return self.merge_and_rank(feed, celebrity_tweets)
```

Giải thích các đánh đổi (trade-offs):

Tại sao kết hợp?

- Vấn đề khuếch đại ghi: Người nổi tiếng có 10 triệu người theo dõi
→ 10 triệu lần cập nhật bộ nhớ đệm cho mỗi tweet = quá tốn kém
- Hiệu năng đọc: Người dùng bình thường (< 10K người theo dõi)
→ Tính trước bảng tin = đọc < 10ms

Đánh đổi: Một chút thiếu nhất quán với tweet của người nổi tiếng (nhất quán cuối cùng trong vài giây) là chấp nhận được

Ví dụ 2: Lược đồ cơ sở dữ liệu (Database Schema)

```
-- Users table
CREATE TABLE users (
  user_id BIGSERIAL PRIMARY KEY,
  username VARCHAR(50) UNIQUE NOT NULL,
  email VARCHAR(255) UNIQUE NOT NULL,
  created_at TIMESTAMP DEFAULT NOW(),
  followers_count INT DEFAULT 0,
  following_count INT DEFAULT 0
);

-- Tweets table (sharded by user_id)
CREATE TABLE tweets (
  tweet_id BIGINT PRIMARY KEY, -- Snowflake ID
  user_id BIGINT NOT NULL,
  content TEXT NOT NULL,
  media_urls TEXT[],
  created_at TIMESTAMP DEFAULT NOW(),
  reply_to BIGINT, -- NULL if not reply

  INDEX idx_user_created (user_id, created_at DESC),
  INDEX idx_created (created_at DESC)
);

-- Relationships table (sharded by follower_id)
CREATE TABLE follows (
  follower_id BIGINT NOT NULL,
  followee_id BIGINT NOT NULL,
  created_at TIMESTAMP DEFAULT NOW(),

  PRIMARY KEY (follower_id, followee_id),
  INDEX idx_followee (followee_id) -- For follower count
);

-- Engagement table (sharded by tweet_id)
CREATE TABLE likes (
  tweet_id BIGINT NOT NULL,
  user_id BIGINT NOT NULL,
  created_at TIMESTAMP DEFAULT NOW(),

  PRIMARY KEY (tweet_id, user_id)
);
```

Giải thích cách phân mảnh dữ liệu (sharding):

Chiến lược phân mảnh:

- tweets: Phân mảnh theo user_id (đặt các tweet của một người dùng cùng chỗ)
- follows: Phân mảnh theo follower_id (tra cứu người theo dõi nhanh)
- likes: Phân mảnh theo tweet_id (đếm lượt thích nhanh)

Tại sao phân mảnh như vậy?

- Phân lớn truy vấn xoay quanh người dùng
- "Lấy các tweet của người dùng" → Truy vấn trên một mảnh duy nhất
- "Lấy bảng tin" → Truy vấn những người mình theo dõi (có thể liên mảnh, nhưng đã được lưu đệm)

Đánh đôi: Một số thao tác cần truy vấn liên mảnh

Ví dụ 3: Điểm nghẽn (Bottlenecks) & Giải pháp

Identify và solve bottlenecks

Bottleneck 1: Fanout queue lag

Problem: 10M followers → 10M messages → Queue backup

Solution:

- Priority queue (VIP users processed first)
- Batch processing (group updates)
- Circuit breaker (skip if queue too long)

Bottleneck 2: Hot celebrity timeline

Problem: Taylor Swift's timeline → 10M cache reads/second

Solution:

- Multi-tier caching (L1: Local cache, L2: Redis)
- CDN caching for celebrity profiles
- Rate limiting per user

Bottleneck 3: Database writes during peaks

Problem: Viral event → 100K tweets/second

Solution:

- Write buffer (queue writes, batch insert)
- Database connection pooling
- Auto-scaling write capacity

Danh Sách Kiểm Tra Cho Phần Đào Sâu

- ☑ Đào sâu 2-3 thành phần quan trọng
- ☑ Trình bày chi tiết triển khai
- ☑ Giải thích thuật toán/logic
- ☑ Thảo luận các đánh đôi
- ☑ Xử lý các điểm nghẽn
- ☑ Nhắc đến cách xử lý sự cố
- ☑ Trình bày chiến lược giám sát

Thời gian dành ra: 10-15 phút

Ở đây chiều sâu là quan trọng. Hãy thể hiện chuyên môn.

Quản Lý Thời Gian: Diễn Tiến 45 Phút

Kỷ luật thời gian nghiêm ngặt giúp bạn thắng buổi phỏng vấn.



Phân bổ thời gian cho mỗi giai đoạn trong buổi phỏng vấn 45 phút

Thời Gian Cho Từng Giai Đoạn

Phút 0-7: Scope

Nên làm:

- ✓ Đặt câu hỏi để làm rõ
- ✓ Liệt kê yêu cầu chức năng
- ✓ Xác nhận yêu cầu phi chức năng
- ✓ Nêu rõ các giả định

KHÔNG nên làm:

- x Nhảy ngay vào thiết kế
- x Bỏ qua yêu cầu
- x Giả định quá nhiều

Phút 7-14: Numbers

Nên làm:

- ✓ Viết phép tính lên bảng
- ✓ Ước lượng lưu lượng (đọc/ghi)
- ✓ Tính toán dung lượng lưu trữ
- ✓ Xác định tỷ lệ đọc/ghi

KHÔNG nên làm:

- x Bỏ qua phần tính toán
- x Đoán mà không tính
- x Quá chính xác (ước lượng gần đúng là được)

Phút 14-21: API

Nên làm:

- ✓ Định nghĩa 4-6 điểm cuối cốt lõi
- ✓ Trình bày yêu cầu/phản hồi
- ✓ Nhắc đến xác thực
- ✓ Ghi chú về phân trang

KHÔNG nên làm:

- x Thiết kế mọi điểm cuối
- x Đi quá chi tiết vào nội dung yêu cầu
- x Tranh luận REST hay GraphQL

Phút 21-33: Key Components

Nên làm:

- ✓ Vẽ sơ đồ kiến trúc
- ✓ Gắn nhãn tất cả các thành phần
- ✓ Thể hiện luồng dữ liệu
- ✓ Giải thích các quyết định then chốt

KHÔNG nên làm:

- x Vẽ quá chi tiết
- x Đưa vào mọi thành phần có thể có
- x Bỏ qua phần giải thích

Phút 33-45: Elaborate

Nên làm:

- ✓ Đào sâu 2-3 phần
- ✓ Trình bày cách triển khai
- ✓ Thảo luận các đánh đổi
- ✓ Xử lý các kịch bản sự cố

KHÔNG nên làm:

- x Có bao quát mọi thứ
- x Mắc kẹt ở một phần
- x Phớt lờ gợi ý của người phỏng vấn

Mẹo Quản Lý Thời Gian

1. Đo đồng hồ

Thỉnh thoảng liếc nhìn đồng hồ`
7 phút → Nên xong phần Scope
14 phút → Nên xong phần Numbers
21 phút → Nên xong phần API

Nếu chậm tiến độ → Tăng tốc
Nếu đi trước → Bổ sung thêm chiều sâu

2. Nói ra nhịp độ của mình

"Em sẽ dành 5 phút cho phần yêu cầu..."
"Em sẽ tính nhanh vài con số..."
"Em xin vẽ kiến trúc, việc này mất khoảng 10 phút..."

→ Cho thấy bạn ý thức về thời gian
→ Đặt kỳ vọng rõ ràng

3. Biết khi nào nên chuyển tiếp

Người phỏng vấn: "Tạm thời vậy là đủ rồi, mình chuyển sang phần khác nhé"
→ Đừng tranh cãi, hãy chuyển tiếp

Bạn bị kẹt ở một chỗ quá 3 phút
→ Nói "Em xin quay lại phần này sau" và chuyển tiếp

4. Giữ lại thời gian dự phòng

Hoàn thành trước phút thứ 40
→ 5 phút dự phòng cho:
- Câu hỏi
- Những phần đào sâu bất ngờ
- Làm rõ thêm
- Sửa sai sót

Những Sai Lầm Thường Gặp & Cách Tránh

Sai Lầm 1: Nhảy Ngay Vào Giải Pháp

Tôi:

Người phỏng vấn: "Thiết kế Twitter"

Ứng viên: "Vâng, vậy mình sẽ dùng vi dịch vụ, Kafka, Cassandra, và..."

- Không thu thập yêu cầu
- Không tính toán
- Tư duy ưu tiên giải pháp

Tốt:

Người phỏng vấn: "Thiết kế Twitter"

Ứng viên: "Trước tiên em xin làm rõ các yêu cầu.

Quy mô dự kiến là bao nhiêu?

Những tính năng nào được ưu tiên?

..."

- Cách tiếp cận bài bản
- Thể hiện quá trình tư duy

Sai Lầm 2: Thiết Kế Thừa (*Over-Engineering*)

Tôi:

"Mình sẽ dùng Kubernetes, Kafka, Cassandra, vi dịch vụ, service mesh, event sourcing, CQRS..."

- Trò chơi "bingo" thuật ngữ thời thượng
- Phức tạp mà không có lý do chính đáng

Tốt:

"Với 100 nghìn người dùng ban đầu, em sẽ bắt đầu với:

- Ứng dụng nguyên khối
- PostgreSQL
- Redis để lưu đệm
- Kiến trúc đơn giản

Mình có thể thêm độ phức tạp khi mở rộng lên hàng triệu người dùng."

- Giải pháp vừa vặn
- Lộ trình phát triển rõ ràng

Sai Lầm 3: Phớt Lờ Người Phỏng Vấn

Tôi:

Người phỏng vấn: "Em xử lý người dùng nổi tiếng thế nào?"

Ứng viên: "Đề'em giải thích xong phân cơ sở dữ liệu trước đã..."

- Không lắng nghe
- Bỏ lỡ sự dẫn dắt

Tốt:

Người phỏng vấn: "Em xử lý người dùng nổi tiếng thế nào?"

Ứng viên: "Câu hỏi hay ạ. Em xin trả lời luôn.

Với người nổi tiếng có > 10K người theo dõi,
em sẽ dùng mô hình kéo (pull) thay vì đẩy (push)..."

- Phản hồi nhanh nhạy
- Linh hoạt

Sai Lầm 4: Không Thảo Luận Về Đánh Đổi (*Trade-off*)

Tôi:

"Mình dùng bộ nhớ đệm vì nó nhanh."

- Hời hợt
- Thiếu chiều sâu

Tốt:

"Mình dùng bộ nhớ đệm cho thao tác đọc vì:

- 95% lưu lượng là đọc
- Yêu cầu độ trễ dưới 10ms

Đánh đổi:

- Tăng thêm độ phức tạp
- Có thể có dữ liệu cũ (giảm thiểu bằng TTL)
- Chi phí bộ nhớ (~\$10K/tháng cho 10TB Redis)

Nhưng sự đánh đổi này xứng đáng với lợi ích về hiệu năng."

- Thể hiện sự thấu hiểu
- Có lý lẽ cho các quyết định

Sai Lầm 5: Thiết Kế Trong Im Lặng

Tôi:

[Lặng lẽ vẽ lên bảng trắng suốt 5 phút]

- Người phỏng vấn không biết bạn đang nghĩ gì
- Không thể đưa ra gợi ý

Tốt:

"Em đang vẽ kiến trúc tổng thể."

Bắt đầu với tầng máy khách, rồi đến máy chủ API, sau đó là tầng dữ liệu. Em sẽ nói chúng bằng mũi tên để thể hiện luồng dữ liệu..."

- Suy nghĩ thành lời
- Tương tác với người phỏng vấn

Mẫu Cấu Trúc Câu Trả Lời

Hãy dùng mẫu này cho mọi bài toán thiết kế hệ thống:

1. SCOPE (5-8 phút)

"Em xin làm rõ các yêu cầu..."

Chức năng:

- Tính năng 1
- Tính năng 2
- Ngoài phạm vi: X, Y

Phi chức năng:

- X triệu người dùng
- Y yêu cầu/giây
- Yêu cầu độ trễ Z

2. NUMBERS (5-8 phút)

"Em xin ước lượng quy mô..."

Lưu lượng:

- Đọc: X req/s
- Ghi: Y req/s
- Tỷ lệ: Đọc:Ghi

Lưu trữ:

- Mỗi ngày: A GB
- Mỗi năm: B TB

Băng thông:

- Vào: C MB/s
- Ra: D GB/s

3. API (5-8 phút)

"Em xin định nghĩa các API chính..."

POST /resource

GET /resource/{id}

...

4. KEY COMPONENTS (10-15 phút)

"Em xin vẽ kiến trúc..."

[Vẽ sơ đồ]

Các thành phần:

- Bộ cân bằng tải: ...
- Máy chủ API: ...
- Bộ nhớ đệm: ...
- Cơ sở dữ liệu: ...

Luồng dữ liệu:

- Luồng ghi: ...
- Luồng đọc: ...

Quyết định then chốt:

- Quyết định 1 + đánh đổi
- Quyết định 2 + đánh đổi

5. ELABORATE (10-15 phút)
"Em xin đào sâu vào..."

[Chọn 2-3 phần]

Phần 1: Chi tiết triển khai

Phần 2: Phân tích đánh đôi

Phần 3: Xử lý sự cố

6. TỔNG KẾT (nếu còn thời gian)
"Tóm lại..."

Điểm nhấn:

- Điểm nhấn 1 → Giải pháp
- Điểm nhấn 2 → Giải pháp

Giám sát:

- Chỉ số 1
- Chỉ số 2

Cải tiến trong tương lai:

- Cải tiến 1
- Cải tiến 2

Bài Tập Thực Hành

Hãy thử SNAKE với bài toán này:

Bài toán: Thiết kế dịch vụ rút gọn URL (bit.ly)

Đến lượt bạn:

Tự bấm giờ: 45 phút

S - Scope:

Viết ra:

- Yêu cầu chức năng (3-5 mục)
- Yêu cầu phi chức năng (quy mô, độ trễ, v.v.)
- Các giả định

Thời gian: 7 phút

N - Numbers:

Tính toán:

- Lưu lượng (đọc, ghi)
- Lưu trữ (5 năm)
- Băng thông

Thời gian: 7 phút

A - API:

Thiết kế:

- POST /shorten
- GET /{shortCode}
- Có thể thêm 1-2 cái nữa

Thời gian: 7 phút

K - Key Components:

Vẽ:

- Sơ đồ kiến trúc
- Gắn nhãn các thành phần
- Thể hiện luồng dữ liệu

Thời gian: 12 phút

E - Elaborate:

Đào sâu:

- Thuật toán sinh mã rút gọn
- Lựa chọn cơ sở dữ liệu
- Chiến lược lưu đệm

Thời gian: 12 phút

Tổng: 45 phút

Tự đánh giá:

Bạn đã:

- ☒ Đi qua đủ các bước SNAKE chưa?
- ☒ Quản lý thời gian tốt chưa?
- ☒ Trình bày phép tính chưa?
- ☒ Có lý lẽ cho các quyết định chưa?
- ☒ Thảo luận các đánh đổi chưa?
- ☒ Suy nghĩ thành lời chưa?

Những Điểm Cốt Lõi

Khung tư duy SNAKE = Cấu trúc dẫn đến thành công

S - Scope: Làm rõ trước khi thiết kế
N - Numbers: Tính toán đề có lý lẽ
A - API: Định nghĩa hợp đồng từ sớm
K - Key Components: Thể hiện tư duy kiến trúc
E - Elaborate: Chứng tỏ chiều sâu

Quản lý thời gian là then chốt:

45 phút trôi qua rất nhanh
Cần kỷ luật nghiêm ngặt
Theo dõi đồng hồ
Biết khi nào nên chuyển tiếp

Giao tiếp quan trọng ngang với kiến thức:

Suy nghĩ thành lời
Tương tác với người phỏng vấn
Linh hoạt
Trình bày lập luận, không chỉ đáp án

Mẫu (Pattern) thành công thường gặp:

Scope + Numbers vững chắc → Yêu cầu rõ ràng
→ Thiết kế API có cơ sở
→ Kiến trúc có lý lẽ thuyết phục
→ Đào sâu kỹ thuật
→ Vượt qua phỏng vấn ✓

Có công mài sắt, có ngày nên kim:

Khung tư duy SNAKE lúc đầu có vẻ gượng gạo
Sau 5-10 buổi luyện tập → Tự nhiên
Sau hơn 20 buổi → Thành phần xạ

Các bài toán đề luyện tập:
- Twitter
- Instagram
- Uber
- Netflix
- Ticketmaster
- WhatsApp

Hãy nhớ:

Phòng vấn kiểm tra quá trình, không phải sự hoàn hảo
Kiến trúc hoàn hảo không tồn tại
Thể hiện tư duy > Thể hiện kiến thức
Có lý lẽ cho quyết định > Áp dụng mẫu thiết kế
Giao tiếp rõ ràng > Biết mọi thứ

Khung tư duy SNAKE đưa bạn từ lan man hỗn loạn → tư duy có cấu trúc.

Hãy làm chủ khung tư duy này. Luyện tập không ngừng. Chinh phục mọi buổi phỏng vấn.

Đánh Đổi (Trade-offs) & Ra Quyết Định Kiến Trúc (Architecture Decision Making)

Sau 3 năm làm việc, tôi được thăng chức lên kỹ sư lâu năm (Senior Engineer).

Quản lý của tôi hỏi: "Em nghĩ điều gì phân biệt một kỹ sư lâu năm với một kỹ sư mới vào nghề?"

Tôi tự tin: "Kỹ sư lâu năm biết nhiều mẫu thiết kế (patterns) hơn, viết code giỏi hơn, có nhiều kinh nghiệm hơn."

Anh ấy lắc đầu: "Không. Kỹ sư lâu năm giỏi ở **tư duy đánh đổi** (trade-off thinking)."

Một tuần sau, trong buổi đánh giá kiến trúc (architecture review):

Kỹ sư mới vào nghề (Junior Engineer): "Em đề xuất dùng vi dịch vụ (microservices) cho dự án mới."

Quản lý: "Tại sao?"

Kỹ sư mới: "Vì vi dịch vụ là thực hành tốt nhất (best practice), mở rộng (scale) tốt, Netflix cũng dùng."

Quản lý: "Với 3 lập trình viên (developers) và 500 người dùng (users), chi phí (cost) của vi dịch vụ là gì? Lợi ích là gì? Sự đánh đổi (Trade-off) đó có đáng không?"

Kỹ sư mới: "... Em chưa nghĩ đến."

Đó là lúc tôi thực sự hiểu.

Kỹ sư lâu năm không phải là người biết nhiều nhất. Mà là người **lập luận về các sự đánh đổi** giỏi nhất.

Tại Sao Tư Duy Đánh Đổi Là Kỹ Năng Cốt Lõi?

Sự Thật Nền Tảng

Trong kiến trúc phần mềm (software architecture), không có giải pháp hoàn hảo.

Giải pháp hoàn hảo = Hoang đường
Giải pháp thực tế = Các sự đánh đổi

Mọi quyết định kỹ thuật đều có:
Lợi ích (được)
Chi phí (mất)

Chọn giải pháp = Chọn chấp nhận sự đánh đổi nào

Tư Duy Của Kỹ Sư Mới Vào Nghề (*Junior*) và Kỹ Sư Lâu Năm (*Senior*)



Kỹ sư mới vào nghề sao chép giải pháp, kỹ sư lâu năm đánh giá các sự đánh đổi

So sánh ví dụ:

Bài toán: Cần một tầng bộ nhớ đệm

Kỹ sư mới:

"Dùng Redis. Nó nhanh và ai cũng dùng."

Kỹ sư lâu năm:

"Hãy đánh giá các lựa chọn lưu đệm:

Phương án 1: Redis

Rất nhanh (< 1ms)

Cấu trúc dữ liệu phong phú

Lưu bên là tùy chọn

Bộ nhớ đắt khi ở quy mô lớn

Chi phí vận hành phụ trội

Chi phí: ~\$500/tháng cho 16GB

Phương án 2: Memcached

Đơn giản, ổn định

Tồn bộ nhớ ít hơn một chút

Không lưu bên

Cấu trúc dữ liệu hạn chế

Chi phí: ~\$400/tháng

Phương án 3: Bộ nhớ đệm cấp ứng dụng (trong tiến trình)

Độ trễ bằng 0

Không tốn chi phí hạ tầng

Đơn giản

Không chia sẻ được giữa các máy chủ

Bị giới hạn bởi bộ nhớ máy chủ

Với bối cảnh của chúng ta:

- 5 máy chủ ứng dụng
- Thiên về đọc (kỳ vọng tỷ lệ trúng bộ nhớ đệm 90%)
- Công ty khởi nghiệp cần tiết kiệm ngân sách
- Tình huống sử dụng đơn giản (chỉ key-value)

Khuyến nghị: Memcached

Lý do: Sự đơn giản + tiết kiệm chi phí có giá trị hơn các tính năng của Redis mà ta không cần.

Ta có thể chuyển sang Redis sau nếu cần lưu bên."

- Đã phân tích các phương án
- Đã liệt kê các đánh đổi
- Đã xem xét bối cảnh
- Đã lý giải quyết định

Thấy sự khác biệt chưa?

Kỹ sư lâu năm không chỉ chọn giải pháp. Họ **lý giải vì sao giải pháp đó phù hợp với bối cảnh**.

Những Đánh Đổi Lớn Mà Mọi Kiến Trúc Sư (Architect) Phải Hiểu

Đánh đổi 1: Tính Nhất Quán (Consistency) và Tính Sẵn Sàng (Availability)

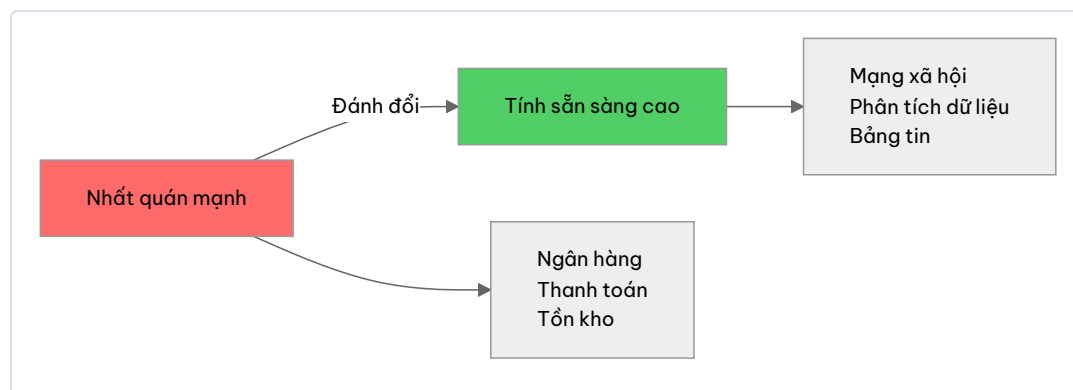
Từ định lý CAP (CAP Theorem):

Trong hệ thống phân tán khi có phân vùng mạng:
Không thể có CẢ tính nhất quán hoàn hảo LÃN tính sẵn sàng hoàn hảo

Phải chọn:

- CP (Nhất quán + Chịu phân vùng)
- AP (Sẵn sàng + Chịu phân vùng)

Diễn giải theo thực tế:



Tính nhất quán và tính sẵn sàng: Chọn dựa trên yêu cầu (requirements) kinh doanh

Kịch bản: Tồn kho thương mại điện tử

```

# Strong Consistency (CP)
def purchase_product(product_id, quantity):
    # Lock row, ensure stock accurate
    with db.transaction():
        product = db.query("SELECT * FROM products WHERE id = ? FOR UPDATE",
                             product_id)

        if product.stock < quantity:
            return {"error": "Out of stock"}

        product.stock -= quantity
        db.commit()

    return {"success": True}

# Trade-offs:
No overselling (critical!)
Accurate inventory
Slow (lock + transaction overhead)
Lower availability (if DB down, can't sell)
Poor scalability (locks limit concurrency)

Why choose this: Overselling = angry customers + revenue loss
Consistency > Availability for this use case

```

Kịch bản: Lướt thích trên mạng xã hội

```

# High Availability (AP)
def like_post(post_id, user_id):
    # Async write, eventual consistency
    cache.increment(f"post:{post_id}:likes")
    queue.publish({
        "event": "post_liked",
        "post_id": post_id,
        "user_id": user_id
    })

    return {"success": True}

# Worker processes queue eventually
@worker.task
def process_like(event):
    db.execute("INSERT INTO likes ...")

# Trade-offs:
Super fast (< 10ms)
Always available
High scalability
Like count might be slightly off
Eventual consistency (delay 1-2s)

Why choose this: User doesn't care if count is 99 or 101
Availability > Consistency for this use case

```

Khung ra quyết định (Decision framework):

```
def choose_consistency_level(feature):
    """Framework để decide consistency vs availability"""

    # Ask critical questions
    questions = {
        "financial_impact": "Sai data = mất tiền?",
        "user_experience": "User notice inconsistency?",
        "legal_requirement": "Compliance yêu cầu consistency?",
        "failure_cost": "Downtime cost bao nhiêu?"
    }

    if feature.financial_impact == "high":
        return "Strong Consistency (CP)"

    if feature.legal_requirement == "yes":
        return "Strong Consistency (CP)"

    if feature.user_experience == "unaffected_by_slight_delay":
        return "Eventual Consistency (AP)"

    if feature.failure_cost == "very_high":
        return "High Availability (AP)"

    # Default: Strong consistency (safer)
    return "Strong Consistency (CP)"

# Examples
choose_consistency_level(Payment)           # → CP (financial)
choose_consistency_level(UserProfile)       # → CP (user-facing)
choose_consistency_level(ViewCount)         # → AP (doesn't matter)
choose_consistency_level(Recommendations)  # → AP (can be stale)
```

Đánh đổi 2: Độ Trễ (Latency) và Chi Phí

Thực tế là: Tốc độ thì tốn tiền.

Độ trễ thấp hơn = Chi phí cao hơn (thường là vậy)

Các cách giảm độ trễ:

- Thêm máy chủ (tốn kém)
- Phần cứng nhanh hơn (tốn kém)
- Đặt gần người dùng hơn (CDN = tốn kém)
- Lưu đệm nhiều hơn (bộ nhớ = tốn kém)
- Tối ưu cơ sở dữ liệu (thời gian kỹ sư = tốn kém)

Kịch bản: Thời gian phản hồi (Response Time) của API


```
# Baseline: No optimization
# Latency: 500ms
# Cost: $500/month (basic setup)

# Option 1: Add Redis cache
# Latency: 50ms (10x improvement)
# Cost: $500 + $300 = $800/month
# ROI: Worth it? Depends on user value

# Option 2: Add CDN for static assets
# Latency: 200ms (2.5x improvement)
# Cost: $500 + $100 = $600/month
# ROI: Better bang for buck

# Option 3: Upgrade database (vertical scale)
# Latency: 300ms (1.7x improvement)
# Cost: $500 + $400 = $900/month
# ROI: Expensive for little gain

# Option 4: All of the above
# Latency: 30ms (16x improvement)
# Cost: $500 + $300 + $100 + $400 = $1,300/month
# ROI: Overkill? Depends on requirements
```

Khung ra quyết định:

```
# Latency requirements tiers

Tier 1: Real-time (< 50ms)
Use cases: Gaming, trading, video calls
Cost: Very high
Solution: Aggressive caching, edge computing, WebSocket

Tier 2: Interactive (< 200ms)
Use cases: Web apps, mobile apps
Cost: Medium
Solution: Cache, CDN, optimized queries

Tier 3: Responsive (< 1s)
Use cases: Reports, batch operations
Cost: Low
Solution: Basic optimization, no over-engineering

Tier 4: Background (< 10s)
Use cases: Email, notifications, analytics
Cost: Very low
Solution: Async processing, queues
```

Phân tích đánh đổi:

Câu hỏi: "Có nên thêm bộ nhớ đệm Redis không?"

Đừng trả lời ngay. Hãy phân tích:

Trạng thái hiện tại:

- Độ trễ API: 400ms
- Truy vấn CSDL: 300ms (điểm nghẽn)
- Người dùng: 10 nghìn
- Phần nản: Ít

Khi có Redis:

- Độ trễ API: 50ms (cải thiện 8 lần)
- Chi phí: +\$300/tháng
- Độ phức tạp: +1 thành phần cần quản lý
- Vô hiệu hoá bộ nhớ đệm: Vấn đề mới

Sự đánh đổi có đáng không?

- 10 nghìn người dùng × \$5/tháng doanh thu = \$50K/tháng
- Trải nghiệm tốt hơn có thể tăng tỷ lệ giữ chân 5% = +\$2.5K/tháng
- Chi phí = \$300/tháng
- ROI = \$2.5K / \$300 = 8.3 lần

Quyết định: Có, đáng làm

Nguyên tắc cá nhân:

Tối ưu độ trễ khi:

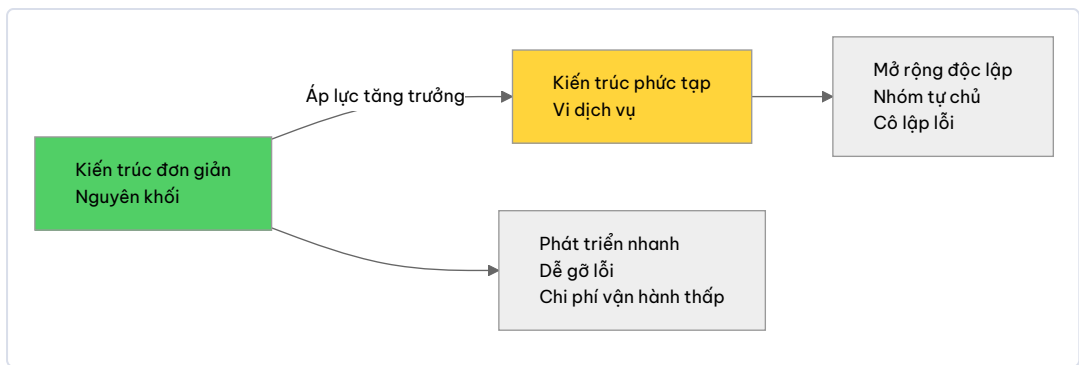
- ✓ Trải nghiệm người dùng bị ảnh hưởng trực tiếp
- ✓ Chỉ số kinh doanh tương quan với tốc độ
- ✓ Chi phí được bù đắp bằng doanh thu/tỷ lệ giữ chân
- ✓ Độ trễ hiện tại vượt quá yêu cầu của phân khúc

Đừng tối ưu khi:

- × Người dùng không nhận ra khác biệt
- × Không có tác động kinh doanh
- × Chi phí quá cao so với lợi ích
- × Quá sớm (chưa có quy mô)

Đánh đổi 3: Sự Đơn Giản và Khả Năng Mở Rộng (Scalability)

Mâu thuẫn muôn thuở: Đơn giản ngay bây giờ hay mở rộng về sau?



Phổ giữa sự đơn giản và khả năng mở rộng: Mỗi giai đoạn cần một cách tiếp cận khác nhau

Kịch bản: Quyết định kiến trúc cho công ty khởi nghiệp (*startup*)

```

# Context
startup = {
  "users": 1000,
  "developers": 3,
  "runway": "12 months",
  "goal": "Product-market fit"
}

# Option 1: Microservices from day 1
architecture_1 = {
  "services": ["user-service", "order-service", "payment-service",
"notification-service"],
  "infrastructure": ["Kubernetes", "Kafka", "Service Mesh"],
  "time_to_market": "6 months",
  "operational_complexity": "High",
  "cost": "$5000/month"
}

# Trade-offs:
"Scales to millions" (not needed yet)
Modern architecture
6 months to MVP (too slow!)
High complexity với 3 devs
Expensive for 1K users
Might not reach PMF before running out of money

# Option 2: Monolith with good structure
architecture_2 = {
  "structure": "Modular monolith",
  "infrastructure": ["Heroku", "PostgreSQL", "Redis"],
  "time_to_market": "6 weeks",
  "operational_complexity": "Low",
  "cost": "$500/month"
}

# Trade-offs:
Ship fast (critical for startup!)
Easy to change (iterate quickly)
Low cost
Simple operations
Might need to refactor later (acceptable!)
Not "impressive" architecture

# Which to choose?
# Startup context → Option 2
# Why: Speed > Scalability at this stage
# Can refactor when/if successful

```

Lộ Trình Tái Cấu Trúc (Refactor) (Thường Tốt Hơn):

Giai đoạn 1: Nguyên khối (0-10 nghìn người dùng)

- Đơn giản, lắp nhanh
- Tìm sự phù hợp giữa sản phẩm và thị trường

Thời gian: 6 tháng

Giai đoạn 2: Nguyên khối dạng mô-đun (10 nghìn-100 nghìn người dùng)

- Tách mô-đun, ranh giới rõ ràng
- Thêm bộ nhớ đệm, bản sao chỉ đọc

Thời gian: +3 tháng

Giai đoạn 3: Vi dịch vụ có chọn lọc (100 nghìn-1 triệu người dùng)

- Chỉ tách các dịch vụ "nóng"
- Giữ phần lõi nguyên khối

Thời gian: +6 tháng

Giai đoạn 4: Vi dịch vụ toàn phần (nếu cần, ở mức 1 triệu+)

- Điểm nghẽn đã được chứng minh
- Có đủ nhóm & nguồn lực

Thời gian: +12 tháng

Tổng: ~27 tháng, nhưng có doanh thu và nhóm lớn dân số với

Vi dịch vụ ngay từ ngày đầu: 6 tháng, có thể không bao giờ ra mắt được

Khung ra quyết định:

```
def choose_architecture(context):  
    """Simple vs Complex architecture decision"""  
  
    if context.users < 10_000:  
        return "Monolith"  
  
    if context.developers < 10:  
        return "Modular Monolith"  
  
    if context.has_proven_bottlenecks and context.can_afford_complexity:  
        return "Selective Microservices"  
  
    if context.users > 10_000_000 and context.developers > 50:  
        return "Full Microservices"  
  
    # Default: Keep it simple  
    return "Monolith"
```

Đánh đổi 4: Tối Ưu Hoá Quá Sớm (*Premature Optimization*)

"Tối ưu hoá quá sớm là nguồn gốc của mọi tội lỗi" - Donald Knuth

Cái bẫy:

"Hãy xây dựng đề'mở rộng được tới 100 triệu người dùng ngay từ ngày đầu!"

Thực tế:

- 99% công ty khởi nghiệp không bao giờ đạt 100 triệu người dùng
- Thiết kế' thừa cho quy mô bạn chưa có = lãng phí thời gian
- Thời gian lãng phí = chi phí cơ hội

Cách tiếp cận tốt hơn:

- Xây dựng cho gấp 2 lần quy mô hiện tại
- Giám sát và tối ưu khi cần
- Lặp lại dựa trên dữ liệu thực

Ví dụ: Tối ưu quá mức

Engineer's proposal for new feature

"We need to build a notification system.
I propose:

- Kafka for event streaming
- Cassandra for storage (scales to billions)
- Microservices architecture
- Service mesh for reliability
- Custom protocol for low latency
- Machine learning for personalization

Timeline: 4 months

Cost: \$50K in infrastructure

Team: 3 developers full-time"

Senior architect's response:

"Let's start simpler:

Current scale:

- 5K users
- ~100 notifications/day
- 99% email, 1% push

MVP approach:

- PostgreSQL for storage (we already have it)
- Background job processor (Celery)
- SendGrid API for emails
- FCM for push notifications

Timeline: 1 week

Cost: \$50/month (SendGrid + FCM)

Team: 1 developer part-time

We can refactor when we hit 100K users.

Right now, shipping fast > perfect architecture."

- 4 months vs 1 week
- \$50K vs \$50/month
- 3 devs vs 1 dev

Which would you choose?

Khi nào nên tối ưu:

Tối ưu khi:

- Có vấn đề hiệu năng thực sự
- Vấn đề ảnh hưởng tới người dùng/doanh thu
- Có dữ liệu chỉ ra điểm nghẽn
- Chi phí của việc không tối ưu > chi phí của việc tối ưu

Đừng tối ưu khi:

- "Biết đâu một ngày nào đó sẽ cần"
- "Thực hành tốt nhất bảo vậy"
- Không có đo lường/dữ liệu
- Người dùng không phàn nàn

Trực Giác Về Định Lý CAP (Vượt Ra Ngoài Lý Thuyết)

Định lý CAP được đơn giản hoá:

C - Consistency (Nhất quán): Mọi nút thấy cùng một dữ liệu

A - Availability (Sẵn sàng): Hệ thống luôn phản hồi

P - Partition tolerance (Chịu phân vùng): Vẫn hoạt động dù mạng gặp sự cố

Trong hệ thống phân tán: Chỉ có thể đảm bảo 2 trong 3

Nhưng trong thực tế, **phân vùng mạng** (*network partitions*) **luôn xảy ra**. Nên lựa chọn thực sự là:

CP (Nhất quán + Chịu phân vùng):

- Chọn tính nhất quán thay vì tính sẵn sàng
- Hệ thống có thể từ chối yêu cầu để giữ nhất quán

AP (Sẵn sàng + Chịu phân vùng):

- Chọn tính sẵn sàng thay vì tính nhất quán
- Hệ thống luôn phản hồi, có thể hiển thị dữ liệu cũ

Ví dụ trực quan:

Máy ATM ngân hàng (Hệ thống CP):

Kịch bản: Phân vùng mạng giữa các ATM và ngân hàng trung tâm

Hành vi CP:

- ATM phát hiện phân vùng
- ATM tạm khoá chức năng rút tiền
- Hiện thị lỗi: "Dịch vụ tạm thời không khả dụng"
- Ngăn số dư không nhất quán

Vì sao chọn CP:

- Không thể mạo hiểm hiện thị sai số dư
- Không thể mạo hiểm rút tiền hai lần
- Tính đúng đắn > Tính sẵn sàng

Tác động tới người dùng:

- Bực bội (không rút được tiền)
- Nhưng giữ được niềm tin (dữ liệu không bị hỏng)

Bảng tin Facebook (Hệ thống AP):

Kịch bản: Phân vùng mạng giữa các trung tâm dữ liệu

Hành vi AP:

- Cả hai trung tâm dữ liệu tiếp tục phục vụ
- Người dùng có thể thấy bảng tin hơi khác nhau
- Cuối cùng sẽ đồng bộ lại khi phân vùng được khắc phục
- Một số lượt thích có thể mất vài giây mới hiện

Vì sao chọn AP:

- Người dùng vẫn duyệt được (trải nghiệm tốt)
- Bảng tin hơi cũ một chút là chấp nhận được
- Tính sẵn sàng > Nhất quán hoàn hảo

Tác động tới người dùng:

- Vẫn dùng được ứng dụng (hài lòng)
- Không để ý độ trễ ~2 giây ở số lượt thích

Ma trận quyết định:

Dùng CP khi:

- ✓ Giao dịch tài chính
- ✓ Quản lý tồn kho
- ✓ Xác thực người dùng
- ✓ Hệ thống đặt chỗ (ghế, vé)
- ✓ Hồ sơ y tế

Dùng AP khi:

- ✓ Bảng tin mạng xã hội
- ✓ Số lượt xem/lượt thích
- ✓ Gợi ý
- ✓ Bảng điều khiển phân tích
- ✓ Kết quả tìm kiếm

Cách Bảo Vệ Các Quyết Định Kiến Trúc

Trong các buổi đánh giá kiến trúc (architecture reviews), bạn sẽ bị chất vấn. Đây là cách để bảo vệ quan điểm:

Khung Bảo Vệ (Defense Framework)

1. Nêu rõ bài toán
2. Liệt kê các phương án đã cân nhắc
3. Giải thích đánh đổi của từng phương án
4. Chỉ ra vì sao giải pháp được chọn phù hợp với bối cảnh
5. Thừa nhận các hạn chế
6. Xác định chỉ số thành công

Ví dụ bảo vệ:

Người chất vấn: "Tại sao bạn chọn PostgreSQL thay vì MongoDB?"

Bảo vệ yếu:

"PostgreSQL tốt hơn."

Bảo vệ mạnh:

"Đề'tôi giải thích lập luận:

Bài toán:

Chúng ta cần lưu dữ liệu người dùng, đơn hàng và các mối quan hệ.
Giao dịch ACID là tối quan trọng (luồng thanh toán).

Các phương án đã cân nhắc:

Phương án 1: PostgreSQL

Giao dịch ACID

Khả năng truy vấn phong phú (JOIN)

Nhóm đã quen thuộc

Đã được chứng minh ở quy mô lớn (Instagram dùng nó)

Giới hạn của mở rộng theo chiều dọc

Thay đổi lược đồ'tôn kém

Mức phù hợp: Cao (khớp với yêu cầu)

Phương án 2: MongoDB

Mở rộng theo chiều ngang dễ'hơn

Lược đồ'linh hoạt

Tốt cho làm mẫu nhanh

Không có ACID giữa các document

Các thao tác kiểu JOIN'tôn kém

Nhóm cần được đào tạo

Mức phù hợp: Trung bình (có vài lợi ích, nhưng có nhược điểm then chốt)

Phương án 3: Cassandra

Quy mô cực lớn (hàng tỷ dòng)

Tính sẵn sàng cao

Nhật quán cuối cùng (không chấp nhận được)

Mô hình vận hành phức tạp

Quá mức cần thiết với quy mô của ta (10 nghìn người dùng)

Mức phù hợp: Thấp (thiết kế'thừa)

Quyết định: PostgreSQL

Lý do:

- Giao dịch ACID là không thể'thương lượng (thanh toán)
- Cần truy vấn phong phú (báo cáo, phân tích)
- Năng suất của nhóm (không phải học thêm)
- Quy mô của ta (10 nghìn → 100 nghìn người dùng) nằm gọn trong năng lực của PostgreSQL

Các đánh đổi được chấp nhận:

- Giới hạn mở rộng theo chiều dọc (chịu được 100 nghìn-1 triệu người dùng, đủ cho hiện tại)
- Chuyển đổi lược đồ'(chấp nhận được nếu có quy trình tốt)

Chỉ số'thành công:

- Độ trễ truy vấn < 100ms (hiện tại 50ms)
- Không có giao dịch nào thất bại
- Duy trì được tốc độ phát triển

Nếu PostgreSQL trở thành điểm nghẽn (> 1 triệu người dùng):

- Thêm bản sao chỉ đọc trước
- Sau đó phân mảnh nếu cần
- Đã có lộ trình chuyển đổi

Có câu hỏi nào không?"

- Phân tích kỹ lưỡng
- Quyết định dựa trên bối cảnh
- Thừa nhận các đánh đổi
- Xác định tiêu chí thành công
- Lên kế hoạch lộ trình phát triển

Những Chất Vấn Thường Gặp & Cách Trả Lời

Chất vấn 1: "Sao không dùng [công nghệ đang thịnh hành]?"

Mẫu trả lời:

"[Công nghệ X] rất tốt cho [tình huống sử dụng cụ thể].
Yêu cầu của chúng ta là [Y], thứ mà [X] không tối ưu cho.
Cụ thể, [đánh đổi Z] không khớp với ưu tiên của chúng ta.
Chúng ta chọn [giải pháp của ta] vì [lập luận]."

Ví dụ:

"Kafka rất tốt cho truyền luồng sự kiện thông lượng cao.
Yêu cầu của chúng ta là hàng đợi tác vụ đơn giản (100 tác vụ/phút).
Độ phức tạp vận hành của Kafka không xứng với lợi ích ở quy mô của ta.
Chúng ta chọn RabbitMQ vì đơn giản và cài đặt nhanh."

Chất vấn 2: "Cái này không mở rộng được đâu!"

Mẫu trả lời:

"Hãy định lượng 'quy mô'.

Hiện tại: [X người dùng/yêu cầu]

Tăng trưởng: [Y% mỗi tháng]

Năng lực: [Giải pháp chịu được Z]

Thời gian: [N tháng trước khi chạm điểm nghẽn]

Kế hoạch: [Chiến lược giảm thiểu]"

Ví dụ:

"Hiện tại: 10 nghìn người dùng, 1 nghìn yêu cầu/phút

Tăng trưởng: 20% mỗi tháng

Một PostgreSQL chịu được: 10 nghìn thao tác ghi/giây

Thời gian: ~24 tháng trước khi chạm điểm nghẽn (ở mức 150 nghìn người dùng)

Kế hoạch: Thêm bản sao chỉ đọc ở mức 50 nghìn người dùng, phân mảnh ở mức 200 nghìn+ nếu cần

Điều này cho ta 2 năm để kiểm chứng sự phù hợp giữa sản phẩm và thị trường."

Chất vấn 3: "Quá phức tạp / Quá đơn giản"

Trả lời:

"Độ phức tạp nên tương xứng với bài toán.

Các ràng buộc của chúng ta:

- [Quy mô/Người dùng/Lưu lượng]
- [Quy mô nhóm/chuyên môn]
- [Thời hạn/Ngân sách]

Với những điều này, [giải pháp] có kích cỡ phù hợp.

Nếu ràng buộc thay đổi, chúng ta sẽ đánh giá lại."

Ví dụ:

"Các ràng buộc của chúng ta:

- 5 nghìn người dùng, 100 req/s
- 2 lập trình viên
- Hạn chót 1 tháng
- Ngân sách \$500/tháng

Vi dịch vụ sẽ là thiết kế thừa.

Một hệ thống nguyên khối có cấu trúc tốt sẽ ra mắt nhanh và chịu được quy mô.

Ta có thể tách ra sau nếu chứng minh được là cần thiết."

Bài Tập Thực Hành: Phân Tích Đánh Đổi

Hãy thử phân tích kịch bản (*scenario*) này:

Bài toán: Thiết kế chiến lược lưu đệm (*caching strategy*) cho trang sản phẩm thương mại điện tử

Bối cảnh:

- 100 nghìn sản phẩm
- 1 triệu lượt xem trang/ngày
- Giá sản phẩm được cập nhật hằng giờ
- Các đợt flash sale (*flash sales*) đổi giá mỗi phút
- 70% lưu lượng (*traffic*) tập trung vào 1 nghìn sản phẩm hàng đầu (sản phẩm "nóng")

Các phương án:

Phương án A: Lưu đệm mạnh tay (TTL dài)

```
cache_ttl = 3600 # 1 hour
```

Benefits:

- Very fast (< 10ms)
- Low database load
- Handle traffic spikes

Costs:

- Stale prices (up to 1 hour old)
- Flash sale prices delayed
- User complaints possible

Phương án B: TTL ngắn

```
cache_ttl = 60 # 1 minute
```

Benefits:

- Reasonably fresh data
- Flash sales mostly accurate
- Balanced approach

Costs:

- More cache misses
- Higher DB load
- Slightly slower

Phương án C: Vô hiệu hoá thông minh (*Smart invalidation*)

```
# Cache-aside pattern with event-driven invalidation
```

```
on_price_update(product_id):  
    cache.delete(f"product:{product_id}")
```

Benefits:

- Always fresh
- Still fast (cached until update)
- Best user experience

Costs:

- Complex implementation
- Invalidation bugs possible
- Need event system

Nhiệm vụ của bạn:

1. Bạn sẽ chọn phương án nào?
2. Tại sao?
3. Bạn đang chấp nhận những sự đánh đổi nào?
4. Bạn sẽ bảo vệ lựa chọn của mình như thế nào?

Phân tích của tôi:

Khuyến nghị: Phương án C (Vô hiệu hoá thông minh)

Lý do:

- Độ chính xác của giá là tối quan trọng (niềm tin trong thương mại điện tử)
- Flash sale là động lực doanh thu (không thể 'dễ' dữ liệu cũ)
- Độ phức tạp được bù đắp bằng giá trị kinh doanh
- 70% là sản phẩm "nóng" = tỷ lệ trúng bộ nhớ đệm vẫn cao

Các đánh đổi được chấp nhận:

- Thời gian triển khai (+1 tuần)
- Độ phức tạp vận hành (giám sát việc vô hiệu hoá)
- Phụ thuộc vào hệ thống sự kiện

Nhưng:

- Tác động doanh thu của giá chính xác > chi phí phát triển
- Hệ thống sự kiện còn hữu ích cho các tính năng khác
- Đàng nào cũng cần giám sát

Phương án thay thế cho MVP: Phương án B

- Nếu thời gian gấp, bắt đầu với TTL 1 phút
- Chuyển sang Phương án C khi đã chứng minh được giá trị
- Cách tiếp cận theo từng giai đoạn giúp giảm rủi ro

Những Điểm Cốt Lõi

Tư duy đánh đổi = Kỹ năng cốt lõi của kỹ sư lâu năm

Kỹ sư mới: "Công nghệ nào là tốt nhất?"

Kỹ sư lâu năm: "Những đánh đổi nào phù hợp với bối cảnh của ta?"

Kinh nghiệm = Thư viện các mẫu

Thâm niên = Lập luận về đánh đổi

Những đánh đổi lớn:

Nhất quán và Sẵn sàng

→ Dữ liệu tài chính: Nhất quán

→ Dữ liệu mạng xã hội: Sẵn sàng

Độ trễ và Chi phí

→ Tính ROI

→ Tối ưu khi có lý do xác đáng

Đơn giản và Khả năng mở rộng

→ Bắt đầu đơn giản

→ Mở rộng khi cần (có dữ liệu)

Tối ưu sớm và Lặp dân

→ Xây cho gấp 2 lần, không phải 100 lần

→ Tối ưu hoá quá sớm = lãng phí

Khung bảo vệ:

1. Nêu rõ bài toán
2. Liệt kê các phương án
3. Giải thích các đánh đổi
4. Lý giải lựa chọn dựa trên bối cảnh
5. Thừa nhận các hạn chế
6. Xác định chỉ số thành công
7. Lên kế hoạch lộ trình phát triển

Trực giác về định lý CAP:

Phân vùng mạng luôn xảy ra (P là mặc định)

Chọn: C hoặc A

C (Nhất quán): Ngân hàng, thanh toán, đặt chỗ~

A (Sẵn sàng): Mạng xã hội, phân tích, bảng tin

Không có câu trả lời hoàn hảo, chỉ có sự phù hợp với bối cảnh

Mô hình tư duy (*Mental model*):

Mọi quyết định kiến trúc:

- Luôn có đánh đổi (luôn luôn!)
- Phụ thuộc vào bối cảnh (không có "tốt nhất" cho mọi trường hợp)
- Cần được lý giải (có lập luận)
- Có thể tiến hoá (không cố định mãi mãi)
- Cần đo lường (không dựa vào cảm tính)

Bài kiểm tra tối thượng:

Một kiến trúc sư giỏi có thể trả lời:

"Tại sao bạn chọn X thay vì Y?"

Bằng:

- ✓ Hiểu rõ bài toán
- ✓ Các phương án thay thế
- ✓ Phân tích đánh đổi
- ✓ Sự khớp với bối cảnh
- ✓ Chỉ số thành công
- ✓ Kế hoạch phát triển

Chứ không phải bằng:

- x "Đó là thực hành tốt nhất"
- x "Ai cũng dùng nó"
- x "Nó mới hơn/nhanh hơn/ngầu hơn"

Hãy nhớ:

Kiến trúc hoàn hảo = Hoang đường

Kiến trúc tốt = Đánh đổi đúng cho bối cảnh của bạn

Làm chủ tư duy đánh đổi

Làm chủ việc ra quyết định kiến trúc

Làm chủ vai trò kỹ sư lâu năm

Bạn đã học các mẫu thiết kế, khung tư duy (*frameworks*), hệ thống. Giờ là lúc học cách **lập luận về chúng**.

Tư duy đánh đổi là thứ phân biệt kỹ sư lâu năm với kỹ sư mới vào nghề. Hãy làm chủ nó.

Chiến Lược Phỏng Vấn Thực Chiến (*Real Interview Strategy*): Áp Dụng Khung Tư Duy (*Framework*) Vào Tình Huống Thực Tế

Sau khi học khung tư duy SNAKE và tư duy đánh đổi (*trade-off thinking*), nhiều người hỏi tôi:

"Khung tư duy thì hiểu rồi, nhưng áp dụng vào Twitter, YouTube... thế nào?"

Đó là câu hỏi đúng.

Tôi đã tham gia hơn 50 buổi phỏng vấn thiết kế hệ thống (*system design interviews*) (cả với vai trò người phỏng vấn (*interviewer*) lẫn ứng viên (*candidate*)). Tôi nhận ra một quy luật:

Ứng viên (*Candidates*) trượt không phải vì thiếu kiến thức. Mà vì không biết người phỏng vấn muốn gì.

Mỗi loại hệ thống có một "câu chuyện" mà người phỏng vấn chờ đợi. Nếu bạn hiểu câu chuyện đó, bạn sẽ sắp xếp câu trả lời một cách hoàn hảo.

Bài này sẽ dạy bạn: **Cách tiếp cận các bài toán thiết kế hệ thống (*system design*) phổ biến nhất.**

Không phải kiến trúc (*architecture*) chi tiết (bạn đã học ở Phần 5). Mà là **chiến lược phỏng vấn (*interview strategy*)**.

Tại Sao Cần Chiến Lược Riêng Cho Mỗi Hệ Thống?

Quy Luật Đằng Sau Các Câu Hỏi

Người phỏng vấn (*Interviewers*) không chọn hệ thống một cách ngẫu nhiên.

Mỗi hệ thống kiểm tra một nhóm kỹ năng cụ thể:

Twitter/Bảng tin mạng xã hội:

- Kiểm tra: Phân phát (fanout), nhât quán cuối cùng, khuếch đại ghi
- Câu chuyện: "Làm sao phân phát nội dung tới hàng triệu người?"

YouTube/Nền tảng video:

- Kiểm tra: CDN, quy trình mã hoá video, tối ưu lưu trữ
- Câu chuyện: "Làm sao phân phối video trên toàn cầu?"

WhatsApp/Nhắn tin:

- Kiểm tra: Giao tiếp thời gian thực, đảm bảo giao tin, mã hoá
- Câu chuyện: "Làm sao đảm bảo tin nhắn được giao tới nơi?"

Google Drive/Lưu trữ:

- Kiểm tra: Đồng bộ tệp, giải quyết xung đột, chia khối (chunking)
- Câu chuyện: "Làm sao đồng bộ tệp giữa các thiết bị?"

Nếu bạn hiểu "câu chuyện", bạn sẽ biết nên tập trung vào đâu.

Sai Lầm Phổ Biến: Đối Xử Với Mọi Hệ Thống Như Nhau

Cách tiếp cận tồi:

"Hệ thống nào cũng cần bộ cân bằng tải, bộ nhớ đệm, cơ sở dữ liệu, hàng đợi thông điệp..."

- Câu trả lời chung chung
- Bỏ lỡ những thách thức đặc thù
- Không có chiều sâu

Cách tiếp cận tốt:

"Thách thức của Twitter là phân phát tới hàng triệu người theo dõi.

Thách thức của YouTube là mã hoá video và CDN.

Thách thức của WhatsApp là đảm bảo giao tin thời gian thực.

Mỗi hệ thống cần một trọng tâm khác nhau."

- Bám sát bài toán
- Thể hiện sự thấu hiểu
- Chứng minh chiều sâu

Chiến Lược 1: Thiết Kế Twitter (Hệ Thống Bảng Tin (Feed) Mạng Xã Hội)

Điều Người Phỏng Vấn Muốn Nghe

Thách thức cốt lõi: Phân phát bài đăng tới hàng triệu người theo dõi (followers) một cách hiệu quả

Các chủ đề chính cần đề cập:

1. Chiến lược phân phát (*fanout*) (đây là phần QUAN TRỌNG NHẤT)
2. Tạo dòng thời gian (*timeline*)
3. Tối ưu cho đọc hay cho ghi
4. Vấn đề người nổi tiếng (*Celebrity problem*)
5. Các đánh đổi (*trade-offs*) của nhất quán cuối cùng (*eventual consistency*)

Áp Dụng SNAKE

S - Scope - Phạm vi (5 phút)

"Cho tôi làm rõ các yêu cầu:

Tính năng cốt lõi:

- ✓ Đăng tweet (280 ký tự)
- ✓ Theo dõi người dùng
- ✓ Xem dòng thời gian trang chủ
- ✓ Thích, retweet

Ngoài phạm vi:

- x Tin nhắn trực tiếp
- x Chủ đề`thịnh hành
- x Tìm kiếm

Quy mô:

- 500 triệu người dùng
- 100 triệu người dùng hoạt động hằng ngày
- 200 triệu tweet/ngày
- Thiên về`đọc (95% là đọc)

Như vậy đúng chưa?"

Điểm cần làm rõ then chốt:

"Với việc tạo dòng thời gian, tôi nên tối ưu cho:

- a) Ghi nhanh (đăng tweet trả về`ngay lập tức)
- b) Đọc nhanh (dòng thời gian tải < 100ms)

Tôi giả định là (b) - tối ưu cho tốc độ đọc?

- Điều này cho thấy bạn hiểu sự đánh đổi cốt lõi
- Người phỏng vấn sẽ xác nhận/điều chỉnh

N - Numbers - Con số (5 phút)

"Cho tôi tính toán quy mô:

Lưu lượng ghi:

200 triệu tweet/ngày / 86400 giây $\approx$ 2.300 tweet/giây

Lưu lượng đọc:

100 triệu người dùng $\times$ 10 lần tải dòng thời gian/ngày = 1 tỷ lần tải/ngày
 $\approx$ 12.000 yêu cầu/giây

Tỷ lệ Đọc:Ghi = 12K:2,3K $\approx$ 5:1

→ Thiên về đọc, lưu đệm là yếu tố sống còn

Lưu trữ (5 năm):

200 triệu tweet/ngày $\times$ 365 $\times$ 5 = 365 tỷ tweet
 $\times$ 1KB mỗi tweet = 365TB

Tính toán phân phát (fanout):

Số người theo dõi trung bình: 200

Khuếch đại ghi: 2.300 tweet/giây $\times$ 200 = 460K lượt cập nhật dòng thời gian/giây

→ Đây chính là điểm nghẽn cần giải quyết!"

→ Cho thấy bạn đã nhận diện được THÁCH THỨC then chốt

A - API (5 phút)

Core APIs:

POST /v1/tweets

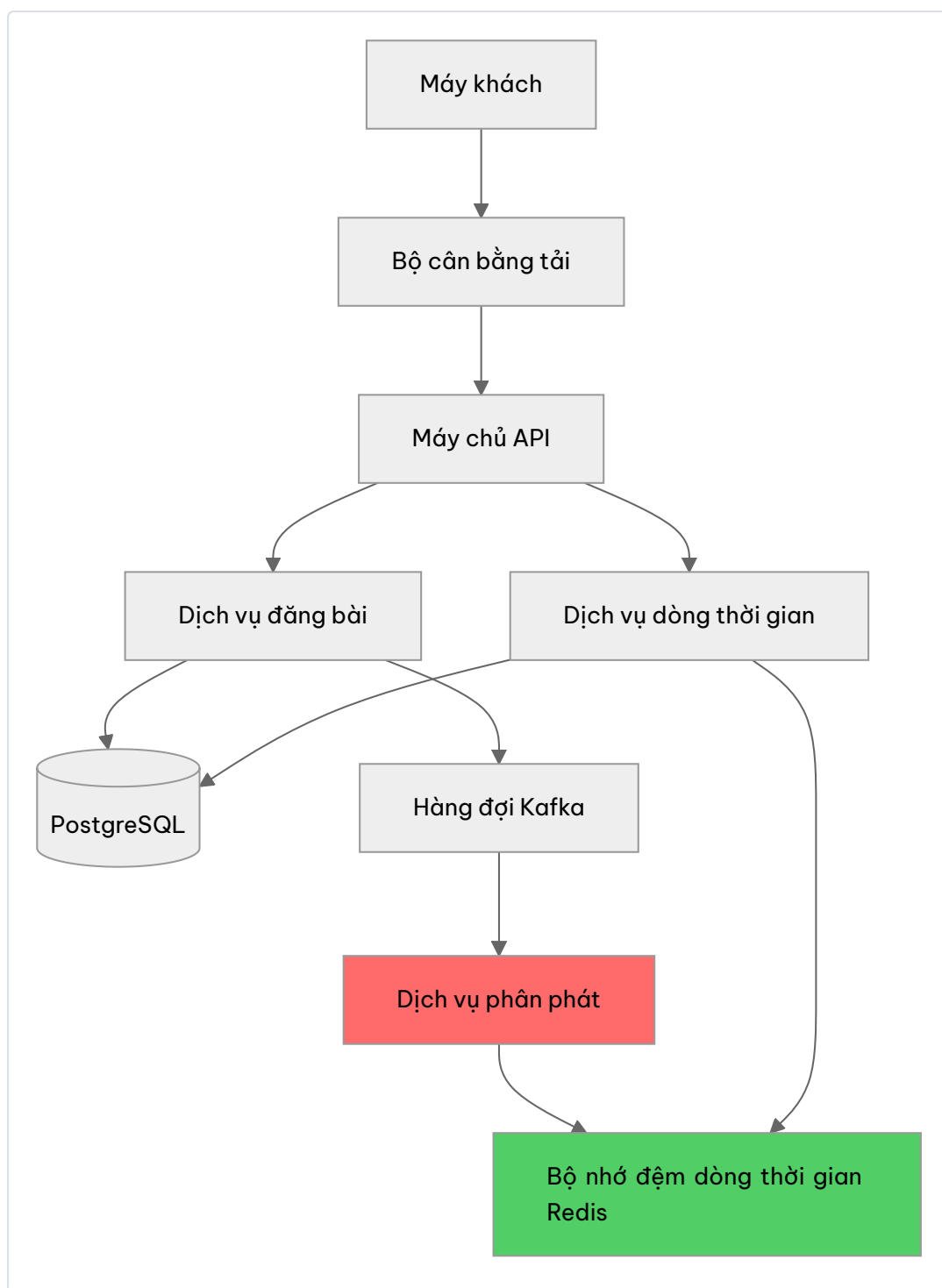
```
{  
  "content": "Hello world!",  
  "media_urls": []  
}
```

GET /v1/timeline?cursor={cursor}&limit=20

POST /v1/tweets/{id}/like

POST /v1/users/{id}/follow

K - Key Components - Thành phần chính (15 phút)



Kiến trúc Twitter: Tập trung vào Dịch vụ phân phát (Fanout Service) và Bộ nhớ đệm dòng thời gian (Timeline Cache)

Phần thảo luận then chốt (dành phần lớn thời gian ở đây):

"Thách thức cốt lõi là phân phát (fanout). Cho tôi giải thích cách tiếp cận của mình:

Chiến lược phân phát lai (hybrid):

Người dùng bình thường (< 10K người theo dõi):

- Phân phát khi ghi (fanout on write)
- Khi người dùng đăng bài → Đẩy tới dòng thời gian của mọi người theo dõi
- Bảng tin tính sẵn = đọc nhanh

Người nổi tiếng (> 10K người theo dõi):

- Phân phát khi đọc (fanout on read)
- Không đẩy tới hàng triệu dòng thời gian
- Kéo về khi người theo dõi tải bảng tin

Đánh đổi:

Đọc nhanh cho hầu hết người dùng (< 50ms từ bộ nhớ đệm)

Tránh bùng nổ phân phát với người nổi tiếng

Tweet của người nổi tiếng hơi trễ một chút (chấp nhận được)

Cách tiếp cận kết hợp = độ phức tạp cao hơn

Cấu trúc bộ nhớ đệm dòng thời gian:

- Một Redis sorted set cho mỗi người dùng
- Khoá: timeline:{user_id}
- Điểm (score): dấu thời gian
- Phần tử: ID của tweet
- Giữ 1000 tweet gần nhất

Vì sao cách này hiệu quả:

- 90% lượt tải dòng thời gian trúng bộ nhớ đệm
- Tính sẵn để trải nghiệm người dùng nhanh
- Tweet của người nổi tiếng được gộp vào khi cần"

→ Đây chính là điều người phỏng vấn muốn nghe

→ Thể hiện sự thấu hiểu sâu sắc

→ Giải quyết đúng thách thức cốt lõi

E - Elaborate - Đào sâu (15 phút)

Chọn 2-3 phần để đào sâu (deep dive):

1. Chi tiết triển khai phân phát

"Cho tôi trình bày logic của tiến trình phân phát (fanout worker):

Khi tweet được đăng:

- Tác vụ bất đồng bộ qua Kafka
- Worker lấy danh sách người theo dõi
- Chèn theo lô vào dòng thời gian Redis (100 người theo dõi/lô)
- Nếu số người theo dõi > 10K → Bỏ qua phân phát, đánh dấu là người nổi tiếng

Cách này xử lý được 460K lượt cập nhật/giây với:

- 100 worker phân phát
- Mỗi worker xử lý 4.600 lượt cập nhật/giây
- Xử lý theo lô giúp giảm số thao tác Redis"

2. Giải pháp cho vấn đề người nổi tiếng

"Với Taylor Swift (100 triệu người theo dõi):

- Không phân phát khi ghi
- Khi người theo dõi tải dòng thời gian:
 - Lấy bảng tin tính sẵn (từ người dùng bình thường)
 - Gộp tweet của người nổi tiếng (truy vấn khi cần)
 - Sắp xếp theo dấu thời gian
 - Áp dụng xếp hạng"

3. Xử lý sự cố

"Nếu hàng đợi Kafka bị dồn ứ:

- Bộ ngắt mạch dừng nhận tác vụ phân phát mới
- Ưu tiên người dùng VIP
- Chuyển sang cơ chế kéo (pull) cho tất cả khi tải cao"

Điều Gì Làm Nên Một Câu Trả Lời Tốt

- ✓ Nhận diện thách thức cốt lõi (phân phát)
- ✓ Đề xuất giải pháp cụ thể (lại)
- ✓ Giải thích rõ ràng các đánh đổi
- ✓ Thể hiện tư duy về khả năng mở rộng
- ✓ Chứng minh chiều sâu ở những phần then chốt

Chiến Lược 2: Thiết Kế YouTube (Nền Tảng Video)

Điều Người Phỏng Vấn Muốn Nghe

Thách thức cốt lõi: Mã hoá (*Encode*), lưu trữ và phân phối video trên toàn cầu ở quy mô (*scale*) cực lớn

Các chủ đề chính cần đề cập:

1. Quy trình tải video lên (*Video upload pipeline*)

2. Chiến lược mã hoá (*encoding*) (nhiều độ phân giải)
3. CDN để phân phối toàn cầu
4. Tối ưu lưu trữ (*storage*)
5. Hệ thống gợi ý (*Recommendation system*) (ở mức tổng quan)

Trọng Tâm Khi Áp Dụng SNAKE

S - Scope - Phạm vi

"Cho tôi làm rõ:

Tính năng cốt lõi:

- ✓ Tải video lên
- ✓ Xem video
- ✓ Tìm kiếm video
- ✓ Gợi ý (chỉ ở mức tổng quan)

Ngoài phạm vi:

- x Phát trực tiếp
- x Hệ thống bình luận
- x Kiểm tiền

Giả định về quy mô:

- 500 triệu người dùng
- 1 tỷ lượt xem video/ngày
- 100 triệu giờ xem/ngày
- 500K video tải lên/ngày

Như vậy đúng chưa?"

N - Numbers - Con số (Then chốt với YouTube)

"Cho tôi tính dung lượng lưu trữ và băng thông:

Lưu trữ video tải lên:

500K lượt tải lên/ngày

Trung bình: video 10 phút ở 1080p = 1GB

Mỗi ngày: $500K \times 1GB = 500TB/ngày$

Mỗi năm: $500TB \times 365 = 180PB/năm$

→ Cần lưu trữ đối tượng phân tán (S3)

Nhưng ta mã hoá ra nhiều độ phân giải:

- 360p (100MB)

- 720p (300MB)

- 1080p (1GB)

- 4K (4GB) - tùy chọn

Tổng mỗi video: ~5GB

→ $500K \times 5GB = 2,5PB$ lưu trữ/ngày!

Băng thông (phân phối):

1 tỷ lượt xem/ngày

Thời gian xem trung bình: 6 phút

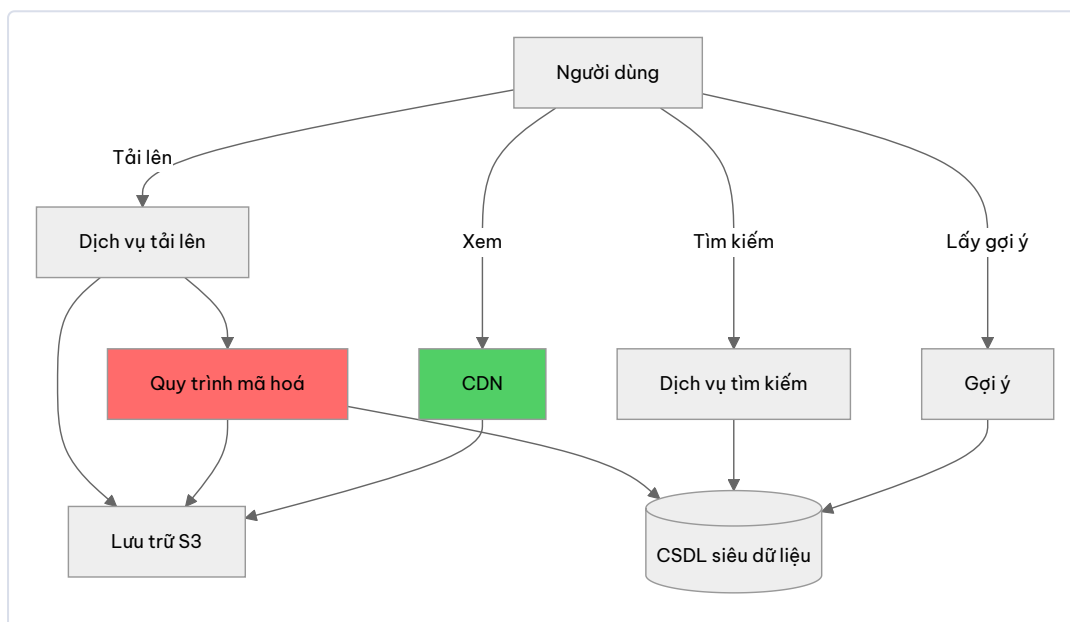
Bitrate trung bình: 5 Mbps

Tổng: $1 \text{ tỷ} \times 6 \text{ phút} \times 5 \text{ Mbps} = 3 \text{ exabyte/tháng}$

→ CDN là tuyệt đối thiết yếu!"

→ Các con số cho thấy bạn hiểu về quy mô

K - Key Components - Thành phần chính (Các phần trọng tâm)



YouTube: Tập trung vào Quy trình mã hoá (Encoding Pipeline) và phân phối qua CDN

Các phần thảo luận then chốt:

1. Quy trình tải lên và mã hoá:

"Luồng tải lên:

1. Người dùng tải video gốc lên → Dịch vụ tải lên
2. Lưu bản gốc vào S3 (lưu trữ lạnh)
3. Kích hoạt quy trình mã hoá (bất đồng bộ)
4. Mã hoá song song ra nhiều độ phân giải:
 - 360p, 720p, 1080p, 4K
 - Nhiều codec khác nhau (H.264, VP9, AV1)
5. Tạo ảnh thu nhỏ
6. Cập nhật cơ sở dữ liệu siêu dữ liệu
7. Thông báo cho người dùng

Mã hoá tốn thời gian:

- Video 10 phút → mất 30 phút để mã hoá mọi định dạng
- Người dùng không phải chờ (xử lý bất đồng bộ)
- Hiện thị trạng thái 'Đang xử lý'

Vì sao cần nhiều độ phân giải:

- Truyền phát với bitrate thích ứng
- Người dùng có kết nối chậm → 360p
- Người dùng có kết nối nhanh → 4K
- Chuyển đổi chất lượng liền mạch"

2. Chiến lược CDN:

"CDN là yếu tố then chốt:

- 3 exabyte/tháng = hơn \$300K nếu không có CDN
- Có CDN: \$50K (tiết kiệm 5 lần!)

Cách hoạt động:

- Video lưu trên S3 (máy chủ gốc)
- CDN lưu đệm các video phổ biến tại điểm biên
- Yêu cầu của người dùng → Điểm biên CDN gần nhất
- Trượt tại điểm biên → Lấy từ máy chủ gốc

Chiến lược lưu đệm:

- 10% video hàng đầu (thịnh hành) → Lưu đệm triệt để
- Phần đuôi dài → Lưu đệm khi có yêu cầu
- Loại bỏ khỏi bộ nhớ đệm: LRU

Vì sao điều này quan trọng:

- Độ trễ thấp (< 100ms để bắt đầu phát)
- Chịu được video lan truyền (hàng triệu người xem đồng thời)
- Tối ưu chi phí"

Điều Gì Làm Nên Một Câu Trả Lời Tốt

- ✓ Con số ở quy mô không lồ (tới exabyte!)
- ✓ Trình bày chi tiết quy trình mã hoá
- ✓ Chiến lược CDN kèm phân tích chi phí
- ✓ Giải thích truyền phát thích ứng
- ✓ Mẫu xử lý bất đồng bộ

Chiến Lược 3: Thiết Kế WhatsApp (Hệ Thống Nhắn Tin)

Điều Người Phỏng Vấn Muốn Nghe

Thách thức cốt lõi: Giao tin nhắn một cách tin cậy theo thời gian thực với tính sẵn sàng cao (*high availability*)

Các chủ đề chính cần đề cập:

1. Giao tin thời gian thực (WebSocket)
2. Đảm bảo giao tin (*Message delivery guarantees*)
3. Xử lý tin nhắn khi người dùng ngoại tuyến
4. Nhắn tin nhóm
5. Mã hoá đầu cuối (*End-to-end encryption*) (ở mức khái niệm)

Trọng Tâm Khi Áp Dụng SNAKE

S - Scope - Phạm vi

"Cho tôi làm rõ:

Tính năng cốt lõi:

- ✓ Nhấn tin 1-1
- ✓ Nhấn tin nhóm (< 256 thành viên)
- ✓ Xác nhận trạng thái (đã gửi, đã nhận, đã đọc)
- ✓ Lưu trữ tin nhắn khi ngoại tuyến

Ngoài phạm vi:

- x Gọi thoại/video
- x Cập nhật trạng thái
- x Tính năng thanh toán

Quy mô:

- 2 tỷ người dùng
- 100 tỷ tin nhắn/ngày
- Cần giao tin thời gian thực
- Đảm bảo giao tin 99,99%

Như vậy đúng chưa?"

N - Numbers - Con số

"Lưu lượng tin nhắn:

100 tỷ tin nhắn/ngày / 86400 giây $\approx$ 1,1 triệu tin nhắn/giây
Cao điểm (gấp 3): 3,3 triệu tin nhắn/giây

Lưu trữ:

Tin nhắn trung bình: 100 byte

100 tỷ tin nhắn/ngày $\times$ 365 $\times$ 2 năm = 73 nghìn tỷ tin nhắn
 $\times$ 100 byte = 7,3 PB

→ Cần lưu trữ phân tán

Tải kết nối:

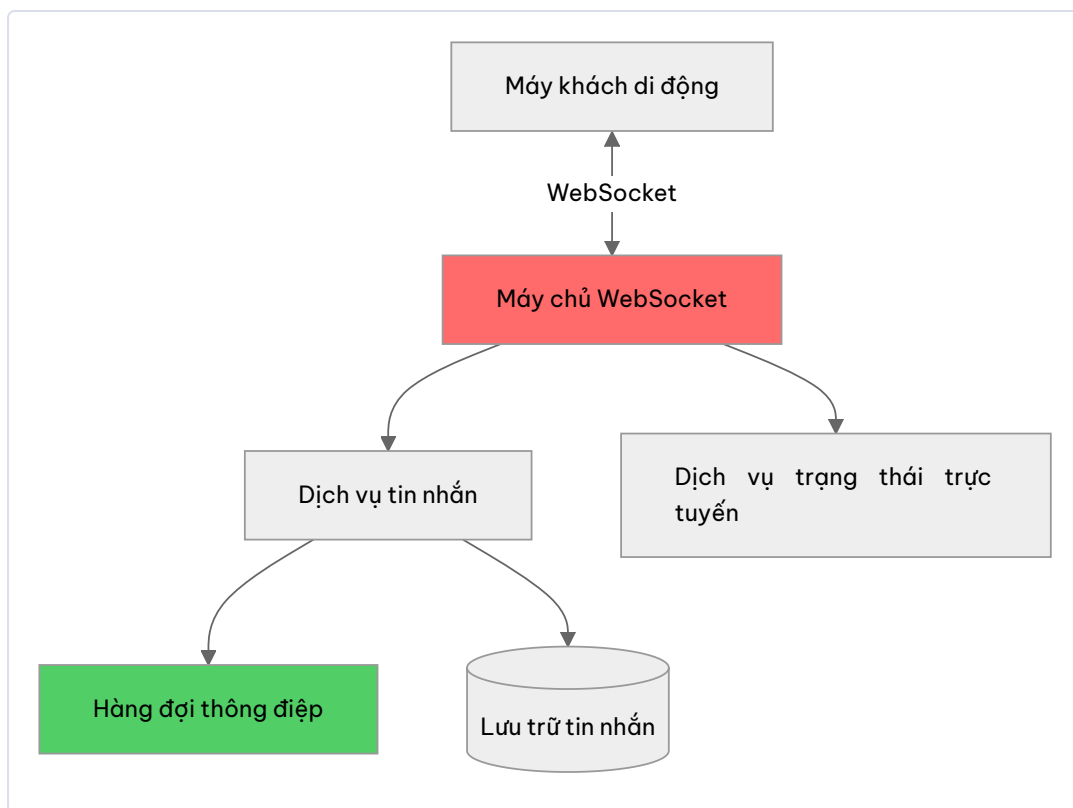
100 triệu người dùng trực tuyến đồng thời

Mỗi người duy trì một kết nối WebSocket

= 100 triệu kết nối lâu dài

→ Cần gom nhóm kết nối (connection pooling), nhiều máy chủ"

K - Key Components - Thành phần chính



WhatsApp: Tập trung vào tầng WebSocket và Hàng đợi thông điệp (Message Queue)

Các phần thảo luận then chốt:

1. Giao tin thời gian thực:

"Mỗi người dùng một kết nối WebSocket:

- Duy trì kết nối lâu dài
- Độ trễ thấp (< 50ms)
- Giao tiếp hai chiều

Luồng tin nhắn:

1. Người gửi → Máy chủ WebSocket A
2. Máy chủ A → Dịch vụ tin nhắn
3. Dịch vụ tin nhắn → Kiểm tra người nhận có trực tuyến không
4. Nếu trực tuyến:
 - Chuyển tiếp tới máy chủ WebSocket của người nhận
 - Giao ngay lập tức
5. Nếu ngoại tuyến:
 - Lưu vào hàng đợi thông điệp
 - Giao khi người nhận trực tuyến trở lại

Vì sao cách này hiệu quả:

- Thời gian thực với người dùng đang trực tuyến
- Đảm bảo giao tin với người dùng ngoại tuyến
- Mở rộng theo chiều ngang (nhiều máy chủ WS)"

2. Đảm bảo giao tin:

"Giao ít nhất một lần (at-least-once):

- Tin nhắn được lưu trong hàng đợi cho tới khi được xác nhận
- Máy khách gửi ACK khi đã nhận
- Thử lại nếu không có ACK trong 30 giây
- Hàng đợi bền vững (Kafka)

Ba dấu tích:

- ✓ Đã gửi: Tin nhắn đã rời thiết bị người gửi
- ✓✓ Đã nhận: Tin nhắn đã tới thiết bị người nhận
- ✓✓✓ Đã đọc: Người nhận đã mở cuộc trò chuyện

Triển khai:

- Mỗi lần cập nhật trạng thái là một tin nhắn riêng
- Được giao qua cùng kênh WebSocket
- Lưu vào cơ sở dữ liệu để làm lịch sử"

3. Xử lý ngoại tuyến:

"Khi người dùng ngoại tuyến:

- Tin nhắn được xếp hàng trong bộ lưu trữ bền vững
- Lưu giữ trong 30 ngày
- Khi người dùng trực tuyến trở lại:
 - Lấy các tin nhắn đang chờ
 - Giao theo đúng thứ tự
 - Đánh dấu là đã nhận
 - Xoá khỏi hàng đợi

Vì sao dựa trên hàng đợi:

- Đảm bảo giao tin

- Xử lý được gián đoạn mạng
- Hỗ trợ nhiều thiết bị"

E - Elaborate - Đào sâu

Nhắn tin nhóm:

"Thách thức: Giao tin tới 256 thành viên một cách hiệu quả

Cách tiếp cận ngây thơ:

- Gửi 256 tin nhắn riêng lẻ
- Vấn đề phân phát (fanout) tương tự Twitter

Cách tiếp cận tốt hơn:

- Một tin nhắn duy nhất được lưu một lần
- 256 con trỏ tới cùng tin nhắn đó
- Giao tới thành viên đang trực tuyến (WebSocket)
- Xếp hàng chờ cho thành viên ngoại tuyến

Đánh đổi:

Tiết kiệm lưu trữ

Tin nhắn nhất quán

Logic đọc phức tạp (cần lần theo con trỏ)"

Mã hoá:

"Mã hoá đầu cuối (mức tổng quan):

- Tin nhắn được mã hoá trên thiết bị người gửi
- Máy chủ không thể giải mã (zero-knowledge)
- Chỉ người nhận mới giải mã được

Hệ quả:

- Tìm kiếm phải thực hiện phía máy khách
- Bản sao lưu được mã hoá
- Máy chủ chỉ định tuyến các byte đã mã hoá"

Điều Gì Làm Nên Một Câu Trả Lời Tốt

- ✓ Tập trung vào thời gian thực (WebSocket)
- ✓ Trình bày chi tiết đảm bảo giao tin
- ✓ Có chiến lược xử lý ngoại tuyến
- ✓ Có số liệu về khả năng mở rộng
- ✓ Tối ưu nhắn tin nhóm

Chiến Lược 4: Thiết Kế Google Drive (Lưu Trữ Đám Mây)

Điều Người Phỏng Vấn Muốn Nghe

Thách thức cốt lõi: Đồng bộ tệp giữa các thiết bị một cách tin cậy, kèm giải quyết xung đột (*conflict resolution*)

Các chủ đề chính cần đề cập:

1. Thuật toán đồng bộ (*sync*) tệp
2. Chiến lược chia khối (*chunking*)
3. Giải quyết xung đột
4. Đồng bộ phần chênh lệch (*Delta sync*) (chỉ những phần đã thay đổi)
5. Cộng tác (chỉnh sửa thời gian thực)

Trọng Tâm Khi Áp Dụng SNAKE

S - Scope - Phạm vi

"Cho tôi làm rõ:

Tính năng cốt lõi:

- ✓ Tải lên/tải xuống tệp
- ✓ Đồng bộ giữa các thiết bị
- ✓ Quản lý phiên bản tệp
- ✓ Chia sẻ (quyền đọc/ghi)

Ngoài phạm vi:

- x Cộng tác chỉnh sửa thời gian thực (Google Docs)
- x Tích hợp ứng dụng bên thứ ba
- x Tìm kiếm nâng cao

Quy mô:

- 1 tỷ người dùng
- 10 tỷ tệp được lưu trữ
- Kích thước tệp trung bình: 5MB
- 1 triệu lượt đồng bộ đang diễn ra/phút

Như vậy đúng chưa?"

N - Numbers - Con số

"Lưu trữ:

$10 \text{ tỷ tệp} \times 5\text{MB} = 50 \text{ exabyte}$

→ Cần lưu trữ đối tượng phân tán

Lưu lượng đồng bộ:

1 triệu lượt đồng bộ/phút

Trung bình: 2 tệp thay đổi mỗi lượt đồng bộ

= 2 triệu thao tác tệp/phút

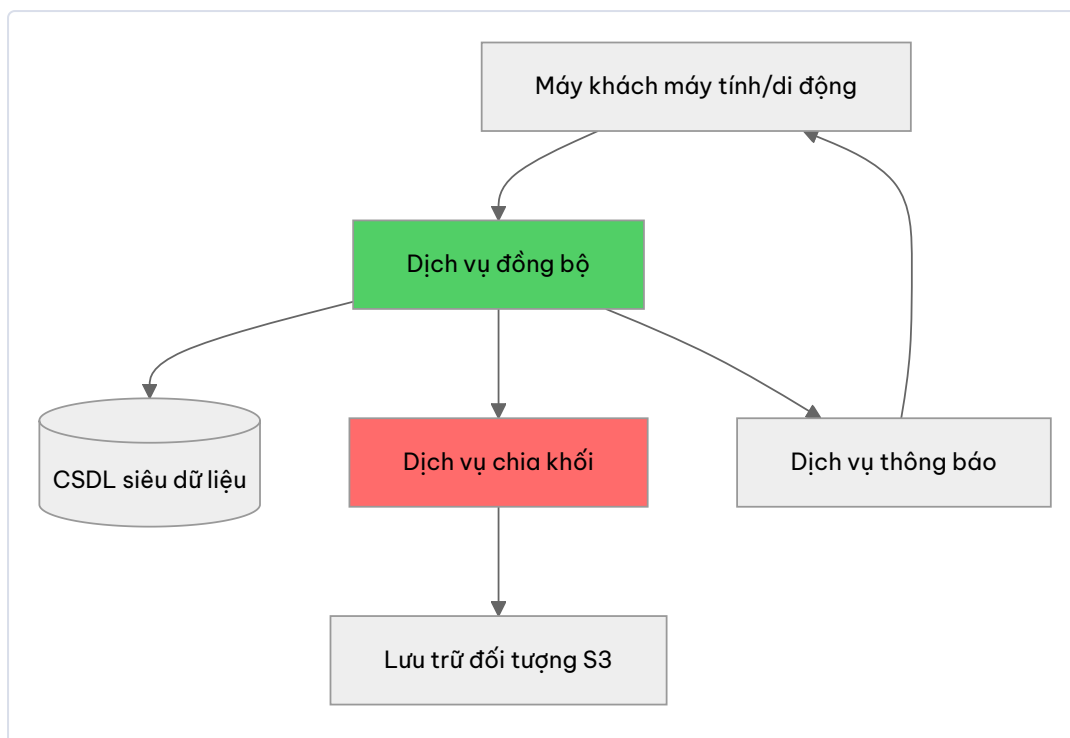
= 33K thao tác/giây

Băng thông:

$33\text{K thao tác} \times 5\text{MB} = 165 \text{ GB/giây}$

→ Cần chia khối để giảm băng thông!"

K - Key Components - Thành phần chính



Google Drive: Tập trung vào Dịch vụ đồng bộ (Sync Service) và chia khối

Các phần thảo luận then chốt:

1. Chiến lược chia khối:

"Vì sao cần chia khối:

- Một tệp 1GB đã được tải lên
- Người dùng sửa 1 dòng
- Không chia khối: Tải lại toàn bộ 1GB
- Có chia khối: Chỉ tải lên khối đã thay đổi

Triển khai:

- Chia tệp thành các khối 4MB
- Băm từng khối (SHA-256)
- Chỉ tải lên các khối có giá trị băm thay đổi

Ví dụ:

Tệp 1GB = 250 khối

Người dùng sửa → 1 khối thay đổi

Tải lên: 4MB thay vì 1GB

→ Tiết kiệm băng thông 250 lần!

Loại bỏ trùng lặp:

- Cùng giá trị băm = cùng nội dung
- Lưu một lần, tham chiếu nhiều lần
- Tiết kiệm lưu trữ (nhiều người dùng tải lên cùng một tệp)"

2. Thuật toán đồng bộ:

"Luồng đồng bộ:

1. Máy khách theo dõi thay đổi của tệp
2. Phát hiện thay đổi → Tính giá trị băm của các khối
3. So sánh với giá trị băm trên máy chủ
4. Chỉ tải lên các khối đã thay đổi
5. Máy chủ dựng lại tệp
6. Thông báo cho các máy khách khác

Phát hiện xung đột:

- Bản ghi cuối cùng thắng (last-write-wins)? (có thể mất dữ liệu)
- Tốt hơn: Lịch sử phiên bản
- Người dùng sửa tệp khi ngoại tuyến trên 2 thiết bị
- Cả hai cùng tải lên khi trực tuyến
- Máy chủ phát hiện xung đột
- Giữ lại cả hai phiên bản
- Đề nghị người dùng tự hợp nhất"

3. Cấu trúc siêu dữ liệu:

"Siêu dữ liệu của tệp:

```
{
  file_id: "abc123",
  name: "document.pdf",
  size: 5242880,
  chunks: [
    {chunk_id: "ch1", hash: "a1b2...", offset: 0},
    {chunk_id: "ch2", hash: "c3d4...", offset: 4194304}
  ],
}
```

```
version: 5,  
modified_by: "user_id",  
modified_at: "2024-03-15T10:30:00Z"  
}
```

Vì sao dùng cấu trúc này:

- Dựng lại tệp từ các khối
- Phát hiện thay đổi (so sánh giá trị băm)
- Lịch sử phiên bản
- Có thể loại bỏ trùng lặp"

E - Elaborate - Đào sâu

Thông báo thời gian thực:

"Khi tệp được đồng bộ:

- Máy chủ đẩy thông báo tới mọi máy khách đang kết nối
- Qua WebSocket hoặc long-polling
- Máy khách kiểm tra xem tệp cục bộ đã lỗi thời chưa
- Tự động tải xuống nếu cần

Vì sao cần thời gian thực:

- Người dùng sửa tệp trên laptop
- Người dùng thấy cập nhật ngay trên điện thoại
- Trải nghiệm liền mạch giữa các thiết bị

Hỗ trợ ngoại tuyến:

"Thách thức: Người dùng sửa khi ngoại tuyến, một người dùng khác sửa cùng tệp

Giải pháp:

- Mỗi lần sửa tạo ra một phiên bản mới
- Khi trực tuyến, tải lên dưới dạng phiên bản mới
- Máy chủ phát hiện có nhiều phiên bản
- Giao diện hiển thị: 'Bản sao xung đột (thiết bị của Người dùng 2)'
- Người dùng tự hợp nhất

Đánh đổi:

Không bao giờ mất dữ liệu

Người dùng nắm quyền kiểm soát

Phải hợp nhất thủ công (trường hợp hiếm)"

Điều Gì Làm Nên Một Câu Trả Lời Tốt

- ✓ Trình bày chi tiết chiến lược chia khối
- ✓ Giải thích đồng bộ phân chênh lệch (delta sync)
- ✓ Có giải quyết xung đột
- ✓ Thể hiện tối ưu bằng thông
- ✓ Thiết kế cấu trúc siêu dữ liệu

Tư Duy Phỏng Vấn: Điều Người Phỏng Vấn Thực Sự Muốn

Vượt Ra Ngoài Kiến Thức Kỹ Thuật

Người phỏng vấn đánh giá:

1. Hiểu bài toán (30%)
 - Bạn có làm rõ yêu cầu không?
 - Bạn có nhận diện được thách thức cốt lõi không?
2. Kỹ năng thiết kế hệ thống (40%)
 - Bạn có thiết kế được kiến trúc có khả năng mở rộng không?
 - Bạn có cân nhắc các đánh đổi không?
 - Bạn có ước lượng được năng lực hệ thống không?
3. Giao tiếp (20%)
 - Bạn có giải thích rõ ràng không?
 - Bạn có vừa nghĩ vừa nói thành lời không?
 - Bạn có tinh thần hợp tác không?
4. Chiều sâu kiến thức (10%)
 - Bạn có hiểu sâu các công nghệ không?
 - Bạn có thảo luận được các phương án thay thế không?

Những Dấu Hiệu Đỏ Khiến Bạn Trượt Phỏng Vấn

Nhảy vào giải pháp mà không làm rõ yêu cầu
Kiến trúc chung chung (không tùy biến theo bài toán)
Không tính toán quy mô
Không thảo luận về đánh đổi
Thiết kế trong im lặng (không nói thành lời suy nghĩ)
Phòng thủ khi bị chất vấn
Toàn thuật ngữ thời thượng, không có chiều sâu

Những Dấu Hiệu Xanh Giúp Bạn Vượt Qua Phỏng Vấn

Đặt câu hỏi làm rõ
Tính toán quy mô (thể hiện tư duy dựa trên dữ liệu)
Giải thích đánh đổi ("Tôi chọn X thay vì Y vì...")
Nói thành lời suy nghĩ của mình
Điều chỉnh dựa trên phản hồi
Thể hiện chiều sâu ở 2-3 phần
Thảo luận các kịch bản sự cố
Đề xuất chiến lược giám sát

Chiến Lược Luyện Tập

Cách Chuẩn Bị

Bước 1: Thành thạo khung tư duy SNAKE (1 tuần)

- Luyện tập 5 bài toán khác nhau với giới hạn thời gian nghiêm ngặt
- Làm quen với trình tự các bước

Bước 2: Nghiên cứu 4 nhóm hệ thống chính (2 tuần)

- Hệ thống mạng xã hội (Twitter, Facebook, Instagram)
- Hệ thống video (YouTube, Netflix, TikTok)
- Hệ thống nhắn tin (WhatsApp, Slack, Discord)
- Hệ thống lưu trữ (Google Drive, Dropbox, S3)

Bước 3: Phỏng vấn thử (*Mock Interviews*) (2-4 tuần)

- Luyện tập với bạn bè, đồng nghiệp
- Tự ghi hình bản thân
- Xin nhận xét, góp ý
- Lặp lại và cải thiện

Bài Toán Luyện Tập Theo Nhóm

Bảng tin mạng xã hội:

- Thiết kế Twitter
- Thiết kế Instagram
- Thiết kế bảng tin LinkedIn
- Thiết kế Reddit

Nền tảng video:

- Thiết kế YouTube
- Thiết kế Netflix
- Thiết kế Twitch
- Thiết kế TikTok

Nhắn tin:

- Thiết kế WhatsApp
- Thiết kế Slack
- Thiết kế Discord
- Thiết kế Telegram

Lưu trữ/Đồng bộ:

- Thiết kế Google Drive
- Thiết kế Dropbox
- Thiết kế iCloud
- Thiết kế S3

Thương mại điện tử:

- Thiết kế Amazon
- Thiết kế Uber
- Thiết kế Airbnb
- Thiết kế ứng dụng giao đồ ăn

Hạ tầng:

- Thiết kế dịch vụ rút gọn URL
- Thiết kế bộ giới hạn tần suất (rate limiter)
- Thiết kế trình thu thập web (web crawler)
- Thiết kế bộ nhớ đệm phân tán

Khuôn Mẫu Quản Lý Thời Gian

- 0-7 phút: Scope (làm rõ yêu cầu)
7-14 phút: Numbers (ước lượng năng lực)
14-21 phút: API (thiết kế giao diện)
21-33 phút: Key Components (kiến trúc)
33-45 phút: Elaborate (đào sâu)

Luyện tập với đồng hồ bấm giờ!

Kỷ luật là chìa khoá

Những Điểm Cốt Lõi

Mỗi loại hệ thống có một "câu chuyện":

Twitter = Bài toán phân phát (fanout)
YouTube = Mã hoá + CDN
WhatsApp = Giao tin thời gian thực
Google Drive = Đồng bộ + giải quyết xung đột

Biết câu chuyện → Biết cần tập trung vào đâu

Công thức thành công khi phỏng vấn:

Kiến thức kỹ thuật × Giao tiếp × Cấu trúc = Thành công

Bạn có thể giỏi kỹ thuật nhưng vẫn trượt nếu thiếu cấu trúc
Bạn có thể có cấu trúc hoàn hảo nhưng vẫn trượt nếu thiếu chiều sâu
Bạn cần cả hai!

Khung tư duy SNAKE mang lại cấu trúc
Luyện tập mang lại chiều sâu kỹ thuật
Phỏng vấn thử cải thiện khả năng giao tiếp

Điều người phỏng vấn muốn:

Không phải sự hoàn hảo
Không phải mọi chi tiết
Không phải lời giải học thuộc lòng

Mà là:
✓ Quy trình tư duy rõ ràng
✓ Ý thức về đánh đổi
✓ Tư duy về khả năng mở rộng
✓ Giao tiếp rõ ràng
✓ Chiều sâu ở những chỗ quan trọng

Chiến lược luyện tập:

Tuần 1: Thành thạo khung tư duy SNAKE
Tuần 2-3: Nghiên cứu các nhóm hệ thống
Tuần 4-6: Phỏng vấn thử
Tuần 7+: Phỏng vấn thật

Luyện tập đều đặn > Học dồn
45 phút × 30 bài toán = Sẵn sàng

Lời khuyên cuối cùng:

Trong buổi phỏng vấn:

- Hít thở sâu (bạn biết những điều này mà!)
- Làm rõ trước khi thiết kế
- Nói thành lời suy nghĩ của mình
- Trình bày các đánh đổi
- Tinh thần hợp tác
- Hãy tận hưởng (nói thật đây!)

Hãy nhớ:

Bạn đã học hành chăm chỉ
Bạn hiểu các hệ thống
Bạn có một khung tư duy
Bạn đã sẵn sàng

Hãy tin vào sự chuẩn bị của mình

Bạn đã có kiến thức (6 phần). Bạn đã có khung tư duy (SNAKE). Bạn đã có chiến lược (bài này).

Giờ là lúc luyện tập và chinh phục những buổi phỏng vấn đó!

Chúc mừng! Bạn đã hoàn thành toàn bộ khoá Thiết Kế Hệ Thống Từ Con Số 0 Đến Chuyên Gia.

Từ Phần 0 (Thay Đổi Mô Hình Tư Duy (*Mental Model*)) đến Phần 6 (Làm Chủ Thiết Kế Hệ Thống), bạn đã lột xác từ lập trình viên (*developer*) thành kiến trúc sư (*architect*).

Hãy tiếp tục luyện tập. Tiếp tục xây dựng. Tiếp tục phát triển.

Bạn đã sẵn sàng. Hãy chinh phục những buổi phỏng vấn đó! 🚀

Các Chủ Đề Kiến Trúc Nâng Cao - Kiến Thức Phân Biệt Kỹ Sư Lâu Năm (*Senior Engineer*) Và Kỹ Sư Cấp Staff (*Staff Engineer*)

Khi bạn đã nắm vững các kiến thức nền tảng về thiết kế hệ thống (*system design*), các mẫu thiết kế (*patterns*) và tư duy đánh đổi (*trade-off thinking*), câu hỏi tiếp theo là: **"Điều gì tạo nên sự khác biệt giữa kỹ sư lâu năm và kỹ sư cấp staff?"**

Câu trả lời không phải là học thêm mẫu thiết kế. Mà là **hiểu sâu về những vấn đề mà chỉ các hệ thống phân tán** (*distributed systems*) **ở quy mô** (*scale*) **lớn mới gặp phải** - và biết cách giải quyết chúng một cách có hệ thống.

Bài học này không phải kiến thức "bắt buộc phải biết" để đi phỏng vấn (*interview*) thiết kế hệ thống. Nhưng nó là **kiến thức giúp bạn tư duy ở tầm cao hơn** - từ "thiết kế một hệ thống" sang "phát triển và duy trì các hệ thống trong môi trường thực tế (*production*) ở quy mô lớn".

Tại Sao Các Chủ Đề Này Lại "Nâng Cao"?

Đối Chiếu Với Thực Tế

Hầu hết các hệ thống không cần event sourcing (lưu trữ dựa trên sự kiện) hay service mesh (lưới dịch vụ). Nhưng khi bạn gặp những vấn đề sau:

- **Nhật ký kiểm toán** (*Audit trail*) **không đầy đủ** → cần phát lại toàn bộ lịch sử
- **Đọc và ghi có yêu cầu** (*requirement*) **hoàn toàn khác nhau** → CQRS
- **Giao tiếp giữa các dịch vụ** (*service*) **phức tạp với hơn 50 vi dịch vụ** (*microservices*) → Service Mesh
- **Không tin hệ thống có thể chịu được sự cố** (*failure*) → Chaos Engineering (kỹ thuật hỗn loạn)
- **Yêu cầu** (*Requests*) **trùng lặp gây hỏng dữ liệu** → Tính lũy đẳng (*Idempotency*)
- **Sự cố dây chuyền** (*Cascading failures*) **lan như domino** → Bộ ngắt mạch (*Circuit Breaker*)

- **Gỡ lỗi (Debug) hệ thống phân tán như mò kim đáy bể** → Truy vết phân tán (Distributed Tracing)

Lúc đó, bạn mới hiểu tại sao những khái niệm này tồn tại.

1. Event Sourcing - Lưu Sự Kiện, Không Lưu Trạng Thái

Vấn Đề Của Cách Lưu Trạng Thái Truyền Thống

Cơ sở dữ liệu truyền thông:
 Số dư người dùng: \$1000 (chỉ có trạng thái hiện tại)

Mất hết lịch sử
 Không biết số dư này từ đâu ra
 Không kiểm toán được
 Không phát lại được

Cách Tiếp Cận Event Sourcing

Ý tưởng cốt lõi: Lưu mọi thay đổi dưới dạng một sự kiện (event), suy ra trạng thái từ các sự kiện (events)



Kho sự kiện (Event Store):

1. UserCreated { userId: 123 }
2. MoneyDeposited { userId: 123, amount: 500 }
3. MoneyWithdrawn { userId: 123, amount: 200 }
4. MoneyDeposited { userId: 123, amount: 700 }

Trạng thái hiện tại = Phát lại tất cả sự kiện
 Số dư = $0 + 500 - 200 + 700 = \$1000$

Tại Sao Dùng Event Sourcing?

1. Nhật Ký Kiểm Toán Đầy Đủ

- Biết chính xác điều gì đã xảy ra, khi nào, tại sao
- Đáp ứng các yêu cầu về tuân thủ và quy định pháp lý

2. Du Hành Thời Gian

- Dừng lại trạng thái tại bất kỳ thời điểm nào trong quá khứ
- Gỡ lỗi các sự cố trong môi trường thực tế bằng cách phát lại sự kiện

3. Nhiều Phép Chiếu (Projections)

- Cùng một luồng sự kiện (*event stream*) → nhiều khung nhìn (*views*) khác nhau
- Phân tích dữ liệu, báo cáo, bảng điều khiển (*dashboards*) thời gian thực

4. Khả Năng Mở Rộng (Scalability)

- Sự kiện chỉ được ghi nối thêm (*append-only*) → cực kỳ nhanh
- Không cần khoá (*locking*) hay giao dịch (*transactions*) phức tạp

Đánh Đổi

Ưu điểm:

- Có đầy đủ lịch sử và nhật ký kiểm toán
- Khả năng du hành thời gian
- Dễ thêm phép chiếu mới
- Phát lại sự kiện để gỡ lỗi (*debugging*)

Nhược điểm:

- Độ phức tạp (*Complexity*) tăng đáng kể
- Tiên hoá lược đồ (*schema*) của sự kiện rất khó
- Lưu trữ (*Storage*) tăng (lưu mọi sự kiện)
- Truy vấn (*Query*) trạng thái hiện tại phải phát lại các sự kiện
- Đường cong học tập dốc

Khi Nào Dùng Event Sourcing?

Dùng khi:

- Ngân hàng, hệ thống tài chính (cần kiểm toán đầy đủ)
- Miền nghiệp vụ (*Domain*) phức tạp với nhiều quy tắc nghiệp vụ (*business rules*)
- Cần lịch sử đầy đủ và khả năng du hành thời gian
- Nhiều phép chiếu từ cùng một dữ liệu

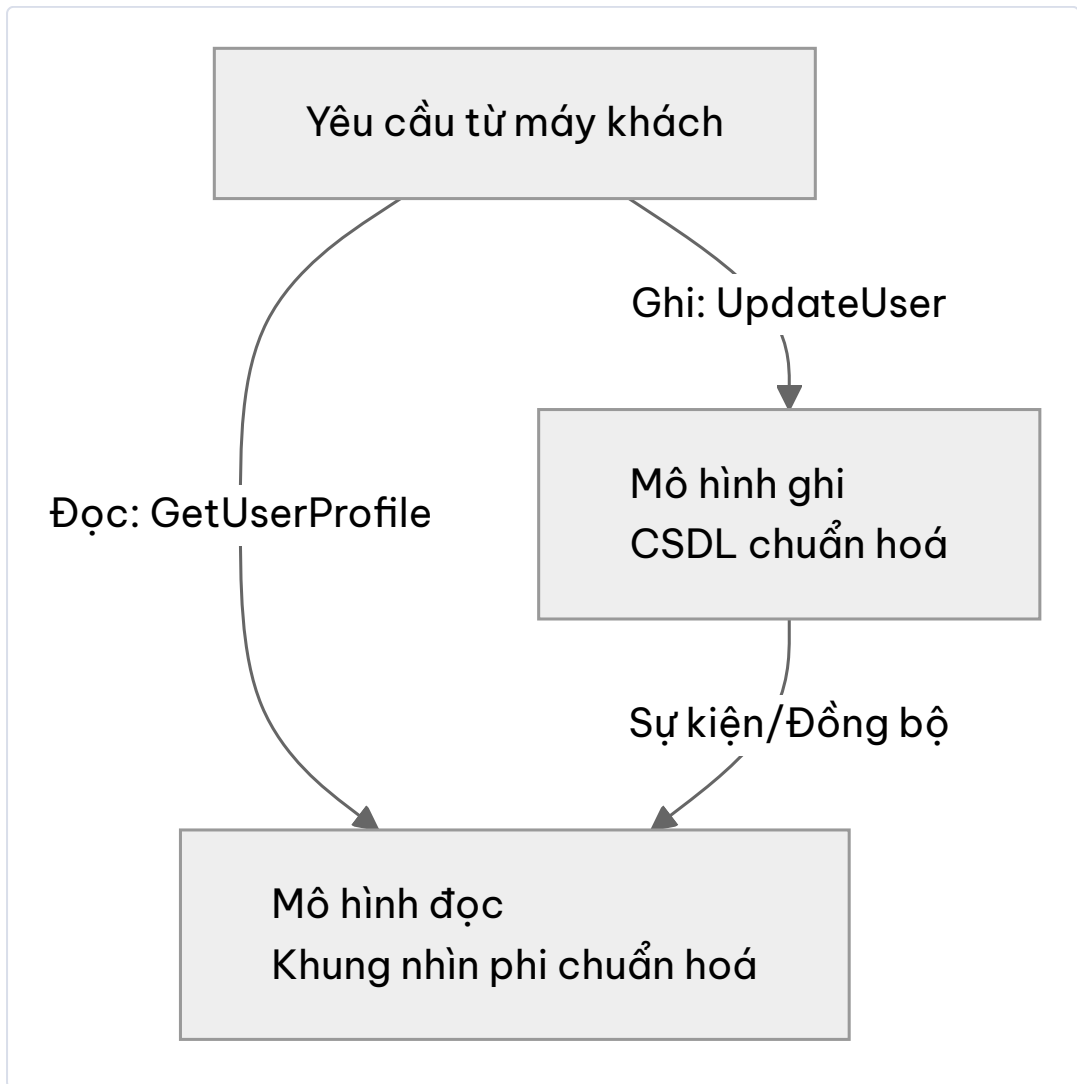
Không dùng khi:

- Ứng dụng CRUD đơn giản
- Nhóm (*Team*) chưa có kinh nghiệm
- Nhật ký kiểm toán không quan trọng
- Độ phức tạp không xứng với lợi ích

2. CQRS - Tách Biệt Trách Nhiệm Lệnh Và Truy Vấn (Command Query Responsibility Segregation)

Khái Niệm Cốt Lõi

CQRS = Tách riêng mô hình (*models*) đọc và ghi



Tại Sao Dùng CQRS?

Thực tế: Đọc và Ghi có yêu cầu (*requirements*) hoàn toàn khác nhau

Phía ghi:

- Kiểm tra quy tắc nghiệp vụ
- Đảm bảo tính nhất quán
- Xử lý giao dịch
- Dữ liệu chuẩn hoá

Phía đọc:

- Truy vấn nhanh
- Phép join phức tạp
- Phép tổng hợp
- Khung nhìn phi chuẩn hoá

Ví Dụ: Trang Sản Phẩm Thương Mại Điện Tử

Không có CQRS (Một mô hình (*model*) duy nhất):

```
-- Mỗi lần load product page:
SELECT p.*, c.name as category_name,
       AVG(r.rating) as avg_rating,
       COUNT(r.id) as review_count,
       i.quantity as stock
FROM products p
JOIN categories c ON p.category_id = c.id
LEFT JOIN reviews r ON p.id = r.product_id
LEFT JOIN inventory i ON p.id = i.product_id
WHERE p.id = ?
GROUP BY p.id

-- Slow vì phải join nhiều tables mỗi request
```

Có CQRS:


```
// Write Side (normalized)
await db.products.update({ id, name, price })
await eventBus.publish('ProductUpdated', { id, name, price })

// Read Side (denormalized view)
await cache.set(`product:${id}`, {
  id, name, price,
  category: 'Electronics',
  avgRating: 4.5,
  reviewCount: 1203,
  stock: 50
})

// Read cực nhanh - chỉ 1 cache lookup
const product = await cache.get(`product:${id}`)
```

Kết Hợp CQRS + Event Sourcing

Phía ghi: Kho sự kiện (Event Store)
 ↓
 Sự kiện: ProductCreated, PriceUpdated, ReviewAdded
 ↓
 Phía đọc: Khung nhìn vật chất hoá (Materialized Views)
 - Khung nhìn chi tiết sản phẩm (bộ nhớ đệm)
 - Chỉ mục tìm kiếm (Elasticsearch)
 - Bảng điều khiển phân tích (BigQuery)

Đánh Đổi

Ưu điểm:

- Tối ưu (*Optimize*) đọc và ghi độc lập
- Mở rộng đọc và ghi riêng biệt
- Nhiều mô hình đọc (*read models*) cho các tình huống sử dụng (*use cases*) khác nhau
- Hiệu năng (*Performance*) cải thiện đáng kể

Nhược điểm:

- Nhất quán cuối cùng (*Eventual consistency*) giữa phía ghi và phía đọc
- Độ phức tạp tăng (2 mô hình thay vì 1)
- Việc đồng bộ dữ liệu phải đáng tin cậy

- Khó gỡ lỗi và bảo trì hơn

Khi Nào Dùng CQRS?

Dùng khi:

- Tỷ lệ Đọc:Ghi chênh lệch lớn (99:1)
- Các truy vấn (*queries*) đọc cực kỳ phức tạp
- Cần mở rộng đọc và ghi theo cách khác nhau
- Nhiều khung nhìn từ cùng một dữ liệu

Không dùng khi:

- CRUD đơn giản với đọc/ghi cân bằng
- Bắt buộc phải có nhất quán mạnh (*strong consistency*)
- Nhóm nhỏ, không đủ nguồn lực để bảo trì

3. Service Mesh - Lớp Hạ Tầng (*Infrastructure*) Cho Giao Tiếp Giữa Các Dịch Vụ

Vấn Đề Giao Tiếp Giữa Các Vi Dịch Vụ

Khi bạn có hơn 50 vi dịch vụ:

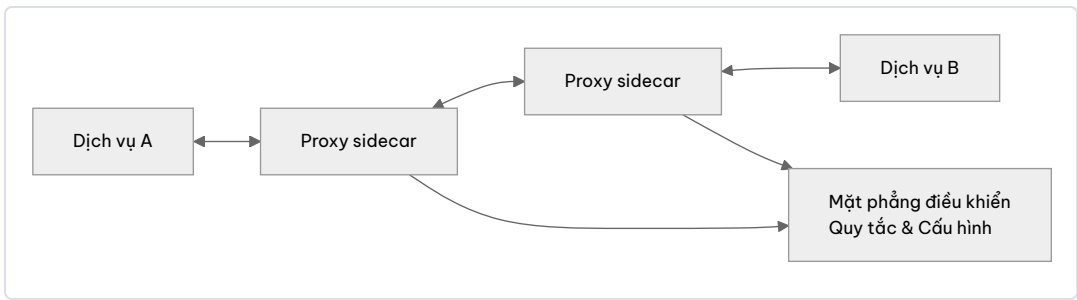
Dịch vụ A gọi Dịch vụ B:

- Logic thu' lại?
- Bộ ngắt mạch?
- Cân bằng tải?
- Mã hoá TLS?
- Truy vết?
- Số' liệu đo?

- Mỗi dịch vụ phải tự cài đặt lại tất cả
- Mã bị trùng lặp
- Hành vi không nhất quán

Giải Pháp Service Mesh

Ý tưởng cốt lõi: Chuyển logic mạng ra khỏi mã ứng dụng, đưa vào lớp hạ tầng



Proxy sidecar (*Sidecar proxy*) (như Envoy) chặn bắt toàn bộ lưu lượng (*traffic*):

Dịch vụ A → Proxy sidecar A → Proxy sidecar B → Dịch vụ B

Proxy đảm nhận:

- Cân bằng tải
- Thử lại
- Ngắt mạch
- TLS
- Số liệu đo
- Truy vết

Các Tính Năng Của Service Mesh

1. Quản Lý Lưu Lượng

- Các thuật toán cân bằng tải (*load balancing*)
- Bộ ngắt mạch (*Circuit breakers*) tự động
- Chính sách thử lại (*retry*) và hết thời gian chờ (*timeout*)
- Chia tách lưu lượng (*canary, blue-green*)

2. Bảo Mật

- TLS hai chiều (*Mutual TLS*) (mTLS) tự động
- Xác thực (*Authentication*) giữa các dịch vụ (*services*)
- Chính sách phân quyền (*authorization*)

3. Khả Năng Quan Sát (*Observability*)

- Truy vết phân tán tự động
- Thu thập số liệu đo (*metrics*)
- Trực quan hoá cấu trúc liên kết giữa các dịch vụ

4. Khả Năng Phục Hồi

- Thử lại (*Retries*) tự động
- Bộ ngắt mạch
- Tiêm lỗi (*Fault injection*) để kiểm thử

Các Service Mesh Phổ Biến

- **Istio** - đầy đủ tính năng, phức tạp
- **Linkerd** - đơn giản hơn, gọn nhẹ
- **Consul Connect** - thuộc hệ sinh thái HashiCorp

Đánh Đổi

Ưu điểm:

- Cấu hình tập trung
- Hành vi nhất quán giữa các dịch vụ
- Có sẵn khả năng quan sát ngay khi cài đặt
- Tính năng bảo mật tự động

Nhược điểm:

- Độ phức tạp cao - thêm 1 lớp hạ tầng
- Chi phí phụ (*Overhead*) về hiệu năng (lớp proxy)
- Đường cong học tập dốc
- Gỡ lỗi khó hơn

Khi Nào Cần Service Mesh?

Dùng khi:

- Hơn 50 vi dịch vụ
- Cần quản lý lưu lượng nhất quán
- Yêu cầu bảo mật cao (mTLS ở mọi nơi)
- Khả năng quan sát là then chốt

Không dùng khi:

- Dưới 10 vi dịch vụ
- Hệ thống nguyên khối (*Monolith*) hoặc kiến trúc đơn giản

- Nhóm chưa thành thạo Kubernetes
- Chi phí phụ về hiệu năng là không chấp nhận được

4. Chaos Engineering - Cố Tình Làm Hỏng Hệ Thống

Triết Lý Cốt Lõi

"Cách tốt nhất để tránh sự cố là liên tục gây ra sự cố (fail)"

Kiểm thử truyền thống:

Kiểm thử → Triển khai → Cầu mong không có gì hỏng

Chaos Engineering:

Triển khai → Cố tình làm hỏng → Khắc phục điểm yếu → Hệ thống bền bỉ hơn

Chaos Monkey Của Netflix

Netflix là người tiên phong về chaos engineering:

Chaos Monkey ngẫu nhiên:

- Tắt các instance trong môi trường thực tế
- Giết các dịch vụ
- Thêm độ trễ
- Làm rơi gói tin mạng

Mục tiêu: Buộc hệ thống phải bền bỉ ngay từ khâu thiết kế

Các Nguyên Tắc Của Chaos Engineering

1. Bắt Đầu Từ Giả Thuyết

Giả thuyết: "Nếu dịch vụ thanh toán gặp sự cố, người dùng vẫn có thể duyệt sản phẩm"

Thí nghiệm: Tắt dịch vụ thanh toán trong môi trường thực tế

Đo lường: Chức năng duyệt sản phẩm còn hoạt động không?

2. Thu Nhỏ Phạm Vi Ảnh Hưởng (Blast Radius)

Bắt đầu nhỏ:

- 1 instance → 1 dịch vụ → 1 vùng → toàn cầu

Không phải: Tắt mọi thứ cùng lúc

3. Tự Động Hoá Các Thí Nghiệm

```
// Automated chaos experiment
const experiment = {
  name: 'Database Failover',
  hypothesis: 'System handles DB failure gracefully',
  blast_radius: '10% traffic',

  fault: {
    type: 'database_failure',
    duration: '5 minutes'
  },

  steady_state: {
    metric: 'api_success_rate',
    threshold: '> 99%'
  }
}

// Run weekly tự động
```

Các Thí Nghiệm Hỗn Loạn Phổ Biến

1. Sự Cố (Failures) Dịch Vụ

- Tắt instance ngẫu nhiên
- Dịch vụ không khả dụng
- Sự cố ở các phụ thuộc (*dependency*)

2. Sự Cố Mạng

- Tiềm độ trễ (*latency*)
- Mất gói tin
- Phân vùng mạng (*Network partition*)

3. Cạn Kiệt Tài Nguyên

- CPU tăng đột biến (*spike*)
- Giả lập rò rỉ bộ nhớ

- Đầy ổ đĩa

4. Hồng Trạng Thái

- Lệch đồng hồ (*Clock skew*)
- Hồng dữ liệu
- Nút (*Nodes*) "xác sống" (*zombie*)

Công Cụ

- **Chaos Monkey** - công cụ nguyên bản của Netflix
- **Gremlin** - nền tảng thương mại
- **Chaos Mesh** - thiết kế riêng cho Kubernetes
- **Litmus** - dự án của CNCF

Đánh Đổi

Lợi ích:

- Phát hiện điểm yếu trước khi xảy ra sự cố trong môi trường thực tế
- Tạo niềm tin vào khả năng phục hồi của hệ thống
- Buộc các nhóm (*teams*) phải suy nghĩ về các kịch bản sự cố (*failure scenarios*)
- Giảm MTTR (Mean Time To Recovery - thời gian khôi phục trung bình)

Thách thức:

- Rủi ro gây ra sự cố ngừng hoạt động (*outages*) thật
- Cần hệ thống giám sát (*monitoring*) và cảnh báo (*alerting*) trưởng thành
- Thay đổi văn hoá - chấp nhận việc làm hỏng hệ thống
- Đầu tư thời gian lớn

Khi Nào Áp Dụng Chaos Engineering?

Dùng khi:

- Hệ thống trong môi trường thực tế có tầm quan trọng then chốt
- Đã có hệ thống giám sát tốt
- Nhóm đã thành thạo với hệ thống phân tán
- Chi phí của thời gian ngừng hoạt động (*downtime*) cao

Chưa dùng khi:

- Hệ thống chưa ổn định
- Giám sát chưa tốt
- Thường xuyên phải "chữa cháy" các sự cố

5. Tính Lũy Đẳng - Xử Lý Yêu Cầu Trùng Lặp Một Cách An Toàn

Vấn Đề: Mạng Không Đáng Tin Cậy

Máy khách → Máy chủ: Tạo thanh toán \$100

Kịch bản 1:

Máy khách gửi → Máy chủ xử lý → Phản hồi bị mất

Máy khách thử lại → Máy chủ trừ tiền \$200 ❌

Kịch bản 2 (Lũy đẳng):

Máy khách gửi kèm ID → Máy chủ xử lý

Máy khách thử lại với cùng ID → Máy chủ trả về kết quả đã lưu đệm

Tổng số tiền bị trừ: \$100 ✅

Khái Niệm Cốt Lõi

Thao tác lũy đẳng (idempotent): Gọi nhiều lần = cùng kết quả như gọi một lần

Theo toán học:

$$f(f(x)) = f(x)$$

Ví dụ:

SET value = 10 → lũy đẳng

value = value + 1 → KHÔNG lũy đẳng

Các Chiến Lược Triển Khai

1. Khoá Lũy Đẳng (Idempotency Keys)


```

// Client generates unique key
const idempotencyKey = uuidv4()

await api.post('/payments', {
  amount: 100,
  idempotency_key: idempotencyKey
})

// Server side
async function processPayment(request) {
  const { amount, idempotency_key } = request

  // Check if already processed
  const cached = await cache.get(idempotency_key)
  if (cached) {
    return cached.result // Return previous result
  }

  // Process payment
  const result = await chargeCard(amount)

  // Cache result
  await cache.set(idempotency_key, { result }, { ttl: 86400 })

  return result
}

```

2. Tính Lũy Đẳng Tự Nhiên

```

// Idempotent by design
PUT /users/123
{
  "name": "John",
  "email": "john@example.com"
}

// Calling nhiều lần → same state

```

3. Ràng Buộc Trong Cơ Sở Dữ Liệu (Database)

```

CREATE UNIQUE INDEX idx_payment_idempotency
ON payments(idempotency_key);

-- Duplicate insert sẽ fail → retry-safe

```

Các Cấp Độ Lũy Đăng

1. Cấp Độ Phương Thức HTTP

GET, PUT, DELETE → lũy đăng theo đặc tả
POST → KHÔNG lũy đăng (cần xử lý tường minh)

2. Cấp Độ Ứng Dụng

```
// Check database state
const existingOrder = await db.orders.findOne({
  userId, productId
})

if (existingOrder) {
  return existingOrder // Already exists
}

// Create new
return await db.orders.create({ userId, productId })
```

3. Cấp Độ Hệ Thống Phân Tán (Distributed System)

Dùng khoá phân tán (distributed lock):

- Giành khoá bằng khoá lũy đăng
- Xử lý nếu giành được khoá
- Nhả khoá
- Thử lại an toàn

Ví Dụ Thực Tế

Thanh toán với Stripe:

```
await stripe.charges.create({
  amount: 1000,
  currency: 'usd',
  source: 'tok_visa',
  idempotency_key: '123e4567-e89b-12d3'
})

// Retry with same key → returns original charge
```

Xử lý thông điệp:

```
// Consumer with deduplication
async function processMessage(message) {
  const messageId = message.id

  const processed = await db.processed_messages
    .findOne({ messageId })

  if (processed) {
    return // Already processed
  }

  // Process message
  await handleMessage(message)

  // Mark as processed
  await db.processed_messages.insert({
    messageId,
    processedAt: Date.now()
  })
}
```

Đánh Đổi

Lợi ích:

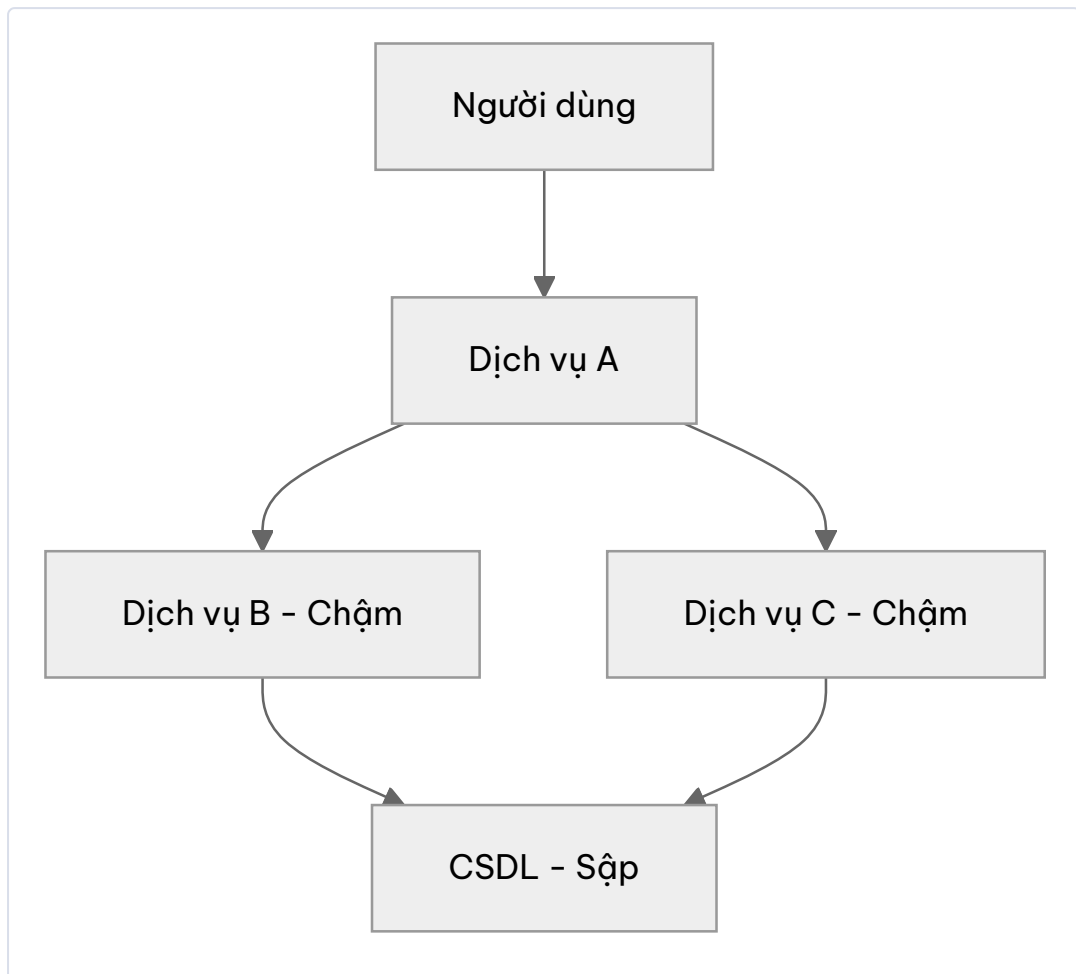
- Thử lại an toàn
- Tránh trừ tiền trùng, đơn hàng trùng, v.v.
- Độ tin cậy (*Reliability*) tốt hơn

Chi phí:

- Tốn thêm lưu trữ (lưu đệm các khoá lũy đẳng)
- Độ phức tạp khi triển khai
- Quản lý TTL (khi nào thì khoá hết hạn?)

6. Bộ Ngắt Mạch - Chặn Đúng Sự Cố Dây Chuyền

Vấn Đề: Sự Cố Dây Chuyền



CSDL sập → Dịch vụ B chậm (chờ hết thời gian chờ)
→ Các luồng của Dịch vụ A bị chặn vì chờ B
→ Dịch vụ A sập
→ Mọi dịch vụ phụ thuộc đều sập
→ Toàn bộ hệ thống sập

Mẫu Bộ Ngắt Mạch

Ý tưởng: Ngừng gọi các dịch vụ đang lỗi, thất bại nhanh (*fail fast*)

Các trạng thái:

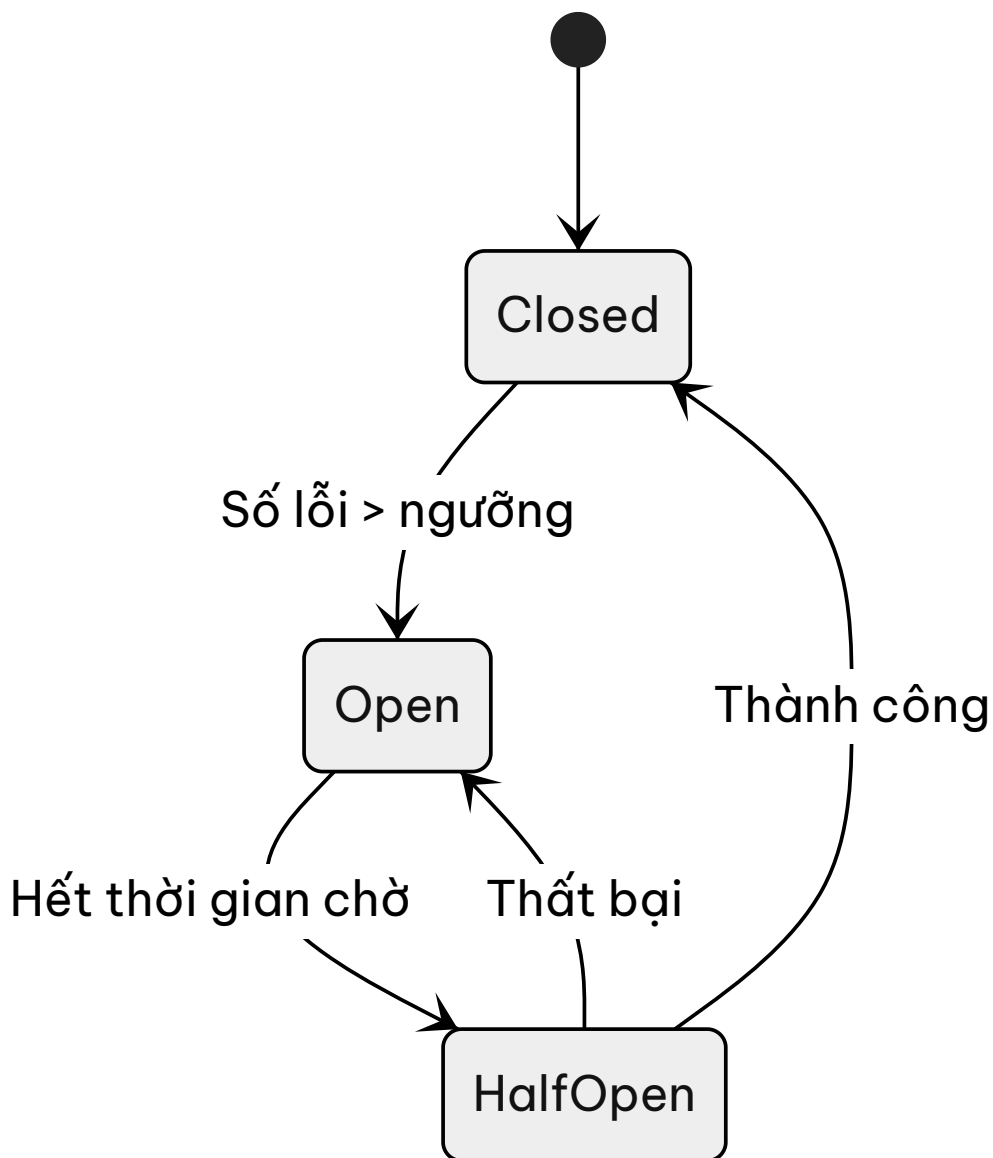
1. CLOSED (đóng) → lời gọi đi qua bình thường
2. OPEN (mở) → lời gọi thất bại ngay lập tức (không chờ)
3. HALF-OPEN (nửa mở) → thử xem dịch vụ đã phục hồi chưa

Closed → (số lỗi > ngưỡng) → Open

Open → (hết thời gian chờ) → Half-Open

Half-Open → (thành công) → Closed

Half-Open → (thất bại) → Open



Triển Khai

```
class CircuitBreaker {
  constructor(options) {
    this.failureThreshold = options.failureThreshold || 5
    this.timeout = options.timeout || 60000 // 1 minute
    this.failures = 0
    this.state = 'CLOSED'
    this.nextAttempt = null
  }

  async call(fn) {
    if (this.state === 'OPEN') {
      if (Date.now() < this.nextAttempt) {
        throw new Error('Circuit breaker OPEN')
      }
      // Try half-open
      this.state = 'HALF-OPEN'
    }

    try {
      const result = await fn()
      this.onSuccess()
      return result
    } catch (error) {
      this.onFailure()
      throw error
    }
  }

  onSuccess() {
    this.failures = 0
    this.state = 'CLOSED'
  }

  onFailure() {
    this.failures++
    if (this.failures >= this.failureThreshold) {
      this.state = 'OPEN'
      this.nextAttempt = Date.now() + this.timeout
    }
  }
}

// Usage
const breaker = new CircuitBreaker({
  failureThreshold: 5,
  timeout: 60000
})

try {
  const data = await breaker.call(() =>
    fetch('https://api.example.com/data'))
}
```

```

    )
  } catch (error) {
    // Circuit open hoặc call failed
    return fallbackData
  }
}

```

Bộ Ngắt Mạch + Phương Án Dự Phòng (Fallback)

```

async function getUserProfile(userId) {
  try {
    return await circuitBreaker.call(() =>
      userService.getProfile(userId)
    )
  } catch (error) {
    // Circuit open → use fallback
    return {
      id: userId,
      name: 'User',
      avatar: 'default-avatar.png',
      _fallback: true
    }
  }
}

```

Nâng Cao: Mẫu Bulkhead (Vách Ngăn)

Cô lập sự cố bằng các nhóm tài nguyên (resource pools):

```

// Separate thread pools cho từng service
const paymentPool = createThreadPool({ size: 10 })
const userPool = createThreadPool({ size: 20 })
const searchPool = createThreadPool({ size: 30 })

// Payment service down → only payment pool exhausted
// User và search services still working

```

Giám Sát Bộ Ngắt Mạch

```
// Metrics quan trọng
circuitBreaker.on('open', () => {
  metrics.increment('circuit_breaker.opened', {
    service: 'payment_service'
  })
  alert.send('Circuit breaker opened: payment_service')
})

circuitBreaker.on('half-open', () => {
  metrics.increment('circuit_breaker.half_open')
})

circuitBreaker.on('closed', () => {
  metrics.increment('circuit_breaker.closed')
})
```

Đánh Đổi

Lợi ích:

- Ngăn chặn sự cố dây chuyền
- Thất bại nhanh thay vì chờ hết thời gian chờ
- Cho dịch vụ đang lỗi thời gian để phục hồi
- Sử dụng tài nguyên hiệu quả hơn

Thách thức:

- Báo động nhầm (mở mạch khi không cần thiết)
- Logic phương án dự phòng cần được bảo trì
- Tinh chỉnh các ngưỡng rất khó
- Độ phức tạp trong giám sát

7. Truy Vết Phân Tán - Gỡ Lỗi Xuyên Suốt Các Dịch Vụ

Vấn Đề: Gỡ Lỗi Vi Dịch Vụ

Người dùng báo: "Thanh toán quá chậm"

Luồng yêu cầu:

Frontend → API Gateway → Auth → Cart → Inventory
→ Payment → Order → Notification → Email

Câu hỏi: Dịch vụ nào chậm?

Cách Tiếp Cận Bằng Ghi Log (Logging) Truyền Thống

```
// Service A
logger.info('Calling Service B')

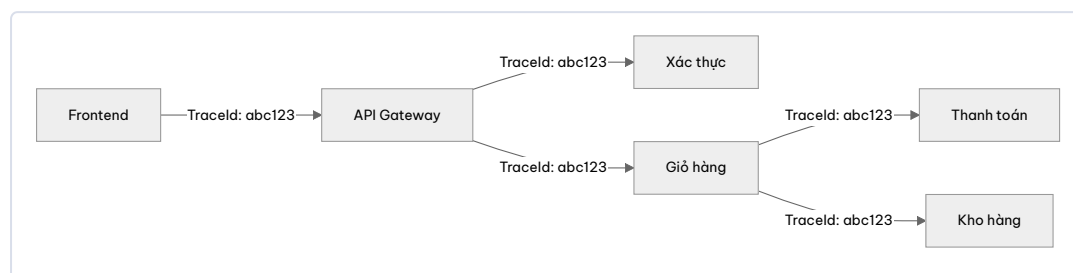
// Service B
logger.info('Processing request')

// Service C
logger.info('Query database')

// Problem: Không biết logs nào belong to cùng request
```

Giải Pháp Truy Vết Phân Tán

Khái niệm cốt lõi: Theo dõi hành trình của yêu cầu (*request*) qua tất cả các dịch vụ



Mỗi yêu cầu có:

- **Trace ID** - ID duy nhất cho toàn bộ yêu cầu
- **Span ID** - ID cho mỗi thao tác trong một trace

- **Parent Span ID** - liên kết các span với nhau

```
Trace: abc123
├─ Span: frontend (5ms)
├─ Span: api-gateway (150ms)
│   └─ Span: auth-service (20ms)
│       └─ Span: cart-service (130ms)
│           └─ Span: inventory-check (50ms)
│               └─ Span: payment-process (80ms) ← Điểm nhấn!
└─ Span: notification (10ms)
```

Triển Khai Với OpenTelemetry

```
const { trace } = require('@opentelemetry/api')
const tracer = trace.getTracer('cart-service')

async function processCheckout(request) {
  // Start span
  const span = tracer.startSpan('checkout.process')

  try {
    // Add context
    span.setAttribute('user_id', request.userId)
    span.setAttribute('cart_size', request.items.length)

    // Call other services (trace propagates automatically)
    const inventory = await checkInventory(request.items)
    const payment = await processPayment(request)

    span.setStatus({ code: SpanStatusCode.OK })
    return { success: true }
  } catch (error) {
    span.recordException(error)
    span.setStatus({ code: SpanStatusCode.ERROR })
    throw error
  } finally {
    span.end()
  }
}
```

Lan Truyền Ngữ Cảnh Truy Vết (Trace Context)

```
// HTTP headers automatically include trace context
const response = await fetch('https://payment-service/charge', {
  headers: {
    'traceparent': '00-abc123-def456-01',
    'tracestate': 'vendor=value'
  }
})

// Next service continues same trace
```

Các Công Cụ Truy Vết Phân Tán

Các nền tảng phổ biến:

- **Jaeger** - mã nguồn mở, dự án của CNCF
- **Zipkin** - hệ thống truy vết (*tracing*) của Twitter
- **Datadog APM** - thương mại, đầy đủ tính năng
- **New Relic** - thương mại, dễ cài đặt
- **Tempo** - backend truy vết của Grafana

Truy Vết + Số Liệu Đo + Log (Logs) = Khả Năng Quan Sát

Yêu cầu chậm?

1. Truy vết → tìm span chậm nhất
2. Số liệu đo → kiểm tra CPU, bộ nhớ của dịch vụ đó
3. Log → xem các lỗi tại thời điểm đó

→ Nguyên nhân gốc: Truy vấn cơ sở dữ liệu không có chỉ mục

Chiến Lược Lấy Mẫu (Sampling)

Vấn đề: Truy vết (Trace) mọi yêu cầu = bùng nổ dung lượng lưu trữ

```
// Solution: Sample intelligently
const sampler = {
  // Always trace errors
  shouldSample: (context) => {
    if (context.hasError) return true

    // Sample 1% normal requests
    if (Math.random() < 0.01) return true

    // Always trace slow requests
    if (context.duration > 1000) return true

    return false
  }
}
```

Đánh Đổi

Lợi ích:

- Trực quan hoá luồng yêu cầu
- Xác định điểm nghẽn (*bottlenecks*) ngay lập tức
- Gỡ lỗi các vấn đề xuyên dịch vụ
- Dữ liệu phục vụ tối ưu (*optimization*) hiệu năng

Chi phí:

- Chi phí phụ (độ trễ + lưu trữ)
- Công sức gắn mã đo đạc (*instrumentation*)
- Đường cong học tập
- Chi phí của các nền tảng truy vết

8. Tư Duy (Mindset) Về Khả Năng Quan Sát - Bạn Không Thể Sửa Thứ Bạn Không Nhìn Thấy

Khả Năng Quan Sát Và Giám Sát

Giám sát (truyền thống):

- Biết CÁI GÌ đã xảy ra
- Số liệu đo và bảng điều khiển định sẵn
- "Dịch vụ có đang chạy không?"
- "CPU có > 80% không?"

Khả năng quan sát:

- Biết TẠI SAO nó xảy ra
- Đặt câu hỏi bất kỳ
- "Tại sao yêu cầu này lại chậm?"
- "Điều gì đã thay đổi trước sự cố?"

Ba Trụ Cột Của Khả Năng Quan Sát

1. Số Liệu Đo (Dữ Liệu Tổng Hợp)

```
// Counters
requests_total{service="api", status="200"} 1024
requests_total{service="api", status="500"} 12

// Gauges
active_connections{service="api"} 45

// Histograms
request_duration_seconds{service="api", le="0.1"} 800
request_duration_seconds{service="api", le="1.0"} 950
```

2. Log (Các Sự Kiện Rời Rạc)

```
{
  "timestamp": "2024-01-15T10:30:00Z",
  "level": "error",
  "service": "payment",
  "trace_id": "abc123",
  "message": "Payment failed",
  "error": "Insufficient funds",
  "user_id": "user_456",
  "amount": 100
}
```

3. Truy Vết (Traces) (Hành Trình Của Yêu Cầu)

Luồng yêu cầu kèm thời gian:
Frontend (5ms)
→ API Gateway (10ms)
→ Auth (50ms)
→ Cơ sở dữ liệu (200ms) ← điêm nghẽn!

Thực Hành Tốt Nhất (Best Practices) Về Ghi Log Có Cấu Trúc (Structured Logging)

```
// Bad: Unstructured logs
logger.info('User user_123 purchased item_456 for $100')

// Good: Structured logs
logger.info('purchase_completed', {
  user_id: 'user_123',
  item_id: 'item_456',
  amount: 100,
  currency: 'USD',
  trace_id: context.traceId,
  duration_ms: 250
})

// Query-able:
// - All purchases by user_123
// - All purchases > $50
// - All slow purchases (duration > 1000ms)
```

Dữ Liệu Có Lực Lượng Cao (High Cardinality)

Khả năng quan sát hiện đại xử lý được dữ liệu có lực lượng cao:

Giám sát kiểu cũ:

- user_id? Không, quá nhiều giá trị khác nhau!
- Chỉ tổng hợp theo dịch vụ, endpoint

Khả năng quan sát hiện đại:

- user_id? Có!
- Request ID? Có!
- Biến thể A/B test? Có!
- Phân khúc khách hàng? Có!

→ Đặt BẤT KỲ câu hỏi nào về hành trình của một người dùng cụ thể

Phát Triển Hướng Khả Năng Quan Sát

```
// Instrument code với context
async function processOrder(order) {
  const span = tracer.startSpan('process_order')

  // Rich context
  span.setAttributes({
    'order.id': order.id,
    'order.value': order.total,
    'order.items_count': order.items.length,
    'user.tier': order.user.tier,
    'user.country': order.user.country
  })

  try {
    // Business logic
    const result = await processPayment(order)

    // Success metrics
    metrics.increment('orders.processed', {
      tier: order.user.tier,
      payment_method: order.paymentMethod
    })

    return result
  } catch (error) {
    // Error context
    span.recordException(error)
    logger.error('order_processing_failed', {
      order_id: order.id,
      error: error.message,
      stack: error.stack
    })
    throw error
  } finally {
    span.end()
  }
}
```

SLI, SLO, SLA

Chỉ số mức dịch vụ - Service Level Indicators (SLIs):

Số liệu đo lường sức khỏe dịch vụ:

- Tính sẵn sàng: 99.9% yêu cầu thành công
- Độ trễ: 95% yêu cầu < 200ms
- Thông lượng: 1000 req/s

Mục tiêu mức dịch vụ - Service Level Objectives (SLOs):

Mục tiêu nội bộ:

- Tính sẵn sàng của API: 99.9% (43 phút ngừng hoạt động/tháng)
- Độ trễ P95: < 200ms
- Tỷ lệ lỗi: < 0.1%

Thỏa thuận mức dịch vụ - Service Level Agreements (SLAs):

Cam kết với khách hàng:

- Đảm bảo 99.9% thời gian hoạt động
- Bị phạt nếu không đạt
- Lời hứa công khai với bên ngoài

Ngân Sách Lỗi (*Error Budget*)

SLO: 99.9% tính sẵn sàng

→ Ngân sách lỗi: 0.1% = 43 phút ngừng hoạt động/tháng

Dùng ngân sách lỗi để:

- Triển khai các tính năng rủi ro hơn
- Chạy các thí nghiệm hỗn loạn
- Các khung thời gian bảo trì

Nếu đã tiêu hết ngân sách lỗi:

- Đóng băng việc triển khai
- Tập trung vào độ tin cậy

Thực Hành Tốt Nhất Về Cảnh Báo

```
// Alert on symptoms, not causes
alert: "CPU > 80%" // So what?

// Alert on user impact
alert: "Error rate > 1% for 5 minutes"
alert: "P95 latency > 1s for 5 minutes"

// Alert with context
{
  alert: "High Error Rate",
  severity: "critical",
  service: "payment",
  error_rate: "5.2%",
  affected_users: 1200,
  trace_samples: ["abc123", "def456"],
  runbook: "https://wiki/payment-errors"
}
```

Các Nền Tảng Khả Năng Quan Sát Hiện Đại

- **Datadog** - tất cả trong một, đắt đỏ
- **New Relic** - APM + khả năng quan sát
- **Grafana Stack** - mã nguồn mở (Loki + Tempo + Mimir)
- **Honeycomb** - chuyên về dữ liệu có lực lượng cao (*high-cardinality*)
- **Lightstep** - cách tiếp cận lấy truy vết làm trọng tâm

Những Điểm Cốt Lỗi

Khi Nào Cần Các Chủ Đề Nâng Cao?

Event Sourcing + CQRS:

- Hệ thống tài chính cần kiểm toán đầy đủ
- Miền nghiệp vụ (*Domains*) phức tạp với nhiều quy tắc nghiệp vụ
- Cần nhiều phép chiếu từ cùng một dữ liệu

Service Mesh:

- Hơn 50 vi dịch vụ

- Yêu cầu bảo mật cao (mTLS)
- Cần khả năng quan sát thống nhất

Chaos Engineering:

- Hệ thống mà môi trường thực tế có tầm quan trọng then chốt
- Chi phí của thời gian ngừng hoạt động cao
- Đã có hệ thống giám sát trưởng thành

Tính Lũy Đẳng:

- Xử lý thanh toán, đơn hàng
- Bất kỳ thao tác nào không muốn bị trùng lặp
- Hệ thống phân tán có cơ chế thử lại

Bộ Ngắt Mạch:

- Vi dịch vụ có nhiều phụ thuộc (*dependencies*)
- Ngăn chặn sự cố dây chuyền
- Cần suy giảm có kiểm soát (*graceful degradation*)

Truy Vết Phân Tán:

- 5 vi dịch vụ
- Gỡ lỗi các vấn đề xuyên dịch vụ
- Tối ưu hiệu năng là then chốt

Khả Năng Quan Sát:

- LUÔN LUÔN - không thể gỡ lỗi hệ thống phân tán nếu không có khả năng quan sát
- Bắt đầu đơn giản, phát triển dần dần

Lộ Trình Phát Triển

Mới vào nghề → Lâu năm (Junior → Senior):

- Nắm vững các mẫu thiết kế hệ thống
- Hiểu các sự đánh đổi
- Thiết kế hệ thống từ con số 0

Lâu năm → Cấp Staff (Senior → Staff):

- Biết khi nào các mẫu phức tạp xứng đáng với chi phí
- Phát triển và duy trì hệ thống ở quy mô lớn
- Xây dựng khả năng quan sát ngay từ đầu
- Suy nghĩ về các kịch bản sự cố
- Cân bằng giữa độ phức tạp và lợi ích

Lời Khuyên Thực Tế

1. Đừng thiết kế thừa (*over-engineer*)

- Bắt đầu đơn giản, chỉ thêm độ phức tạp khi cần
- Đo lường trước khi tối ưu
- Tự hỏi liệu lợi ích có lớn hơn chi phí không

2. Khả năng quan sát trước tiên

- Không thể gỡ lỗi thứ bạn không nhìn thấy
- Gắn mã đo đạc (*Instrument*) ngay từ ngày đầu
- Ghi log có cấu trúc, truy vết, số liệu đo

3. Học từ sự cố

- Viết báo cáo hậu kiểm (*post-mortems*) sau các sự cố
- Áp dụng chaos engineering khi đã sẵn sàng
- Xây dựng khả năng phục hồi dần dần

4. Tập trung vào tác động tới người dùng

- Số liệu đo phải gắn với trải nghiệm người dùng
- SLO dựa trên kỳ vọng của người dùng
- Cảnh báo (*Alert*) dựa trên những triệu chứng mà người dùng cảm nhận được

Lời Kết

Các chủ đề nâng cao không phải để "khoe kiến thức kỹ thuật". Chúng là công cụ giải quyết những vấn đề thực tế mà chỉ các hệ thống trong môi trường thực tế ở quy mô lớn mới gặp phải.

Kỹ sư cấp staff khác kỹ sư lâu năm (*senior*) ở chỗ: **Biết khi nào nên dừng, khi nào không nên dừng, và lý do tại sao.**

Đừng cố học mọi thứ. Hãy học khi bạn cần đến chúng. Tập trung vào kiến thức nền tảng trước.

Tư Duy Kiến Trúc Sư & Tư Duy Vận Hành Thực Tế - Từ Kỹ Sư Đến Kiến Trúc Sư Đáng Tin Cậy

Sau khi đã nắm vững mọi kiến thức kỹ thuật - mẫu thiết kế (*patterns*), hệ thống phân tán (*distributed systems*), khả năng mở rộng (*scalability*) - câu hỏi cuối cùng là: **"Điều gì tạo nên một kiến trúc sư đáng tin cậy (*trusted architect*)?"**

Câu trả lời không phải là kỹ năng kỹ thuật. Mà là **sự chuyển dịch tư duy (*mindset*)** từ **"viết mã"** sang **"làm chủ hệ thống"**.

Kỹ sư (*Engineer*) viết mã để giải quyết vấn đề. Kiến trúc sư (*Architect*) thiết kế hệ thống và **chịu trách nhiệm khi môi trường thực tế (*production*) sập lúc 3 giờ sáng**.

Bài này không dạy thêm mẫu thiết kế nào nữa. Nó dạy cách **tư duy như người phải trực sự cố (*on-call*)**, **phải giải thích sự cố ngừng dịch vụ (*outage*) cho CEO**, **phải lên kế hoạch chuyển đổi (*migration*) cho 100 triệu người dùng**.

Đổi Chiều Thực Tế: Bạn Làm Chủ Hệ Thống

Tư Duy Kỹ Sư So Với Tư Duy Kiến Trúc Sư

Tư duy kỹ sư:

"Tính năng chạy được trên laptop của tôi ✓"

"Đã qua hết các bài kiểm thử" ✓

"Đã triển khai lên môi trường thực tế" ✓

→ Chuyển sang ticket tiếp theo

Tư duy kiến trúc sư:

"Cái này có chạy được khi tải gấp 10 lần không?"

"Nếu cơ sở dữ liệu sập thì sao?"

"Chúng ta hoàn tác (*rollback*) thế nào?"

"Chiến lược giám sát là gì?"

"Ai trực sự cố khi cái này hỏng?"

→ Làm chủ hệ thống

Khác biệt cốt lõi: Kiến trúc sư nghĩ xa hơn kịch bản suôn sẻ (*happy path*).

Phép Thử 3 Giờ Sáng

Kịch bản: Cảnh báo trên môi trường thực tế lúc 3 giờ sáng

Phản ứng của kỹ sư:

"Không phải mã của tôi, không phải việc của tôi"

"Đề'mai kiểm tra"

Phản ứng của kiến trúc sư:

"Người dùng bị ảnh hưởng thế' nào?"

"Làm sao giảm thiểu ngay bây giờ?"

"Phân tích nguyên nhân gốc đề'mai"

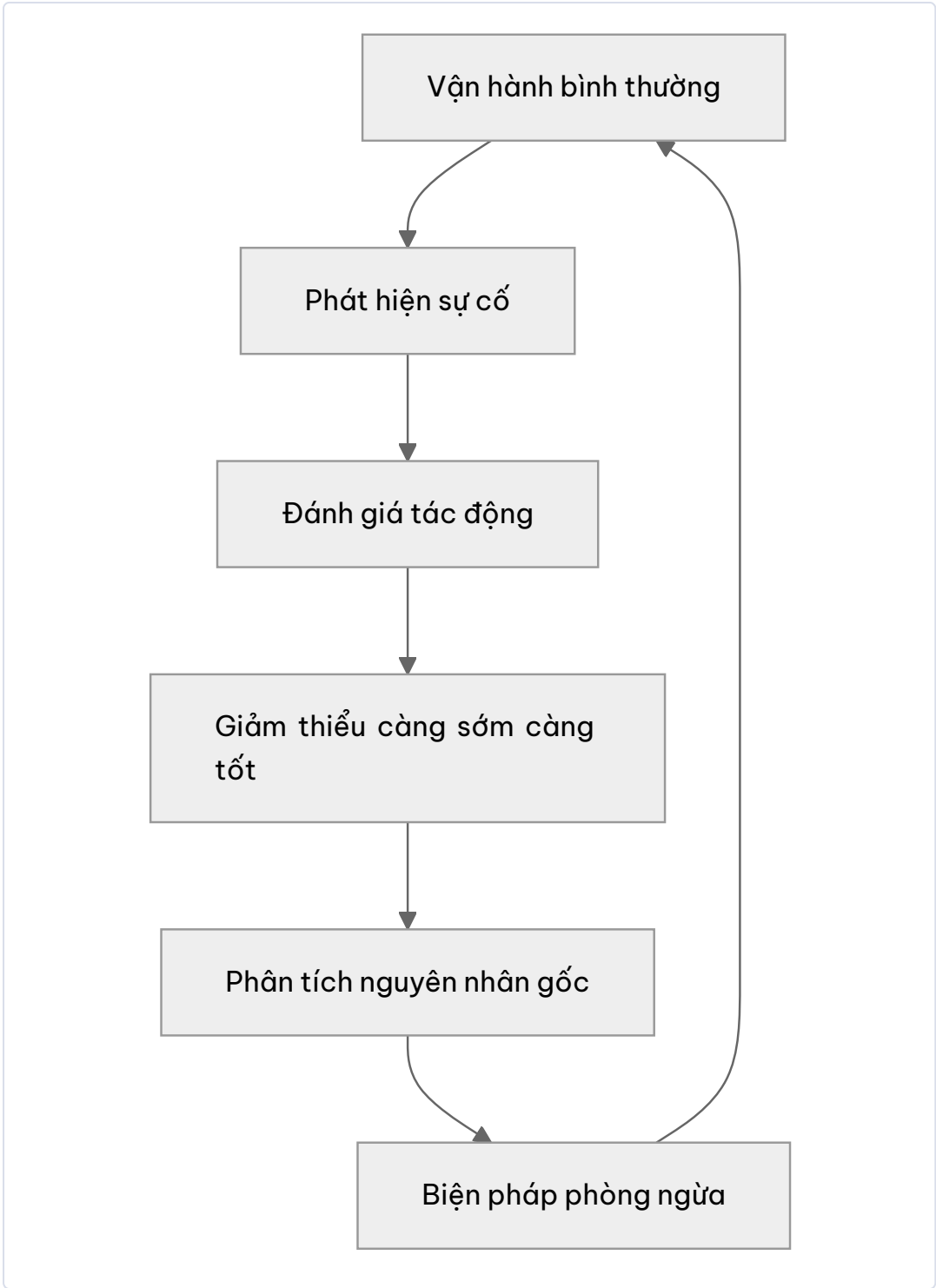
"Làm sao ngăn chuyện này tái diễn?"

Tư duy kiến trúc sư = Tư duy làm chủ.

1. Tư Duy Về Sự Cố (Incident) Thực Tế - Luôn Luôn Trước Sự Cố

Mọi Thứ Rồi Sẽ Hỏng

Sự thật nền tảng: Mọi hệ thống đều sẽ gặp sự cố. Việc của bạn là giảm thiểu tác động.



Khung Tư Duy (Framework) Ứng Phó Sự Cố

Giai đoạn 1: Phát hiện (Từng phút đầu quan trọng)

```
// Bad: Discover từ user complaints
User tweet: "Your site is down!"
→ 30 minutes before team knows

// Good: Proactive monitoring
Alert: "API error rate: 5% (threshold: 1%)"
Slack: "@oncall immediate action required"
→ Team knows trong 2 minutes
```

Giai đoạn 2: Đánh giá (Hiểu rồi mới hành động)

Các câu hỏi then chốt:

1. Tác động đến người dùng? (Bao nhiêu người dùng bị ảnh hưởng?)
2. Tác động đến kinh doanh? (Mất doanh thu? Vi phạm SLA?)
3. Phạm vi? (Một dịch vụ? Toàn bộ hệ thống?)
4. Xu hướng? (Đang tệ đi? Ổn định?)

Ví dụ:

- 15% yêu cầu thanh toán đang lỗi
 - ~500 người dùng bị ảnh hưởng
 - Chỉ dịch vụ thanh toán
 - Tỷ lệ lỗi ổn định trong 10 phút gần nhất
- Mức độ nghiêm trọng cao, phạm vi hoành vùng được

Giai đoạn 3: Giảm thiểu (Sửa ngay, tìm hiểu sau)

Ưu tiên: Cầm máu trước đã

Các phương án (theo thứ tự):

1. Hoàn tác - rollback (an toàn nhất, nhanh nhất)
2. Tắt cờ tính năng - feature flag (vô hiệu hoá mã mới)
3. Định tuyến lưu lượng (chuyển sang các instance khoẻ mạnh)
4. Suy giảm có kiểm soát (tắt các tính năng không thiết yếu)
5. Sửa khẩn cấp (phương án cuối cùng)

Ví dụ:

Dịch vụ thanh toán sập?

→ Hoàn tác về phiên bản trước (5 phút)

Không phải: Gỡ lỗi tìm nguyên nhân gốc ngay trên môi trường thực tế (hàng giờ)

Giai đoạn 4: Truyền thông (Giữ các bên liên quan (stakeholders) luôn nắm tình hình)

Mẫu cập nhật sự cố:

Trạng thái: ĐANG ĐIỀU TRA

Tác động: 15% lượt thanh toán thất bại

Bắt đầu: 14:30 UTC

Hành động hiện tại: Đang hoàn tác dịch vụ thanh toán về v2.3.1

Cập nhật tiếp theo: 14:45 UTC

Đối tượng nhận:

- Nhóm nội bộ (Slack #incidents)
- Nhóm hỗ trợ (thông báo ngăn cho khách hàng)
- Ban quản lý (tác động kinh doanh)
- Khách hàng nếu cần (trạng trạng thái)

Giai đoạn 5: Phân Tích Nguyên Nhân Gốc (*Root Cause Analysis*) (**Học từ thất bại**)

Báo cáo sau sự cố (Post-Mortem): Dịch vụ thanh toán ngừng hoạt động

Ngày: 2024-02-15

Thời lượng: 14:30 - 14:52 UTC (22 phút)

Tác động: 15% lượt thanh toán thất bại, ~500 người dùng bị ảnh hưởng

Dòng thời gian

14:30 - Triển khai v2.3.2 với logic thử lại (retry) mới
14:32 - Tỷ lệ lỗi bắt đầu tăng
14:35 - Cảnh báo được kích hoạt
14:37 - Người trực sự cố xác nhận
14:40 - Quyết định hoàn tác
14:45 - Hoàn tác xong
14:52 - Dịch vụ phục hồi

Nguyên nhân gốc

Logic thử lại gây ra vòng lặp vô hạn với một số loại lỗi.
Khi tải cao, làm cạn kiệt connection pool.

Tác động

- 500 người dùng gặp lỗi khi thanh toán
- Doanh thu ước tính bị mất: \$5.000
- Không mất hay hỏng dữ liệu

Việc cần làm

- [] Thêm bộ ngắt mạch vào logic thử lại (Phụ trách: @alice, Hạn: 2/20)
- [] Kiểm thử tải cho các kịch bản thử lại (Phụ trách: @bob, Hạn: 2/22)
- [] Giám sát connection pool (Phụ trách: @carol, Hạn: 2/18)
- [] Rà soát danh sách kiểm tra triển khai (Phụ trách: @dave, Hạn: 2/25)

Những gì làm tốt

- Phát hiện nhanh (2 phút từ lúc triển khai đến lúc có cảnh báo)
- Quy trình hoàn tác rõ ràng
- Truyền thông tốt

Những gì cần cải thiện

- Kiểm thử tải đã không phát hiện được kịch bản này
- Logic thử lại chưa được review đủ kỹ
- Cần quan sát connection pool tốt hơn

Văn Hoá Không Đồ Lỗi

Báo cáo sau sự cố'tôi:

"Bob đã triển khai mã có lỗi"

"Alice đã không review cẩn thận"

→ Mọi người che giấu sai lầm

Báo cáo sau sự cố'tốt:

"Quy trình triển khai đã không bắt được vòng lặp vô hạn"

"Cần tăng độ bao phủ của kiểm thử'tải"

"Danh sách review nên có các mẫu logic thử'lại"

→ Tập trung cải thiện hệ thống

2. Tư Duy Về Sự Cố - Thiết Kế Để Chịu Được Sự Cố

Định Luật Murphy Cho Kiến Trúc Sư

"Cái gì có thể hỏng, thì sẽ hỏng."

Việc của kiến trúc sư: Giả định sự cố'sẽ xảy ra, thiết kế'để'hệ thống chống chịu được

Các sự cố'thường gặp:

- Máy chủ sập
- Phân vùng mạng
- Cơ sở dữ liệu chậm/sập
- Đầy ổ'đĩa
- Rò rỉ bộ nhớ
- Phụ thuộc bên ngoài hết thời gian chờ
- Hỏng dữ liệu
- Lỗi cấu hình
- Lỗi con người (khi triển khai)
- Tấn công DDoS

Phân Tích Các Kiểu Sự Cố (Failure Mode)

Trước khi thiết kế hệ thống, hãy liệt kê các kịch bản sự cố:

Thanh toán thương mại điện tử - Các kịch bản sự cố

Sự cố thành phần

- Công thanh toán sập → Thu' lại + hàng đợi
- Dịch vụ tiền kho chậm → Bộ ngắt mạch + bộ nhớ đệm
- Cạn kiệt connection pool của CSDL → Giới hạn số' kết nối
- Bộ nhớ đệm Redis sập → Dự phòng sang cơ sở dữ liệu

Sự cố mạng

- Hết thời gian chờ khi gọi dịch vụ thanh toán → Xu' lý bất đồng bộ
- Mất gói tin → Thu' lại với exponential backoff
- Phân vùng mạng → Nhặt quán cuối cùng

Sự cố dữ liệu

- Thanh toán trùng lặp → Khoá lũy đẳng (idempotency key)
- Race condition trên tiền kho → Khoá lạc quan (optimistic locking)
- Hỏng dữ liệu → Checksum + kiểm tra hợp lệ

Sự cố vận hành

- Triển khai lỗi → Phát hành dần + kế' hoạch hoàn tác
- Lỗi khi đổi cấu hình → Kiểm tra cấu hình + chạy thử' (dry-run)
- Chứng chỉ hết hạn → Tự động gia hạn + cảnh báo

Thiết Kế Cho Suy Giảm Có Kiểm Soát (*Degradation*)

Không phải tính năng nào cũng thiết yếu như nhau:

```
// Example: E-commerce homepage

Critical (must work):
- Browse products ✓
- Search ✓
- Add to cart ✓
- Checkout ✓

Non-critical (can degrade):
- Recommendations ⚠️ → Show static list
- Reviews ⚠️ → Hide section
- Personalization ⚠️ → Generic experience
- Real-time inventory ⚠️ → Show "In stock" always

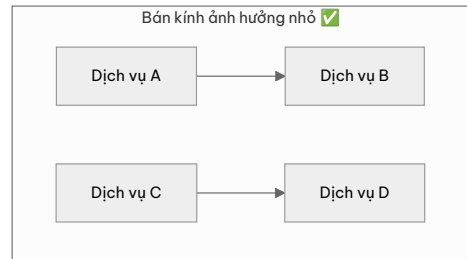
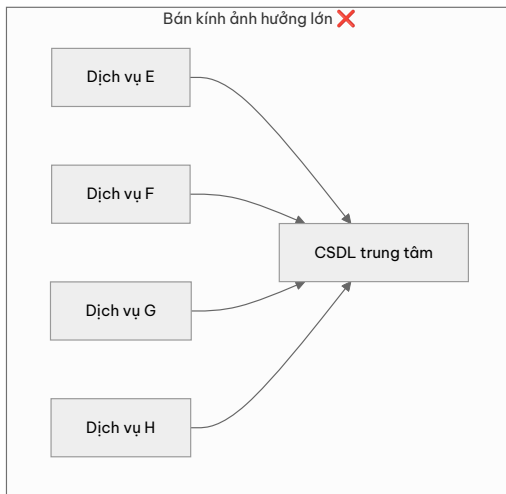
// Implementation
async function getHomepage() {
  const products = await getProducts() // Critical

  let recommendations = []
  try {
    recommendations = await getRecommendations()
  } catch (error) {
    // Degrade gracefully
    recommendations = DEFAULT_RECOMMENDATIONS
    logger.warn('Recommendations service down, using defaults')
  }

  return { products, recommendations }
}
```

Tư Duy Về Bán Kính Ảnh Hưởng (Blast Radius)

Giới hạn tác động của sự cố:



Bán kính ảnh hưởng nhỏ:

- Dịch vụ B lỗi → chỉ Dịch vụ A bị ảnh hưởng
- Vùng sự cố được cô lập

Bán kính ảnh hưởng lớn:

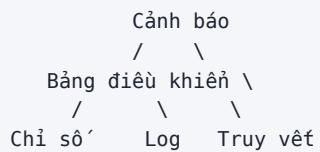
- CSDL trung tâm lỗi → TẤT CẢ dịch vụ sập
- Điểm lỗi duy nhất

Các chiến lược thu hẹp bán kính ảnh hưởng:

- Cô lập bằng vi dịch vụ (*microservices*)
- Phân mảnh dữ liệu (*Database sharding*)
- Triển khai đa vùng (*Multi-region deployment*)
- Bulkhead pattern (mẫu vách ngăn)
- Bộ ngắt mạch (*Circuit breakers*)

3. Chiến Lược Giám Sát (Monitoring) & Cảnh Báo (Alerting) - Biết Trước Khi Người Dùng Biết

Kim Tự Tháp Giám Sát



Đáy: Thu thập mọi thứ
Giữa: Trực quan hoá những gì quan trọng
Đỉnh: Chỉ cảnh báo những gì nghiêm trọng

Cần Giám Sát Những Gì

1. Chỉ số (Metrics) Phía Người Dùng (Quan Trọng Nhất)

```
// These affect users directly
const userMetrics = {
  availability: '99.9%', // Can users access?
  latency_p50: '100ms', // How fast?
  latency_p95: '300ms', // Slow requests?
  latency_p99: '1000ms', // Worst case?
  error_rate: '0.1%', // How many failures?
  success_rate: '99.9%' // Overall health
}
```

2. Chỉ số Sức Khỏe Hệ Thống

```
const systemMetrics = {
  cpu_usage: '45%',
  memory_usage: '60%',
  disk_usage: '70%',
  network_io: '500 Mbps',
  connection_pool: '80/100 active'
}
```

3. Chỉ số Kinh Doanh

```
const businessMetrics = {
  orders_per_minute: 150,
  revenue_per_hour: '$5000',
  active_users: 2500,
  checkout_conversion: '3.2%'
}
```

Bốn Tín Hiệu Vàng (*The Golden Signals*) (Google SRE)

Hãy giám sát 4 chỉ số này cho mọi dịch vụ (service):

1. Độ trễ (Latency) - Yêu cầu mất bao lâu?
2. Lưu lượng (Traffic) - Nhu cầu bao nhiêu?
3. Lỗi (Errors) - Bao nhiêu lần thất bại?
4. Độ bão hoà (Saturation) - Hệ thống "đầy" đến mức nào?

Ví dụ: Dịch vụ API

- Độ trễ: p95 = 200ms
- Lưu lượng: 1000 req/s
- Lỗi: tỷ lệ lỗi 0,5%
- Độ bão hoà: CPU 60%, bộ nhớ 70%

Triết Lý Cảnh Báo

Cảnh báo dựa trên triệu chứng, không phải nguyên nhân:

Cảnh báo tồi (nguyên nhân):

- CPU > 80%
- Bộ nhớ > 90%
- Ổ đĩa > 85%

Vì sao tồi? Người dùng không quan tâm đến CPU.

Cảnh báo tốt (triệu chứng):

- Tỷ lệ lỗi > 1% trong 5 phút
- Độ trễ P95 > 1s trong 5 phút
- Tính sẵn sàng < 99,9% trong giờ vừa qua

Vì sao tốt? Những thứ này ảnh hưởng đến người dùng.

Phòng Tránh Một Mỏi Vì Cảnh Báo (Alert Fatigue)

```
// Problem: Too many alerts
Alert: CPU > 80% // Every day, not actionable
Alert: Disk > 70% // Not urgent
Alert: Memory > 60% // Normal

Team: Ignores all alerts

// Solution: Alert only on user impact
const alertConfig = {
  error_rate: {
    threshold: '1%',
    duration: '5 minutes',
    severity: 'critical',
    action: 'Page on-call immediately'
  },

  latency_p95: {
    threshold: '1000ms',
    duration: '10 minutes',
    severity: 'warning',
    action: 'Slack notification'
  }
}
```

Sổ Tay Xử Lý (Runbooks) Cho Các Sự Cố Thường Gặp

Sổ tay xử lý (Runbook): Cảnh báo tỷ lệ lỗi cao

Cảnh báo

Tỷ lệ lỗi > 1% trong 5 phút

Hành động tức thì

1. Kiểm tra trạng thái: <https://status.company.com>
2. Kiểm tra các lần triển khai gần đây (2 giờ qua)
3. Kiểm tra sức khỏe các phụ thuộc (thanh toán, cơ sở dữ liệu)

Các bước chẩn đoán

1. Xem log lỗi:
``kubectl logs -l app=api --tail=100 | grep ERROR``
2. Kiểm tra truy vết của các yêu cầu chậm
3. Xem bảng điều khiển chỉ số

Nguyên nhân thường gặp

- Vừa triển khai → Hoàn tác
- Phụ thuộc hết thời gian chờ → Bộ ngắt mạch đã kích hoạt chưa?
- Cơ sở dữ liệu chậm → Kiểm tra slow query log
- Chạm giới hạn tần suất → Kiểm tra lưu lượng tăng đột biến

Giảm thiểu

- Hoàn tác: ``./scripts/rollback.sh``
- Tắt tính năng: ``feature-flag payment-v2 off``
- Mở rộng: ``kubectl scale deployment api --replicas=10``

Leo thang

Nếu không xử lý xong trong 15 phút:

- Leo thang lên @architect
- Cập nhật kênh sự cố
- Liên hệ nhà cung cấp nếu là vấn đề từ phụ thuộc

4. Hoạch Định Năng Lực (*Capacity Planning*) - Mở Rộng Trước Khi Cần Đến

Dự Phóng Tăng Trưởng

```
// Current metrics (Feb 2024)
const current = {
  daily_users: 100_000,
  peak_requests_per_second: 1_000,
  database_size: '500 GB',
  monthly_cost: '$10_000'
}

// Growth rate: 20% per month
const projections = {
  '3_months': {
    users: 172_000,    //  $1.2^3 = 1.72x$ 
    rps: 1_720,
    db_size: '860 GB',
    cost: '$17_200'
  },

  '6_months': {
    users: 298_000,    //  $1.2^6 = 2.98x$ 
    rps: 2_980,
    db_size: '1.5 TB',
    cost: '$29_800'
  },

  '12_months': {
    users: 890_000,    //  $1.2^{12} = 8.9x$ 
    rps: 8_900,
    db_size: '4.5 TB',
    cost: '$89_000'
  }
}
```

Ma Trận Hoạch Định Tài Nguyên

Kế hoạch năng lực - 6 tháng tới

Năng lực hiện tại

- Máy chủ API: 10 instance, CPU trung bình 60%
- Cơ sở dữ liệu: 1 máy chủ chính + 2 bản sao, dùng 70% kết nối
- Bộ nhớ đệm: Redis 50GB, dùng 40% bộ nhớ
- Năng lực tối đa: ~1500 RPS trước khi bắt đầu suy giảm

Dự phóng 6 tháng

- Tải dự kiến: 3000 RPS (gấp 2 lần mức tối đa hiện tại)
- Lưu lượng đỉnh: 4500 RPS (mùa lễ hội)

Kế hoạch hành động

Thời gian	Hành động	Chi phí	Phụ trách
-----	-----	-----	-----
Tháng 1	Thêm 10 instance API	+\$2k/tháng	DevOps
Tháng 2	Bản sao chỉ đọc của CSDL (2 → 4)	+\$3k/tháng	DBA
Tháng 2	Nâng cấp cụm Redis (50GB → 100GB)	+\$1k/tháng	DevOps
Tháng 3	Chuẩn bị phân mảnh CSDL	\$0	Backend
Tháng 4	Triển khai phân mảnh CSDL	\$0	Backend
Tháng 5	Nâng cấp CDN cho tài nguyên tĩnh	+\$1k/tháng	Frontend
Tháng 6	Kiểm thử tải ở mức 5000 RPS	\$0	QA

Đánh giá rủi ro

- ⚠ Phân mảnh CSDL phức tạp, dành ra 2 tháng
- ⚠ Cân môi trường kiểm thử tải
- Mở rộng API thì đơn giản

Chiến Lược Kiểm Thử Tải (Load Testing)

```
// Gradual load test
const loadTest = {
  baseline: {
    users: 1000,
    duration: '10 minutes',
    expected: 'p95 < 200ms, 0% errors'
  },

  target: {
    users: 3000,
    duration: '30 minutes',
    expected: 'p95 < 500ms, < 0.1% errors'
  },

  stress: {
    users: 5000,
    duration: '10 minutes',
    expected: 'Find breaking point'
  },

  spike: {
    users: '0 → 5000 in 1 minute',
    duration: '15 minutes',
    expected: 'System recovers gracefully'
  }
}

// What to monitor during load test
const metrics = [
  'Response time (p50, p95, p99)',
  'Error rate',
  'CPU, memory, disk usage',
  'Database connections',
  'Queue depth',
  'Cache hit rate'
]
```

Ngân Sách Cho Tăng Trưởng

Quy tắc kinh nghiệm: Chi phí hạ tầng = 5-10% doanh thu

Ví dụ:

- Doanh thu hằng tháng: \$1M
- Chi phí hạ tầng chấp nhận được: \$50k-\$100k
- Chi phí hiện tại: \$30k
- Dư địa tăng trưởng: \$20k-\$70k

Chiến lược:

- Tối ưu trước khi mở rộng (giảm 20% chi phí)
- Lên kế hoạch tăng dần
- Dành sẵn ngân sách cho sự cố

5. Chiến Lược Chuyển Đổi - Thay Đổi Môi Trường Thực Tế Một Cách An Toàn

Những Thách Thức Khi Chuyển Đổi

Vì sao chuyển đổi lại rủi ro:

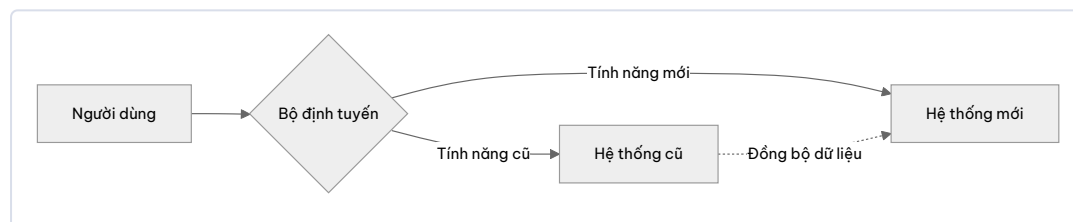
- Yêu cầu không có thời gian ngừng hoạt động
- Khối lượng dữ liệu lớn
- Không dễ hoàn tác
- Người dùng bị ảnh hưởng nếu thất bại
- Áp lực kinh doanh (doanh thu)

Kịch bản ví dụ:

- Chuyển đổi cơ sở dữ liệu (PostgreSQL → MySQL)
- Chuyển đổi đám mây (AWS → GCP)
- Thay đổi kiến trúc (nguyên khối → vi dịch vụ)
- Thay đổi mô hình dữ liệu (tiến hoá lược đồ)

Strangler Fig Pattern (mẫu cây đa bóp nghẹt)

Thay thế dần hệ thống cũ bằng hệ thống mới:



Giai đoạn 1: Xây hệ thống mới chạy song song với hệ thống cũ

Giai đoạn 2: Định tuyến lưu lượng mới sang hệ thống mới

Giai đoạn 3: Chuyển dần dữ liệu cũ

Giai đoạn 4: Chuyển hướng thêm lưu lượng sang hệ thống mới

Giai đoạn 5: Ngừng sử dụng hệ thống cũ

Thời gian: Từ vài tháng đến vài năm, không phải chuyển một lần (big-bang)

Ví Dụ Chuyển Đổi Cơ Sở Dữ Liệu (Database Migration)

```
// Phase 1: Dual Write
async function updateUser(userId, data) {
  // Write to old database
  await oldDB.users.update(userId, data)

  // Also write to new database
  try {
    await newDB.users.update(userId, data)
  } catch (error) {
    // Log but don't fail
    logger.error('New DB write failed', error)
  }
}

// Phase 2: Verify Data Consistency
async function verifyData() {
  const oldData = await oldDB.users.find(userId)
  const newData = await newDB.users.find(userId)

  if (!isEqual(oldData, newData)) {
    alert('Data inconsistency detected')
  }
}

// Phase 3: Dual Read (verify new DB)
async function getUser(userId) {
  const data = await oldDB.users.find(userId)

  // Compare with new DB
  const newData = await newDB.users.find(userId)
  if (!isEqual(data, newData)) {
    logger.warn('Data mismatch', { userId, oldData: data, newData })
  }

  return data // Still return old DB data
}

// Phase 4: Switch Reads to New DB
async function getUser(userId) {
  const data = await newDB.users.find(userId)

  // Fallback to old DB if failed
  if (!data) {
    logger.error('New DB read failed, falling back')
    return await oldDB.users.find(userId)
  }

  return data
}

// Phase 5: Stop Writing to Old DB
```



```

async function updateUser(userId, data) {
  await newDB.users.update(userId, data)
  // Old DB write removed
}

// Phase 6: Deprecate Old DB
// Remove old database entirely

```

Cờ Tính Năng (Feature Flag) Cho Chuyển Đổi

```

const featureFlags = {
  use_new_payment_service: {
    enabled: true,
    rollout: '10%', // Start with 10% traffic
    rules: [
      { segment: 'internal_users', enabled: true },
      { segment: 'beta_testers', enabled: true }
    ]
  }
}

async function processPayment(order) {
  if (featureFlags.use_new_payment_service.enabled) {
    if (shouldEnableForUser(order.userId)) {
      try {
        return await newPaymentService.process(order)
      } catch (error) {
        // Fallback to old service
        logger.error('New payment failed, fallback', error)
        return await oldPaymentService.process(order)
      }
    }
  }

  return await oldPaymentService.process(order)
}

```

Danh Sách Kiểm Tra Khi Chuyển Đổi

Danh sách kiểm tra chuyển đổi

Trước khi chuyển đổi

- [] Kiểm thử'tải hệ thống mới ở mức lưu lượng dự kiến
- [] Đã cài đặt cờ tính năng (feature flag)
- [] Đã viết tài liệu kế'hoạch hoàn tác
- [] Script kiểm tra dữ liệu đã sẵn sàng
- [] Bảng điều khiển giám sát đã chuẩn bị
- [] Nhóm trực sự cố'đã được phổ'biến
- [] Các bên liên quan đã được thông báo
- [] Đã lên lịch khung giờ bảo trì (nếu cần)

Trong khi chuyển đổi

- [] Bắt đầu với 1% lưu lượng
- [] Theo dõi sát tỷ lệ lỗi
- [] Kiểm tra tính nhất quán dữ liệu mỗi giờ
- [] Phát hành dần: 1% → 5% → 10% → 25% → 50% → 100%
- [] Mỗi bước: theo dõi 24-48 giờ trước khi sang bước tiếp
- [] Ghi lại mọi vấn đề'gặp phải

Sau khi chuyển đổi

- [] Giám sát đầy đủ trong 1 tuần
- [] Báo cáo kiểm tra dữ liệu
- [] So sánh hiệu năng (cũ và mới)
- [] Họp rút kinh nghiệm sau sự kiện (post-mortem)
- [] Ghi lại bài học kinh nghiệm
- [] Lên kế'hoạch ngừng sử'dụng hệ thống cũ

6. Lãnh Đạo Kỹ Thuật - Tạo Ảnh Hưởng Mà Không Cần Quyền Hạn

Kiến Trúc Sư Với Vai Trò Người Dẫn Dắt

Lãnh đạo kỹ thuật $\neq$ Quản lý

Bạn không có quyền:

- Giao việc
- Sa thải người khác
- Duyệt ngân sách

Nhưng bạn Có sức ảnh hưởng thông qua:

- Chuyên môn kỹ thuật
- Niềm tin đã giành được
- Giao tiếp rõ ràng
- Thành tích ra quyết định đúng

Xây Dựng Niềm Tin Qua Kết Quả Thực Tế

Làm sao để giành được niềm tin với vai trò kiến trúc sư:

Mang lại kết quả:

- Thiết kế thực sự chạy tốt trên môi trường thực tế
- Giải pháp mở rộng được khi cần
- Chuyển đổi suôn sẻ, không sự cô

Đáng tin cậy:

- Có mặt khi có sự cô
- Làm đến nơi đến chốn những gì đã cam kết
- Trả lời câu hỏi nhanh chóng

Thừa nhận sai lầm:

- "Tôi đã sai về X"
- "Cách tiếp cận tốt hơn là Y"
- Học hỏi một cách công khai

Ghi nhận công lao:

- "Ý tưởng dùng bộ nhớ đệm của Sarah đã cứu chúng ta"
- Làm nổi bật đóng góp của nhóm
- Chia sẻ thành công

Ra Quyết Định Kỹ Thuật

Khung tư duy cho các quyết định kiến trúc:

Hồ sơ quyết định kiến trúc (Architecture Decision Record - ADR)

Bối cảnh

Chúng ta cần thiết kế lại hệ thống thông báo.

Hệ thống hiện tại:

- Gửi 1 triệu thông báo/ngày
- Tăng 50% mỗi quý
- Thường xuyên bị trễ và lỗi

Quyết định

Áp dụng kiến trúc hướng sự kiện với Kafka

Các phương án đã cân nhắc

1. ****RabbitMQ + worker****

- Ưu điểm: Nhóm đã quen, đơn giản hơn
- Nhược điểm: Hạn chế về mở rộng, không phát lại (replay) được

2. ****AWS SNS/SQS****

- Ưu điểm: Dịch vụ được quản lý sẵn, tin cậy
- Nhược điểm: Phụ thuộc nhà cung cấp, tốn kém khi quy mô lớn

3. ****Kafka**** (ĐÃ CHỌN)

- Ưu điểm: Thông lượng cao, phát lại được, hệ sinh thái mạnh
- Nhược điểm: Phức tạp khi vận hành, phải mất thời gian học

Hệ quả

Tích cực:

- Xử lý được hơn 10 triệu thông báo/ngày
- Phát lại thông điệp để gỡ lỗi
- Có thể phân tích thời gian thực

Tiêu cực:

- Nhóm cần được đào tạo về Kafka
- Thêm chi phí phụ cho vận hành
- Chuyển đổi mất 3 tháng

Lộ trình

- Tháng 1: Cài đặt Kafka + đào tạo
- Tháng 2: Xây dựng dịch vụ thông báo mới
- Tháng 3: Chuyển đổi dần

Tạo Ảnh Hưởng Bằng Dữ Liệu

```
// Instead of: "We need to rewrite this!"
// Use data to make the case:

const currentMetrics = {
  deployment_frequency: '1/week',
  lead_time: '2 weeks',
  mttr: '4 hours',
  change_failure_rate: '15%'
}

const industryBenchmark = {
  elite_performers: {
    deployment_frequency: 'Multiple/day',
    lead_time: '< 1 day',
    mttr: '< 1 hour',
    change_failure_rate: '0-15%'
  }
}

const proposal = `
Problem: Our lead time is 14x slower than elite teams
Impact: Competitors ship features faster
Solution: Move to microservices + CI/CD
Expected improvement: Lead time 2 weeks → 2 days
Cost: 3 months, 4 engineers
ROI: Ship 7x faster, competitive advantage
`
```

Kỹ Năng Giao Tiếp

Giải thích chuyện kỹ thuật cho người không làm kỹ thuật:

Biệt ngữ kỹ thuật:

"Chúng ta cần triển khai horizontal pod autoscaling bằng Kubernetes HPA dựa trên custom metrics từ Prometheus để xử lý lưu lượng tăng đột biến"

Ngôn ngữ kinh doanh:

"Hiện tại, lưu lượng tăng đột biến làm sập trang web."

Điều này gây mất doanh thu.

Giải pháp: Tự động thêm máy chủ khi hệ thống bận.

Chi phí: \$5k/tháng

Lợi ích: Không còn sập, bảo vệ \$50k doanh thu mỗi giờ"

Viết tài liệu kỹ thuật rõ ràng:

Tài liệu tồi

Dùng Redis để lưu đệm. Cấu hình TTL cho phù hợp.

Tài liệu tốt

Chiến lược lưu đệm

Chúng ta dùng Redis để lưu đệm phản hồi API, giảm 80% tải cho cơ sở dữ liệu.

Khi nào nên lưu đệm

- Nên lưu đệm: Hồ sơ người dùng, danh sách sản phẩm (hiếm khi thay đổi)
- Không lưu đệm: Tổng kho thời gian thực, giá (thay đổi thường xuyên)

Hướng dẫn đặt TTL

- Hồ sơ người dùng: 1 giờ (hơi cũ một chút cũng không sao)
- Danh sách sản phẩm: 5 phút (cập nhật đều đặn)
- Kết quả tìm kiếm: 30 giây (cân bằng giữa độ mới và hiệu năng)

Ví dụ

```
```javascript
await redis.set(`user:${id}`, data, { ttl: 3600 })
```
```

Giám sát

- Tỷ lệ trùng bộ nhớ đệm: <https://dashboard/redis>
- Cảnh báo nếu tỷ lệ trùng < 80%

Review Mã Một Cách Xuất Sắc

```
// Bad review
"This is wrong. Use Redis instead."

// Good review
/*
Concern: In-memory cache won't scale across multiple instances.

Issue:
- Each instance has separate cache
- Cache miss rate will be high
- Duplicated memory usage

Suggestion:
Consider shared Redis cache:
- Single source of truth
- Better hit rate
- Scales horizontally

If Redis not possible:
- Document limitations
- Add metrics for cache hit rate
- Plan migration when needed

Happy to discuss trade-offs!
*/
```

Kèm Cặp Các Kỹ Sư

Kỹ sư mới vào nghề: "Em nên dùng MongoDB hay PostgreSQL?"

Câu trả lời tôi:

"Dùng PostgreSQL đi, nó tốt hơn."

Câu trả lời tốt:

"Câu hỏi hay đấy! Cùng nghĩ qua các đánh đổi nhé:

Dữ liệu của em trông thế nào?

- Có cấu trúc, có quan hệ? → PostgreSQL
- Phi cấu trúc, lược đồ linh hoạt? → MongoDB

Những thao tác nào?

- Truy vấn phức tạp, phép join? → PostgreSQL
- CRUD đơn giản, ghi nhanh? → MongoDB

Nhóm của em quen với cái gì?

- Có kinh nghiệm SQL? → PostgreSQL dễ hơn

Với tình huống của em (hồ sơ người dùng có quan hệ),
anh khuyên dùng PostgreSQL vì:

1. Em có dữ liệu có cấu trúc
2. Cần truy vấn join (users + orders)
3. Nhóm đã biết SQL

Nhưng cái nào cũng có thể dùng được. Em muốn làm thử nguyên mẫu cả hai không?"

7. Quy Trình Review Kiến Trúc - Ngăn Vấn Đề Từ Sớm

Khi Nào Cần Review

Khi nào bắt buộc review kiến trúc:

Luôn review:

- Dịch vụ/hệ thống mới
- Tái cấu trúc lớn
- Thay đổi lược đồ cơ sở dữ liệu
- Áp dụng công nghệ mới
- Thay đổi nhạy cảm về bảo mật
- Phụ thuộc liên nhóm

 Cần nhắc review:

- Luồng xử lý quan trọng về hiệu năng
- Thuật toán phức tạp
- Tích hợp API bên ngoài

Không cần review:

- Sửa lỗi
- Thay đổi giao diện
- Cập nhật nội dung chữ
- Tái cấu trúc nhỏ

Mẫu Review

Review kiến trúc: Thiết kế lại dịch vụ thanh toán

Người đề xuất

Sarah Chen, Trưởng nhóm Backend

Tổng quan

Thiết kế lại dịch vụ thanh toán để hỗ trợ thanh toán quốc tế

Hệ thống hiện tại

- Một nhà cung cấp thanh toán duy nhất (Stripe)
- Chỉ USD
- Xử lý đồng bộ
- ~1000 giao dịch/ngày

Hệ thống đề xuất

- Nhiều nhà cung cấp thanh toán (Stripe, PayPal, Adyen)
- Hỗ trợ đa tiền tệ
- Xử lý bất đồng bộ qua Kafka
- Dự kiến 10k giao dịch/ngày

Sơ đồ kiến trúc

[Chèn sơ đồ]

Các quyết định thiết kế chính

1. Nhiều nhà cung cấp thanh toán

Quyết định: Hỗ trợ từ 3 nhà cung cấp trở lên

Lý do: Ưu tiên theo khu vực, dự phòng

Đánh đổi: Độ phức tạp và tính linh hoạt

2. Xử lý bất đồng bộ

Quyết định: Dùng Kafka cho các sự kiện thanh toán

Lý do: Tách rời, chịu được tăng đột biến, thư lại

Đánh đổi: Nhất quán cuối cùng

3. Trừu tượng hoá nhà cung cấp

Quyết định: Ẩn nhà cung cấp sau một interface

Lý do: Dễ thêm/bớt nhà cung cấp

Đánh đổi: Thêm một lớp trừu tượng

Môi lo ngại & rủi ro

1. **Quy đổi tiền tệ**

- Rủi ro: Tỷ giá thay đổi trong lúc xử lý
- Giảm thiểu: Chốt tỷ giá lúc thanh toán

2. **Chuyển đổi dự phòng nhà cung cấp**

- Rủi ro: Nhà cung cấp chính bị sập
- Giảm thiểu: Tự động chuyển sang nhà cung cấp phụ

3. **Tính nhất quán dữ liệu**

- Rủi ro: Mất sự kiện trong Kafka

- Giảm thiểu: Gửi ít nhất một lần (at-least-once), tính lũy đẳng

Cân nhắc về quy mô

- Hiện tại: 1k giao dịch/ngày
- 1 năm tới: 50k giao dịch/ngày (gấp 50 lần)
- Thiết kế chịu được: 100k giao dịch/ngày

Bảo mật

- Duy trì tuân thủ PCI
- Không lưu dữ liệu thẻ
- Mã hoá token của nhà cung cấp
- Ghi log kiểm toán

Giám sát

- Tỷ lệ giao dịch thành công theo từng nhà cung cấp
- Độ trễ p95, p99
- Độ chính xác của quy đổi tiền tệ
- Cảnh báo thanh toán thất bại

Lộ trình

- Tuần 1-2: Xây lớp trừu tượng nhà cung cấp
- Tuần 3-4: Tích hợp các nhà cung cấp
- Tuần 5-6: Cài đặt Kafka + bộ xử lý sự kiện
- Tuần 7-8: Chuyển đổi + kiểm thử
- Tuần 9: Phát hành sản phẩm

Câu hỏi cho người review

1. Lớp trừu tượng nhà cung cấp có quá phức tạp không?
2. Kafka có quá tay so với quy mô hiện tại không?
3. Có bỏ sót kịch bản sự cố nào không?

Người review

- @architect-alice (hạ tầng)
- @architect-bob (bảo mật)
- @devops-lead (vận hành)

Hướng Dẫn Review

Dành cho người review:

Tập trung vào:

- Có bỏ sót kịch bản sự cố?
- Đã cân nhắc về` quy mô?
- Vấn đề` bảo mật?
- Độ phức tạp khi vận hành?
- Có phương án tốt hơn?

Tránh:

- Sa đà vào tiêu tiết (bikeshedding: đặt tên, định dạng)
- "Tôi sẽ làm khác" mà không có lý lẽ
- Cầu toàn ("Cái này chưa tối ưu")

🎯 Mục tiêu:

Giúp cải thiện thiết kế, không phải cản trở tiến độ

Dành cho tác giả thiết kế:

Hãy cởi mở với:

- Phản hồi mang tính phản biện
- Các cách tiếp cận khác
- Giải pháp đơn giản hơn

Tránh:

- Phản ứng phòng thủ
- "Cứ tin tôi, nó sẽ chạy"
- Gạt bỏ các mối lo ngại

🎯 Mục tiêu:

Học hỏi, cải thiện thiết kế, xây dựng đồng thuận

Tiêu Chí Phê Duyệt

Thế nào là được phê duyệt

Thiết kế có thể được phê duyệt khi:

- [] Các kịch bản sự cố đã được xác định và có biện pháp giảm thiểu
- [] Các cân nhắc về quy mô đã được giải quyết
- [] Đã review bảo mật
- [] Đã hiểu rõ tác động vận hành
- [] Đã xác định chiến lược giám sát
- [] Có kế hoạch hoàn tác
- [] Từ 2 kiến trúc sư trở lên đã phê duyệt
- [] Không còn mối lo ngại mang tính chặn

⚠ Phê duyệt có điều kiện:

- Các mối lo ngại nhỏ cần xử lý trong lúc hiện thực
- Đã lên lịch các buổi review tiếp theo

Không phê duyệt:

- Rủi ro lớn chưa được giảm thiểu
- Các mối lo ngại về quy mô chưa được giải quyết
- Vấn đề bảo mật mang tính chặn

8. Tư Duy Sau Cùng - Bạn Làm Chủ Hệ Thống

Các Nguyên Tắc Làm Chủ Môi Trường Thực Tế

1. Nghĩ Dài Hạn

Đoạn mã bạn viết hôm nay:

- Bạn sẽ phải bảo trì trong nhiều năm
- Người khác sẽ mở rộng nó
- Sẽ chạy ở quy mô gấp 10 lần
- Rồi sẽ có lúc hỏng

Hãy viết mã mà bạn tự hào được làm chủ.

2. Thấu Cảm Với Người Trực Sự Cố (Oncall)

Trước khi merge:

"Cái này có đánh thức ai đó lúc 3 giờ sáng không?"

"Cái này có gỡ lỗi được không?"

"Có hoàn tác được không?"

"Thông báo lỗi có chỉ ra được việc cần làm không?"

Nếu câu trả lời là "có lẽ" → hãy cải thiện thiết kế.

3. Tài Liệu Như Một Bức Thư Tình

Viết tài liệu cho chính bạn trong tương lai (6 tháng sau):

- Vì sao lại thiết kế thế này?
- Những đánh đổi là gì?
- Gỡ lỗi thế nào?
- Mở rộng thế nào?

Bạn sẽ thâm cảm ơn chính mình.

4. Đo Lường Mọi Thứ

"Chúng ta tin vào Chúa. Còn lại, ai cũng phải mang dữ liệu đến."

- W. Edwards Deming

Không có chỉ số = Không nhìn thấy gì = Không thể cải thiện

Thêm chỉ số trước, tối ưu sau.

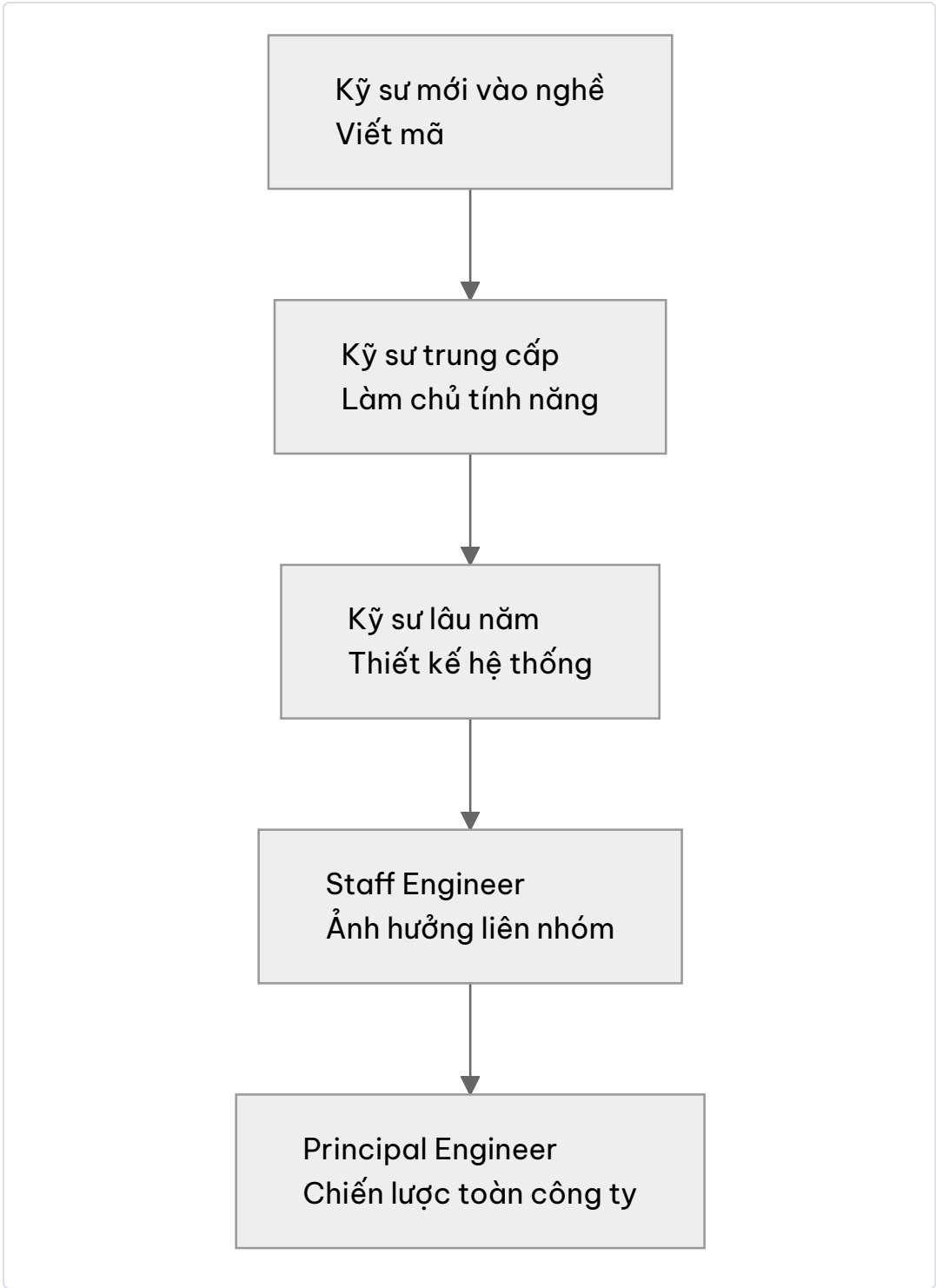
5. Tự Động Hoá Việc Lặp Lại Nhàm Chán (Toil)

Các tác vụ lặp đi lặp lại:

- Triển khai
- Hoàn tác
- Mở rộng
- Thiết lập giám sát
- Ứng phó sự cố

Tự động hoá một lần, hưởng lợi mãi mãi.

Lộ Trình Phát Triển Sự Nghiệp



Ở mỗi cấp độ:

Mới vào nghề → Trung cấp:

- Chất lượng mã
- Làm chủ tính năng
- Học thiết kế hệ thống

Trung cấp → Lâu năm:

- Thiết kế hệ thống
- Tư duy đánh đổi
- Kèm cặp người khác

Lâu năm → Staff:

- Ảnh hưởng liên nhóm
- Tư duy vận hành thực tế
- Chiến lược kỹ thuật

Staff → Principal:

- Kiến trúc toàn công ty
- Tuyển dụng & xây dựng nhóm
- Định hướng kỹ thuật

Điều Gì Làm Nên Những Kiến Trúc Sư Xuất Sắc

Kiến trúc sư trung bình:

- Biết các mẫu thiết kế
- Làm theo thực hành tốt nhất
- Thiết kế hệ thống

Kiến trúc sư xuất sắc:

- Nghĩ đến sự cô
- Đơn giản hoá sự phức tạp
- Giúp người khác làm được việc
- Làm chủ môi trường thực tế
- Giao tiếp rõ ràng
- Giành được niềm tin
- Mang lại kết quả

10 Điều Răn Của Hệ Thống Vận Hành Thực Tế

1. ****Giám sát mọi thứ**** - Bạn không thể cải thiện điều mình không đo được
2. ****Thiết kế để chịu được sự cô**** - Mọi thứ đều sẽ hỏng. Hãy lên kế hoạch cho điều đó.
3. ****Bắt đầu đơn giản**** - Chỉ thêm độ phức tạp khi cần
4. ****Tư duy theo đánh đôi**** - Không có giải pháp hoàn hảo. Hãy chọn khôn ngoan.
5. ****Ghi lại các quyết định**** - Bạn của tương lai sẽ hỏi "Tại sao?"
6. ****Tự động hoá việc lặp lại**** - Con người dễ suy nghĩ, máy móc dễ lặp lại
7. ****Bảo mật mặc định**** - Nói lỏng dễ hơn siết chặt về sau
8. ****Kiềm thu ở quy mô lớn**** - Hãy kiềm thu tài trước khi người dùng làm điều đó thay bạn
9. ****Phát hành dần dần**** - Triển khai một lần (big bang) = sự cô một lần cực lớn
10. ****Làm chủ mã của bạn**** - Bạn xây nó, bạn vận hành nó

Những Điểm Cốt Lõi

Những Chuyển Dịch Tư Duy Từ Kỹ Sư → Kiến Trúc Sư

1. Từ Mã → Hệ Thống

- Kỹ sư: "Tính năng chạy được trên máy của tôi"
- Kiến trúc sư: "Hệ thống chạy được ở quy mô lớn trên môi trường thực tế"

2. Từ Kịch Bản Suôn Sẻ → Các Kịch Bản Sự Cố

- Kỹ sư: "Nếu mọi thứ đều chạy..."
- Kiến trúc sư: "Khi mọi thứ hỏng..."

3. Từ Một Mình → Cả Nhóm

- Kỹ sư: Viết mã một mình
- Kiến trúc sư: Giúp cả nhóm thành công

4. Từ Xây Dựng → Làm Chủ

- Kỹ sư: Giao xong rồi chuyển sang việc khác
- Kiến trúc sư: Làm chủ nó trên môi trường thực tế

5. Từ Kỹ Thuật → Kinh Doanh

- Kỹ sư: "Cái này tốt hơn về mặt kỹ thuật"
- Kiến trúc sư: "Cái này mang lại giá trị kinh doanh"

Danh Sách Kiểm Tra Tư Duy Vận Hành Thực Tế

Trước khi đưa lên môi trường thực tế:

- ☐ Bán kính ảnh hưởng là bao nhiêu nếu cái này hỏng?
- ☐ Có hoàn tác được trong dưới 5 phút không?
- ☐ Đã có giám sát cho cái này chưa?
- ☐ Thông báo lỗi có chỉ ra được việc cần làm không?
- ☐ Đã kiểm thử tải ở mức gấp 2 lưu lượng dự kiến chưa?
- ☐ Có sổ tay xử lý (runbook) cho các sự cố thường gặp không?
- ☐ Kỹ sư trực sự cố có gỡ lỗi được cái này lúc 3 giờ sáng không?
- ☐ Tác động là gì nếu dịch vụ này sập?
- ☐ Có kế hoạch phát hành dần không?
- ☐ Đã cân nhắc hết các kiểu sự cố chưa?

Các Nguyên Tắc Lãnh Đạo Kỹ Thuật

Giao tiếp:

- Giải thích chuyện kỹ thuật bằng ngôn ngữ kinh doanh
- Viết tài liệu mà người khác hiểu được
- Giao tiếp nhiều hơn mức cần thiết khi có sự cố (*incidents*)

Ra quyết định:

- Dùng dữ liệu, không dùng ý kiến chủ quan
- Ghi lại các đánh đổi (*trade-offs*)
- Để nhóm cùng tham gia ra quyết định

Tạo ảnh hưởng:

- Xây dựng niềm tin bằng kết quả thực tế

- Ghi nhận công lao cho người khác, nhận trách nhiệm về mình
- Kèm cặp một cách hào phóng

Làm chủ:

- Có mặt khi có sự cố
- Làm đến nơi đến chốn những gì đã cam kết
- Nghĩ dài hạn

Lời Kết

Hành trình từ kỹ sư đến kiến trúc sư không phải là học thêm nhiều mẫu thiết kế. Mà là dám nhận lấy trách nhiệm làm chủ.

Bạn không cần biết mọi công nghệ. Bạn cần:

- Tư duy theo đánh đổi
- Thiết kế để chịu được sự cố (*failure*)
- Làm chủ môi trường thực tế
- Giúp nhóm của bạn làm được việc
- Giao tiếp rõ ràng
- Mang lại kết quả

Kiến trúc sư xuất sắc không được tạo nên trong một ngày. Họ được tôi luyện qua:

- Ứng phó các sự cố trên môi trường thực tế
- Mắc sai lầm và học hỏi
- Giành được niềm tin theo thời gian
- Đưa vào vận hành những hệ thống mở rộng được
- Giúp người khác thành công

Bạn đã sẵn sàng khi nhận ra: Hệ thống là của bạn. Hãy làm chủ nó.

Chuẩn Bị Cho Phỏng Vấn Thiết Kế Hệ Thống (System Design Interview)

Tài liệu này dành cho lập trình viên muốn chuẩn bị nghiêm túc cho **phỏng vấn thiết kế hệ thống** ở cấp độ trung cấp đến lâu năm.

Mục tiêu không phải là học thuộc "khuôn mẫu trả lời", mà là xây dựng tư duy kiến trúc có cấu trúc, giải thích được sự đánh đổi (*trade-off*), và giao tiếp rõ ràng dưới áp lực phỏng vấn.

Bối cảnh lộ trình: nội dung này phù hợp nhất khi bạn đã đi qua phần nền tảng từ Phần 0 đến Phần 5 và đang bước vào tư duy của **Phần 6 — Làm Chủ Thiết Kế Hệ Thống**.

1) Phỏng Vấn Thiết Kế Hệ Thống Là Gì?

Phỏng vấn thiết kế hệ thống kiểm tra cách bạn tư duy khi đứng trước một bài toán hệ thống mở, thiếu dữ kiện và có nhiều ràng buộc thực tế.

Mục Đích Chính

- Đánh giá khả năng **thiết kế hệ thống** chứ không chỉ viết mã.
- Đánh giá cách bạn ra quyết định trong điều kiện không hoàn hảo.
- Đánh giá khả năng cân bằng giữa sản phẩm, kỹ thuật và vận hành.

Khác Gì Với Phỏng Vấn Lập Trình (Coding Interview)?

- Phỏng vấn lập trình: tập trung vào thuật toán, tính đúng đắn (*correctness*) và chi tiết cài đặt (*implementation*).
- Phỏng vấn thiết kế hệ thống: tập trung vào kiến trúc tổng thể, quy mô (*scale*), độ tin cậy (*reliability*), sự đánh đổi và khả năng giao tiếp.

Trọng Tâm

Bạn được kỳ vọng thể hiện tư duy của một kỹ sư có thể thiết kế hệ thống cho môi trường vận hành thực tế (*production*), chứ không chỉ là người hoàn thành một hàm.

2) Người Phỏng Vấn (*Interviewer*) Thực Sự Đánh Giá Điều Gì?

1. Làm Rõ Yêu Cầu (*Requirement Clarification*)

- Bạn có biết hỏi đúng câu hỏi để làm rõ phạm vi (*scope*) không?
- Bạn có phân biệt được yêu cầu chức năng (*functional requirement*) và yêu cầu phi chức năng (*non-functional requirement*) không?

2. Lập Luận Về Đánh Đổi

- Bạn có giải thích được vì sao chọn phương án A thay vì B?
- Bạn có nhìn thấy chi phí vận hành và độ phức tạp đi kèm không?

3. Tư Duy Về Khả Năng Mở Rộng (*Scalability*)

- Bạn có biết đặt giả định về QPS, kích thước dữ liệu, tốc độ tăng trưởng không?
- Bạn có biết hệ thống sẽ vỡ ở đâu khi quy mô tăng?

4. Giao Tiếp Có Cấu Trúc

- Câu trả lời có mạch lạc, theo thứ tự, dễ theo dõi không?
- Bạn có chủ động dẫn dắt buổi trao đổi không?

5. Ý Thức Về Vận Hành Thực Tế

- Bạn có nghĩ tới độ tin cậy, khả năng quan sát (*observability*), xử lý sự cố (*failure handling*), hoàn tác (*rollback*)?
 - Bạn có biết khi nào nên giữ đơn giản và khi nào phải nâng cấp kiến trúc?
-

3) Quy Trình Phỏng Vấn Chuẩn (Từng Bước)

Bước 1 – Làm Rõ Yêu Cầu (*Clarify Requirements*)

- Xác định rõ bài toán cốt lõi.
- Chốt các tình huống sử dụng (*use case*) ưu tiên.
- Làm rõ các giới hạn: độ trễ (*latency*), tính nhất quán (*consistency*), tính sẵn sàng (*availability*), chi phí (*cost*).

Bước 2 – Xác Định Quy Mô & Ràng Buộc (*Constraints*)

- Ước lượng số người dùng, QPS, dung lượng lưu trữ (*storage*), băng thông (*bandwidth*).
- Xác định các "điểm nóng" cần ưu tiên xử lý trước.

Bước 3 – Kiến Trúc Tổng Quan (*High-level Architecture*)

- Vẽ kiến trúc ở mức các khối lớn.
- Xác định các dịch vụ (*service*) chính, luồng dữ liệu (*data flow*) và ranh giới trách nhiệm.

Bước 4 – Mô Hình Dữ Liệu (*Data Model*) & API

- Thiết kế các thực thể (*entity*) chính và mẫu truy cập (*access pattern*) quan trọng.
- Đề xuất các API cốt lõi để kiểm chứng tính khả thi.

Bước 5 – Đào Sâu (*Deep Dive*) Vào Thành Phần Quan Trọng

- Chọn 1-2 điểm quan trọng để đào sâu: bộ nhớ đệm (*cache*), hàng đợi (*queue*), phân vùng (*partition*) CSDL, mô hình nhất quán (*consistency model*)...

Bước 6 – Phân Tích Điểm Nghẽn (*Bottleneck*) & Sự Cố (*Failure*)

- Chỉ ra điểm lỗi duy nhất (*SPOF*), điểm nóng (*hotspot*), các kiểu sự cố (*failure mode*).
- Nêu chiến lược giảm rủi ro và phương án dự phòng.

Bước 7 – Đánh Đổi & Tiến Hoá

- Tóm tắt điểm mạnh/điểm yếu của phương án.

- Nêu lộ trình (*roadmap*) nâng cấp hệ thống theo từng giai đoạn tăng trưởng.
-

4) Khung Tư Duy (*Framework*) Gợi Ý: SNAKE

Một khung tư duy ngắn gọn, dễ nhớ để bạn không bị "trôi" trong buổi phỏng vấn.

S – Scope (Phạm Vi)

- Bài toán là gì?
- Tình huống sử dụng nào được ưu tiên?
- Điều gì không nằm trong phạm vi hôm nay?

N – Numbers (Các Con Số)

- Ước lượng QPS, kích thước dữ liệu gửi kèm (*payload*), dung lượng lưu trữ, lưu lượng cao điểm (*peak traffic*).
- Đặt các con số gần đúng để định hướng kiến trúc.

A – API

- Đề xuất các API cốt lõi để chốt hợp đồng giao tiếp (*contract*) giữa máy khách (*client*) và máy chủ (*server*).

K – Key Components (Các Thành Phần Chính)

- Dựng kiến trúc chính: bộ cân bằng tải (*LB*), ứng dụng, bộ nhớ đệm, CSDL, hàng đợi, tiến trình xử lý (*worker*), CDN...

E – Explore Deep Dive (Đào Sâu)

- Đào sâu 1-2 vấn đề có rủi ro cao nhất.
- Nói rõ sự đánh đổi và kế hoạch mở rộng tiếp theo.

Ví Dụ Nhanh (Thiết Kế Dịch Vụ Rút Gọn URL (*URL Shortener*))

- **Phạm vi:** tạo URL rút gọn, chuyển hướng (*redirect*) nhanh, thống kê lượt nhấp cơ bản.

- **Các con số:** 50 nghìn yêu cầu/giây cho đọc, 5 nghìn yêu cầu/giây cho ghi, lưu dữ liệu 5 năm.
 - **API:** POST /shorten, GET /{code}.
 - **Các thành phần chính:** dịch vụ API + kho khoá-giá trị (KV store) + bộ nhớ đệm + đường ống phân tích (analytics pipeline).
 - **Đào sâu:** chiến lược xử lý trùng mã (collision), khoá nóng (hot key), nhất quán cuối cùng (eventual consistency) cho phân phân tích.
-

5) Các Câu Hỏi Phỏng Vấn Phổ Biến Và Điều Được Kiểm Tra

Thiết Kế Twitter / X

Kiểm tra: chiến lược phân phát (fan-out), cách sinh bảng tin (feed), đánh đổi giữa đọc và ghi, tính nhất quán của dòng thời gian (timeline).

Thiết Kế YouTube

Kiểm tra: đường ống tải lên (upload pipeline), chuyển mã video (transcoding), chiến lược CDN, kho siêu dữ liệu (metadata store), ranh giới của hệ thống gợi ý (recommendation).

Thiết Kế Hệ Thống Nhắn Tin (Chat System)

Kiểm tra: truyền tin thời gian thực (real-time delivery), quản lý kết nối (connection management), thứ tự (ordering) tin nhắn, đồng bộ ngoại tuyến (offline sync), trạng thái trực tuyến (presence).

Thiết Kế Dịch Vụ Rút Gọn URL

Kiểm tra: sinh ID (ID generation), độ trễ khi chuyển hướng, xử lý khoá nóng, tối ưu cho hệ thống thiên về đọc (read-heavy).

Thiết Kế Hệ Thống Thông Báo (Notification System)

Kiểm tra: gửi tin qua nhiều kênh (multi-channel), chính sách thử lại (retry policy), khử trùng lặp (deduplication), giới hạn tần suất (rate limiting), khả năng quan sát.

6) Cách Chuẩn Bị Hiệu Quả

Theo Lộ Trình Học

- Củng cố nền tảng (Phần 0-2).
- Nâng độ sâu về hệ thống phân tán (*distributed*) và quy mô (Phần 3-4).
- Luyện các mẫu (*pattern*) thực tế và cách kể lại quá trình ra quyết định (*decision narrative*) (Phần 5-6).

Chiến Lược Luyện Tập

- 3-4 đề/tuần, mỗi đề 45-60 phút.
- 1 buổi/tuần chỉ để ôn lại và sửa cách trình bày.
- Luyện cả độ rộng (*breadth*) (nhiều loại đề) lẫn độ sâu (*depth*) (đào sâu một đề).

Luyện Vẽ Sơ Đồ Trên Bảng Trắng (*Whiteboard*)

- Vẽ nhanh kiến trúc ở mức thành phần trước.
- Giảm chi tiết thừa, làm rõ luồng dữ liệu.

Luyện Giao Tiếp

- Nói thành tiếng khi giải đề.
- Ghi âm 1-2 lần/tuần để tự phát hiện chỗ trình bày còn rối.

Phỏng Vấn Thử (*Mock Interviews*)

- Tập với bạn học hoặc người hướng dẫn (*mentor*).
- Sau mỗi buổi, chốt 3 điểm mạnh và 3 điểm cần sửa.

Học Từ Hệ Thống Thật

- Đọc các blog kỹ thuật (*engineering blog*) (Uber, Netflix, Cloudflare, Meta...).
 - Tóm tắt lại theo khung: bài toán -> giải pháp -> đánh đổi -> bài học.
-

7) Các Lỗi Rất Thường Gặp

- Nhảy vào giải pháp quá sớm khi chưa chốt yêu cầu.
- Không hỏi số liệu về quy mô nên kiến trúc thiếu cơ sở.
- Nói quá nhiều thành phần nhưng không giải thích sự đánh đổi.
- Thiết kế thừa (*Over-engineering*) ngay từ đầu.
- Không phân biệt "bắt buộc phải có bây giờ" và "tiến hoá sau này".
- Bỏ qua độ tin cậy, xử lý sự cố, khả năng quan sát.

Danh sách kiểm tra chống lỗi: trước khi kết thúc, luôn tự hỏi "Nếu lưu lượng (*traffic*) tăng gấp 10 lần, hệ thống sẽ vỡ ở đâu trước?"

8) Lộ Trình Chuẩn Bị Đề Xuất (8 Tuần)

Tuần 1-2

- Ôn khung tư duy trả lời (SNAKE).
- Luyện 4 đề cơ bản: rút gọn URL, bộ giới hạn tần suất (*rate limiter*), hệ thống thông báo, hệ thống nhắn tin cơ bản.

Tuần 3-4

- Tăng độ sâu về hệ thống phân tán (*distributed systems*): tính nhất quán, phân vùng, hàng đợi, vô hiệu hoá bộ nhớ đệm (*cache invalidation*).
- Luyện 4 đề trung bình: bảng tin, lưu trữ tệp (*file storage*), tìm kiếm cơ bản, đường ống phân tích.

Tuần 5-6

- Tập các đề từ trung bình đến khó: YouTube, tổng quan hệ thống điều phối gọi xe, tổng quan hệ thống phân phối quảng cáo.
- Tập phân tích điểm nghẽn/sự cố kỹ hơn.

Tuần 7

- Phỏng vấn thử (*Mock interview*) cường độ cao: 3-4 buổi/tuần.
- Chấm theo thang tiêu chí (*rubric*): độ rõ ràng, cấu trúc, sự đánh đổi, độ sâu.

Tuần 8

- Ôn tập lần cuối: các lỗi cá nhân hay mắc nhất, các mẫu quan trọng nhất cần nhớ, các câu hỏi làm rõ hay dùng nhất.
 - Giảm cường độ trước buổi phỏng vấn, tập trung vào sự rõ ràng và bình tĩnh.
-

9) Mẹo Giao Tiếp Trong Phỏng Vấn

Nghĩ Thành Lời (*Think Aloud*) Có Cấu Trúc

Nói rõ bạn đang ở bước nào: yêu cầu -> các con số -> kiến trúc -> đào sâu -> đánh đổi.

Dẫn Dắt Cuộc Trao Đổi

- Chủ động đề xuất giả định khi thiếu dữ liệu.
- Thường xuyên dừng lại để xác nhận: "Anh/chị muốn em đào sâu phần nào tiếp theo?"

Đặt Câu Hỏi Tốt

- Lưu lượng cao điểm hay trung bình?
- Ưu tiên độ trễ hay tính nhất quán?
- Làm MVP trước hay tính đến quy mô dài hạn trước?

Khi Gặp Bài Toán Chưa Từng Làm

- Bình tĩnh quay về nguyên lý gốc.
 - Chia nhỏ bài toán.
 - Chọn phiên bản nền (*baseline*) đơn giản trước rồi mở rộng dần.
-

10) Bắt Đầu Ngay

Nếu bạn muốn chuẩn bị cho phỏng vấn thiết kế hệ thống một cách bài bản:

- Hoàn thành các phần nền tảng nếu còn thiếu.
- Vào **Phần 6 — Làm Chủ Thiết Kế Hệ Thống** để luyện khung tư duy cho phỏng vấn.
- Lên lịch phỏng vấn thử định kỳ ngay từ tuần này.

Bảng thuật ngữ Anh – Việt

Các thuật ngữ chuyên ngành đã được Việt hoá trong sách. Lần đầu xuất hiện trong mỗi bài, thuật ngữ gốc được ghi kèm trong ngoặc.

| Tiếng Anh | Tiếng Việt |
|----------------------------|--|
| a/b testing | kiểm thử a/b, thử nghiệm a/b |
| access logs | log truy cập |
| access pattern | mẫu truy cập |
| access patterns | mẫu truy cập |
| acknowledge | xác nhận |
| active connections | kết nối đang hoạt động |
| active invalidation | vô hiệu hoá chủ động |
| active-active multi-region | đa vùng chủ động – chủ động |
| aggregation | tổng hợp |
| aggregations | tổng hợp |
| agreement mechanism | cơ chế thỏa thuận |
| alert | cảnh báo |
| alert fatigue | mệt mỏi vì cảnh báo |
| alert strategy | chiến lược cảnh báo |
| alerting | cảnh báo |
| alternatives | các phương án thay thế, phương án thay thế |
| amdahl's law | định luật amdahl |
| analytics pipeline | đường ống phân tích dữ liệu, đường ống phân tích |
| anti-pattern | phản mẫu |
| anti-patterns | phản mẫu |
| api calls | lời gọi api |

| Tiếng Anh | Tiếng Việt |
|---|-----------------------------|
| <i>append-only</i> | chỉ ghi nối thêm |
| <i>application cache</i> | bộ nhớ đệm ứng dụng |
| <i>application layer</i> | tầng ứng dụng |
| <i>application performance monitoring</i> | giám sát hiệu năng ứng dụng |
| <i>application server</i> | máy chủ ứng dụng |
| <i>application servers</i> | máy chủ ứng dụng |
| <i>application-level</i> | tầng ứng dụng |
| <i>application-level merge</i> | hợp nhất ở tầng ứng dụng |
| <i>approach</i> | cách tiếp cận |
| <i>approaches</i> | cách tiếp cận |
| <i>architect</i> | kiến trúc sư |
| <i>architect mindset</i> | tư duy kiến trúc sư |
| <i>architects</i> | kiến trúc sư |
| <i>architecture</i> | kiến trúc |
| <i>architecture decision making</i> | ra quyết định kiến trúc |
| <i>architecture evolution</i> | tiến hoá kiến trúc |
| <i>architecture mindset</i> | tư duy kiến trúc |
| <i>architecture patterns</i> | mẫu kiến trúc |
| <i>architecture review</i> | đánh giá kiến trúc |
| <i>architecture reviews</i> | đánh giá kiến trúc |
| <i>architectures</i> | kiến trúc |
| <i>archival policies</i> | chính sách lưu trữ dài hạn |
| <i>archiving</i> | lưu trữ lâu dài |
| <i>assumption</i> | giả định |
| <i>assumptions</i> | giả định |
| <i>async</i> | bất đồng bộ |

| Tiếng Anh | Tiếng Việt |
|-------------------------------|--|
| async jobs | tác vụ bất đồng bộ |
| async patterns | mẫu bất đồng bộ |
| async pipeline | đường ống bất đồng bộ |
| async processing | xử lý bất đồng bộ |
| async replication | nhân bản bất đồng bộ |
| asynchronous | bất đồng bộ |
| asynchronous communication | giao tiếp bất đồng bộ |
| asynchronous processing | xử lý bất đồng bộ |
| at-least-once | ít nhất một lần, phân phối ít nhất một lần |
| at-least-once delivery | phân phối ít nhất một lần |
| at-most-once | tối đa một lần |
| at-most-once delivery | phân phối tối đa một lần |
| atomic operations | thao tác nguyên tử |
| atomicity | tính nguyên tử |
| audit trail | nhật ký kiểm toán |
| auth | xác thực |
| authentication | xác thực |
| authorization | phân quyền |
| auto-increment | tự tăng |
| auto-increment ids | id tự tăng |
| auto-scaling | tự động mở rộng |
| auto-scaling policies | chính sách tự động mở rộng |
| availability | tính sẵn sàng, sẵn sàng |
| back-of-envelope | ước tính nhanh |
| back-of-envelope calculations | ước tính nhanh |
| backend | phía máy chủ |

| Tiếng Anh | Tiếng Việt |
|---------------------------|---------------------------------------|
| backend server | máy chủ phía sau |
| backend servers | máy chủ phía sau |
| backends | máy chủ phía sau |
| background jobs | tác vụ nền |
| background worker | tiến trình xử lý nền |
| background workers | tiến trình xử lý nền |
| backpressure | áp lực ngược |
| bandwidth | băng thông |
| baseline | phiên bản nền |
| baseline test | kiểm thử cơ sở |
| batch | theo lô |
| batch loading | tải theo lô |
| batch processing | xử lý theo lô |
| best practice | thực hành tốt nhất |
| best practices | thực hành tốt nhất |
| big bang rewrite | viết lại toàn bộ một lần |
| binary log shipping | chuyển binary log |
| blast radius | bán kính ảnh hưởng, phạm vi ảnh hưởng |
| blocking | chặn |
| blocks | chặn |
| blue-green deployment | triển khai blue-green |
| bottleneck | điểm nghẽn |
| bottleneck identification | xác định điểm nghẽn |
| bottleneck thinking | tư duy điểm nghẽn |
| bottlenecks | điểm nghẽn |
| bottom-up | từ dưới lên |

| Tiếng Anh | Tiếng Việt |
|----------------------|-------------------------|
| breadth | độ rộng |
| breaking point | điểm gãy |
| browser cache | bộ nhớ đệm trình duyệt |
| brute force | vét cạn |
| bug | lỗi |
| bugs | lỗi |
| building block | khối xây dựng |
| building blocks | khối xây dựng |
| bulkhead | vách ngăn |
| bulkhead pattern | mẫu vách ngăn |
| bulkheads | vách ngăn |
| bundle | gói mã |
| burn rate | tốc độ tiêu hao |
| burst traffic | lưu lượng bùng nổ |
| bursts | bùng nổ |
| bursty | đợt bùng nổ |
| business logic | logic nghiệp vụ |
| business logic layer | tầng nghiệp vụ |
| business rules | quy tắc nghiệp vụ |
| cache | bộ nhớ đệm, lưu đệm |
| cache busting | phá bộ nhớ đệm |
| cache eviction | loại bỏ khỏi bộ nhớ đệm |
| cache headers | tiêu đề bộ nhớ đệm |
| cache hit | trúng bộ nhớ đệm |
| cache hit rate | tỷ lệ trúng bộ nhớ đệm |
| cache hit rates | tỷ lệ trúng bộ nhớ đệm |

| Tiếng Anh | Tiếng Việt |
|--------------------------|--|
| cache invalidation | vô hiệu hoá bộ nhớ đệm |
| cache key | khoá bộ nhớ đệm |
| cache keys | khoá bộ nhớ đệm |
| cache layer | lớp bộ nhớ đệm, tầng bộ nhớ đệm |
| cache miss | trượt bộ nhớ đệm |
| cache miss rate | tỷ lệ trượt bộ nhớ đệm |
| cache misses | trượt bộ nhớ đệm |
| cache patterns | mẫu lưu đệm |
| cache stampede | giảm đập bộ nhớ đệm |
| cache strategies | chiến lược lưu đệm |
| cache strategy | chiến lược lưu đệm |
| cache versioning | đánh phiên bản bộ nhớ đệm |
| cache warming | làm ấm bộ nhớ đệm, làm nóng bộ nhớ đệm |
| cache-aside | cache-aside |
| cache-first | ưu tiên bộ nhớ đệm |
| cache-first architecture | kiến trúc ưu tiên bộ nhớ đệm |
| caching | lưu đệm |
| caching layers | lớp bộ nhớ đệm |
| caching strategies | chiến lược lưu đệm |
| caching strategy | chiến lược lưu đệm |
| calculation framework | khung tính toán |
| canary deployments | triển khai canary |
| canary release | phát hành canary |
| candidate | ứng viên |
| candidates | ứng viên |
| cap theorem | định lý cap |

| Tiếng Anh | Tiếng Việt |
|----------------------|----------------------------------|
| capacity | năng lực, năng lực xử lý |
| capacity estimation | ước lượng năng lực |
| capacity planning | hoạch định năng lực |
| cascade failures | lỗi dây chuyền |
| cascading failure | sự cố dây chuyền |
| cascading failures | sự cố dây chuyền, lỗi dây chuyền |
| cascading timeout | hết thời gian chờ dây chuyền |
| case studies | nghiên cứu tình huống |
| case study | nghiên cứu tình huống |
| causal | quan hệ nhân quả |
| causal dependencies | phụ thuộc nhân quả |
| causal events | sự kiện có quan hệ nhân quả |
| causal relationship | quan hệ nhân quả |
| causal relationships | quan hệ nhân quả |
| causality | quan hệ nhân quả |
| celebrity problem | vấn đề người nổi tiếng |
| chain | chuỗi |
| chat system | hệ thống nhắn tin |
| checklist | danh sách kiểm tra |
| choreography | phối hợp phân tán |
| chunking | chia khối |
| circuit breaker | bộ ngắt mạch |
| circuit breakers | bộ ngắt mạch |
| clarify requirements | làm rõ yêu cầu |
| clean architecture | kiến trúc sạch |
| clean code | mã sạch |

| Tiếng Anh | Tiếng Việt |
|------------------------|-----------------------------------|
| client | máy khách |
| clients | máy khách |
| clock | đồng hồ |
| clock drift | độ trôi đồng hồ |
| clock problem | bài toán đồng hồ |
| clock skew | lệch đồng hồ |
| clocks | đồng hồ |
| cloud-native | môi trường đám mây gốc |
| cluster | cụm |
| code splitting | chia nhỏ mã |
| code-centric thinking | tư duy lấy mã nguồn làm trung tâm |
| code-first | lấy mã nguồn làm trung tâm |
| code-first thinking | tư duy ưu tiên viết mã |
| codebase | mã nguồn |
| coding interview | phỏng vấn lập trình |
| cold data | dữ liệu lạnh |
| cold start | khởi động nguội |
| cold storage | lưu trữ lạnh |
| collision | xung đột, trùng mã |
| collisions | xung đột |
| commit | cam kết |
| committed | commit |
| communication | giao tiếp |
| communication pattern | mẫu giao tiếp |
| communication patterns | mẫu giao tiếp, các mẫu giao tiếp |
| compensable | bù trừ |

| Tiếng Anh | Tiếng Việt |
|---------------------------|----------------------------------|
| compensated | bù trừ |
| compensating transactions | giao dịch bù trừ |
| compensation | bù trừ |
| compensations | bù trừ |
| complexity | độ phức tạp |
| component | thành phần |
| component interaction | tương tác giữa các thành phần |
| component interactions | tương tác giữa các thành phần |
| components | thành phần |
| composite index | chỉ mục kết hợp |
| compute | tính toán |
| concurrency | tính đồng thời, mức độ đồng thời |
| concurrency model | mô hình xử lý đồng thời |
| concurrent | đồng thời |
| concurrent events | sự kiện đồng thời |
| concurrent transactions | giao dịch đồng thời |
| conditional requests | yêu cầu có điều kiện |
| configuration management | quản lý cấu hình |
| conflict | xung đột |
| conflict resolution | giải quyết xung đột |
| conflicting | xung đột |
| conflicts | xung đột |
| connection management | quản lý kết nối |
| connection pool | nhóm kết nối |
| connection pooling | gộp kết nối, gom nhóm kết nối |
| connection pools | nhóm kết nối |

| Tiếng Anh | Tiếng Việt |
|-------------------------------|------------------------------|
| consensus | đồng thuận |
| consensus algorithm | thuật toán đồng thuận |
| consensus algorithms | thuật toán đồng thuận |
| consistency | tính nhất quán, nhất quán |
| consistency guarantee | bảo đảm nhất quán |
| consistency model | mô hình nhất quán |
| consistency models | mô hình nhất quán |
| consistent | nhất quán |
| consistent hashing | băm nhất quán |
| constraint | ràng buộc |
| constraint analysis framework | khung phân tích ràng buộc |
| constraints | ràng buộc |
| consume | tiêu thụ |
| consumer | bên tiêu thụ |
| consumers | bên tiêu thụ |
| content delivery network | mạng phân phối nội dung |
| content delivery system | hệ thống phân phối nội dung |
| content hash | mã băm nội dung |
| content-based routing | định tuyến dựa trên nội dung |
| context | bối cảnh |
| contract | hợp đồng giao tiếp |
| conversion rate | tỷ lệ chuyển đổi |
| coordinate | điều phối |
| coordination | phối hợp, sự phối hợp |
| coordinator | bộ điều phối |
| correctness | tính đúng đắn |

| Tiếng Anh | Tiếng Việt |
|--------------------------|-----------------------------|
| correlation ids | id tương quan |
| cost | chi phí |
| counter | bộ đếm |
| counters | bộ đếm |
| coupled | liên kết |
| couples | liên kết chặt |
| coupling | liên kết |
| covering index | chỉ mục bao phủ |
| critical path | đường găng |
| critical section | vùng găng |
| cross-shard queries | truy vấn xuyên mảnh |
| cross-shard transactions | giao dịch xuyên mảnh |
| crystal oscillator | bộ dao động thạch anh |
| cursor-based | phân trang dựa trên con trỏ |
| cursor-based pagination | phân trang dựa trên con trỏ |
| custom headers | tiêu đề tùy biến |
| dashboard | bảng điều khiển |
| dashboards | bảng điều khiển |
| data aggregation | tổng hợp dữ liệu |
| data consistency | tính nhất quán dữ liệu |
| data flow | luồng dữ liệu, dữ liệu chảy |
| data hot | dữ liệu "nóng" |
| data integrity | toàn vẹn dữ liệu |
| data layer | tầng dữ liệu |
| data lifecycle | vòng đời dữ liệu |
| data model | mô hình dữ liệu |

| Tiếng Anh | Tiếng Việt |
|-----------------------|--------------------------------|
| data payload | dữ liệu gửi kèm |
| data pipelines | đường ống dữ liệu |
| data reconciliation | đối soát dữ liệu |
| data residency | nơi lưu trữ dữ liệu |
| data retention policy | chính sách lưu giữ dữ liệu |
| database | cơ sở dữ liệu |
| database capacity | dung lượng cơ sở dữ liệu |
| database migration | chuyển đổi cơ sở dữ liệu |
| database queries | truy vấn cơ sở dữ liệu |
| database scaling | mở rộng cơ sở dữ liệu |
| database schema | lược đồ cơ sở dữ liệu |
| database servers | máy chủ cơ sở dữ liệu |
| database sharding | phân mảnh dữ liệu |
| databases | cơ sở dữ liệu |
| datacenter | trung tâm dữ liệu |
| datacenters | trung tâm dữ liệu |
| db | cơ sở dữ liệu |
| dead letter queue | hàng đợi thư chết |
| dead letter queues | hàng đợi thư chết |
| death spiral | vòng xoáy tử thần |
| debug | gỡ lỗi |
| debugging | gỡ lỗi |
| decision framework | khung ra quyết định |
| decision narrative | kể lại quá trình ra quyết định |
| decision tree | cây quyết định |
| decouple | tách rời |

| Tiếng Anh | Tiếng Việt |
|---------------------|---------------------------------|
| decoupled | tách rời |
| decouples | tách rời |
| decoupling | tách rời |
| decoupling services | tách rời dịch vụ |
| deduplication | khử trùng lặp |
| deep dive | đào sâu |
| defense framework | khung bảo vệ |
| defense in depth | phòng thủ nhiều lớp |
| degradation | suy giảm có kiểm soát |
| delivery guarantees | bảo đảm phân phối |
| delta sync | đồng bộ phần chênh lệch |
| denormalization | phi chuẩn hoá |
| denormalize | phi chuẩn hoá |
| denormalized | phi chuẩn hoá |
| dependencies | phụ thuộc, thành phần phụ thuộc |
| dependency | phụ thuộc |
| deploy | triển khai |
| deployment | triển khai |
| deployments | triển khai |
| depth | độ sâu |
| design patterns | mẫu thiết kế |
| design system | thiết kế hệ thống |
| deterministic | tắt định |
| developer | lập trình viên |
| developers | lập trình viên |
| development | môi trường phát triển |

| Tiếng Anh | Tiếng Việt |
|-------------------------------|---|
| diminishing returns | lợi ích giảm dần |
| direct lookup | tra cứu trực tiếp |
| directed acyclic graph | đồ thị có hướng không chu trình |
| directives | chỉ thị |
| distributed | phân tán, hệ thống phân tán |
| distributed agreement | thỏa thuận phân tán, hệ thống phân tán, thỏa thuận phân tán |
| distributed agreement problem | thỏa thuận phân tán |
| distributed cache | bộ nhớ đệm phân tán |
| distributed coordination | phối hợp phân tán |
| distributed database | cơ sở dữ liệu phân tán |
| distributed databases | cơ sở dữ liệu phân tán |
| distributed id generation | sinh id phân tán |
| distributed lock | khóa phân tán |
| distributed monolith | hệ thống nguyên khối phân tán |
| distributed patterns | mẫu phân tán |
| distributed rate limiting | giới hạn tần suất phân tán |
| distributed reasoning | tư duy phân tán |
| distributed sessions | phiên phân tán |
| distributed system | hệ thống phân tán, hệ phân tán |
| distributed systems | hệ thống phân tán |
| distributed thinking | tư duy phân tán |
| distributed tracing | truy vết phân tán |
| distributed transaction | giao dịch phân tán |
| distributed transactions | giao dịch phân tán |
| dns prefetch | tải trước dns |
| domain | lĩnh vực, miền nghiệp vụ |

| Tiếng Anh | Tiếng Việt |
|-------------------------|--|
| domain logic | logic nghiệp vụ |
| domains | miền nghiệp vụ |
| dominant access pattern | mẫu truy cập chủ đạo |
| downstream | hạ nguồn |
| downtime | thời gian ngừng hoạt động, ngừng hoạt động |
| drift | độ trôi |
| drift rate | tốc độ trôi |
| durability | độ bền dữ liệu |
| dynamic content | nội dung động |
| dynamic threshold | ngưỡng động |
| e-commerce | thương mại điện tử |
| eager loading | nạp sớm, tải sớm |
| edge | điểm biên |
| edge computing | điện toán biên |
| edge locations | vị trí biên, điểm biên |
| edge-first | ưu tiên điểm biên |
| election process | quá trình bầu chọn |
| election timeout | thời gian chờ bầu chọn |
| encode | mã hoá |
| encoding | mã hoá |
| encoding pipeline | quy trình mã hoá |
| end-to-end | từ đầu đến cuối |
| end-to-end encryption | mã hoá đầu cuối |
| endpoint | điểm cuối |
| endpoints | điểm cuối |
| endurance test | kiểm thử độ bền |

| Tiếng Anh | Tiếng Việt |
|------------------------------|--|
| engagement | mức độ tương tác |
| engineer | kỹ sư |
| engineer mindset | tư duy kỹ sư |
| engineering blog | blog kỹ thuật |
| engineers | kỹ sư |
| entity | thực thể |
| entity-based sharding | phân mảnh theo thực thể |
| error budget | ngân sách lỗi |
| error budgets | ngân sách lỗi |
| error rate | tỷ lệ lỗi |
| estimate | ước lượng |
| estimate capacity | ước lượng năng lực |
| estimates | ước lượng |
| estimation | ước lượng |
| estimations | ước lượng |
| event | sự kiện |
| event notification | thông báo sự kiện |
| event ordering | thứ tự sự kiện |
| event sourcing | lưu trữ dựa trên sự kiện, event sourcing, lưu trữ theo sự kiện |
| event stream | luồng sự kiện |
| event streams | luồng sự kiện |
| event-carried state transfer | sự kiện mang theo trạng thái |
| event-driven | hướng sự kiện |
| event-driven architecture | kiến trúc hướng sự kiện |
| event-loop | vòng lặp sự kiện |
| events | sự kiện |

| Tiếng Anh | Tiếng Việt |
|-----------------------|--------------------------------|
| eventual | nhất quán cuối cùng |
| eventual consistency | nhất quán cuối cùng |
| eventually consistent | nhất quán cuối cùng |
| eviction | loại bỏ |
| exactly-once | đúng một lần |
| exactly-once delivery | phân phối đúng một lần |
| exceptions | ngoại lệ |
| execution time | thời gian thực thi |
| exponential backoff | lùi thời gian theo cấp số nhân |
| external dependencies | phụ thuộc bên ngoài |
| fail | sự cố, gặp sự cố |
| fail fast | thất bại nhanh |
| failover | chuyển đổi dự phòng |
| failure | sự cố |
| failure awareness | ý thức về sự cố |
| failure handling | xử lý sự cố |
| failure mode | kiểu sự cố, các kiểu sự cố |
| failure model | mô hình sự cố |
| failure models | mô hình sự cố |
| failure modes | các kiểu sự cố |
| failure scenarios | kịch bản sự cố |
| failure-first | đặt sự cố lên đầu |
| failures | sự cố |
| fallback | phương án dự phòng, dự phòng |
| fallbacks | dự phòng |
| fan-out | phân phát |

| Tiếng Anh | Tiếng Việt |
|-------------------------|---------------------------------------|
| <i>fan-out/fan-in</i> | phân tán – gom về |
| <i>fanout</i> | phân phát |
| <i>fanout on read</i> | phân phát khi đọc |
| <i>fanout on write</i> | phân phát khi ghi |
| <i>fanout service</i> | dịch vụ phân phát |
| <i>fanout-on-read</i> | phân phát khi đọc |
| <i>fanout-on-write</i> | phân phát khi ghi |
| <i>fault injection</i> | tiêm lỗi |
| <i>fault isolation</i> | cô lập lỗi |
| <i>fault tolerance</i> | khả năng chịu lỗi |
| <i>feature flag</i> | cờ tính năng |
| <i>feed</i> | bảng tin |
| <i>feed engineer</i> | kỹ sư bảng tin |
| <i>feeds</i> | bảng tin |
| <i>file storage</i> | lưu trữ tệp |
| <i>first paint</i> | hiển thị đầu tiên |
| <i>first principles</i> | nguyên lý gốc |
| <i>flapping</i> | chập chờn |
| <i>flash sale</i> | giảm giá chớp nhoáng |
| <i>flash sales</i> | flash sale |
| <i>flow</i> | luồng, luồng xử lý |
| <i>follower</i> | nút theo, người theo dõi |
| <i>followers</i> | người theo dõi, nút theo |
| <i>foreign keys</i> | khoá ngoại |
| <i>forward</i> | chuyển tiếp |
| <i>framework</i> | khung tư duy, khung, khung phát triển |

| Tiếng Anh | Tiếng Việt |
|------------------------------|-----------------------------------|
| frameworks | khung tư duy |
| full table scan | quét toàn bộ bảng, quét toàn bảng |
| functional requirement | yêu cầu chức năng |
| functional requirements | yêu cầu chức năng |
| geo routing | định tuyến theo địa lý |
| geographic latency | độ trễ địa lý |
| geographic sharding | phân mảnh theo địa lý |
| geography-based sharding | phân mảnh theo địa lý |
| geolocation | vị trí địa lý |
| geolocation-based routing | định tuyến theo vị trí địa lý |
| golden signals | tín hiệu vàng |
| graceful degradation | suy giảm có kiểm soát |
| gradual rollout | triển khai dần |
| happened-before | xảy ra trước |
| happened-before relationship | quan hệ xảy-ra-trước |
| happens-before relationship | quan hệ xảy-ra-trước |
| happy path | kịch bản suôn sẻ |
| hash | hàm băm, băm |
| hash partitioning | phân vùng theo hàm băm |
| hash-based sharding | phân mảnh theo hàm băm |
| header | tiêu đề |
| headers | tiêu đề |
| headroom | dư địa |
| health check | kiểm tra sức khỏe |
| health checks | kiểm tra sức khỏe |
| heartbeats | tín hiệu nhịp tim |

| Tiếng Anh | Tiếng Việt |
|--------------------------------|--|
| high availability | tính sẵn sàng cao |
| high cardinality | lực lượng cao |
| high traffic | lưu lượng cao |
| high-availability architecture | kiến trúc có tính sẵn sàng cao |
| high-cardinality | lực lượng cao |
| high-level architecture | kiến trúc tổng quan, kiến trúc tổng thể |
| highly available | tính sẵn sàng cao |
| historical data | dữ liệu lịch sử |
| hit rate | tỷ lệ trúng, trúng bộ nhớ đệm |
| horizontal scaling | mở rộng theo chiều ngang, theo chiều ngang |
| hot data | dữ liệu "nóng", dữ liệu nóng |
| hot key | khoá nóng |
| hot storage | lưu trữ nóng |
| hotspot | điểm nóng |
| hotspots | điểm nóng |
| hybrid | lai |
| hybrid clocks | đồng hồ lai |
| hybrid fanout | phân phát lai |
| hybrid logical clocks | đồng hồ logic lai |
| id generation | sinh id |
| idempotency | tính lũy đẳng, lũy đẳng |
| idempotency key | khoá lũy đẳng |
| idempotency keys | khoá lũy đẳng |
| idempotent | lũy đẳng |
| idempotent operation | thao tác lũy đẳng |
| identity | định danh |

| Tiếng Anh | Tiếng Việt |
|-----------------------|------------------------------|
| immediate consistency | nhất quán tức thời |
| implementation | cài đặt |
| in-memory | lưu trong bộ nhớ |
| incident | sự cố |
| incidents | sự cố |
| inconsistent state | trạng thái không nhất quán |
| index | chỉ mục |
| indexes | chỉ mục |
| indexing | đánh chỉ mục, chỉ mục |
| infinite scroll | cuộn vô hạn |
| infra | hạ tầng |
| infrastructure | hạ tầng |
| instance | phiên bản chạy |
| instances | phiên bản máy chủ, phiên bản |
| instrument | gắn đo đặc, gắn mã đo đặc |
| instrumentation | gắn đo đặc, gắn mã đo đặc |
| integration | tích hợp, kiểm thử tích hợp |
| integration tests | kiểm thử tích hợp |
| integration thinking | tư duy tích hợp |
| integrity guarantees | bảo đảm toàn vẹn |
| intelligent routing | định tuyến thông minh |
| interactions | tương tác |
| interview | phỏng vấn |
| interview strategy | chiến lược phỏng vấn |
| interviewer | người phỏng vấn |
| interviewers | người phỏng vấn |

| Tiếng Anh | Tiếng Việt |
|-----------------------|----------------------------------|
| interviews | phỏng vấn |
| invalidate | vô hiệu hoá |
| invalidation | vô hiệu hoá |
| ip whitelist | danh sách ip cho phép |
| isolation | tính cô lập |
| junior | mới vào nghề, kỹ sư mới vào nghề |
| junior engineer | kỹ sư mới vào nghề |
| junior engineers | kỹ sư mới vào nghề |
| keys | khoá |
| kv store | kho khoá-giá trị |
| lamport timestamps | dấu thời gian lamport |
| last write wins | ghi sau cùng thắng |
| latency | độ trễ |
| latency-based routing | định tuyến theo độ trễ |
| layer 4 | tầng 4 |
| layer 7 | tầng 7 |
| layer 7 lb | bộ cân bằng tải tầng 7 |
| lazy loading | tải lười |
| lb | bộ cân bằng tải |
| leader | nút chủ |
| leader election | bầu chọn nút chủ |
| leader lease | hợp đồng thuê quyền nút chủ |
| legacy systems | hệ thống cũ |
| linearizability | tính tuyến tính hóa |
| list partitioning | phân vùng theo danh sách |
| load | tải |

| Tiếng Anh | Tiếng Việt |
|------------------------|----------------------------|
| load balancer | bộ cân bằng tải |
| load balancers | bộ cân bằng tải |
| load balancing | cân bằng tải |
| load control | kiểm soát tải |
| load shedding | cắt giảm tải |
| load test | kiểm thử tải |
| load testing | kiểm thử tải |
| load testing thinking | tư duy kiểm thử tải |
| local clock | đồng hồ cục bộ |
| local solution | giải pháp cục bộ |
| local transactions | giao dịch cục bộ |
| locality of reference | tính cục bộ của tham chiếu |
| lock | khoá |
| locking | khoá |
| log aggregation | tổng hợp log |
| log levels | mức log |
| logging | ghi log, log |
| logging strategy | chiến lược ghi log |
| logical clock | đồng hồ logic |
| logical clocks | đồng hồ logic |
| logical counter | bộ đếm logic |
| logs | log |
| long-lived connections | kết nối kéo dài |
| long-polling | thăm dò dài |
| loose coupling | liên kết lỏng |
| maintain | bảo trì |

| Tiếng Anh | Tiếng Việt |
|------------------------------------|-----------------------------------|
| <i>maintenance window</i> | khung giờ bảo trì |
| <i>majority quorum</i> | túc số đa số |
| <i>managed database</i> | cơ sở dữ liệu được quản lý |
| <i>managed service</i> | dịch vụ được quản lý |
| <i>managed services</i> | dịch vụ được quản lý |
| <i>manual invalidation</i> | vô hiệu hoá thủ công |
| <i>master</i> | nút chủ, máy chủ chính |
| <i>master election</i> | bầu chọn máy chủ chính |
| <i>master-slave</i> | chủ-tớ |
| <i>material views</i> | khung nhìn cụ thể hoá |
| <i>materialized views</i> | khung nhìn cụ thể hoá |
| <i>mean time to resolve</i> | thời gian trung bình để khắc phục |
| <i>median</i> | trung vị |
| <i>memory</i> | bộ nhớ |
| <i>memory leak</i> | rò rỉ bộ nhớ |
| <i>mental framework</i> | khung tư duy |
| <i>mental model</i> | mô hình tư duy |
| <i>mental models</i> | mô hình tư duy |
| <i>mentor</i> | người hướng dẫn |
| <i>merge conflicts</i> | xung đột khi hợp nhất |
| <i>message</i> | thông điệp |
| <i>message broker</i> | môi giới thông điệp |
| <i>message delivery guarantees</i> | đảm bảo giao tin |
| <i>message duplication</i> | trùng lặp thông điệp |
| <i>message queue</i> | hàng đợi thông điệp |
| <i>message queues</i> | hàng đợi thông điệp |

| Tiếng Anh | Tiếng Việt |
|----------------------------|---------------------------------|
| messages | thông điệp |
| metadata | siêu dữ liệu |
| metadata store | kho siêu dữ liệu |
| metrics | chỉ số, số liệu đo |
| microservice | vi dịch vụ |
| microservices | vi dịch vụ |
| microservices architecture | kiến trúc vi dịch vụ |
| migrate | chuyển đổi |
| migrate data | di chuyển dữ liệu |
| migration | chuyển đổi |
| mindset | tư duy |
| mindsets | tư duy |
| miss | trượt |
| ml inference | suy luận ml |
| ml pipeline | quy trình học máy |
| ml ranking pipeline | quy trình xếp hạng bằng học máy |
| mock interview | phỏng vấn thử |
| mock interviews | phỏng vấn thử |
| model | mô hình |
| models | mô hình |
| modular | mô-đun |
| modular monolith | nguyên khối mô-đun hoá |
| modules | mô-đun |
| monitor | giám sát |
| monitoring | giám sát |
| monitoring metrics | số liệu giám sát |

| Tiếng Anh | Tiếng Việt |
|----------------------------|-----------------------------------|
| monolith | hệ thống nguyên khối, nguyên khối |
| monolithic | nguyên khối |
| multi-channel | nhiều kênh |
| multi-cloud | đa đám mây |
| multi-layer cache | bộ nhớ đệm nhiều tầng |
| multi-layer caching | lưu đệm nhiều tầng |
| multi-node | nhiều nút |
| multi-region | đa vùng |
| multi-region active-active | đa vùng chủ động – chủ động |
| multi-region deployment | triển khai đa vùng |
| multi-tenant | đa khách thuê |
| multiplexing | ghép kênh |
| mutual tls | tls hai chiều |
| n+1 problem | vấn đề n+1 |
| n+1 queries | truy vấn n+1 |
| n+1 query | truy vấn n+1 |
| naive scaling | mở rộng ngây thơ |
| nano-services | dịch vụ siêu nhỏ |
| natural idempotency | lũy đẳng tự nhiên |
| network | mạng |
| network delay | độ trễ mạng |
| network jitter | dao động độ trễ mạng |
| network latency | độ trễ mạng |
| network partition | phân vùng mạng |
| network partitions | phân vùng mạng |
| network round trip | một vòng khứ hồi mạng |

| Tiếng Anh | Tiếng Việt |
|-----------------------------|--|
| news feed | bảng tin |
| newsfeed | bảng tin |
| node | nút |
| nodes | nút |
| non-functional requirement | yêu cầu phi chức năng |
| non-functional requirements | yêu cầu phi chức năng |
| normalization | chuẩn hoá |
| notification system | hệ thống thông báo |
| observability | khả năng quan sát |
| offline sync | đồng bộ ngoại tuyến |
| offset-based pagination | phân trang dựa trên offset |
| on-call | trực sự cố |
| oncall | trực sự cố |
| one-size-fits-all | một cỡ vừa cho tất cả |
| operability | khả năng vận hành |
| optimization | tối ưu |
| optimization strategy | chiến lược tối ưu |
| optimize | tối ưu |
| orchestration | điều phối tập trung |
| order | đơn hàng, thứ tự |
| ordering | thứ tự |
| origin | máy chủ gốc |
| origin server | máy chủ gốc |
| outage | sự cố ngừng dịch vụ |
| outages | sự cố ngừng dịch vụ, sự cố ngừng hoạt động |
| outliers | giá trị ngoại lai |

| Tiếng Anh | Tiếng Việt |
|---------------------|---|
| over-engineer | thiết kế thừa |
| over-engineering | thiết kế thừa |
| overhead | chi phí phụ |
| overload | quá tải |
| overselling | bán vượt tồn kho |
| ownership | làm chủ, trách nhiệm làm chủ |
| packet loss | mất gói tin |
| pagination | phân trang |
| pain points | điểm đau |
| parallel processing | xử lý song song |
| parallelization | song song hoá |
| parallelize | song song hoá |
| partial failure | sự cố cục bộ |
| partial index | chỉ mục một phần |
| partition | phân vùng, chịu phân vùng |
| partition pruning | cắt tỉa phân vùng |
| partition tolerance | khả năng chịu phân vùng |
| partitioning | phân vùng |
| partitions | phân vùng |
| pattern | mẫu, mẫu thiết kế, mẫu hình |
| patterns | mẫu thiết kế, mẫu, mẫu truy cập |
| payload | dữ liệu gửi kèm, kích thước dữ liệu gửi kèm |
| payloads | dữ liệu gửi kèm |
| payment | thanh toán |
| peak | cao điểm |
| peak hour | giờ cao điểm |

| Tiếng Anh | Tiếng Việt |
|--------------------------|-----------------------------|
| peak hours | giờ cao điểm |
| peak traffic | lưu lượng cao điểm |
| percentile | bách phân vị |
| percentiles | bách phân vị |
| perf | hiệu năng |
| perform | hiệu năng |
| performance | hiệu năng |
| performance bottleneck | điểm nghẽn hiệu năng |
| performance debugging | gỡ lỗi hiệu năng |
| performance optimization | tối ưu hiệu năng |
| performance-first | đặt hiệu năng lên đầu |
| permanent redirects | chuyển hướng vĩnh viễn |
| persistence | lưu bền |
| persistent storage | lưu trữ bền vững |
| personalized | cá nhân hoá |
| physical clock | đồng hồ vật lý |
| physical clocks | đồng hồ vật lý |
| pii | thông tin định danh cá nhân |
| pipeline | đường ống |
| point of presence | điểm hiện diện |
| poison message | thông điệp độc |
| polyglot architecture | kiến trúc đa ngôn ngữ |
| pool | nhóm máy chủ |
| pools | nhóm máy chủ |
| port | cổng |
| post-mortems | báo cáo hậu kiểm |

| Tiếng Anh | Tiếng Việt |
|-------------------------------|---|
| <i>pre-compute</i> | tính trước |
| <i>premature complexity</i> | phức tạp hoá sớm |
| <i>premature optimization</i> | tối ưu quá sớm, tối ưu hoá sớm, tối ưu hoá quá sớm |
| <i>prematurely optimize</i> | tối ưu hoá quá sớm |
| <i>prepare</i> | chuẩn bị |
| <i>prepare phase</i> | pha chuẩn bị |
| <i>presence</i> | trạng thái trực tuyến |
| <i>presentation layer</i> | tầng hiển thị, lớp trình bày |
| <i>primary</i> | máy chủ chính |
| <i>primary database</i> | cơ sở dữ liệu chính |
| <i>problem-first</i> | lấy bài toán làm trung tâm |
| <i>problem-first thinking</i> | lấy bài toán làm trung tâm, tư duy ưu tiên vấn đề |
| <i>process</i> | tiến trình |
| <i>processes</i> | tiến trình |
| <i>processing pipeline</i> | đường ống xử lý |
| <i>producer</i> | bên phát |
| <i>producers</i> | bên phát |
| <i>product-market-fit</i> | độ phù hợp sản phẩm – thị trường |
| <i>production</i> | môi trường thực tế, môi trường vận hành thực tế, vận hành thực tế |
| <i>production-grade</i> | vận hành thực tế |
| <i>production-ready</i> | sẵn sàng cho vận hành thực tế |
| <i>profile performance</i> | phân tích hiệu năng |
| <i>profiling</i> | phân tích hiệu năng |
| <i>projections</i> | phép chiếu |
| <i>propagation delay</i> | độ trễ lan truyền |
| <i>protection layer</i> | lớp bảo vệ |

| Tiếng Anh | Tiếng Việt |
|------------------------------|-----------------------------------|
| protocol | giao thức |
| pub/sub | mô hình xuất bản – đăng ký |
| publish | phát hành |
| publish-subscribe | xuất bản – đăng ký |
| pull model | mô hình kéo |
| purge | xoá bộ nhớ đệm |
| purge api | api xoá bộ nhớ đệm |
| push model | mô hình đẩy |
| push notification | thông báo đẩy |
| quartz crystal oscillator | bộ dao động thạch anh |
| queries | truy vấn |
| query | truy vấn |
| query cache | bộ nhớ đệm truy vấn |
| query execution plan | kế hoạch thực thi truy vấn |
| query optimization | tối ưu truy vấn |
| query parameters | tham số truy vấn |
| query planner | bộ lập kế hoạch truy vấn |
| query tuning | tinh chỉnh truy vấn |
| queue | hàng đợi |
| queue backlog | hàng đợi tồn đọng |
| queue depth | độ sâu hàng đợi |
| queue-worker | hàng đợi – worker |
| queue-worker pattern | mẫu hàng đợi – worker |
| queues | hàng đợi |
| random jitter | độ lệch ngẫu nhiên |
| randomized election timeouts | thời gian chờ bầu chọn ngẫu nhiên |

| Tiếng Anh | Tiếng Việt |
|------------------------------|---------------------------------------|
| range partitioning | phân vùng theo khoảng |
| range queries | truy vấn theo khoảng |
| range-based sharding | phân mảnh theo khoảng |
| ranking | xếp hạng |
| rate limit | giới hạn tần suất |
| rate limiter | bộ giới hạn tần suất |
| rate limiting | giới hạn tần suất |
| rate limits | giới hạn tần suất |
| re-hash | băm lại |
| read | đọc |
| read heavy | thiên về đọc |
| read models | mô hình đọc |
| read path | đường đọc |
| read replica | bản sao chỉ đọc |
| read replica pattern | mẫu bản sao chỉ đọc |
| read replicas | bản sao chỉ đọc |
| read traffic | lưu lượng đọc |
| read-after-write | đọc-sau-khi-ghi, đọc ngay sau khi ghi |
| read-after-write consistency | nhất quán đọc-sau-ghi |
| read-heavy | thiên về đọc, hệ thống thiên về đọc |
| read-heavy system | hệ thống thiên về đọc |
| read-heavy systems | hệ thống thiên về đọc |
| read-heavy workload | khối lượng công việc thiên về đọc |
| real interview strategy | chiến lược phỏng vấn thực chiến |
| real-time | thời gian thực |
| real-time delivery | truyền tin thời gian thực |

| Tiếng Anh | Tiếng Việt |
|---------------------------|------------------------------------|
| rebalance | tái cân bằng |
| rebalancing | tái cân bằng |
| recommendation | gợi ý |
| recommendation system | hệ thống gợi ý |
| redirect | chuyển hướng |
| redirects | chuyển hướng |
| redundancy | dự phòng |
| refactor | tái cấu trúc |
| regions | vùng |
| reliability | độ tin cậy |
| reliable | đáng tin cậy |
| replica | bản sao |
| replicas | bản sao |
| replicate | nhân bản |
| replicated | nhân bản |
| replication | nhân bản |
| replication lag | độ trễ nhân bản |
| request | yêu cầu |
| request body | thân yêu cầu |
| request flow | luồng yêu cầu |
| request pattern | mẫu yêu cầu |
| request-driven | hướng yêu cầu |
| request-response | yêu cầu-phản hồi, yêu cầu-phản hồi |
| requests | yêu cầu |
| requirement | yêu cầu |
| requirement clarification | làm rõ yêu cầu |

| Tiếng Anh | Tiếng Việt |
|---------------------------|-------------------------------|
| requirements | yêu cầu |
| reserved instances | máy ảo đặt trước |
| resharding | phân mảnh lại |
| resilient | bền bỉ |
| resolution strategy | chiến lược giải quyết |
| resource pools | nhóm tài nguyên |
| response | phản hồi |
| response time | thời gian phản hồi |
| responses | phản hồi |
| responsive images | hình ảnh đáp ứng, ảnh đáp ứng |
| resume-driven development | phát triển hướng cv |
| retries | thử lại |
| retry | thử lại |
| retry policy | chính sách thử lại |
| revalidate | xác thực lại |
| revalidation | xác thực lại |
| reverse engineer | phân tích ngược |
| reverse engineering | phân tích ngược |
| roadmap | lộ trình |
| roi | lợi tức đầu tư |
| rollback | hoàn tác |
| rolling deployment | triển khai cuốn chiếu |
| rolling deployments | triển khai cuốn chiếu |
| root cause | nguyên nhân gốc |
| root cause analysis | phân tích nguyên nhân gốc |
| round trip | khứ hồi, một vòng khứ hồi |

| Tiếng Anh | Tiếng Việt |
|-----------------------|--|
| route | định tuyến |
| routes | định tuyến |
| routing | định tuyến |
| routing algorithm | thuật toán định tuyến |
| rubric | thang tiêu chí |
| runbook | sổ tay xử lý |
| runbooks | sổ tay xử lý |
| sampling | lấy mẫu |
| saturation | độ bão hòa |
| scalability | khả năng mở rộng |
| scalability patterns | mẫu mở rộng |
| scalable | có khả năng mở rộng, mở rộng, khả năng mở rộng |
| scalable architecture | kiến trúc có khả năng mở rộng |
| scale | quy mô, mở rộng, khả năng mở rộng |
| scale horizontal | mở rộng theo chiều ngang |
| scale intuition | trực giác về quy mô |
| scale triggers | tín hiệu cần mở rộng |
| scale vertically | mở rộng theo chiều dọc |
| scaling | mở rộng |
| scenario | kịch bản |
| scenarios | kịch bản |
| scheduled jobs | tác vụ định kỳ |
| schema | lược đồ |
| scope | phạm vi |
| semi-synchronous | bán đồng bộ |
| senior | kỹ sư lâu năm, lâu năm |

| Tiếng Anh | Tiếng Việt |
|--------------------------|------------------------------|
| senior architect | kiến trúc sư lâu năm |
| senior engineer | kỹ sư lâu năm |
| serialization | tuần tự hoá |
| serialize | tuần tự hoá |
| server | máy chủ |
| serverless | kiến trúc phi máy chủ |
| servers | máy chủ |
| service | dịch vụ |
| service boundaries | ranh giới dịch vụ |
| service decoupling | tách rời dịch vụ |
| service level agreement | thỏa thuận mức dịch vụ |
| service level indicator | chỉ số mức dịch vụ |
| service level indicators | chỉ số mức dịch vụ |
| service level objective | mục tiêu mức dịch vụ |
| service workers | service worker |
| services | dịch vụ |
| session | phiên |
| session affinity | gắn phiên |
| session stickiness | gắn phiên |
| sessions | phiên |
| shard | mảnh, phân mảnh |
| shard key | khoá phân mảnh |
| shard router | bộ định tuyến mảnh |
| sharded database | cơ sở dữ liệu phân mảnh |
| sharding | phân mảnh dữ liệu, phân mảnh |
| shards | mảnh |

| Tiếng Anh | Tiếng Việt |
|-------------------------|-----------------------------|
| shared cache | bộ nhớ đệm dùng chung |
| shared session store | kho lưu phiên dùng chung |
| shedding | cắt giảm tải |
| sidecar proxy | proxy sidecar |
| silent data corruption | hỏng dữ liệu thâm lặng |
| simple baseline | phiên bản nền đơn giản |
| single point of failure | điểm lỗi duy nhất |
| single-machine | một máy duy nhất |
| single-server | một máy chủ |
| single-threaded | đơn luồng |
| single-writer | một nút ghi |
| slave | nút tớ |
| slaves | nút tớ, bản sao |
| slow query | truy vấn chậm |
| slow query log | nhật ký truy vấn chậm |
| smart invalidation | vô hiệu hoá thông minh |
| soak test | kiểm thử ngâm |
| software architecture | kiến trúc phần mềm |
| solution-first thinking | tư duy ưu tiên giải pháp |
| source of truth | nguồn dữ liệu gốc |
| speedup | tăng tốc |
| spike | tăng đột biến |
| spike test | kiểm thử đột biến |
| spikes | tăng đột biến, đột biến tải |
| split brain | não phân đôi |
| spof | điểm lỗi duy nhất |

| Tiếng Anh | Tiếng Việt |
|--------------------|---|
| spot instances | máy ảo giá rẻ ngắt quãng |
| ssl termination | kết thúc ssl |
| staff | kỹ sư cấp cao |
| staff engineer | kỹ sư cấp staff |
| staging | môi trường thử nghiệm |
| stakeholder | bên liên quan |
| stakeholders | bên liên quan |
| stale | cũ |
| stale data | dữ liệu cũ |
| stampede | dồn dập |
| startup | công ty khởi nghiệp |
| startups | công ty khởi nghiệp |
| state | trạng thái |
| state machines | máy trạng thái |
| stateful | có trạng thái |
| stateless | phi trạng thái |
| states | trạng thái |
| static assets | tài nguyên tĩnh |
| static content | nội dung tĩnh |
| static threshold | ngưỡng tĩnh |
| sticky session | phiên dính |
| sticky sessions | phiên dính |
| storage | lưu trữ, bộ lưu trữ, dung lượng lưu trữ |
| storage strategy | chiến lược lưu trữ |
| streaming | truyền phát |
| streaming platform | nền tảng truyền dữ liệu luồng |

| Tiếng Anh | Tiếng Việt |
|----------------------------------|--------------------------------------|
| stress test | kiểm thử áp lực |
| strong consistency | nhất quán mạnh |
| structured logging | log có cấu trúc, ghi log có cấu trúc |
| subscribe | đăng ký |
| subscriber | bên đăng ký |
| subscribers | bên đăng ký |
| surrogate keys | khoá thay thế |
| sync | đồng bộ |
| sync service | dịch vụ đồng bộ |
| synchronization | đồng bộ hoá |
| synchronous | đồng bộ |
| synchronous communication | giao tiếp đồng bộ |
| synchronous processing | xử lý đồng bộ |
| synthetic monitoring | giám sát tổng hợp |
| system architecture | kiến trúc hệ thống |
| system capacity | năng lực hệ thống |
| system clock | đồng hồ hệ thống |
| system design | thiết kế hệ thống |
| system design interview | phỏng vấn thiết kế hệ thống |
| system design interviews | phỏng vấn thiết kế hệ thống |
| system design thinking | tư duy thiết kế hệ thống |
| system design thinking framework | khung tư duy thiết kế hệ thống |
| system question framework | khung câu hỏi hệ thống |
| system thinking | tư duy hệ thống |
| system thinking framework | khung tư duy hệ thống |
| system-level thinking | tư duy ở tầm hệ thống |

| Tiếng Anh | Tiếng Việt |
|----------------------------|-----------------------------|
| system-wide integrity | toàn vẹn toàn hệ thống |
| tail latency | độ trễ đuôi |
| target | mục tiêu |
| task queues | hàng đợi tác vụ |
| team | nhóm |
| team ownership | quyền sở hữu theo nhóm |
| teams | nhóm |
| tech stack | bộ công nghệ |
| technology-first | lấy công nghệ làm trung tâm |
| template | khuôn mẫu |
| term numbers | số nhiệm kỳ |
| termination | kết thúc |
| the golden signals | bốn tín hiệu vàng |
| the pattern | quy luật |
| the read/write pattern | mẫu đọc/ghi |
| the weakest link principle | nguyên lý mắt xích yếu nhất |
| think aloud | nghĩ thành lời |
| thinking framework | khung tư duy |
| thinking in constraints | tư duy theo ràng buộc |
| thread pool | nhóm luồng |
| thread pools | nhóm luồng |
| thresholds | ngưỡng |
| throughput | thông lượng |
| thundering herd | đàn bò dồn dập |
| tiered | phân tầng |
| tiered cache | bộ nhớ đệm phân tầng |

| Tiếng Anh | Tiếng Việt |
|----------------------|--|
| tight coupling | liên kết chặt |
| time to live | thời gian sống |
| time-series | chuỗi thời gian |
| timeline | dòng thời gian |
| timeline cache | bộ nhớ đệm dòng thời gian |
| timeout | hết thời gian chờ, thời gian chờ |
| timeouts | thời gian chờ, hết thời gian chờ |
| timestamp | đầu thời gian |
| timestamps | đầu thời gian |
| toil | việc lặp lại nhàm chán |
| top-down | từ trên xuống |
| trace | vết, truy vết |
| trace context | ngữ cảnh truy vết |
| traces | vết, truy vết |
| tracing | truy vết |
| trade-off | sự đánh đổi, đánh đổi |
| trade-off thinking | tư duy đánh đổi |
| trade-offs | đánh đổi, sự đánh đổi |
| traffic | lưu lượng |
| traffic distribution | phân bố lưu lượng |
| traffic pattern | mẫu lưu lượng, mô hình lưu lượng |
| traffic shaping | định hình lưu lượng |
| traffic spikes | lưu lượng tăng đột biến, tăng đột biến lưu lượng |
| transaction | giao dịch |
| transaction strategy | chiến lược giao dịch |
| transactions | giao dịch |

| Tiếng Anh | Tiếng Việt |
|---------------------|-----------------------------------|
| transcode | chuyển mã |
| transcoding | chuyển mã video |
| transitivity | tính bắc cầu |
| transport layer | tầng giao vận |
| trending | xu hướng |
| trusted architect | kiến trúc sư đáng tin cậy |
| tunable consistency | tính nhất quán có thể điều chỉnh |
| under-engineering | thiết kế thiếu |
| unit tests | kiểm thử đơn vị |
| upload pipeline | đường ống tải lên |
| upstream | thượng nguồn |
| uptime | thời gian hoạt động |
| url shortener | dịch vụ rút gọn url, rút gọn url |
| use case | tình huống sử dụng |
| use cases | tình huống sử dụng |
| user | người dùng |
| user experience | trải nghiệm người dùng |
| user interactions | tương tác của người dùng |
| users | người dùng |
| ux | trải nghiệm người dùng |
| vector clock | đồng hồ vector |
| vector clocks | đồng hồ vector |
| version control | quản lý phiên bản |
| versioned urls | url có phiên bản |
| versioning | đánh phiên bản, quản lý phiên bản |
| vertical scale | mở rộng theo chiều dọc |

| Tiếng Anh | Tiếng Việt |
|--------------------------|----------------------------|
| vertical scaling | mở rộng theo chiều dọc |
| video streaming | phát video trực tuyến |
| video upload pipeline | quy trình tải video lên |
| views | khung nhìn |
| volume | khối lượng |
| wal shipping | chuyển wal |
| wall clock | giờ đồng hồ thực |
| war room | phòng họp khẩn cấp |
| web application firewall | tường lửa ứng dụng web |
| web server | máy chủ web |
| weight | trọng số |
| weights | trọng số |
| whiteboard | bảng trắng |
| worker | tiến trình xử lý |
| workers | tiến trình xử lý |
| workload | khối lượng công việc |
| write | ghi |
| write heavy | thiên về ghi |
| write traffic | lưu lượng ghi |
| write-ahead log | nhật ký ghi trước |
| write-heavy | thiên về ghi |
| writes | thao tác ghi |
| zero-downtime deploy | triển khai không gián đoạn |
| zero-downtime deployment | triển khai không gián đoạn |